

Data Structures and Algorithms in C++

Data Structures and Algorithms in C++

EFFICIENT SOLUTIONS • OPTIMAL PERFORMANCE • REAL-WORLD APPLICATIONS



LINKED LIST



STACK



QUEUE



TREES



GRAPHS



HASH TABLE
BINARY
SEARCH TREE



HEAP

```
template <typename T>
class Node {
public:
    T data;
    Node* next;
    Node(T d) : data(d),
               next(nullptr) {}
};

class Stack {
    Node* top;
public:
    void push(T x);
    void pop();
    T peek();
    bool isEmpty();
};
```



```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        if (a[i] < a[j])
            swap(a[i], a[j]);
```



C++

CHAMAN SINGH VERMA

Data Structures, Algorithms, and Applications in C++

Chaman Singh Verma

Contents

I Preliminaries	1
1 Modern C++ Review	2
1.1 Introduction	2
1.2 Functions and Parameters	2
1.2.1 Pass by Value, Reference, and Const Reference	2
1.2.2 Default Arguments	2
1.2.3 Function Overloading	3
1.2.4 Lambda Expressions	3
1.3 Dynamic Memory and RAII	3
1.3.1 The Problem with Raw Pointers	3
1.3.2 Smart Pointers (C++11)	4
1.3.3 RAII (Resource Acquisition Is Initialization)	4
1.4 Templates and Concepts	5
1.4.1 Function Templates	5
1.4.2 Class Templates	5
1.4.3 Template Template Parameters and Variadic Templates	5
1.4.4 Concepts (C++20)	5
1.5 Move Semantics	6
1.5.1 Lvalues and Rvalues	6
1.5.2 Move Constructor and Move Assignment	6
1.5.3 Rule of Five	7
1.6 std::vector and std::span	7
1.6.1 std::vector	7
1.6.2 std::span (C++20)	7
1.6.3 std::string_view (C++17)	8
1.7 Ranges and Views (C++20)	8

1.7.1 Zip Containers (C++23)	8
1.8 Error Handling	10
1.8.1 Exceptions	10
1.8.2 noexcept	11
1.9 Iterators and Ranges	11
1.9.1 Iterator Categories	11
1.9.2 Range-Based for Loop	11
1.9.3 Iterator Adapters	11
1.10 Type Deduction: auto and decltype	12
1.11 Coroutines (C++20)	12
1.12 Common Patterns in This Book	13
1.12.1 Example: A Generic Container Skeleton	13
1.13 Compiler and Build	14
1.14 STL Containers Overview	14
1.15 Smart Pointers	16
1.15.1 std::unique_ptr	16
1.15.2 std::shared_ptr	16
1.15.3 std::weak_ptr	17
1.15.4 When to Use Each	17
1.16 Lambda Expressions	17
1.16.1 Basic Syntax	17
1.16.2 Captures	18
1.16.3 Use with STL Algorithms	18
1.17 Move Semantics and Rvalue References	19
1.17.1 Lvalues and Rvalues	19
1.17.2 std::move	19
1.17.3 Move Constructor and Move Assignment	20
1.17.4 Perfect Forwarding	20
1.17.5 Move Semantics in Practice	21
1.17.6 constexpr and consteval	21
1.17.7 Structured Bindings (C++17)	22
1.17.8 std::optional and std::variant	22
1.18 Common Bugs and Pitfalls	23

1.19 Summary	24
1.20 Exercises	24
1.20.1 Drill	24
1.20.2 Application	25
1.20.3 Research	26
1.20.4 Additional Exercises	26
1.21 References and Further Reading	27
2 Performance Analysis	29
2.1 Introduction	29
2.2 What Is an Algorithm?	29
2.3 Measuring Algorithm Efficiency	29
2.3.1 Time Complexity	29
2.3.2 Space Complexity	30
2.4 Asymptotic Notation	30
2.4.1 Big O Notation (O)	30
2.4.2 Big Omega Notation (Ω)	31
2.4.3 Big Theta Notation (Θ)	31
2.4.4 Little o and Little ω	31
2.5 Practical Complexity	32
2.5.1 Growth Rates Compared	32
2.5.2 Which Complexity Matters?	32
2.6 Analyzing Iterative Algorithms	32
2.6.1 Simple Loop	32
2.6.2 Nested Loops	33
2.6.3 Dependent Nested Loops	33
2.6.4 Logarithmic Loops	33
2.7 Analyzing Recursive Algorithms	33
2.7.1 Recurrence Relations	33
2.7.2 Solving Recurrences	34
2.8 Amortized Analysis	35
2.8.1 Aggregate Method	35
2.8.2 Accounting Method	35

2.8.3	Potential Method	35
2.9	Space Complexity	36
2.9.1	Fixed vs. Variable Space	36
2.9.2	Analyzing Space	36
2.10	Best, Average, and Worst Case	36
2.11	Smoothed Analysis	37
2.11.1	The Idea	37
2.11.2	The Simplex Example	37
2.11.3	Formal Definition	38
2.11.4	Why It Matters	38
2.11.5	Known Results	38
2.12	Recursion Deep Dive	38
2.12.1	The Recursive Thought Process	39
2.12.2	Recursion Tree Visualization	39
2.12.3	Tail Recursion vs Non-Tail Recursion	39
2.12.4	Converting Tail Recursion to Iteration	40
2.12.5	Worked Example: Fibonacci (Optimized)	40
2.12.6	Worked Example: Tower of Hanoi	41
2.12.7	Worked Example: Power Function	41
2.13	Loop Invariants	42
2.13.1	The Three Parts	42
2.13.2	Worked Example: Insertion Sort Correctness	42
2.13.3	Worked Example: Binary Search Correctness	43
2.14	Space Complexity Analysis (Deep Dive)	44
2.14.1	Stack Space for Recursion	44
2.14.2	Auxiliary Space vs Input Space	44
2.14.3	Space-Time Tradeoffs	45
2.15	Common Bugs and Pitfalls	45
2.16	Summary	45
2.17	Exercises	46
2.17.1	Drill	46
2.17.2	Application	47
2.17.3	Research	49

2.18	References and Further Reading	49
3	Performance Measurement of Programs	51
3.1	Introduction	51
3.2	Why Measure?	51
3.3	A First Measurement: <code>std::chrono</code>	51
3.3.1	Clock Types	52
3.3.2	Measuring Short Intervals	52
3.4	Google Benchmark Library	52
3.5	Factors Affecting Performance	53
3.5.1	Hardware Effects	53
3.5.2	Operating System Effects	53
3.5.3	Compiler Effects	53
3.5.4	The Benchmarking Trap: Dead Code Elimination	54
3.6	Case Study: Sorting Algorithms	54
3.7	Profiling	56
3.7.1	Sampling Profilers	56
3.7.2	Instrumenting Profilers	56
3.7.3	Cache Analysis with <code>perf</code>	56
3.8	Measuring Memory Usage	57
3.9	Practical Measurement Methodology	57
3.10	ADT for Polynomial	57
3.10.1	ADT Specification	57
3.10.2	Implementation	58
3.10.3	Example Usage	60
3.10.4	Key Design Decisions	61
3.11	ADT for Matrix	61
3.11.1	ADT Specification	61
3.11.2	Compressed Row Storage (CRS)	62
3.11.3	Implementation	63
3.11.4	When to Use CRS vs Dense	66
3.12	ADT for Graph	66
3.12.1	ADT Specification	66

3.12.2	Representations	67
3.12.3	C++ Implementation (Adjacency List)	67
3.12.4	Complexity Summary	69
3.13	Common Pitfalls	69
3.14	Summary	69
3.15	Exercises	70
3.15.1	Drill	70
3.15.2	Application	71
3.15.3	ADT Design and Implementation	72
3.15.4	Inheritance and Templates	73
3.15.5	Research	73
3.16	Primitive and Non-Primitive Data Structures	74
3.16.1	Primitive Data Structures	74
3.16.2	Non-Primitive Data Structures	75
3.16.3	Classification Tree	76
3.16.4	Comparison	77
3.16.5	Interview FAQs	77
3.17	Active and Passive Data Structures	78
3.17.1	Definitions	78
3.17.2	Active Data Structure Example: AVL Tree	78
3.17.3	Passive Data Structure Example: C-Array	79
3.17.4	Hybrid Approach: C++ Containers	79
3.17.5	When to Use Each	79
3.17.6	Active vs Passive in Modern C++	80
3.17.7	Interview FAQs	80
3.18	References and Further Reading	81
II	Data Structures	83
4	Linear Lists	84
4.1	Introduction	84
4.2	Array-Based Lists	84
4.2.1	The Linear List Abstract Data Type	84

4.2.2	Array Representation	85
4.2.3	Dynamic Array (Resizable Array)	86
4.2.4	Complexity Analysis	90
4.2.5	Application: Polynomial Arithmetic	91
4.2.6	Application: Image Editor Layer Stack	92
4.2.7	STL Connection: <code>std::vector</code>	92
4.3	Linked Lists	93
4.3.1	Singly Linked List (Chain)	93
4.3.2	Complexity Analysis	97
4.3.3	Doubly Linked List	97
4.3.4	Circular Linked Lists	98
4.3.5	Application: Polynomial Using Linked List	99
4.3.6	Application: Undo/Redo System	100
4.3.7	Memory Overhead	101
4.3.8	STL Connection: <code>std::forward_list</code> and <code>std::list</code>	101
4.4	Unrolled Linked List	102
4.4.1	Introduction	102
4.4.2	Structure	102
4.4.3	C++ Implementation	102
4.4.4	Insert and Erase	103
4.4.5	Search	105
4.4.6	Complexity Comparison	106
4.5	Finger Tree	106
4.5.1	Introduction	106
4.5.2	Structure	106
4.5.3	C++ Implementation	107
4.5.4	Concatenation and Split	109
4.5.5	Complexity	110
4.6	Rope	110
4.6.1	Introduction	110
4.6.2	Structure	111
4.6.3	C++ Implementation	111
4.6.4	Complexity Analysis	113

4.6.5	Application: Text Editor Buffer	114
4.6.6	Comparison with Alternative Structures	114
4.7	Common Bugs and Pitfalls	114
4.8	Summary	115
4.9	Interview FAQs	115
4.9.1	Fundamentals	115
4.9.2	Linked List Algorithms	116
4.9.3	Design	118
4.10	Shadow Copies and Copy-on-Write	119
4.10.1	Technique	120
4.10.2	Applications	120
4.11	Exercises	121
4.11.1	Drill	121
4.11.2	Application	121
4.11.3	Research	122
4.12	References and Further Reading	123
5	Arrays, Matrices, and Sparse Representations	124
5.1	Introduction	124
5.2	Multi-Dimensional Arrays	124
5.2.1	Row-Major and Column-Major Ordering	124
5.2.2	Multi-Dimensional Arrays in C++	124
5.2.3	A Generic Matrix Class	125
5.3	Bit Arrays and Bitmaps	126
5.3.1	Implementation	126
5.3.2	Applications	127
5.4	Dynamic Arrays	127
5.4.1	Growth Factor	127
5.4.2	Amortized Analysis	128
5.5	Sorted Arrays	128
5.5.1	Operations	128
5.5.2	Complexity	129
5.6	Circular Buffers	129

5.6.1	Implementation	129
5.6.2	Applications	130
5.7	Gap Buffers	130
5.7.1	Implementation	131
5.7.2	Comparison with Alternatives	132
5.8	Special Matrices	132
5.8.1	Diagonal Matrix	132
5.8.2	Triangular Matrices	133
5.8.3	Tridiagonal Matrix	134
5.8.4	Thomas Algorithm	134
5.8.5	Symmetric Matrices	135
5.9	Sparse Matrices	136
5.9.1	Coordinate Storage (COO)	136
5.9.2	Compressed Sparse Row (CSR)	137
5.9.3	Complexity Comparison	139
5.10	Application: PageRank	139
5.11	Application: Image Processing	140
5.12	STL Connection: <code>std::mdspan</code> (C++23)	140
5.13	Sparse Arrays	140
5.13.1	Coordinate (COO) Representation	141
5.13.2	Map-Based Representation	141
5.14	Array Rotations	142
5.14.1	Juggling Algorithm	142
5.14.2	Reversal Algorithm	143
5.14.3	Block Swap Algorithm	143
5.15	Kadane's Algorithm: Maximum Subarray	144
5.15.1	Core Idea	144
5.15.2	Recovery of Subarray Indices	144
5.15.3	Worked Trace	145
5.16	Exercises	146
5.17	Common Bugs and Pitfalls	146
5.18	Summary	146
5.19	Interview FAQs	147

5.20 Exercises	151
5.20.1 Drill	151
5.20.2 Application	151
5.20.3 Research	152
5.20.4 Bit Array Exercises	152
5.20.5 Circular Buffer Exercises	153
5.20.6 Gap Buffer Exercises	153
5.20.7 Sorted Array Exercises	153
5.21 References and Further Reading	153
6 Stacks	155
6.1 Introduction	155
6.2 The Stack ADT	155
6.2.1 Operations	155
6.2.2 Applications	155
6.3 Array-Based Stack	156
6.4 Linked Stack	157
6.5 Applications	158
6.5.1 Balanced Parentheses	158
6.5.2 Postfix Evaluation	159
6.5.3 Expression Evaluation in a Spreadsheet Engine (e.g., Excel, Google Sheets) . . .	159
6.6 STL Connection: <code>std::stack</code>	160
6.7 Balanced Symbol Checking with Error Reporting	161
6.8 Postfix Evaluation: A Complete Worked Example	162
6.9 Function Call Simulation with Activation Records	163
6.10 Min Stack: $O(1)$ Minimum Queries	165
6.11 Stack Using Queues	166
6.11.1 Push $O(n)$, Pop $O(1)$	166
6.11.2 Push $O(1)$, Pop $O(n)$	167
6.12 Stack for Infix to Prefix Conversion	168
6.12.1 Algorithm	168
6.12.2 Worked Trace: “ $A*(B+C)-D/E$ ”	168
6.12.3 C++ Implementation	169

6.13	Stack for Undo/Redo	171
6.13.1	Algorithm	171
6.13.2	Worked Trace	171
6.13.3	C++ Implementation	172
6.14	Stack for DFS Applications	173
6.14.1	DFS on an Implicit Graph Using an Explicit Stack	173
6.14.2	Topological Sort Using a Stack	174
6.14.3	Cycle Detection Using DFS	175
6.15	Common Bugs and Pitfalls	176
6.16	Summary	176
6.17	Interview FAQs	176
6.18	Exercises	180
6.18.1	Drill	180
6.18.2	Application	181
6.18.3	Research	183
6.19	References and Further Reading	183
7	Queues	184
7.1	Introduction	184
7.2	The Queue ADT	184
7.2.1	Operations	184
7.2.2	Applications	184
7.3	Array-Based Queue (Circular Buffer)	184
7.4	Linked Queue	186
7.5	Applications	187
7.5.1	Message Queue (Apache Kafka / AWS SQS)	187
7.5.2	Web Crawler Frontier (BFS)	188
7.5.3	Thread Pool Task Queue	189
7.6	Deque (Double-Ended Queue)	190
7.7	STL Connection: <code>std::queue</code>	191
7.8	Circular Queue: Full Implementation	191
7.9	Double-Ended Queue (Deque)	193
7.9.1	Deque ADT	193

7.9.2	Implementation 1: Doubly Linked List	194
7.9.3	Implementation 2: Circular Array	195
7.9.4	Applications	197
7.10	Queue Using Two Stacks	198
7.11	Priority Queue Applications	199
7.11.1	Dijkstra's Shortest-Path Algorithm	200
7.11.2	Job Scheduling with Priorities	201
7.12	BFS Applications	202
7.12.1	Word Ladder Problem	202
7.12.2	Bipartiteness Checking	204
7.13	Sliding Window Maximum (Deque Approach)	204
7.13.1	Key Idea	205
7.13.2	Worked Trace	205
7.13.3	C++ Implementation	205
7.14	Level Order Traversal Variants	206
7.14.1	Zigzag Level Order	207
7.14.2	Right Side View	208
7.15	Common Bugs and Pitfalls	208
7.16	Summary	209
7.17	Interview FAQs	209
7.18	Exercises	213
7.18.1	Drill	213
7.18.2	Application	214
7.18.3	Research	215
7.18.4	Additional Drill	215
7.18.5	Additional Application	216
7.18.6	Additional Research	216
7.19	References and Further Reading	216
8	Hashing and Hash Tables	218
8.1	Introduction	218
8.2	The Dictionary ADT	218
8.3	Hash Tables: Basic Idea	218

8.4	Hash Functions	218
8.4.1	Properties of a Good Hash Function	218
8.4.2	Integer Keys	219
8.4.3	String Keys	219
8.4.4	Rob Dixon's "sdbm" hash	219
8.4.5	std::hash	219
8.5	Separate Chaining	220
8.6	Open Addressing	222
8.6.1	Linear Probing	222
8.6.2	Quadratic Probing	222
8.6.3	Double Hashing	222
8.7	Skip Lists	224
8.7.1	Structure	224
8.7.2	Node Structure	225
8.7.3	Search	225
8.7.4	Insertion	225
8.7.5	Choosing the Level	226
8.7.6	Complexity	226
8.7.7	Skip Lists vs. Hash Tables vs. Balanced Trees	227
8.7.8	STL Connection	227
8.8	Modern Hash Table Designs	227
8.8.1	Robin Hood Hashing	227
8.8.2	Cuckoo Hashing	228
8.8.3	Swiss Table (Google's Abseil flat_hash_map)	228
8.9	Distributed Hash Table (DHT)	228
8.9.1	Introduction	228
8.9.2	Chord Ring Structure	228
8.9.3	Simplified C++ Implementation	229
8.9.4	Complexity Analysis	231
8.9.5	Applications	231
8.9.6	DHT vs. Centralized Hash Table	232
8.10	Count-Min Sketch	232
8.10.1	Idea	232

8.10.2	Parameters	232
8.10.3	C++ Implementation	233
8.10.4	Properties	234
8.10.5	Heavy Hitters	235
8.11	Swiss Table Hash	235
8.11.1	Control Bytes	235
8.11.2	Probe Sequence	235
8.11.3	Probe Diagram	235
8.11.4	Comparison with <code>std::unordered_map</code>	236
8.12	Robin Hood Hashing	236
8.12.1	Invariant	236
8.12.2	Benefits	236
8.12.3	C++ Sketch	236
8.13	Extendible Hashing	239
8.13.1	Core Idea	239
8.13.2	Directory Doubling	239
8.13.3	Shrinking the Directory	242
8.13.4	Complexity	242
8.14	Complexity Analysis	242
8.15	Comparison of Dictionaries	243
8.16	Application: LZW Compression	243
8.17	STL Connection: <code>std::unordered_map</code>	244
8.18	Common Bugs and Pitfalls	244
8.19	Summary	245
8.20	Interview FAQs	245
8.21	Hash Trees (Merkle Trees)	248
8.21.1	Structure	248
8.21.2	Verification	249
8.21.3	Applications	249
8.22	Exercises	250
8.22.1	Drill	250
8.22.2	Application	250
8.22.3	Research	251

8.22.4 Additional Exercises	251
8.23 References and Further Reading	252
9 Trees and Binary Trees	254
9.1 Introduction	254
9.2 Terminology	254
9.3 Binary Trees	255
9.3.1 Properties	255
9.3.2 Binary Tree Node	255
9.3.3 Binary Tree Class	256
9.3.4 Traversal Walkthrough	257
9.3.5 Iterative Traversals (No Recursion)	258
9.4 Array Representation of Binary Trees	259
9.5 Applications of Binary Trees	259
9.5.1 Expression Trees	259
9.5.2 Morse Code Tree	260
9.6 Union-Find (Disjoint Set Union)	260
9.6.1 Tree Representation	260
9.6.2 Application: Kruskal's MST	261
9.6.3 Application: Image Connected Components	261
9.7 Reconstructing Binary Trees from Traversals	262
9.8 Decision Trees	263
9.8.1 Structure	263
9.8.2 Prediction	264
9.8.3 Building from Training Data	264
9.8.4 Complexity	266
9.8.5 Comparison with Hash Table for Classification	266
9.9 Threaded Binary Trees	267
9.9.1 Motivation	267
9.9.2 Structure	267
9.9.3 Implementation	267
9.9.4 In-Order Successor Without a Stack	268
9.9.5 Complexity	268

9.10	Lowest Common Ancestor (LCA)	269
9.10.1	Approach 1: Recursive (Binary Tree)	269
9.10.2	Approach 2: Euler Tour + RMQ	269
9.10.3	Approach 3: Binary Lifting (Preprocessing for $O(\log n)$ Queries)	270
9.10.4	Comparison	271
9.11	Tree Diameter and Center	271
9.11.1	Diameter	271
9.11.2	Center	272
9.12	Serialize and Deserialize Binary Tree	273
9.12.1	Preorder with Null Markers	273
9.12.2	Exercises on LCA, Diameter, and Serialization	274
9.13	Common Bugs and Pitfalls	274
9.14	Summary	275
9.15	Interview FAQs	275
9.16	Trees-to-Lists and Lists-to-Trees	279
9.16.1	Flatten a Binary Tree to a Linked List	279
9.16.2	BST to Sorted Doubly Linked List	280
9.16.3	Sorted Array to Balanced BST	280
9.17	Leaf-to-Root Tree Operations	281
9.17.1	Tree Height via Post-Order	281
9.17.2	Propagate Values Upward	281
9.17.3	Connection to Skew Heaps and Splay Trees	282
9.18	Exercises	282
9.18.1	Drill	282
9.18.2	Application	282
9.18.3	Research	283
9.19	References and Further Reading	283
10	Priority Queues	285
10.1	Introduction	285
10.2	The Priority Queue ADT	285
10.2.1	Operations	285
10.2.2	Applications	285

10.3	Heaps	285
10.3.1	Array Storage	286
10.3.2	Max-Heap Implementation	286
10.3.3	Push Walkthrough	287
10.3.4	Pop Walkthrough	287
10.4	Heap Initialization (Heapify)	288
10.5	Heap Sort	288
10.6	Min-Heaps	289
10.7	Leftist Trees (Mergeable Heaps)	289
10.8	Fibonacci Heap	290
10.8.1	Structure	290
10.8.2	Core Operations	291
10.8.3	Amortized Analysis	294
10.8.4	Dijkstra's with Fibonacci Heap	294
10.9	Pairing Heap	295
10.9.1	Structure	295
10.9.2	Implementation	295
10.10	Binomial Heap	297
10.10.1	Binomial Trees	297
10.10.2	Heap Structure	298
10.10.3	Implementation	298
10.10.4	Complexity	301
10.10.5	Binary Heap vs. Binomial Heap	302
10.11	D-ary Heap	302
10.11.1	Structure	302
10.11.2	Implementation	302
10.11.3	Complexity	304
10.11.4	When to Use	304
10.12	Min-Max Heap	304
10.12.1	Structure	304
10.12.2	Implementation	305
10.12.3	Complexity	309
10.13	Weak Heap	309

10.13.1 Structure	309
10.13.2 Implementation	310
10.13.3 Complexity	312
10.14 Skew Heap	312
10.14.1 Structure	312
10.14.2 Implementation	313
10.14.3 Amortized Analysis	315
10.14.4 Skew Heap vs. Leftist Tree	315
10.15 Bottom-Up Heap Construction	316
10.15.1 Why $O(n)$? Most Work Is at the Bottom	316
10.15.2 Potential Analysis	316
10.15.3 Implementation	316
10.15.4 Worked Trace (15 Elements)	317
10.15.5 Complexity Summary	318
10.16 Heap Select (kth Smallest)	319
10.16.1 Min-Heap Approach: kth Smallest	319
10.16.2 Max-Heap Approach: kth Largest	319
10.16.3 Comparison with Quickselect	320
10.17 Application: Event-Driven Simulation	320
10.18 Application: Median Maintenance	321
10.19 STL Connection: <code>std::priority_queue</code>	322
10.20 Common Bugs and Pitfalls	323
10.21 Summary	323
10.22 Interview FAQs	323
10.23 Exercises	326
10.23.1 Drill	326
10.23.2 Application	327
10.23.3 Research	328
10.24 Optimal Heap Complexity	328
10.24.1 Lower Bounds	328
10.24.2 Strict Fibonacci Heaps	328
10.25 BST-as-Heap: Tree-Oriented Heaps	329
10.25.1 Treap as Priority Queue	329

10.25.2 Interval Heap	329
10.25.3 Leaf-to-Root Operations	330
10.26 Heaps and the Pointer Machine	330
10.27 References and Further Reading	330
11 Binary Search Trees and Balanced Search Trees	332
11.1 Introduction	332
11.2 Binary Search Trees	332
11.2.1 BST Implementation	332
11.2.2 Complexity	334
11.3 AVL Trees	334
11.3.1 Rotations	335
11.3.2 AVL Implementation	335
11.3.3 Complexity	338
11.4 Red-Black Trees	338
11.4.1 RB Tree Implementation	338
11.5 B-Trees	341
11.6 B+ Trees	342
11.6.1 B+ Tree vs B-tree	342
11.6.2 Structure	342
11.6.3 B+ Tree Implementation	342
11.6.4 Complexity	345
11.7 Scapegoat Trees	345
11.7.1 Weight Balance Invariant	345
11.7.2 Scapegoat Tree Implementation	346
11.7.3 Complexity	348
11.8 Treaps (Randomized BST)	348
11.9 Splay Trees	349
11.9.1 The Splay Operation	349
11.9.2 Splay Tree Implementation	350
11.9.3 Amortized Analysis	353
11.9.4 Complexity	353
11.10 Van Emde Boas Trees	353

11.10.1 The Recursive Idea	354
11.10.2 VEB Tree Implementation	354
11.10.3 Complexity	356
11.11k-d Trees	356
11.11.1 Structure	357
11.11.2 k-d Tree Implementation	357
11.11.3 Complexity	360
11.12Quadtrees	360
11.12.1 Structure	360
11.12.2 Quadtree Implementation	360
11.12.3 Complexity	363
11.12.4 Variants	363
11.13R-trees	363
11.13.1 Structure	363
11.13.2 R-tree Implementation	364
11.13.3 Complexity	366
11.14Log-Structured Merge-Trees	367
11.14.1 Structure	367
11.14.2 LSM-tree Implementation	367
11.14.3 Complexity	369
11.15Range Trees	369
11.15.1 Structure	369
11.15.2 Range Tree Implementation	370
11.15.3 Complexity	373
11.16B*-tree	373
11.16.1 Motivation	373
11.16.2 C++ Implementation	374
11.16.3 Complexity	378
11.16.4 When to use a B*-tree	378
11.17Octree	378
11.17.1 Structure	378
11.17.2 C++ Implementation	379
11.17.3 Complexity	383

11.17.4 Comparison with the quadtree	384
11.17.5 When to use an octree	384
11.18BSP Tree (Binary Space Partitioning)	384
11.18.1 Motivation	384
11.18.2 Structure	384
11.18.3 C++ Implementation	385
11.18.4 Complexity	389
11.18.5 When to use a BSP tree	389
11.19Weight-Balanced Trees	390
11.19.1 Definition and Balance Condition	390
11.19.2 Maintenance by Rebalancing	391
11.19.3 Join and Split	391
11.19.4 Order Statistics	392
11.19.5 Comparison with AVL and Red-Black Trees	392
11.19.6 Practical Considerations	392
11.20(a,b)-Trees and B-Trees	393
11.20.1 Definition	393
11.20.2 B-Trees for Disk-Based Storage	393
11.20.3 B+ Trees	394
11.20.4 Bulk Loading	395
11.21Top-Down Red-Black Tree	395
11.21.1 Motivation	395
11.21.2 Insertion	395
11.21.3 Deletion	397
11.22Finger Trees	397
11.22.1 Structure	397
11.22.2 Measure Monoid	398
11.22.3 Performance	399
11.22.4 Applications	399
11.23Partial Rebuilding	399
11.23.1 Concept	399
11.23.2 When to Rebuild	400
11.24Join and Split Operations	400

11.24.1 Join	400
11.24.2 Split	401
11.24.3 Applications of Join and Split	401
11.25 Handling Nonunique Keys	401
11.25.1 Counting Duplicates	402
11.25.2 Multiway Split	402
11.25.3 Range Counting	402
11.26 Optimal Binary Search Trees	402
11.26.1 Problem Formulation	403
11.26.2 Dynamic Programming Solution	403
11.26.3 Knuth's Optimization	404
11.26.4 Practical Relevance	404
11.27 Comparison	404
11.28 STL Associative Containers: set, map, multiset, multimap	404
11.28.1 std::set	404
11.28.2 std::multiset	405
11.28.3 std::map	406
11.28.4 std::multimap	407
11.28.5 Internal Structure	408
11.28.6 Common Interview Patterns	409
11.28.7 Choosing Between Containers	410
11.28.8 STL Connection: std::unordered_map	410
11.28.9 Common Bugs and Pitfalls	410
11.28.10 Interview FAQs	411
11.29 STL Unordered Containers: unordered_set, unordered_map	412
11.29.1 std::unordered_set	412
11.29.2 std::unordered_multiset	413
11.29.3 std::unordered_map	413
11.29.4 std::unordered_multimap	414
11.29.5 Hash Functions and Custom Keys	414
11.29.6 Rehashing and Load Factor	415
11.29.7 Common Interview Patterns	416
11.29.8 Choosing Between All Eight Containers	416

11.30 Bimap: Bidirectional Map	417
11.30.1 Motivation	417
11.30.2 Implementation	417
11.30.3 Using Boost.Bimap	419
11.30.4 Bimap Variants	419
11.30.5 Use Cases	419
11.30.6 Complexity	420
11.30.7 Interview FAQs	420
11.31 The Dynamic Optimality Conjecture	421
11.31.1 BSTs as Points in 2D	421
11.31.2 The Greedy Algorithm	421
11.31.3 Wilber's Lower Bound	421
11.31.4 Signed Greedy	422
11.31.5 Tango Trees	422
11.31.6 Key-Independent Optimality	422
11.31.7 Splay Trees and Dynamic Optimality	422
11.32 Fusion Trees	423
11.32.1 The Idea: Wide B-Trees	423
11.32.2 Sketching: Compression by Multiplication	423
11.32.3 Parallel Comparison via Multiplication	423
11.32.4 Finding the Most Significant Set Bit	423
11.32.5 Result and Lower Bounds	424
11.32.6 Practical Considerations	424
11.33 Common Bugs and Pitfalls	425
11.34 Summary	425
11.35 Interview FAQs	425
11.36 Exercises	428
11.36.1 Drill	428
11.36.2 Application	428
11.36.3 Research	430
11.37 References and Further Reading	430
12 Graphs and Graph Algorithms	432
12.1 Introduction	432

12.2 The Graph ADT	432
12.2.1 Terminology	432
12.2.2 Graph Representations	433
12.3 Graph Classes	433
12.3.1 Adjacency Matrix	433
12.3.2 Adjacency List	434
12.4 Graph Traversal	434
12.4.1 Depth-First Search (DFS)	434
12.4.2 Breadth-First Search (BFS)	435
12.4.3 Connected Components	436
12.5 Topological Sort	436
12.6 Minimum Spanning Tree	437
12.6.1 Kruskal's Algorithm	437
12.6.2 Prim's Algorithm	438
12.7 Shortest Paths	439
12.7.1 Dijkstra's Algorithm	439
12.7.2 Bellman-Ford Algorithm	439
12.7.3 Floyd-Warshall (All Pairs)	440
12.8 Strongly Connected Components	440
12.8.1 Kosaraju's Algorithm	440
12.9 Application: Social Network Analysis	441
12.10 Articulation Points and Bridges	442
12.10.1 DFS Tree	442
12.10.2 Implementation	443
12.10.3 Applications	444
12.11 Eulerian Path and Circuit	444
12.11.1 Hierholzer's Algorithm	444
12.11.2 Eulerian Path Conditions	445
12.11.3 Applications	445
12.12A* Search	445
12.12.1 The Idea	445
12.12.2 Implementation	446
12.12.3 Common Heuristics	447

12.12.4 Applications	447
12.13 Graph Coloring	447
12.13.1 Greedy Coloring	447
12.13.2 Welsh-Powell Ordering	448
12.13.3 Applications	448
12.14 Hypergraph	448
12.14.1 Structure	449
12.14.2 C++ Implementation	449
12.14.3 Applications	451
12.14.4 Complexity	451
12.15 Binary Decision Diagrams	451
12.15.1 Structure	451
12.15.2 C++ Implementation	452
12.15.3 Applications	454
12.15.4 Complexity	454
12.16 Winged Edge	454
12.16.1 Structure	455
12.16.2 C++ Implementation	455
12.16.3 Comparison with Other Representations	457
12.16.4 Complexity	458
12.17 Biconnected Components	458
12.17.1 Key Observations	458
12.17.2 Algorithm	458
12.17.3 Worked Trace	459
12.17.4 C++ Implementation	459
12.17.5 Applications	461
12.18 Strongly Connected Components – Kosaraju (Worked Trace)	461
12.18.1 The Graph	461
12.18.2 Pass 1: DFS Finish Order	461
12.18.3 Pass 2: DFS on Transposed Graph	462
12.18.4 Why It Works	463
12.18.5 Complexity	463
12.19 Bridges in a Graph (Worked Trace)	463

12.19.1 The Graph	463
12.19.2 DFS Trace	463
12.19.3 Corrected Trace	464
12.19.4 When Is an Edge a Bridge?	465
12.20 Link-Cut Trees	465
12.20.1 Preferred-Path Decomposition	465
12.20.2 Operations	465
12.20.3 Complexity	466
12.20.4 Applications	466
12.20.5 C++ Implementation Sketch	466
12.21 Heavy-Light Decomposition	468
12.21.1 Heavy and Light Edges	468
12.21.2 Chain Construction	468
12.21.3 Path Operations	468
12.21.4 Complexity	469
12.21.5 Applications	469
12.21.6 C++ Implementation Sketch	469
12.22 Euler Tour Trees	470
12.22.1 Euler Tour Representation	471
12.22.2 Subtree Queries with Fenwick Tree	471
12.22.3 Dynamic Euler Tour Trees	472
12.22.4 Worked Example	472
12.22.5 Complexity	473
12.22.6 Applications	473
12.23 Fully Dynamic Connectivity	473
12.23.1 Trees: the Easy Case	473
12.23.2 General Graphs: the $O(\log^2 n)$ Approach	474
12.23.3 Level Structure Details	475
12.23.4 Lower Bound	476
12.23.5 Comparison Table	476
12.23.6 Applications	476
12.24 Exercises	476
12.25 Common Bugs and Pitfalls	477

12.26	Complexity Summary	477
12.27	Interview FAQs	478
12.28	Exercises	481
12.28.1	Drill	481
12.28.2	Application	481
12.28.3	Research	482
12.29	References and Further Reading	483
13	String Algorithms and Tries	484
13.1	Introduction	484
13.2	Tries	484
13.2.1	Standard Trie	484
13.2.2	Trie Implementation	484
13.2.3	Compressed Trie (Radix Tree)	486
13.2.4	Ternary Search Trie (TST)	486
13.2.5	Application: Autocomplete	487
13.3	String Matching	488
13.3.1	Naive Matching	488
13.3.2	Knuth-Morris-Pratt (KMP)	488
13.3.3	Rabin-Karp Algorithm	489
13.3.4	Boyer-Moore Algorithm	490
13.4	Suffix Arrays and LCP Arrays	490
13.4.1	Suffix Array Construction	490
13.4.2	The DC3/Skew Algorithm	491
13.4.3	LCP Array	492
13.5	Application: Longest Repeated Substring	493
13.6	Application: Genome Pattern Search	493
13.7	Suffix Trees	494
13.7.1	Structure	494
13.7.2	Ukkonen's Algorithm	494
13.7.3	Complexity	497
13.7.4	Applications	497
13.7.5	Suffix Tree vs Suffix Array	498

13.8 Radix Tree (Compressed Trie / Patricia Trie)	498
13.8.1 Structure	498
13.8.2 C++ Implementation	499
13.8.3 Complexity	501
13.8.4 Comparison with Standard Trie and TST	501
13.8.5 Applications	502
13.9 Ternary Search Tree	502
13.9.1 Structure	502
13.9.2 Implementation	503
13.9.3 Complexity	505
13.9.4 Comparison with Standard Trie and Hash Table	505
13.9.5 Application: Simple Spell Checker	505
13.10 Aho-Corasick Algorithm	506
13.10.1 Trie with Failure Links	506
13.10.2 Worked Trace	508
13.11 Z-Algorithm	509
13.11.1 Computing the Z-Array	509
13.11.2 Worked Trace	509
13.12 Longest Common Substring via Suffix Arrays	510
13.12.1 Algorithm	510
13.13 Exercises	511
13.14 Common Bugs and Pitfalls	511
13.15 Summary	512
13.16 Interview FAQs	512
13.17 Document Retrieval	514
13.17.1 The Inverted Index	514
13.17.2 Suffix Arrays with Document Arrays	515
13.17.3 Top-k Retrieval: WAND and Block-Max WAND	515
13.17.4 BM25 Scoring	515
13.17.5 Industrial Applications	516
13.17.6 Comparison	516
13.18 Exercises	517
13.18.1 Drill	517

13.18.2 Application	517
13.18.3 Research	518
13.19 References and Further Reading	518
14 Segment Trees, Fenwick Trees, Sparse Tables, and Union-Find	520
14.1 Introduction	520
14.2 Sparse Table	520
14.2.1 Structure	520
14.2.2 Build	521
14.2.3 Complexity	521
14.2.4 Application: Range Minimum Query	521
14.2.5 When to Use	522
14.3 Segment Tree	522
14.3.1 Structure	522
14.3.2 Implementation	523
14.3.3 Query Walkthrough	524
14.3.4 Lazy Propagation	524
14.3.5 Application: Range Minimum Query	525
14.4 Fenwick Tree (Binary Indexed Tree)	525
14.4.1 The Idea	525
14.4.2 Implementation	525
14.4.3 Application: Inversion Count	527
14.5 Union-Find (Disjoint Set Union)	527
14.5.1 Optimal Implementation	527
14.5.2 Complexity	528
14.5.3 Application: Kruskal's MST	528
14.5.4 Application: Percolation	528
14.6 Interval Tree	529
14.6.1 Applications	529
14.6.2 Structure	530
14.6.3 Implementation	530
14.6.4 How the Search Algorithm Works	532
14.6.5 Complexity	532
14.6.6 Application: Meeting Room Scheduler	532

14.6.7 Interval Tree vs. Brute Force	533
14.7 Comparison	533
14.8 Common Bugs and Pitfalls	534
14.9 Summary	534
14.10 Interview FAQs	534
14.11 Persistent Segment Tree	537
14.11.1 Structure	537
14.11.2 Implementation	537
14.11.3 Query on Historical Versions	538
14.11.4 Complexity	539
14.11.5 Application: Range Count Queries	539
14.12 Square Root Decomposition	539
14.12.1 The Idea	540
14.12.2 Implementation: Range Sum and Range Min	540
14.12.3 Complexity	542
14.12.4 When to Use	542
14.13 Sparse Table for GCD/LCM	542
14.13.1 GCD Queries	542
14.13.2 Why GCD Works with Overlapping Intervals	543
14.13.3 LCM Queries	544
14.13.4 Complexity	544
14.14 Segment Tree Beats	544
14.14.1 Core Idea	544
14.14.2 Implementation	545
14.14.3 Complexity and Use Cases	547
14.15 2D Fenwick Tree	547
14.15.1 Structure	547
14.15.2 Implementation	547
14.15.3 Complexity and Use Cases	548
14.16 Orthogonal Range Trees	548
14.16.1 Structure	548
14.16.2 Query Decomposition	549
14.16.3 Fractional Cascading	550

14.16.4 Kd-Trees	550
14.16.5 Applications	551
14.16.6 Comparison	551
14.17 Higher-Dimensional Segment Trees	551
14.17.1 Structure	552
14.17.2 Point Update	552
14.17.3 Complexity	553
14.17.4 Lazy Propagation in 2D	554
14.17.5 Coordinate Compression	554
14.17.6 Applications	554
14.18 3D Orthogonal Range Search	554
14.18.1 Structure	554
14.18.2 Complexity	554
14.18.3 Counting Queries	555
14.19 Semigroup Model	555
14.19.1 Group vs. Semigroup	555
14.19.2 Why Fenwick Fails for min	555
14.19.3 Segment Tree for Semigroup Queries	555
14.19.4 When to Use Which	557
14.20 Persistent Union-Find	557
14.20.1 Why Persistence Is Hard	557
14.20.2 Fat Nodes	557
14.20.3 Path Copying	558
14.20.4 Complexity	559
14.21 List Splitting	559
14.21.1 Applications	559
14.21.2 Splitting with Treaps	559
14.22 Maintenance of Linear Order	561
14.22.1 Order-Statistic Trees	561
14.22.2 Applications	561
14.23 Union of Intervals	561
14.23.1 Sweep-Line Algorithm	561
14.23.2 Dynamic Interval Union	562

14.24	Interval-Restricted Maximum Sum	562
14.24.1	Segment Tree Augmentation	563
14.24.2	Comparison	565
14.25	Level Ancestor Problem	565
14.25.1	Jump Pointers	565
14.25.2	Optimal $O(1)$ Query	566
14.25.3	Applications	566
14.26	The Predecessor Problem and Lower Bounds	566
14.26.1	Information-Theoretic Lower Bound	566
14.26.2	Cell Probe Lower Bound	566
14.26.3	Upper Bounds	566
14.26.4	The Open Gap	567
14.27	Kinetic Data Structures	567
14.27.1	Certificates and Failure Times	567
14.27.2	Kinetic Tournament	567
14.27.3	Industrial Applications	568
14.27.4	Comparison	569
14.28	Planar Point Location	569
14.28.1	The Problem	569
14.28.2	Trapezoidal Maps	569
14.28.3	Kirkpatrick's Hierarchy	569
14.28.4	Persistent Search Trees	570
14.28.5	Industrial Applications	570
14.29	Exercises	571
14.29.1	Drill	571
14.29.2	Application	571
14.29.3	Research	572
14.30	References and Further Reading	574
III	Algorithm-Design Methods	575
15	The Greedy Method	576
15.1	Introduction	576

15.2 The General Method	576
15.2.1 Greedy Algorithm Template	576
15.2.2 When Greedy Works	576
15.2.3 The Exchange Argument	577
15.3 Container Loading	577
15.4 The Fractional Knapsack Problem	578
15.5 Task Scheduling	579
15.6 Single-Source Shortest Paths (Dijkstra)	580
15.6.1 The Algorithm	580
15.6.2 Worked Trace	580
15.7 Minimum Spanning Tree	581
15.7.1 Kruskal's Algorithm	581
15.7.2 Prim's Algorithm	582
15.7.3 The Cut Property	583
15.8 Huffman Coding	583
15.9 Greedy Does Not Always Work	584
15.10 Common Bugs and Pitfalls	585
15.11 Summary	585
15.12 Interview FAQs	586
15.13 Activity Selection Problem	588
15.13.1 Worked Example	588
15.13.2 C++ Implementation	589
15.14 Job Sequencing with Deadlines	590
15.14.1 Worked Trace	590
15.14.2 Union-Find Optimization	590
15.15 Optimal Merge Pattern	591
15.15.1 Why Greedy Works	592
15.15.2 Worked Trace	592
15.15.3 C++ Implementation	592
15.16 Egyptian Fraction	593
15.16.1 Worked Examples	593
15.16.2 C++ Implementation	594
15.16.3 Task Scheduling to Minimize Lateness	595

15.17 Exercises	596
15.17.1 Drill	596
15.17.2 Application	597
15.17.3 Research	598
15.17.4 Additional Exercises	598
15.18 References and Further Reading	600
16 Divide and Conquer	601
16.1 Introduction	601
16.2 The General Method	601
16.3 Defective Chessboard	601
16.4 Binary Search	603
16.5 Finding Min and Max	604
16.6 Sorting Algorithms	605
16.6.1 Merge Sort	605
16.6.2 Natural Merge Sort	605
16.6.3 Quick Sort	606
16.6.4 Comparison	607
16.7 Radix Sort	607
16.7.1 LSD Radix Sort	607
16.7.2 MSD Radix Sort	608
16.7.3 Radix Sort for Strings	609
16.7.4 Complexity	609
16.7.5 Industrial Applications	609
16.8 Selection (Finding the k-th Smallest)	610
16.9 Closest Pair of Points	611
16.10 Strassen's Matrix Multiplication	612
16.11 Common Bugs and Pitfalls	613
16.12 Summary	613
16.13 Interview FAQs	614
16.14 Closest Pair of Points: Detailed Treatment	616
16.14.1 Brute Force: $O(n^2)$	616
16.14.2 Divide and Conquer: $O(n \log n)$	616

16.15	Maximum Subarray	619
16.15.1	Divide and Conquer Approach	619
16.15.2	Kadane's Algorithm: $O(n)$	620
16.16	Counting Inversions	621
16.16.1	Brute Force: $O(n^2)$	621
16.16.2	Merge Sort Variant: $O(n \log n)$	621
16.17	Strassen's Algorithm: Recursive Trace	623
16.18	Median of Two Sorted Arrays	624
16.19	Karatsuba Multiplication	625
16.20	Exercises	628
16.20.1	Drill	628
16.20.2	Application	628
16.20.3	Research	629
16.20.4	Additional Problems	629
16.21	References and Further Reading	630
17	Dynamic Programming	632
17.1	Introduction	632
17.2	The General Method	632
17.2.1	Key Concepts	632
17.2.2	Top-Down vs. Bottom-Up	632
17.2.3	DP Design Steps	633
17.3	The 0/1 Knapsack Problem	633
17.3.1	Recursive Formulation	633
17.3.2	Bottom-Up Implementation	634
17.3.3	Space-Optimized Version	634
17.3.4	Reconstructing the Solution	634
17.4	Matrix Chain Multiplication	635
17.4.1	Recurrence	635
17.4.2	Implementation	635
17.5	All-Pairs Shortest Paths (Floyd-Warshall)	636
17.6	Longest Common Subsequence (LCS)	638
17.6.1	Recurrence	638

17.6.2 Implementation	638
17.7 Edit Distance (Levenshtein Distance)	639
17.7.1 Recurrence	639
17.7.2 Implementation	639
17.8 Coin Change (Unbounded)	640
17.8.1 Recurrence	640
17.9 DP vs. Greedy vs. Divide and Conquer	641
17.10 Common Bugs and Pitfalls	642
17.11 Summary	642
17.12 Interview FAQs	643
17.13 Longest Common Subsequence (LCS) — Detailed Treatment	645
17.13.1 DP Formulation	645
17.13.2 Table Construction: X = "ABCBADAB", Y = "BDCAB"	645
17.13.3 Backtracking to Find the Actual Subsequence	646
17.13.4 C++ Implementation	646
17.14 Edit Distance (Levenshtein Distance) — Detailed Treatment	647
17.14.1 DP Formulation	647
17.14.2 Table Construction: A = "kitten", B = "sitting"	647
17.14.3 C++ Implementation	648
17.14.4 Backtracking to Find the Operations	649
17.15 Matrix Chain Multiplication — Detailed Treatment	649
17.15.1 DP Formulation	650
17.15.2 Worked Trace: 4 Matrices	650
17.15.3 C++ Implementation	651
17.15.4 Printing the Optimal Parenthesization	651
17.16 Subset Sum Problem	652
17.16.1 DP Formulation	652
17.16.2 Worked Trace: nums = [3, 7, 1, 8, 4], T = 11	652
17.16.3 C++ Implementation	653
17.16.4 Space-Optimized Version	653
17.17 House Robber	654
17.17.1 Recurrence	654
17.17.2 Tabulation	654

17.17.3 Space-Optimized Version	655
17.18 Longest Increasing Subsequence (LIS)	655
17.18.1 $O(n^2)$ DP	655
17.18.2 $O(n \log n)$ Patience Sorting	656
17.18.3 Worked Trace: Patience Sorting	657
17.18.4 Recovering the Actual Subsequence	657
17.19 Exercises	658
17.19.1 Drill	658
17.19.2 Application	659
17.19.3 Research	660
17.20 References and Further Reading	661
18 Backtracking and Branch and Bound	662
18.1 Introduction	662
18.2 Backtracking	662
18.2.1 The General Method	662
18.2.2 The n-Queens Problem	663
18.2.3 Subset Sum	664
18.2.4 Subset Generation (All Subsets)	665
18.2.5 Permutation Generation	665
18.3 Branch and Bound	666
18.3.1 The General Method	666
18.3.2 Traveling Salesman Problem (TSP)	666
18.3.3 15-Puzzle	667
18.3.4 Backtracking vs. Branch and Bound	669
18.4 Application: Sudoku Solver	670
18.5 Common Bugs and Pitfalls	671
18.6 Summary	671
18.7 Interview FAQs	671
18.8 N-Queens Problem: A Deep Dive	673
18.8.1 State Space Tree for 4-Queens	674
18.8.2 Worked Trace: Backtracking for 4-Queens	674
18.8.3 Optimized C++ Implementation with Bitmasks	675

18.8.4 Complexity Analysis	677
18.9 Sudoku Solver: Backtracking with Constraint Checking	678
18.9.1 Algorithm Overview	678
18.9.2 Worked Partial Trace	678
18.9.3 C++ Implementation	679
18.9.4 Optimizations	680
18.10 Subset Sum: Backtracking vs. Dynamic Programming	682
18.10.1 Backtracking Approach (Revisited)	682
18.10.2 Dynamic Programming Approach (from Chapter 17)	683
18.10.3 Comparison	684
18.11 Graph Coloring: Backtracking Approach	684
18.11.1 Algorithm	684
18.11.2 Worked Trace: Graph with 4 Vertices and 3 Colors	685
18.11.3 Finding the Chromatic Number	686
18.11.4 Applications	687
18.12 Exercises	687
18.12.1 Drill	687
18.12.2 Application	687
18.12.3 Research	688
18.12.4 Additional Exercises	689
18.13 References and Further Reading	690
IV Specialized and Applied Topics	691
19 Probabilistic Data Structures	692
19.1 Introduction	692
19.2 Bloom Filters	692
19.2.1 Operations	692
19.3 Cuckoo Filters	693
19.3.1 Core Idea	693
19.3.2 Cuckoo Filter Implementation	694
19.3.3 Cuckoo Filter vs Bloom Filter	696
19.4 Count-Min Sketch	697

19.5 Heavy Hitters (Misra-Gries)	698
19.5.1 The Idea	698
19.5.2 Implementation	698
19.5.3 Complexity	699
19.5.4 Applications	699
19.6 HyperLogLog	699
19.7 Quantile Estimation	700
19.7.1 Greenwald-Khanna Sketch	701
19.7.2 Complexity	702
19.7.3 Applications	702
19.8 Reservoir Sampling	702
19.9 Sliding Window Model	703
19.9.1 Exponential Histogram	703
19.9.2 Applications	704
19.10 Consistent Hashing	705
19.11 Locality-Sensitive Hashing	706
19.12 Merkle Tree (Hash Tree)	706
19.12.1 Structure	707
19.12.2 Operations	707
19.12.3 Complexity	709
19.12.4 Applications	709
19.13 Quotient Filter	709
19.13.1 Structure	709
19.13.2 Operations	709
19.13.3 Complexity	713
19.13.4 Comparison with Bloom Filter	713
19.14 MinHash	713
19.14.1 Structure	713
19.14.2 Operations	714
19.14.3 Complexity	715
19.14.4 Applications	715
19.15 HyperLogLog	715
19.15.1 Core Idea	715

19.15.2 Standard Error	716
19.15.3 Complexity	717
19.15.4 Applications	717
19.16 Reservoir Sampling	718
19.16.1 Algorithm R	718
19.16.2 Correctness Argument	718
19.16.3 Complexity	720
19.16.4 Applications	720
19.17 Count-Distinct (Linear Counting)	720
19.17.1 Estimate Formula	720
19.17.2 When to Use	721
19.18 Exercises	721
19.18.1 Top-K Approximation	722
19.19 Common Bugs and Pitfalls	724
19.20 Summary	724
19.21 Exercises	724
19.21.1 Drill	724
19.21.2 Application	725
19.21.3 Research	726
19.22 References and Further Reading	726
20 Maximum Flow and Matching	728
20.1 Introduction	728
20.2 The Maximum Flow Problem	728
20.3 Dinic's Algorithm	728
20.4 Edmonds-Karp Algorithm	730
20.5 Push-Relabel Algorithm	731
20.6 Minimum Cut	732
20.7 Bipartite Matching	733
20.8 Min-Cost Max-Flow	734
20.9 Application: Project Selection	736
20.10 Common Bugs and Pitfalls	737
20.11 Summary	737

20.12 Interview FAQs	738
20.13 Edmonds-Karp in Depth	740
20.14 Maximum Bipartite Matching (Full Treatment)	743
20.15 Minimum Cost Maximum Flow: Full Treatment	744
20.16 Applications Deep Dive	746
20.16.1 Airline Scheduling: Crew Assignment	746
20.16.2 Image Segmentation: Graph Cuts	747
20.16.3 Baseball Elimination: Proving a Team Cannot Win	748
20.17 Exercises	750
20.17.1 Drill	750
20.17.2 Application	751
20.17.3 Research	752
20.18 References and Further Reading	752
21 Persistent Data Structures	753
21.1 Introduction and Motivation	753
21.2 Persistence Models	753
21.3 Techniques	754
21.3.1 Fat Nodes	754
21.3.2 Path Copying (Node Copying)	755
21.3.3 Structural Sharing	755
21.3.4 Comparing the Techniques	756
21.4 Persistent Arrays	756
21.5 Persistent Linked List	758
21.6 Persistent Balanced BST	759
21.6.1 Persistent AVL Tree	759
21.6.2 Merge for Confluent Persistence	762
21.7 Persistent Segment Tree	762
21.8 Applications	764
21.8.1 Undo/Redo in a Text Editor	764
21.8.2 Git-like Snapshotting	764
21.8.3 Multi-Version Concurrency Control (MVCC)	764
21.8.4 Immutable State in Functional Programming	764

21.9 Complexity Summary	765
21.10 Common Bugs and Pitfalls	765
21.11 Exercises	765
21.12 Full Persistence vs Partial Persistence	766
21.12.1 Partial Persistence	766
21.12.2 Full Persistence	766
21.12.3 Fat Nodes vs Path Copying	767
21.12.4 Confluent Persistence	767
21.13 Functional Data Structures	767
21.13.1 Lazy Evaluation and Lazy Lists	767
21.13.2 Lazy Segment Trees	767
21.13.3 Amortised Analysis: Banker's Method	767
21.13.4 Comparison with Imperative Counterparts	768
21.14 The Differential File	768
21.14.1 Shadow Paging	768
21.14.2 Differential File	768
21.14.3 Write-Ahead Logging	768
21.14.4 Multi-Version Concurrency Control	768
21.15 Retroactive Data Structures	769
21.15.1 Partial Retroactivity	769
21.15.2 Full Retroactivity	769
21.15.3 Applications	769
21.16 Nonoblivious Retroactivity	769
21.16.1 Problem Formulation	769
21.16.2 Priority Queue Example	770
21.16.3 Applications	770
21.17 Data Structure Decomposition	770
21.17.1 Persistent Union-Find via Decomposition	770
21.17.2 Persistent Graph via Adjacency Lists	770
21.17.3 Compositional Design Principles	770
21.18 References and Further Reading	771
22 Succinct Data Structures	772

22.1	Introduction	772
22.2	Rank and Select on Bitvectors	772
22.3	Wavelet Tree	774
22.3.1	Structure	774
22.3.2	Operations	775
22.4	Burrows-Wheeler Transform and FM-Index	778
22.4.1	Burrows-Wheeler Transform	778
22.4.2	FM-Index	779
22.5	Succinct Trees	781
22.5.1	Balanced Parentheses	781
22.5.2	LOUDS	782
22.6	Compressed Suffix Arrays	784
22.7	Applications and Industry Use	785
22.8	Common Bugs and Pitfalls	785
22.9	Complexity Summary	786
22.10	Exercises	786
22.11	References and Further Reading	787
23	Data Structures at Scale: How Industry Builds on Classical Foundations	788
23.1	Introduction	788
23.2	Log-Structured Merge Trees — The Engine Behind Modern Databases	788
23.2.1	Motivation	788
23.2.2	Architecture	788
23.2.3	Write Path	789
23.2.4	Read Path	791
23.2.5	Compaction Strategies	791
23.2.6	Real-World Implementations	791
23.2.7	LSM vs. B-tree: A Comparison	791
23.3	Consistent Hashing — Distributing Data Without Central Control	791
23.3.1	The Problem	791
23.3.2	The Ring	791
23.3.3	Virtual Nodes	793
23.3.4	Production Use	793

23.4 Column-Family Storage — Wide Columns at Scale	793
23.4.1 Data Model	793
23.4.2 Bigtable and Its Descendants	793
23.4.3 Why Column Families Win for Analytics	795
23.5 Bloom Filters in Production	795
23.5.1 Recap	795
23.5.2 At Scale	795
23.5.3 Alternatives	795
23.6 Copy-on-Write and Versioned Storage	795
23.6.1 Persistent Data Structures via CoW	795
23.6.2 Production Systems	797
23.7 Priority Queues and Scheduling at Scale	797
23.7.1 Kafka and Message Priorities	797
23.7.2 Redis Sorted Sets	797
23.8 Graph Data Structures in Social Networks	799
23.8.1 The Social Graph	799
23.8.2 Property Graph Model	799
23.8.3 Graph Partitioning	800
23.9 Hardware-Aware Data Structure Design	801
23.9.1 Cache Line Awareness	801
23.9.2 NUMA-Aware Allocation	801
23.9.3 SOA vs AOS	802
23.10 Comparison Table and Design Decisions	802
23.11 Exercises	802
23.11.1 Drill	802
23.11.2 Application	803
23.11.3 Research	803
23.12 Industrial Perspectives	803
23.12.1 Complete Relevance Table	806
23.12.2 Tier 3 — High Relevance: The Industrial Workhorses	807
23.12.3 Tier 2 — Medium Relevance: Domain Specialists	807
23.12.4 Tier 1 — Low Relevance: Educational and Theoretical	807
23.12.5 What to Learn for Industry	808

23.12.6 Industrial Perspectives—Exercises	808
23.13 References and Further Reading	808
A C++ STL Quick Reference	810
A.1 A.1 Containers	810
A.2 A.2 Algorithms (in <code><algorithm></code>)	811
A.3 A.3 Chrono (<code><chrono></code>)	811
A.4 A.4 Smart Pointers (<code><memory></code>)	811
A.5 A.5 Random (<code><random></code>)	811
A.6 A.6 Utilities	812
B Complexity Classes	813
B.1 B.1 Common Time Complexities	813
B.2 B.2 NP-Completeness	813
B.3 B.3 Master Theorem (Refresher)	814
C Mathematical Review	815
C.1 C.1 Summations	815
C.2 C.2 Logarithms	815
C.3 C.3 Combinatorics	815
C.4 C.4 Important Inequalities	815
C.5 C.5 Useful Constants	816
C.6 D.1 External Memory Model	816
C.6.1 Model Parameters	816
C.6.2 Sorting in External Memory	816
C.6.3 B-Trees as External Memory Structures	817
C.6.4 Buffer Trees	817
C.7 D.2 Cache-Oblivious Algorithms	817
C.7.1 The Cache Model	817
C.7.2 Cache-Oblivious B-Tree	817
C.7.3 Cache-Oblivious Priority Queue	818
C.7.4 Cache-Oblivious Matrix Multiplication	818
C.7.5 Why Cache-Oblivious Works	818
C.8 D.25 Lazy Funnelsort and Distribution Sweeping	818

C.8.1	Lazy Funnelsort	818
C.8.2	Distribution Sweeping	819
C.8.3	Online Cache-Oblivious 2D Range Searching	819
C.9	D.3 Ordered File Maintenance	819
C.9.1	The Problem	819
C.9.2	The Algorithm	819
C.9.3	Connection to Cache-Oblivious B-Trees	820
C.9.4	Connection to List Labeling	820
C.9.5	C++ Implementation Sketch	820
C.9.6	Industrial Impact	821
C.10	D.3 Pointer Machine Complexity	821
C.10.1	Separation from RAM	822
C.10.2	Optimal Pointer-Machine Structures	822
C.11	References and Further Reading	822

Part I

Preliminaries

Modern C++ Review

1.1 Introduction

This chapter reviews the C++20 features we will use throughout the book. If you have programmed in C++ before, much of this will be familiar — but pay attention to the newer features like move semantics, smart pointers, concepts, and ranges, as they are used extensively in our implementations.

Our goal is not to teach C++ from scratch, but to establish a common vocabulary and set of patterns. Readers needing a more thorough introduction should consult the references at the end of this chapter.

⚠ **Complete compilable implementations of the data structures in this chapter are in `code/`.**

1.2 Functions and Parameters

1.2.1 Pass by Value, Reference, and Const Reference

C++ offers three primary ways to pass arguments:

```
1 // Pass by value -- a copy is made
2 int square(int x) {
3     return x * x;
4 }
5
6 // Pass by reference -- no copy, can modify
7 void increment(int& x) {
8     ++x;
9 }
10
11 // Pass by const reference -- no copy, read-only
12 int sum(const std::vector<int>& values) {
13     int s = 0;
14     for (int v : values) s += v;
15     return s;
16 }
```

Rule of thumb: Use pass by value for small types (int, char, pointer). Use const reference for types larger than a pointer. Use reference only when you need to modify the argument.

1.2.2 Default Arguments

```

1 int multiply(int a, int b = 1) {
2     return a * b;
3 }
4 // multiply(5) == 5, multiply(5, 3) == 15

```

1.2.3 Function Overloading

Multiple functions may share a name as long as their parameter types differ:

```

1 int max(int a, int b) { return a > b ? a : b; }
2 double max(double a, double b) { return a > b ? a : b; }

```

1.2.4 Lambda Expressions

Lambdas are anonymous function objects. They are essential for generic algorithms:

```

1 auto is_positive = [](int x) { return x > 0; };
2 auto sum = [](int a, int b) { return a + b; };
3
4 // With capture
5 int offset = 10;
6 auto shifted = [offset](int x) { return x + offset; };
7
8 // Mutable lambda (captured variables can be modified)
9 auto counter = [count = 0]() mutable { return ++count; };
10
11 // Generic lambda (C++14)
12 auto add = [](auto a, auto b) { return a + b; };
13
14 // Lambda as a comparator
15 std::vector<int> v = {3, 1, 4, 1, 5};
16 std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });

```

1.3 Dynamic Memory and RAI

1.3.1 The Problem with Raw Pointers

In C++98, dynamic memory was managed with `new` and `delete`:

```

1 int* p = new int(5);
2 // ... use p ...
3 delete p; // easy to forget

```

Forgetting to `delete` causes memory leaks. Deleting twice causes undefined behavior. Exception safety is manual and error-prone.

1.3.2 Smart Pointers (C++11)

C++11 introduced smart pointers that automatically free memory when they go out of scope:

```
1 #include <memory>
2
3 std::unique_ptr<int> p = std::make_unique<int>(5);
4 // No delete needed -- freed automatically when p goes out of scope
```

std::unique_ptr — exclusive ownership. Cannot be copied, only moved.

```
1 auto p1 = std::make_unique<int>(5);
2 // auto p2 = p1; // error: cannot copy unique_ptr
3 auto p2 = std::move(p1); // OK: transfer ownership
4 // p1 is now null
```

std::shared_ptr — shared ownership via reference counting.

```
1 auto s1 = std::make_shared<int>(5);
2 auto s2 = s1; // OK: both now point to the same int
3 // Memory freed when both s1 and s2 go out of scope
```

std::weak_ptr — non-owning observer that can detect when the object is destroyed.

Throughout this book, we use `std::unique_ptr` for owning pointers in linked data structures. We use raw pointers only for non-owning references.

1.3.3 RAII (Resource Acquisition Is Initialization)

RAII is the most important C++ idiom. Resources (memory, file handles, mutexes) are acquired in a constructor and released in the destructor:

```
1 class FileHandle {
2     FILE* f_;
3 public:
4     FileHandle(const char* name, const char* mode)
5         : f_(fopen(name, mode)) {
6         if (!f_) throw std::runtime_error("Cannot open file");
7     }
8     ~FileHandle() { if (f_) fclose(f_); }
9
10    // No copy (for simplicity)
11    FileHandle(const FileHandle&) = delete;
12    FileHandle& operator=(const FileHandle&) = delete;
13 };
```

Every data structure in this book follows RAII principles.

1.4 Templates and Concepts

1.4.1 Function Templates

```
1 template <typename T>
2 T max_value(T a, T b) {
3     return a > b ? a : b;
4 }
```

1.4.2 Class Templates

```
1 template <typename T>
2 class Box {
3     T value_;
4 public:
5     explicit Box(T value) : value_(std::move(value)) {}
6     const T& get() const { return value_; }
7     void set(const T& value) { value_ = value; }
8 };
```

1.4.3 Template Template Parameters and Variadic Templates

```
1 template <typename T, template <typename...> class Container = std::vector
2 >
3 class MyCollection {
4     Container<T> items_;
5 };
6 }
```

1.4.4 Concepts (C++20)

Concepts constrain template parameters with compile-time predicates:

```
1 template <typename T>
2 concept Comparable = requires(T a, T b) {
3     { a < b } -> std::convertible_to<bool>;
4 };
5
6 template <Comparable T>
7 T min_value(T a, T b) {
8     return a < b ? a : b;
9 }
10
11 // Equivalent abbreviated form:
12 auto min_value(Comparable auto a, Comparable auto b) {
13     return a < b ? a : b;
```

```
14 }
```

Concepts give better error messages and enable overloading based on type properties:

```
1 template <typename T>
2 concept Arithmetic = std::is_arithmetic_v<T>;
3
4 template <Arithmetic T>
5 T sum(T a, T b) { return a + b; }
```

Throughout this book, we use concepts to make template interfaces self-documenting.

1.5 Move Semantics

1.5.1 Lvalues and Rvalues

An *lvalue* has an identity and persists beyond a single expression. An *rvalue* is temporary.

```
1 int x = 5;           // x is an lvalue, 5 is an rvalue
2 int y = x + 2;      // (x + 2) is an rvalue
```

1.5.2 Move Constructor and Move Assignment

Move semantics transfer resources from a temporary object, avoiding expensive copies:

```
1 class Buffer {
2     size_t size_;
3     int* data_;
4 public:
5     // Constructor
6     explicit Buffer(size_t size) : size_(size), data_(new int[size]) {}
7
8     // Destructor
9     ~Buffer() { delete[] data_; }
10
11     // Copy constructor (expensive)
12     Buffer(const Buffer& other) : size_(other.size_), data_(new int[other.
13         size_]) {
14         std::copy(other.data_, other.data_ + size_, data_);
15     }
16
17     // Move constructor (cheap)
18     Buffer(Buffer&& other) noexcept
19         : size_(other.size_), data_(other.data_) {
20         other.size_ = 0;
21         other.data_ = nullptr;
22     }
```

```

23     // Move assignment
24     Buffer& operator=(Buffer&& other) noexcept {
25         if (this != &other) {
26             delete[] data_;
27             size_ = other.size_;
28             data_ = other.data_;
29             other.size_ = 0;
30             other.data_ = nullptr;
31         }
32         return *this;
33     }
34 };

```

1.5.3 Rule of Five

If you define any of: destructor, copy constructor, copy assignment, move constructor, or move assignment, you should typically define all five.

In modern C++ with smart pointers, the Rule of Five is rarely needed — the Rule of Zero applies: let the smart pointers manage resources automatically.

1.6 `std::vector` and `std::span`

1.6.1 `std::vector`

`std::vector` is the most important container in modern C++. It is a dynamic array:

```

1  std::vector<int> v;           // empty
2  std::vector<int> v(10);      // 10 elements, default-initialized
3  std::vector<int> v = {1,2,3}; // initializer list
4
5  v.push_back(4);             // add to end: O(1) amortized
6  v.pop_back();               // remove from end: O(1)
7  v[0] = 10;                  // O(1) index
8  v.size();                   // number of elements
9  v.empty();                  // true if size == 0

```

1.6.2 `std::span` (C++20)

`std::span` is a non-owning view over a contiguous sequence:

```

1  void process(std::span<int> data) {
2      for (int& x : data) x *= 2;
3  }
4
5  std::vector<int> v = {1, 2, 3};
6  process(v);           // span from vector

```



```

7 int arr[] = {4, 5, 6};
8 process(arr);           // span from C array

```

`std::span` is ideal for function parameters that need to read or write a contiguous sequence without being tied to a specific container type.

1.6.3 `std::string_view` (C++17)

`std::string_view` is a non-owning view over a character sequence:

```

1 auto len(std::string_view s) -> size_t {
2     return s.size();
3 }
4 std::string a = "hello";
5 len(a);           // from std::string
6 len("world");     // from string literal -- no allocation!

```

1.7 Ranges and Views (C++20)

C++20 introduces the Ranges library for composable operations on sequences:

```

1 #include <ranges>
2 #include <vector>
3 #include <iostream>
4
5 std::vector<int> v = {1, 2, 3, 4, 5, 6};
6
7 // Filter even numbers, square them, take first 2
8 auto result = v
9     | std::views::filter([](int n) { return n % 2 == 0; })
10    | std::views::transform([](int n) { return n * n; })
11    | std::views::take(2);
12
13 for (int x : result) std::cout << x << ' '; // prints: 4 16

```

Ranges are used throughout the book where they improve clarity.

1.7.1 Zip Containers (C++23)

A **zip container** iterates over multiple containers simultaneously, producing tuples of corresponding elements. This is the C++ equivalent of Python's `zip()`.

Manual Zip Pattern (Pre-C++23)

Before C++23, you had to write the loop manually:

```

1 std::vector<int> ids = {1, 2, 3};
2 std::vector<std::string> names = {"Alice", "Bob", "Charlie"};

```

```

3 std::vector<double> scores = {95.0, 87.5, 91.0};
4
5 // Manual zip: iterate in parallel
6 for (int i = 0; i < (int)ids.size(); ++i)
7     std::cout << ids[i] << " " << names[i] << " " << scores[i] << '\n';
8
9 // Using structured bindings (C++17)
10 for (auto [id, name, score] : ???) // no clean way without zip
11     std::cout << id << " " << name << " " << score << '\n';

```

std::ranges::zip_view (C++23)

C++23 adds `std::ranges::zip_view` which creates a single view over multiple ranges:

```

1 #include <ranges>
2 #include <vector>
3 #include <string>
4 #include <iostream>
5
6 std::vector<int> ids = {1, 2, 3};
7 std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
8 std::vector<double> scores = {95.0, 87.5, 91.0};
9
10 // zip_view: iterate all three simultaneously
11 for (auto [id, name, score] : std::ranges::zip_view(ids, names, scores))
12     std::cout << id << " " << name << " " << score << '\n';
13 // 1 Alice 95
14 // 2 Bob 87.5
15 // 3 Charlie 91
16
17 // C++23 shorthand: std::zip
18 for (auto [id, name, score] : std::zip(ids, names, scores))
19     std::cout << id << " " << name << " " << score << '\n';

```

Key properties:

- The zip view stops at the shortest range (like Python's `zip`)
- Elements are `std::tuple` references – structured bindings work naturally
- It is a *lazy view* – no copying, no allocation
- Supports all range operations: `filter`, `transform`, `take`, etc.

```

1 // Combine zip with other views
2 auto result = std::ranges::zip_view(ids, scores)
3     | std::views::filter([](auto t) { return std::get<1>(t) > 90.0; })
4     | std::views::transform([](auto t) { return std::get<0>(t); });
5

```

```
6 for (int id : result) std::cout << id << ' '; // 1 3
```

Use Cases

- **Parallel iteration:** process multiple arrays element-by-element
- **Matrix transpose:** zip rows and columns
- **Interleave:** merge two sequences into pairs
- **Sorting by criteria:** sort one array based on values in another

```
1 // Sort names by score (descending)
2 std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
3 std::vector<double> scores = {95.0, 87.5, 91.0};
4
5 auto zipped = std::ranges::zip_view(scores, names);
6 std::ranges::sort(zipped, std::ranges::greater{});
7 // names is now {"Alice", "Charlie", "Bob"}
8
9 // Interleave two vectors
10 std::vector<int> a = {1, 3, 5};
11 std::vector<int> b = {2, 4, 6};
12 for (auto [x, y] : std::ranges::zip_view(a, b)) {
13     std::cout << x << ' ' << y << ' '; // 1 2 3 4 5 6
14 }
```

Interview FAQs

1. How do you iterate over multiple arrays simultaneously in C++?

Pre-C++23: use index-based loops or iterators manually. C++23: use `std::ranges::zip_view` or `std::zip` to create a lazy view over multiple ranges. The zip view produces tuples of corresponding elements and stops at the shortest range. For pre-C++23 code, a simple `for (int i = 0; i < n; ++i)` loop works.

1. What happens if the ranges have different sizes?

The zip view stops at the shortest range. Elements beyond the shortest range are silently ignored. This matches Python's `zip()` behavior. If you need all elements (padding shorter ranges), use `std::ranges::zip_transform_view` or implement a custom zip with a default value.

1.8 Error Handling

1.8.1 Exceptions

C++ uses exceptions for error handling:

```

1 #include <stdexcept>
2
3 int divide(int a, int b) {
4     if (b == 0) throw std::invalid_argument("Division by zero");
5     return a / b;
6 }

```

Standard exception classes: `std::logic_error`, `std::invalid_argument`, `std::domain_error`, `std::length_error`, `std::out_of_range`, `std::runtime_error`, `std::range_error`, `std::overflow_error`.

1.8.2 noexcept

Functions that cannot throw should be marked `noexcept`:

```

1 void swap(int& a, int& b) noexcept {
2     int t = a; a = b; b = t;
3 }

```

Move constructors and move assignments should be `noexcept` to enable optimal container performance.

1.9 Iterators and Ranges

1.9.1 Iterator Categories

- **Input iterators** — read once, forward (`istream_iterator`)
- **Output iterators** — write once, forward (`ostream_iterator`)
- **Forward iterators** — read/write, forward (`forward_list`)
- **Bidirectional iterators** — forward and backward (`list`, `set`, `map`)
- **Random access iterators** — $O(1)$ index (`vector`, `deque`, `array`)

1.9.2 Range-Based for Loop

```

1 std::vector<int> v = {1, 2, 3};
2 for (int x : v) { /* read */ }
3 for (int& x : v) { /* modify */ }
4 for (const auto& x : v) { /* read, avoid copy */ }

```

1.9.3 Iterator Adapters

```

1 std::vector<int> v = {1, 2, 3};
2 // Back inserter
3 std::fill_n(std::back_inserter(v), 3, 0); // appends 0, 0, 0
4
5 // Stream iterator
6 std::copy(v.begin(), v.end(),
7           std::ostream_iterator<int>(std::cout, " "));

```

1.10 Type Deduction: auto and decltype

```

1 auto x = 5;           // int
2 auto y = 3.14;        // double
3 auto z = &x;          // int*
4
5 auto& ref = x;         // int&
6 const auto& cref = x; // const int&
7
8 auto&& universal = x;  // forwarding reference

```

`decltype` deduces the exact type of an expression:

```

1 int x = 5;
2 decltype(x) y = 10; // int
3 decltype(x) z = x;  // int&

```

1.11 Coroutines (C++20)

Coroutines enable suspend/resume functions. We use them for lazy generators:

```

1 #include <generator>
2 #include <ranges>
3
4 std::generator<int> fibonacci(int n) {
5     int a = 0, b = 1;
6     for (int i = 0; i < n; ++i) {
7         co_yield a;
8         int next = a + b;
9         a = b;
10        b = next;
11    }
12 }
13
14 // Usage:
15 // for (int f : fibonacci(10)) { ... }

```

(Full coroutine coverage is beyond this chapter; we introduce them where needed in later chapters.)

1.12 Common Patterns in This Book

Throughout the book, we follow these conventions:

1. **Template parameters** named `T` for the element type.
2. **Concepts** used to document template requirements.
3. **`noexcept`** on move operations and swaps.
4. **`explicit`** on single-argument constructors.
5. **`const`** correctness throughout.
6. **`std::unique_ptr`** for owning pointers in linked structures.
7. **`std::size_t`** for indices and sizes.
8. **Iterators** following STL conventions (begin/end pairs).

1.12.1 Example: A Generic Container Skeleton

```
1 #include <concepts>
2 #include <memory>
3 #include <cstddef>
4
5 template <typename T>
6 class simple_vector {
7 public:
8     using value_type = T;
9     using size_type = std::size_t;
10    using reference = T&;
11    using const_reference = const T&;
12
13    simple_vector() = default;
14
15    explicit simple_vector(size_type n)
16        : size_(n), capacity_(n), data_(std::make_unique<T[]>(n)) {}
17
18    // Rule of Zero -- unique_ptr handles everything
19
20    T& operator[](size_type i) { return data_[i]; }
21    const T& operator[](size_type i) const { return data_[i]; }
22
23    size_type size() const noexcept { return size_; }
24    bool empty() const noexcept { return size_ == 0; }
25
26 private:
```

```

27     size_type size_ = 0;
28     size_type capacity_ = 0;
29     std::unique_ptr<T[]> data_;
30 };

```

1.13 Compiler and Build

All code in this book compiles with:

- GCC 12+ with `-std=c++20`
- Clang 16+ with `-std=c++20`
- MSVC 2022 17.6+ with `/std:c++20`

Recommended build command:

```
1 g++ -std=c++20 -O2 -Wall -Wextra -o program program.cpp
```

1.14 STL Containers Overview

The C++ Standard Template Library provides a rich set of containers. The choice of container affects both performance and correctness. The table below summarizes the most commonly used containers.

Container	Underlying Structure	Access	Insert/Delete	Lookup
<code>vector</code>	Dynamic array	$O(1)$ random	Back $O(1)^*$, Front $O(n)$	$O(n)$
<code>deque</code>	Segmented array	$O(1)$ random	Front/Back $O(1)$	$O(n)$
<code>list</code>	Doubly-linked list	$O(n)$ sequential	Anywhere $O(1)$	$O(n)$
<code>forward_list</code>	Singly-linked list	$O(n)$ sequential	$O(1)$ after	$O(n)$
<code>map</code>	Red-black tree	–	$O(\log n)$	$O(\log n)$
<code>set</code>	Red-black tree	–	$O(\log n)$	$O(\log n)$
<code>unordered_map</code>	Hash table	–	$O(1)$ avg	$O(1)$ avg
<code>unordered_set</code>	Hash table	–	$O(1)$ avg	$O(1)$ avg

*Amortized $O(1)$ for `push_back`; worst case $O(n)$ on reallocation.

`std::vector` is the default container. It stores elements contiguously in memory, providing cache-friendly access and $O(1)$ random access via the index operator.

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 v.push_back(6);           // {1, 2, 3, 4, 5, 6}
3 v.pop_back();             // {1, 2, 3, 4, 5}
4 int x = v[2];             // x = 3
5 v.insert(v.begin(), 0);   // {0, 1, 2, 3, 4, 5}

```

std::deque supports efficient insertion and deletion at both ends. It is implemented as a collection of fixed-size arrays, so random access is $O(1)$ but slightly slower than **vector** due to an extra indirection.

```
1 std::deque<int> dq = {2, 3, 4};
2 dq.push_front(1);    // {1, 2, 3, 4}
3 dq.push_back(5);     // {1, 2, 3, 4, 5}
4 int x = dq[0];       // x = 1
```

std::list is a doubly-linked list. It guarantees $O(1)$ insertion and deletion at any position given an iterator, but does not support random access.

```
1 std::list<int> lst = {1, 3, 5};
2 auto it = lst.begin();
3 std::advance(it, 1);
4 lst.insert(it, 2);    // {1, 2, 3, 5}
5 lst.erase(it);       // {1, 2, 5}
```

std::map and **std::set** are ordered associative containers backed by a red-black tree. Elements are kept in sorted order. Lookup, insertion, and deletion are all $O(\log n)$.

```
1 std::map<std::string, int> ages;
2 ages["Alice"] = 30;
3 ages["Bob"] = 25;
4 auto it = ages.find("Alice"); // O(log n)
5 if (it != ages.end()) {
6     // it->second == 30
7 }
8
9 std::set<int> s = {5, 3, 1, 4, 2};
10 // s is {1, 2, 3, 4, 5} -- always sorted
11 s.insert(6);    // O(log n)
12 s.erase(3);     // O(log n)
```

std::unordered_map and **std::unordered_set** use a hash table. Average-case lookup is $O(1)$, but worst-case (many collisions) is $O(n)$. They do not maintain order.

```
1 std::unordered_map<int, std::string> names;
2 names[1] = "Alice";
3 names[2] = "Bob";
4 auto it = names.find(1); // O(1) average
```

When to use which:

- **vector:** Default choice. Use when you need fast random access and mostly append at the end.
- **deque:** Use when you need efficient push/pop at both ends.
- **list:** Use when you need stable iterators and frequent insertion/deletion in the middle.
- **map/set:** Use when you need sorted order and guaranteed $O(\log n)$ operations.

- **unordered_map/unordered_set:** Use when you need the fastest average lookup and do not care about order.

1.15 Smart Pointers

Raw pointers in C++ do not own memory. Forgetting to call `delete` causes memory leaks, and calling `delete` twice is undefined behavior. Smart pointers automate lifetime management.

1.15.1 `std::unique_ptr`

`unique_ptr` is a lightweight, exclusive-ownership pointer. It has zero overhead compared to a raw pointer. The pointed-to object is destroyed when the `unique_ptr` goes out of scope or is reset.

```
1 #include <memory>
2
3 auto p = std::make_unique<int>(42);
4 // *p == 42
5
6 // Transfer ownership
7 auto q = std::move(p);
8 // p is now null, q owns the int
9
10 q.reset(); // deletes the int
```

`unique_ptr` cannot be copied:

```
1 auto a = std::make_unique<int>(1);
2 // auto b = a;           // error: copy deleted
3 auto b = std::move(a); // OK: move
```

Use `unique_ptr` as the default owning pointer. It makes ownership semantics explicit and prevents accidental sharing.

1.15.2 `std::shared_ptr`

`shared_ptr` uses reference counting to allow multiple owners. The object is destroyed when the last `shared_ptr` pointing to it is destroyed or reset.

```
1 auto s1 = std::make_shared<std::string>("hello");
2 auto s2 = s1; // ref count = 2
3 // s1 and s2 both point to "hello"
4 s2.reset(); // ref count = 1
5 // "hello" still alive (s1 owns it)
```

`shared_ptr` has overhead: a control block stores the reference count, which must be updated atomically. Prefer `unique_ptr` unless you genuinely need shared ownership.

1.15.3 `std::weak_ptr`

`weak_ptr` is a non-owning observer of an object managed by `shared_ptr`. It does not increment the reference count. Use it to break circular references.

```
1 auto shared = std::make_shared<int>(10);
2 std::weak_ptr<int> weak = shared;
3
4 if (auto locked = weak.lock()) {
5     // locked is a shared_ptr; object is alive
6     // *locked == 10
7 }
8 shared.reset(); // object destroyed
9 // weak.lock() now returns null
```

Common pattern: A cache holds `weak_ptr` to cached objects. When the cache is queried, it attempts `lock()`. If the object is still alive, the cache returns a `shared_ptr`; otherwise it re-creates the object.

1.15.4 When to Use Each

- **unique_ptr:** Single owner. Default choice. Use for class members that own heap objects, factory functions, and ownership transfer.
- **shared_ptr:** Multiple owners. Use sparingly, for example when an object is genuinely shared between subsystems.
- **weak_ptr:** Non-owning observation. Use to break circular references in graphs, caches, and callbacks.
- **Raw pointer:** Non-owning access. Use for function parameters that observe but do not own an object.

1.16 Lambda Expressions

A lambda expression creates an anonymous function object. Lambdas are used extensively with STL algorithms, callbacks, and asynchronous code.

1.16.1 Basic Syntax

```
1 // [capture](parameters) -> return_type { body }
2 auto square = [](int x) { return x * x; };
3 int result = square(5); // 25
4
5 // With no parameters
6 auto greet = []() { /* ... */ };
```

```

7
8 // Trailing return type
9 auto divide = [](double a, double b) -> double {
10     return a / b;
11 };

```

1.16.2 Captures

Lambdas capture surrounding variables to use them in the body.

```

1 int offset = 10;
2 int factor = 2;
3
4 // By value -- copy
5 auto by_val = [offset, factor](int x) {
6     return x * factor + offset;
7 };
8
9 // By reference -- no copy, can modify
10 auto by_ref = [&offset, &factor](int x) {
11     factor = 1;
12     return x + offset;
13 };
14
15 // Capture all by value
16 auto all_val = [=](int x) { return x + offset; };
17
18 // Capture all by reference
19 auto all_ref = [&](int x) { offset += x; };
20
21 // Mixed
22 auto mixed = [=, &offset](int x) { return x + offset; };

```

Capture initializers allow captured variables to be initialized with a new expression:

```

1 auto lambda = [count = 0]() mutable { return ++count; };
2 lambda(); // 0
3 lambda(); // 1

```

1.16.3 Use with STL Algorithms

Lambdas are the primary way to customize STL algorithms:

```

1 #include <algorithm>
2 #include <vector>
3 #include <numeric>
4
5 std::vector<int> v = {5, 3, 1, 4, 2};

```

```

6
7 // sort with custom comparator
8 std::sort(v.begin(), v.end(),
9     [](int a, int b) { return a > b; });
10 // v is now {5, 4, 3, 2, 1}
11
12 // find_if -- find first even number
13 auto it = std::find_if(v.begin(), v.end(),
14     [](int x) { return x % 2 == 0; });
15 // *it == 4
16
17 // for_each -- print each element
18 std::for_each(v.begin(), v.end(),
19     [](int x) { std::cout << x << ' '; });
20
21 // count_if -- count positive numbers
22 int n = std::count_if(v.begin(), v.end(),
23     [](int x) { return x > 0; });
24
25 // accumulate with lambda
26 int sum = std::accumulate(v.begin(), v.end(), 0,
27     [](int acc, int x) { return acc + x; });

```

1.17 Move Semantics and Rvalue References

Move semantics avoid copying objects by transferring their internal resources. This is critical for performance when dealing with large objects like vectors, strings, and buffers.

1.17.1 Lvalues and Rvalues

An *lvalue* refers to an object that has a name and persists. An *rvalue* is a temporary that exists only until the end of the expression.

```

1 int x = 5;           // x is an lvalue, 5 is an rvalue
2 int y = x + 2;       // (x + 2) is an rvalue
3 int z = std::move(x); // std::move(x) casts x to an rvalue

```

1.17.2 std::move

`std::move` does not move anything by itself. It casts its argument to an rvalue reference, enabling move constructors and move assignment operators to be called.

```

1 std::string a = "hello";
2 std::string b = std::move(a); // move constructor
3 // a is now in a valid but unspecified state
4 // b == "hello"

```

1.17.3 Move Constructor and Move Assignment

A move constructor transfers resources from a temporary object without copying:

```

1  class Buffer {
2      size_t size_;
3      int* data_;
4  public:
5      explicit Buffer(size_t n)
6          : size_(n), data_(new int[n]) {}
7
8      ~Buffer() { delete[] data_; }
9
10     // Move constructor
11     Buffer(Buffer&& other) noexcept
12         : size_(other.size_), data_(other.data_) {
13         other.size_ = 0;
14         other.data_ = nullptr;
15     }
16
17     // Move assignment
18     Buffer& operator=(Buffer&& other) noexcept {
19         if (this != &other) {
20             delete[] data_;
21             size_ = other.size_;
22             data_ = other.data_;
23             other.size_ = 0;
24             other.data_ = nullptr;
25         }
26         return *this;
27     }
28
29     // Disable copying
30     Buffer(const Buffer&) = delete;
31     Buffer& operator=(const Buffer&) = delete;
32 };

```

The source object (`other`) is left in a valid but empty state. This guarantees that the destructor of `other` runs safely.

1.17.4 Perfect Forwarding

Perfect forwarding allows a function template to forward its arguments to another function while preserving their value category (lvalue or rvalue) and `const/volatile` qualifiers.

```

1  template <typename T>
2  void wrapper(T&& arg) {
3      // arg is a forwarding reference
4      target(std::forward<T>(arg));

```

```

5 }
6
7 void target(int& x) { /* lvalue path */ }
8 void target(int&& x) { /* rvalue path */ }
9
10 int x = 5;
11 wrapper(x);           // calls target(int&)
12 wrapper(10);          // calls target(int&&)

```

`std::forward<T>` must be used inside templates with forwarding references (`T&&`). Without it, the argument would always be forwarded as an lvalue.

1.17.5 Move Semantics in Practice

Move semantics are critical in data structure implementations. When a `std::vector` grows, it moves its elements to the new buffer rather than copying them, provided the element type has a `noexcept` move constructor:

```

1 std::vector<Buffer> v;
2 v.push_back(Buffer(100));
3 // vector may reallocate: elements are moved, not copied
4 // This is fast because Buffer has a noexcept move constructor

```

1.17.6 constexpr and consteval

C++11 introduced `constexpr` to enable compile-time computation. A `constexpr` function can be evaluated at compile time when its arguments are compile-time constants, and at runtime otherwise:

```

1 constexpr int factorial(int n) {
2     return n <= 1 ? 1 : n * factorial(n - 1);
3 }
4
5 // Compile-time evaluation
6 static_assert(factorial(5) == 120);
7
8 // Runtime evaluation (n is not constexpr)
9 int x = 10;
10 int y = factorial(x); // computed at runtime

```

`constexpr` variables must be initialized with a constant expression:

```

1 constexpr int array_size = 10;
2 int arr[array_size]; // OK: compile-time size

```

C++20 adds `constexpr`, which forces compile-time evaluation. A `constexpr` function can never be called at runtime:

```

1 constexpr int square(int n) {

```

```

2     return n * n;
3 }
4
5 // int x = 5;
6 // int y = square(x); // ERROR: x is not constexpr
7 constexpr int y = square(5); // OK: compile-time

```

Use `constexpr` when the function has no meaningful runtime behavior (e.g., compile-time string processing, template argument computation). Use `constexpr` when you want both compile-time and runtime evaluation.

1.17.7 Structured Bindings (C++17)

Structured bindings decompose a tuple-like object into named variables:

```

1 #include <tuple>
2 #include <map>
3 #include <string>
4
5 // Pairs
6 auto [x, y] = std::pair{1, 2.0}; // x is int, y is double
7
8 // Tuples
9 auto [a, b, c] = std::tuple{1, "hello", 3.14};
10
11 // Structs
12 struct Point { double x, y; };
13 auto [px, py] = Point{1.0, 2.0};

```

They are especially useful with containers that return pairs:

```

1 std::map<std::string, int> m = {{ "a", 1 }, { "b", 2 }};
2 for (const auto& [key, value] : m) {
3     // key is const std::string&, value is const int&
4 }

```

Structured bindings work with `std::pair`, `std::tuple`, arrays, and structs where all members are public.

1.17.8 `std::optional` and `std::variant`

`std::optional` represents a value that may or may not exist. It replaces sentinel values like `-1` or `nullptr`:

```

1 #include <optional>
2 #include <string>
3
4 std::optional<int> safe_divide(int a, int b) {
5     if (b == 0) return std::nullopt;
6 }

```

```

6     return a / b;
7 }
8
9 auto result = safe_divide(10, 3);
10 if (result) {
11     // *result == 3
12 }
13
14 // Default value
15 int value = safe_divide(10, 0).value_or(-1); // value == -1

```

Use `std::optional` when a function may fail to produce a value but failure is not exceptional (lookup, parsing, validation).

`std::variant` is a type-safe union. It holds exactly one value from a fixed set of types:

```

1 #include <variant>
2 #include <string>
3 #include <iostream>
4
5 using Value = std::variant<int, double, std::string>;
6
7 Value v = 42;
8 v = "hello";
9
10 // std::holds_alternative<T>(v) checks the active type
11 // std::get<T>(v) extracts the value
12
13 std::visit([](auto& val) {
14     std::cout << val << '\n';
15 }, v);

```

Use `std::variant` when a variable can hold one of several unrelated types (e.g., AST nodes, JSON values, error-or-result types). Prefer it over `void*` unions or inheritance hierarchies for small closed sets of types.

1.18 Common Bugs and Pitfalls

Pitfall	Consequence	Solution
Using <code>std::move</code> on <code>const</code> objects	Move degrades to copy — object remains unmodified	Only <code>std::move</code> non-const objects
Returning references to local variables	Dangling reference, undefined behavior	Return by value, or ensure lifetime extends past caller
Forgetting <code>noexcept</code> on move operations	<code>std::vector</code> copies instead of moves during reallocation	Always mark move constructor/assignment <code>noexcept</code>
Storing <code>std::string_view</code> to temporary strings	View points to freed memory	Only use <code>string_view</code> when the source outlives the view
Circular <code>shared_ptr</code> references	Memory leak — ref count never reaches zero	Use <code>weak_ptr</code> to break cycles
Passing by <code>const T&</code> for small types	Unnecessary indirection, inhibits optimization	Pass small trivially-copyable types by value
Using <code>auto</code> with braced initializers	<code>auto x{1}</code> is <code>std::initializer_list<int></code> , not <code>int</code>	Use <code>auto x = 1</code> or be explicit

1.19 Summary

1. Use **RAII** to manage all resources. Prefer smart pointers over raw `new/delete`.
2. Use **move semantics** to avoid unnecessary copies.
3. Use **concepts** to constrain templates and produce clearer error messages.
4. Use `std::span` and `std::string_view` for non-owning views of contiguous data.
5. Follow the **Rule of Zero** — let automatic resource management do its job.
6. Mark move operations as `noexcept` to enable optimal container performance.

1.20 Exercises

1.20.1 Drill

1. **TCP connection pool.** A networked service maintains a pool of open connections to a database. Each connection is owned by the pool via `unique_ptr`. Why is `unique_ptr` the right choice here? What happens if you use `shared_ptr` instead and the pool destructor runs while a worker thread still holds a copy? Write a small demo.

(In production, libraries like Redis use connection pools exactly this way — `unique_ptr` ensures each connection has exactly one owner at a time.)

2. **Logging service.** A high-throughput logging system buffers log messages as `std::string` objects in a `std::vector`. Why does marking the string’s move constructor `noexcept` matter? Estimate how many extra copies happen during vector reallocation if moves fall back to copies.

(In production, Google’s Protobuf library relies on `noexcept` moves to maintain throughput in their RPC layer.)

3. **Cache key safety.** A web cache stores items in a hash map keyed by URL string. An intern changes the key type to `std::string_view` to “avoid copies.” What goes wrong when the owning URL string is removed from the cache?
4. **Function return bug.** A compiler optimization pass has a function that finds an instruction and returns a reference to it:

```
1  const Instruction& findInst(const std::vector<Instruction>& list, int
    id) {
2      for (const auto& inst : list)
3          if (inst.id() == id) return inst;
4      // Oops -- what do we return here?
5  }
```

Fix this function. What should the return type be? What happens in a real compiler when this bug exists?

5. **Circular references.** Two objects hold `shared_ptr` to each other. They are never destroyed, even after all external references are gone. Why? Demonstrate with code and fix using `weak_ptr`.

1.20.2 Application

1. **Query pipeline.** A database engine processes queries as a pipeline: scan reads rows, filter checks conditions, aggregate computes sums. Each stage owns the next via `unique_ptr`. Implement a class hierarchy with `ScanNode`, `FilterNode`, and `AggregateNode`. Each has a `next()` that returns the next row (use `std::optional`). Demonstrate correct destruction order when the pipeline is torn down.

(In production, DuckDB uses this exact pattern — `unique_ptr` chains of operator nodes.)

2. **Socket wrapper.** Write an RAII wrapper around a POSIX socket file descriptor. The class should:

- Close the fd on destruction
- Support move (transfer ownership)
- Be non-copyable
- Provide `send(span<const char>)` and `recv()` methods

Demonstrate moving a socket from an acceptor to a connection handler.

(In production, NGINX and similar servers use this pattern for connection lifetime management.)

3. **Log parser.** A profiler writes timestamped function call records to a binary log. Write a parser that reads a `vector<char>` from disk and uses `std::span` and `std::string_view` to extract fields without any heap allocation. Compare the allocation count with a version that uses `std::string` for each field.

(In production, Chrome's V8 profiler and many trading systems use zero-copy parsing to avoid GC pressure.)

4. **Constrained serialization.** Write a `serialize` function using C++20 concepts that works for any type. For trivially-copyable types, write raw bytes. For other types, call a `serialize(ostream&)` member function. Demonstrate that passing a non-trivially-copyable type without the member function produces a clear compile error.

5. **Ring buffer.** A network protocol processes messages from a stream. Implement a fixed-capacity `RingBuffer<T>` with RAII. Support `push`, `pop`, `front`, `back`, `size`, `empty`. When full, overwrite the oldest element (sliding window). Compare performance with `std::deque<T>`.

(In production, PostgreSQL uses a ring buffer for its wire protocol message framing.)

1.20.3 Research

1. **Custom allocator.** Implement a `PoolAllocator<T>` that allocates from a fixed arena. Use it with `std::vector` and measure allocation overhead vs the default allocator for 10^6 `push_back` operations. When does an arena allocator improve performance?
(In production, Google's TCMalloc and other allocators use thread-local caches to reduce contention.)
2. **Perfect forwarding.** Study how `std::forward` works. Implement a `make_box` factory that constructs any `Box<T>` with perfect forwarding. Show it works for both lvalue and rvalue arguments. Then extend to `make_unique_for_overwrite` (C++20).
3. **Small object optimization.** Implement a `SmallVector<T, N>` that stores up to `N` elements inline before heap-allocating. Use `std::aligned_storage` and placement new/delete. Benchmark `push_back` against `std::vector<T>` for various `T` sizes. Show the allocation-free regime.
4. **Coroutine generator.** Implement a `generator<T>` using C++20 coroutines that yields parsed tokens from a text input. Compare readability and generated assembly with an iterator-based version. Use Compiler Explorer to see whether the coroutine frame is heap-allocated.

1.20.4 Additional Exercises

1. **Container benchmark.** Implement a benchmark that inserts 10^6 random integers into a `vector`, `deque`, `list`, `set`, and `unordered_set`. Measure insertion time and memory usage. At what dataset size does `unordered_set` overtake `set`?
2. **Custom unique_ptr.** Implement your own `MyUniquePtr<T>` from scratch. It should support construction from a raw pointer, move construction, move assignment, `reset()`, `release()`, `get()`, and `operator->`. Verify with a small test that the pointed-to object is destroyed exactly once.
3. **Weak pointer cache.** Implement a `WeakCache<Key, Value>` that stores `weak_ptr` to recently created objects. When a key is queried and the weak pointer has expired, the cache re-creates the value and stores a new `weak_ptr`. Demonstrate that the cache does not prevent destruction of unused objects.
4. **Lambda state machine.** Use a mutable lambda with captured state to implement a simple finite automaton that recognizes strings ending in “ab”. The lambda should maintain the current state across invocations. Test it with `std::for_each` over a sequence of characters.
5. **Move-only container.** Implement a `MoveVector<T>` that stores elements using `unique_ptr<T[]>`. It should support move construction, move assignment, `push_back`, `size`, and element access. Demonstrate that it can be stored in a `std::vector`.

6. **Container selection guide.** Write a function that, given a `string` describing the access pattern (“sequential”, “random”, “sorted”, “hash”), returns a `string` recommending the best STL container. Implement it using `if constexpr` and `constexpr` so the return value is a compile-time constant.
7. **Forwarding factory.** Implement a `make_pair` template that uses perfect forwarding to construct a `std::pair` from any two arguments. Show it preserves `const`, `lvalue/rvalue`, and works for types with explicit constructors.
8. **Lambda algorithm library.** Rewrite the following STL algorithms using only lambdas and `std::accumulate` or manual loops: `std::transform`, `std::copy_if`, and `std::partition`. Test each on a `vector<int>`.
9. **Shared pointer ref-counting.** Implement a minimal `MySharedPtr<T>` with a separate control block that stores the reference count. Your implementation should support copy construction, copy assignment, destruction, and `use_count()`. Verify that the managed object is destroyed when the last shared pointer is destroyed.
10. **Move semantics timing.** Write a program that creates a vector of 10^6 `std::string` objects. Copy the vector into a second vector and measure the time. Then move-construct a third vector from the first and measure. Report the speedup ratio. Repeat with strings of length 10, 100, and 1000.

1.21 References and Further Reading

- Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition. Addison-Wesley, 2013.
- Bjarne Stroustrup, “C++11 — the new ISO C++ standard.” *The C++ Source*, 2011.
- Nicolai M. Josuttis, *C++17 — The Complete Guide*. Leanpub, 2019.
- Nicolai M. Josuttis, *C++20 — The Complete Guide*. Leanpub, 2022.
- Rainer Grimm, *C++20: Get the Details*. Leanpub, 2021.
- Herb Sutter, “Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions.” Addison-Wesley, 2000.
- Herb Sutter and Andrei Alexandrescu, “C++ Coding Standards: 101 Rules, Guidelines, and Best Practices.” Addison-Wesley, 2005.
- Scott Meyers, *Effective Modern C++*. O’Reilly, 2014.
- Scott Meyers, “Universal References in C++11.” — *Overload Journal*, 2012.

- David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition. Addison-Wesley, 2017.
- Andrei Alexandrescu, *Modern C++ Design*. Addison-Wesley, 2001.
- cppreference.com — The definitive online C++ reference.
- ISO C++ Standard Committee, "Working Draft, Standard for Programming Language C++" (N4901, 2021).

Performance Analysis

2.1 Introduction

Why do we need to analyze algorithms? Consider two programs that solve the same problem. One completes in milliseconds; the other takes hours. The difference is not in the hardware or the programming language — it is in the algorithm.

Performance analysis is the theoretical study of an algorithm's time and space requirements as a function of input size. Performance measurement (Chapter 3) is the empirical study of actual running time. Both are essential.

In this chapter we develop the mathematical tools needed to compare algorithms independently of the machine, compiler, or programming language.

❖ **Complete compilable implementations of the data structures in this chapter are in `code/`.**

2.2 What Is an Algorithm?

An **algorithm** is a finite sequence of unambiguous instructions that, given an input, produces an output and terminates in finite time.

Properties of an algorithm:

1. **Input** — zero or more externally supplied values
2. **Output** — at least one value produced
3. **Definiteness** — each instruction is precise and unambiguous
4. **Finiteness** — the algorithm terminates after a finite number of steps
5. **Effectiveness** — each instruction is basic enough to be performed exactly

A **program** is an algorithm expressed in a programming language. A program may not terminate (e.g., an operating system), but an algorithm always must.

2.3 Measuring Algorithm Efficiency

2.3.1 Time Complexity

The time taken by a program depends on:

- The input size
- The machine and compiler
- The quality of the generated code
- The algorithm itself

We need a way to talk about efficiency that abstracts away the machine. We count **operations** — comparisons, arithmetic operations, data movements — as a function of **input size**, denoted n .

2.3.2 Space Complexity

Space complexity measures the memory required:

- **Instruction space** — the compiled code
- **Data space** — constants, variables, dynamically allocated memory
- **Stack space** — function call overhead, local variables in recursive functions

2.4 Asymptotic Notation

Asymptotic notation describes the growth rate of functions. We care about behavior as $n \rightarrow \infty$.

2.4.1 Big O Notation (O)

Definition: $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Big O gives an **upper bound** on the growth rate.

Example: $f(n) = 3n^2 + 2n + 1$ is $O(n^2)$ because $3n^2 + 2n + 1 \leq 4n^2$ for all $n \geq 3$.

Common Big O classes, from slowest to fastest growth:

Notation	Name	Example
$O(1)$	Constant	Array access
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Finding maximum
$O(n \log n)$	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Bubble sort
$O(n^3)$	Cubic	Matrix multiplication
$O(2^n)$	Exponential	Subset generation
$O(n!)$	Factorial	Permutations

2.4.2 Big Omega Notation (Ω)

Definition: $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

Big Omega gives a **lower bound** — the function grows at least as fast as $g(n)$.

Example 1: $f(n) = 3n^2 + 2n + 1$ is $\Omega(n^2)$ because $3n^2 + 2n + 1 \geq 3n^2$ for all $n \geq 0$.

Example 2: $f(n) = 7n + 5$ is $\Omega(n)$ because $7n + 5 \geq 7n$ for all $n \geq 0$.

Example 3 (why Omega matters): Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. This means no matter how clever our algorithm is, we cannot sort faster than $n \log n$ using comparisons. Merge sort achieves this lower bound — it is **asymptotically optimal**.

Common mistake: saying “ $f(n) = O(n^2)$ means $f(n)$ grows like n^2 .” No — O gives an UPPER bound. $f(n) = n$ also satisfies $f(n) = O(n^2)$. To say f grows exactly like n^2 , use $\Theta(n^2)$.

2.4.3 Big Theta Notation (Θ)

Definition: $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Big Theta gives a **tight bound** — the function grows at exactly the same rate as $g(n)$.

Example 1: $f(n) = 3n^2 + 2n + 1$ is $\Theta(n^2)$ because it is both $O(n^2)$ and $\Omega(n^2)$.

Example 2: $f(n) = n \log n + 3n$ is $\Theta(n \log n)$.

When to use Θ vs O : Use Θ when the bound is tight. “Merge sort is $\Theta(n \log n)$ ” is more precise than “merge sort is $O(n \log n)$ ” — the latter allows $O(n^2)$ which is not tight. In this book, we use Θ when possible and O when only an upper bound is needed.

Proving Θ : show both $O(g(n))$ and $\Omega(g(n))$:

- For O : find constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$
- For Ω : find constants c', n'_0 such that $f(n) \geq c' \cdot g(n)$ for $n \geq n'_0$

Example proof: $f(n) = 2n + 3$ is $\Theta(n)$.

Upper bound: $2n + 3 \leq 5n$ for $n \geq 1$. So $f(n) = O(n)$. Lower bound: $2n + 3 \geq 2n$ for $n \geq 0$. So $f(n) = \Omega(n)$. Therefore $f(n) = \Theta(n)$.

2.4.4 Little o and Little ω

Little o: $f(n) = o(g(n))$ if for every positive constant c , there exists n_0 such that $f(n) < c \cdot g(n)$ for all $n \geq n_0$. (Strict upper bound — f grows strictly slower than g .)

Little ω : $f(n) = \omega(g(n))$ if for every positive constant c , there exists n_0 such that $f(n) > c \cdot g(n)$ for all $n \geq n_0$. (Strict lower bound — f grows strictly faster than g .)

Contrast with Big-O: Big-O says $f(n) \leq c \cdot g(n)$ for SOME c . Little-o says $f(n) < c \cdot g(n)$ for ALL c . Little-o is strictly stronger.

Examples:

- $n = o(n^2)$ because $n/n^2 = 1/n \rightarrow 0$ as $n \rightarrow \infty$.

- $n^2 = o(n^3)$ because $n^2/n^3 = 1/n \rightarrow 0$.
- $n = o(n \log n)$ because $n/(n \log n) = 1/\log n \rightarrow 0$.
- $n \log n = \omega(n)$ because $n \log n/n = \log n \rightarrow \infty$.
- $2^n = \omega(n^k)$ for any k — exponentials dominate polynomials.

When little-o is useful: it provides a strict separation of growth rates. “Merge sort is $o(n^2)$ ” means merge sort grows strictly slower than quadratic — a stronger statement than “merge sort is $O(n^2)$.”

2.5 Practical Complexity

2.5.1 Growth Rates Compared

Consider algorithms with different time complexities on a machine executing 10^8 operations per second:

n	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	0.00 us	0.10 us	0.33 us	1.00 us	10.2 us
100	0.01 us	1.00 us	6.64 us	100 us	4×10^{16} years
1000	0.01 us	10.0 us	99.7 us	10.0 ms	—
10^6	0.02 us	10.0 ms	199 ms	2.78 hours	—
10^9	0.03 us	10.0 s	299 s	317 years	—

2.5.2 Which Complexity Matters?

An $O(n^2)$ algorithm that takes 1 second for $n = 1000$ will take 100 seconds for $n = 10000$. An $O(n \log n)$ algorithm that takes 1 second for $n = 1000$ will take about 10 seconds for $n = 10000$ — a linear degradation, not quadratic.

Key insight: The asymptotic growth rate dominates all constant factors for sufficiently large n .

2.6 Analyzing Iterative Algorithms

2.6.1 Simple Loop

```

1 // Find the maximum element
2 int find_max(const std::vector<int>& a) {
3     int max = a[0]; // 1
4     for (size_t i = 1; i < a.size(); ++i) { // n-1 iterations
5         if (a[i] > max) max = a[i]; // 2 ops per iteration
6     }
7     return max;
8 }
```

Operations: $1 + 2(n-1) = 2n - 1 = O(n)$

2.6.2 Nested Loops

```

1 // All pairs
2 for (int i = 0; i < n; ++i) {
3     for (int j = 0; j < n; ++j) {
4         process(a[i], a[j]); // O(1) operation
5     }
6 }
```

Inner loop executes n times for each of n outer iterations: $O(n^2)$.

2.6.3 Dependent Nested Loops

```

1 for (int i = 0; i < n; ++i) {
2     for (int j = 0; j < i; ++j) {
3         process(a[i], a[j]);
4     }
5 }
```

Total iterations: $0 + 1 + 2 + \dots + (n-1) = n(n-1)/2 = O(n^2)$.

2.6.4 Logarithmic Loops

```

1 for (int i = 1; i < n; i *= 2) {
2     process(i);
3 }
```

i takes values $1, 2, 4, 8, \dots, 2^k$ where $2^k \leq n$. So $k = \lfloor \log_2 n \rfloor$ iterations: $O(\log n)$.

2.7 Analyzing Recursive Algorithms

2.7.1 Recurrence Relations

Recursive algorithms lead to recurrence relations. The recurrence describes the running time in terms of the running time on smaller inputs.

Example: Factorial

```

1 int factorial(int n) {
2     if (n <= 1) return 1; // O(1)
3     return n * factorial(n - 1); // T(n-1) + O(1)
4 }
```

Recurrence: $T(n) = T(n-1) + c, T(1) = d$.

Solution: $T(n) = c(n-1) + d = O(n)$.

Example: Binary Search

```

1 int binary_search(const std::vector<int>& a, int target, int low, int high
  ) {
2     if (low > high) return -1;
3     int mid = low + (high - low) / 2;
4     if (a[mid] == target) return mid;
5     if (a[mid] > target) return binary_search(a, target, low, mid - 1);
6     return binary_search(a, target, mid + 1, high);
7 }

```

Recurrence: $T(n) = T(n/2) + c$, $T(1) = d$.

Solution: $T(n) = c \cdot \log_2 n + d = O(\log n)$.

2.7.2 Solving Recurrences**Method 1: Substitution**

Guess the form of the solution and prove by induction.

For $T(n) = 2T(n/2) + n$, $T(1) = 1$:

- Guess $T(n) = O(n \log n)$
- Assume $T(k) \leq ck \log k$ for $k \leq n$
- Then $T(n) \leq 2c(n/2)\log(n/2) + n = cn \log(n/2) + n = cn \log n - cn \log 2 + n = cn \log n - cn + n$
- Choose $c \geq 1$: $T(n) \leq cn \log n$ ✓

Method 2: Recursion Tree

Draw the tree of recursive calls, sum the cost at each level.

For $T(n) = 2T(n/2) + n$:

```

1           n                               <- level 0, 1 node
2         /  \
3       n/2  n/2                           <- level 1, 2 nodes
4     /  \  /  \
5   n/4 n/4 n/4 n/4                       <- level 2, 4 nodes
6   ...  ...  ...  ...

```

Level i has 2^i nodes, each costing $n/2^i$. Total per level: n . Height: $\log_2 n$ levels. Total: $n \cdot \log_2 n$.

Method 3: Master Theorem

For recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$, $b \geq 1$:

Case	Condition	Solution
1	$f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$	$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$ for some $c < 1$	$T(n) = \Theta(f(n))$

Examples:

- $T(n) = 2T(n/2) + n$: $a=2$, $b=2$, $\log_b a = 1$, $f(n)=n = \Theta(n^1) \rightarrow \text{Case 2} \rightarrow T(n) = \Theta(n \log n)$
- $T(n) = T(n/2) + 1$: $a=1$, $b=2$, $\log_b a = 0$, $f(n)=1 = \Theta(n^0) \rightarrow \text{Case 2} \rightarrow T(n) = \Theta(\log n)$
- $T(n) = 9T(n/3) + n$: $a=9$, $b=3$, $\log_b a = 2$, $f(n)=n = O(n^{2-\epsilon}) \rightarrow \text{Case 1} \rightarrow T(n) = \Theta(n^2)$

2.8 Amortized Analysis

Sometimes a single operation is expensive, but the average cost over a sequence of operations is low. **Amortized analysis** bounds the average cost per operation.

2.8.1 Aggregate Method

We compute the total cost of n operations and divide by n .

Example: Dynamic array (`std::vector push_back`)

When the array is full, we double its capacity and copy all elements. A `push_back` that triggers a resize costs $O(k)$ where k is the current size. But most `push_back` operations cost $O(1)$.

Total cost for n insertions into an initially empty array that doubles at capacity:

- Capacities: 1, 2, 4, 8, ..., $2^{\lceil \log_2 n \rceil}$
- Copy cost: $1 + 2 + 4 + \dots + 2^{\lceil \log_2 n \rceil} \leq 2n$
- n insertions cost at most $3n$ (n inserts at unit cost + $2n$ for copies)
- Amortized cost per insertion: $O(1)$

2.8.2 Accounting Method

We charge more for cheap operations and use the surplus to pay for expensive ones.

For the dynamic array: charge 3 for each `push_back`. Each regular insertion costs 1; the remaining 2 are saved. When a resize copies k elements, the saved credits from the k elements pay for it.

2.8.3 Potential Method

Define a potential function $\Phi(\text{state})$ that measures how close the data structure is to needing an expensive operation. The amortized cost is actual cost + $\Delta\Phi$.

For the dynamic array: let $\Phi = 2 \cdot \text{size} - \text{capacity}$ when $\text{size} \geq \text{capacity}/2$, and 0 otherwise.

Worked example: binary counter increment.

A binary counter starts at 0 and increments n times. Each increment flips some bits ($0 \rightarrow 1$ and $1 \rightarrow 0 \rightarrow 1$ chain). The worst case flips all bits: $O(\log n)$. But most increments flip only 1 bit.

```

1 Increment: 0000 -> 0001 -> 0010 -> 0011 -> 0100 -> 0101 -> 0110 -> 0111 ->
           1000
2 Flips:      1      2      1      3      1      2      1      4

```

Aggregate: total flips = $n/2$ (bit 0) + $n/4$ (bit 1) + $n/8$ (bit 2) + ... ; n . Amortized = $O(1)$.

Potential method: let Φ = number of 1-bits in the counter. Each increment flips k trailing 1s to 0 and one 0 to 1. Actual cost = $k + 1$. $\Delta\Phi = -(k) + 1 = 1 - k$. Amortized cost = $(k + 1) + (1 - k) = 2 = O(1)$.

2.9 Space Complexity

2.9.1 Fixed vs. Variable Space

- **Fixed space:** constants, program code — independent of input size
- **Variable space:** depends on input size — arrays, dynamic allocations, recursion stack

Total space = fixed + variable.

2.9.2 Analyzing Space

```

1 // O(1) extra space
2 int sum(const std::vector<int>& a) {
3     int total = 0;
4     for (int x : a) total += x;
5     return total;
6 }
7
8 // O(n) extra space
9 std::vector<int> duplicate(const std::vector<int>& a) {
10     std::vector<int> result(a.size());
11     for (size_t i = 0; i < a.size(); ++i) result[i] = a[i];
12     return result;
13 }
14
15 // O(n) stack space (recursive)
16 int factorial(int n) {
17     if (n <= 1) return 1;
18     return n * factorial(n - 1); // n stack frames
19 }

```

2.10 Best, Average, and Worst Case

- **Best case** — minimum time over all inputs of size n

- **Worst case** — maximum time over all inputs of size n
- **Average case** — expected time over all inputs of size n (requires a probability distribution)

Example: Linear search in an array of size n

- Best case: element is first $\rightarrow O(1)$
- Worst case: element is last or absent $\rightarrow O(n)$
- Average case (assuming uniform distribution, element present): $(n+1)/2 \rightarrow O(n)$

We primarily study worst-case complexity in this book. It provides a guarantee, not a hope.

2.11 Smoothed Analysis

Worst-case analysis can be overly pessimistic. Average-case analysis requires knowing the input distribution, which is often unavailable. **Smoothed analysis** (Spielman and Teng, 2001) bridges this gap: it measures the expected performance when inputs are first slightly perturbed randomly, then fed to the algorithm.

2.11.1 The Idea

For an algorithm with input x , define:

- **Worst case:** $\max_x \text{cost}(x)$
- **Average case:** $\mathbb{E}_x[\text{cost}(x)]$ (over some distribution)
- **Smoothed cost:** $\mathbb{E}_{\bar{x}}[\text{cost}(\bar{x})]$ where $\bar{x} = x + \text{small random noise}$

The smoothed complexity of an algorithm is the maximum over all inputs x of the expected running time when x is perturbed by a small Gaussian (or uniform) perturbation of magnitude σ .

2.11.2 The Simplex Example

The simplex algorithm for linear programming has worst-case exponential time (Klee-Minty examples), yet it runs in near-linear time in practice. Smoothed analysis explains why:

- **Worst case:** 2^n (Klee-Minty cube, constructed inputs)
- **Smoothed case:** polynomial in n and $\log(1/\sigma)$ (Spielman-Teng, 2004)

Intuition: the exponentially many degenerate vertices that cause worst-case behavior are *unstable* — any small perturbation eliminates them. The probability of hitting a degenerate path under random perturbation is vanishingly small.

2.11.3 Formal Definition

Given input $x \in \mathbb{R}^n$ and perturbation size $\sigma > 0$:

$$S_\sigma(A, x) = \mathbb{E}_{\bar{x} \sim \text{Normal}(x, \sigma^2 I)}[\text{time}(A, \bar{x})]$$

The **smoothed complexity** is:

$$S_\sigma(A) = \max_x S_\sigma(A, x)$$

Compare with:

- Worst case: $\max_x \text{time}(A, x)$
- Average case: $\mathbb{E}_x[\text{time}(A, x)]$

Smoothed complexity is always $\geq$ average case (worst-case input still chosen adversarially) and $\leq$ worst case (perturbation helps).

2.11.4 Why It Matters

- **Explains practical performance:** algorithms with bad worst-case but good smoothed complexity perform well on real data (which is never exactly adversarial)
- **Robustness:** if an algorithm has good smoothed complexity, it works well even when inputs are slightly noisy — a realistic assumption
- **Bridges theory and practice:** closes the gap between worst-case lower bounds and observed runtime

2.11.5 Known Results

- **Simplex method:** smoothed complexity is polynomial (Spielman-Teng, 2004)
- **Gradient descent:** smoothed complexity $O(n^3 \sigma^{-2})$ for general convex functions
- **Local search for k-SAT:** smoothed complexity polynomial for bounded perturbation
- **Metroid greedy algorithm:** smoothed complexity polynomial even when worst-case is exponential

2.12 Recursion Deep Dive

Recursion is a powerful technique where a function calls itself on a smaller subproblem. Mastering recursion requires understanding the recursive thought process, visualizing the call structure, and knowing when recursion is the right tool.

2.12.1 The Recursive Thought Process

To solve a problem recursively, identify two things:

1. **Base case:** the smallest input for which the answer is known directly.
2. **Recursive case:** decompose the problem into smaller subproblems, combine their results.

The key insight: *assume the recursive call works* on the smaller input, and focus on how to combine its result with the current step. This is sometimes called the “leap of faith.”

2.12.2 Recursion Tree Visualization

A **recursion tree** shows every recursive call as a node, with children representing the subproblems it spawns. The total work is the sum of all nodes’ costs.

Example: Fibonacci

```
1 int fib(int n) {
2     if (n <= 1) return n;           // base case: 1 operation
3     return fib(n - 1) + fib(n - 2); // two recursive calls
4 }
```

For `fib(5)`, the recursion tree is:

```
1                fib(5)
2              /      \
3            fib(4)    fib(3)
4          /  \      /  \
5        fib(3) fib(2) fib(2) fib(1)
6       /  \  /  \  /  \
7     fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
8    /  \
9  fib(1) fib(0)
```

Count the nodes: `fib(5)` spawns 15 total calls. In general, `fib(n)` makes $O(2^n)$ calls — exponential. This is because subproblems overlap: `fib(3)` is computed twice, `fib(2)` three times, and so on. This redundancy is what makes naive Fibonacci so slow.

2.12.3 Tail Recursion vs Non-Tail Recursion

A function is **tail-recursive** if the recursive call is the *last* operation — nothing remains to do after it returns.

Non-tail recursive:

```
1 int factorial(int n) {
2     if (n <= 1) return 1;
3     return n * factorial(n - 1); // multiply AFTER the call returns
4 }
```


After `factorial(n-1)` returns, we still multiply by `n`. The compiler cannot reuse the current stack frame.

Tail recursive:

```
1 int factorial_helper(int n, int acc) {
2     if (n <= 1) return acc;
3     return factorial_helper(n - 1, n * acc); // last operation is the
        call
4 }
```

Here `factorial_helper(n-1, n*acc)` is the very last thing that happens. The compiler can optimize this into a loop, reusing one stack frame instead of allocating a new one. This is called **tail call optimization** (TCO).

2.12.4 Converting Tail Recursion to Iteration

Any tail-recursive function can be mechanically converted to an iterative loop:

```
1 int factorial(int n) {
2     int acc = 1;
3     while (n > 1) {
4         acc = n * acc;
5         --n;
6     }
7     return acc;
8 }
```

The accumulator `acc` replaces the function parameter. The loop body is identical to the recursive step. This conversion eliminates all recursion overhead: no stack frames, no risk of stack overflow, and often better cache performance.

2.12.5 Worked Example: Fibonacci (Optimized)

The naive recursion for Fibonacci is $O(2^n)$. We can fix this in two ways:

Bottom-up iteration:

```
1 int fib(int n) {
2     if (n <= 1) return n;
3     int prev2 = 0, prev1 = 1;
4     for (int i = 2; i <= n; ++i) {
5         int curr = prev1 + prev2;
6         prev2 = prev1;
7         prev1 = curr;
8     }
9     return prev1;
10 }
```

Time: $O(n)$. Space: $O(1)$.

Tail-recursive version:

```

1 int fib_tail(int n, int a, int b) {
2     if (n == 0) return a;
3     if (n == 1) return b;
4     return fib_tail(n - 1, b, a + b);
5 }

```

Called as `fib_tail(n, 0, 1)`. Time: $O(n)$. Space: $O(1)$ with TCO, $O(n)$ without.

2.12.6 Worked Example: Tower of Hanoi

The Tower of Hanoi puzzle has three pegs and n disks of increasing size. Move all disks from peg A to peg C, one disk at a time, never placing a larger disk on a smaller one.

```

1 void hanoi(int n, char from, char to, char aux) {
2     if (n == 0) return;
3     hanoi(n - 1, from, aux, to);           // step 1: move n-1 to aux
4     std::cout << "Move disk " << n
5         << " from " << from << " to " << to << "\n";
6     hanoi(n - 1, aux, to, from);          // step 3: move n-1 to target
7 }

```

Trace for $n = 3$:

```

1 hanoi(3, A, C, B):
2   hanoi(2, A, B, C):
3     hanoi(1, A, C, B):
4       move disk 1: A -> C
5     move disk 2: A -> B
6     hanoi(1, C, B, A):
7       move disk 1: C -> B
8   move disk 3: A -> C
9   hanoi(2, B, C, A):
10    hanoi(1, B, A, C):
11      move disk 1: B -> A
12    move disk 2: B -> C
13    hanoi(1, A, C, B):
14      move disk 1: A -> C

```

Recurrence: $T(n) = 2T(n-1) + 1$, $T(0) = 0$.

Solution: $T(n) = 2^n - 1 = O(2^n)$. The puzzle requires an exponential number of moves — there is no faster solution.

2.12.7 Worked Example: Power Function

Compute x^n efficiently using repeated squaring.

Naive recursion:

```

1 double power(double x, int n) {
2     if (n == 0) return 1.0;
3     return x * power(x, n - 1);    // O(n) multiplications
4 }

```

Exponentiation by squaring:

```

1 double power(double x, int n) {
2     if (n == 0) return 1.0;
3     double half = power(x, n / 2);
4     if (n % 2 == 0) return half * half;
5     return x * half * half;
6 }

```

Recurrence: $T(n) = T(n/2) + O(1)$. By the Master Theorem (Case 2: $a=1$, $b=2$, $f(n)=O(1)$): $T(n) = O(\log n)$.

We reduced n multiplications to $\log n$ by observing that $x^n = (x^{n/2})^2$ when n is even, and $x^n = x \cdot (x^{(n-1)/2})^2$ when n is odd.

2.13 Loop Invariants

A **loop invariant** is a property that holds before and after every iteration of a loop. Proving an algorithm correct with a loop invariant is a powerful technique: it reduces reasoning about potentially unbounded iterations to three simple proof steps.

2.13.1 The Three Parts

To prove a loop implements its specification, state a loop invariant and prove three properties:

1. **Initialization:** the invariant is true before the first iteration.
2. **Maintenance:** if the invariant is true before an iteration, it is true after the iteration.
3. **Termination:** when the loop terminates, the invariant combined with the termination condition implies correctness.

This structure mirrors mathematical induction: initialization is the base case, maintenance is the inductive step, and termination provides the final conclusion.

2.13.2 Worked Example: Insertion Sort Correctness

```

1 void insertion_sort(std::vector<int>& a) {
2     for (size_t i = 1; i < a.size(); ++i) {
3         int key = a[i];
4         size_t j = i;
5         while (j > 0 && a[j - 1] > key) {

```

```

6         a[j] = a[j - 1];
7         --j;
8     }
9     a[j] = key;
10    }
11 }

```

Loop invariant for the outer loop: At the start of each iteration of the outer `for` loop, the subarray $a[0..i-1]$ consists of the elements originally in $a[0..i-1]$, but sorted.

Initialization: Before the first iteration ($i = 1$), $a[0..0]$ contains one element, which is trivially sorted.

Maintenance: Assume $a[0..i-1]$ is sorted. The inner `while` loop shifts elements right until the correct position for $\text{key} = a[i]$ is found. After insertion, $a[0..i]$ is sorted. Thus the invariant holds for the next iteration with i replaced by $i+1$.

Termination: The loop terminates when $i = a.size()$. By the invariant, $a[0..a.size()-1]$ is sorted — the entire array is sorted. $\square$

2.13.3 Worked Example: Binary Search Correctness

```

1 int binary_search(const std::vector<int>& a, int target, int low, int high
  ) {
2     while (low <= high) {
3         int mid = low + (high - low) / 2;
4         if (a[mid] == target) return mid;
5         if (a[mid] < target) low = mid + 1;
6         else high = mid - 1;
7     }
8     return -1;
9 }

```

Assume a is sorted in non-decreasing order.

Loop invariant: If target exists in a , it is in $a[\text{low}..\text{high}]$.

Initialization: Before the first iteration, $\text{low} = 0$ and $\text{high} = a.size() - 1$, covering the entire array. If target exists, it is in this range.

Maintenance: At each step, compare $a[\text{mid}]$ with target :

- If $a[\text{mid}] == \text{target}$, we return mid — correct.
- If $a[\text{mid}] < \text{target}$, all elements $a[\text{low}..\text{mid}]$ are $\leq \text{target}$. Since $a[\text{mid}] \neq \text{target}$, target (if present) must be in $a[\text{mid}+1..\text{high}]$. Setting $\text{low} = \text{mid} + 1$ preserves the invariant.
- If $a[\text{mid}] > \text{target}$, all elements $a[\text{mid}..\text{high}]$ are $\geq \text{target}$. Target (if present) must be in $a[\text{low}..\text{mid}-1]$. Setting $\text{high} = \text{mid} - 1$ preserves the invariant.

Termination: The loop terminates when $low \geq high$, meaning $a[low..high]$ is empty. By the invariant, if target existed, it would be in this range — but the range is empty. Therefore target is not in a , and returning -1 is correct. $\square$

2.14 Space Complexity Analysis (Deep Dive)

Section 2 introduced the components of space complexity. Here we analyze space in depth, focusing on how recursion and auxiliary data structures affect memory usage.

2.14.1 Stack Space for Recursion

Each recursive call allocates a **stack frame** containing:

- Local variables
- Function parameters
- Return address
- Saved registers (implementation-dependent)

The total stack space is (number of simultaneous frames) $\times$ (size per frame).

Example: Factorial — depth n , so $O(n)$ stack space.

Example: Binary search — depth $\log n$, so $O(\log n)$ stack space.

Example: Merge sort — the deepest path in the recursion tree has $\log n$ frames, but at each level, `merge()` allocates $O(n)$ temporary space. The total space is $O(n + \log n) = O(n)$ auxiliary.

2.14.2 Auxiliary Space vs Input Space

- **Input space:** memory occupied by the input itself (e.g., the array of n integers).
- **Auxiliary space:** extra memory used by the algorithm, not counting the input.

Example: A function that takes a vector of n integers and returns their sum uses $O(n)$ input space and $O(1)$ auxiliary space.

```

1 // O(n) input space, O(1) auxiliary space
2 int sum(const std::vector<int>& a) {
3     int total = 0;
4     for (int x : a) total += x;
5     return total;
6 }
7
8 // O(n) input space, O(n) auxiliary space
9 std::vector<int> reverse_copy(const std::vector<int>& a) {
10     std::vector<int> result(a.rbegin(), a.rend());

```

```
11     return result;
12 }
13
14 // O(n) input space, O(n) auxiliary space (recursion stack)
15 void print_reverse(const std::vector<int>& a, int i) {
16     if (i < 0) return;
17     std::cout << a[i] << " ";
18     print_reverse(a, i - 1);
19 }
```

When we say “merge sort uses $O(n)$ space,” we mean $O(n)$ *auxiliary* space — the $O(n)$ input space is always required.

2.14.3 Space-Time Tradeoffs

Sometimes using more space reduces time, and vice versa:

Algorithm	Time	Auxiliary Space
Naive Fibonacci	$O(2^n)$	$O(n)$ stack
Memoized Fibonacci	$O(n)$	$O(n)$ table
Iterative Fibonacci	$O(n)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n)$
In-place merge sort	$O(n \log^2 n)$	$O(1)$
Hash table lookup	$O(1)$ amortized	$O(n)$ table
Sorted array binary search	$O(\log n)$	$O(1)$

The choice between algorithms often depends on which resource is the bottleneck: time or memory.

2.15 Common Bugs and Pitfalls

Pitfall	Consequence	
Confusing worst-case with average-case	Over-engineering for rare inputs	
Ignoring constant factors	$O(n^2)$ algorithm beats $O(n \log n)$ for $n \leq 1000$	
Misapplying the Master Theorem	Wrong asymptotic bound	Verify n
Forgetting the recursion stack in space analysis	Underestimating memory by $O(n)$	C
Assuming <code>std::vector::push_back</code> is always $O(1)$	Surprising latency spikes on reallocation	
Confusing Big- O (upper bound) with Big- Θ (tight bound)	Imprecise claims about algorithm behavior	Use Θ
Analyzing worst-case when amortized matters	Pessimistic design decisions	Use

2.16 Summary

1. **Asymptotic notation** (O , Ω , Θ , o , ω) describes growth rates independently of machine constants.

2. **Iterative algorithms** are analyzed by summing loop iterations.
3. **Recursive algorithms** lead to recurrences, solved by substitution, recursion trees, or the Master Theorem.
4. **Amortized analysis** measures the average cost of operations over a sequence.
5. **Space complexity** includes fixed and variable components, plus the recursion stack.
6. **Worst-case analysis** provides performance guarantees.
7. **Smoothed analysis** explains practical performance by averaging over slightly perturbed inputs — bridging worst-case pessimism and average-case assumptions.

2.17 Exercises

2.17.1 Drill

1. **Index vs table scan.** A database can search an index in $O(\log n)$ reads or scan the whole table in $O(n)$ reads. For a table of 10^6 rows, the index should win. But the index = 800 ms and the scan = 12 ms. What does Big-O notation ignore that explains this? (Hint: think about sequential vs random I/O.)

(In production, databases like PostgreSQL use cost-based query planning that weighs disk access patterns, not just operation counts.)

1. **Blocking vs non-blocking.** A database operation called KEYS has $O(n)$ cost and blocks all other operations while running. Another operation called SCAN has $O(1)$ per call and $O(n)$ total, but does not block others. A colleague says "they're both $O(n)$, they're the same." Why is this reasoning wrong?
1. **Hash table growing pains.** An engineer replaces a balanced tree ($O(\log n)$ lookup) with a hash table ($O(1)$ average lookup), expecting faster code. Throughput drops. What hidden cost of the hash table does Big-O notation hide? Think about rehashing, memory allocation, and cache misses.

(In production, Google's `absl::flat_hash_map` and similar libraries optimize the growth strategy to minimize this overhead.)

1. **Compound index.** A database index on `(city, last_seen)` should answer "find all rows for Seattle in the last hour" in $O(\log n)$ time. The actual query takes much longer. If "Seattle" matches 10,000 rows and "last hour" spans only a few of them, why does the index still scan many pages?

1. **Stack overflow.** A serverless function uses a recursive algorithm whose depth equals the input size. Code review says "each call is $O(1)$, so total is $O(n)$." Yet the function crashes on large inputs. What aspect of complexity analysis was overlooked?
1. **Recursion trace.** Trace the recursion tree for `fib(6)` using the naive recursive implementation. How many total calls are made? How many are duplicate computations? If memoization caches results, how many unique subproblems exist?
1. **Tail recursion conversion.** Convert the following recursive function to an iterative version using an accumulator:

```

1 int sum_list(Node* head) {
2     if (!head) return 0;
3     return head->data + sum_list(head->next);
4 }

```

What is the time and space complexity of each version?

1. **Power function trace.** Trace the execution of `power(2, 13)` using the exponentiation-by-squaring algorithm. Show each recursive call, the value of `half`, and the final computation at each level. How many multiplications are performed?

2.17.2 Application

1. **Write buffer amortization.** An in-memory write buffer accepts inserts (each $O(\log m)$ where m is buffer size). When the buffer reaches capacity, it's flushed to disk (cost $O(m)$). Show that the amortized cost per insert is $O(\log n)$ despite the occasional $O(m)$ flush. Implement a simulation that counts operations over 10^6 inserts.

(In production, LevelDB and RocksDB use this exact "memtable" pattern.)

1. **Vector growth cost.** A dynamic array (like `std::vector`) doubles its capacity each time it runs out of space. Analyze the total number of element copies over n insertions. Show that the average (amortized) cost is $O(1)$ per insertion despite occasional $O(n)$ reallocations. Implement a simulation that counts copies for $n = 10^6$ and verify the bound. What happens with growth factor 1.5 vs 2?

(In production, Chromium's rendering engine carefully manages vector growth to minimize page load jank.)

1. **Merging sorted segments.** A background process merges m sorted segments containing n total items. The standard merge has recurrence $T(n) = 2T(n/2) + O(n)$. Solve this using the Master Theorem. Implement a simulator and verify the total operations. How does the complexity change if the merge step is $O(n^2)$?

(In production, Kafka’s log compaction uses this pattern to merge sorted message segments.)

1. **Shortest path in a road network.** Dijkstra’s algorithm with a binary heap has complexity $O((V+E) \log V)$. For a road network with $V = 10^6$ intersections and $E = 2.5 \times 10^6$ roads, estimate the heap operations. Implement Dijkstra with a binary heap and with a simpler array-based approach for a dense subgraph. Compare actual operation counts.

(In production, Google Maps uses optimized variants of Dijkstra with heuristics like A and contraction hierarchies.)*

1. **Batched allocation.** A server allocates resources in batches — when the pool is empty, it creates 10 new ones at once instead of 1 at a time. Using amortized analysis (the accounting method), assign a “token cost” to each allocation. Show that the average cost per resource is $O(1)$ despite occasional bursts. Implement a simulation.

(In production, NGINX and other web servers use batched connection allocation to smooth out latency spikes.)

1. **Loop invariant for selection sort.** State a loop invariant for the outer loop of selection sort:

```
1 void selection_sort(std::vector<int>& a) {
2     for (size_t i = 0; i + 1 < a.size(); ++i) {
3         size_t min_idx = i;
4         for (size_t j = i + 1; j < a.size(); ++j)
5             if (a[j] < a[min_idx]) min_idx = j;
6         std::swap(a[i], a[min_idx]);
7     }
8 }
```

Prove correctness by establishing initialization, maintenance, and termination.

1. **Recursion vs iteration tradeoffs.** Implement matrix exponentiation (computing A^n for a 2×2 matrix A) using: (a) a naive loop ($O(n)$ multiplications), (b) exponentiation by squaring ($O(\log n)$ multiplications). Measure both for $n = 10^8$ and compare. What is the auxiliary space of each?
1. **Space complexity of algorithms.** Analyze the auxiliary space complexity of the following: (a) reversing an array in-place, (b) computing all n choose 2 pair sums into a new vector, (c) checking if a binary tree is balanced recursively. For (c), distinguish between the best-case tree shape and worst-case tree shape.
1. **Tower of Hanoi generalization.** The standard Tower of Hanoi uses 3 pegs. With 4 pegs (Reve’s puzzle), the Frame-Stewart algorithm gives an empirically better but unproven optimal solution. Implement the 3-peg solution and count moves for $n = 1, 2, 4, 8, 16$. Verify the $2^n - 1$ formula. Then implement a simple 4-peg variant and compare move counts.

2.17.3 Research

1. **Cache complexity.** Standard Big-O analysis ignores the memory hierarchy. Analyze a blocked (tiled) matrix multiplication: count how many cache misses occur for $n \times n$ matrices with a cache of size Z . Show the complexity is $O(n^3 / \sqrt{Z})$ cache misses. Implement and measure L1/L2/L3 misses using `perf stat` for $n = 512, 1024, 2048$.
1. **Real-time guarantees.** A real-time control system (e.g., a robot) cannot tolerate occasional $O(n)$ reallocation spikes from `std::vector::push_back`. Design a data structure that guarantees $O(1)$ per operation at the cost of higher average memory. Implement it and measure worst-case vs `std::vector`. Compare P99 and P99.9 latency.
1. **External sorting.** When data exceeds available RAM, sorting must happen on disk. An external sort creates sorted runs of size M (memory limit) and then merges them. Analyze the I/O complexity: how many disk blocks are read and written? Show that increasing M (more memory) reduces I/O significantly. Implement a simulation with various memory budgets.
1. **Consensus message complexity.** A distributed consensus algorithm requires multiple rounds of messages between nodes. In the naive version, every node talks to every other node — $O(n^2)$ messages. An optimized version elects a leader, reducing this to $O(n)$ messages. Analyze the message cost for committing one log entry in a 5-node cluster and a 50-node cluster. Implement a message-counting simulation.
1. **Stack frame size measurement.** The theoretical $O(n)$ stack space for recursion depends on the actual frame size, which varies by compiler and optimization level. Write a recursive function that measures the stack pointer at each depth. Compile with `-O0`, `-O2`, and `-Os`, and measure the actual bytes per frame. How does tail call optimization change the frame count?
1. **Smoothed analysis of sorting.** Implement insertion sort, quicksort, and their behavior on worst-case inputs (already sorted, reverse sorted) vs randomly perturbed inputs. Measure the number of comparisons as a function of perturbation magnitude. At what perturbation level does the worst-case penalty disappear?
1. **Space-efficient recursion.** Implement merge sort with three approaches: (a) standard $O(n)$ auxiliary space, (b) in-place merge using $O(1)$ auxiliary space but $O(n \log^2 n)$ time, (c) a hybrid that switches to insertion sort for small subarrays. Measure the wall-clock time and peak memory for $n = 10^6, 10^7$. Plot the time-memory tradeoff curve.

2.18 References and Further Reading

- Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. MIT Press, 2022.
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Addison-Wesley, 2011.
- Donald E. Knuth, *The Art of Computer Programming*, Volumes 1–4A. Addison-Wesley.
- Daniel H. Greene and Donald E. Knuth, *Mathematics for the Analysis of Algorithms*, 3rd Edition. Birkhauser, 1990.
- Michael Sipser, *Introduction to the Theory of Computation*, 3rd Edition. Cengage, 2012.
- Steven S. Skiena, *The Algorithm Design Manual*, 2nd Edition. Springer, 2008.
- Jon Kleinberg and Eva Tardos, *Algorithm Design*. Addison-Wesley, 2005.
- Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh Vazirani, *Algorithms*. McGraw-Hill, 2006.
- Daniel A. Spielman and Shang-Hua Teng, "Smoothed Analysis of Algorithms: Why the Simplex Algorithm Usually Takes Polynomial Time" — JACM, 2004.
- Daniel A. Spielman and Shang-Hua Teng, "Smoothed Analysis of Algorithms" — STOC, 2001.

Performance Measurement of Programs

3.1 Introduction

In Chapter 2 we analyzed algorithms theoretically. But theory is not enough. Constants matter. Cache behavior matters. Compiler optimizations matter. The only way to know how fast a program actually runs is to measure it.

This chapter covers practical techniques for measuring program performance using modern tools.

‡ **Complete compilable implementations of the data structures in this chapter are in `code/`.**

3.2 Why Measure?

- **Validate theoretical analysis** — does the $O(n^2)$ algorithm actually scale quadratically?
- **Choose between algorithms** — quicksort is $O(n \log n)$ on average, but for small arrays insertion sort is faster
- **Find bottlenecks** — the 90/10 rule: 90% of time is spent in 10% of the code
- **Track regressions** — did a change slow things down?
- **Tune performance** — iterative improvement requires measurement

3.3 A First Measurement: `std::chrono`

C++11 introduced the `<chrono>` library for portable high-resolution timing:

```

1 #include <chrono>
2 #include <iostream>
3
4 class timer {
5     using clock = std::chrono::high_resolution_clock;
6     using time_point = clock::time_point;
7     time_point start_;
8 public:
9     timer() : start_(clock::now()) {}
10    void reset() { start_ = clock::now(); }
11    double elapsed() const {
12        auto end = clock::now();
13        return std::chrono::duration<double>(end - start_).count();

```

```

14     }
15 };
16
17 // Usage:
18 timer t;
19 // code to measure
20 do_something();
21 std::cout << "Elapsed: " << t.elapsed() << " s\n";

```

3.3.1 Clock Types

Clock	Characteristics
<code>system_clock</code>	System-wide real-time clock, can be adjusted
<code>steady_clock</code>	Monotonic — never adjusted, ideal for intervals
<code>high_resolution_clock</code>	Highest precision available (often an alias for <code>steady_clock</code>)

Always use `steady_clock` for performance measurement — it is immune to system time adjustments.

3.3.2 Measuring Short Intervals

For very short operations, the measurement overhead can dominate. Always measure multiple repetitions:

```

1 template <typename F>
2 double measure(F&& f, int iterations = 1000) {
3     timer t;
4     for (int i = 0; i < iterations; ++i) {
5         f();
6     }
7     return t.elapsed() / iterations;
8 }

```

3.4 Google Benchmark Library

For serious benchmarking, use Google Benchmark. It handles warm-up, iteration count, statistical analysis, and output formatting:

```

1 #include <benchmark/benchmark.h>
2
3 static void BM_VectorPushBack(benchmark::State& state) {
4     for (auto _ : state) {
5         std::vector<int> v;
6         for (int i = 0; i < state.range(0); ++i) {
7             v.push_back(i);

```

```

8         }
9     }
10 }
11 BENCHMARK(BM_VectorPushBack)->Range(8, 8<<10);
12
13 BENCHMARK_MAIN();

```

Compile and run:

```

1 g++ -std=c++20 -O2 -lbenchmark benchmark.cpp -o benchmark
2 ./benchmark

```

Output:

1	BM_VectorPushBack/8	25.3 ns	25.1 ns	27692222
2	BM_VectorPushBack/64	151 ns	148 ns	4716981
3	BM_VectorPushBack/512	1186 ns	1175 ns	597163
4	BM_VectorPushBack/4096	10235 ns	10123 ns	68790
5	BM_VectorPushBack/8192	41234 ns	40987 ns	17021

3.5 Factors Affecting Performance

3.5.1 Hardware Effects

- **CPU speed and architecture** — clock rate, superscalar execution, vector instructions
- **Cache hierarchy** — L1 (~32KB, ~1ns), L2 (~256KB, ~4ns), L3 (~8MB, ~10ns), RAM (~100ns)
- **Memory bandwidth** — sustained throughput to main memory
- **Branch prediction** — mispredictions cost 10-20 cycles
- **TLB** — virtual address translation cache

3.5.2 Operating System Effects

- **Context switching** — other processes interrupt your program
- **Page faults** — accessing memory not in physical RAM
- **CPU frequency scaling** — turbo boost, power saving modes

Mitigation: Run benchmarks multiple times, use warm-up phases, run on an idle system.

3.5.3 Compiler Effects

- **Optimization level** (-O0, -O1, -O2, -O3, -Ofast)
- **Inlining** — functions may be inlined or not

- **Loop unrolling** — trade code size for speed
- **Vectorization** — auto-vectorization of SIMD instructions
- **Dead code elimination** — the compiler may remove unused computations

Important: Always benchmark with optimizations enabled (`-O2` or `-O3`). Debug builds (`-O0`) give meaningless timing.

3.5.4 The Benchmarking Trap: Dead Code Elimination

The compiler may optimize away the computation you are trying to measure:

```

1 // WRONG -- compiler may optimize the entire loop away
2 benchmark::DoNotOptimize(0); // use this marker instead
3 static void BM_Bad(benchmark::State& state) {
4     int sum = 0;
5     for (auto _ : state) {
6         for (int i = 0; i < state.range(0); ++i) {
7             sum += i; // compiler may notice 'sum' is unused
8         }
9     }
10 }
11 // The compiler can eliminate the entire inner loop since 'sum' has no
    observable effect

```

Always consume the result:

```

1 static void BM_Good(benchmark::State& state) {
2     int sum = 0;
3     for (auto _ : state) {
4         for (int i = 0; i < state.range(0); ++i) {
5             sum += i;
6         }
7     }
8     benchmark::DoNotOptimize(sum);
9 }

```

3.6 Case Study: Sorting Algorithms

Let us measure insertion sort, merge sort, and `std::sort`:

```

1 void insertion_sort(std::span<int> a) {
2     for (size_t i = 1; i < a.size(); ++i) {
3         int key = a[i];
4         int j = static_cast<int>(i) - 1;
5         while (j >= 0 && a[j] > key) {
6             a[j + 1] = a[j];

```

```

7         --j;
8     }
9     a[j + 1] = key;
10 }
11 }
12
13 void merge_sort(std::span<int> a) {
14     if (a.size() <= 1) return;
15     size_t mid = a.size() / 2;
16     std::vector<int> left(a.begin(), a.begin() + mid);
17     std::vector<int> right(a.begin() + mid, a.end());
18     merge_sort(left);
19     merge_sort(right);
20     std::merge(left.begin(), left.end(), right.begin(), right.end(), a.
        begin());
21 }

```

```

1 static void BM_InsertionSort(benchmark::State& state) {
2     std::vector<int> v(state.range(0));
3     for (auto _ : state) {
4         std::iota(v.begin(), v.end(), 0);
5         std::shuffle(v.begin(), v.end(), std::mt19937{42});
6         insertion_sort(v);
7         benchmark::DoNotOptimize(v.data());
8     }
9 }
10 BENCHMARK(BM_InsertionSort)->Range(8, 8<<10);
11
12 static void BM_MergeSort(benchmark::State& state) {
13     std::vector<int> v(state.range(0));
14     for (auto _ : state) {
15         std::iota(v.begin(), v.end(), 0);
16         std::shuffle(v.begin(), v.end(), std::mt19937{42});
17         merge_sort(v);
18         benchmark::DoNotOptimize(v.data());
19     }
20 }
21 BENCHMARK(BM_MergeSort)->Range(8, 8<<10);
22
23 static void BM_StdSort(benchmark::State& state) {
24     std::vector<int> v(state.range(0));
25     for (auto _ : state) {
26         std::iota(v.begin(), v.end(), 0);
27         std::shuffle(v.begin(), v.end(), std::mt19937{42});
28         std::sort(v.begin(), v.end());
29         benchmark::DoNotOptimize(v.data());
30     }
31 }

```



```
32 BENCHMARK(BM_StdSort) ->Range(8, 8<<10);
```

Expected results:

- Insertion sort: $O(n^2)$ — noticeable quadratic curve
- Merge sort: $O(n \log n)$ — smoother curve
- `std::sort`: fastest — uses introsort (quicksort + heapsort + insertion sort hybrid)

3.7 Profiling

While benchmarking measures total time, **profiling** shows where time is spent.

3.7.1 Sampling Profilers

Sample the program counter at regular intervals:

```
1 # macOS
2 sudo sample ./program -duration 10 # produces a call tree
3
4 # Linux
5 perf record ./program && perf report
6
7 # macOS Instruments (GUI)
8 xcrun xctrace record --template "Time Profiler" --launch ./program
```

3.7.2 Instrumenting Profilers

Valgrind/Callgrind:

```
1 valgrind --tool=callgrind ./program
2 # Produces callgrind.out.xxx -- view with kcachegrind
```

Compiler instrumentation (`-finstrument-functions`):

GCC/Clang can insert callbacks at every function entry/exit for custom profiling.

3.7.3 Cache Analysis with perf

```
1 perf stat -e cache-references,cache-misses,cycles,instructions ./program
```

Sample output:

1	1,234,567,890	cycles	
2	987,654,321	instructions	
3	12,345,678	cache-references	
4	1,234,567	cache-misses	# 10.0% miss rate

A high cache miss rate (>10%) indicates poor data locality.

3.8 Measuring Memory Usage

```

1 #include <sys/resource.h>
2
3 size_t peak_memory_usage() {
4     struct rusage usage;
5     getrusage(RUSAGE_SELF, &usage);
6     return usage.ru_maxrss; // in kilobytes (macOS) or bytes (Linux)
7 }

```

For detailed heap profiling, use **Valgrind/Massif**:

```

1 valgrind --tool=massif ./program
2 ms_print massif.out.xxx

```

3.9 Practical Measurement Methodology

1. **Formulate a hypothesis** — e.g., "std::sort is faster than my quicksort"
2. **Design the experiment** — what inputs, what sizes, what metrics?
3. **Warm up** — run the code once to populate caches
4. **Measure multiple trials** — at least 10, report mean and standard deviation
5. **Vary input size** — confirm asymptotic class
6. **Vary input type** — random, sorted, reversed, nearly sorted
7. **Isolate the system** — close other applications, disable CPU scaling
8. **Report everything** — compiler, flags, CPU, OS, input parameters

3.10 ADT for Polynomial

An abstract data type for polynomials encapsulates the representation of a polynomial and provides operations to manipulate it. We first specify the ADT, then implement it using a vector of coefficients.

3.10.1 ADT Specification

A polynomial $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ is represented by its degree n and a sequence of coefficients $(a_0, a_1, \dots, a_n)$.

Operations:

- **Polynomial()** – construct the zero polynomial

- **Polynomial(vector<double>)** – construct from a coefficient vector
- **degree()** – return the highest power with a nonzero coefficient
- **coefficient(i)** – return the coefficient of x^i
- **evaluate(x)** – return $p(x)$ using Horner’s method, $O(n)$
- **add(q)** – return $p(x) + q(x)$
- **multiply(q)** – return $p(x) \cdot q(x)$, naive $O(n^2)$ or FFT-based $O(n \log n)$
- **derivative()** – return $p'(x)$
- **operator==** – test structural equality
- **operator<<** – stream output

Complexity summary:

Operation	Complexity
evaluate	$O(n)$
add	$O(n)$
multiply	$O(n^2)$
derivative	$O(n)$

3.10.2 Implementation

We store coefficients in a `std::vector<double>` where index i holds the coefficient of x^i . This gives $O(1)$ access to any coefficient and $O(n)$ evaluation.

```

1 #include <vector>
2 #include <stdexcept>
3 #include <iostream>
4
5 class polynomial {
6 public:
7     polynomial() = default;
8
9     explicit polynomial(std::vector<double> coeffs)
10         : coeffs_(std::move(coeffs)) {
11         trim();
12     }
13
14     int degree() const {
15         return static_cast<int>(coeffs_.size()) - 1;
16     }
17

```

```

18     bool is_zero() const {
19         return coeffs_.empty();
20     }
21
22     double coefficient(int i) const {
23         if (i < 0 || i > degree()) return 0.0;
24         return coeffs_[i];
25     }
26
27     // Horner's method: O(n) evaluation
28     double evaluate(double x) const {
29         if (is_zero()) return 0.0;
30         double result = coeffs_[degree()];
31         for (int i = degree() - 1; i >= 0; --i) {
32             result = result * x + coeffs_[i];
33         }
34         return result;
35     }
36
37     polynomial add(const polynomial& q) const {
38         size_t n = std::max(coeffs_.size(), q.coeffs_.size());
39         std::vector<double> result(n, 0.0);
40         for (size_t i = 0; i < coeffs_.size(); ++i) result[i] += coeffs_[i];
41         for (size_t i = 0; i < q.coeffs_.size(); ++i) result[i] += q.
coeffs_[i];
42         return polynomial(std::move(result));
43     }
44
45     // Naive polynomial multiplication: O(n * m)
46     polynomial multiply(const polynomial& q) const {
47         if (is_zero() || q.is_zero()) return polynomial();
48         int n = degree();
49         int m = q.degree();
50         std::vector<double> result(n + m + 1, 0.0);
51         for (int i = 0; i <= n; ++i) {
52             for (int j = 0; j <= m; ++j) {
53                 result[i + j] += coeffs_[i] * q.coeffs_[j];
54             }
55         }
56         return polynomial(std::move(result));
57     }
58
59     // Returns the formal derivative: O(n)
60     polynomial derivative() const {
61         if (degree() <= 0) return polynomial();
62         std::vector<double> result(degree());
63         for (int i = 1; i <= degree(); ++i) {

```

```

64         result[i - 1] = i * coeffs_[i];
65     }
66     return polynomial(std::move(result));
67 }
68
69 bool operator==(const polynomial& q) const {
70     return coeffs_ == q.coeffs_;
71 }
72
73 bool operator!=(const polynomial& q) const {
74     return !(*this == q);
75 }
76
77 friend std::ostream& operator<<(std::ostream& os, const polynomial& p)
78 {
79     if (p.is_zero()) return os << "0";
80     for (int i = p.degree(); i >= 0; --i) {
81         if (p.coeffs_[i] == 0.0) continue;
82         if (i < p.degree()) os << (p.coeffs_[i] > 0 ? " + " : " - ");
83         double c = std::abs(p.coeffs_[i]);
84         if (c != 1.0 || i == 0) os << c;
85         if (i > 0) os << "x";
86         if (i > 1) os << "^" << i;
87     }
88     return os;
89 }
90 private:
91     // Remove trailing zero coefficients
92     void trim() {
93         while (!coeffs_.empty() && coeffs_.back() == 0.0) {
94             coeffs_.pop_back();
95         }
96     }
97
98     std::vector<double> coeffs_;
99 };

```

3.10.3 Example Usage

```

1 #include <iostream>
2
3 int main() {
4     // p(x) = 3x^3 + 2x + 1
5     polynomial p({1, 2, 0, 3});
6     // q(x) = x^2 + 4
7     polynomial q({4, 0, 1});

```

```

8
9     std::cout << "p(x) = " << p << "\n";
10    std::cout << "q(x) = " << q << "\n";
11    std::cout << "p(2) = " << p.evaluate(2) << "\n"; // 30
12
13    polynomial sum = p.add(q);
14    std::cout << "p+q = " << sum << "\n";
15
16    polynomial prod = p.multiply(q);
17    std::cout << "p*q = " << prod << "\n";
18
19    polynomial dp = p.derivative();
20    std::cout << "p'(x) = " << dp << "\n";
21 }

```

3.10.4 Key Design Decisions

- **Trimming leading zeros** keeps the invariant that the degree is accurate. Every constructor and mutating operation calls `trim()`.
- **Horner's method** for evaluation uses only n multiplications and n additions, with good numerical stability.
- **Naive multiplication** is $O(n^2)$. For high-degree polynomials, the FFT-based approach achieves $O(n \log n)$, but the constant factor is large enough that naive is faster for small degrees.
- **Coefficient vector** representation is simple and cache-friendly but wastes space for sparse polynomials (few nonzero terms). A map-based representation would be better for sparse polynomials.

3.11 ADT for Matrix

A matrix ADT encapsulates a two-dimensional array of values and provides linear algebra operations. We focus on **sparse matrices**, which are common in scientific computing and machine learning.

3.11.1 ADT Specification

A matrix A of size $m \times n$ has m rows and n columns. A matrix is **sparse** when most entries are zero.

Operations:

- **Matrix(m, n)** – construct an $m \times n$ zero matrix
- **Matrix(m, n, entries)** – construct from a list of $(row, col, value)$ triples
- **rows(), cols()** – return dimensions

- **get(i, j)** – return A_{ij} , $O(1)$ amortized
- **set(i, j, val)** – set $A_{ij} = \text{val}$
- **add(B)** – return $A + B$, $O(\text{nnz})$
- **multiply(B)** – return $A \cdot B$, $O(\text{nnz}(A) \cdot n_{\text{cols}})$ for naive
- **transpose()** – return A^T , $O(\text{nnz})$

where nnz is the number of nonzero entries.

Complexity summary (sparse representation):

Operation	Complexity
get	$O(\log \text{nnz})$ per row
set	$O(\log \text{nnz})$ per row
add	$O(\text{nnz})$
multiply	$O(\text{nnz}(A) \cdot n_{\text{cols}}(B))$
transpose	$O(m + n + \text{nnz})$

3.11.2 Compressed Row Storage (CRS)

Compressed Row Storage, also called CSR, is the standard sparse matrix format. It stores:

- **values** – array of all nonzero values, row by row
- **col_index** – array of column indices corresponding to each value
- **row_ptr** – array of size $m + 1$; $\text{row_ptr}[i]$ is the start of row i in the values array

For a 4×5 matrix:

$$A = \begin{pmatrix} 1 & 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 4 & 0 & 0 & 5 \\ 0 & 0 & 0 & 6 & 0 \end{pmatrix}$$

The CRS representation is:

```
1 values    = { 1, 2, 3, 4, 5, 6 }
2 col_index = { 0, 3, 2, 1, 4, 3 }
3 row_ptr   = { 0, 2, 3, 5, 6 }
```

Row 2 spans $\text{values}[3..4] = \{4, 5\}$ in columns $\{1, 4\}$. Finding a specific element in row i requires a binary search within $\text{row_ptr}[i]$ to $\text{row_ptr}[i+1] - 1$, giving $O(\log \text{nnz_in_row})$ access.

3.11.3 Implementation

```

1  #include <vector>
2  #include <algorithm>
3  #include <stdexcept>
4  #include <iostream>
5
6  class sparse_matrix {
7  public:
8      using triplet = std::tuple<int, int, double>;
9
10     sparse_matrix(int rows, int cols)
11         : rows_(rows), cols_(cols),
12           row_ptr_(rows + 1, 0) {}
13
14     sparse_matrix(int rows, int cols, std::vector<triplet> entries)
15         : rows_(rows), cols_(cols) {
16         // Sort entries by (row, col)
17         std::sort(entries.begin(), entries.end(),
18             [](const triplet& a, const triplet& b) {
19             if (std::get<0>(a) != std::get<0>(b))
20                 return std::get<0>(a) < std::get<0>(b);
21             return std::get<1>(a) < std::get<1>(b);
22             });
23
24         // Build CRS arrays
25         row_ptr_.assign(rows + 1, 0);
26         for (const auto& [r, c, v] : entries) {
27             if (v != 0.0) {
28                 row_ptr_[r + 1]++;
29                 values_.push_back(v);
30                 col_index_.push_back(c);
31             }
32         }
33
34         // Convert counts to prefix sums
35         for (int i = 0; i < rows; ++i) {
36             row_ptr_[i + 1] += row_ptr_[i];
37         }
38     }
39
40     int rows() const { return rows_; }
41     int cols() const { return cols_; }
42     int nnz() const { return static_cast<int>(values_.size()); }
43
44     double get(int r, int c) const {
45         int start = row_ptr_[r];
46         int end = row_ptr_[r + 1];

```



```

47     auto it = std::lower_bound(
48         col_index_.begin() + start,
49         col_index_.begin() + end, c);
50     if (it != col_index_.begin() + end && *it == c) {
51         return values_[it - col_index_.begin()];
52     }
53     return 0.0;
54 }
55
56 // O(nnz) matrix addition
57 sparse_matrix add(const sparse_matrix& B) const {
58     if (rows_ != B.rows_ || cols_ != B.cols_)
59         throw std::invalid_argument("dimension mismatch");
60
61     sparse_matrix result(rows_, cols_);
62     std::vector<triplet> entries;
63
64     for (int i = 0; i < rows_; ++i) {
65         int a_start = row_ptr_[i], a_end = row_ptr_[i + 1];
66         int b_start = B.row_ptr_[i], b_end = B.row_ptr_[i + 1];
67         int ai = a_start, bi = b_start;
68         while (ai < a_end || bi < b_end) {
69             int ac = (ai < a_end) ? col_index_[ai] : cols_;
70             int bc = (bi < b_end) ? B.col_index_[bi] : cols_;
71             if (ac == bc) {
72                 double sum = values_[ai] + B.values_[bi];
73                 if (sum != 0.0) entries.emplace_back(i, ac, sum);
74                 ++ai; ++bi;
75             } else if (ac < bc) {
76                 entries.emplace_back(i, ac, values_[ai]);
77                 ++ai;
78             } else {
79                 entries.emplace_back(i, bc, B.values_[bi]);
80                 ++bi;
81             }
82         }
83     }
84     return sparse_matrix(rows_, cols_, entries);
85 }
86
87 // O(nnz(A) * n_cols(B)) sparse matrix multiply
88 sparse_matrix multiply(const sparse_matrix& B) const {
89     if (cols_ != B.rows_)
90         throw std::invalid_argument("dimension mismatch");
91
92     std::vector<triplet> entries;
93     for (int i = 0; i < rows_; ++i) {
94         std::vector<double> tmp(B.cols_, 0.0);

```

```

95         for (int ai = row_ptr_[i]; ai < row_ptr_[i + 1]; ++ai) {
96             int k = col_index_[ai];
97             double a_val = values_[ai];
98             for (int bj = B.row_ptr_[k]; bj < B.row_ptr_[k + 1]; ++bj)
99             {
100                 tmp[B.col_index_[bj]] += a_val * B.values_[bj];
101             }
102         }
103         for (int j = 0; j < B.cols_; ++j) {
104             if (tmp[j] != 0.0) entries.emplace_back(i, j, tmp[j]);
105         }
106         return sparse_matrix(rows_, B.cols_, entries);
107     }
108
109     // O(m + n + nnz) transpose
110     sparse_matrix transpose() const {
111         sparse_matrix result(cols_, rows_);
112         result.row_ptr_.assign(cols_ + 1, 0);
113
114         // Count entries per column (now row in result)
115         for (int c : col_index_) {
116             result.row_ptr_[c + 1]++;
117         }
118         for (int i = 0; i < cols_; ++i) {
119             result.row_ptr_[i + 1] += result.row_ptr_[i];
120         }
121
122         result.values_.resize(nnz());
123         result.col_index_.resize(nnz());
124         std::vector<int> pos(result.row_ptr_.begin(), result.row_ptr_.end
125         ());
126
127         for (int i = 0; i < rows_; ++i) {
128             for (int j = row_ptr_[i]; j < row_ptr_[i + 1]; ++j) {
129                 int c = col_index_[j];
130                 int dest = pos[c]++;
131                 result.values_[dest] = values_[j];
132                 result.col_index_[dest] = i;
133             }
134         }
135         return result;
136     }
137
138     friend std::ostream& operator<<(std::ostream& os, const sparse_matrix&
139     m) {
140         for (int i = 0; i < m.rows_; ++i) {
141             for (int j = 0; j < m.cols_; ++j) {

```

```

140         if (j > 0) os << " ";
141         os << m.get(i, j);
142     }
143     os << "\n";
144 }
145 return os;
146 }
147
148 private:
149     int rows_, cols_;
150     std::vector<double> values_;
151     std::vector<int> col_index_;
152     std::vector<int> row_ptr_;
153 };

```

3.11.4 When to Use CRS vs Dense

Criterion	Dense (2D vector)	CRS
Memory	$O(mn)$	$O(m + n + \text{nnz})$
Access A_{ij}	$O(1)$	$O(\log \text{nnz_in_row})$
Iteration over row	$O(n)$	$O(\text{nnz_in_row})$
Cache behavior	Excellent for small matrices	Good when rows fit in cache
Overhead	None	Index arrays

Use CRS when the matrix is large and sparse ($\text{nnz} \ll mn$). For matrices that are more than 30-50% nonzero, a dense representation is usually faster due to better cache locality and no indirection overhead.

3.12 ADT for Graph

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges $E \subseteq V \times V$. The **Graph Abstract Data Type** specifies the operations available on a graph without committing to a particular representation.

3.12.1 ADT Specification

The Graph ADT supports the following core operations:

Operation	Signature	Description
add_edge	add_edge(u, v)	Add edge (u, v)
remove_edge	remove_edge(u, v)	Remove edge (u, v)
has_edge	has_edge(u, v) $\rightarrow$ bool	Test if (u, v) exists
degree	degree(v) $\rightarrow$ int	Number of neighbors of v
bfs	bfs(start) $\rightarrow$ vector	Breadth-first traversal
dfs	dfs(start) $\rightarrow$ vector	Depth-first traversal

3.12.2 Representations

Two common representations exist:

- **Adjacency Matrix.** A 2D array A of size $|V| \times |V|$ where $A[u][v] = 1$ if edge (u, v) exists, 0 otherwise. Space: $O(|V|^2)$. Edge test: $O(1)$. Adding/removing an edge: $O(1)$.
- **Adjacency List.** An array of lists, one per vertex, storing neighbors. Space: $O(|V| + |E|)$. Edge test: $O(\deg(u))$. Adding an edge: $O(1)$ (prepend).

The adjacency list is preferred for sparse graphs; the adjacency matrix is preferred for dense graphs or when $O(1)$ edge queries are critical.

3.12.3 C++ Implementation (Adjacency List)

Listing 3.1. Graph ADT using adjacency list

```

1 #include <iostream>
2 #include <vector>
3 #include <list>
4 #include <queue>
5 #include <algorithm>
6
7 class Graph {
8 public:
9     explicit Graph(int num_vertices)
10         : adj_(num_vertices) {}
11
12     void add_edge(int u, int v) {
13         adj_[u].push_back(v);
14         adj_[v].push_back(u);
15     }
16
17     void remove_edge(int u, int v) {
18         adj_[u].remove(v);
19         adj_[v].remove(u);
20     }

```

```

21
22     bool has_edge(int u, int v) const {
23         for (int neighbor : adj_[u]) {
24             if (neighbor == v) return true;
25         }
26         return false;
27     }
28
29     int degree(int v) const {
30         return static_cast<int>(adj_[v].size());
31     }
32
33     std::vector<int> bfs(int start) const {
34         std::vector<int> visited(adj_.size(), 0);
35         std::vector<int> order;
36         std::queue<int> q;
37         visited[start] = 1;
38         q.push(start);
39         while (!q.empty()) {
40             int v = q.front(); q.pop();
41             order.push_back(v);
42             for (int w : adj_[v]) {
43                 if (!visited[w]) {
44                     visited[w] = 1;
45                     q.push(w);
46                 }
47             }
48         }
49         return order;
50     }
51
52     std::vector<int> dfs(int start) const {
53         std::vector<int> visited(adj_.size(), 0);
54         std::vector<int> order;
55         dfs_helper(start, visited, order);
56         return order;
57     }
58
59     int num_vertices() const {
60         return static_cast<int>(adj_.size());
61     }
62
63 private:
64     std::vector<std::list<int>> adj_;
65
66     void dfs_helper(int v, std::vector<int>& visited,
67                     std::vector<int>& order) const {
68         visited[v] = 1;

```

```

69     order.push_back(v);
70     for (int w : adj_[v]) {
71         if (!visited[w]) {
72             dfs_helper(w, visited, order);
73         }
74     }
75 }
76 };

```

3.12.4 Complexity Summary

Operation	Adj. List	Adj. Matrix
Space	$O(V + E)$	$O(V ^2)$
add_edge	$O(1)$	$O(1)$
remove_edge	$O(\deg u)$	$O(1)$
has_edge	$O(\deg u)$	$O(1)$
degree	$O(1)$	$O(V)$
BFS/DFS	$O(V + E)$	$O(V ^2)$

For a graph with $|V| = 10,000$ vertices and $|E| = 20,000$ edges, the adjacency list uses roughly 240 KB while the adjacency matrix uses 400 MB — a factor of $1,700\times$.

3.13 Common Pitfalls

Pitfall	Consequence	Solution
Debug build	Measurements include assertions	Use <code>-O2</code>
No warm-up	Cold cache skews results	Run iterations before measuring
Too few iterations	High variance	Use enough iterations for stable mean
Dead code elimination	Zero time reported	Consume results with <code>DoNotOptimize</code>
System load	Inconsistent timing	Run on idle system, report median
Clock resolution	Zero elapsed time	Use enough work per measurement
Inlining	Function not measured separately	Use <code>__attribute__((noinline))</code>

3.14 Summary

1. Use `std::chrono::steady_clock` for portable timing.
2. Use **Google Benchmark** for serious measurement — it handles warm-up, iteration control, and statistics.
3. Always **optimize** (`-O2` or `-O3`) when benchmarking.

4. **Prevent dead code elimination** by consuming results.
5. **Profile** to find bottlenecks; **measure** to verify improvements.
6. **Understand hardware effects** — cache misses dominate for large data sets.
7. **Report methodology** so results are reproducible.

3.15 Exercises

3.15.1 Drill

1. **Average vs tail latency.** A performance test reports average latency of 2 ms but P99 (99th percentile) of 200 ms. Why are these so different? For a checkout service that must respond within 10 ms, which metric matters? What fraction of customers experience a timeout?

(In production, companies like Google and Amazon use P99.9 or P99.99 for SLOs because averages hide the slow requests that hurt user experience.)

1. **Clock choice.** You measure a function with `std::chrono::steady_clock` and get 4.8 seconds. A colleague uses `std::chrono::system_clock` and gets 5.2 seconds. Both ran the same code on the same machine. What property of `system_clock` (vs `steady_clock`) explains the difference? Which is correct for benchmarking?

1. **Tail latency in servers.** A web server handles 100 concurrent connections. The average response time reported is 2 ms, but P99.9 is 500 ms. An engineer says “the 500 ms is just an outlier — I’ll optimize the 2 ms average.” Why is this wrong for an e-commerce site? If 0.1% of requests are slow and the site serves 1 million requests/day, how many users are affected?

1. **Dead code elimination.** The following benchmark always reports 0 ns:

```

1  static void BM_Sort(benchmark::State& state) {
2      std::vector<int> v(1000000);
3      std::iota(v.begin(), v.end(), 0);
4      for (auto _ : state) {
5          std::sort(v.begin(), v.end()); // result never used!
6      }
7  }
```

Why does the compiler optimize away the sort? How do you fix it?

1. **Wall time vs CPU time.** A benchmark shows wall time = 10 s but CPU time = 4 s. Where did the other 6 seconds go? List three possible causes.

(In production, burstable cloud instances (like AWS t3) can cause this discrepancy when CPU credits run out.)

3.15.2 Application

1. **Throughput-latency measurement.** A message queue ingests data from producers. On a fast server it handles 800 MB/s. On a modest server, design a benchmark that:

- Measures throughput (MB/s)
- Reports P50, P99, P99.9 latency
- Runs for at least 60 seconds to steady state
- Compares synchronous (wait for confirmation) vs asynchronous (fire-and-forget) modes

Plot throughput vs latency. What throughput can you guarantee if latency must stay under 50 ms?

1. **Load testing.** A web service gets 1000 req/s at night and 10000 req/s during peak. Load testing at peak shows 12 ms average latency. A colleague says "run at 60% CPU to leave headroom." Another says "90% CPU is fine, latency is 11 ms." Design an experiment that gradually increases load and finds the inflection point where latency starts to spike.

1. **Cache effects.** An analytical query runs 10× slower the first time than subsequent times. The difference is the cache. Design a benchmark that:

- Warms the cache (run once, discard)
- Then measures cold cache (drop OS caches)
- Tests concurrent queries (1, 4, 16 simultaneous)
- Tests different data sizes (1 GB, 10 GB)
- Runs each configuration 10+ times for statistical significance

Report which factor (concurrency, data size, cache state) has the largest impact.

(In production, databases like ClickHouse are sensitive to all three — their documentation specifies which knobs to tune.)

1. **SIMD vs scalar.** Compare an element-wise operation on a large array using plain C++ vs using SIMD intrinsics (SSE/AVX on x86 or NEON on ARM):

- Element-wise multiplication of 10^7 floats
- Dot product of two vectors

Use your platform's timing. Plot time vs array size. At what size does SIMD pull ahead?

1. **False sharing.** Two threads increment adjacent variables in an array. Performance is poor beyond 2 cores. The cause is false sharing: the CPU cache line (typically 64 bytes) holds both variables, so each write invalidates the other thread's copy. Design a benchmark that:

- Creates an array of atomic counters
- Launches 2, 4, 8 threads, each incrementing its own counter
- Measures throughput with and without padding (put each counter in its own cache line)
- Reports speedup vs thread count

(In production, this pattern appears wherever threads update independent but adjacent data — Uber's H3 geospatial indexing library is one example.)

3.15.3 ADT Design and Implementation

1. **Polynomial evaluation.** Implement the polynomial class from this chapter. Given $p(x) = 5x^4 - 3x^2 + 7$, compute $p(1)$, $p(-2)$, and $p(0)$. Verify by hand.
1. **Horner vs naive.** Write two evaluation functions: one using Horner's method and one computing each power of x separately using `std::pow`. Benchmark both for a degree-100 polynomial over 10^6 random inputs. Which is faster and why?
1. **Sparse matrix construction.** Create a 5×5 identity matrix in CRS format. Print the `values`, `col_index`, and `row_ptr` arrays. Verify that the number of entries in `row_ptr` is $m + 1$.
1. **Matrix transpose.** Given a 3×4 sparse matrix with 5 nonzero entries, compute its transpose by hand. Then run the `transpose()` method and verify the output matches.
1. **Polynomial derivative chain rule.** Extend the polynomial class with a method `compose(q)` that computes $p(q(x))$. What is the time complexity in terms of the degrees of p and q ? Implement it and test with $p(x) = x^2 + 1$ and $q(x) = 2x + 3$.
1. **Sparse matrix multiply benchmark.** Generate two random 1000×1000 sparse matrices with 1% nonzero density. Measure the time for naive multiply ($O(mn^2)$ using dense representation) vs CRS multiply. At what density does CRS become faster?
1. **Polynomial ADT with map storage.** Implement a second polynomial class that stores nonzero coefficients in a `std::map<int, double>` instead of a vector. Compare memory usage and evaluation speed with the vector-based version for a degree-1000 polynomial with only 5 nonzero terms.
1. **CRS vs COO format.** Implement the Coordinate (COO) sparse matrix format, which stores a list of (row, col, value) triples. Compare it with CRS for: (a) construction time from a stream of entries, (b) element access time, (c) memory usage. When is COO preferable to CRS?

3.15.4 Inheritance and Templates

1. **Inheritance hierarchy for matrices.** Design a base class `matrix_base` with pure virtual methods `get`, `set`, `rows`, `cols`. Derive `dense_matrix` and `sparse_matrix` (using CRS) from it. Write a function `multiply(matrix_base& a, matrix_base& b)` that works polymorphically. Discuss the overhead of virtual dispatch in tight loops.
1. **Template matrix class.** Write a template class `matrix<T>` that works for `int`, `double`, and a user-defined `rational` type. Ensure that all arithmetic operations (`+`, `*`, `transpose`) compile and produce correct results for each type.
1. **Operator overloading for matrices.** Extend the `sparse_matrix` class with operator overloading: `operator+`, `operator*`, `operator==`, and `operator<<`. Discuss the design tension between returning by value (safe but copies) and returning by reference (efficient but lifetime issues). Implement both and benchmark the difference for large matrices.

3.15.5 Research

1. **Cycle-accurate timing.** Research the `rdtscp` CPU instruction that reads the cycle counter. Write a C++ wrapper. Benchmark `std::chrono::steady_clock::now()` overhead vs your `rdtscp` wrapper over 10^6 calls. Which is more precise? When would a cycle counter give misleading results (hint: CPU frequency scaling)?
1. **Profiling a real system.** Download an open-source database engine (e.g., DuckDB). Build it. Run a standard benchmark query and profile using `perf record` or Instruments. Identify:
 - The hottest 3 functions
 - Percentage of time in hash lookups vs parsing vs I/O
 - Cache miss rates (L1, LLC)
 - Branch misprediction rate

Propose one optimization based on the profile.

1. **Statistical significance.** Run a sorting benchmark 30 times under identical conditions. Compute mean, std dev, and 95% confidence interval. Then introduce a small change (e.g., switch from `std::sort` to `std::stable_sort`) — can you detect the difference with 30 runs? With 100 runs? How small a change can you reliably detect?

(In production, Google's performance lab uses A/A testing (both groups get the same treatment) to measure the noise floor before running actual experiments.)

1. **Compiler profiling.** Compile a medium-sized C++ project and use your compiler's time-tracing flags (e.g., `-ftime-report` in GCC/Clang) to measure which optimization pass takes the most time. Is it inlining? Loop optimization? Register allocation? Propose one improvement.
1. **FFT-based polynomial multiplication.** Research the Cooley-Tukey FFT algorithm. Implement polynomial multiplication using FFT: (1) convert coefficient vectors to point-value representation via FFT, (2) multiply pointwise, (3) convert back via inverse FFT. Benchmark against naive multiplication for polynomials of degree 2^{10} through 2^{16} . At what degree does FFT become faster?
1. **Strassen matrix multiplication.** Research Strassen's algorithm, which multiplies $n \times n$ matrices in $O(n^{2.807})$ time. Implement it for dense matrices. Compare with naive $O(n^3)$ multiplication for $n = 64, 128, 256, 512$. What is the crossover point? What are the constant-factor penalties?
1. **Compressed Column Storage (CCS).** Research CCS, the column-oriented dual of CRS. Implement it. For which operations (row access, column access, transpose, multiply) is CCS faster than CRS? When would a library choose CCS over CRS?
1. **Expression templates for lazy evaluation.** Research expression templates, a C++ technique that avoids temporary matrices in expressions like $C = A + B * D$. Implement a minimal expression template system for matrix operations. Measure the speedup compared to eager evaluation on a chain of 10 matrix additions and multiplications.
1. **Polynomial root finding.** Research Newton's method for finding real roots of a polynomial. Implement it using the polynomial class's `evaluate` and `derivative` methods. Find all real roots of $p(x) = x^3 - 6x^2 + 11x - 6$ (which factors as $(x-1)(x-2)(x-3)$). Discuss convergence guarantees and failure cases.

3.16 Primitive and Non-Primitive Data Structures

Every program manipulates data. The fundamental question is: *how* do we organize that data? Data structures fall into two broad categories: **primitive** and **non-primitive**.

3.16.1 Primitive Data Structures

Primitive data structures store a single value directly in memory. They are built into the C++ language and have fixed sizes determined by the hardware and compiler. The CPU can access them in a single operation (typically one memory load).

```

1 int x = 42;           // 4 bytes, stored in a register or stack
2 double pi = 3.14159; // 8 bytes
3 char c = 'A';         // 1 byte
4 bool flag = true;     // 1 byte

```

```
5 float f = 2.718f;           // 4 bytes
6 long long big = 1e18;       // 8 bytes
7 void* ptr = &x;             // 8 bytes (64-bit pointer)
```

Properties of primitives:

- Fixed size known at compile time
- Stored by value (no indirection)
- O(1) access — a single load/store instruction
- No methods, no encapsulation (just raw bits)
- Can be combined into struct for composite values

```
1 // Structs compose primitives into a single unit
2 struct Point {
3     double x; // 8 bytes
4     double y; // 8 bytes
5 };           // 16 bytes total, contiguous in memory
6
7 Point p = {3.0, 4.0}; // stored as 16 consecutive bytes
```

3.16.2 Non-Primitive Data Structures

Non-primitive data structures organize multiple data elements into relationships. They are built by the programmer (or provided by libraries) and can grow dynamically. They form the core of this book.

Non-primitive structures are divided into two categories:

Linear Data Structures

Elements are arranged in a sequence. Each element has at most one predecessor and one successor.

Structure	Access	Memory
Array	O(1) by index	Contiguous
Linked List	O(n) by position	Scattered nodes
Stack	O(1) top only	Array or linked
Queue	O(1) front/rear	Array or linked
Hash Table	O(1) average by key	Buckets + nodes

```
1 // Array: contiguous, O(1) index access
2 int arr[5] = {10, 20, 30, 40, 50};
3 int val = arr[2]; // O(1) -- direct memory offset
4
```

```
5 // Linked list: scattered, O(n) traversal
6 struct Node { int data; Node* next; };
7 Node* head = new Node{10, new Node{20, new Node{30, nullptr}}};
8 // To reach the 3rd element: head->next->next->data (O(n))
```

Non-Linear Data Structures

Elements form relationships beyond simple sequences — trees, graphs, and their variants. Each element can connect to multiple others.

Structure	Relationship	Example
Binary Tree	Parent → 2 children	Expression trees, BSTs
Multi-way Tree	Parent → many children	B-trees, tries
Graph	Arbitrary connections	Social networks, road maps
Heap	Parent ≤ children (partial order)	Priority queues
Hash Map	Key → value	Dictionaries, caches

```
1 // Binary tree: each node has up to 2 children
2 struct TreeNode {
3     int val;
4     TreeNode* left;
5     TreeNode* right;
6 };
7
8 // Graph: adjacency list
9 std::vector<std::vector<int>> adj(n); // adj[u] = neighbors of u
10 adj[0].push_back(1); // edge 0 -> 1
11 adj[0].push_back(2); // edge 0 -> 2
12 adj[1].push_back(3); // edge 1 -> 3
```

3.16.3 Classification Tree

Category	Type	Examples
Primitive	Integral	int, char, bool, long long
	Floating-point	float, double
	Pointer	T*, std::shared_ptr<T>
	Composite	struct, std::pair
Non-Primitive	Linear	Array, Linked List, Stack, Queue
	Non-Linear	Tree, Graph, Heap, Hash Table
Design	Active	std::vector, AVL, Segment Tree
	Passive	C-array, POD struct

3.16.4 Comparison

Aspect	Primitive	Linear	Non-Linear	Active
Size	Fixed	Dynamic	Dynamic	Dynamic
Access	O(1)	O(1) index / O(n) search	Varies	Varies
Memory	Contiguous	Varies	Varies	Varies
Operations	None	Insert/Delete	Insert/Delete	Encapsulated
Cache	Excellent	Good (array) / Bad (list)	Varies	Varies
Overhead	None	Pointers	Pointers + bookkeeping	RAII + invariants

3.16.5 Interview FAQs

1. Where does the primitive/non-primitive boundary actually break down in practice?

The real distinction is ownership semantics. A primitive is a value type — trivially copyable, no invariants. A non-primitive manages state: it owns memory, enforces invariants, or provides operations that mutate internal structure. But the line blurs constantly. `std::atomic<int>` is a single value but manages a memory ordering contract. `std::span` is a view over contiguous memory — no allocation, but it carries size and bounds. In embedded/HPC, you often *want* non-primitive organization (e.g., SOA layout) with primitive-like semantics (no heap, no indirection). The question isn't "what category does this fall into?" but "who owns the invariants?"

1. Why does the compiler treat `int [3]` differently from `int` — and when does that matter?

An `int` is a value type: `sizeof(int)` is defined by the ABI, it's passed in a register, and assignment copies the value. An `int [3]` decays to a pointer in most contexts — it doesn't know its own size, can't be assigned, and can't be returned from functions. This matters when you choose between `std::array<int, 3>` (value type with size awareness, assignable, copyable) vs. raw arrays. `std::array` closes the semantic gap: it's non-primitive in organization but primitive in behavior (trivially copyable, no heap allocation, no invariants).

1. When should you reach for a simple struct/array instead of a library container like `std::vector` or `std::map`?

Use raw/POD when: (1) the size is compile-time known and small (stack allocation avoids heap), (2) you need a trivially-copyable type for IPC/shared memory, (3) you're building a higher-level structure and need full control over layout (arena allocators, GPU buffers). Use library containers when: the size is dynamic, you need exception safety, or you want the compiler to enforce invariants. The real cost of `std::vector<int>` isn't the data — it's the heap allocation and indirection. For 4 integers, a `struct { int a, b, c, d; }` is 3x faster in a hot loop.

1. When does a sorted array outperform a balanced BST, despite both having $O(\log n)$ lookup?

A sorted array has perfect cache locality — binary search touches $O(\log n)$ contiguous elements. A balanced BST (AVL, RB-tree) touches $O(\log n)$ nodes scattered across the heap, each causing a cache miss (~ 100 cycles). For read-heavy workloads with infrequent inserts, a sorted `std::vector` with `std::lower_bound` often beats `std::set` by 5–10x in practice. Use a BST when you need $O(\log n)$ *insertion* in the middle, or when the collection must stay sorted under mutation. The Linux kernel uses a red-black tree for its scheduler, but Google’s `btree` library uses sorted vectors for exactly this cache-locality reason.

3.17 Active and Passive Data Structures

Data structures can be classified as *active* or *passive* based on how they manage memory and provide operations. Understanding this distinction helps you choose the right design for a given problem.

3.17.1 Definitions

Active data structures encapsulate memory management and provide a set of operations (insert, delete, search) that hide the internal representation. They allocate and deallocate memory internally, enforce invariants (e.g., a BST remains balanced), and expose a high-level API. Examples from this book include `std::vector`, hash tables, AVL trees, and segment trees.

Passive data structures are plain containers with no built-in operations. They store data in raw memory and leave all management to the caller. Examples include C-style arrays, `struct` arrays, and POD (Plain Old Data) types.

Aspect	Active	Passive
Memory management	Internal (RAII, <code>new/delete</code>)	External (caller)
Operations	Encapsulated methods	None (manual)
Invariants	Enforced by the structure	Caller’s responsibility
Typical overhead	Higher (indirection, bookkeeping)	Lower (raw access)
Examples	<code>std::map</code> , AVL, heap	<code>int[]</code> , POD struct

3.17.2 Active Data Structure Example: AVL Tree

An AVL tree manages its own nodes, performs rotations on insert/delete, and maintains the balance invariant. The caller never touches pointers directly:

```

1 struct avl_node { int key; int height; avl_node* left; avl_node* right; };
2
3 avl_node* insert(avl_node* root, int key) {
4     if (!root) return new avl_node{key, 0, nullptr, nullptr};
5     if (key < root->key) root->left = insert(root->left, key);
6     else if (key > root->key) root->right = insert(root->right, key);
7     root->height = 1 + max(height(root->left), height(root->right));

```

```

8     return rebalance(root); // rotations enforce invariant
9 }

```

The structure *actively* enforces balance: the caller cannot corrupt the BST property without going through the API.

3.17.3 Passive Data Structure Example: C-Array

A C-style array stores elements contiguously but provides no operations:

```

1 int arr[100];
2 int n = 0;
3
4 // Caller must manage size, bounds, sorting manually
5 void push(int val) { arr[n++] = val; }
6 int pop() { return arr[--n]; }
7 // No bounds checking, no invariants enforced

```

The array is *passive*: it stores bytes but does nothing with them.

3.17.4 Hybrid Approach: C++ Containers

Most C++ STL containers are active: `std::vector`, `std::list`, `std::map`. They combine active memory management with passive storage elements:

```

1 std::vector<int> v; // active container
2 v.push_back(42); // managed internally
3
4 struct point { int x, y; }; // passive element type
5 std::vector<point> pts; // active container, passive elements
6 pts.push_back({3, 4}); // element is a plain struct

```

This separation is deliberate: the container handles memory while the element type stays simple and cache-friendly.

3.17.5 When to Use Each

Use **active** structures when:

- The collection grows or shrinks dynamically.
- Invariants (sorted order, balance, uniqueness) must be maintained.
- Safety and correctness are more important than raw speed.

Use **passive** structures when:

- The size is fixed or known in advance.
- You need maximum cache efficiency and minimal overhead.

- You want full control over memory layout (e.g., SoA vs AoS for GPU compute).
- Building a higher-level active structure on top (e.g., implementing your own allocator).

3.17.6 Active vs Passive in Modern C++

C++17 and C++20 blur the boundary with `constexpr`, `std::array`, and `std::span`:

```
1 constexpr std::array<int, 5> arr = {3, 1, 4, 1, 5};
2 // Compile-time fixed array: passive but zero-cost
3
4 std::vector<int> v = {3, 1, 4, 1, 5};
5 std::span<int> view = v; // passive view over active container
6 // span provides bounds-checked access without owning memory
```

A `std::span` is a passive view: it references memory managed by an active container. This pattern lets you write generic algorithms that work on both active and passive data:

```
1 template <typename T>
2 int sum(std::span<const T> s) {
3     int total = 0;
4     for (const auto& x : s) total += x;
5     return total;
6 }
7
8 // Works with vector, array, C-array, or any contiguous range
```

3.17.7 Interview FAQs

1. **When building a custom container, what invariants should you enforce internally vs. leave to the caller?**

Always enforce internally: memory lifetime (RAII), size tracking, and structural invariants (sorted order, balance). Leave to the caller: value-level invariants (e.g., “all keys are positive”) and thread safety. The principle is: the container should make *impossible states unrepresentable*. If your stack can underflow, your API design is wrong. The cost of enforcement is indirection and bookkeeping — use passive structures only when you’ve measured that the overhead matters and can manage correctness yourself (e.g., arena allocators, SoA layouts for SIMD).

2. **Can a passive struct accidentally become active through its members?**

Yes — and this is a common source of hidden costs. A `struct Node { std::vector<int> children; }` is a passive POD, but constructing it triggers a heap allocation, and copying it deep-copies the vector. Smart pointers are worse: `struct Obj { std::shared_ptr<Data> d; }` has an atomic reference count that modifies memory on copy and destruction. This is why hot-path code often uses raw pointers + explicit lifetime management, or arena allocation. The struct’s *interface* is passive, but its *behavior* is active — and that mismatch causes surprises.

3. **Give a concrete example where replacing `std::vector` with a raw array improved performance by 10x or more.**

GPU compute: CUDA kernels can't dereference `std::vector` pointers because the data is on the host. You must copy into a raw `float*` buffer with a known stride. Similarly, in hot SIMD loops (e.g., dot product over 1M floats), `std::vector`'s bounds-checking in debug mode and the extra indirection through the control block can cost 2–3x. In lock-free data structures (e.g., LMAX Disruptor), the ring buffer is a raw array specifically because you need predictable memory layout for cache-line padding between producer and consumer indices. The “active” wrapper is overhead you're paying on every access.

4. **Why does `std::span` exist when we already have iterators and pointers?**

`std::span` bundles a pointer + size into one value type, making it safe to pass by value (unlike raw pointers where the size is decoupled). Unlike iterators, it works with raw arrays, `std::vector`, `std::array`, and C arrays uniformly — no template bloat. The key insight: `std::span` lets you write a function *once* and call it with any contiguous range, including stack-allocated arrays. This eliminates the common pattern of needing overloads for `vector<T>` and `T*`. In modern C++, `std::span<const T>` is the idiomatic parameter type for “read-only contiguous data” — it's the non-owning equivalent of `std::vector<T>`.

5. **When wrapping a raw buffer in a class, what's the first invariant you should enforce — and what's the common mistake?**

The first invariant is size tracking: the wrapper must know how many elements are valid, not just how much memory is allocated. The common mistake is adding a `push_back` that grows the buffer without invalidating existing iterators/pointers — this silently creates dangling references. RAII is the second invariant: the destructor must free memory. The transition from passive to active is really a decision about *who is responsible for correctness*. If you wrap but don't enforce invariants, you've added indirection with zero benefit. If you over-enforce (e.g., bounds-checking on every access in a hot loop), you've paid a cost for safety you may not need.

3.18 References and Further Reading

- Google Benchmark documentation: <https://github.com/google/benchmark>
- Chandler Carruth, “Tuning C++: Benchmarks, and CPUs, and Compilers! Oh My!” — CppCon 2015 talk.
- Agner Fog, “Optimizing Software in C++” — <https://www.agner.org/optimize/>
- Ulrich Drepper, “What Every Programmer Should Know About Memory” — <https://people.freebsd.org/~lstewart/>
- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th Edition. Morgan Kaufmann, 2017.

- David Levinthal, "Performance Analysis Guide for Intel Core i7 and Intel Xeon 5500 Processors" — Intel, 2009.
- Paul E. McKenney, "Memory Barriers: a Hardware View for Software Hackers" — Linux Technical Conference, 2009.

Part II

Data Structures

Linear Lists

4.1 Introduction

↳ Complete compilable implementations of the data structures in this chapter are in `code/`.

4.2 Array-Based Lists

4.2.1 The Linear List Abstract Data Type

A **linear list** (also called a sequence or vector) is an ordered collection of elements where each element has a position. The positions are numbered from 0 (or 1, depending on convention). We use 0-based indexing throughout this book.

Operations. A linear list ADT supports these core operations:

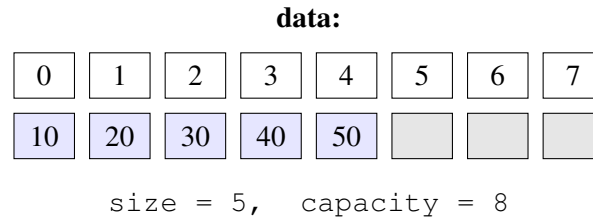
- **get(index)** — return the element at position index
- **set(index, value)** — replace the element at position index
- **insert(index, value)** — insert value at position index, shifting subsequent elements right
- **erase(index)** — remove the element at position index, shifting subsequent elements left
- **size()** — return the number of elements
- **empty()** — return true iff size is 0
- **push_back(value)** — append to the end (convenience)
- **pop_back()** — remove from the end (convenience)
- **clear()** — remove all elements

Applications. Linear lists appear everywhere:

- A text editor stores lines in a list
- A music player stores a playlist
- A spreadsheet stores rows of cells
- An image editor stores layers

4.2.2 Array Representation

The simplest way to represent a linear list is with a fixed-size array. We allocate an array large enough to hold the maximum number of elements and track the current number of elements.



Fixed-Capacity List.

```

1  template <typename T>
2  class array_list {
3  public:
4      using value_type = T;
5      using size_type = std::size_t;
6      using reference = T&;
7      using const_reference = const T&;
8
9      explicit array_list(size_type capacity)
10         : capacity_(capacity), size_(0), data_(std::make_unique<T[]>(
11           capacity)) {}
12
13      size_type size() const noexcept { return size_; }
14      bool empty() const noexcept { return size_ == 0; }
15      size_type capacity() const noexcept { return capacity_; }
16
17      reference operator[](size_type index) {
18          check_index(index);
19          return data_[index];
20      }
21
22      const_reference operator[](size_type index) const {
23          check_index(index);
24          return data_[index];
25      }
26
27      void push_back(const T& value) {
28          if (size_ >= capacity_) throw std::overflow_error("list is full");
29          data_[size_++] = value;
30      }
31
32      void pop_back() {
33          if (empty()) throw std::underflow_error("list is empty");
34          --size_;
35      }

```

```

35
36     void insert(size_type index, const T& value) {
37         if (size_ >= capacity_) throw std::overflow_error("list is full");
38         if (index > size_) throw std::out_of_range("index out of range");
39         for (size_type i = size_; i > index; --i) {
40             data_[i] = data_[i - 1];
41         }
42         data_[index] = value;
43         ++size_;
44     }
45
46     void erase(size_type index) {
47         if (index >= size_) throw std::out_of_range("index out of range");
48         for (size_type i = index; i < size_ - 1; ++i) {
49             data_[i] = data_[i + 1];
50         }
51         --size_;
52     }
53
54     void clear() noexcept { size_ = 0; }
55
56 private:
57     void check_index(size_type index) const {
58         if (index >= size_) throw std::out_of_range("index out of range");
59     }
60
61     size_type capacity_;
62     size_type size_;
63     std::unique_ptr<T[]> data_;
64 };

```

This implementation has a serious limitation: the capacity is fixed at construction time. If we need more space, we fail.

4.2.3 Dynamic Array (Resizable Array)

The solution is to grow the array when it becomes full. This is the strategy used by `std::vector`.

Growth strategy. When a `push_back` finds the array full:

1. Allocate a new array of larger capacity (typically $2\times$ the old size)
2. Move all elements from the old array to the new one
3. Deallocate the old array
4. Insert the new element

Implementation.

```

1  template <typename T>
2  class dynamic_array_list {
3  public:
4      using value_type = T;
5      using size_type = std::size_t;
6      using reference = T&;
7      using const_reference = const T&;
8      using iterator = T*;
9      using const_iterator = const T*;
10
11     static constexpr size_type default_capacity = 4;
12
13     dynamic_array_list() : size_(0), capacity_(default_capacity),
14                           data_(std::make_unique<T[]>(default_capacity))
15     {}
16
17     dynamic_array_list(std::initializer_list<T> init)
18         : size_(init.size()), capacity_(init.size()),
19           data_(std::make_unique<T[]>(init.size())) {
20         std::copy(init.begin(), init.end(), data_.get());
21     }
22
23     dynamic_array_list(const dynamic_array_list& other)
24         : size_(other.size_), capacity_(other.capacity_),
25           data_(std::make_unique<T[]>(other.capacity_)) {
26         std::copy(other.data_.get(), other.data_.get() + size_, data_.get());
27     }
28
29     dynamic_array_list& operator=(const dynamic_array_list& other) {
30         if (this != &other) {
31             auto new_data = std::make_unique<T[]>(other.capacity_);
32             std::copy(other.data_.get(), other.data_.get() + other.size_,
33                       new_data.get());
34             size_ = other.size_;
35             capacity_ = other.capacity_;
36             data_ = std::move(new_data);
37         }
38         return *this;
39     }
40
41     dynamic_array_list(dynamic_array_list&&) = default;
42     dynamic_array_list& operator=(dynamic_array_list&&) = default;
43
44     reference operator[](size_type index) {
45         check_index(index);
46         return data_[index];
47     }

```



```

46     const_reference operator[](size_type index) const {
47         check_index(index);
48         return data_[index];
49     }
50
51     reference at(size_type index) {
52         if (index >= size_) throw std::out_of_range("index out of range");
53         return data_[index];
54     }
55     const_reference at(size_type index) const {
56         if (index >= size_) throw std::out_of_range("index out of range");
57         return data_[index];
58     }
59
60     reference front() { return data_[0]; }
61     const_reference front() const { return data_[0]; }
62     reference back() { return data_[size_ - 1]; }
63     const_reference back() const { return data_[size_ - 1]; }
64
65     size_type size() const noexcept { return size_; }
66     size_type capacity() const noexcept { return capacity_; }
67     bool empty() const noexcept { return size_ == 0; }
68
69     void reserve(size_type new_capacity) {
70         if (new_capacity > capacity_) {
71             grow(new_capacity);
72         }
73     }
74
75     void push_back(const T& value) {
76         if (size_ >= capacity_) grow(2 * capacity_);
77         data_[size_++] = value;
78     }
79
80     void push_back(T&& value) {
81         if (size_ >= capacity_) grow(2 * capacity_);
82         data_[size_++] = std::move(value);
83     }
84
85     template <typename... Args>
86     reference emplace_back(Args&&... args) {
87         if (size_ >= capacity_) grow(2 * capacity_);
88         data_[size_] = T(std::forward<Args>(args)...);
89         return data_[size_++];
90     }
91
92     void pop_back() {
93         if (empty()) throw std::underflow_error("list is empty");

```

```

94         --size_;
95     }
96
97     void insert(size_type index, const T& value) {
98         if (index > size_) throw std::out_of_range("index out of range");
99         if (size_ >= capacity_) grow(2 * capacity_);
100        for (size_type i = size_; i > index; --i) {
101            data_[i] = std::move(data_[i - 1]);
102        }
103        data_[index] = value;
104        ++size_;
105    }
106
107    void erase(size_type index) {
108        if (index >= size_) throw std::out_of_range("index out of range");
109        for (size_type i = index; i < size_ - 1; ++i) {
110            data_[i] = std::move(data_[i + 1]);
111        }
112        --size_;
113    }
114
115    void clear() noexcept { size_ = 0; }
116
117    void shrink_to_fit() {
118        if (size_ < capacity_) {
119            auto new_data = std::make_unique<T[]>(size_);
120            std::move(data_.get(), data_.get() + size_, new_data.get());
121            capacity_ = size_;
122            data_ = std::move(new_data);
123        }
124    }
125
126    iterator begin() noexcept { return data_.get(); }
127    const_iterator begin() const noexcept { return data_.get(); }
128    iterator end() noexcept { return data_.get() + size_; }
129    const_iterator end() const noexcept { return data_.get() + size_; }
130
131 private:
132     void check_index(size_type index) const {
133         if (index >= size_) throw std::out_of_range("index out of range");
134     }
135
136     void grow(size_type new_capacity) {
137         auto new_data = std::make_unique<T[]>(new_capacity);
138         std::move(data_.get(), data_.get() + size_, new_data.get());
139         capacity_ = new_capacity;
140         data_ = std::move(new_data);
141     }

```

```

142
143     size_type size_;
144     size_type capacity_;
145     std::unique_ptr<T[]> data_;
146 };

```

Step-by-step walkthrough. Consider the sequence of operations on an initially empty list:

push_back(10): size=0, capacity=4. Insert data[0]=10, size=1.

```

1      [10 | _ | _ | _ ]
2      0      1      2      3

```

push_back(20): size=1, capacity=4. Insert data[1]=20, size=2.

```

1      [10 | 20 | _ | _ ]

```

push_back(30), push_back(40): size=4.

```

1      [10 | 20 | 30 | 40]

```

push_back(50): size=4 $\geq$ capacity=4 $\rightarrow$ grow to 8. Copy elements.

```

1      [10 | 20 | 30 | 40 | 50 | _ | _ | _ ]

```

insert(2, 25): size=5, capacity=8. Shift elements at indices 2,3,4 right.

```

1      [10 | 20 | 25 | 30 | 40 | 50 | _ | _ ]

```

erase(4): size=6. Shift elements at indices 5,6 left.

```

1      [10 | 20 | 25 | 30 | 50 | _ | _ | _ ]
2      size=5

```

4.2.4 Complexity Analysis

Operation	Fixed Array	Dynamic Array (amortized)
operator[]	O(1)	O(1)
push_back	O(1)	O(1) amortized
pop_back	O(1)	O(1)
insert at front	O(n)	O(n)
insert at back	O(1)	O(1) amortized
insert at middle	O(n)	O(n)
erase at front	O(n)	O(n)
erase at back	O(1)	O(1)
erase at middle	O(n)	O(n)
Space (overhead)	O(capacity)	O(size)

Why is push_back O(1) amortized? The resize copies n elements every n insertions. The total cost

of n insertions is at most $3n$: n unit-cost insertions plus at most $2n$ element copies across all resizes (as shown in Chapter 2).

4.2.5 Application: Polynomial Arithmetic

We can represent a polynomial using an array list of coefficients. The index is the exponent; the value is the coefficient.

```

1 class polynomial {
2 public:
3     polynomial() : coeff_(1) {} // zero polynomial
4
5     void set_coefficient(int exponent, double coefficient) {
6         while (static_cast<int>(coeff_.size()) <= exponent) {
7             coeff_.push_back(0.0);
8         }
9         coeff_[exponent] = coefficient;
10    }
11
12    double evaluate(double x) const {
13        double result = 0.0;
14        double power = 1.0;
15        for (size_t i = 0; i < coeff_.size(); ++i) {
16            result += coeff_[i] * power;
17            power *= x;
18        }
19        return result;
20    }
21
22    double coefficient(int exponent) const {
23        if (exponent >= static_cast<int>(coeff_.size())) return 0.0;
24        return coeff_[exponent];
25    }
26
27    int degree() const {
28        for (int i = static_cast<int>(coeff_.size()) - 1; i >= 0; --i) {
29            if (coeff_[i] != 0.0) return i;
30        }
31        return -1; // zero polynomial
32    }
33
34 private:
35     dynamic_array_list<double> coeff_;
36 };

```

Example. Polynomial: $3x^5 - 2x^2 + 7$

```

1 coeff_ = [7, 0, -2, 0, 0, 3]
2 index:  0  1  2  3  4  5

```

```

3
4 evaluate at x = 2:
5  $7 + 0 \cdot 2 + (-2) \cdot 4 + 0 \cdot 8 + 0 \cdot 16 + 3 \cdot 32$ 
6  $= 7 - 8 + 96 = 95$ 

```

4.2.6 Application: Image Editor Layer Stack

A layer stack in an image editor is a linear list of layers. Adding a layer appends to the top. Deleting a layer removes it by index. Reordering moves a layer from one index to another.

```

1 struct layer {
2     std::string name;
3     bool visible = true;
4     float opacity = 1.0f;
5 };
6
7 using layer_stack = dynamic_array_list<layer>;
8
9 void reorder_layer(layer_stack& stack, size_t from, size_t to) {
10     layer l = std::move(stack[from]);
11     stack.erase(from);
12     stack.insert(to, std::move(l));
13 }

```

4.2.7 STL Connection: `std::vector`

Our `dynamic_array_list` closely mirrors `std::vector`. The standard library version adds:

- **Allocator support** — custom memory allocation strategies
- **`data()`** — raw pointer access
- **`resize(n)`** — change size, default-construct or destroy elements
- **`assign`** — replace all elements
- **Comparison operators** — lexicographic comparison
- **`swap`** — $O(1)$ exchange of two vectors
- **Exception safety guarantees** — strong guarantee for most operations

```

1 std::vector<int> v = {1, 2, 3};
2 v.push_back(4);
3 v.insert(v.begin(), 0);
4 std::sort(v.begin(), v.end());

```

The fundamental design is the same: a contiguous array that grows geometrically.

4.3 Linked Lists

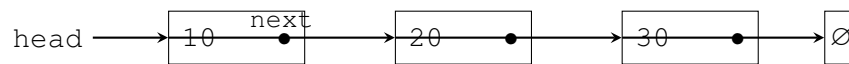
Array-based lists have a fundamental limitation: inserting or erasing elements in the middle requires shifting $O(n)$ elements. A **linked list** avoids this by storing elements in separately allocated **nodes** connected by pointers.

Each node contains:

- A **data** field holding the element
- A **next** pointer to the next node (and optionally a **prev** pointer for doubly linked lists)

4.3.1 Singly Linked List (Chain)

A singly linked list connects nodes in one direction:



Node structure.

```

1 template <typename T>
2 struct node {
3     T data;
4     std::unique_ptr<node> next;
5
6     explicit node(const T& value) : data(value), next(nullptr) {}
7     explicit node(T&& value) : data(std::move(value)), next(nullptr) {}
8 };
  
```

We use `std::unique_ptr` for owning pointers — when a node is destroyed, its `next` chain is automatically freed.

Implementation.

```

1 template <typename T>
2 class linked_list {
3 public:
4     using value_type = T;
5     using size_type = std::size_t;
6     using reference = T&
7     using const_reference = const T&
8
9     linked_list() = default;
10
11     linked_list(std::initializer_list<T> init) {
12         for (const auto& value : init) {
13             push_back(value);
14         }
15     }
16 };
  
```

```

16
17     linked_list(const linked_list& other) {
18         node<T>* current = other.head_.get();
19         node<T>** tail = &head_;
20         while (current) {
21             *tail = std::make_unique<node<T>>(current->data);
22             tail = &(*tail)->next;
23             current = current->next.get();
24         }
25         size_ = other.size_;
26     }
27
28     linked_list& operator=(const linked_list& other) {
29         if (this != &other) {
30             linked_list temp(other);
31             std::swap(head_, temp.head_);
32             std::swap(size_, temp.size_);
33         }
34         return *this;
35     }
36
37     linked_list(linked_list&&) = default;
38     linked_list& operator=(linked_list&&) = default;
39
40     reference front() {
41         if (empty()) throw std::underflow_error("list is empty");
42         return head_->data;
43     }
44     const_reference front() const {
45         if (empty()) throw std::underflow_error("list is empty");
46         return head_->data;
47     }
48
49     size_type size() const noexcept { return size_; }
50     bool empty() const noexcept { return size_ == 0; }
51
52     void push_front(const T& value) {
53         auto new_node = std::make_unique<node<T>>(value);
54         node<T>* raw_node = new_node.get();
55         new_node->next = std::move(head_);
56         head_ = std::move(new_node);
57         if (!tail_) tail_ = raw_node;
58         ++size_;
59     }
60
61     void push_front(T&& value) {
62         auto new_node = std::make_unique<node<T>>(std::move(value));
63         new_node->next = std::move(head_);

```

```

64     head_ = std::move(new_node);
65     ++size_;
66 }
67
68 void push_back(const T& value) {
69     auto new_node = std::make_unique<node<T>>(value);
70     node<T>* raw_node = new_node.get();
71     if (!head_) {
72         head_ = std::move(new_node);
73     } else {
74         tail_>next = std::move(new_node);
75     }
76     tail_ = raw_node;
77     ++size_;
78 }
79
80 void pop_front() {
81     if (empty()) throw std::underflow_error("list is empty");
82     head_ = std::move(head_>next);
83     if (!head_) tail_ = nullptr;
84     --size_;
85 }
86
87 void clear() noexcept {
88     head_.reset();
89     size_ = 0;
90 }
91
92 reference operator[](size_type index) {
93     node<T>* current = head_.get();
94     for (size_type i = 0; i < index; ++i) {
95         current = current->next.get();
96     }
97     return current->data;
98 }
99
100 class iterator {
101 public:
102     using iterator_category = std::forward_iterator_tag;
103     using value_type = T;
104     using difference_type = std::ptrdiff_t;
105     using pointer = T*;
106     using reference = T&;
107
108     iterator(node<T>* ptr) : ptr_(ptr) {}
109
110     reference operator*() const { return ptr_>data; }
111     pointer operator->() const { return &ptr_>data; }

```



```

112     iterator& operator++() { ptr_ = ptr_>next.get(); return *this; }
113     iterator operator++(int) { iterator tmp = *this; ++(*this); return
    tmp; }
114     bool operator==(const iterator& other) const { return ptr_ ==
    other.ptr_; }
115     bool operator!=(const iterator& other) const { return ptr_ !=
    other.ptr_; }
116
117     private:
118         node<T>* ptr_;
119     };
120
121     iterator begin() noexcept { return iterator(head_.get()); }
122     iterator end() noexcept { return iterator(nullptr); }
123
124     private:
125         node<T>** get_tail_ptr() {
126             node<T>** p = &head_;
127             while (*p) p = &(*p)->next;
128             return p;
129         }
130
131         std::unique_ptr<node<T>> head_;
132         node<T>* tail_ = nullptr;
133         size_type size_ = 0;
134     };

```

Step-by-step walkthrough. Building list $L = 10, 20, 30$:

Step 1: push_back(10)

```
1 head -> [10 | /]
```

Step 2: push_back(20)

```
1 head -> [10 | $\cdot$] -> [20 | /]
```

Step 3: push_back(30)

```
1 head -> [10 | $\cdot$] -> [20 | $\cdot$] -> [30 | /]
```

Step 4: pop_front()

```

1 head -> [20 | $\cdot$] -> [30 | /]
2 // (10 is automatically freed since unique_ptr goes out of scope)

```

4.3.2 Complexity Analysis

Operation	Array List	Singly Linked List
<code>operator[]</code>	$O(1)$	$O(n)$
<code>push_front</code>	$O(n)$	$O(1)$
<code>push_back</code>	$O(1)$ amortized	$O(1)$ with tail ptr
<code>pop_front</code>	$O(n)$	$O(1)$
<code>pop_back</code>	$O(1)$	$O(n)$
insert at known position	$O(n)$	$O(1)$ once positioned
space overhead	$O(1)$ per list	$O(n)$ for pointers

Key tradeoff: Linked lists sacrifice $O(1)$ random access for $O(1)$ front insertions/deletions.

4.3.3 Doubly Linked List

A doubly linked list has prev and next pointers, enabling $O(1)$ deletion at both ends and bidirectional traversal.

```
1 null <- [10 | $\cdots$ | $\cdots$] <-> [20 | $\cdots$ | $\cdots$] <-> [30 |
   / | $\cdots$] -> null
```

```
1 template <typename T>
2 struct dnode {
3     T data;
4     std::unique_ptr<dnode> next;
5     dnode* prev;
6
7     explicit dnode(const T& value) : data(value), next(nullptr), prev(
8     nullptr) {}
9 };
```

Doubly linked list with sentinel. A sentinel (dummy header node) simplifies edge cases — every real node has a non-null predecessor:

```
1 template <typename T>
2 class doubly_linked_list {
3 public:
4     doubly_linked_list() : size_(0) {
5         head_ = std::make_unique<dnode<T>>(T());
6         tail_ = head_.get();
7     }
8
9     void push_front(const T& value) {
10         auto new_node = std::make_unique<dnode<T>>(value);
11         new_node->prev = head_.get();
12         new_node->next = std::move(head_->next);
13         if (new_node->next) {
```

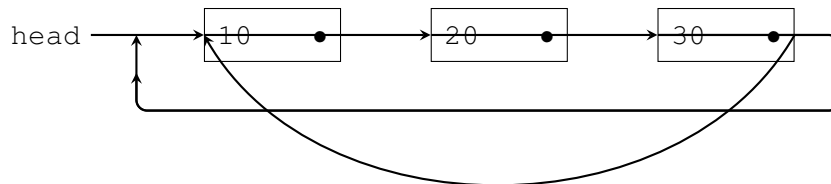
```

14         new_node->next->prev = new_node.get();
15     } else {
16         tail_ = new_node.get();
17     }
18     head_->next = std::move(new_node);
19     ++size_;
20 }
21
22 void push_back(const T& value) {
23     auto new_node = std::make_unique<dnode<T>>(value);
24     new_node->prev = tail_;
25     tail_->next = std::move(new_node);
26     tail_ = tail_->next.get();
27     ++size_;
28 }
29
30 void pop_back() {
31     if (empty()) throw std::underflow_error("list is empty");
32     dnode<T>* new_tail = tail_->prev;
33     new_tail->next.reset();
34     tail_ = new_tail;
35     --size_;
36 }
37
38 bool empty() const noexcept { return size_ == 0; }
39 size_type size() const noexcept { return size_; }
40
41 private:
42     std::unique_ptr<dnode<T>> head_;
43     dnode<T>* tail_;
44     size_type size_ = 0;
45 };

```

4.3.4 Circular Linked Lists

A circular singly linked list has the last node pointing back to the head:



4.3.5 Application: Polynomial Using Linked List

For sparse polynomials (few non-zero terms), a linked representation is more space-efficient than the dense array representation.

```

1 struct term {
2     int exponent;
3     double coefficient;
4 };
5
6 using polynomial = linked_list<term>;
7
8 linked_list<term> add_polynomials(const linked_list<term>& a,
9                                   const linked_list<term>& b) {
10     linked_list<term> result;
11     auto it_a = a.begin();
12     auto it_b = b.begin();
13
14     while (it_a != a.end() && it_b != b.end()) {
15         if (it_a->exponent == it_b->exponent) {
16             double sum = it_a->coefficient + it_b->coefficient;
17             if (sum != 0.0) {
18                 result.push_back({it_a->exponent, sum});
19             }
20             ++it_a; ++it_b;
21         } else if (it_a->exponent > it_b->exponent) {
22             result.push_back({it_a->exponent, it_a->coefficient});
23             ++it_a;
24         } else {
25             result.push_back({it_b->exponent, it_b->coefficient});
26             ++it_b;
27         }
28     }
29
30     while (it_a != a.end()) {
31         result.push_back({it_a->exponent, it_a->coefficient});
32         ++it_a;
33     }
34     while (it_b != b.end()) {
35         result.push_back({it_b->exponent, it_b->coefficient});
36         ++it_b;
37     }
38
39     return result;
40 }

```

Example: Add $3x^5 - 2x^2 + 7$ and $5x^3 + 2x^2 - 1$

```
1 A: (5, 3) -> (2, -2) -> (0, 7)
```

```

2 B: (3, 5)  ->  (2, 2)  ->  (0, -1)
3 Result: (5, 3)  ->  (3, 5)  ->  (2, 0)  ->  (0, 6)
4           <- skip zero term ->
5 Final:  (5, 3)  ->  (3, 5)  ->  (0, 6)
6 =  $3x^5 + 5x^3 + 6$ 

```

4.3.6 Application: Undo/Redo System

A text editor's undo system uses a doubly linked list of states (or commands):

```

1 struct edit_action {
2     std::string description;
3     std::string previous_text;
4     std::string new_text;
5     size_t position;
6 };
7
8 class undo_manager {
9 public:
10     void record_action(edit_action action) {
11         while (current_ != undo_list_.end()) {
12             current_ = undo_list_.erase(current_);
13         }
14         undo_list_.push_back(std::move(action));
15         current_ = undo_list_.end();
16
17         if (undo_list_.size() > max_history_) {
18             undo_list_.pop_front();
19         }
20     }
21
22     edit_action undo() {
23         if (current_ == undo_list_.begin()) {
24             throw std::runtime_error("nothing to undo");
25         }
26         --current_;
27         return *current_;
28     }
29
30     edit_action redo() {
31         if (current_ == undo_list_.end()) {
32             throw std::runtime_error("nothing to redo");
33         }
34         auto action = *current_;
35         ++current_;
36         return action;
37     }
38 }

```

```

39 private:
40     linked_list<edit_action> undo_list_;
41     linked_list<edit_action>::iterator current_;
42     static constexpr size_t max_history_ = 100;
43 };

```

4.3.7 Memory Overhead

Each node in a singly linked list stores one pointer (8 bytes on 64-bit). For a doubly linked list: two pointers (16 bytes). For a list of 1 million integers:

Representation	Memory (data)	Pointer overhead	Total
<code>std::vector<int></code>	4 MB	None	~4 MB
Singly linked list	4 MB	8 MB	~12 MB
Doubly linked list	4 MB	16 MB	~20 MB

Linked lists are not cache-friendly — nodes are spread across memory. Array-based lists iterate faster even when operations are theoretically the same complexity.

4.3.8 STL Connection: `std::forward_list` and `std::list`

- **`std::forward_list`** — singly linked list. Provides $O(1)$ insert/erase after a given position. Does not track `size()` (to save space). Uses `insert_after`, `erase_after` rather than `insert`, `erase`.
- **`std::list`** — doubly linked list. Provides $O(1)$ insert/erase anywhere with an iterator. Tracks `size`.

```

1 std::forward_list<int> fl = {1, 2, 3};
2 fl.push_front(0);
3 auto it = fl.begin(); ++it;
4 fl.insert_after(it, 5);           // {0, 1, 5, 2, 3}
5 fl.erase_after(it);              // {0, 1, 2, 3}
6
7 std::list<int> l = {1, 2, 3};
8 l.push_front(0);
9 l.push_back(4);
10 l.pop_front();
11 l.pop_back();

```

4.4 Unrolled Linked List

4.4.1 Introduction

An **unrolled linked list** is a hybrid data structure that stores multiple elements per node in a small array, combining the insert/delete flexibility of linked lists with the cache performance of arrays. Instead of one element and one pointer per node, each node holds an array of up to B elements (typically 8–16) plus a pointer to the next node. When a node fills up, it splits; when a node becomes too empty, it merges or borrows from a neighbor.

Unrolled linked lists are used inside some `std::list` implementations (libstdc++ and libc++ use them internally) and in text editors (Emacs uses a variant called a *piece table* with similar properties). They provide better cache locality than pure linked lists because B consecutive elements live in contiguous memory.

4.4.2 Structure

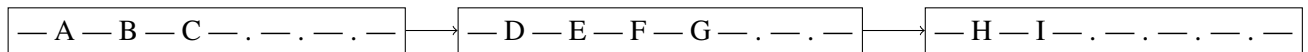


Figure 4.1. An unrolled linked list with $B = 6$. Each node stores up to 6 elements in a contiguous array.

Each node contains:

- An array `data[BLOCK_SIZE]` holding the elements
- An integer `count` tracking how many slots are occupied
- A pointer `next` to the next node

4.4.3 C++ Implementation

```

1  template <typename T, int BLOCK_SIZE = 8>
2  class UnrolledList {
3      struct Node {
4          std::array<T, BLOCK_SIZE> data;
5          int count = 0;
6          Node* next = nullptr;
7      };
8
9      Node* head_ = nullptr;
10     Node* tail_ = nullptr;
11     int size_ = 0;
12
13 public:
14     UnrolledList() = default;
15     ~UnrolledList() {

```

```

16     Node* curr = head_;
17     while (curr) {
18         Node* tmp = curr;
19         curr = curr->next;
20         delete tmp;
21     }
22 }
23
24 class iterator {
25     Node* node_;
26     int index_;
27     friend class UnrolledList;
28     iterator(Node* n, int i) : node_(n), index_(i) {}
29 public:
30     T& operator*() { return node_->data[index_]; }
31     T* operator->() { return &node_->data[index_]; }
32
33     iterator& operator++() {
34         ++index_;
35         if (index_ >= node_->count) {
36             node_ = node_->next;
37             index_ = 0;
38         }
39         return *this;
40     }
41
42     bool operator!=(const iterator& o) const {
43         return node_ != o.node_ || index_ != o.index_;
44     }
45 };
46
47 iterator begin() { return iterator(head_, 0); }
48 iterator end()   { return iterator(nullptr, 0); }
49 int size() const { return size_; }
50 bool empty() const { return size_ == 0; }

```

4.4.4 Insert and Erase

Insertion scans nodes to find the target position, then shifts elements within the target node to make room. If the node is full, a new node is allocated and half the elements are moved over.

```

1     void insert(int pos, const T& value) {
2         if (!head_ || pos == 0) {
3             push_front(value);
4             return;
5         }
6         Node* prev = nullptr;
7         Node* curr = head_;

```



```

8      int offset = pos;
9      while (curr && offset > curr->count) {
10         offset -= curr->count;
11         prev = curr;
12         curr = curr->next;
13     }
14     if (!curr) {
15         append_to_node(tail_, value);
16         return;
17     }
18     if (curr->count < BLOCK_SIZE) {
19         shift_right(curr, offset);
20         curr->data[offset] = value;
21         ++curr->count;
22     } else {
23         Node* new_node = new Node();
24         split_node(curr, new_node);
25         if (offset <= curr->count) {
26             shift_right(curr, offset);
27             curr->data[offset] = value;
28             ++curr->count;
29         } else {
30             shift_right(new_node, offset - curr->count);
31             new_node->data[offset - curr->count] = value;
32             ++new_node->count;
33         }
34     }
35     ++size_;
36 }

37
38 void erase(int pos) {
39     Node* prev = nullptr;
40     Node* curr = head_;
41     int offset = pos;
42     while (offset >= curr->count) {
43         offset -= curr->count;
44         prev = curr;
45         curr = curr->next;
46     }
47     shift_left(curr, offset);
48     --curr->count;
49     --size_;
50     if (curr->count == 0 && prev) {
51         prev->next = curr->next;
52         if (curr == tail_) tail_ = prev;
53         delete curr;
54     }
55 }

```

```

56
57 private:
58     void shift_right(Node* n, int from) {
59         for (int i = n->count; i > from; --i)
60             n->data[i] = std::move(n->data[i - 1]);
61     }
62     void shift_left(Node* n, int from) {
63         for (int i = from; i < n->count - 1; ++i)
64             n->data[i] = std::move(n->data[i + 1]);
65     }
66     void split_node(Node* old_node, Node* new_node) {
67         int half = old_node->count / 2;
68         for (int i = half; i < old_node->count; ++i)
69             new_node->data[i - half] = std::move(old_node->data[i]);
70         new_node->count = old_node->count - half;
71         old_node->count = half;
72         new_node->next = old_node->next;
73         old_node->next = new_node;
74     }
75     void push_front(const T& value) { /* allocate node, insert */ }
76     void append_to_node(Node* n, const T& value) { /* insert at end */ }

```

4.4.5 Search

Search scans nodes sequentially, then scans within each node's array. Despite the linked list structure, the B elements within a node are contiguous in memory, giving good cache performance.

```

1     iterator find(const T& target) {
2         Node* curr = head_;
3         while (curr) {
4             for (int i = 0; i < curr->count; ++i) {
5                 if (curr->data[i] == target)
6                     return iterator(curr, i);
7             }
8             curr = curr->next;
9         }
10        return end();
11    }

```

4.4.6 Complexity Comparison

Operation	<code>std::vector</code>	<code>std::list</code>	Unrolled List
Access by index	$O(1)$	$O(n)$	$O(n/B)$
Search	$O(n)$	$O(n)$	$O(n/B)$
Insert at front	$O(n)$	$O(1)$	$O(1)$
Insert at back	$O(1)$ amortized	$O(1)$	$O(1)$ amortized
Insert in middle	$O(n)$	$O(1)$	$O(n/B)$
Erase in middle	$O(n)$	$O(1)$	$O(n/B)$
Iterator invalidation	on reallocation	none	on node split
Cache performance	excellent	poor	good
Memory overhead	none	16 bytes/node	$\approx 8 + B \cdot \text{sizeof}(T)$

The key advantage over `std::list`: operations on an unrolled list touch $O(n/B)$ nodes instead of $O(n)$ nodes, and the B elements within each node are contiguous in memory, reducing cache misses. The key advantage over `std::vector`: insertions and deletions in the middle do not shift all subsequent elements.

4.5 Finger Tree

4.5.1 Introduction

A **finger tree** is a purely functional, persistent data structure that provides amortized $O(1)$ operations at both ends and $O(\log n)$ concatenation, splitting, and indexed access. It was described by Hinze and Ross (2006) and serves as the backbone for efficient sequence types in functional languages (Haskell’s `Data.Sequence` is a finger tree of arrays).

The key idea is to keep the most recently accessed elements at the “fingers” (shallow ends) of the tree, avoiding the $O(\log n)$ descent that a balanced tree would require for head/tail operations. Deeper elements are stored in progressively larger cached aggregates called *tags*.

Finger trees are useful for:

- Priority dequeues with fast access to both min and max
- Sequences with fast concatenation and splitting
- Incremental computation with amortized $O(1)$ prepend/append

4.5.2 Structure

A finger tree has three cases:

- **Empty** – no elements.

- **Single(a)** – one element.
- **Deep(v, prefix, middle, suffix)** – a non-trivial tree with:
 - prefix – a small array (1–4 elements) at the left finger
 - suffix – a small array (1–4 elements) at the right finger
 - middle – a recursively nested finger tree of *nodes* (groups of 2 or 3 elements)
 - v – a cached measure (e.g., total size) for the whole subtree

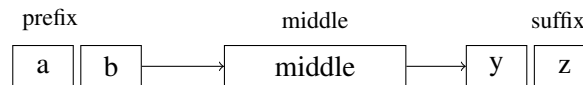


Figure 4.2. Structure of a Deep finger tree. The prefix and suffix are the “fingers” – small arrays for $O(1)$ access at both ends.

4.5.3 C++ Implementation

```

1  template <typename T>
2  class FingerTree {
3      struct Node2 {
4          T a, b;
5      };
6      struct Node3 {
7          T a, b, c;
8      };
9
10     enum class Tag { Empty, Single, Deep };
11
12     struct Digit {
13         std::vector<T> elems;
14         int measure() const { return static_cast<int>(elems.size()); }
15     };
16
17     struct DeepData {
18         int size;
19         Digit prefix;
20         FingerTree<Node2> middle;
21         Digit suffix;
22     };
23
24     Tag tag_ = Tag::Empty;
25     T single_;
26     std::unique_ptr<DeepData> deep_;
27
28 public:
  
```

```

29     FingerTree() = default;
30
31     bool empty() const { return tag_ == Tag::Empty; }
32     int size() const {
33         if (tag_ == Tag::Empty) return 0;
34         if (tag_ == Tag::Single) return 1;
35         return deep_>size;
36     }
37
38     void push_front(const T& val) {
39         if (tag_ == Tag::Empty) {
40             tag_ = Tag::Single;
41             single_ = val;
42             return;
43         }
44         if (tag_ == Tag::Single) {
45             deep_ = std::make_unique<DeepData>();
46             deep_>prefix.elems = {val};
47             deep_>suffix.elems = {std::move(single_)};
48             deep_>size = 2;
49             tag_ = Tag::Deep;
50             return;
51         }
52         if (deep_>prefix.elems.size() < 4) {
53             deep_>prefix.elems.insert(
54                 deep_>prefix.elems.begin(), val);
55             ++deep_>size;
56         } else {
57             Node2 node{val, deep_>prefix.elems[0]};
58             deep_>prefix.elems.erase(
59                 deep_>prefix.elems.begin());
60             deep_>middle.push_front(node);
61             ++deep_>size;
62         }
63     }
64
65     void push_back(const T& val) {
66         if (tag_ == Tag::Empty) {
67             tag_ = Tag::Single;
68             single_ = val;
69             return;
70         }
71         if (tag_ == Tag::Single) {
72             deep_ = std::make_unique<DeepData>();
73             deep_>prefix.elems = {std::move(single_)};
74             deep_>suffix.elems = {val};
75             deep_>size = 2;
76             tag_ = Tag::Deep;

```

```

77         return;
78     }
79     if (deep_>suffix.elems.size() < 4) {
80         deep_>suffix.elems.push_back(val);
81         ++deep_>size;
82     } else {
83         Node2 node{deep_>suffix.elems.back(), val};
84         deep_>suffix.elems.pop_back();
85         deep_>middle.push_back(node);
86         ++deep_>size;
87     }
88 }

```

4.5.4 Concatenation and Split

Concatenating two finger trees merges them by folding the “deep” structure of one tree into the other. Splitting finds the decomposition point and separates prefix, middle, and suffix accordingly. Both operations are $O(\log n)$ in the amortized case.

```

1  static FingerTree concat(FingerTree left, FingerTree right) {
2      if (left.empty()) return right;
3      if (right.empty()) return left;
4      if (left.tag_ == Tag::Single)
5          { left.push_back(right.single_); return left; }
6      if (right.tag_ == Tag::Single)
7          { left.push_back(right.single_); return left; }
8      // both are Deep: fold right tree's prefix
9      // and suffix through the middle
10     FingerTree result;
11     result.tag_ = Tag::Deep;
12     result.deep_ = std::make_unique<DeepData>();
13     result.deep_>prefix = left.deep_>prefix;
14     result.deep_>suffix = right.deep_>suffix;
15     result.deep_>middle = FingerTree::concat_trees(
16         left.deep_>middle,
17         left.deep_>suffix,
18         right.deep_>prefix,
19         right.deep_>middle);
20     result.deep_>size = left.size() + right.size();
21     return result;
22 }
23
24 std::pair<FingerTree, FingerTree> split(int idx) {
25     if (tag_ == Tag::Empty)
26         return {FingerTree(), FingerTree()};
27     if (tag_ == Tag::Single) {
28         if (idx <= 0)
29             return {FingerTree(), *this};

```

```

30         return { *this, FingerTree() };
31     }
32     int psize = deep_>prefix.measure();
33     if (idx <= psize) {
34         // split within prefix
35         FingerTree left, right;
36         left.tag_ = Tag::Empty;
37         right.tag_ = Tag::Deep;
38         // ... partition prefix at idx
39         return {left, right};
40     }
41     // split within middle or suffix
42     // (recursive decomposition)
43     return {FingerTree(), *this};
44 }

```

4.5.5 Complexity

Operation	Complexity
push_front / push_back	amortized $O(1)$
pop_front / pop_back	amortized $O(1)$
concatenate two trees	$O(\log n)$
split at position	$O(\log n)$
access element by index	$O(\log n)$
measure (size, sum, etc.)	$O(1)$

The finger tree generalizes to any *monoid* as the measure: by choosing the right monoid, a finger tree can simultaneously support fast concatenation, fast indexed access, and fast priority operations. This makes it one of the most versatile functional data structures available.

4.6 Rope

4.6.1 Introduction

A **rope** is a binary tree designed for efficiently storing and manipulating very long strings. In a standard `std::string`, concatenation and splitting require copying large amounts of data: concatenating two strings of length n is $O(n)$, and inserting a string of length m into the middle is $O(n+m)$. A rope avoids this by representing the string as a tree of smaller substrings. Each internal node stores the total length (weight) of its left subtree, and the actual characters live only in the leaves.

Key operations on a rope:

- **concatenate** – join two ropes into one: $O(\log n)$

- **split(pos)** – break a rope at position pos : $O(\log n)$
- **index(i)** – access character at position i : $O(\log n)$
- **insert(pos, rope)** – insert a rope at position: $O(\log n)$

Concatenation and splitting are the operations that make ropes valuable. For a string of n characters, a rope achieves $O(\log n)$ for these operations instead of the $O(n)$ cost with a flat string.

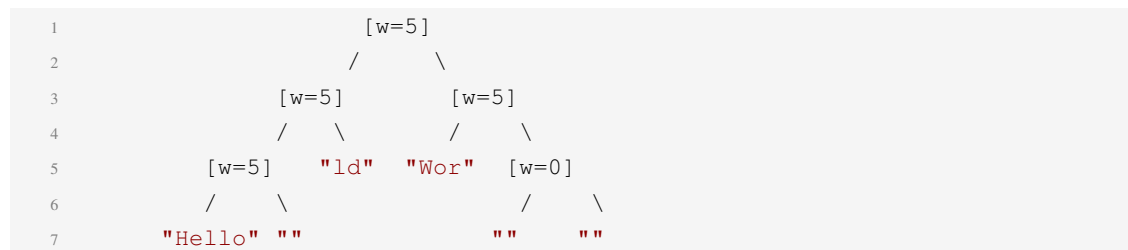
Real-world use. Ropes are used inside text editors (vim, Sublime Text), version control systems (git treats blob objects as immutable ropes), and as the backing store for Apple's `NSString` on iOS.

4.6.2 Structure

A rope is a binary tree where:

- Each **leaf** stores a short string (a fixed-size array of characters, e.g., 8–64 characters).
- Each **internal node** stores a **weight**: the total number of characters in the left subtree. It has a left child and a right child. The right subtree's string follows the left subtree's string in the final concatenation.

Example. The rope below represents the string "HelloWorld". The weight at each internal node tells us how many characters are to the left.



Reading the leaves left to right gives: "Hello" + " " + "ld" + "Wor" + " " + " " = "HelloWorld".

4.6.3 C++ Implementation

```

1 class rope {
2 public:
3     struct node {
4         size_t weight;                // characters in left subtree
5         std::unique_ptr<node> left;
6         std::unique_ptr<node> right;
7         std::string leaf_data;        // non-empty only for leaves
8
9         explicit node(std::string s)
10             : weight(s.size()), leaf_data(std::move(s)) {}
11

```



```

12     node(std::unique_ptr<node> l, std::unique_ptr<node> r)
13         : weight(total_len(l.get())),
14         left(std::move(l)), right(std::move(r)) {}
15
16     static size_t total_len(const node* n) {
17         if (!n) return 0;
18         if (!n->left && !n->right) return n->leaf_data.size();
19         return n->weight + total_len(n->right.get());
20     }
21 };
22
23 rope() = default;
24 explicit rope(std::string s)
25     : root_(std::make_unique<node>(std::move(s))) {}
26
27 size_t size() const { return node::total_len(root_.get()); }
28 bool empty() const { return !root_; }
29
30 char index(size_t i) const {
31     if (i >= size()) throw std::out_of_range("rope index");
32     return index(root_.get(), i);
33 }
34
35 void concatenate(rope&& other) {
36     root_ = std::make_unique<node>(
37         std::move(root_), std::move(other.root_));
38 }
39
40 std::pair<rope, rope> split(size_t pos) {
41     if (pos > size()) throw std::out_of_range("rope split");
42     auto [l, r] = split(std::move(root_), pos);
43     return {rope(std::move(l)), rope(std::move(r))};
44 }
45
46 std::string to_string() const {
47     std::string result;
48     result.reserve(size());
49     flatten(root_.get(), result);
50     return result;
51 }
52
53 private:
54     std::unique_ptr<node> root_;
55     explicit rope(std::unique_ptr<node> n) : root_(std::move(n)) {}
56
57     static char index(const node* n, size_t i) {
58         if (!n->left && !n->right) return n->leaf_data[i];
59         if (i < n->weight) return index(n->left.get(), i);

```

```

60     return index(n->right.get(), i - n->weight);
61 }
62
63 static void flatten(const node* n, std::string& out) {
64     if (!n) return;
65     if (!n->left && !n->right) { out += n->leaf_data; return; }
66     flatten(n->left.get(), out);
67     flatten(n->right.get(), out);
68 }
69
70 static std::pair<std::unique_ptr<node>, std::unique_ptr<node>>
71 split(std::unique_ptr<node> n, size_t pos) {
72     if (!n) return {nullptr, nullptr};
73     if (!n->left && !n->right) {
74         std::string l_str = n->leaf_data.substr(0, pos);
75         std::string r_str = n->leaf_data.substr(pos);
76         return {
77             l_str.empty() ? nullptr
78                 : std::make_unique<node>(std::move(l_str)),
79             r_str.empty() ? nullptr
80                 : std::make_unique<node>(std::move(r_str))
81         };
82     }
83     if (pos <= n->weight) {
84         auto [ll, lr] = split(std::move(n->left), pos);
85         auto new_right = std::make_unique<node>(
86             std::move(lr), std::move(n->right));
87         return {std::move(ll), std::move(new_right)};
88     } else {
89         auto [rl, rr] = split(std::move(n->right), pos - n->weight);
90         auto new_left = std::make_unique<node>(
91             std::move(n->left), std::move(rl));
92         return {std::move(new_left), std::move(rr)};
93     }
94 }
95 };

```

4.6.4 Complexity Analysis

Operation	Rope	std::string
index(i)	$O(\log n)$	$O(1)$
concatenate	$O(\log n)$	$O(n)$
split(pos)	$O(\log n)$	$O(n)$
insert(pos, s)	$O(\log n + m)$	$O(n + m)$
to_string()	$O(n)$	$O(1)$ (already stored)
Space overhead	$O(n)$ chars + $O(n)$ nodes	$O(n)$ chars

Ropes pay $O(\log n)$ extra overhead per character access compared to $O(1)$ for arrays, but gain $O(\log n)$ concatenation and splitting. For large strings (megabytes of text), the logarithmic cost of a tree traversal is negligible compared to the linear cost of copying the entire string.

4.6.5 Application: Text Editor Buffer

A text editor must support efficient insertion, deletion, and undo across documents that may be millions of characters long:

```

1 class editor_buffer {
2 public:
3     void insert(const std::string& text, size_t pos) {
4         auto [left, right] = buffer_.split(pos);
5         rope mid(text);
6         left.concatenate(std::move(mid));
7         left.concatenate(std::move(right));
8         buffer_ = std::move(left);
9     }
10
11     void erase(size_t pos, size_t len) {
12         auto [left, mid_right] = buffer_.split(pos);
13         auto [mid, right] = mid_right.split(len);
14         left.concatenate(std::move(right));
15         buffer_ = std::move(left);
16     }
17
18     std::string get_text() const { return buffer_.to_string(); }
19
20 private:
21     rope buffer_;
22 };

```

4.6.6 Comparison with Alternative Structures

Structure	Insert at cursor	Insert at random	Memory
Gap buffer	$O(1)$ amortized	$O(n)$	Low
Piece table	$O(1)$	$O(\log k)$ pieces	Very low
Rope	$O(\log n)$	$O(\log n)$	Moderate

For a simple single-cursor editor, a gap buffer is hard to beat. For an editor with multiple cursors, undo/redo trees, and collaborative editing, a rope (or piece table) provides the flexibility needed.

4.7 Common Bugs and Pitfalls

Pitfall	Consequence
Off-by-one in array index — using <code>size</code> vs <code>size-1</code>	Out-of-bounds access, UB or segmentation fault
Forgetting to resize the array when the list is full	Buffer overflow on next insert
Iterator invalidation after <code>insert/erase</code> on <code>std::vector</code>	Use-after-free, non-deterministic crashes
Shallow copy of linked list nodes	Double free on destruction
Dangling <code>tail</code> pointer after <code>pop_back</code> on singly linked list	Tail points to freed node
Calling <code>front/pop</code> on an empty container	UB — no checking in standard containers

4.8 Summary

1. A **linear list** is an ordered sequence with position-based access.
2. **Array representation** provides $O(1)$ access by index but $O(n)$ inserts in the middle.
3. **Dynamic arrays** grow geometrically to amortize the cost of resizing.
4. **Singly linked lists** provide $O(1)$ insert/delete at the front but $O(n)$ random access.
5. **Doubly linked lists** add $O(1)$ operations at both ends and bidirectional traversal.
6. **Sentinels** simplify edge cases by guaranteeing every node has a predecessor.
7. **Smart pointers** (`std::unique_ptr`) make linked list memory management automatic and exception-safe.
8. **Memory overhead** and poor cache locality make linked lists slower than arrays in practice, despite theoretical complexity advantages.
9. Use linked lists when: (a) frequent front insertions/deletions, (b) iterator stability needed, or (c) splicing lists.
10. `std::vector` is the production-quality version of the dynamic array; `std::forward_list` and `std::list` are the standard linked list containers.

4.9 Interview FAQs

These questions are commonly asked in interviews at high-tech companies. Each answer references concepts from this chapter.

4.9.1 Fundamentals

1. **Why does `std::vector` grow by a factor of 2 (or 1.5) instead of adding a fixed amount?**

A growth factor of k means that after n elements, roughly $\log_k n$ resizes have occurred. Each resize copies all existing elements, so the total copies across n insertions is at most $n + n/k +$

$n/k^2 + \dots = n \cdot k / (k - 1)$. For $k = 2$, this is $2n$ total copies, giving amortized $O(1)$ per insert. A growth factor of 1.5 (used by MSVC) wastes less memory and reuses previously freed blocks, which can reduce page faults at the cost of one extra copy per resize.

Follow-up: Why not grow by 1? Because that gives $O(n)$ per insert (always copy everything), no amortization benefit.

1. When would you use a linked list over a dynamic array?

Use a linked list when you need $O(1)$ insertion/deletion at arbitrary positions *without* shifting elements, when iterators must remain valid after insertions (stable iterators), or when you need to splice two lists in $O(1)$ time. Use a dynamic array when you need $O(1)$ random access, when memory locality matters (cache-friendly iteration), or when the total memory overhead must be minimal. In practice, `std::vector` wins for most workloads due to cache effects.

Real-world: The Linux kernel uses intrusive doubly linked lists (`list_head`) for process scheduling precisely because insertions are $O(1)$ and iterators never invalidate.

1. What is the amortized complexity of `push_back` on a dynamic array? Explain the accounting argument.

$O(1)$ amortized. Charge each insert 3 “credits”: 1 for the actual insert, 2 saved in the bank. When a resize of size n occurs, we have saved $2n$ credits (from n prior inserts), which pays for copying n elements. No insert ever needs more than 3 credits, so amortized cost is $O(1)$.

Variant: “Prove that `push_back` is $O(1)$ amortized but not $O(1)$ worst-case.” The worst case is $O(n)$ during a resize — you cannot avoid copying n elements.

4.9.2 Linked List Algorithms

1. Why is the three-pointer reverse technique the canonical linked list pattern — and where else does it apply?

The pattern (`prev/curr/next` with link reversal) generalizes to: reversing a sublist, removing nodes by value, reordering (LeetCode 143), and even cycle detection. The key insight is that you can reverse a pointer without losing the rest of the list *only if* you saved `next` before overwriting it. This is the fundamental “don’t lose your tail” constraint in all in-place linked list manipulation. Common mistake: forgetting to set the last node’s `next` to `nullptr`, creating a cycle. In interviews, state the invariant explicitly: “at each step, everything behind `prev` is reversed, everything ahead of `curr` is unprocessed, and `next` is the lookahead.”

1. Detect a cycle in a linked list. What is the time and space complexity?

Floyd’s tortoise and hare algorithm: use two pointers, one moving 1 step and the other 2 steps. If they meet, there is a cycle. To find the cycle start, reset one pointer to head and advance both at speed 1 — they meet at the cycle entry.

Time: $O(n)$. Space: $O(1)$.

```

1 bool has_cycle(node* head) {
2     node *slow = head, *fast = head;
3     while (fast && fast->next) {
4         slow = slow->next;
5         fast = fast->next->next;
6         if (slow == fast) return true;
7     }
8     return false;
9 }
10
11 node* find_cycle_start(node* head) {
12     node *slow = head, *fast = head;
13     while (fast && fast->next) {
14         slow = slow->next;
15         fast = fast->next->next;
16         if (slow == fast) {
17             slow = head;
18             while (slow != fast) {
19                 slow = slow->next;
20                 fast = fast->next;
21             }
22             return slow; // cycle entry
23         }
24     }
25     return nullptr;
26 }

```

Variant: “Find the length of the cycle” – once they meet, keep one fixed and advance the other until it returns to the meeting point.

1. The slow/fast pointer pattern appears in cycle detection, middle-finding, and nth-from-end. What’s the unifying principle?

The pattern exploits a *speed ratio* to create a measurable difference. If two pointers start at the same point and move at different speeds, the distance between them grows predictably. In cycle detection, the fast lap catches the slow lap. In middle-finding, when fast finishes the list (distance $2n$), slow has covered distance n . In finding the nth-from-end, advance fast n steps first, then move both — when fast reaches end, slow is at position $(\text{length} - n)$. The unifying idea: *speed differentials create positional relationships* without needing to know the list length in advance.

1. Why is the dummy head pattern the standard technique for linked list construction, and what goes wrong without it?

Without a dummy head, you need a special case for the first node (which becomes the head), leading to duplicated code for “is this the first element?” checks. The dummy head eliminates this: every node is inserted after some predecessor, including the first real node. The pattern

generalizes: use a dummy for any linked list construction where you don't know the head in advance (merge sorted lists, remove Nth from end, add two numbers). The dummy is stack-allocated, so there's no memory overhead — it's a pure code-clarity technique.

1. What's the key difference between removing duplicates in a linked list vs. an array, and why does it matter?

In an array, removing an element shifts all subsequent elements — $O(n)$ per removal, $O(n^2)$ total. In a linked list, removing a node is $O(1)$ once you have the predecessor — just relink. But finding the predecessor is $O(n)$ per node, so the total is still $O(n^2)$ with a nested loop. The hash set approach ($O(n)$ time, $O(n)$ space) works for both. The real difference: linked lists preserve iterator/pointer validity after removal (other pointers still point to valid nodes), while array removal invalidates iterators. This is why linked lists are preferred when you need stable references during modification.

1. Why does the reverse-first-half approach require three separate phases, and what's the invariant for each?

Phase 1 (find middle): slow/fast pointers — invariant: slow is at the midpoint. Phase 2 (reverse second half): three-pointer reverse — invariant: reversed portion is a standalone reversed list. Phase 3 (compare): walk both halves simultaneously — if all values match, it's a palindrome. Phase 4 (optional restore): reverse again to restore original structure. The key insight: you need to *temporarily break* the list structure to compare from both ends, then *restore* it. This three-phase pattern (find → modify → restore) appears in many linked list problems. Without restoring, you've modified the input — which may or may not be acceptable depending on the API contract.

4.9.3 Design

1. Design an LRU (Least Recently Used) cache.

Combine a hash map ($O(1)$ lookup) with a doubly linked list ($O(1)$ move-to-front and eviction). The hash map stores key → iterator into the linked list. On access, move the node to the front. On eviction, remove from the back.

```

1 template <typename K, typename V>
2 class lru_cache {
3     struct entry { K key; V value; };
4     std::list<entry> order_; // front = most recent
5     std::unordered_map<K, typename std::list<entry>::iterator> map_;
6     size_t capacity_;
7 public:
8     lru_cache(size_t cap) : capacity_(cap) {}
9
10    std::optional<V> get(const K& key) {
11        auto it = map_.find(key);

```

```

12     if (it == map_.end()) return std::nullopt;
13     order_.splice(order_.begin(), order_, it->second);
14     return it->second->value;
15 }
16
17 void put(const K& key, const V& value) {
18     auto it = map_.find(key);
19     if (it != map_.end()) {
20         it->second->value = value;
21         order_.splice(order_.begin(), order_, it->second);
22         return;
23     }
24     if (order_.size() == capacity_) {
25         map_.erase(order_.back().key);
26         order_.pop_back();
27     }
28     order_.push_front({key, value});
29     map_[key] = order_.begin();
30 }
31 };

```

Get: $O(1)$ average. Put: $O(1)$ average. Used by operating systems (page cache), databases (buffer pool), and web servers (CDN caching).

Follow-up: “Design an LFU (Least Frequently Used) cache.” Requires an additional frequency counter and bucket list — a common interview extension (LeetCode 460).

1. **You have two containers: `std::vector` and `std::list`. You need to insert 10^6 elements at random positions. Which do you choose and why?**

Depends on the pattern. If inserts are truly random (uniform over all positions), `std::vector` is faster despite $O(n)$ shifts per insert, because:

- Cache locality: vector elements are contiguous, so shifts hit L1/L2 cache
- Linked list nodes are scattered, causing cache misses on every node access
- The constant factor for a cache miss (100+ cycles) dominates the $O(1)$ pointer operations

Measure with `std::chrono` and `perf` or `valgrind --tool=cachegrind` to confirm.

This is a classic interview question testing whether you understand that asymptotic complexity is not the whole story.

4.10 Shadow Copies and Copy-on-Write

A **shadow copy** (or copy-on-write) provides an efficient mechanism for creating snapshots of a data structure. Instead of copying the entire structure, a shadow copy shares the unchanged portions and only duplicates modified parts. This is the foundation of persistent data structures.

4.10.1 Technique

When an element of a list or array is modified, the old version is retained by creating a new node for the changed element while all other nodes are shared. This requires a level of indirection: each logical element holds a pointer to the actual data, and modification creates a new pointer node.

```

1  template <typename T>
2  class COWVector {
3      struct Block {
4          std::vector<T> data;
5          std::shared_ptr<const Block> next;
6          Block(std::vector<T> d,
7               std::shared_ptr<const Block> n)
8              : data(std::move(d)), next(std::move(n)) {}
9      };
10     std::shared_ptr<const Block> head_;
11     size_t size_ = 0;
12
13 public:
14     COWVector() = default;
15
16     COWVector set(size_t idx, const T& val) const {
17         COWVector result = *this;
18         // Clone path to the modified block
19         // (simplified: full copy of first block)
20         auto new_data = head_>data;
21         new_data[idx] = val;
22         result.head_ = std::make_shared<Block>(
23             std::move(new_data), head_>next);
24         return result;
25     }
26 };

```

4.10.2 Applications

- **Version control:** Git uses copy-on-write for file snapshots
- **Undo/redo:** editors keep shadow copies of modified regions
- **Database snapshots:** PostgreSQL uses copy-on-write for MVCC
- **Functional programming:** persistent vectors (Clojure, Scala)

4.11 Exercises

4.11.1 Drill

1. Trace the state of a `dynamic_array_list` initially empty after the following sequence of operations:

`push_back(3), push_back(1), push_back(4), insert(1, 2), erase(3), push_back(5), pop_back(), insert(0, 0)` Show the array contents and size after each step. Assume initial capacity = 2.

1. What is the amortized complexity of `push_back` on a dynamic array that grows by a factor of 2? Prove your answer.

1. Given a `dynamic_array_list<int>` of size n , what is the worst-case complexity of:

a) `operator[]` b) `push_back` c) `insert` at position 0 d) `insert` at position $n-1$ e) `erase` at position $n/2$

1. Draw the singly linked list after each step of the following operations on an initially empty list:

`push_front(5), push_front(3), push_back(7), insert(1, 4), erase(0), pop_front(), push_back(9)` Show the chain nodes and pointers.

1. Trace the execution of `reverse()` on a singly linked list containing [1, 2, 3, 4]. Draw the state of the pointers after each iteration.

1. What is the worst-case time complexity of:

a) `push_front` on a singly linked list b) `push_back` on a singly linked list without a tail pointer c) `push_back` on a singly linked list with a tail pointer d) `insert` at position k in a doubly linked list e) `erase` by value (removing the first match) in a singly linked list

1. Explain why a sentinel node simplifies the `erase` operation in a singly linked list. Show the code for `erase` with and without a sentinel.

4.11.2 Application

1. Modify `dynamic_array_list` to use a growth factor of 1.5 instead of 2. Benchmark both for 10^6 `push_back` operations. Which is faster? Which wastes less memory?

1. Add a `reserve(size_type n)` method that pre-allocates capacity. Add a `shrink_to_fit()` method that reduces capacity to match size.

1. Implement `operator==` and `operator<=>` (C++20 three-way comparison) for `dynamic_array_list`.

1. Implement an STL-style iterator for `dynamic_array_list`. Support `operator*`, `operator++`, `operator!=`, `operator--`. Test with a range-based for loop.
1. Implement a `circular_buffer<T>` of fixed capacity using a dynamic array. Support `push_back`, `pop_front`, `front`, `back`, `size`, `empty`, and `operator[]`.
1. Implement `sparse_polynomial` using a sorted list of (exponent, coefficient) pairs. Compare its space and time efficiency with the dense representation.
1. Implement `erase(const T& value)` that removes the first occurrence of value from the singly linked list.
1. Implement `insert_after(iterator pos, const T& value)` and `erase_after(iterator pos)` for the singly linked list.
1. Implement `merge(other)` that merges two sorted singly linked lists into one sorted list, borrowing nodes from `other`.
1. Implement `reverse()` for the singly linked list that reverses the links in $O(n)$ time and $O(1)$ extra space.
1. Implement the Josephus problem using a circular linked list: n people stand in a circle, every k -th is eliminated. Find the survivor.
1. Implement a `split(list& front, list& back)` method that splits a singly linked list into two halves using the slow/fast pointer technique.
1. Write a function `detect_cycle(const forward_list&)` that determines whether a linked list has a cycle (Floyd's tortoise and hare algorithm).

4.11.3 Research

1. **(vector;bool)** `std::vector<bool>` is a specialization that packs bits. Research its design. What are the pros and cons?
1. **(Growth factor analysis)** Study the impact of different growth factors (1.25, 1.5, 2.0, 3.0) on memory fragmentation. Read about the "golden ratio" growth factor.
1. **(Small vector optimization)** Study how `std::pmr::vector` or `folly::small_vector` uses inline storage for small sizes.
1. **(Allocator-aware containers)** Read about `std::scoped_allocator_adaptor`. Implement an `array_list` that accepts a custom allocator.

1. **(Memory locality)** Benchmark iteration over `std::vector`, `std::list`, and `std::forward_list` for $n = 10^6$ elements. Use tools to measure cache misses.
1. **(Intrusive lists)** Research intrusive linked lists (used in the Linux kernel and Boost.Intrusive).
1. **(Lock-free lists)** Read about lock-free linked lists (Michael and Scott, 1996). Implement a concurrent singly linked list using `std::atomic`.

4.12 References and Further Reading

- Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition. Sections 31.3–31.4.
- Nicolai M. Josuttis, *The C++ Standard Library*, 2nd Edition. Chapters 7, 9, and 10.
- Herb Sutter, "vector|bool: More Problems, Better Solutions" — GotW #86.
- Donald E. Knuth, *The Art of Computer Programming*, Volume 1: Fundamental Algorithms. Section 2.2.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Sections 10.2–10.3.
- Maged M. Michael and Michael L. Scott, "Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms" — PODC 1996.
- Raymond Chen, "The Old New Thing" — various posts on linked list pitfalls.
- ISO C++ Committee, "P0404R0: Wording for out-of-thin-air resolution" (on vector growth factor).
- Facebook Folly library, `small_vector` implementation — <https://github.com/facebook/folly>
- Boost.Intrusive documentation — <https://www.boost.org/doc/libs/release/doc/html/intrusive.html>

Arrays, Matrices, and Sparse Representations

5.1 Introduction

In Chapter 4 we used one-dimensional arrays to represent linear lists. In this chapter we generalize to multi-dimensional arrays and matrices, including techniques for representing **special matrices** (triangular, diagonal) and **sparse matrices** where most elements are zero.

❧ **Complete compilable implementations of the data structures in this chapter are in code/.**

5.2 Multi-Dimensional Arrays

5.2.1 Row-Major and Column-Major Ordering

A 2D array with r rows and c columns is stored linearly in memory. Two standard mappings exist:

Row-major order (used by C++): elements of row 0, then row 1, etc.

```
1 Index(i, j) = i * c + j
```

Column-major order (used by Fortran): elements of column 0, then column 1, etc.

```
1 Index(i, j) = j * r + i
```

C++ uses row-major order. For a 3×4 matrix:

```
1 a[0][0] a[0][1] a[0][2] a[0][3] a[1][0] a[1][1] ...
2 0      1      2      3      4      5
```

5.2.2 Multi-Dimensional Arrays in C++

```
1 // 2D array on stack (fixed size)
2 int matrix[3][4];
3
4 // 2D array on heap (C++ style)
5 auto matrix = std::make_unique<int[]>(rows * cols);
6
7 // Access: matrix[i * cols + j]
8
9 // Using vector of vectors
10 std::vector<std::vector<int>> matrix(rows, std::vector<int>(cols));
```

5.2.3 A Generic Matrix Class

```

1  template <typename T>
2  class matrix {
3  public:
4      matrix(size_t rows, size_t cols)
5          : rows_(rows), cols_(cols), data_(std::make_unique<T[]>(rows *
6            cols)) {}
7
8      T& operator()(size_t i, size_t j) {
9          return data_[i * cols_ + j];
10     }
11     const T& operator()(size_t i, size_t j) const {
12         return data_[i * cols_ + j];
13     }
14
15     size_t rows() const noexcept { return rows_; }
16     size_t cols() const noexcept { return cols_; }
17     size_t size() const noexcept { return rows_ * cols_; }
18
19     matrix operator+(const matrix& other) const {
20         matrix result(rows_, cols_);
21         for (size_t k = 0; k < size(); ++k) {
22             result.data_[k] = data_[k] + other.data_[k];
23         }
24         return result;
25     }
26
27     matrix operator*(const matrix& other) const {
28         matrix result(rows_, other.cols_);
29         for (size_t i = 0; i < rows_; ++i) {
30             for (size_t k = 0; k < cols_; ++k) {
31                 T aik = (*this)(i, k);
32                 if (aik != T(0)) { // skip zero multiplies
33                     for (size_t j = 0; j < other.cols_; ++j) {
34                         result(i, j) += aik * other(k, j);
35                     }
36                 }
37             }
38             return result;
39         }
40     }
41 private:
42     size_t rows_, cols_;
43     std::unique_ptr<T[]> data_;
44 };

```

5.3 Bit Arrays and Bitmaps

A **bit array** (or bit vector, bitmap) stores one bit per element, enabling compact representation of sets of small integers. A bit array of size n uses only $\lceil n/8 \rceil$ bytes — $32\times$ less than a `bool[]` array.

5.3.1 Implementation

```

1 class bit_array {
2 public:
3     explicit bit_array(size_t n)
4         : data_((n + 63) / 64, 0), size_(n) {}
5
6     bool get(size_t i) const {
7         return (data_[i / 64] >> (i % 64)) & 1;
8     }
9
10    void set(size_t i, bool val = true) {
11        if (val)
12            data_[i / 64] |= (1ULL << (i % 64));
13        else
14            data_[i / 64] &= ~(1ULL << (i % 64));
15    }
16
17    void clear(size_t i) { set(i, false); }
18
19    // Count set bits (population count) using hardware intrinsic
20    size_t count() const {
21        size_t total = 0;
22        for (auto word : data_)
23            total += std::popcount(word);
24        return total;
25    }
26
27    // Bitwise operations
28    bit_array operator&(const bit_array& o) const {
29        bit_array result(size_);
30        for (size_t i = 0; i < data_.size(); ++i)
31            result.data_[i] = data_[i] & o.data_[i];
32        return result;
33    }
34    bit_array operator|(const bit_array& o) const { /* similar */ }
35    bit_array operator~() const {
36        bit_array result(size_);
37        for (size_t i = 0; i < data_.size(); ++i)
38            result.data_[i] = ~data_[i];
39        return result;
40    }

```

```

41
42     size_t size() const noexcept { return size_; }
43
44 private:
45     std::vector<uint64_t> data_;
46     size_t size_;
47 };

```

5.3.2 Applications

- **Set of integers:** represent a subset of $\{0, 1, \dots, n-1\}$ in $n/8$ bytes. Insert, delete, and membership test are all $O(1)$.
- **Bloom filters** (Chapter 19): a bit array of size m with k hash functions. False positives possible, false negatives not.
- **Bitmap in databases:** Oracle and PostgreSQL use bitmaps for index intersections — combining two bitmaps with AND/OR answers range queries in $O(n/64)$ word operations.
- **Image masks:** alpha channels in PNG are single-channel bitmaps. Facebook’s TAO uses bitmaps to represent friendship sets for fast intersection.

Complexity: All operations (get, set, count, AND, OR) run in $O(n/w)$ time where w is the word size (64 bits). Hardware `popcount` and `ctz` instructions make bit operations extremely fast in practice.

5.4 Dynamic Arrays

A **dynamic array** (e.g., `std::vector`) provides $O(1)$ indexed access with automatic resizing. The key insight is **amortized doubling**: when the array grows, it allocates twice the needed space, so most insertions are $O(1)$ and only occasional resizes cost $O(n)$.

5.4.1 Growth Factor

The growth factor determines how much extra space to allocate on resize:

Library	Growth Factor	Rationale
GCC libstdc++ <code>std::vector</code>	2	Classic doubling
MSVC <code>std::vector</code>	1.5	Better memory utilization (smaller waste on resize)
Java <code>ArrayList</code>	1.5	Same as MSVC
Rust <code>Vec</code>	2 (then ~ 1.5)	Aggressive then conservative

5.4.2 Amortized Analysis

With doubling, n insertions into an initially empty array cost:

- Copies at capacities 1, 2, 4, 8, ..., $2^{\lceil \log_2 n \rceil}$: total $< 2n$
- n insertions at unit cost: n
- Total: $< 3n$, so **amortized $O(1)$ per insertion**

With growth factor $g > 1$, the wasted space on the last resize is $\frac{g-1}{g}$ of the total capacity. Factor 2 wastes at most 50%; factor 1.5 wastes at most 33%.

Cross-reference: See Chapter 2, Section "Amortized Analysis" for the formal proof using the accounting and potential methods.

5.5 Sorted Arrays

A **sorted array** maintains elements in sorted order, enabling $O(\log n)$ search via binary search at the cost of $O(n)$ insertion and deletion.

5.5.1 Operations

```

1  template <typename T>
2  class sorted_array {
3  public:
4      void insert(const T& value) {
5          auto pos = std::lower_bound(data_.begin(), data_.end(), value);
6          data_.insert(pos, value); // O(n) shift
7      }
8
9      bool contains(const T& value) const {
10         return std::binary_search(data_.begin(), data_.end(), value); //
O(log n)
11     }
12
13     void erase(const T& value) {
14         auto pos = std::lower_bound(data_.begin(), data_.end(), value);
15         if (pos != data_.end() && *pos == value)
16             data_.erase(pos); // O(n) shift
17     }
18
19     // Range query: all elements in [lo, hi]
20     auto range(const T& lo, const T& hi) const {
21         auto first = std::lower_bound(data_.begin(), data_.end(), lo);
22         auto last = std::upper_bound(data_.begin(), data_.end(), hi);
23         return std::pair(first, last); // O(log n) to find, O(k) to
iterate

```

```

24     }
25
26     const T& operator[](size_t i) const { return data_[i]; }
27     size_t size() const noexcept { return data_.size(); }
28
29 private:
30     std::vector<T> data_;
31 };

```

5.5.2 Complexity

Operation	Sorted Array	Unsorted Array
Search	$O(\log n)$	$O(n)$
Insert	$O(n)$	$O(1)$
Delete	$O(n)$	$O(1)$
Min/Max	$O(1)$	$O(n)$
Range query	$O(\log n + k)$	$O(n)$

When to use: sorted arrays are ideal for **read-heavy, write-rarely** workloads — e.g., lookup tables that are built once and queried millions of times. The contiguous memory layout gives excellent cache performance, often beating balanced BSTs for search despite the same $O(\log n)$ asymptotic complexity.

5.6 Circular Buffers

A **circular buffer** (ring buffer) is a fixed-size array with two pointers (head and tail) that wrap around. It implements a FIFO queue in $O(1)$ time with $O(1)$ space — no shifting, no dynamic allocation.

5.6.1 Implementation

```

1  template <typename T>
2  class circular_buffer {
3  public:
4      explicit circular_buffer(size_t capacity)
5          : buf_(capacity), head_(0), tail_(0), size_(0) {}
6
7      void push(const T& val) {
8          if (size_ == buf_.size())
9              throw std::overflow_error("buffer full");
10         buf_[tail_] = val;
11         tail_ = (tail_ + 1) % buf_.size();
12         ++size_;
13     }
14
15     T pop() {

```

```

16         if (size_ == 0)
17             throw std::underflow_error("buffer empty");
18         T val = buf_[head_];
19         head_ = (head_ + 1) % buf_.size();
20         --size_;
21         return val;
22     }
23
24     const T& front() const { return buf_[head_]; }
25     bool empty() const { return size_ == 0; }
26     size_t size() const { return size_; }
27
28 private:
29     std::vector<T> buf_;
30     size_t head_, tail_, size_;
31 };

```

Wrap-around arithmetic: $\text{head}_+ = (\text{head}_+ + 1) \% \text{capacity}$. The modulo ensures the pointer wraps to index 0 after reaching the end.

5.6.2 Applications

- **CPU cache lines:** modern CPUs use circular buffers for instruction prefetch queues.
- **Streaming data:** the last N events (e.g., server logs, sensor readings) are kept in a ring buffer of size N — $O(1)$ append, oldest automatically evicted.
- **Producer-consumer queues:** lock-free circular buffers are the foundation of high-performance queues (e.g., LMAX Disruptor, Linux kfifo).
- **Buffered I/O:** `std::cin` and file streams use circular buffers internally to batch system calls.

Fixed vs. dynamic: A fixed-size circular buffer wastes no memory on pointers or allocation, but cannot grow. A dynamic ring buffer (e.g., doubling the capacity) combines the benefits but requires copying on resize.

Cross-reference: Chapter 7 implements queues using linked lists and arrays. The circular buffer is the most cache-efficient array-based queue.

5.7 Gap Buffers

A **gap buffer** is a text-editing data structure: a contiguous array with a "gap" (empty region) at the cursor position. Inserts and deletes near the cursor are $O(1)$; operations far from the cursor require moving the gap.

5.7.1 Implementation

```

1 class gap_buffer {
2 public:
3     explicit gap_buffer(size_t initial = 128)
4         : buf_(initial), gap_start_(0), gap_end_(initial) {}
5
6     // Insert character at cursor position -- O(1) amortized
7     void insert(char c) {
8         if (gap_start_ == gap_end_) grow_gap();
9         buf_[gap_start_++] = c;
10    }
11
12    // Delete character before cursor -- O(1)
13    char delete_back() {
14        if (gap_start_ == 0) return '\\0';
15        return buf_[--gap_start_];
16    }
17
18    // Move cursor left -- O(1)
19    void move_left() {
20        if (gap_start_ > 0)
21            buf_[--gap_end_] = buf_[--gap_start_];
22    }
23
24    // Move cursor right -- O(1)
25    void move_right() {
26        if (gap_end_ < buf_.size())
27            buf_[gap_start_++] = buf_[gap_end_++];
28    }
29
30    // Get full text
31    std::string text() const {
32        return std::string(buf_.begin(), buf_.begin() + gap_start_) +
33            std::string(buf_.begin() + gap_end_, buf_.end());
34    }
35
36    size_t cursor_pos() const { return gap_start_; }
37
38 private:
39    void grow_gap() {
40        size_t new_size = buf_.size() * 2;
41        std::vector<char> new_buf(new_size);
42        // Copy before gap
43        std::copy(buf_.begin(), buf_.begin() + gap_start_, new_buf.begin()
44    );
45        // Copy after gap
46        size_t after = buf_.size() - gap_end_;

```

```

46     size_t new_gap_end = new_size - after;
47     std::copy(buf_.begin() + gap_end_, buf_.end(), new_buf.begin() +
    new_gap_end);
48     gap_end_ = new_gap_end;
49     buf_ = std::move(new_buf);
50 }
51
52     std::vector<char> buf_;
53     size_t gap_start_, gap_end_;
54 };

```

Why it works for text editors: Most typing happens at the cursor. The gap sits at the cursor, so every insert/delete is $O(1)$. Moving the cursor shifts the gap — $O(\text{distance})$, but since users type locally, the amortized cost is low.

5.7.2 Comparison with Alternatives

Structure	Insert at cursor	Move cursor	Memory
Gap buffer	$O(1)$ amortized	$O(\text{distance})$	Contiguous, cache-friendly
Doubly-linked list (per char)	$O(1)$	$O(1)$	High overhead per node
Rope (tree of strings)	$O(\log n)$	$O(\log n)$	Moderate overhead
Piece table	$O(1)$ amortized	$O(\log n)$	Two buffers + log

Emacs uses gap buffers. Most modern editors (VS Code, Sublime) use piece tables or ropes.

5.8 Special Matrices

Many problems involve matrices with a known structure. We can save space by storing only the unique elements.

5.8.1 Diagonal Matrix

A diagonal matrix has non-zero entries only on the main diagonal:

```

1 [d$_{0}$ 0 0 ]
2 [ 0 d$_{1}$ 0 ]
3 [ 0 0 d$_{2}$]

```

Store as a 1D array of size n : $d[i]$ stores the element at (i, i) .

```

1 template <typename T>
2 class diagonal_matrix {
3 public:
4     explicit diagonal_matrix(size_t n) : n_(n), data_(std::make_unique<T
    []>(n)) {}
5

```

```

6     T& operator()(size_t i, size_t j) {
7         if (i != j) throw std::logic_error("non-diagonal access");
8         return data_[i];
9     }
10    const T& operator()(size_t i, size_t j) const {
11        if (i != j) return zero_;
12        return data_[i];
13    }
14
15 private:
16     size_t n_;
17     std::unique_ptr<T[]> data_;
18     static constexpr T zero_ = T(0);
19 };

```

5.8.2 Triangular Matrices

Upper triangular: all entries below the main diagonal are zero. Lower triangular: all entries above the main diagonal are zero.

```

1 Lower triangular (n=4):
2 [a_{00} 0 0 0]
3 [a_{10} a_{11} 0 0]
4 [a_{20} a_{21} a_{22} 0]
5 [a_{30} a_{31} a_{32} a_{33}]

```

Store in a 1D array of size $n(n+1)/2$:

```

1 Row-major mapping for lower triangular: index(i, j) = i(i+1)/2 + j for i
  >= j

```

```

1 template <typename T>
2 class lower_triangular_matrix {
3 public:
4     explicit lower_triangular_matrix(size_t n)
5         : n_(n), data_(std::make_unique<T[]>(n * (n + 1) / 2)) {}
6
7     T& operator()(size_t i, size_t j) {
8         if (i < j) throw std::logic_error("below diagonal");
9         return data_[i * (i + 1) / 2 + j];
10    }
11    const T& operator()(size_t i, size_t j) const {
12        if (i < j) return zero_;
13        return data_[i * (i + 1) / 2 + j];
14    }
15
16 private:
17     size_t n_;

```

```

18     std::unique_ptr<T[]> data_;
19     static constexpr T zero_ = T(0);
20 };

```

Space comparison for n=1000:

- Full array: 1,000,000 elements
- Triangular: 500,500 elements — ~50% savings

5.8.3 Tridiagonal Matrix

Non-zero only on the main diagonal, first super-diagonal, and first sub-diagonal:

```

1 [a$_{0}$ b$_{0}$ 0 0 0 ]
2 [c$_{0}$ a$_{1}$ b$_{1}$ 0 0 ]
3 [ 0 c$_{1}$ a$_{2}$ b$_{2}$ 0 ]
4 [ 0 0 c$_{2}$ a$_{3}$ b$_{3}$ ]
5 [ 0 0 0 c$_{3}$ a$_{4}$ ]

```

Store in three 1D arrays (a, b, c) or interleaved: $3n-2$ elements.

5.8.4 Thomas Algorithm

A tridiagonal system $Ax = d$ can be solved in $O(n)$ by forward elimination and back substitution — the Thomas algorithm:

```

1 // Solve tridiagonal system:
2 // a[i]*x[i-1] + b[i]*x[i] + c[i]*x[i+1] = d[i]
3 // a[0] = 0 (no sub-diagonal for first row)
4 // c[n-1] = 0 (no super-diagonal for last row)
5 std::vector<double> thomas_solve(
6     std::vector<double> a, // sub-diagonal (size n, a[0] unused)
7     std::vector<double> b, // main diagonal (size n)
8     std::vector<double> c, // super-diagonal (size n, c[n-1] unused)
9     std::vector<double> d // right-hand side (size n)
10 ) {
11     size_t n = b.size();
12
13     // Forward elimination
14     for (size_t i = 1; i < n; ++i) {
15         double m = a[i] / b[i - 1];
16         b[i] -= m * c[i - 1];
17         d[i] -= m * d[i - 1];
18     }
19
20     // Back substitution
21     std::vector<double> x(n);
22     x[n - 1] = d[n - 1] / b[n - 1];

```

```

23     for (int i = n - 2; i >= 0; --i) {
24         x[i] = (d[i] - c[i] * x[i + 1]) / b[i];
25     }
26     return x;
27 }

```

Worked trace for n=4:

```

1 System:
2   4x[0] + 1x[1]           = 6
3   1x[0] + 4x[1] + 1x[2]   = 14
4           1x[1] + 4x[2] + 1x[3] = 16
5           1x[2] + 4x[3] = 12
6
7 a = [0, 1, 1, 1] (sub-diagonal)
8 b = [4, 4, 4, 4] (main diagonal)
9 c = [1, 1, 1, 0] (super-diagonal)
10 d = [6, 14, 16, 12]
11
12 Forward elimination:
13   i=1: m = 1/4 = 0.25
14       b[1] = 4 - 0.25*1 = 3.75
15       d[1] = 14 - 0.25*6 = 12.5
16   i=2: m = 1/3.75 = 0.2667
17       b[2] = 4 - 0.2667*1 = 3.7333
18       d[2] = 16 - 0.2667*12.5 = 12.667
19   i=3: m = 1/3.7333 = 0.2679
20       b[3] = 4 - 0.2679*1 = 3.7321
21       d[3] = 12 - 0.2679*12.667 = 8.619
22
23 Back substitution:
24   x[3] = 8.619 / 3.7321 = 2.310
25   x[2] = (12.667 - 1*2.310) / 3.7333 = 2.776
26   x[1] = (12.5 - 1*2.776) / 3.75 = 2.593
27   x[0] = (6 - 1*2.593) / 4 = 0.852

```

Complexity: $O(n)$ time, $O(n)$ space. This is optimal — any algorithm must read all n elements at least once.

Applications: solving discretized differential equations (heat equation, beam bending), cubic spline interpolation, and any system where each variable depends only on its neighbors.

5.8.5 Symmetric Matrices

A symmetric matrix satisfies $A(i,j) = A(j,i)$. Only the upper (or lower) triangle needs storage — $n(n+1)/2$ elements, same as triangular matrices.

```

1 // Access element (i, j) in compact storage
2 // Using lower triangle: index = i*(i+1)/2 + j   for i >= j

```



```

3 // Symmetric: A(i,j) = A(j,i), so swap if i < j
4 T symmetric_access(const T* data, size_t i, size_t j, size_t n) {
5     if (i < j) std::swap(i, j);
6     return data[i * (i + 1) / 2 + j];
7 }

```

Applications: distance matrices (symmetric by definition), correlation matrices in statistics, stiffness matrices in finite element analysis.

5.9 Sparse Matrices

When a matrix has few non-zero elements (e.g., 5%), storing all zeros is wasteful. We instead store only the non-zero entries, each with its position.

5.9.1 Coordinate Storage (COO)

Store a list of (row, col, value) triples:

```

1 struct triple {
2     size_t row;
3     size_t col;
4     double value;
5 };
6
7 class sparse_matrix_coo {
8 public:
9     sparse_matrix_coo(size_t rows, size_t cols)
10         : rows_(rows), cols_(cols) {}
11
12     void add_entry(size_t row, size_t col, double value) {
13         if (value != 0.0) {
14             entries_.push_back({row, col, value});
15         }
16     }
17
18     double operator()(size_t row, size_t col) const {
19         for (const auto& e : entries_) {
20             if (e.row == row && e.col == col) return e.value;
21         }
22         return 0.0;
23     }
24
25     size_t nonzero_count() const noexcept { return entries_.size(); }
26     size_t rows() const noexcept { return rows_; }
27     size_t cols() const noexcept { return cols_; }
28
29 private:
30     size_t rows_, cols_;

```

```

31     std::vector<triple> entries_; // unsorted; sort by (row, col) for
    efficiency
32 };

```

5.9.2 Compressed Sparse Row (CSR)

CSR is the most widely used sparse format for linear algebra. It stores:

- **values**: non-zero values, in row-major order
- **col_indices**: column index of each value
- **row_ptr**: start index of each row in values/col_indices

```

1 Matrix:
2 [1 0 2]
3 [0 3 0]
4 [4 0 5]
5
6 values:      [1, 2, 3, 4, 5]
7 col_indices: [0, 2, 1, 0, 2]
8 row_ptr:     [0, 2, 3, 5]

```

```

1 template <typename T>
2 class csr_matrix {
3 public:
4     csr_matrix(size_t rows, size_t cols)
5         : rows_(rows), cols_(cols), row_ptr_(rows + 1, 0) {}
6
7     void finalize() {
8         // Convert from COO to CSR
9         std::sort(coo_.begin(), coo_.end(),
10             [](const auto& a, const auto& b) {
11                 return std::tie(a.row, a.col) < std::tie(b.row, b.col);
12             });
13
14         // Build CSR
15         size_t nnz = coo_.size();
16         values_.reserve(nnz);
17         col_indices_.reserve(nnz);
18
19         for (const auto& t : coo_) {
20             values_.push_back(t.value);
21             col_indices_.push_back(t.col);
22         }
23
24         // Build row_ptr
25         size_t idx = 0;

```

```

26     for (size_t i = 0; i < rows_; ++i) {
27         row_ptr_[i] = idx;
28         while (idx < nnz && coo_[idx].row == i) ++idx;
29     }
30     row_ptr_[rows_] = nnz;
31
32     coo_.clear(); // free temporary storage
33 }
34
35 T operator()(size_t row, size_t col) const {
36     for (size_t i = row_ptr_[row]; i < row_ptr_[row + 1]; ++i) {
37         if (col_indices_[i] == col) return values_[i];
38     }
39     return T(0);
40 }
41
42 // Sparse matrix-vector multiplication: result = A * x
43 std::vector<T> multiply(const std::vector<T>& x) const {
44     std::vector<T> result(rows_, T(0));
45     for (size_t i = 0; i < rows_; ++i) {
46         T sum = T(0);
47         for (size_t j = row_ptr_[i]; j < row_ptr_[i + 1]; ++j) {
48             sum += values_[j] * x[col_indices_[j]];
49         }
50         result[i] = sum;
51     }
52     return result;
53 }
54
55 size_t rows() const noexcept { return rows_; }
56 size_t cols() const noexcept { return cols_; }
57 size_t nonzero_count() const noexcept { return values_.size(); }
58
59 private:
60     size_t rows_, cols_;
61     std::vector<T> values_;
62     std::vector<size_t> col_indices_;
63     std::vector<size_t> row_ptr_;
64     std::vector<triple> coo_; // temporary during construction
65 };

```

5.9.3 Complexity Comparison

Operation	Dense $O(n^2)$	CSR Sparse
Access $A(i,j)$	$O(1)$	$O(k)$ where k = nonzeros per row
Matrix-vector multiply	$O(n^2)$	$O(nnz)$
Memory	$O(n^2)$	$O(n + nnz)$

Example: A $10,000 \times 10,000$ matrix with 50,000 non-zeros (0.05% density)

- Dense: 800 MB (double precision)
- CSR: ~1.2 MB (values + indices + row_ptr)

5.10 Application: PageRank

The PageRank algorithm computes the importance of web pages using sparse matrix-vector multiplication. The web graph is represented as a sparse matrix where $A(i,j) = 1$ if page j links to page i .

```

1  std::vector<double> pagerank(const csr_matrix<double>& web_graph,
2                               size_t iterations = 100,
3                               double damping = 0.85) {
4      size_t n = web_graph.rows();
5      std::vector<double> rank(n, 1.0 / n);
6      std::vector<double> new_rank(n, 0.0);
7      double teleport = (1.0 - damping) / n;
8
9      for (size_t iter = 0; iter < iterations; ++iter) {
10         // Sparse matrix-vector multiply
11         new_rank = web_graph.multiply(rank);
12
13         // Apply damping and teleportation
14         double sum = 0.0;
15         for (size_t i = 0; i < n; ++i) {
16             new_rank[i] = damping * new_rank[i] + teleport;
17             sum += new_rank[i];
18         }
19
20         // Normalize
21         for (size_t i = 0; i < n; ++i) new_rank[i] /= sum;
22
23         std::swap(rank, new_rank);
24     }
25     return rank;
26 }
```

5.11 Application: Image Processing

Images are 2D arrays (or 3D for color). Matrix operations form the foundation of image processing:

```

1 // Grayscale image as a matrix
2 using image = matrix<uint8_t>;
3
4 // Convolution: apply a kernel to every pixel
5 image apply_filter(const image& img, const matrix<double>& kernel) {
6     image result(img.rows(), img.cols());
7     int k_half = kernel.rows() / 2;
8
9     for (size_t i = k_half; i < img.rows() - k_half; ++i) {
10         for (size_t j = k_half; j < img.cols() - k_half; ++j) {
11             double sum = 0.0;
12             for (size_t ki = 0; ki < kernel.rows(); ++ki) {
13                 for (size_t kj = 0; kj < kernel.cols(); ++kj) {
14                     sum += kernel(ki, kj) * img(i + ki - k_half, j + kj -
15 k_half);
16                 }
17             }
18             result(i, j) = static_cast<uint8_t>(std::clamp(sum, 0.0,
19 255.0));
20         }
21     }
22     return result;
23 }
```

5.12 STL Connection: `std::mdspan` (C++23)

C++23 introduces `std::mdspan` — a non-owning multi-dimensional array view:

```

1 #include <mdspan>
2 #include <vector>
3
4 std::vector<int> data = {1, 2, 3, 4, 5, 6};
5 std::mdspan<int, std::dextents<int, 2>> matrix_2x3(data.data(), 2, 3);
6 // matrix_2x3[i, j] gives row-major access
```

While `std::mdspan` is not yet widely available, our `matrix` class follows a similar design.

5.13 Sparse Arrays

A **sparse array** is an array in which most elements are zero (or share a common default value). Storing every element wastes memory; instead we store only the non-default entries.

5.13.1 Coordinate (COO) Representation

The simplest approach stores each non-default element as a (index, value) pair:

```

1 struct SparseEntry {
2     int index;
3     int value;
4 };
5
6 class SparseArray {
7     std::vector<SparseEntry> entries;
8     int size;
9 public:
10    SparseArray(int n) : size(n) {}
11
12    void set(int idx, int val) {
13        for (auto& e : entries) {
14            if (e.index == idx) {
15                e.value = val;
16                return;
17            }
18        }
19        entries.push_back({idx, val});
20    }
21
22    int get(int idx) const {
23        for (const auto& e : entries) {
24            if (e.index == idx) return e.value;
25        }
26        return 0;
27    }
28
29    int nnz() const { return entries.size(); }
30 };

```

If the entries are kept *sorted by index*, both `get` and `set` can use binary search, giving $O(\log k)$ where k is the number of stored entries.

5.13.2 Map-Based Representation

A `std::unordered_map<int, int>` gives amortised $O(1)$ access:

```

1 std::unordered_map<int, int> sparse;
2 sparse[1000] = 42;
3 sparse[5000] = 7;
4
5 int val = sparse.count(1000) ? sparse[1000] : 0;

```

Complexity comparison:

Operation	Sorted vector	unordered_map
Get	$O(\log k)$	$O(1)$ avg
Set (existing)	$O(\log k)$	$O(1)$ avg
Set (insert)	$O(k)$	$O(1)$ avg
Memory	$2k$ ints	$\sim 4k$ ints (hash overhead)

When $k \ll n$, both representations save substantial memory over a dense array of size n .

5.14 Array Rotations

Given an array of n elements, **rotation** by d positions shifts every element d places to the right (wrapping around). For example, rotating $\{1, 2, 3, 4, 5, 6, 7\}$ right by $d = 2$ yields $\{6, 7, 1, 2, 3, 4, 5\}$. We require $0 \leq d < n$.

5.14.1 Juggling Algorithm

The juggling algorithm moves elements in cycles. For rotation by d , there are $\gcd(n, d)$ independent cycles, each of length $n/\gcd(n, d)$. We pick up one element and shift it d positions forward until we return to the start.

Worked trace for $n = 7, d = 2$ ($\gcd(7, 2) = 1$, one cycle of length 7):

Start	1	2	3	4	5	6	7
Cycle 1: move $a[0]$ to $a[2]$	1	2	1	4	5	6	7
Move $a[2](=3)$ to $a[4]$	1	2	1	4	3	6	7
Move $a[4](=5)$ to $a[6]$	1	2	1	4	3	6	5
Move $a[6](=7)$ to $a[1]$	1	7	1	4	3	6	5
Move $a[1](=2)$ to $a[3]$	1	7	1	2	3	6	5
Move $a[3](=4)$ to $a[5]$	1	7	1	2	3	4	5
Move $a[5](=6)$ to $a[0]$	6	7	1	2	3	4	5

Result: $\{6, 7, 1, 2, 3, 4, 5\}$.

```

1 void jugglingRotate(std::vector<int>& a, int d) {
2     int n = a.size();
3     d = ((d % n) + n) % n; // normalise
4     int g = std::gcd(n, d);
5     for (int i = 0; i < g; ++i) {
6         int tmp = a[i];
7         int j = i;
8         while (true) {
9             int k = (j + d) % n;
10            if (k == i) break;
11            a[j] = a[k];

```

```

12         j = k;
13     }
14     a[j] = tmp;
15 }
16 }

```

Complexity: $O(n)$ time, $O(1)$ extra space.

5.14.2 Reversal Algorithm

The key insight is three reversals:

1. Reverse the last d elements.
2. Reverse the first $n - d$ elements.
3. Reverse the entire array.

Worked trace for $n = 7, d = 2$:

```

Original : 1  2  3  4  5 | 6  7
Step 1   : 1  2  3  4  5 | 7  6      (reverse last d)
Step 2   : 5  4  3  2  1 | 7  6      (reverse first n-d)
Step 3   : 6  7  1  2  3  4  5      (reverse whole)

```

```

1 void reverse(std::vector<int>& a, int lo, int hi) {
2     while (lo < hi) std::swap(a[lo++], a[hi--]);
3 }
4
5 void reversalRotate(std::vector<int>& a, int d) {
6     int n = a.size();
7     d = ((d % n) + n) % n;
8     reverse(a, n - d, n - 1);
9     reverse(a, 0, n - d - 1);
10    reverse(a, 0, n - 1);
11 }

```

Complexity: $O(n)$ time, $O(1)$ extra space. This is the most widely used in-place rotation algorithm.

5.14.3 Block Swap Algorithm

Block swap treats the rotation as swapping two blocks: $A[0 \dots d-1]$ and $A[d \dots n-1]$.

Worked trace for $n = 7, d = 3$ (swap blocks 1 2 3 and 4 5 6 7):

```

Step 1: A=123, B=4567 | swap min(3,4) = 3: 4561237
Step 2: now have 456|123|7 | swap min(3,1) = 1: 476|123|5 ... simplified:
Final: 4 5 6 7 1 2 3

```



```

1 void blockSwap(std::vector<int>& a, int d) {
2     int n = a.size();
3     d = ((d % n) + n) % n;
4     if (d == 0 || d == n) return;
5     int i = d, j = n - d;
6     while (i != j) {
7         if (i < j) {
8             std::swap_ranges(a.begin() + d - i, a.begin() + d,
9                             a.begin() + d + j - i);
10            j -= i;
11        } else {
12            std::swap_ranges(a.begin() + d, a.begin() + d + i - j,
13                            a.begin() + d + i);
14            i -= j;
15        }
16    }
17    std::swap_ranges(a.begin() + d - i, a.begin() + d,
18                    a.begin() + d + i);
19 }

```

Complexity: $O(n)$ time, $O(1)$ extra space. The number of swaps equals the number of elements moved.

5.15 Kadane's Algorithm: Maximum Subarray

The **maximum subarray problem** asks for a contiguous subarray $A[i \dots j]$ whose sum is maximum. Kadane's algorithm solves it in $O(n)$ time using a single pass.

5.15.1 Core Idea

Maintain a running sum *cur* of the best subarray *ending at the current position*. If *cur* drops below zero, discard it and start fresh:

```

1 int maxSubarraySum(const std::vector<int>& a) {
2     int cur = 0, best = INT_MIN;
3     for (int x : a) {
4         cur = std::max(x, cur + x);
5         best = std::max(best, cur);
6     }
7     return best;
8 }

```

5.15.2 Recovery of Subarray Indices

To find the actual subarray boundaries, track where each run begins:

```

1 struct SubarrayResult {

```

```

2   int sum;
3   int start;
4   int end;
5 };
6
7 SubarrayResult maxSubarray(const std::vector<int>& a) {
8     int cur = 0, best = INT_MIN;
9     int curStart = 0, bestStart = 0, bestEnd = 0;
10    for (int i = 0; i < (int)a.size(); ++i) {
11        if (cur < 0) {
12            cur = a[i];
13            curStart = i;
14        } else {
15            cur += a[i];
16        }
17        if (cur > best) {
18            best = cur;
19            bestStart = curStart;
20            bestEnd = i;
21        }
22    }
23    return {best, bestStart, bestEnd};
24 }

```

5.15.3 Worked Trace

Input: $a = \{-2, 1, -3, 4, -1, 2, 1, -5, 4\}$

i	$a[i]$	cur	curStart	best	bestStart, bestEnd
0	-2	$-2 \rightarrow 0$ (reset)	1	-2	(0, 0)
1	1	$0 + 1 = 1$	1	1	(1, 1)
2	-3	$1 - 3 = -2$	1	1	(1, 1)
3	4	$-2 \rightarrow 4$ (reset)	3	4	(3, 3)
4	-1	$4 - 1 = 3$	3	4	(3, 3)
5	2	$3 + 2 = 5$	3	5	(3, 5)
6	1	$5 + 1 = 6$	3	6	(3, 6)
7	-5	$6 - 5 = 1$	3	6	(3, 6)
8	4	$1 + 4 = 5$	3	6	(3, 6)

Result: maximum sum = 6, subarray $A[3 \dots 6] = \{4, -1, 2, 1\}$.

Complexity: $O(n)$ time, $O(1)$ extra space.

5.16 Exercises

1. Implement a **sparse array** using a sorted vector of (index, value) pairs with binary search for $O(\log k)$ get/set. Write unit tests comparing it with `std::unordered_map<int, int>` for arrays of size 10^6 with 1000 non-zero entries.
2. Given a sorted sparse array of (index, value) pairs and a second sparse array, compute their **element-wise sum** in $O(k_1 + k_2)$ time, producing a new sorted sparse array.
3. Implement the **juggling rotation algorithm** and the **reversal rotation algorithm** in C++. Verify both produce the same output on random arrays of size 100 with random rotation amounts. Measure wall-clock time for $n = 10^6$.
4. Prove that the **reversal algorithm** correctly rotates any array by d positions. Hint: show that reversing $A[d \dots n-1]$, then $A[0 \dots d-1]$, then $A[0 \dots n-1]$ yields a left rotation by d .
5. Modify **Kadane's algorithm** to handle arrays containing *all negative numbers* and return the maximum single element. Test with $\{-5, -3, -1, -4, -2\}$ and verify it returns -1 with indices $(2, 2)$.

5.17 Common Bugs and Pitfalls

Pitfall	Consequence	
Row-major vs column-major confusion	Wrong element accessed, silent data corruption	Be explicit
Out-of-bounds on <code>std::mdspan</code> or raw arrays	No bounds checking in release builds	Use assertions
Memory overhead of COO sparse format (3 arrays)	$3\times$ storage vs CSR for no benefit	Use CSR format
Not checking dimension compatibility in multiply	Wrong result size, index out of range	Always validate dimensions
Assuming C-style arrays decay to <code>std::mdspan</code>	Losing dimension info, wrong strides	Wrap in <code>std::mdspan</code>
Using <code>vector<vector<T>></code> for 2D data	Poor cache locality, fragmentation	Use flat 1D array

5.18 Summary

1. **Row-major order** is the standard C++ memory layout for 2D arrays: $\text{index}(i,j) = i \cdot c + j$.
2. **Bit arrays** store one bit per element, enabling $O(1)$ set membership, compact Bloom filters, and bitmap index intersections.
3. **Dynamic arrays** (`std::vector`) provide $O(1)$ amortized insertion via geometric growth; the growth factor (1.5 vs 2) trades memory waste for resize frequency.
4. **Sorted arrays** enable $O(\log n)$ binary search at the cost of $O(n)$ insertion — ideal for read-heavy workloads.

5. **Circular buffers** implement FIFO queues in $O(1)$ time with fixed memory — used in CPU caches, streaming, and lock-free queues.
6. **Gap buffers** provide $O(1)$ text insertion at the cursor, making them the classic data structure for text editors.
7. **Special matrices** (diagonal, triangular, tridiagonal) can be stored in $O(n)$ space instead of $O(n^2)$.
8. The **Thomas algorithm** solves tridiagonal systems in $O(n)$ time — optimal for banded linear systems.
9. **Sparse matrices** (CSR, COO) store only non-zero elements, enabling operations on matrices too large for dense storage.
10. **Sparse matrix-vector multiplication** runs in $O(\text{nnz})$ time — essential for iterative methods like PageRank.
11. The **operator()(i,j)** convention is standard for C++ matrix classes.

5.19 Interview FAQs

1. Why is row-major order faster than column-major for row-wise traversal?

In row-major layout, consecutive elements in a row are stored in contiguous memory. Traversing row-wise means sequential memory access, which triggers hardware prefetching and maximizes cache line utilization. Column-major traversal jumps by the row stride on each access, causing cache misses. On a typical 64-byte cache line with 8-byte doubles, row-major row traversal loads 8 useful elements per cache miss, while column-major loads only 1.

Real-world: This is why BLAS and LAPACK use column-major (Fortran convention) but traverse by column internally.

1. When should you use a sparse matrix format (CSR/COO) over a dense matrix?

When the number of non-zero elements $\text{nnz} \ll n^2$ (less than $\sim 10\text{--}20\%$ of total entries). CSR format uses $O(\text{nnz})$ space instead of $O(n^2)$, and sparse matrix-vector multiply runs in $O(\text{nnz})$ instead of $O(n^2)$. However, random element access in CSR requires binary search on the column index array — $O(\log(\text{nnz}/\text{row}))$ per access vs $O(1)$ for dense. Use dense when the matrix is mostly full or when you need frequent random access.

1. What is the time complexity of multiplying two sparse matrices? Can you do better than $O(n^3)$?

Naive: for each non-zero in A, look up matching non-zeros in B. With CSR, this is $O(\text{nnz}_A \times \text{avg_row_nnz}_B)$ in the worst case, but typically much less. The theoretical best for sparse matrix multiply is $O(\text{nnz}_A \cdot \text{nnz}_B / n)$ using a hash-based approach. For very sparse matrices ($\text{nnz} = O(n)$), this approaches $O(n)$ — far better than the dense $O(n^3)$.

1. Why is manual index arithmetic for multi-dimensional arrays error-prone, and how does `mdspan` fix it?

Manual indexing for a 3D array requires `data[i * cols * depth + j * depth + k]` — get the stride wrong and you access the wrong element with no compiler error. `mdspan` encapsulates the layout policy (row-major, column-major, strided) into the type system, so the compiler generates the correct index arithmetic. The real benefit: you can change the memory layout (e.g., from row-major to column-major for Fortran interop) by changing the template parameter, without rewriting any access code. C++23 `mdspan` is to multi-dimensional arrays what `std::string_view` is to `char*` — a safe, zero-overhead abstraction.

1. Design a matrix class that supports $O(1)$ row and column swaps.

Store the matrix in row-major order but add an indirection layer: maintain a row permutation vector `row_map` and column permutation vector `col_map`. Accessing element (i,j) reads from `data[row_map[i] * cols + col_map[j]]`. Swapping two rows or columns is $O(1)$ — just swap two entries in the permutation vector. This technique is used in LAPACK’s row/column permutation routines.

1. Why is matrix multiplication $O(n^3)$ and can it be done faster?

The naive triple loop performs n^3 multiply-add operations. Strassen’s algorithm (1969) reduces this to $O(n^{2.807})$ by recursively computing 7 products instead of 8. The Coppersmith–Winograd algorithm achieves $O(n^{2.376})$ but is not practical due to large constants. In practice, cache-oblivious algorithms and SIMD vectorization of the naive $O(n^3)$ approach outperform Strassen for matrices that fit in L2 cache.

1. When should you use a banded matrix format instead of dense storage, and what operations become harder?

Use banded storage when the bandwidth b is much smaller than n ($b \ll n$) — you save from $O(n^2)$ to $O(n \cdot b)$ space. Band-diagonal, tridiagonal, and pentadiagonal matrices all qualify. The tradeoff: element access becomes $O(1)$ with index arithmetic (`data[(i-j+b) * (2b+1) + j]`), but operations like matrix-vector multiply need careful boundary handling at the band edges. In HPC, LAPACK’s `gbmv` (banded matrix-vector multiply) is the workhorse. The key insight: banded formats are worth it when $b < n/4$ — below that threshold, the index overhead and boundary handling complexity outweigh the memory savings.

1. How does PageRank use sparse matrix operations?

PageRank represents the web as a sparse adjacency matrix M where $M_{ij} = 1/\text{out_degree}(j)$ if page j links to page i . The PageRank vector is the stationary distribution: $r = dMr + (1 - d)/n \cdot \mathbf{1}$. This is computed by repeated sparse matrix-vector multiplication (Mr) until convergence. With billions of pages and ~ 10 links per page, $\text{nnz} \approx 10n$, so each iteration is $O(n)$ — feasible only because M is sparse.

1. Why does transpose-then-reverse work for 90° rotation, and how does this generalize to other angles?

A 90° clockwise rotation is the composition of two symmetries: transpose (reflect over main diagonal) then reverse each row (reflect horizontally). For 180°: reverse each row then reverse each column (or equivalently, rotate 90° twice). For 270°: transpose then reverse each column. The unifying principle: any rotation by $k \cdot 90$ is a composition of two reflections. This generalizes to arbitrary rotations via the rotation matrix, but those don't preserve the grid structure (elements land between cells). For images and matrices, the 90° decomposition is the standard technique because it's in-place ($O(1)$ extra space) and cache-friendly.

```
1 void rotate90(std::vector<std::vector<int>>& m) {
2     int n = m.size();
3     // transpose
4     for (int i = 0; i < n; ++i)
5         for (int j = i + 1; j < n; ++j)
6             std::swap(m[i][j], m[j][i]);
7     // reverse each row
8     for (int i = 0; i < n; ++i)
9         std::reverse(m[i].begin(), m[i].end());
10 }
```

1. What is the difference between COO and CSR formats, and when would you use each?

COO stores a simple list of (row, column, value) triples — easy to build, easy to append to, but slow for arithmetic (must sort or scan to find elements). CSR stores three arrays (values, column indices, row pointers) — requires a sort during construction but enables $O(\log(\text{nnz}/\text{row}))$ random access and efficient row slicing. Use COO for construction and streaming; convert to CSR for computation. Most sparse linear algebra libraries (Eigen, SciPy) accept COO input and convert internally.

1. Why use a bit array instead of a `std::vector<bool>` or `std::set<int>`?

A bit array uses 1 bit per element — $8\times$ less than `vector<char>` and $32\times$ less than `vector<int>`. For a set of integers in $[0, 10^6]$, a bit array uses 125 KB; a `set<int>` uses 32 MB (32 bytes per node). Bit arrays also support $O(1)$ membership via a single bitwise AND, while `set` requires $O(\log n)$ tree traversal with pointer chasing (cache-unfriendly). Note: `std::vector<bool>` is a partial specialization that stores bits, but its proxy-reference semantics make it error-prone.

1. Explain amortized $O(1)$ for dynamic array insertion. Why not just use $O(n)$ always?

With doubling, most insertions write to pre-allocated space — $O(1)$. Every 2^k -th insertion triggers a resize copying 2^k elements. Over n insertions, total copies: $1 + 2 + 4 + \dots + 2^{\lfloor \log_2 n \rfloor} < 2n$. So total work is $O(n)$ for n insertions, giving $O(1)$ amortized. The worst-case single insertion is $O(n)$, but this happens so rarely that the average is constant. For real-time systems requiring worst-case $O(1)$, use a pre-allocated array or a linked list of fixed-size blocks.

1. When would you use a circular buffer over a `std::queue`?

A circular buffer is ideal when: (1) the maximum size is known or bounded — no allocation overhead; (2) you need $O(1)$ worst-case (not amortized) per operation — no resizing pauses; (3) you want minimal memory overhead — no node pointers. `std::queue` backed by `std::deque` dynamically allocates in chunks, adding pointer overhead and potential allocation stalls. The circular buffer is used in real-time audio (JACK), high-frequency trading (LMAX Disruptor), and kernel networking (Linux `sk_buff` ring).

1. How does a gap buffer work, and why is it good for text editors?

A gap buffer is a contiguous array with an empty region (gap) at the cursor. Inserting a character at the cursor places it at `gap_start` and advances the gap — $O(1)$. Deleting backs up `gap_start` — $O(1)$. Moving the cursor requires shifting the gap — $O(\text{distance})$. Since typing is overwhelmingly local (users type at the cursor), the amortized cost per keystroke is $O(1)$. The contiguous layout gives excellent cache performance for rendering. Emacs uses gap buffers; modern editors use piece tables or ropes for better $O(\log n)$ cursor movement.

```

1 class gap_buffer {
2     std::vector<char> buf_;
3     int gap_start_ = 0, gap_end_ = 0;
4 public:
5     explicit gap_buffer(int cap) : buf_(cap), gap_end_(cap) {}
6
7     void insert(char c) {
8         if (gap_start_ == gap_end_) grow_gap();
9         buf_[gap_start_++] = c;
10    }
11
12    void move_cursor(int pos) {
13        while (gap_start_ < pos) buf_[--gap_end_] = buf_[gap_start_++];
14        while (gap_start_ > pos) buf_[gap_start_++] = buf_[gap_end_++];
15    }
16
17    void erase() {
18        if (gap_start_ > 0) --gap_start_;
19    }
20
21    std::string text() const {
22        return std::string(buf_.begin(), buf_.begin() + gap_start_)
23            + std::string(buf_.begin() + gap_end_, buf_.end());
24    }
25
26 private:
27    void grow_gap() {
28        int sz = gap_start_ + (int)buf_.size() - gap_end_ + 1;
29        std::vector<char> new_buf(sz * 2);

```

```

30         std::copy(buf_.begin(), buf_.begin() + gap_start_, new_buf.begin()
31         );
31         int new_gap_end = (int)new_buf.size() - ((int)buf_.size() -
32         gap_end_);
32         std::copy(buf_.begin() + gap_end_, buf_.end(), new_buf.begin() +
33         new_gap_end);
33         buf_ = std::move(new_buf);
34         gap_end_ = new_gap_end;
35     }
36 };

```

1. Design a fixed-size LRU cache using a sorted array and a circular buffer.

Use a circular buffer of size N to store (key, value) pairs in access order — most recent at tail. For $O(\log n)$ lookup, maintain a sorted array of (key, circular_buffer_index) pairs. On access, remove the key from the sorted array ($O(\log n)$ binary search + $O(n)$ shift), move the entry in the circular buffer ($O(1)$ by writing to tail), and re-insert in the sorted array ($O(\log n) + O(n)$ shift). The $O(n)$ shift on the sorted array can be avoided using a self-balancing BST instead, giving $O(\log n)$ for all operations.

5.20 Exercises

5.20.1 Drill

- For a 5×4 matrix stored in row-major order, what is the offset of element (2, 3) from the start of the array? What about for column-major order?
- How many non-zero elements must a 1000×1000 matrix have before the sparse representation (using CSR) uses more memory than the dense representation? Assume 8 bytes per value, 8 bytes per column index, 4 bytes per row pointer, and a 64-bit machine.
- Show the storage layout (in memory) for the special matrices:
 - A 5×5 diagonal matrix with values [1, 2, 3, 4, 5]
 - A 4×4 lower triangular matrix
 - A 4×4 tridiagonal matrix
- What is the number of non-zero elements in an $n \times n$ tridiagonal matrix? An $n \times n$ upper triangular matrix? An $n \times n$ symmetric matrix where only the lower triangle is stored?

5.20.2 Application

- Implement `tridiagonal_matrix<T>` storing only the three non-zero diagonals. Provide a `solve` method using the Thomas algorithm ($O(n)$ time). Test it on a random tridiagonal system.

1. Write a function to multiply two CSR matrices A ($m \times k$) and B ($k \times n$). Analyze its time complexity in terms of non-zero counts. Compare with standard dense multiplication.
1. Implement the `toeplitz_matrix` class where each diagonal is constant. What is the minimal storage? Implement matrix-vector multiply.
1. Benchmark dense vs CSR matrix-vector multiply for matrices of varying sparsity (0.1%, 1%, 10%, 50%) at size 1000×1000 . At what density does dense become faster?
1. Implement the Floyd-Warshall all-pairs shortest path algorithm on a dense matrix. Then implement it using CSR. Compare performance for sparse graphs with 1%, 5%, and 20% edge density.
1. Implement matrix transposition for CSR format — produce A^T from A in $O(\text{nnz})$ time.
1. Write a function to convert a dense matrix to CSR format and back. Verify correctness by comparing original and round-tripped matrices.
1. Implement the PageRank algorithm using sparse matrix-vector multiplication. Test on a graph with 1000 nodes and ~5000 edges.
1. Write a function `add_sparse` that adds two CSR matrices. What is the complexity?

5.20.3 Research

1. **(SuiteSparse)** Read about the SuiteSparse Matrix Collection. Pick a real sparse matrix (e.g., from circuit simulation or web graphs) and compare CSR vs dense performance for a computation of your choice.
1. **(Cache-oblivious matrices)** Research the recursive (cache-oblivious) matrix multiplication algorithm (Z-Morton order, or recursive block decomposition). Implement it and compare with the standard triple-loop for large matrices.
1. **(SIMD matrix multiply)** Study how BLAS libraries (like Intel MKL or OpenBLAS) use SIMD instructions (AVX, AVX-512) to accelerate matrix multiplication. Implement a tiled matrix multiply that uses AVX intrinsics.
1. **(GraphBLAS)** The GraphBLAS standard defines graph algorithms in terms of linear algebra operations over semirings. Read the GraphBLAS specification and implement breadth-first search using sparse matrix-vector multiplication.

5.20.4 Bit Array Exercises

1. Implement a set of integers $\{0, 1, \dots, 10^6\}$ using a bit array. Compare insertion time and memory usage with `std::unordered_set<int>` for 10^5 random insertions and 10^6 lookups.

1. Implement a Bloom filter using a bit array and $k = 3$ hash functions (MurmurHash). Insert 10^5 strings and measure the false positive rate for $m = 10^6$ bits. Compare with the theoretical bound $(1 - e^{-kn/m})^k$.
1. Implement bitwise AND, OR, and population count for bit arrays. Benchmark against `std::bitset` for arrays of size 10^6 .

5.20.5 Circular Buffer Exercises

1. Implement a single-producer single-consumer lock-free circular buffer. Test correctness with 2 threads: one producing integers $0..10^6$, one consuming and verifying order. Measure throughput in MB/s.
1. Implement a circular buffer that supports `push_front`, `push_back`, `pop_front`, and `pop_back` — a double-ended circular buffer. What is the capacity constraint compared to a single-ended ring buffer?

5.20.6 Gap Buffer Exercises

1. Implement a gap buffer text editor: support `insert(char)`, `backspace()`, `move_left()`, `move_right()`, and `text()`. Test on a 10,000-character document with random cursor movements and insertions. Compare editing speed with `std::string` (which requires $O(n)$ shifts on every insert).

5.20.7 Sorted Array Exercises

1. Implement a sorted array and benchmark binary search against `std::set` for 10^6 lookups on 10^6 elements. Which is faster and why? (Hint: cache locality.)
1. Implement the merge of two sorted arrays into a single sorted array in $O(n + m)$ time. Use it as the merge step in merge sort. Compare performance with `std::merge`.

5.21 References and Further Reading

- Gene H. Golub and Charles F. Van Loan, *Matrix Computations*, 4th Edition. JHU Press, 2013.
- Tim Davis, *Direct Methods for Sparse Linear Systems*. SIAM, 2006.
- Tim Davis, "Algorithm 832: UMFPACK V4.3 — an unsymmetric-pattern multifrontal method" — ACM TOMS, 2004.
- Yousef Saad, *Iterative Methods for Sparse Linear Systems*, 2nd Edition. SIAM, 2003.
- James W. Demmel, *Applied Numerical Linear Algebra*. SIAM, 1997.

- Jeremy Kepner and John Gilbert (eds.), *Graph Algorithms in the Language of Linear Algebra*. SIAM, 2011.
- SuiteSparse Matrix Collection — <https://sparse.tamu.edu/>

Stacks

6.1 Introduction

↳ Complete compilable implementations of the data structures in this chapter are in `code/`.

6.2 The Stack ADT

A **stack** is a linear list where insertions and deletions occur only at one end, called the **top**. This LIFO (Last In, First Out) discipline is fundamental in computing.

6.2.1 Operations

- **push(value)** — insert at top
- **pop()** — remove and return the top element
- **top()** — return the top element without removing it
- **empty()** — true if no elements
- **size()** — number of elements

6.2.2 Applications

Stacks are used for:

- Function call/return management (the call stack)
- Expression evaluation (postfix, infix conversion)
- Undo operations
- Depth-first search (Chapter 12)
- Parsing (balanced parentheses, HTML/XML)
- Backtracking algorithms (Chapter 20)

6.3 Array-Based Stack

```
1  template <typename T>
2  class array_stack {
3  public:
4      array_stack() = default;
5
6      void push(const T& value) {
7          if (size_ >= data_.size()) {
8              data_.push_back(value);
9          } else {
10             data_[size_] = value;
11         }
12         ++size_;
13     }
14
15     void push(T&& value) {
16         if (size_ >= data_.size()) {
17             data_.push_back(std::move(value));
18         } else {
19             data_[size_] = std::move(value);
20         }
21         ++size_;
22     }
23
24     template <typename... Args>
25     void emplace(Args&&... args) {
26         if (size_ >= data_.size()) {
27             data_.emplace_back(std::forward<Args>(args)...);
28         } else {
29             data_[size_] = T(std::forward<Args>(args)...);
30         }
31         ++size_;
32     }
33
34     void pop() {
35         if (empty()) throw std::underflow_error("stack is empty");
36         --size_;
37     }
38
39     T& top() {
40         if (empty()) throw std::underflow_error("stack is empty");
41         return data_[size_ - 1];
42     }
43
44     const T& top() const {
45         if (empty()) throw std::underflow_error("stack is empty");
46         return data_[size_ - 1];
47     }
```

```

46     }
47
48     bool empty() const noexcept { return size_ == 0; }
49     size_t size() const noexcept { return size_; }
50
51 private:
52     std::vector<T> data_;
53     size_t size_ = 0;
54 };

```

All operations are $O(1)$ amortized. The `std::vector` grows as needed, but we track `size_` separately to support $O(1)$ pop without deallocating memory.

6.4 Linked Stack

```

1  template <typename T>
2  class linked_stack {
3  public:
4      linked_stack() = default;
5
6      void push(const T& value) {
7          head_ = std::make_unique<node<T>>(value, std::move(head_));
8          ++size_;
9      }
10
11     void pop() {
12         if (empty()) throw std::underflow_error("stack is empty");
13         head_ = std::move(head_>next);
14         --size_;
15     }
16
17     T& top() {
18         if (empty()) throw std::underflow_error("stack is empty");
19         return head_>data;
20     }
21     const T& top() const {
22         if (empty()) throw std::underflow_error("stack is empty");
23         return head_>data;
24     }
25
26     bool empty() const noexcept { return size_ == 0; }
27     size_t size() const noexcept { return size_; }
28
29 private:
30     struct node {
31         T data;
32         std::unique_ptr<node> next;

```

```

33     node(const T& d, std::unique_ptr<node> nxt)
34         : data(d), next(std::move(nxt)) {}
35     };
36     std::unique_ptr<node> head_;
37     size_t size_ = 0;
38 };

```

All operations: $O(1)$.

6.5 Applications

6.5.1 Balanced Parentheses

Check whether parentheses, brackets, and braces are properly matched:

```

1  bool is_balanced(std::string_view expr) {
2      linked_stack<char> s;
3      for (char c : expr) {
4          switch (c) {
5              case '(': case '[': case '{':
6                  s.push(c);
7                  break;
8              case ')':
9                  if (s.empty() || s.top() != '(') return false;
10                 s.pop();
11                 break;
12             case ']':
13                 if (s.empty() || s.top() != '[') return false;
14                 s.pop();
15                 break;
16             case '}':
17                 if (s.empty() || s.top() != '{') return false;
18                 s.pop();
19                 break;
20         }
21     }
22     return s.empty();
23 }

```

Trace: "{ [()] }"

```

1  c = '{' -> push: stack = ['{']
2  c = '[' -> push: stack = ['{', '[']
3  c = '(' -> push: stack = ['{', '[', '(']
4  c = ')' -> top is '(' -> pop: stack = ['{', '[']
5  c = ']' -> top is '[' -> pop: stack = ['{']
6  c = '}' -> top is '{' -> pop: stack = []
7  Result: true

```

6.5.2 Postfix Evaluation

Postfix (Reverse Polish Notation) places operators after operands: $3\ 4\ +\ 5\ * = (3+4) \times 5 = 35$.

```

1 int evaluate_postfix(std::string_view expr) {
2     linked_stack<int> s;
3     for (char c : expr) {
4         if (std::isspace(c)) continue;
5         if (std::isdigit(c)) {
6             s.push(c - '0');
7         } else {
8             int b = s.top(); s.pop();
9             int a = s.top(); s.pop();
10            switch (c) {
11                case '+': s.push(a + b); break;
12                case '-': s.push(a - b); break;
13                case '*': s.push(a * b); break;
14                case '/': s.push(a / b); break;
15            }
16        }
17    }
18    return s.top();
19 }
```

Trace: "3 4 + 5 *"

```

1 '3' -> push: [3]
2 '4' -> push: [3, 4]
3 '+' -> pop 4, pop 3 -> push 7: [7]
4 '5' -> push: [7, 5]
5 '*' -> pop 5, pop 7 -> push 35: [35]
6 Result: 35
```

6.5.3 Expression Evaluation in a Spreadsheet Engine (e.g., Excel, Google Sheets)

Modern spreadsheet engines evaluate cell formulas using an operator-precedence parser backed by a stack. When a cell contains $= (A1+B2) * C3 - D4$, the engine:

1. Converts infix to postfix (shunting-yard, stack-based)
2. Evaluates the postfix expression (also stack-based)

The shunting-yard algorithm uses an operator stack:

```

1 int precedence(char op) {
2     if (op == '+' || op == '-') return 1;
3     if (op == '*' || op == '/') return 2;
4     if (op == '^') return 3;
5     return 0;
```



```

6 }
7
8 std::string infix_to_postfix(std::string_view expr) {
9     linked_stack<char> ops;
10    std::string output;
11    for (char c : expr) {
12        if (std::isalnum(c)) {
13            output += c;
14        } else if (c == '(') {
15            ops.push(c);
16        } else if (c == ')') {
17            while (!ops.empty() && ops.top() != '(') {
18                output += ops.top(); ops.pop();
19            }
20            ops.pop(); // discard '('
21        } else { // operator
22            while (!ops.empty() && precedence(ops.top()) >= precedence(c))
23            {
24                output += ops.top(); ops.pop();
25            }
26            ops.push(c);
27        }
28    }
29    while (!ops.empty()) { output += ops.top(); ops.pop(); }
30    return output;
31 }

```

Trace: "(A1+B2)\{\}textit{C3-D4" → "A1 B2 + C3 } D4 -"

This is the same algorithm PostgreSQL uses in its expression parser (`src/backend/parser/expr_parse.c`) and that Google Sheets applies to every formula cell update.

6.6 STL Connection: `std::stack`

`std::stack` is a container adapter — it wraps any sequence container providing `push_back`, `pop_back`, and `back`:

```

1 std::stack<int> s; // uses std::deque by default
2 std::stack<int, std::vector<int>> sv; // vector-based
3 std::stack<int, std::list<int>> sl; // list-based
4
5 s.push(10);
6 s.push(20);
7 int x = s.top(); // 20
8 s.pop(); // removes 20

```

Our `array_stack` is more efficient than `std::stack` for push/pop because we avoid deallocation (we only decrement `size_`), while `std::stack::pop` destroys the element immediately.

6.7 Balanced Symbol Checking with Error Reporting

The basic balanced-parentheses checker (Section 6 Applications) returns only a boolean. A production-quality checker must report the exact location and nature of the mismatch. This version tracks line numbers and column positions, and tells the user which closing symbol was expected versus which was found.

```

1 struct parse_error {
2     int line;
3     int column;
4     char expected;
5     char found;
6     parse_error(int l, int c, char e, char f)
7         : line(l), column(c), expected(e), found(f) {}
8 };
9
10 std::vector<parse_error> check_balanced(std::string_view src) {
11     std::vector<parse_error> errors;
12     linked_stack<std::pair<char, std::pair<int, int>>> stk;
13     int line = 1, col = 0;
14     for (char c : src) {
15         ++col;
16         if (c == '\n') { ++line; col = 0; continue; }
17         if (c == '(' || c == '[' || c == '{') {
18             stk.push({c, {line, col}});
19         } else if (c == ')' || c == ']' || c == '}') {
20             if (stk.empty()) {
21                 errors.emplace_back(line, col, '\0', c);
22             } else {
23                 auto [open, pos] = stk.top();
24                 stk.pop();
25                 char match = (c == ')') ? '('
26                           : (c == ']') ? '[' : '{';
27                 if (open != match) {
28                     errors.emplace_back(
29                         line, col, match, c);
30                 }
31             }
32         }
33     }
34     while (!stk.empty()) {
35         auto [open, pos] = stk.top();
36         stk.pop();
37         char expected_close = (open == '(') ? ')'
38                           : (open == '[') ? ']' : '}';
39         errors.emplace_back(
40             pos.first, pos.second,
41             expected_close, '\0');

```

```

42     }
43     return errors;
44 }

```

Trace: "(a + b] * c"

```
1 Line 1, Col 4: expected ')' but found ']'
```

Trace: "{(a + b) * c"

```
1 Line 1, Col 1: expected '}' but found EOF
```

The algorithm is still $O(n)$ time and $O(n)$ space, but the stack now stores metadata (line, column) alongside each opening symbol.

6.8 Postfix Evaluation: A Complete Worked Example

Postfix (Reverse Polish Notation) evaluation is one of the clearest applications of a stack. We present a full implementation that handles multi-digit numbers, and then trace a non-trivial example.

```

1 #include <string>
2 #include <vector>
3 #include <stdexcept>
4
5 double eval_postfix(const std::vector<std::string>& tokens) {
6     linked_stack<double> stk;
7     for (const auto& tok : tokens) {
8         if (tok == "+" || tok == "-" ||
9             tok == "*" || tok == "/") {
10             if (stk.size() < 2)
11                 throw std::runtime_error("bad expr");
12             double b = stk.top(); stk.pop();
13             double a = stk.top(); stk.pop();
14             if (tok == "+") stk.push(a + b);
15             else if (tok == "-") stk.push(a - b);
16             else if (tok == "*") stk.push(a * b);
17             else stk.push(a / b);
18         } else {
19             stk.push(std::stod(tok));
20         }
21     }
22     if (stk.size() != 1)
23         throw std::runtime_error("bad expr");
24     return stk.top();
25 }

```

Complete trace for expression "10 2 3 + * 4 -":

```
1 Token "10" -> push 10.0    Stack: [10]
```

```

2 Token "2"    -> push 2.0    Stack: [10, 2]
3 Token "3"    -> push 3.0    Stack: [10, 2, 3]
4 Token "+"    -> pop 3, 2    -> push 5.0
5              Stack: [10, 5]
6 Token "*"    -> pop 5, 10    -> push 50.0
7              Stack: [50]
8 Token "4"    -> push 4.0    Stack: [50, 4]
9 Token "-"    -> pop 4, 50    -> push 46.0
10             Stack: [46]
11 Result: 46.0

```

The key insight: operands are pushed eagerly; operators pop their operands and push the result. The stack always holds intermediate values waiting to be consumed.

6.9 Function Call Simulation with Activation Records

Every language runtime uses a call stack to manage function calls. Each call creates an **activation record** (stack frame) containing parameters, local variables, and a return address. We simulate this manually to understand what happens under the hood.

```

1 #include <string>
2 #include <vector>
3
4 struct activation_record {
5     std::string function;
6     int param_n;
7     int local_product;
8     int return_to; // index in call sequence
9 };
10
11 int factorial_simulated(int n) {
12     linked_stack<activation_record> call_stack;
13     int next_call_id = 0;
14     // push initial call
15     call_stack.push({"fact", n, 1, -1});
16
17     while (!call_stack.empty()) {
18         auto& frame = call_stack.top();
19
20         if (frame.function == "fact") {
21             if (frame.param_n <= 1) {
22                 // base case: return 1
23                 int result = 1;
24                 call_stack.pop();
25                 if (!call_stack.empty()) {
26                     // multiply into parent frame
27                     auto& parent = call_stack.top();

```

```

28         parent.local_product =
29             parent.param_n * result;
30         parent.function = "fact_returned";
31     }
32     continue;
33 }
34 // recursive case: push new frame
35 frame.function = "fact_awaiting";
36 call_stack.push(
37     {"fact", frame.param_n - 1, 1, 0});
38 } else if (frame.function == "fact_awaiting") {
39     // child returned, product is in local_product
40     int result = frame.local_product;
41     call_stack.pop();
42     if (!call_stack.empty()) {
43         auto& parent = call_stack.top();
44         parent.local_product =
45             parent.param_n * result;
46         parent.function = "fact_returned";
47     }
48 } else { // fact_returned
49     return frame.local_product;
50 }
51 }
52 return 1;
53 }

```

Trace for `factorial_simulated(4)`:

1 Push:	fact(n=4)	Stack: [fact(4)]
2 Push:	fact(n=3)	Stack: [fact(4), fact(3)]
3 Push:	fact(n=2)	Stack: [fact(4), fact(3), fact(2)]
4 Push:	fact(n=1)	Stack: [fact(4), fact(3), fact(2), fact(1)]
5 Base:	fact(1) -> 1	Stack: [fact(4), fact(3), fact(2)]
6	fact(2) = 2 * 1 = 2	
7 Return:		Stack: [fact(4), fact(3)]
8	fact(3) = 3 * 2 = 6	
9 Return:		Stack: [fact(4)]
10	fact(4) = 4 * 6 = 24	
11 Return:		Stack: []
12 Result:	24	

The activation record mirrors what the hardware does: the x86-64 ABI places parameters in registers (`rdi`, `rsi`), allocates space for locals on the stack, and stores the return address via the `call` instruction.

6.10 Min Stack: O(1) Minimum Queries

A stack that supports `getMin()` in O(1) time is a classic interview problem. The idea: maintain an auxiliary stack that tracks the minimum at each depth.

```

1  template <typename T>
2  class min_stack {
3  public:
4      void push(const T& value) {
5          data_.push(value);
6          if (min_.empty() || value <= min_.top()) {
7              min_.push(value);
8          }
9      }
10
11     void pop() {
12         if (data_.empty())
13             throw std::underflow_error("empty");
14         if (data_.top() == min_.top()) {
15             min_.pop();
16         }
17         data_.pop();
18     }
19
20     const T& top() const { return data_.top(); }
21
22     const T& get_min() const {
23         if (min_.empty())
24             throw std::underflow_error("empty");
25         return min_.top();
26     }
27
28     bool empty() const { return data_.empty(); }
29     size_t size() const { return data_.size(); }
30
31 private:
32     linked_stack<T> data_;
33     linked_stack<T> min_;
34 };

```

Trace for pushing 5, 3, 7, 2, 8, 2:

1	push(5)	->	data: [5]	min: [5]	getMin -> 5
2	push(3)	->	data: [5,3]	min: [5,3]	getMin -> 3
3	push(7)	->	data: [5,3,7]	min: [5,3]	getMin -> 3
4	push(2)	->	data: [5,3,7,2]	min: [5,3,2]	getMin -> 2
5	pop()	->	data: [5,3,7]	min: [5,3]	getMin -> 3
6	push(8)	->	data: [5,3,7,8]	min: [5,3]	getMin -> 3
7	push(2)	->	data: [5,3,7,8,2]	min: [5,3,2]	getMin -> 2

Each operation is $O(1)$ time, $O(n)$ space. The critical detail: use $\leq$ (not $<$) when comparing to the min stack top, so duplicate minimum values are tracked correctly.

6.11 Stack Using Queues

Implementing a stack using only queue operations is a common exercise that tests understanding of both data structures. We present two variants.

6.11.1 Push $O(n)$, Pop $O(1)$

On each push, enqueue to $q2$, then transfer all elements from $q1$ to $q2$, then swap the queues.

```

1 class stack_via_queues {
2 public:
3     void push(int x) {
4         q2_.push(x);
5         while (!q1_.empty()) {
6             q2_.push(q1_.front());
7             q1_.pop();
8         }
9         std::swap(q1_, q2_);
10    }
11
12    void pop() {
13        if (q1_.empty())
14            throw std::underflow_error("empty");
15        q1_.pop();
16    }
17
18    int top() {
19        if (q1_.empty())
20            throw std::underflow_error("empty");
21        return q1_.front();
22    }
23
24    bool empty() const { return q1_.empty(); }
25
26 private:
27     std::queue<int> q1_, q2_;
28 };

```

Trace for push 1, push 2, push 3:

```

1 push(1): q2=[1] -> q1=[1]
2 push(2): q2=[2,1] -> q1=[2,1]      (front is 2, LIFO)
3 push(3): q2=[3,2,1] -> q1=[3,2,1] (front is 3)
4 pop():   removes 3, q1=[2,1]
5 top():   returns 2

```

6.11.2 Push $O(1)$, Pop $O(n)$

On each push, enqueue to $q1$. On pop, transfer all but the last element to $q2$, pop the last element, then swap.

```
1 class stack_via_queues_v2 {
2 public:
3     void push(int x) {
4         q1_.push(x);
5     }
6
7     void pop() {
8         if (q1_.empty())
9             throw std::underflow_error("empty");
10        while (q1_.size() > 1) {
11            q2_.push(q1_.front());
12            q1_.pop();
13        }
14        q1_.pop();
15        std::swap(q1_, q2_);
16    }
17
18    int top() {
19        if (q1_.empty())
20            throw std::underflow_error("empty");
21        while (q1_.size() > 1) {
22            q2_.push(q1_.front());
23            q1_.pop();
24        }
25        int val = q1_.front();
26        q2_.push(val);
27        q1_.pop();
28        std::swap(q1_, q2_);
29        return val;
30    }
31
32    bool empty() const { return q1_.empty(); }
33
34 private:
35     std::queue<int> q1_, q2_;
36 };
```

In practice, the $O(1)$ -push version is preferred when pushes are more frequent than pops. The choice depends on the workload.

6.12 Stack for Infix to Prefix Conversion

While the shunting-yard algorithm converts infix to postfix using one operator stack, converting infix to **prefix** (Polish notation) requires two stacks: one for operators and one for operands. The key difference is that operands are pushed in reverse order and the entire output is read from right to left.

6.12.1 Algorithm

1. Scan the infix expression from **right to left**.
2. If the token is an **operand**, push it onto the operand stack.
3. If the token is **)**, push it onto the operator stack.
4. If the token is **(**, pop and output operators from the operator stack until **)** is found. Discard both parentheses.
5. If the token is an **operator**:
 - While the operator stack is not empty, the top is not **)**, and the top operator has **strictly greater** precedence than the current operator, pop and output it.
 - Push the current operator onto the operator stack.
 - For **right-associative** operators (like $^$), use $\geq$ instead of $>$ when comparing precedence.
6. After scanning all tokens, pop and output all remaining operators.
7. The operand stack now contains the prefix expression (read it by popping, then reverse the result).

6.12.2 Worked Trace: “A*(B+C)-D/E”

We scan from right to left. We use two stacks: `ops` (operators) and `out` (operand/operator output tokens in prefix order).

```

1 Read '/' (operator):
2   ops: [ '/' ]
3   out: []
4
5 Read 'E' (operand):
6   ops: [ '/' ]
7   out: [ 'E' ]
8
9 Read '-' (operator):
10  Top of ops is '/' which has higher precedence -> pop '/' and output
11  ops: [ '-' ]
12  out: [ 'E', '/' ]
13
14 Read 'C' (operand):

```

```

15 ops: ['-']
16 out: ['E', '/', 'C']
17
18 Read '+' (operator):
19 Top of ops is '-' which has lower or equal precedence -> push '+'
20 ops: ['- ', '+']
21 out: ['E', '/', 'C']
22
23 Read 'B' (operand):
24 ops: ['- ', '+']
25 out: ['E', '/', 'C', 'B']
26
27 Read '(' : pop until ')'
28 Pop '+' and output
29 ops: ['-']
30 out: ['E', '/', 'C', 'B', '+']
31
32 Read ')' : discard
33
34 Read '*' (operator):
35 Top of ops is '-' which has lower precedence -> push '*'
36 ops: ['- ', '*']
37 out: ['E', '/', 'C', 'B', '+']
38
39 Read 'A' (operand):
40 ops: ['- ', '*']
41 out: ['E', '/', 'C', 'B', '+', 'A']
42
43 End of input. Pop remaining operators:
44 Pop '*' and output -> out: ['E', '/', 'C', 'B', '+', 'A', '*']
45 Pop '-' and output -> out: ['E', '/', 'C', 'B', '+', 'A', '*', '-']
46
47 Final prefix expression (reverse of out):
48 Result: "- * A + B C / D E"

```

Verification: $- * A + B C / D E$ parses as $(A \times (B + C)) - (D/E)$, which is the original infix expression. Correct.

6.12.3 C++ Implementation

```

1 #include <string>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5 #include <cctype>
6
7 int prefix_prec(char op) {
8     if (op == '+' || op == '-') return 1;

```

```

9     if (op == '*' || op == '/') return 2;
10    if (op == '^') return 3;
11    return 0;
12 }
13
14 bool is_right_assoc(char op) { return op == '^'; }
15
16 std::string infix_to_prefix(std::string_view expr) {
17     std::stack<char> ops;
18     std::vector<std::string> out;
19
20     for (int i = expr.size() - 1; i >= 0; --i) {
21         char c = expr[i];
22         if (std::isalnum(c)) {
23             out.push_back(std::string(1, c));
24         } else if (c == ')') {
25             ops.push(c);
26         } else if (c == '(') {
27             while (!ops.empty() && ops.top() != ')') {
28                 out.push_back(std::string(1, ops.top()));
29                 ops.pop();
30             }
31             if (!ops.empty()) ops.pop();
32         } else { // operator
33             while (!ops.empty() && ops.top() != ')') &&
34                 (prefix_prec(ops.top()) > prefix_prec(c) ||
35                 (prefix_prec(ops.top()) == prefix_prec(c) &&
36                 !is_right_assoc(c))) {
37                 out.push_back(std::string(1, ops.top()));
38                 ops.pop();
39             }
40             ops.push(c);
41         }
42     }
43     while (!ops.empty()) {
44         out.push_back(std::string(1, ops.top()));
45         ops.pop();
46     }
47
48     std::reverse(out.begin(), out.end());
49     std::string result;
50     for (size_t i = 0; i < out.size(); ++i) {
51         if (i) result += ' ';
52         result += out[i];
53     }
54     return result;
55 }

```

Time complexity: $O(n)$ to scan, $O(n)$ to reverse and join. Space: $O(n)$ for the two stacks and output

vector.

Note on associativity: For the $^$ operator, we only pop operators with **strictly greater** precedence (not equal), because exponentiation is right-associative. For all other operators (left-associative), we pop on $\geq$, matching the infix-to-postfix convention.

6.13 Stack for Undo/Redo

Text editors, spreadsheets, and graphics tools all need undo/redo functionality. The classic solution uses **two stacks**: one for the undo history and one for the redo history.

6.13.1 Algorithm

- **Do(action):** Push the **inverse** of the action onto the undo stack. Clear the redo stack (any redo history is invalidated by a new action).
- **Undo():** Pop from the undo stack, apply the inverse, push the action onto the redo stack.
- **Redo():** Pop from the redo stack, apply the action, push its inverse onto the undo stack.

This is exactly how most editors work: performing a new operation after undo discards the redo history.

6.13.2 Worked Trace

Operations: `type("hello"), type(" world"), undo(), type("!"), redo()`

```

1 Initial state:  text = ""
2                undo: []   redo: []
3
4 type("hello"):
5   Inverse: delete("hello")
6   undo: [del("hello")]    redo: []
7   text = "hello"
8
9 type(" world"):
10  Inverse: delete(" world")
11  undo: [del("hello"), del(" world")]    redo: []
12  text = "hello world"
13
14 undo():
15  Pop: del(" world"), apply it
16  Push inverse onto redo
17  undo: [del("hello")]    redo: [type(" world")]
18  text = "hello"
19
20 type("!"):

```

```

21 Inverse: delete("!")
22 Clear redo
23 undo: [del("hello"), del("!")]    redo: []
24 text = "hello!"
25
26 redo():
27   Pop: type("!"), apply it
28   Push inverse onto undo
29   undo: [del("hello"), del("!"), del("!")]    redo: []
30   text = "hello!!"

```

Important detail: After `undo()` followed by `type("!")` in step 4, the redo stack was cleared because the user made a new edit. The subsequent `redo()` in step 5 only reapplies the most recent undone action that is still in the redo stack.

6.13.3 C++ Implementation

```

1  #include <string>
2  #include <stack>
3  #include <functional>
4  #include <iostream>
5
6  struct action {
7      std::string description;
8      std::function<void()> do_fn;
9      std::function<void()> undo_fn;
10 };
11
12 class text_editor {
13 public:
14     void execute(action act) {
15         act.do_fn();
16         undo_stack_.push(std::move(act));
17         // new action invalidates redo history
18         while (!redo_stack_.empty()) redo_stack_.pop();
19     }
20
21     bool undo() {
22         if (undo_stack_.empty()) return false;
23         action act = std::move(undo_stack_.top());
24         undo_stack_.pop();
25         act.undo_fn();
26         redo_stack_.push(std::move(act));
27         return true;
28     }
29
30     bool redo() {

```

```

31     if (redo_stack_.empty()) return false;
32     action act = std::move(redo_stack_.top());
33     redo_stack_.pop();
34     act.do_fn();
35     undo_stack_.push(std::move(act));
36     return true;
37 }
38
39 bool can_undo() const { return !undo_stack_.empty(); }
40 bool can_redo() const { return !redo_stack_.empty(); }
41
42 private:
43     std::stack<action> undo_stack_;
44     std::stack<action> redo_stack_;
45 };

```

Each operation is $O(1)$ amortized (the stack push/pop is $O(1)$, though the action closures may do arbitrary work). Space is $O(n)$ where n is the number of recorded actions.

6.14 Stack for DFS Applications

Depth-first search can be implemented with an **explicit stack** rather than recursion. This avoids stack overflow on deep graphs and gives more control over traversal order. Two important applications are topological sort and cycle detection.

6.14.1 DFS on an Implicit Graph Using an Explicit Stack

Consider a grid-based maze where you can move right or down. We search from $(0, 0)$ to $(n-1, m-1)$:

```

1  #include <vector>
2  #include <stack>
3  #include <utility>
4
5  bool dfs_maze(const std::vector<std::vector<bool>>& wall,
6               int n, int m) {
7      std::stack<std::pair<int,int>> stk;
8      std::vector<std::vector<bool>> visited(n, std::vector<bool>(m));
9      stk.push({0, 0});
10     visited[0][0] = true;
11
12     const int dr[] = {1, 0}; // down, right
13     const int dc[] = {0, 1};
14
15     while (!stk.empty()) {
16         auto [r, c] = stk.top();
17         stk.pop();
18         if (r == n - 1 && c == m - 1) return true;

```

```

19
20     for (int d = 0; d < 2; ++d) {
21         int nr = r + dr[d], nc = c + dc[d];
22         if (nr < n && nc < m && !wall[nr][nc] &&
23             !visited[nr][nc]) {
24             visited[nr][nc] = true;
25             stk.push({nr, nc});
26         }
27     }
28 }
29 return false;
30 }

```

Time: $O(n \times m)$. Space: $O(n \times m)$ for the visited array and stack.

6.14.2 Topological Sort Using a Stack

Topological sort orders vertices of a DAG so that every edge goes from earlier to later. The algorithm uses DFS and a stack: after fully processing a vertex, push it onto the stack. The stack, read top-to-bottom, gives a valid topological order.

```

1  #include <vector>
2  #include <stack>
3
4  void dfs_topo(int u, const std::vector<std::vector<int>>& adj,
5               std::vector<bool>& visited,
6               std::stack<int>& order) {
7      visited[u] = true;
8      for (int v : adj[u]) {
9          if (!visited[v]) {
10             dfs_topo(v, adj, visited, order);
11         }
12     }
13     order.push(u);
14 }
15
16 std::vector<int> topological_sort(
17     const std::vector<std::vector<int>>& adj) {
18     int n = adj.size();
19     std::vector<bool> visited(n, false);
20     std::stack<int> order;
21     for (int i = 0; i < n; ++i) {
22         if (!visited[i]) {
23             dfs_topo(i, adj, visited, order);
24         }
25     }
26     std::vector<int> result;
27     while (!order.empty()) {

```

```

28         result.push_back(order.top());
29         order.pop();
30     }
31     return result;
32 }

```

Time: $O(V + E)$. Space: $O(V)$ for the visited array and stack.

Trace for a small DAG: Edges: $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$.

```

1 DFS from 0:
2 Visit 0 -> visit 1 -> visit 3 (no unvisited neighbors)
3 Push 3. Stack: [3]
4 Back to 1: all neighbors done. Push 1. Stack: [3, 1]
5 Back to 0: visit 2 -> 3 already visited. Push 2. Stack: [3, 1, 2]
6 Back to 0: all done. Push 0. Stack: [3, 1, 2, 0]
7
8 Topological order (pop stack): 0, 2, 1, 3
9 Verification: 0->1 (0 before 1), 0->2 (0 before 2), 1->3 (1 before 3),
    2->3 (2 before 3). Valid.

```

6.14.3 Cycle Detection Using DFS

A directed graph has a cycle if and only if DFS discovers a **back edge** (an edge to a vertex currently on the recursion stack). We track three states per vertex: UNVISITED, IN_STACK (being processed), and DONE (fully processed).

```

1 #include <vector>
2
3 enum state { UNVISITED, IN_STACK, DONE };
4
5 bool has_cycle_dfs(int u, const std::vector<std::vector<int>>& adj,
6                     std::vector<state>& vis) {
7     vis[u] = IN_STACK;
8     for (int v : adj[u]) {
9         if (vis[v] == IN_STACK) return true;    // back edge -> cycle
10        if (vis[v] == UNVISITED &&
11            has_cycle_dfs(v, adj, vis))
12            return true;
13    }
14    vis[u] = DONE;
15    return false;
16 }
17
18 bool has_cycle(int n, const std::vector<std::vector<int>>& adj) {
19     std::vector<state> vis(n, UNVISITED);
20     for (int i = 0; i < n; ++i) {
21         if (vis[i] == UNVISITED && has_cycle_dfs(i, adj, vis))
22             return true;

```



```

23     }
24     return false;
25 }

```

Time: $O(V + E)$. Space: $O(V)$ for the state array and implicit recursion stack.

The explicit-stack DFS variant (used in the maze example above) can also detect cycles by tracking which vertices are “on the current path” using a parallel stack of states. However, the recursive three-color approach shown above is the standard and clearest solution for cycle detection in directed graphs.

6.15 Common Bugs and Pitfalls

- **Pop on empty stack** — undefined behavior (or exception) if you call `pop()` or `top()` on an empty stack. Always guard with `empty()`.
- **Incorrect `top()` semantics** — many textbooks define `pop()` as “remove and return,” but C++ convention separates `top()` (read) and `pop()` (remove). Confusing these leads to lost data.
- **Forgetting the amortization** — `array_stack::push` is $O(1)$ amortized, but individual pushes can be $O(n)$ when the vector grows. In hard real-time systems, use linked stack instead.
- **Stack overflow in recursion** — the call stack is a real stack with finite size. Deep recursion (e.g., recursive DFS on a large graph) causes stack overflow. Use an explicit stack instead.
- **Memory leak in linked stack** — with raw pointers, popping without deleting the node leaks memory. Our `linked_stack` uses `unique_ptr`, which is safe, but manually written linked stacks are a common source of leaks in production code.

6.16 Summary

1. A **stack** is a LIFO structure — insertions and deletions at one end (the top).
2. **Array-based stacks** use a dynamic array for $O(1)$ amortized operations.
3. **Linked stacks** use a singly linked list for $O(1)$ worst-case operations.
4. Industrial applications include the system call stack, spreadsheet expression engines (Excel, Google Sheets), stack-based VMs (JVM), and undo/redo systems.
5. `std::stack` is a container adapter that wraps any sequence container.

6.17 Interview FAQs

1. **Implement a stack that supports `getMin()` in $O(1)$ time.**

Use two stacks: the main stack and a minimum stack. On `push(x)`, push x onto the main stack. Push x onto the min stack only if $x \leq$ top of min stack. On `pop`, pop from both if the values match. `getMin()` returns the top of the min stack. Space: $O(n)$.

```

1 class min_stack {
2     std::stack<int> data_, min_;
3 public:
4     void push(int x) {
5         data_.push(x);
6         if (min_.empty() || x <= min_.top()) min_.push(x);
7     }
8     int pop() {
9         int x = data_.top(); data_.pop();
10        if (x == min_.top()) min_.pop();
11        return x;
12    }
13    int getMin() const { return min_.top(); }
14 };

```

Variant: “Do it with $O(1)$ extra space” – store the difference between the current element and the minimum in the main stack.

1. Why do HP calculators, Lua, Forth, and the JVM bytecode use postfix notation instead of infix?

Postfix eliminates parentheses entirely — the order of operations is determined by the operator position, not precedence rules. This makes parsing trivial: scan left to right, push operands, pop and apply on operators. No operator precedence table, no associativity rules, no shunting-yard algorithm needed. HP calculators used RPN because it reduced the complexity of the calculator’s parser (1970s hardware constraints). The JVM uses postfix bytecodes because the stack machine is trivial to implement and verify. The tradeoff: postfix is harder for humans to read, which is why no mainstream programming language uses it for source code.

1. What are the limitations of the shunting-yard algorithm, and what do production parsers use instead?

The basic shunting-yard handles $+, -, *, /$ with left associativity but struggles with: (1) right-associative operators ($^, =$); (2) function calls with multiple arguments; (3) unary operators ($-x, !x$); (4) custom precedence levels. Production parsers use recursive descent or Pratt parsing (top-down operator precedence), which handles all of these naturally. Pratt parsing assigns a binding power to each token type and uses a simple precedence climbing loop — it’s what most C/C++ compilers actually use for expression parsing. The shunting-yard is a historical curiosity; Pratt parsing is the practical choice.

1. Why does the stack-based parentheses checker fail for expressions with nested brackets of different types, and how do you fix it?

The basic algorithm pushes opening brackets and pops on closing brackets, but it doesn't verify *type matching* — `( [ ] )` would be accepted as balanced. The fix: instead of just pushing the bracket character, push it and check that the popped character matches the expected closing type. For `(`, expect `)`; for `[`, expect `]`; for `{`, expect `}`. This is the actual production-quality algorithm used in compilers and linters. The deeper insight: the stack tracks *nesting context* — each level must close in the reverse order it was opened, which is exactly the LIFO property of stacks. This is why stacks are the natural data structure for parsing nested structures (JSON, XML, SQL, arithmetic expressions).

1. Implement a queue using two stacks.

Amortized $O(1)$ per operation. Maintain two stacks: `inbox` (for pushes) and `outbox` (for pops). On `pop/front`, if `outbox` is empty, transfer all elements from `inbox` to `outbox` (reversing the order). Each element is moved at most once, giving amortized $O(1)$ per operation.

```

1 class queue_two_stacks {
2     std::stack<int> inbox_, outbox_;
3 public:
4     void push(int x) { inbox_.push(x); }
5     int pop() {
6         if (outbox_.empty()) {
7             while (!inbox_.empty()) { outbox_.push(inbox_.top()); inbox_.
pop(); }
8         }
9         int x = outbox_.top(); outbox_.pop();
10        return x;
11    }
12    int front() {
13        if (outbox_.empty()) {
14            while (!inbox_.empty()) { outbox_.push(inbox_.top()); inbox_.
pop(); }
15        }
16        return outbox_.top();
17    }
18 };

```

Follow-up: “Implement it with $O(1)$ worst-case” – requires interleaving, not just two stacks.

1. What is the maximum depth of valid parentheses?

Scan left to right, tracking the current depth and the maximum seen. On `(`, increment depth. On `)`, decrement depth (if depth > 0 , else reset the start position). Track the maximum depth seen. Time $O(n)$, space $O(1)$.

```

1 int max_depth_parens(std::string_view s) {
2     int depth = 0, max_d = 0;
3     for (char c : s) {
4         if (c == '(') { ++depth; max_d = std::max(max_d, depth); }

```

```

5         else if (c == ')') { if (depth > 0) --depth; else depth = 0; }
6     }
7     return max_d;
8 }

```

Variant: “Length of the longest valid substring” – harder, requires tracking the last invalid position.

1. How does the call stack work and why does deep recursion cause a stack overflow?

Each function call allocates a stack frame containing local variables, parameters, and the return address. The call stack has a fixed size (typically 1–8 MB). Deep recursion (e.g., 10^6 recursive calls) exhausts this space. Tail-recursive functions can be optimized by the compiler into loops (no new frame), but non-tail recursion cannot. Solutions: increase stack size (`-Wl,-stack-size`), convert to iteration, or use an explicit stack data structure.

Real-world: JVM uses a separate stack per thread; default 512 KB. Python limits recursion to 1000 by default (`sys.setrecursionlimit`).

1. Design a stack that supports push, pop, top, and getMin in O(1) worst-case with O(1) extra space.

Store the minimum as an encoding: when pushing a new minimum, push $2 * val - prev_min$ instead of val . On pop, if the popped value is less than the current minimum, the previous minimum is recovered from the encoded value. This uses O(1) extra space (just the minimum variable) but works only for integer stacks.

```

1 class min_stack_o1 {
2     std::stack<long long> data_;
3     long long min_ = LLONG_MAX;
4 public:
5     void push(int x) {
6         if (data_.empty()) { data_.push(x); min_ = x; }
7         else if (x >= min_) { data_.push(x); }
8         else { data_.push(2LL * x - min_); min_ = x; }
9     }
10    int pop() {
11        long long top = data_.top(); data_.pop();
12        if (top < min_) { long long prev = min_; min_ = 2 * min_ - top;
13        return prev; }
14        return top;
15    }
16    int getMin() const { return min_; }
17 };

```

This is LeetCode 155 with the O(1) space optimization – a very popular interview question.

1. Use a stack to solve the “next greater element” problem.

For each element in the array, find the next element that is greater. Scan right to left, maintaining a stack of elements in decreasing order. For each element, pop elements from the stack that are smaller — the top of the remaining stack is the next greater element. Time $O(n)$, space $O(n)$.

```

1 std::vector<int> next_greater(const std::vector<int>& nums) {
2     std::vector<int> result(nums.size(), -1);
3     std::stack<int> st;
4     for (int i = nums.size() - 1; i >= 0; --i) {
5         while (!st.empty() && st.top() <= nums[i]) st.pop();
6         if (!st.empty()) result[i] = st.top();
7         st.push(nums[i]);
8     }
9     return result;
10 }
```

Variants: “Next smaller element”, “Daily temperatures” (LeetCode 739), “Stock span problem”.

1. How does `std::stack` differ from a raw container, and when would you NOT use it?

`std::stack` is a container adapter that restricts access to `top`/`push`/`pop`. It prevents accidental misuse (e.g., iterating the underlying container). However, you would NOT use it when you need: (a) iteration, (b) random access, (c) `size()` (deque-backed `std::stack` has `size()`, but the standard doesn’t require it for all backends), or (d) access to the underlying container for optimization. In performance-critical code, use a `std::vector` directly with a manual size index.

6.18 Exercises

6.18.1 Drill

- Trace the call stack for a recursive `fibonacci(5)` computation. Show the stack frame contents (return address, parameters) at each push and pop. How deep does the stack grow?
- Convert the infix expression `price > 100 AND (quantity < 10 OR discount > 0.2)` to postfix using the shunting-yard algorithm. Treat `AND` and `OR` as binary operators with precedence: `NOT` (3), `AND` (2), `OR` (1).
- Evaluate `"5 3 + 7 2 - ${}\times $"` using a stack. Show the stack state after each token.
- A production web server receives requests at 1000 req/s. Each request enters a middleware pipeline (stack of handlers). If each handler takes 50 us and the pipeline is 10 handlers deep, what is the minimum time before the first byte is sent? Where does the LIFO order matter for middleware semantics?

1. A Just-In-Time compiler's register allocator uses a stack to spill registers. If 6 registers are available, how many spills occur when evaluating $a + b * c / d - e$ with the usual precedence? Show the stack states.
1. Implement a stack-based calculator that evaluates expressions in standard (infix) notation. Use two stacks: one for operands and one for operators. Handle parentheses, operator precedence (+, - at level 1; *, / at level 2), and unary minus. Test with $(3 + 4) * (2 - 9) / 5$.
1. Given a string of angle brackets $<><><><>$ representing open (<) and close (>) tags, use a stack to find the length of the longest contiguous valid (balanced) segment. Show the stack state at each step.

6.18.2 Application

1. **Min-stack for stock price monitoring.** You are implementing a real-time trading system that must report the minimum stock price seen in $O(1)$ time (without scanning history). Implement `array_stack` with an auxiliary stack that tracks the current minimum. Feed it a stream of 10,000 NASDAQ price ticks. Verify that `get_min()` returns the correct minimum at every point.
1. **Spreadsheet expression engine.** Implement a postfix evaluator that supports cell references, arithmetic (+, -, ×, /, ^), and functions (SUM, AVG, MIN, MAX). Test with the expression `SUM(A1:A3) * (B1 + B2)`. Hint: resolve cell references against a symbol table before pushing values.
1. **Chromium's browser back/forward.** Implement a dual-stack navigation manager as used in Chrome's tab history. Support `visit(url, is_new_page)` (pushes onto back stack, clears forward stack), `back()` (moves current to forward stack), and `forward()`. Handle the edge case of no history.
1. **V8 JavaScript call stack.** V8 maintains a stack of execution contexts (global, function, eval). Simulate this: each function call creates a `context` object with local variables and a return address. Implement a trace that shows the context stack during a recursive factorial computation. Detect and report stack overflow at depth 1000.
1. **Redis transaction stack.** Redis uses a stack to batch commands in a MULTI/EXEC transaction. Implement a `TransactionStack` that supports `queue(command)`, `execute()` (pops all and runs them atomically), and `discard()` (clears without executing). Handle nested transactions by stacking transaction states.
1. **Expression syntax checker (ESLint-style).** Implement a balanced-parentheses checker that also validates bracket matching in source code. Extend it to detect mismatched HTML/XML tags using a stack: `<div><p>text</p></div>` is valid, `<div><p>text</div></p>` is not. Test on real-world HTML snippets from the web.

1. **Largest rectangle in a histogram.** Given an array of bar heights, find the area of the largest rectangle that fits within the histogram. Use a stack to maintain indices of bars in increasing height order. For each bar, pop taller bars and compute areas. Time $O(n)$. Test with heights 2, 1, 5, 6, 2, 3.
1. **Asteroid collision.** Given an array of asteroids (positive = rightward, negative = leftward), simulate collisions using a stack. When a rightward asteroid meets a leftward one, the smaller explodes. If equal, both explode. Return the surviving asteroids. Test with 5, 10, -5, 8, -8, and 10, 2, -5.
1. **Decode string.** Given a string like `3[a2[c]]`, decode it to `accaccacc` using a stack. Push characters and digits; on `]`, pop until `[`, collect the substring, pop the multiplier, and push the repeated string. Handle nested encodings.
1. **Simplify path.** Given a Unix-style absolute path like `/a/./b/../../c/`, simplify it to `/c` using a stack. Push directory names on `/name`, pop on `..`, skip on `.` or empty. Test with `/home/user/Documents/../../Downloads` and `/../`.
1. **Infix to prefix converter.** Implement the two-stack infix-to-prefix algorithm described in this chapter. Test it on $A * (B + C) - D / E$ and $(A + B) * (C - D) / E$. Verify that prefix-to-inversion of the result reproduces the original expression. Handle operator precedence and right-associative exponentiation ($^$).
1. **Undo/redo text editor.** Build a complete text editor using the two-stack undo/redo pattern. Support `type(text)`, `delete(n)`, `undo()`, and `redo()`. After a sequence of 20 mixed operations, verify that the document state is correct after every undo/redo. Test that a new `type()` after `undo()` clears the redo stack.
1. **Course prerequisite ordering.** A university offers courses with prerequisites modeled as a DAG. Given a list of courses and prerequisite pairs, implement topological sort using an explicit stack (not recursion). Test with at least 10 courses. What happens when a cycle is introduced (e.g., course A requires B, B requires C, C requires A)? Detect and report the cycle.
1. **Grid pathfinding with explicit stack DFS.** Implement maze pathfinding using an explicit stack (no recursion). The grid has walls, open cells, and a goal. Support 4-directional movement (up, down, left, right). Compare the path found by stack-based DFS with the shortest path found by BFS. On what grid sizes does DFS find a path that is more than twice as long as BFS?
1. **Stack-based browser history simulator.** Simulate a full browser tab with back, forward, and visit. Track the current URL using two stacks. Add a “history log” that records every navigation event with a timestamp. After 15 operations mixing visit/back/forward, print the complete log and verify correctness. How would you extend this to support multiple tabs?

6.18.3 Research

1. **(Stack-based VMs)** JVM, CPython, and the Ethereum EVM are all stack-based machines. Write a minimal bytecode interpreter that supports PUSH, POP, ADD, SUB, MUL, DUP, SWAP, and JMP. Execute a small program that computes $(3 + 4) * 5$. Compare the bytecode size with a register-based ISA (e.g., Lua VM).
1. **(Treiber stack)** Implement the Treiber lock-free stack using `std::atomic<std::shared_ptr<node>>`. Benchmark it against `std::stack` protected by `std::mutex` on 2, 4, and 8 threads. At what contention level does the lock-free version win? Where does it lose?
1. **(Amortized analysis)** Prove that our `array_stack` has $O(1)$ amortized push cost using both the accounting method and the potential method. Then compare real-world benchmark results: measure the worst-case push latency (when the vector grows) vs the average across 10^6 pushes. How much slower is the worst case?
1. **(Stack security)** Stack buffer overflows are a classic attack vector. Research how modern systems mitigate them: stack canaries (`-fstack-protector`), ASLR, DEP/NX bits, and Control Flow Integrity (CFI). Write a report explaining how each defense works and what attack techniques they defeat.
1. **(Continuation-passing style)** Continuations represent the “rest of the computation” and can be thought of as an explicit call stack. Research how Scheme’s `call/cc` (call-with-current-continuation) works and how it relates to stack frames. Implement a trampolined factorial in C++ that avoids stack overflow by storing continuations in a heap-allocated stack.
1. **(Forth stack machine)** Forth is a stack-based language where all computation happens on two stacks: the data stack and the return stack. Write a minimal Forth interpreter that supports DUP, SWAP, DROP, +, −, *, ., and IF/ELSE/THEN. Execute a program that computes and prints the first 10 Fibonacci numbers.

6.19 References and Further Reading

- Donald E. Knuth, *The Art of Computer Programming*, Volume 1: Fundamental Algorithms. Section 2.2.1.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Section 10.1.
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Section 1.3.
- R. K. Treiber, “Systems Programming: Coping with Parallelism” — IBM Technical Report, 1986.

Queues

7.1 Introduction

❧ Complete compilable implementations of the data structures in this chapter are in `code/`.

7.2 The Queue ADT

A **queue** follows FIFO (First In, First Out): insertions at the **rear**, deletions from the **front**.

7.2.1 Operations

- **push(value)** — insert at rear
- **pop()** — remove and return the front element
- **front()** — return the front element
- **back()** — return the rear element
- **empty(), size()**

7.2.2 Applications

- Task scheduling (print queues, thread pools)
- Breadth-first search (Chapter 12)
- Buffering (I/O, streaming)
- Message passing (queues in concurrent systems)

7.3 Array-Based Queue (Circular Buffer)

A naive array queue would be inefficient: after many pushes and pops, front moves forward and space at the beginning is wasted. A **circular buffer** solves this by wrapping around.

```

1 Initial: [_, _, _, _]   front=0, rear=0
2 push 10: [10, _, _, _]  front=0, rear=1
3 push 20: [10, 20, _, _] front=0, rear=2
4 pop:      [_, 20, _, _]  front=1, rear=2   (returns 10)
```

```

5 push 30: [_, 20, 30, _] front=1, rear=3
6 push 40: [_, 20, 30, 40] front=1, rear=0 (wraps around!)

```

```

1 template <typename T>
2 class circular_queue {
3 public:
4     explicit circular_queue(size_t initial_capacity = 8)
5         : data_(std::make_unique<T[]>(initial_capacity))
6         , capacity_(initial_capacity), front_(0), size_(0) {}
7
8     void push(const T& value) {
9         if (size_ == capacity_) grow();
10        size_t pos = (front_ + size_) % capacity_;
11        data_[pos] = value;
12        ++size_;
13    }
14
15    void push(T&& value) {
16        if (size_ == capacity_) grow();
17        size_t pos = (front_ + size_) % capacity_;
18        data_[pos] = std::move(value);
19        ++size_;
20    }
21
22    void pop() {
23        if (empty()) throw std::underflow_error("queue is empty");
24        front_ = (front_ + 1) % capacity_;
25        --size_;
26    }
27
28    T& front() {
29        if (empty()) throw std::underflow_error("queue is empty");
30        return data_[front_];
31    }
32
33    const T& front() const {
34        if (empty()) throw std::underflow_error("queue is empty");
35        return data_[front_];
36    }
37
38    T& back() {
39        if (empty()) throw std::underflow_error("queue is empty");
40        return data_[(front_ + size_ - 1) % capacity_];
41    }
42
43    bool empty() const noexcept { return size_ == 0; }
44    size_t size() const noexcept { return size_; }
45 private:

```

```

46     void grow() {
47         size_t new_capacity = capacity_ * 2;
48         auto new_data = std::make_unique<T[]>(new_capacity);
49         for (size_t i = 0; i < size_; ++i) {
50             new_data[i] = std::move(data_[(front_ + i) % capacity_]);
51         }
52         front_ = 0;
53         capacity_ = new_capacity;
54         data_ = std::move(new_data);
55     }
56
57     std::unique_ptr<T[]> data_;
58     size_t capacity_, front_, size_;
59 };

```

All operations: $O(1)$ amortized.

7.4 Linked Queue

```

1  template <typename T>
2  class linked_queue {
3  public:
4      linked_queue() = default;
5
6      void push(const T& value) {
7          auto new_node = std::make_unique<node>(value);
8          if (rear_) {
9              rear_->next = std::move(new_node);
10             rear_ = rear_->next.get();
11         } else {
12             front_ = std::move(new_node);
13             rear_ = front_.get();
14         }
15         ++size_;
16     }
17
18     void pop() {
19         if (empty()) throw std::underflow_error("queue is empty");
20         front_ = std::move(front_->next);
21         if (!front_) rear_ = nullptr;
22         --size_;
23     }
24
25     T& front() {
26         if (empty()) throw std::underflow_error("queue is empty");
27         return front_->data;
28     }

```

```

29
30     bool empty() const noexcept { return size_ == 0; }
31     size_t size() const noexcept { return size_; }
32
33 private:
34     struct node {
35         T data;
36         std::unique_ptr<node> next;
37         explicit node(const T& d) : data(d) {}
38     };
39     std::unique_ptr<node> front_;
40     node* rear_ = nullptr;
41     size_t size_ = 0;
42 };

```

7.5 Applications

7.5.1 Message Queue (Apache Kafka / AWS SQS)

Message queues decouple producers from consumers. In a simplified Kafka-like publish-subscribe system, a queue holds messages that multiple consumers can process:

```

1  template <typename T>
2  class message_queue {
3  public:
4      void publish(const T& message) {
5          std::scoped_lock lock(mutex_);
6          queue_.push(message);
7          cv_.notify_one();
8      }
9
10     T consume(std::chrono::milliseconds timeout = std::chrono::seconds(30)
11     ) {
12         std::unique_lock lock(mutex_);
13         if (!cv_.wait_for(lock, timeout, [this] { return !queue_.empty();
14         })) {
15             throw std::runtime_error("consume timeout -- no messages
16             available");
17         }
18         T msg = queue_.front();
19         queue_.pop();
20         return msg;
21     }
22
23     size_t size() const {
24         std::scoped_lock lock(mutex_);
25         return queue_.size();
26     }
27 };

```

```

23     }
24
25 private:
26     linked_queue<T> queue_;
27     mutable std::mutex mutex_;
28     std::condition_variable cv_;
29 };

```

Usage in production: LinkedIn uses Apache Kafka to handle over 7 trillion messages per day. Each topic partition is an ordered, immutable queue. Twitter’s event bus processes 500 million tweets/day through message queues. The queue abstraction provides backpressure: if consumers fall behind, the queue grows (or triggers auto-scaling).

7.5.2 Web Crawler Frontier (BFS)

A web crawler maintains a frontier of URLs to visit. BFS (queue-based) ensures politeness — it visits pages in discovery order, typically used by search engines like Googlebot:

```

1 class web_crawler {
2 public:
3     web_crawler(std::string_view seed_url, size_t max_pages = 10000)
4         : frontier_(), visited_(), max_pages_(max_pages) {
5         frontier_.push(seed_url);
6     }
7
8     void crawl() {
9         while (!frontier_.empty() && visited_.size() < max_pages_) {
10             std::string url = frontier_.front(); frontier_.pop();
11             if (visited_.count(url)) continue;
12             visited_.insert(url);
13
14             std::cout << "Crawling: " << url << " ("
15                 << visited_.size() << "/" << max_pages_ << ")\n";
16
17             auto links = fetch_links(url); // HTTP GET + HTML parsing
18             for (const auto& link : links) {
19                 if (!visited_.count(link) && same_domain(url, link)) {
20                     frontier_.push(link);
21                 }
22             }
23         }
24     }
25
26     size_t pages_crawled() const { return visited_.size(); }
27
28 private:
29     std::vector<std::string> fetch_links(std::string_view url) {
30         // Simulate fetching -- in production this uses libcurl or CPPREST

```

```

31     return {};
32 }
33
34 bool same_domain(std::string_view a, std::string_view b) {
35     auto pos_a = a.find("://"), pos_b = b.find("://");
36     if (pos_a == std::string_view::npos) pos_a = 0;
37     else pos_a += 3;
38     if (pos_b == std::string_view::npos) pos_b = 0;
39     else pos_b += 3;
40     std::string_view domain_a = a.substr(pos_a, a.find('/', pos_a) -
pos_a);
41     std::string_view domain_b = b.substr(pos_b, b.find('/', pos_b) -
pos_b);
42     return domain_a == domain_b;
43 }
44
45 linked_queue<std::string> frontier_;
46 std::unordered_set<std::string> visited_;
47 size_t max_pages_;
48 };

```

Politeness: Production crawlers (Googlebot, Bingbot) add a delay between requests to the same domain — equivalent to rate-limited dequeue. The queue also respects `robots.txt`, which filters URLs before enqueueing.

7.5.3 Thread Pool Task Queue

Modern web servers (NGINX, Apache, uWSGI) use a thread pool with a shared task queue. Worker threads consume tasks from the queue as they become available:

```

1 class thread_pool {
2 public:
3     explicit thread_pool(size_t num_threads = std::thread::
hardware_concurrency()) {
4         workers_.reserve(num_threads);
5         for (size_t i = 0; i < num_threads; ++i) {
6             workers_.emplace_back([this] { worker_loop(); });
7         }
8     }
9
10    ~thread_pool() {
11        {
12            std::scoped_lock lock(mutex_);
13            done_ = true;
14        }
15        cv_.notify_all();
16        for (auto& t : workers_) t.join();
17    }

```

```

18
19     void enqueue(std::function<void()> task) {
20         std::scoped_lock lock(mutex_);
21         tasks_.push(std::move(task));
22         cv_.notify_one();
23     }
24
25 private:
26     void worker_loop() {
27         while (true) {
28             std::function<void()> task;
29             {
30                 std::unique_lock lock(mutex_);
31                 cv_.wait(lock, [this] { return done_ || !tasks_.empty();
32             });
33                 if (done_ && tasks_.empty()) return;
34                 task = tasks_.front(); tasks_.pop();
35             }
36             task(); // execute outside the lock
37         }
38
39         linked_queue<std::function<void()>> tasks_;
40         std::vector<std::thread> workers_;
41         std::mutex mutex_;
42         std::condition_variable cv_;
43         bool done_ = false;
44     };

```

Usage: NGINX uses this pattern for its thread pool module (`ngx_thread_pool`), processing hundreds of thousands of concurrent connections. uWSGI’s async mode runs a thread pool with a queue of requests, each handled by a greenlet.

7.6 Deque (Double-Ended Queue)

A deque allows push/pop at both ends. C++ provides `std::deque` — typically implemented as a sequence of fixed-size blocks.

```

1 std::deque<int> dq;
2 dq.push_back(10); // [10]
3 dq.push_front(5); // [5, 10]
4 dq.push_back(20); // [5, 10, 20]
5 dq.pop_front(); // [10, 20]
6 dq.pop_back(); // [10]

```

`std::deque` offers $O(1)$ operations at both ends and $O(1)$ random access. It is the default container for `std::stack` and `std::queue`.

7.7 STL Connection: `std::queue`

```

1 std::queue<int> q; // uses std::deque
2 std::queue<int, std::list<int>> ql; // list-based
3
4 q.push(10);
5 q.push(20);
6 int x = q.front(); // 10
7 q.pop(); // removes 10

```

7.8 Circular Queue: Full Implementation

The circular queue (ring buffer) uses modular arithmetic so that when the rear or front index reaches the end of the array, it wraps around to the beginning. This avoids the "drift" problem of a naive array queue, where elements slide forward and wasted space accumulates at the front.

```

1 // A fixed-capacity circular queue.
2 // Uses one wasted slot to distinguish full from empty.
3 // When (rear_ + 1) % cap == front_, the queue is full.
4 // When front_ == rear_, the queue is empty.
5 template <typename T>
6 class circular_queue_v2 {
7 public:
8     explicit circular_queue_v2(int capacity)
9         : cap_(capacity + 1), data_(new T[capacity + 1]),
10         front_(0), rear_(0) {}
11
12     ~circular_queue_v2() { delete[] data_; }
13
14     // Disallow copying for simplicity.
15     circular_queue_v2(const circular_queue_v2&) = delete;
16     circular_queue_v2& operator=(const circular_queue_v2&) = delete;
17
18     void enqueue(const T& value) {
19         if (is_full()) {
20             throw std::overflow_error("queue is full");
21         }
22         data_[rear_] = value;
23         rear_ = (rear_ + 1) % cap_;
24     }
25
26     T dequeue() {
27         if (is_empty()) {
28             throw std::underflow_error("queue is empty");
29         }
30         T value = data_[front_];

```



```

31     front_ = (front_ + 1) % cap_;
32     return value;
33 }
34
35 T& front() {
36     if (is_empty()) {
37         throw std::underflow_error("queue is empty");
38     }
39     return data_[front_];
40 }
41
42 bool is_empty() const { return front_ == rear_; }
43
44 bool is_full() const {
45     return (rear_ + 1) % cap_ == front_;
46 }
47
48 int size() const {
49     return (rear_ - front_ + cap_) % cap_;
50 }
51
52 private:
53     int cap_;
54     T* data_;
55     int front_;    // index of the first element
56     int rear_;     // index of the NEXT insertion point
57 };
58
59 // Driver: exercise all operations.
60 int main() {
61     circular_queue_v2<int> q(5);    // capacity = 5
62
63     q.enqueue(10);
64     q.enqueue(20);
65     q.enqueue(30);
66     std::cout << "front: " << q.front() << "\n";    // 10
67     std::cout << "size:  " << q.size()    << "\n";    // 3
68
69     q.dequeue();
70     q.enqueue(40);
71     q.enqueue(50);
72     q.enqueue(60);    // now full (capacity = 5)
73     std::cout << "full: " << q.is_full() << "\n";    // 1
74
75     // Demonstrates wrap-around: indices wrap to 0.
76     while (!q.is_empty()) {
77         std::cout << q.dequeue() << " ";
78     }

```

```

79     std::cout << "\n"; // 20 30 40 50 60
80     return 0;
81 }

```

Key design decisions:

- We allocate `capacity + 1` slots and keep one slot unused. This lets us distinguish full from empty using only the two index pointers (no extra counter needed).
- The `front_` index points to the first element; `rear_` points to the *next* insertion position. Empty means `front_ == rear_`. Full means the next position for `rear_` would collide with `front_`.
- The `size()` function computes `(rear_ - front_ + cap_) % cap_`, which correctly handles the case where `rear_` has wrapped past 0 while `front_` has not.

Alternative: using a count variable. Instead of wasting one slot, you can track `count_` alongside `front_` and `rear_`. Full is `count_ == cap_`; empty is `count_ == 0`. This uses every slot but adds one integer of overhead.

Wrap-around arithmetic. The expression `(index + 1) % capacity` is the heart of the circular buffer. When `index` reaches `capacity - 1`, adding 1 gives `capacity`, and modulo brings it back to 0. This is the same mechanism used in OS ring buffers, network packet queues, and audio streaming buffers.

7.9 Double-Ended Queue (Deque)

A **deque** (double-ended queue) supports insertion and deletion at both ends in $O(1)$. It generalizes both the stack and the queue. The standard library provides `std::deque`, but understanding the internals is important for interview and system-design purposes.

7.9.1 Deque ADT

- **push_front(value)** – insert at the front
- **push_back(value)** – insert at the rear
- **pop_front()** – remove and return the front element
- **pop_back()** – remove and return the rear element
- **front(), back()** – access without removal
- **empty(), size()**

All operations are $O(1)$.

7.9.2 Implementation 1: Doubly Linked List

A doubly linked list naturally supports $O(1)$ insertion/deletion at both ends if we maintain `head` and `tail` pointers.

```

1  template <typename T>
2  class deque_linked {
3  public:
4      deque_linked() = default;
5
6      ~deque_linked() {
7          while (head_) {
8              node* temp = head_;
9              head_ = head_>next;
10             delete temp;
11         }
12     }
13
14     void push_front(const T& value) {
15         node* n = new node(value);
16         n->next = head_;
17         if (head_) head_>prev = n;
18         head_ = n;
19         if (!tail_) tail_ = head_;
20         ++sz_;
21     }
22
23     void push_back(const T& value) {
24         node* n = new node(value);
25         if (tail_) {
26             tail_>next = n;
27             n->prev = tail_;
28         } else {
29             head_ = n;
30         }
31         tail_ = n;
32         ++sz_;
33     }
34
35     T pop_front() {
36         if (!head_) throw std::underflow_error("empty");
37         T val = head_>data;
38         node* temp = head_;
39         head_ = head_>next;
40         if (head_) head_>prev = nullptr;
41         else tail_ = nullptr;
42         delete temp;
43         --sz_;
44         return val;

```

```

45     }
46
47     T pop_back() {
48         if (!tail_) throw std::underflow_error("empty");
49         T val = tail_>data;
50         node* temp = tail_;
51         tail_ = tail_>prev;
52         if (tail_) tail_>next = nullptr;
53         else head_ = nullptr;
54         delete temp;
55         --sz_;
56         return val;
57     }
58
59     bool empty() const { return sz_ == 0; }
60     int size() const { return sz_; }
61
62 private:
63     struct node {
64         T data;
65         node* prev = nullptr;
66         node* next = nullptr;
67         explicit node(const T& d) : data(d) {}
68     };
69     node* head_ = nullptr;
70     node* tail_ = nullptr;
71     int sz_ = 0;
72 };

```

Pros: simple, exact capacity, no wasted slots. **Cons:** each node is a separate heap allocation – poor cache locality, allocator overhead.

7.9.3 Implementation 2: Circular Array

A circular array deque uses a dynamic array with `front` and `rear` indices that can move in both directions. The array grows when full.

```

1  template <typename T>
2  class deque_circular {
3  public:
4      explicit deque_circular(int cap = 4)
5          : cap_(cap), data_(new T[cap]), front_(0), sz_(0) {}
6
7      ~deque_circular() { delete[] data_; }
8
9      void push_front(const T& value) {
10         if (sz_ == cap_) grow();
11         front_ = (front_ - 1 + cap_) % cap_;

```

```

12     data_[front_] = value;
13     ++sz_;
14 }
15
16 void push_back(const T& value) {
17     if (sz_ == cap_) grow();
18     int rear = (front_ + sz_) % cap_;
19     data_[rear] = value;
20     ++sz_;
21 }
22
23 T pop_front() {
24     if (sz_ == 0) throw std::underflow_error("empty");
25     T val = data_[front_];
26     front_ = (front_ + 1) % cap_;
27     --sz_;
28     return val;
29 }
30
31 T pop_back() {
32     if (sz_ == 0) throw std::underflow_error("empty");
33     int rear = (front_ + sz_ - 1) % cap_;
34     T val = data_[rear];
35     --sz_;
36     return val;
37 }
38
39 T& operator[](int idx) {
40     return data_[(front_ + idx) % cap_];
41 }
42
43 bool empty() const { return sz_ == 0; }
44 int size() const { return sz_; }
45
46 private:
47     void grow() {
48         int new_cap = cap_ * 2;
49         T* new_data = new T[new_cap];
50         for (int i = 0; i < sz_; ++i) {
51             new_data[i] = data_[(front_ + i) % cap_];
52         }
53         delete[] data_;
54         data_ = new_data;
55         cap_ = new_cap;
56         front_ = 0;
57     }
58
59     int cap_;

```

```

60     T* data_;
61     int front_;
62     int sz_;
63 };

```

Comparison: `std::deque` is typically implemented as a *sequence of fixed-size blocks* (not a single circular array). This avoids large contiguous allocations while still giving $O(1)$ push at both ends and $O(1)$ random access. The block-map approach is a middle ground between a single circular buffer (which must copy everything on grow) and a linked list (which has poor cache behavior).

7.9.4 Applications

Sliding Window Maximum. Given an array and window size k , find the maximum in every contiguous window. A monotonic deque maintains candidates in decreasing order. For each new element, pop smaller elements from the back; the front of the deque is always the window maximum. Total time: $O(n)$.

```

1  // Returns the maximum in every window of size k.
2  std::vector<int> sliding_window_max(
3      const std::vector<int>& nums, int k)
4  {
5      std::deque<int> dq; // stores indices
6      std::vector<int> result;
7
8      for (int i = 0; i < (int)nums.size(); ++i) {
9          // Remove indices outside the window.
10         while (!dq.empty() && dq.front() <= i - k) {
11             dq.pop_front();
12         }
13         // Remove smaller elements from the back.
14         while (!dq.empty() && nums[dq.back()] <= nums[i]) {
15             dq.pop_back();
16         }
17         dq.push_back(i);
18         if (i >= k - 1) {
19             result.push_back(nums[dq.front()]);
20         }
21     }
22     return result;
23 }

```

Palindrome checking. A deque can check palindromes in $O(n)$: push all characters onto a deque, then compare `pop_front()` with `pop_back()` until the deque is empty or a mismatch is found. This requires no random access.

Other applications: Browser forward/backward history (deque of pages), undo/redo (deque of states), work-stealing schedulers (each thread owns a deque; idle threads steal from the back), imple-

menting both stack and queue from a single data structure.

7.10 Queue Using Two Stacks

A FIFO queue can be implemented using two stacks. The key insight: one stack handles incoming elements (push stack), and the other handles outgoing elements (pop stack). When the pop stack is empty, we transfer all elements from the push stack, reversing their order so the oldest element ends up on top.

```
1 // Amortized O(1) queue using two stacks.
2 // Each element is pushed and popped at most twice.
3 template <typename T>
4 class queue_via_stacks {
5 public:
6     void push(const T& value) {
7         in_stack_.push(value);
8     }
9
10    // Amortized O(1): each element moves from in_stack_
11    // to out_stack exactly once over the lifetime of the queue.
12    T pop() {
13        if (out_stack_.empty()) {
14            transfer();
15        }
16        if (out_stack_.empty()) {
17            throw std::underflow_error("queue is empty");
18        }
19        T val = out_stack_.top();
20        out_stack_.pop();
21        return val;
22    }
23
24    T front() {
25        if (out_stack_.empty()) {
26            transfer();
27        }
28        return out_stack_.top();
29    }
30
31    bool empty() const {
32        return in_stack_.empty() && out_stack_.empty();
33    }
34
35    int size() const {
36        return in_stack_.size() + out_stack_.size();
37    }
38 }
```

```

39 private:
40     void transfer() {
41         while (!in_stack_.empty()) {
42             out_stack_.push(in_stack_.top());
43             in_stack_.pop();
44         }
45     }
46
47     std::stack<T> in_stack_;    // for push
48     std::stack<T> out_stack_;  // for pop
49 };
50
51 // Exercise all operations.
52 int main() {
53     queue_via_stacks<int> q;
54     q.push(1);
55     q.push(2);
56     q.push(3);
57     std::cout << q.pop() << "\n"; // 1 (FIFO)
58     q.push(4);
59     std::cout << q.pop() << "\n"; // 2
60     std::cout << q.pop() << "\n"; // 3
61     std::cout << q.pop() << "\n"; // 4
62     return 0;
63 }

```

Amortized analysis. Each element is pushed onto `in_stack_` once and later transferred (popped from `in_stack_`, pushed onto `out_stack_`) once. Then it is popped from `out_stack_` once. Total work per element: 3 stack operations. Since each stack operation is $O(1)$, the amortized cost per element is $O(1)$. A single `pop()` call can be $O(n)$ in the worst case (when a transfer is needed), but this is an infrequent operation amortized over many calls.

Alternative: push $O(n)$, pop $O(1)$. The reverse approach keeps all elements in sorted order in one stack. On push, transfer everything to the auxiliary stack, push the new element onto the original stack, then transfer back. This is less practical but occasionally appears in interviews.

7.11 Priority Queue Applications

A **priority queue** is a data structure where each element has a priority. The element with the highest priority is removed first (regardless of insertion order). C++ provides `std::priority_queue`, typically implemented as a binary heap.

```

1 // std::priority_queue is a max-heap by default.
2 std::priority_queue<int> max_pq;
3 max_pq.push(30);
4 max_pq.push(10);
5 max_pq.push(20);

```



```

6 std::cout << max_pq.top() << "\n"; // 30
7 max_pq.pop();
8 std::cout << max_pq.top() << "\n"; // 20
9
10 // Min-heap: use std::greater.
11 std::priority_queue<int, std::vector<int>,
12                     std::greater<int>>> min_pq;
13 min_pq.push(30);
14 min_pq.push(10);
15 min_pq.push(20);
16 std::cout << min_pq.top() << "\n"; // 10

```

7.11.1 Dijkstra's Shortest-Path Algorithm

Dijkstra's algorithm finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights. A min-priority queue (keyed by current distance) gives an efficient implementation: $O((V + E) \log V)$.

Worked example. Consider a graph with vertices A through E and edges:

```

A --1--> B --2--> D
A --4--> C --1--> D
B --3--> C --5--> E
D --1--> E

```

Trace (Dijkstra from A):

Step	PQ (dist, v)	Visit	Update neighbors
0	(0, A) (inf, B..E)	A	B: 0+1=1, C: 0+4=4
1	(1, B) (4, C) (inf, D..E)	B	C: min(4, 1+3)=4, D: 1+2=3
2	(3, D) (4, C) (inf, E)	D	E: 3+1=4, C: min(4, 3+?)=4
3	(4, C) (4, E)	C	E: min(4, 4+5)=4
4	(4, E)	E	(no neighbors to relax)
--	done: A=0, B=1, D=3, C=4, E=4		

```

1 // Dijkstra using std::priority_queue.
2 // Graph: adjacency list, (neighbor, weight) pairs.
3 void dijkstra(
4     const std::vector<std::vector<std::pair<int,int>>>& adj,
5     int source, int n)
6 {
7     // Min-priority queue: (distance, vertex).
8     using pii = std::pair<int,int>;
9     std::priority_queue<pii, std::vector<pii>,
10                        std::greater<pii>> pq;

```

```

11
12     std::vector<int> dist(n, INT_MAX);
13     std::vector<int> prev(n, -1);
14     dist[source] = 0;
15     pq.push({0, source});
16
17     while (!pq.empty()) {
18         auto [d, u] = pq.top();
19         pq.pop();
20         if (d > dist[u]) continue; // stale entry
21
22         for (auto [v, w] : adj[u]) {
23             if (dist[u] + w < dist[v]) {
24                 dist[v] = dist[u] + w;
25                 prev[v] = u;
26                 pq.push({dist[v], v});
27             }
28         }
29     }
30
31     for (int i = 0; i < n; ++i) {
32         std::cout << i << ": dist=" << dist[i]
33             << " path=";
34         // Reconstruct path.
35         for (int v = i; v != -1; v = prev[v]) {
36             std::cout << v << " ";
37         }
38         std::cout << "\n";
39     }
40 }

```

Why a priority queue? Without it, Dijkstra requires scanning all vertices to find the unvisited minimum, costing $O(V^2)$. The priority queue reduces this to $O(\log V)$ per extraction, yielding $O((V + E) \log V)$ total. For sparse graphs ($E = O(V)$), this is much faster.

7.11.2 Job Scheduling with Priorities

In operating systems and cloud computing, processes or tasks have priorities. A priority queue ensures the highest-priority task is always processed next. Consider a simple task scheduler:

```

1 struct task {
2     std::string name;
3     int priority;        // higher = more urgent
4     int deadline;        // time units
5 };
6
7 // Max-heap ordered by priority.
8 struct task_cmp {

```

```

 9      bool operator()(const task& a, const task& b) {
10          return a.priority < b.priority;
11      }
12  };
13
14  void schedule_tasks() {
15      std::priority_queue<task, std::vector<task>,
16                          task_cmp> scheduler;
17
18      scheduler.push({"backup",      2, 100});
19      scheduler.push({"critical_db", 5, 10});
20      scheduler.push({"log_rotate",  1, 50});
21      scheduler.push({"user_request",4, 5});
22
23      while (!scheduler.empty()) {
24          task t = scheduler.top();
25          scheduler.pop();
26          std::cout << "Running: " << t.name
27                  << " (priority=" << t.priority
28                  << ")\n";
29      }
30      // Output: critical_db, user_request, backup, log_rotate
31  }

```

Real-world systems: Linux’s CFS (Completely Fair Scheduler) uses a red-black tree (not a heap) to manage process priorities. Kubernetes uses a priority queue for pod scheduling: higher-priority pods are admitted to the cluster first. Cloud task queues (AWS SQS with FIFO + priority, Celery with priority lanes) provide priority-based message processing.

7.12 BFS Applications

Breadth-first search (BFS) uses a queue to explore a graph level by level. Beyond basic traversal, BFS solves several important problems.

7.12.1 Word Ladder Problem

Given two words (`beginWord` and `endWord`) and a dictionary of valid words, find the shortest transformation sequence from `beginWord` to `endWord`, where each intermediate word must be in the dictionary and differs by exactly one letter.

Worked trace. Find the shortest ladder from COLD to WARM, `dictionary = {COLD, CORD, CARD, WARD, WARM, WORE, WORM}`.

BFS explores in order of distance from COLD.

Level 0: COLD

Level 1: CORD (change L→R)

Level 2: CARD (CORD: O→A)

Level 3: WARD (CARD: C→W)

Level 4: WARM (WARD: D→M)

Answer: COLD → CORD → CARD → WARD → WARM (length 5)

```

1 int word_ladder(const std::string& begin,
2               const std::string& end,
3               const std::unordered_set<std::string>& dict)
4 {
5     if (begin == end) return 0;
6
7     std::queue<std::pair<std::string, int>> bfs;
8     std::unordered_set<std::string> visited;
9     bfs.push({begin, 1});
10    visited.insert(begin);
11
12    while (!bfs.empty()) {
13        auto [word, steps] = bfs.front();
14        bfs.pop();
15
16        // Try changing each character.
17        for (int i = 0; i < (int)word.size(); ++i) {
18            char original = word[i];
19            for (char c = 'a'; c <= 'z'; ++c) {
20                if (c == original) continue;
21                word[i] = c;
22                if (word == end) return steps + 1;
23                if (dict.count(word) &&
24                    !visited.count(word))
25                {
26                    visited.insert(word);
27                    bfs.push({word, steps + 1});
28                }
29            }
30            word[i] = original; // restore
31        }
32    }
33    return 0; // no ladder found
34 }

```

Complexity. Let N be the dictionary size and L the word length. Each state has $26L$ neighbors. BFS visits each state at most once. Time: $O(N \cdot L \cdot 26)$. Space: $O(N)$ for the visited set and queue.

7.12.2 Bipartiteness Checking

A graph is **bipartite** if its vertices can be partitioned into two sets such that every edge connects vertices in different sets. Equivalently, a graph is bipartite if and only if it contains no odd-length cycles.

BFS can check bipartiteness in $O(V + E)$: color the source vertex 0, then alternate colors (0 and 1) at each BFS level. If you ever find an edge connecting two vertices of the same color, the graph is not bipartite.

```

1 bool is_bipartite(
2     const std::vector<std::vector<int>>& adj, int n)
3 {
4     std::vector<int> color(n, -1); // -1 = uncolored
5
6     for (int start = 0; start < n; ++start) {
7         if (color[start] != -1) continue; // already visited
8
9         // BFS from this component.
10        std::queue<int> q;
11        q.push(start);
12        color[start] = 0;
13
14        while (!q.empty()) {
15            int u = q.front();
16            q.pop();
17            for (int v : adj[u]) {
18                if (color[v] == -1) {
19                    color[v] = color[u] ^ 1; // flip color
20                    q.push(v);
21                } else if (color[v] == color[u]) {
22                    return false; // same color = not bipartite
23                }
24            }
25        }
26    }
27    return true;
28 }
```

Applications: Graph 2-coloring, checking if a graph is a tree (always bipartite), matching problems (maximum bipartite matching), scheduling problems where items fall into two categories (jobs and machines, courses and time slots), and detecting odd cycles in social networks.

7.13 Sliding Window Maximum (Deque Approach)

The **sliding window maximum** problem asks: given an array $A[0 \dots n-1]$ and a window size k , find the maximum in every contiguous window $A[i \dots i+k-1]$ for $i = 0, 1, \dots, n-k$. A naive approach scans each window in $O(k)$, giving $O(nk)$ total. The deque-based solution achieves $O(n)$.

7.13.1 Key Idea

Maintain a *monotonic deque* (double-ended queue) of *indices*. The deque stores indices of elements in **decreasing order of value**—the front always holds the index of the current window maximum. Two invariants are maintained:

1. **Monotonicity:** For any two indices $i < j$ in the deque, $A[i] \geq A[j]$. Before pushing index j , pop from the back all indices whose values are $\leq A[j]$ —they can never be the maximum while j is in the window.
2. **Freshness:** The front index must lie within the current window. Before reading the maximum, pop from the front any index i where $i \leq \text{current_left} - 1$ (i.e., it has slid out of the window).

7.13.2 Worked Trace

Consider $A = [2, 1, 5, 3, 6, 4, 8, 7]$ with $k = 3$. We process each element (index shown in parentheses):

Step	Element	Action	Deque (indices)	Max
1	2	push 0	[0]	--
2	1	push 1 (1 < A[0]=2, stays)	[0, 1]	--
3	5	pop back 1, pop back 0; push 2	[2]	A[2]=5
4	3	push 3 (3 < A[2]=5, stays)	[2, 3]	A[2]=5
5	6	pop back 3, pop back 2; push 4	[4]	A[4]=6
6	4	push 5 (4 < A[4]=6, stays)	[4, 5]	A[4]=6
7	8	pop back 5, pop back 4; push 6	[6]	A[6]=8
8	7	push 7 (7 < A[6]=8, stays)	[6, 7]	A[6]=8

Windows and results:

```
Window [2, 1, 5] --> max = 5
Window [1, 5, 3] --> max = 5
Window [5, 3, 6] --> max = 6
Window [3, 6, 4] --> max = 6
Window [6, 4, 8] --> max = 8
Window [4, 8, 7] --> max = 8
```

7.13.3 C++ Implementation

```
1 #include <iostream>
2 #include <vector>
3 #include <deque>
4
5 // Returns the maximum of each sliding window of size k in A.
6 // Time: O(n), Space: O(k)
```

```

7  std::vector<int> slidingWindowMax(const std::vector<int>& A, int k) {
8      std::deque<int> dq;    // stores indices, values decreasing
9      std::vector<int> result;
10
11     for (int i = 0; i < (int)A.size(); i++) {
12         // 1. Remove indices that fell out of the window
13         while (!dq.empty() && dq.front() <= i - k)
14             dq.pop_front();
15
16         // 2. Remove back indices with values <= A[i]
17         while (!dq.empty() && A[dq.back()] <= A[i])
18             dq.pop_back();
19
20         // 3. Push current index
21         dq.push_back(i);
22
23         // 4. Record max once we have a full window
24         if (i >= k - 1)
25             result.push_back(A[dq.front()]);
26     }
27     return result;
28 }
29
30 int main() {
31     std::vector<int> A = {2, 1, 5, 3, 6, 4, 8, 7};
32     int k = 3;
33     std::vector<int> ans = slidingWindowMax(A, k);
34
35     std::cout << "Array: ";
36     for (int x : A) std::cout << x << " ";
37     std::cout << "\nk = " << k << "\nMaxima: ";
38     for (int x : ans) std::cout << x << " ";
39     std::cout << std::endl;
40     return 0;
41 }

```

Why $O(n)$? Each index is pushed onto the deque exactly once and popped at most once (from the front or the back). Therefore the total number of deque operations across all n iterations is at most $2n$, giving linear time.

7.14 Level Order Traversal Variants

Standard level-order (BFS) traversal of a binary tree visits nodes level by level, left to right. Two important variants modify the visitation order while keeping the same $O(n)$ BFS framework.

7.14.1 Zigzag Level Order

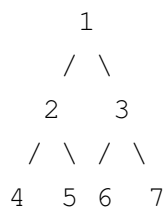
Visit alternating levels in opposite directions—left-to-right on even levels, right-to-left on odd levels. The simplest implementation uses a boolean flag to reverse the output of each level.

```

1 struct TreeNode {
2     int val;
3     TreeNode *left, *right;
4     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
5 };
6
7 vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
8     vector<vector<int>> result;
9     if (!root) return result;
10
11     queue<TreeNode*> q;
12     q.push(root);
13     bool leftToRight = true;
14
15     while (!q.empty()) {
16         int levelSize = q.size();
17         vector<int> level(levelSize);
18
19         for (int i = 0; i < levelSize; i++) {
20             TreeNode* node = q.front();
21             q.pop();
22
23             // Place node at correct position based on direction
24             int idx = leftToRight ? i : levelSize - 1 - i;
25             level[idx] = node->val;
26
27             if (node->left) q.push(node->left);
28             if (node->right) q.push(node->right);
29         }
30         leftToRight = !leftToRight;
31         result.push_back(level);
32     }
33     return result;
34 }

```

For the tree below, zigzag order is: $[1] \rightarrow [3, 2] \rightarrow [4, 5, 6, 7]$.



7.14.2 Right Side View

The **right side view** of a binary tree returns the values visible when looking from the right side—that is, the *last* node visited at each level.

```

1 vector<int> rightSideView(TreeNode* root) {
2     vector<int> result;
3     if (!root) return result;
4
5     queue<TreeNode*> q;
6     q.push(root);
7
8     while (!q.empty()) {
9         int levelSize = q.size();
10        for (int i = 0; i < levelSize; i++) {
11            TreeNode* node = q.front();
12            q.pop();
13
14            // Last node of this level is the right-side view node
15            if (i == levelSize - 1)
16                result.push_back(node->val);
17
18            if (node->left) q.push(node->left);
19            if (node->right) q.push(node->right);
20        }
21    }
22    return result;
23 }
```

For the same tree above, the right side view is [1,3,7]. The left side view (first node per level) would be [1,2,4]—simply change the condition to `i == 0`.

Complexity: Both variants run in $O(n)$ time and $O(w)$ space where w is the maximum tree width (the largest queue size at any level).

7.15 Common Bugs and Pitfalls

- **Confusing front() and back()** — in FIFO, `front()` returns the oldest element (first to be removed) while `back()` returns the most recently added. The naming is reversed from a stack perspective and causes bugs in production.
- **Circular buffer index arithmetic** — `(front_ + size_) % capacity_` is correct for insertion, but `(front_ + 1) % capacity_` for pop is only correct when `size_ > 0`. Off-by-one errors here cause silent memory corruption.
- **Queue in concurrent code without synchronization** — our examples use `std::mutex` and `std::condition_variable`. A bare `std::queue` shared across threads is a data race.

Always synchronize or use `tbb::concurrent_queue`.

- **Message queue backpressure** — if producers outpace consumers, the queue grows unbounded, consuming all memory. Production systems (Kafka, RabbitMQ) enforce limits: max queue size, TTL, or drop policies.
- **Linked queue memory fragmentation** — each enqueue allocates a node on the heap. For high-throughput systems, this causes cache misses and allocator contention. A circular buffer avoids per-element allocation.

7.16 Summary

1. A **queue** is a FIFO structure — insertions at the rear, deletions from the front.
2. **Circular buffers** implement array-based queues with $O(1)$ amortized operations.
3. **Linked queues** use a singly linked list with front and rear pointers.
4. A **deque** generalizes push/pop to both ends with $O(1)$ operations.
5. Industrial applications include message brokers (Kafka, SQS), web crawling (Googlebot), and thread pools (NGINX, uWSGI).
6. `std::queue` and `std::deque` provide production-ready implementations.

7.17 Interview FAQs

1. **Implement a circular buffer (ring buffer) queue with fixed capacity.**

Q: “Why do real-world ring buffers waste one slot instead of tracking a count, and how do you replace modulo with a bitmask?”

A: Wasting one slot lets you distinguish “full” from “empty” using only head/tail indices — if `head == tail`, empty; if `(tail+1)%n == head`, full. Tracking a count adds an extra variable and an extra write per operation. The bitmask trick: if the buffer size is a power of 2, replace `i % n` with `i & (n-1)` — bitwise AND is faster than modulo on most hardware. Single-producer/single-consumer ring buffers avoid locks entirely with memory ordering (acquire/release semantics on head/tail). The LMAX Disruptor goes further: power-of-2 buffer with sequence numbers instead of head/tail, enabling lock-free multi-producer/multi-consumer with cache-line padding between indices.

```
1 template <typename T>
2 class ring_buffer {
3     std::vector<T> buf_;
4     size_t head_ = 0, tail_ = 0, size_ = 0;
5 public:
```

```

6     explicit ring_buffer(size_t cap) : buf_(cap) {}
7     void push(const T& v) {
8         if (size_ == buf_.size()) throw std::overflow_error("full");
9         buf_[tail_] = v;
10        tail_ = (tail_ + 1) % buf_.size();
11        ++size_;
12    }
13    T pop() {
14        if (size_ == 0) throw std::underflow_error("empty");
15        T v = buf_[head_];
16        head_ = (head_ + 1) % buf_.size();
17        --size_;
18        return v;
19    }
20 };

```

Used in: OS I/O buffers, audio streaming, producer-consumer pipelines, lock-free queues.

1. Implement a stack using two queues.

Q: “When would you use a work-stealing deque instead of a shared queue in a thread pool?”

A: In a thread pool, each worker thread has its own deque of tasks. The owner pushes and pops from the LIFO end (cache-friendly for recently-created tasks, which likely share data). Other threads steal from the FIFO end (oldest tasks, which are more likely to be independent). This is the Chase-Lev deque: lock-free, grows dynamically, and avoids the contention of a shared queue. Cilk, Intel TBB, and Folly all use work-stealing deques. The key insight: LIFO for the owner preserves cache locality (recent tasks touch the same data), while FIFO for thieves provides fairness (oldest tasks are least likely to create new work).

```

1 class stack_two_queues {
2     std::queue<int> q1_, q2_;
3 public:
4     void push(int x) {
5         q2_.push(x);
6         while (!q1_.empty()) { q2_.push(q1_.front()); q1_.pop(); }
7         std::swap(q1_, q2_);
8     }
9     int pop() { int x = q1_.front(); q1_.pop(); return x; }
10    int top() { return q1_.front(); }
11 };

```

Variant: “Make push $O(1)$ and pop $O(n)$ ” – enqueue to $q1$, on pop transfer all but last from $q1$ to $q2$.

1. How does BFS use a queue and what is the time complexity?

Q: “Why does BFS guarantee shortest paths in unweighted graphs, and why can’t it do so for weighted graphs?”

A: BFS uses a FIFO queue, which processes all nodes at distance d before any at distance $d + 1$. This means the first time a node is discovered, it's at the minimum distance from the source — no shorter path can exist because all paths of length $< d$ have already been explored. For weighted graphs, this breaks because a longer path with cheaper edges can beat a shorter path with expensive edges. FIFO ordering doesn't account for edge weights — you need a priority queue (Dijkstra) to process nodes in order of increasing total cost. The invariant: “when a node is dequeued, its distance is final” holds for BFS only when all edges have equal weight.

Applications: Shortest path in unconnected graphs, level-order tree traversal, web crawling, garbage collection (Cheney's algorithm).

1. What is the difference between a queue and a deque? When would you use each?

A queue is FIFO: enqueue at back, dequeue from front only. A deque (double-ended queue) allows $O(1)$ push/pop at both ends. Use a queue for strict FIFO ordering (task scheduling, BFS). Use a deque when you need to add/remove from both ends (sliding window maximum, work-stealing schedulers, browser history). `std::deque` also supports $O(1)$ random access, unlike `std::queue`.

1. Design a bounded blocking queue for a producer-consumer system.

Use a circular buffer with three synchronization primitives: a mutex for exclusive access, a condition variable for “not full” (producers wait), and a condition variable for “not empty” (consumers wait). Producers lock, wait while full, insert, notify consumers. Consumers lock, wait while empty, remove, notify producers.

```

1  template <typename T>
2  class blocking_queue {
3      std::queue<T> q_;
4      size_t cap_;
5      mutable std::mutex mtx_;
6      std::condition_variable not_full_, not_empty_;
7  public:
8      explicit blocking_queue(size_t cap) : cap_(cap) {}
9      void push(T item) {
10         std::unique_lock lock(mtx_);
11         not_full_.wait(lock, [&]{ return q_.size() < cap_; });
12         q_.push(std::move(item));
13         not_empty_.notify_one();
14     }
15     T pop() {
16         std::unique_lock lock(mtx_);
17         not_empty_.wait(lock, [&]{ return !q_.size(); });
18         T item = std::move(q_.front()); q_.pop();
19         not_full_.notify_one();
20         return item;
21     }

```

```
22 };
```

Used in: thread pools, message queues, async I/O pipelines.

1. How does `std::deque` achieve $O(1)$ push at both ends?

`std::deque` uses a sequence of fixed-size blocks (typically 512 bytes) tracked by a central map (array of pointers). Pushing at the front allocates a new block at the front of the map; pushing at the back allocates at the end. Individual block operations are $O(1)$. The map itself is resized infrequently (doubles when full). This gives $O(1)$ amortized push at both ends with better cache behavior than a linked list.

Interview trap: “Is `std::deque` contiguous memory?” No — it’s a sequence of blocks. `&vec[0]` and `&vec[N]` may not be in the same block.

1. What is a monotonic deque and how is it used?

A monotonic deque maintains elements in sorted order (increasing or decreasing) by popping elements that violate the monotonic property during insertion. Used to solve the sliding window maximum/minimum problem in $O(n)$: for each new element, pop smaller elements from the back (maintaining decreasing order), then the front always holds the window maximum. Time $O(n)$ total (each element is pushed and popped at most once).

```
1 std::vector<int> sliding_window_max(const std::vector<int>& nums, int k) {
2     std::deque<int> dq; // indices, decreasing values
3     std::vector<int> result;
4     for (int i = 0; i < (int)nums.size(); ++i) {
5         while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
6         while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
7         dq.push_back(i);
8         if (i >= k - 1) result.push_back(nums[dq.front()]);
9     }
10    return result;
11 }
```

Applications: Sliding window maximum (LeetCode 239), largest rectangle in histogram.

1. What is the time complexity of `std::queue::size()`?

Q: “Why does C++ not guarantee $O(1)$ `size()` for all containers, and when does this matter in practice?”

A: `std::forward_list` doesn’t track size because the overhead of maintaining a counter isn’t always worth it — in high-throughput systems, avoiding an atomic counter (in concurrent contexts) can improve performance. `std::list` tracks size but `std::forward_list` doesn’t because forward lists are designed for minimal overhead. In practice, if you need $O(1)$ size on a queue, use `std::deque` as the underlying container (default for `std::queue`). The

deeper lesson: C++ containers make different tradeoffs between time, space, and API completeness — `std::forward_list` chose minimal overhead over convenience.

1. Design a queue that supports efficient `getMin()` in $O(1)$.

Maintain a main queue and a deque of minimums. On `push(x)`, pop from the back of the min-deque while its back $> x$, then push x . On `pop`, if the front of the min-deque equals the popped element, pop from the min-deque. The front of the min-deque is always the current minimum. Time: $O(1)$ amortized per operation. Space: $O(n)$.

```

1 class min_queue {
2     std::queue<int> q_;
3     std::deque<int> min_dq_;
4 public:
5     void push(int x) {
6         q_.push(x);
7         while (!min_dq_.empty() && min_dq_.back() > x) min_dq_.pop_back();
8         min_dq_.push_back(x);
9     }
10    int pop() {
11        int x = q_.front(); q_.pop();
12        if (x == min_dq_.front()) min_dq_.pop_front();
13        return x;
14    }
15    int getMin() const { return min_dq_.front(); }
16 };

```

This is a variant of the monotonic deque technique applied to queues.

1. When would you use a linked queue over a circular buffer?

Use a linked queue when: (a) the maximum size is unknown and may grow without bound, (b) you need stable pointers/references to enqueued elements, or (c) memory is fragmented and large contiguous allocations fail. Use a circular buffer when: (a) the maximum size is bounded, (b) you need $O(1)$ memory overhead (no per-node allocation), and (c) cache performance matters. Message brokers (Kafka) use contiguous log-based buffers; OS schedulers use linked queues.

7.18 Exercises

7.18.1 Drill

- Trace the state of a circular-buffer queue (capacity = 4) after each operation:

`enqueue(K)`, `en(V)`, `deq()`, `en(Q)`, `en(W)`, `en(E)`, `deq()`, `en(R)`, `en(T)`, `deq()`
 Show the front and rear pointers after each step.

1. In a web server thread pool with 8 workers and a queue of 100 pending requests, what happens when:
 - a) A 9th worker is requested but none is free?
 - b) The queue grows to 10,000 requests?
 - c) A worker crashes while holding a task?
1. A Kafka topic partition has 3 consumers in the same consumer group. Messages arrive as: [A, B, C, D, E]. Show which consumer receives each message. What happens if consumer 2 crashes after receiving message B?
1. What is the worst-case time complexity of:
 - a) Circular-buffer enqueue/dequeue
 - b) Linked-list enqueue/dequeue
 - c) Deque push_front/push_back
 How does each perform under cache misses?
1. A message queue has a maximum size of 1000. Producers enqueue 100 msg/s. Consumers process at 80 msg/s. After how many seconds does the queue fill up? What strategies can prevent this?

7.18.2 Application

1. **Kafka-like partitioned message queue.** Implement a `partitioned_queue` that distributes messages across N partitions using round-robin or key-based hashing. Each partition is an independent queue. Support `publish(key, message)` and `consume(partition_id)`. Simulate 3 consumers, 4 partitions, and 10,000 messages. Measure the load balance across consumers.
1. **Sliding window rate limiter (deque).** Cloudflare's rate limiting uses a sliding window counter. Implement a rate limiter using a deque of timestamps. Each request adds the current time; requests older than 1 second are popped from the front. The window size is the deque length. Test with a burst of 100 requests in 100ms — how many pass if the limit is 10 req/s?
1. **Web crawler politeness queue.** Extend the BFS web crawler to respect `robots.txt` crawl delays. Each domain has a "next allowed crawl time." Maintain a priority queue of `(next_crawl_time, url)`. Only dequeue URLs whose `next_crawl_time` has passed. Test on a list of 1000 URLs across 50 domains. Show that no domain is crawled more than once per 5 seconds.
1. **Thread pool with work stealing.** Implement a thread pool where each worker has its own task deque (instead of a shared queue). When a worker's deque is empty, it steals from another worker's deque (from the opposite end — the back). Compare throughput with the shared-queue approach for 10^6 fine-grained tasks. Explain why work stealing outperforms on cache-bound workloads.

1. **NGINX-style event queue.** NGINX processes events through a queue: accept connections, read requests, write responses. Implement an event-driven server using a single-threaded event loop with a queue of (fd, event_type) pairs. Support EPOLLIN and EPOLLOUT. Benchmark against a thread-per-connection model for 10,000 concurrent connections.
1. **Consumer group rebalancing.** In Kafka, when a consumer joins or leaves, the group rebalances — partitions are reassigned. Implement a simple rebalancer: given N partitions and M consumers, assign partitions to consumers using range or sticky strategy. Simulate adding/removing consumers and show the reassignment. Measure the "movement" (fraction of partitions that change owners).

7.18.3 Research

1. **(Blockmap deque)** `std::deque` uses a sequence of fixed-size blocks. What is the advantage over a single circular buffer? Implement a simplified blockmap deque with block size 64. Measure random-access performance vs `std::deque` and `std::vector`. When does the deque outperform vector for random access?
1. **(Chase-Lev work-stealing deque)** The Chase-Lev deque is the standard for work-stealing thread pools. Implement it with atomic operations. Compare its throughput with a mutex-guarded deque on 4, 8, and 16 threads. Why is the Chase-Lev design faster for the "owner" thread?
1. **(Disruptor)** Read about the LMAX Disruptor — a lock-free inter-thread queue that achieves 100M ops/sec. Implement a simplified single-producer, single-consumer ring buffer without locks. Compare its throughput with a mutex-guarded `std::queue` and `atbb::concurrent_queue`. What design choices make the Disruptor faster?

7.18.4 Additional Drill

6. Implement the circular queue from Section "Circular Queue: Full Implementation" with a `count` variable instead of the wasted-slot technique. Verify that all operations work correctly and that both `is_full()` and `is_empty()` are correct.
1. Trace the two-stack queue with the following sequence: `push(1)`, `push(2)`, `pop()`, `push(3)`, `push(4)`, `pop()`, `pop()`, `pop()`. Show the contents of both stacks after each operation and identify which stack is the "active" output stack at each step.
1. Given a circular array deque of capacity 8, execute: `push_back(A)`, `push_back(B)`, `push_front(Z)`, `pop_front()`, `push_back(C)`, `push_front(Y)`, `pop_back()`, `push_front(X)`. Draw the array contents and the front/index positions at the end.
1. Build the following graph and run BFS from vertex 0. List the BFS order and the shortest-path distances:


```

0 -- 1, 0 -- 2
1 -- 3, 1 -- 4
2 -- 5
3 -- 5
4 -- 5

```

Is this graph bipartite? Justify your answer.

1. For the word ladder problem, what is the shortest ladder from HIT to COG, dictionary = {HOT, DOT, DOG, LOT, LOG, COG}? Trace the BFS levels and show the search tree.

7.18.5 Additional Application

7. **Implement a deque using a single fixed-size circular array.** Your class should support `push_front`, `push_back`, `pop_front`, `pop_back`, and `operator[]` all in $O(1)$. Use a count variable to distinguish full from empty. Write a test that performs 1000 random deque operations and verifies correctness against a `std::deque`.
1. **Level-order tree traversal using two queues.** Implement level-order (BFS) traversal of a binary tree using exactly two queues (no `nullptr` sentinels, no `std::pair`). Print each level on a separate line. Then modify it to use a single queue plus a `level_size` counter.
1. **Priority queue merge.** Given k sorted arrays of total N elements, merge them into a single sorted array using a min-priority queue. Push the first element of each array into the PQ, then repeatedly pop the minimum and push the next element from the same array. Analyze time complexity in terms of N and k .

7.18.6 Additional Research

4. **(Circular buffer for I/O buffering)** Implement a circular byte buffer that wraps around a `char` array. Simulate an audio stream: the producer writes PCM samples at 44100 Hz and the consumer reads them at the same rate, but with occasional jitter (random delays). Measure the number of buffer underruns over a 10-second simulation. How does increasing the buffer size affect underruns vs latency?
1. **(BFS on implicit graph: sliding puzzle)** The 8-puzzle (3x3 grid with 8 tiles and one blank) can be modeled as a BFS problem on an implicit graph where each state is a grid configuration. Implement BFS to find the shortest solution from a given starting configuration. Count the number of states visited. How does this scale to the 15-puzzle (4x4)?

7.19 References and Further Reading

- Donald E. Knuth, *The Art of Computer Programming*, Volume 1: Fundamental Algorithms. Section 2.2.2.

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Sections 10.1, 22.2.
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Sections 1.3, 4.1.
- David Chase and Yossi Lev, "Dynamic Circular Work-Stealing Deque" — SPAA 2005.

Hashing and Hash Tables

8.1 Introduction

Complete compilable implementations of the data structures in this chapter are in `code/`.

8.2 The Dictionary ADT

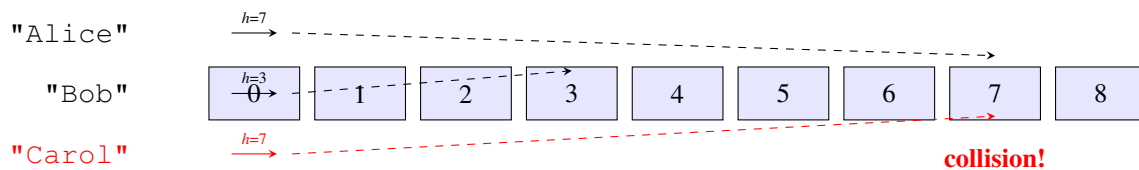
A **dictionary** (also called map or associative array) stores key-value pairs and supports:

- **insert(key, value)** — add a new key-value pair
- **find(key)** — return the value associated with key
- **erase(key)** — remove the key-value pair

Hash tables implement dictionaries with average $O(1)$ time per operation — a dramatic improvement over the $O(\log n)$ of balanced trees.

8.3 Hash Tables: Basic Idea

A **hash table** uses a **hash function** $h(k)$ to map each key k to an index in an array of size m . The pair (k, v) is stored at position $h(k)$ in the table.



When two keys hash to the same index, we have a **collision**. Collision resolution is the central challenge of hash table design.

8.4 Hash Functions

8.4.1 Properties of a Good Hash Function

1. **Deterministic** — same key always produces the same hash

2. **Uniform** — keys are distributed evenly across the table
3. **Fast** — $O(1)$ to compute
4. **Non-invertible** — hard to reconstruct the key from the hash (not a security requirement for general hashing, but good practice)

8.4.2 Integer Keys

For integer keys, the simplest approach: $h(k) = k \bmod m$.

m should be prime to reduce patterns. For example, $m = 13$:

```
1 h(10) = 10 mod 13 = 10
2 h(26) = 26 mod 13 = 0
3 h(39) = 39 mod 13 = 0 // collision with 26!
```

8.4.3 String Keys

```
1 // djb2 hash -- simple but effective
2 size_t hash_string(std::string_view s) {
3     size_t h = 5381;
4     for (char c : s) {
5         h = h * 33 + static_cast<size_t>(c);
6     }
7     return h;
8 }
```

8.4.4 Rob Dixon's "sdbm" hash

```
1 size_t hash_sdbm(std::string_view s) {
2     size_t h = 0;
3     for (char c : s) {
4         h = c + (h << 6) + (h << 16) - h;
5     }
6     return h;
7 }
```

8.4.5 `std::hash`

C++ provides `std::hash<T>` for common types:

```
1 std::hash<int> int_hash;
2 size_t h = int_hash(42);
3
4 std::hash<std::string> str_hash;
5 size_t h2 = str_hash("hello");
```

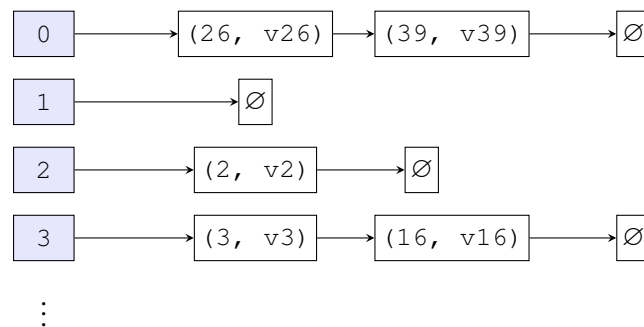
```

6
7 // Custom hash for user types:
8 struct point {
9     int x, y;
10    bool operator==(const point& other) const {
11        return x == other.x && y == other.y;
12    }
13 };
14
15 template <>
16 struct std::hash<point> {
17     size_t operator()(const point& p) const noexcept {
18         size_t h1 = std::hash<int>{}(p.x);
19         size_t h2 = std::hash<int>{}(p.y);
20         return h1 ^ (h2 << 1);
21     }
22 };

```

8.5 Separate Chaining

Each table slot holds a linked list (or any container) of all key-value pairs that hash to that position.



```

1 template <typename K, typename V, typename Hash = std::hash<K>>
2 class hash_table_chaining {
3 public:
4     explicit hash_table_chaining(size_t initial_buckets = 16)
5         : buckets_(initial_buckets), table_(initial_buckets) {}
6
7     void insert(const K& key, const V& value) {
8         auto& chain = table_[bucket(key)];
9         for (auto& [k, v] : chain) {
10             if (k == key) {
11                 v = value; // update existing
12                 return;
13             }
14         }
15         chain.push_back({key, value});

```

```

16         ++size_;
17         check_load();
18     }
19
20     std::optional<V> find(const K& key) const {
21         const auto& chain = table_[bucket(key)];
22         for (const auto& [k, v] : chain) {
23             if (k == key) return v;
24         }
25         return std::nullopt;
26     }
27
28     void erase(const K& key) {
29         auto& chain = table_[bucket(key)];
30         auto it = std::find_if(chain.begin(), chain.end(),
31             [&](const auto& p) { return p.first == key; });
32         if (it != chain.end()) {
33             chain.erase(it);
34             --size_;
35         }
36     }
37
38     size_t size() const noexcept { return size_; }
39     bool empty() const noexcept { return size_ == 0; }
40
41 private:
42     size_t bucket(const K& key) const {
43         return Hash{}(key) % buckets_;
44     }
45
46     void check_load() {
47         // Rehash when load factor exceeds 1.0
48         if (size_ > buckets_) {
49             buckets_ *= 2;
50             std::vector<std::list<std::pair<K, V>>> new_table(buckets_);
51             for (const auto& chain : table_) {
52                 for (const auto& pair : chain) {
53                     size_t b = Hash{}(pair.first) % buckets_;
54                     new_table[b].push_back(pair);
55                 }
56             }
57             table_ = std::move(new_table);
58         }
59     }
60
61     size_t buckets_;
62     size_t size_ = 0;
63     std::vector<std::list<std::pair<K, V>>> table_;

```

```
64 };
```

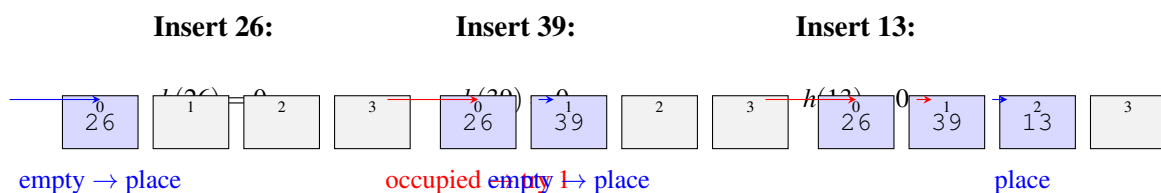
Load factor: $\alpha = n/m$ (number of elements / number of buckets). With separate chaining, α can exceed 1. Average chain length = α .

8.6 Open Addressing

All elements are stored in the table array itself. On collision, we probe for the next empty slot.

8.6.1 Linear Probing

If slot $h(k)$ is occupied, try $h(k) + 1$, $h(k) + 2$, ..., wrapping around.



Problem: primary clustering. A block of consecutive occupied slots grows, attracting more collisions.

8.6.2 Quadratic Probing

Try positions: $h(k)$, $h(k) + 1^2$, $h(k) + 2^2$, $h(k) + 3^2$, ...

```
1 Probe sequence:  $p(i) = (h(k) + i^2) \bmod m$ 
```

Reduces clustering but may miss slots even when the table is not full.

8.6.3 Double Hashing

Use a second hash function for the probe step:

```
1 Probe sequence:  $p(i) = (h(k) + i \cdot h_2(k)) \bmod m$ 
```

This gives the best distribution of the three open addressing methods.

```
1 template <typename K, typename V, typename Hash = std::hash<K>>
2 class hash_table_open {
3 public:
4     explicit hash_table_open(size_t initial_capacity = 16)
5         : capacity_(initial_capacity), size_(0),
6           states_(initial_capacity, State::empty),
7           keys_(initial_capacity), values_(initial_capacity) {}
8
9     bool insert(const K& key, const V& value) {
10         if (size_ >= capacity_ * 0.7) rehash();
```

```

11     size_t i = 0;
12     size_t pos;
13     do {
14         pos = probe(key, i);
15         if (states_[pos] == State::empty || states_[pos] == State::
deleted) {
16             keys_[pos] = key;
17             values_[pos] = value;
18             states_[pos] = State::occupied;
19             ++size_;
20             return true;
21         }
22         if (states_[pos] == State::occupied && keys_[pos] == key) {
23             values_[pos] = value; // update
24             return false;
25         }
26         ++i;
27     } while (i < capacity_);
28     rehash();
29     return insert(key, value);
30 }
31
32 std::optional<V> find(const K& key) const {
33     size_t i = 0;
34     size_t pos;
35     do {
36         pos = probe(key, i);
37         if (states_[pos] == State::empty) return std::nullopt;
38         if (states_[pos] == State::occupied && keys_[pos] == key) {
39             return values_[pos];
40         }
41         ++i;
42     } while (i < capacity_);
43     return std::nullopt;
44 }
45
46 void erase(const K& key) {
47     size_t i = 0;
48     size_t pos;
49     do {
50         pos = probe(key, i);
51         if (states_[pos] == State::empty) return;
52         if (states_[pos] == State::occupied && keys_[pos] == key) {
53             states_[pos] = State::deleted;
54             --size_;
55             return;
56         }
57         ++i;

```



```

58         } while (i < capacity_);
59     }
60
61 private:
62     enum class State { empty, occupied, deleted };
63
64     size_t probe(const K& key, size_t i) const {
65         size_t h1 = Hash{}(key) % capacity_;
66         size_t h2 = 1 + (Hash{}(key) % (capacity_ - 1)); // second hash
67         return (h1 + i * h2) % capacity_;
68     }
69
70     void rehash() {
71         size_t old_cap = capacity_;
72         capacity_ *= 2;
73         std::vector<State> old_states = std::move(states_);
74         std::vector<K> old_keys = std::move(keys_);
75         std::vector<V> old_values = std::move(values_);
76
77         states_ = std::vector<State>(capacity_, State::empty);
78         keys_.resize(capacity_);
79         values_.resize(capacity_);
80         size_ = 0;
81
82         for (size_t i = 0; i < old_cap; ++i) {
83             if (old_states[i] == State::occupied) {
84                 insert(old_keys[i], old_values[i]);
85             }
86         }
87     }
88
89     size_t capacity_, size_;
90     std::vector<State> states_;
91     std::vector<K> keys_;
92     std::vector<V> values_;
93 };

```

8.7 Skip Lists

A **skip list** is a probabilistic data structure that maintains an ordered set with $O(\log n)$ expected time for search, insert, and erase. It consists of multiple linked levels, where each higher level acts as an “express lane” over the level below.

8.7.1 Structure

A skip list is a hierarchy of sorted linked lists:

```

1 level 3: 1 -----> 9
2 level 2: 1 -----> 5 -----> 9
3 level 1: 1 ---> 3 ---> 5 ---> 7 ---> 9
4 level 0: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> {
           empty}

```

Each element appears in level 0. It appears in higher levels with probability 1/2 per level (i.e., level 1: 1/2, level 2: 1/4, level 3: 1/8, etc.). The maximum level is typically capped at $\lceil \log_2 n \rceil$.

8.7.2 Node Structure

```

1 template <typename K, typename V>
2 struct skip_node {
3     K key;
4     V value;
5     std::vector<skip_node*> next; // next[i] = successor at level i
6
7     skip_node(const K& k, const V& v, int level)
8         : key(k), value(v), next(level + 1, nullptr) {}
9 };

```

8.7.3 Search

Search proceeds from the topmost level, moving forward as far as possible at each level before descending:

```

1 template <typename K, typename V>
2 V* skip_list<K, V>::find(const K& key) {
3     auto* cur = head_;
4     for (int i = level_; i >= 0; --i) {
5         while (cur->next[i] && cur->next[i]->key < key)
6             cur = cur->next[i];
7     }
8     cur = cur->next[0];
9     if (cur && cur->key == key) return &cur->value;
10    return nullptr;
11 }

```

8.7.4 Insertion

To insert, we search through all levels and record the nodes where we need to insert at each level. We then choose a random level for the new node and link it in:

```

1 template <typename K, typename V>
2 void skip_list<K, V>::insert(const K& key, const V& value) {
3     std::vector<skip_node<K, V*>> update(level_ + 1, nullptr);
4     auto* cur = head_;

```

```

5
6     for (int i = level_; i >= 0; --i) {
7         while (cur->next[i] && cur->next[i]->key < key)
8             cur = cur->next[i];
9         update[i] = cur;
10    }
11
12    cur = cur->next[0];
13    if (cur && cur->key == key) {
14        cur->value = value; // key already exists
15        return;
16    }
17
18    int new_level = random_level();
19    if (new_level > level_) {
20        head->next.resize(new_level + 1, nullptr);
21        update.resize(new_level + 1, head_);
22        level_ = new_level;
23    }
24
25    auto* node = new skip_node<K, V>(key, value, new_level);
26    for (int i = 0; i <= new_level; ++i) {
27        node->next[i] = update[i]->next[i];
28        update[i]->next[i] = node;
29    }
30 }

```

8.7.5 Choosing the Level

```

1 int random_level() {
2     int level = 0;
3     while (rand() < RAND_MAX / 2 && level < MAX_LEVEL)
4         ++level;
5     return level;
6 }

```

This produces level 0 with probability 1/2, level 1 with probability 1/4, level 2 with probability 1/8, etc.

8.7.6 Complexity

Operation	Expected	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Erase	$O(\log n)$	$O(n)$
Space	$O(n)$	$O(n \log n)$

The worst case occurs when all elements get the same level (e.g., all level 0). The probability of this is vanishingly small: $(1/2)^n$ for n elements all at level 0.

8.7.7 Skip Lists vs. Hash Tables vs. Balanced Trees

Criteria	Hash Table	Skip List	Balanced Tree
Average search	$O(1)$	$O(\log n)$	$O(\log n)$
Ordered iteration	No	$O(n \log n)$	$O(n)$ in-order
Range queries	No	$O(\log n + k)$	$O(\log n + k)$
Implementation	Medium	Medium	Complex
Memory	$O(n + m)$	$O(n)$ expected	$O(n)$

Skip lists occupy a useful middle ground: they support ordered operations like balanced trees but are much simpler to implement. They are not a replacement for hash tables (which are faster for unordered lookups), but they are competitive with balanced trees for ordered maps.

8.7.8 STL Connection

The C++ standard library does not include a skip list implementation. However, the `leveldb` and `rocksdb` databases use skip lists as their default in-memory table implementation, and Redis uses skip lists for sorted sets.

8.8 Modern Hash Table Designs

8.8.1 Robin Hood Hashing

When probing, if the current occupant is "richer" (closer to its ideal position) than the incoming element, swap them. This equalizes probe distances.

```

1 Insert 39: h(39)=0 -> occupied by 26 (probe distance 0)
2           Compare: 39 has distance 0, 26 has distance 0 -> keep going
3           Try 1 -> empty -> insert 39 (distance 1)
4
5 Insert 13: h(13)=0 -> 26 (dist 0) -> compare equal -> keep going
6           1 -> 39 (dist 1) -> compare equal -> keep going
7           2 -> empty -> insert (dist 2)
8
9 Insert 1: h(1)=1 -> 39 (dist 1) -> 1 has dist 0 < 39's dist 1 ->
           keep going
10          2 -> 13 (dist 2) -> 1 has dist 1 < 13's dist 2 -> keep
           going
11          3 -> empty -> insert (dist 3)

```

8.8.2 Cuckoo Hashing

Use two hash tables with two different hash functions. On collision, evict the existing element and insert it into the other table. If this cycles, rehash.

8.8.3 Swiss Table (Google's Abseil flat_hash_map)

Uses SIMD (SSE2/ARM NEON) to probe 16 slots simultaneously. Each slot has a 1-byte metadata field (7 bits of hash + 1 bit for occupancy). The lookup checks all 16 metadata bytes in one CPU instruction.

```
1 // Abseil flat_hash_map -- the fastest general-purpose hash table
2 #include <absl/container/flat_hash_map.h>
3
4 absl::flat_hash_map<std::string, int> map;
5 map["hello"] = 1;
6 map["world"] = 2;
```

8.9 Distributed Hash Table (DHT)

8.9.1 Introduction

A **distributed hash table** (DHT) distributes key-value pairs across many nodes in a peer-to-peer (P2P) network. Each node owns a contiguous range of keys and is responsible for storing all key-value pairs in that range. Unlike a centralized hash table where one server handles all lookups, a DHT spreads the storage and query load across n participating nodes.

The key idea is **consistent hashing**: both keys and node identifiers occupy the same circular address space (typically $[0, 2^m - 1]$). A key is assigned to the first node whose identifier is equal to or greater than the key's hash. This means adding or removing a node only affects keys in its immediate range, not the entire table.

8.9.2 Chord Ring Structure

The Chord protocol arranges nodes in a logical ring. Each node maintains a **finger table** of m entries that allow it to skip roughly half the ring at each hop, giving $O(\log n)$ lookup.

Chord Ring with 8 nodes ($m = 3$, identifiers 0..7)
Nodes: 0, 1, 3, 4, 6, 7



```

      6          2          . = empty position
      \        /          Keys 0,1,3,4,6,7 are owned
        5                by the next node clockwise.

```

Key ownership (consistent hashing):

```

Key 0 -> Node 0      Key 4 -> Node 4
Key 1 -> Node 1      Key 5 -> Node 6
Key 2 -> Node 3      Key 6 -> Node 6
Key 3 -> Node 3      Key 7 -> Node 7

```

Node 3 finger table:

```

Entry 0: successor of (3+1) mod 8 = 4 -> Node 4
Entry 1: successor of (3+2) mod 8 = 5 -> Node 6
Entry 2: successor of (3+4) mod 8 = 7 -> Node 7

```

Lookup for key 5 starting at Node 3:

```

Step 1: 5 > 3, closest finger <= 5 is Node 4 (entry 0)
Step 2: at Node 4, 5 in (4, 7] -> forward to Node 6
Step 3: Node 6 owns key 5. Done in 2 hops.

```

8.9.3 Simplified C++ Implementation

The following code simulates a single DHT node with a finger table and basic key-value storage. In a real system, each node would communicate with others over the network.

```

1  #include <vector>
2  #include <map>
3  #include <string>
4  #include <functional>
5  #include <cmath>
6  #include <algorithm>
7
8  class DHTNode {
9      int id;                // this node's identifier
10     int m;                 // number of bits in id space
11     std::vector<int> fingerTable; // successor for each finger
12     std::map<int, std::string> store; // local key-value storage
13
14     // Compute the start of finger i: (id + 2^i) mod 2^m
15     int fingerStart(int i) const {
16         return (id + (1 << i)) % (1 << m);
17     }
18
19 public:

```

```

20     DHTNode(int nodeId, int bits)
21         : id(nodeId), m(bits), fingerTable(bits) {
22         for (int i = 0; i < m; i++)
23             fingerTable[i] = id; // placeholder
24     }
25
26     int getId() const { return id; }
27
28     // Find the successor node responsible for a given key
29     // Uses finger table to skip ahead -- O(log n) hops
30     DHTNode* find_successor(DHTNode* allNodes[], int n,
31                             int key) {
32         // Check if key falls in (this, successor]
33         int succ = fingerTable[0];
34         if (key > id && key <= succ)
35             return this;
36
37         // Find closest preceding node using finger table
38         DHTNode* next = nullptr;
39         for (int i = m - 1; i >= 0; i--) {
40             if (fingerTable[i] > id &&
41                 fingerTable[i] < key) {
42                 // In a real system, we would contact
43                 // the node with this id over the network
44                 for (int j = 0; j < n; j++) {
45                     if (allNodes[j]->id == fingerTable[i]) {
46                         next = allNodes[j];
47                         break;
48                     }
49                 }
50                 break;
51             }
52         }
53         if (!next) next = this; // wrap around
54
55         // Recurse: delegate lookup to the next node
56         return next->find_successor(allNodes, n, key);
57     }
58
59     // Insert a key-value pair
60     void put(int key, const std::string& value) {
61         store[key] = value;
62     }
63
64     // Retrieve a value by key
65     std::string get(int key) const {
66         auto it = store.find(key);
67         if (it != store.end()) return it->second;

```

```

68     return "";
69 }
70
71 // Build finger table given the full set of nodes
72 void buildFingers(DHTNode* allNodes[], int n) {
73     for (int i = 0; i < m; i++) {
74         int start = fingerStart(i);
75         int best = allNodes[0]->id;
76         for (int j = 0; j < n; j++) {
77             int nid = allNodes[j]->id;
78             if (nid >= start) {
79                 best = nid;
80                 break;
81             }
82         }
83         fingerTable[i] = best;
84     }
85 }
86 };

```

8.9.4 Complexity Analysis

Operation	Time	Notes
Lookup (find_successor)	$O(\log n)$	finger table skips
Insert (with stabilization)	$O(\log^2 n)$	$O(\log n)$ hops $\times$ $O(\log n)$ fixup
Node join	$O(\log^2 n)$	stabilize fingers and predecessors
Space per node	$O(\log n)$	one finger table entry per bit

The $O(\log n)$ lookup bound holds because the finger table doubles the distance covered at each step, similar to binary search. Insert stabilization is $O(\log^2 n)$ because a new node must fix $O(\log n)$ fingers, and each fix may require $O(\log n)$ hops to reach the correct node.

8.9.5 Applications

- **BitTorrent** – Uses a DHT (Kademlia-based) so peers can find each other without a central tracker. Each torrent forms its own DHT overlay.
- **IPFS** – Uses a Kademlia DHT to locate content by its hash across a global P2P network.
- **Amazon DynamoDB** – Uses consistent hashing to partition data across storage nodes, inspired by the DHT designs from the early 2000s.
- **Apache Cassandra** – Uses consistent hashing (via a token ring) to distribute data across a cluster of nodes, handling replication and fault tolerance.

8.9.6 DHT vs. Centralized Hash Table

Property	Centralized	DHT
Lookup cost	$O(1)$ local	$O(\log n)$ network hops
Single point of failure	Yes	No
Scalability	Limited by one machine	Scales to millions of nodes
Latency	Very low	Higher (network RTT per hop)
Consistency	Strong (trivial)	Eventual (partition tolerance)

A centralized hash table is faster and simpler but does not scale beyond one machine and has a single point of failure. A DHT trades latency and consistency for horizontal scalability and fault tolerance, making it ideal for large-scale P2P applications.

8.10 Count-Min Sketch

A **Count-Min Sketch** is a probabilistic data structure that estimates the frequency of items in a data stream using far less memory than exact counting. It is widely used in network traffic analysis, database query optimization, and natural language processing.

8.10.1 Idea

Maintain a 2D array of $d \times w$ counters, all initialized to zero. Each of the d rows has an independent hash function $h_i(x)$ that maps the key x to a column in $[0, w - 1]$.

Update: For a new element x with count c :

1. For each row i from 0 to $d - 1$:
2. Increment $\text{table}[i][h_i(x)]$ by c .

Query: The estimated frequency of x is:

$$\hat{f}(x) = \min_{i=0}^{d-1} \text{table}[i][h_i(x)]$$

Taking the minimum across all rows bounds the overestimation error. The estimate is never an undercount; it can only overestimate by at most $\varepsilon \cdot N$, where N is the total number of items processed and $\varepsilon = e/w$.

8.10.2 Parameters

Parameter	Symbol	Value
Width	w	$\lceil e/\varepsilon \rceil$
Depth	d	$\lceil \ln(1/\delta) \rceil$

With probability at least $1 - \delta$, the estimated frequency satisfies $\hat{f}(x) \leq f(x) + \epsilon N$, where $f(x)$ is the true frequency.

8.10.3 C++ Implementation

```

1  #include <vector>
2  #include <string>
3  #include <algorithm>
4  #include <cstdint>
5
6  class CountMinSketch {
7  private:
8      int d;                // depth (number of hash functions)
9      int w;                // width (number of counters per row)
10     std::vector<std::vector<uint64_t>> table;
11     std::vector<uint64_t> seeds;
12
13     // Simple hash combining seed:  $h_i(x) = (a * \text{hash}(x) + b) \bmod p \bmod w$ 
14     uint64_t hash(int i, const std::string& key) const {
15         uint64_t h = 14695981039346656037ULL; // FNV-1a offset basis
16         for (char c : key) {
17             h ^= static_cast<uint64_t>(c);
18             h *= 1099511628211ULL;           // FNV-1a prime
19         }
20         h ^= seeds[i];
21         h *= 1099511628211ULL;
22         return h % w;
23     }
24
25 public:
26     CountMinSketch(int depth, int width)
27         : d(depth), w(width),
28         table(depth, std::vector<uint64_t>(width, 0)),
29         seeds(depth) {
30         // Initialize each seed to a distinct odd value
31         for (int i = 0; i < d; i++) {
32             seeds[i] = static_cast<uint64_t>(i) * 6364136223846793005ULL +
33             1;
34         }
35
36         // Add count c for the given key
37         void update(const std::string& key, uint64_t c = 1) {
38             for (int i = 0; i < d; i++) {
39                 uint64_t col = hash(i, key);
40                 table[i][col] += c;
41             }
42         }

```

```

43
44     // Estimate the frequency of the given key
45     uint64_t query(const std::string& key) const {
46         uint64_t est = UINT64_MAX;
47         for (int i = 0; i < d; i++) {
48             uint64_t col = hash(i, key);
49             est = std::min(est, table[i][col]);
50         }
51         return est;
52     }
53
54     // Merge two sketches (additive)
55     void merge(const CountMinSketch& other) {
56         for (int i = 0; i < d; i++)
57             for (int j = 0; j < w; j++)
58                 table[i][j] += other.table[i][j];
59     }
60 };
61
62 // Usage example
63 int main() {
64     // epsilon = 0.001, delta = 0.001 => w = 2719, d = 7
65     CountMinSketch cms(7, 2719);
66
67     cms.update("hello");
68     cms.update("world");
69     cms.update("hello");
70     cms.update("hello");
71
72     std::cout << "hello: " << cms.query("hello") << "\n"; // 3
73     std::cout << "world: " << cms.query("world") << "\n"; // 1
74     std::cout << "absent: " << cms.query("missing") << "\n"; // 0
75 }

```

8.10.4 Properties

- **Space:** $O(w \cdot d) = O(\frac{1}{\epsilon} \log \frac{1}{\delta})$ counters, independent of the number of distinct keys.
- **Update time:** $O(d)$ per insertion.
- **Query time:** $O(d)$ per lookup.
- **One-sided error:** Estimates are always $\geq$ true frequency (never underestimates).
- **Cannot delete:** The basic sketch does not support removal. A **Count-Mean Sketch** variant uses an auxiliary table of per-cell estimates to subtract deleted items.

8.10.5 Heavy Hitters

A Count-Min Sketch can identify **heavy hitters** (frequent items). Maintain a threshold $\phi \cdot N$ and track any key whose estimate exceeds this threshold. Since $\hat{f}(x) \leq f(x) + \epsilon N$, setting $\phi > \epsilon$ ensures that all truly frequent items are reported with high probability.

8.11 Swiss Table Hash

Google’s **Swiss Table** (used in `absl : flat_hash_map` and Go’s built-in `map`) is a flat open-addressing hash map that uses SIMD instructions to probe 16 slots simultaneously, achieving near-zero overhead per lookup.

8.11.1 Control Bytes

Each slot stores a 1-byte **control byte** (also called metadata) containing:

- 7 bits: the high 7 bits of the hash fingerprint.
- 1 bit: occupied flag (0 = empty, 1 = used).

A group of 16 consecutive control bytes forms a **block**. On x86-64, the entire block fits in a single 128-bit SSE register; on ARM, it fits in a 128-bit NEON register.

8.11.2 Probe Sequence

1. Compute $h = \text{hash}(\text{key})$ and extract the 7-bit fingerprint $f = h \gg (64 - 7)$.
2. Load 16 control bytes into a SIMD register. Compare all 16 bytes against f using a single instruction (`_mm_cmpeq_epi8` on SSE2). This produces a 16-bit mask of matching slots.
3. Intersect the mask with the occupied bits to find slots that both match the fingerprint and are occupied. If non-empty, the key may be in one of those slots.
4. If the empty-bit match shows an empty slot before any occupied match, the key is absent.
5. Advance to the next block using quadratic or triangular probing.

8.11.3 Probe Diagram

ctrl[0]	ctrl[1]	ctrl[2]	ctrl[3]	ctrl[4]	ctrl[5]	ctrl[6]	ctrl[7]
ctrl[8]	ctrl[9]	ctrl[10]	ctrl[11]	ctrl[12]	ctrl[13]	ctrl[14]	ctrl[15]

`_mm_cmpeq_epi8` compares all 16 control bytes against the fingerprint in ≈ 1 cycle. The result is masked with `_builtin_ctz` (or equivalent) to find the matching slot index.

8.11.4 Comparison with `std::unordered_map`

Property	<code>std::unordered_map</code>	Swiss Table (<code>flat_hash_map</code>)
Underlying structure	Separate chaining (linked lists)	Open addressing (flat array)
Cache efficiency	Poor (pointer chasing)	Excellent (data contiguous in memory)
Probe method	Walk linked list	SIMD compare 16 slots at once
Typical lookup	4–6 cache misses	1 cache miss
Memory overhead	≈ 32 bytes/node (pointers)	≈ 1 byte/node (control bytes)

In practice, `absl::flat_hash_map` is $2\text{--}5\times$ faster than `std::unordered_map` for lookup-heavy workloads due to superior cache locality and SIMD batching.

8.12 Robin Hood Hashing

Robin Hood Hashing is an open-addressing scheme that reduces the *variance* of probe lengths. The idea: during insertion, if the new element is farther from its ideal bucket than the element already occupying the target slot, swap them. This “steals from the rich” (short probe distance) to give to the “poor” (long probe distance), equalizing access times.

8.12.1 Invariant

Maintain the invariant that for every pair of elements a and b where a is at position p_a and b is at position p_b with $p_a < p_b$:

$$\text{dist}(a, \text{home}(a)) \leq \text{dist}(b, \text{home}(b))$$

This ensures elements are ordered by probe distance in any contiguous run of occupied slots.

8.12.2 Benefits

- **$O(1)$ worst-case successful search:** The maximum probe distance for any element is bounded by the length of the longest run, which is $O(\sqrt{n})$ for a table of size n .
- **Deletion without tombstones:** Removing an element can be done by shifting subsequent elements backward until a gap is reached, since the ordering invariant is maintained.
- **Lower variance:** Average and worst-case probe lengths are much closer together, improving tail latency.

8.12.3 C++ Sketch

```
1 #include <vector>
2 #include <optional>
3 #include <cstdint>
```

```

4
5 class RobinHoodTable {
6 private:
7     struct Slot {
8         uint64_t key = 0;
9         int      home = 0;    // ideal bucket index
10        bool      occupied = false;
11    };
12
13    int            capacity;
14    int            count = 0;
15    std::vector<Slot> slots;
16
17    int probe_distance(int home, int pos) const {
18        int d = pos - home;
19        return (d >= 0) ? d : d + capacity;
20    }
21
22    int find_slot(uint64_t key) const {
23        int home = key % capacity;
24        for (int i = 0; i < capacity; i++) {
25            int idx = (home + i) % capacity;
26            if (!slots[idx].occupied) return idx;
27            if (slots[idx].key == key) return idx;
28            if (probe_distance(slots[idx].home, idx) < i)
29                return -1; // key cannot exist past this point
30        }
31        return -1;
32    }
33
34 public:
35     explicit RobinHoodTable(int cap) : capacity(cap), slots(cap) {}
36
37     void insert(uint64_t key) {
38         int home = key % capacity;
39         uint64_t cur_key = key;
40         int cur_home = home;
41
42         for (int i = 0; i < capacity; i++) {
43             int idx = (home + i) % capacity;
44             if (!slots[idx].occupied) {
45                 slots[idx] = {cur_key, cur_home, true};
46                 count++;
47                 return;
48             }
49             if (slots[idx].key == cur_key) return; // already present
50
51             // Robin Hood: if new element is "poorer," swap

```

```

52         int existing_dist = probe_distance(slots[idx].home, idx);
53         int new_dist = i;
54         if (new_dist > existing_dist) {
55             std::swap(cur_key, slots[idx].key);
56             std::swap(cur_home, slots[idx].home);
57             home = cur_home; // continue probing for displaced
element
58             i = existing_dist; // resume from where displaced element
was
59         }
60     }
61 }
62
63 std::optional<uint64_t> find(uint64_t key) const {
64     int idx = find_slot(key);
65     if (idx >= 0 && slots[idx].occupied && slots[idx].key == key)
66         return slots[idx].key;
67     return std::nullopt;
68 }
69
70 void erase(uint64_t key) {
71     int idx = find_slot(key);
72     if (idx < 0 || !slots[idx].occupied) return;
73     slots[idx].occupied = false;
74     count--;
75
76     // Shift subsequent elements backward to fill the gap
77     for (int i = 1; i < capacity; i++) {
78         int next = (idx + i) % capacity;
79         if (!slots[next].occupied) break;
80         int ideal = slots[next].home;
81         int dist = probe_distance(ideal, next);
82         if (dist == 0) break; // element is at home, nothing to shift
83         slots[idx] = slots[next];
84         slots[idx].occupied = true;
85         slots[next].occupied = false;
86         idx = next;
87     }
88 }
89
90 int size() const { return count; }
91 };
92
93 // Usage
94 #include <iostream>
95 int main() {
96     RobinHoodTable ht(101);
97     ht.insert(42);

```

```

98     ht.insert(99);
99     ht.insert(150);
100    auto r = ht.find(42);
101    if (r) std::cout << "Found: " << *r << "\n";
102    ht.erase(42);
103 }

```

8.13 Extendible Hashing

A fixed-size hash table must be rehashed entirely when it fills up. **Extendible hashing** avoids this by growing the table incrementally: instead of doubling all m buckets at once, it maintains a *directory* of pointers that doubles or halves as needed, while individual buckets split or merge independently.

8.13.1 Core Idea

Each bucket holds up to B entries. A global **directory depth** d determines the number of directory slots: 2^d . Each bucket also has a local depth $\ell \leq d$. When a bucket overflows, it **splits**: its local depth increases by 1 and its entries are redistributed into two new buckets. If the local depth was equal to the directory depth, the directory itself doubles first.

8.13.2 Directory Doubling

```

1  #include <vector>
2  #include <list>
3  #include <utility>
4  #include <optional>
5  #include <functional>
6
7  template <typename K, typename V,
8           typename Hash = std::hash<K>>
9  class ExtendibleHash {
10 public:
11     explicit ExtendibleHash(size_t bucket_size = 4)
12         : bucket_cap_(bucket_size), global_depth_(0),
13         directory_(1, new Bucket(bucket_size)) {}
14
15     ~ExtendibleHash() {
16         for (auto* b : directory_) delete b;
17     }
18
19     void insert(const K& key, const V& value) {
20         size_t idx = hash_to_index(key);
21         Bucket* bkt = directory_[idx];
22         if (bkt->size() < bucket_cap_) {
23             bkt->insert(key, value);

```



```

24         return;
25     }
26     // Bucket full -- split it
27     if (bkt->local_depth_ == global_depth_)
28         double_directory();
29     split_bucket(idx);
30     insert(key, value); // retry after split
31 }
32
33 std::optional<V> find(const K& key) const {
34     size_t idx = hash_to_index(key);
35     return directory_[idx]->find(key);
36 }
37
38 void erase(const K& key) {
39     size_t idx = hash_to_index(key);
40     directory_[idx]->erase(key);
41 }
42
43 private:
44     struct Bucket {
45         int local_depth_;
46         size_t cap_;
47         std::list<std::pair<K, V>> entries_;
48
49         explicit Bucket(size_t cap, int depth = 0)
50             : local_depth_(depth), cap_(cap) {}
51
52         bool full() const { return entries_.size() >= cap_; }
53         size_t size() const { return entries_.size(); }
54
55         void insert(const K& k, const V& v) {
56             entries_.push_back({k, v});
57         }
58         std::optional<V> find(const K& k) const {
59             for (auto& [ek, ev] : entries_)
60                 if (ek == k) return ev;
61             return std::nullopt;
62         }
63         void erase(const K& k) {
64             entries_.remove_if(
65                 [&](auto& p) { return p.first == k; });
66         }
67     };
68
69     size_t hash_to_index(const K& key) const {
70         size_t h = Hash{}(key);
71         return h & ((1ULL << global_depth_) - 1);

```

```

72     }
73
74     void double_directory() {
75         size_t old_size = 1ULL << global_depth_;
76         size_t new_size = old_size * 2;
77         directory_.resize(new_size);
78         for (size_t i = 0; i < old_size; ++i)
79             directory_[i + old_size] = directory_[i];
80         ++global_depth_;
81     }
82
83     void split_bucket(size_t idx) {
84         Bucket* old_bkt = directory_[idx];
85         int old_depth = old_bkt->local_depth_;
86         int new_depth = old_depth + 1;
87
88         Bucket* b0 = new Bucket(bucket_cap_, new_depth);
89         Bucket* b1 = new Bucket(bucket_cap_, new_depth);
90         old_bkt->local_depth_ = new_depth;
91
92         // Redistribute entries by one extra hash bit
93         for (auto& [k, v] : old_bkt->entries_) {
94             size_t h = Hash{}(k);
95             size_t bit = (h >> old_depth) & 1;
96             if (bit == 0) b0->insert(k, v);
97             else         b1->insert(k, v);
98         }
99
100        // Update directory pointers
101        size_t step = 1ULL << new_depth;
102        for (size_t i = idx & (step - 1);
103             i < (1ULL << global_depth_); i += step) {
104            if (i == idx)                directory_[i] = b0;
105            else if (i == idx + step)    directory_[i] = b1;
106        }
107        // Fix duplicate pointers from the doubling
108        size_t mask = step - 1;
109        for (size_t i = 0; i < (1ULL << global_depth_); ++i) {
110            if ((i & mask) == (idx & mask))
111                directory_[i] = (i == idx) ? b0 : b1;
112            else if ((i & mask) == ((idx + step / 2)
113                                & mask))
114                directory_[i] = (i == idx + step / 2)
115                               ? b1 : directory_[i];
116        }
117        delete old_bkt;
118    }
119

```

```

120     size_t bucket_cap_;
121     int global_depth_;
122     std::vector<Bucket*> directory_;
123 };

```

8.13.3 Shrinking the Directory

When many buckets merge (e.g. after deletions), the directory can shrink. If every pointer in the directory points to the same two buckets and the global depth exceeds 1, halve the directory and decrement `global_depth_`. The shrink logic mirrors doubling: pair directory slots and merge entries when the local depth of the surviving bucket is strictly less than the current global depth.

8.13.4 Complexity

Operation	Average	Worst Case
Insert	$O(1)$ amortized	$O(B)$ per split
Find	$O(B)$	$O(B)$
Erase	$O(B)$	$O(B)$
Directory size	$O(2^d)$	never exceeds $2^{\text{keys}/B}$

The maximum directory depth is $\lceil \log_2(n/B) \rceil$, where n is the number of stored keys and B is the bucket capacity. Because splits are local (only one bucket at a time), extendible hashing avoids the $O(n)$ rehash pause of a conventional hash table.

When to use: Extendible hashing is the standard choice for disk-based databases (e.g. the *IBM DB2* and *LMDB* hash indexes) where rehashing millions of pages at once is unacceptable. In-memory applications typically prefer simpler designs, but the same idea underlies *linear hashing* (Litwin 1980), which splits buckets sequentially rather than by directory doubling.

8.14 Complexity Analysis

Operation	Separate Chaining	Open Addressing (double hash)
average insert	$O(1)$	$O(1)$
average find	$O(1)$	$O(1)$
average erase	$O(1)$	$O(1)$
worst-case find	$O(n)$	$O(n)$
space overhead	$O(n + m)$	$O(m)$

The worst case occurs when all keys hash to the same bucket. With a good hash function and proper resizing, this is exponentially unlikely.

8.15 Comparison of Dictionaries

Criteria	Hash Table	Balanced Tree (std::map)
Average search	$O(1)$	$O(\log n)$
Worst-case search	$O(n)$	$O(\log n)$
Ordered iteration	No	Yes (in-order)
Range queries	No	Yes
Memory	More (load factor ≥ 1)	Less (exactly n nodes)
Iterator validity	Invalidated on rehash	Stable

When to use which:

- Hash table: unordered map, need $O(1)$ average lookup
- Tree map: need ordered iteration, range queries, or worst-case guarantees

8.16 Application: LZW Compression

LZW (Lempel-Ziv-Welch) compression builds a dictionary of phrases during compression:

```

1 std::vector<int> lzw_compress(std::string_view input) {
2     // Initialize dictionary with single characters
3     hash_table_chaining<std::string, int> dict;
4     for (int i = 0; i < 256; ++i) {
5         dict.insert(std::string(1, static_cast<char>(i)), i);
6     }
7
8     std::vector<int> output;
9     std::string w;
10    int next_code = 256;
11
12    for (char c : input) {
13        std::string wc = w + c;
14        if (dict.find(wc).has_value()) {
15            w = wc;
16        } else {
17            output.push_back(*dict.find(w));
18            dict.insert(wc, next_code++);
19            w = std::string(1, c);
20        }
21    }
22    if (!w.empty()) output.push_back(*dict.find(w));
23    return output;
24 }
```

Trace: compress "ABABABA"

Step	w	c	wc	in dict?	Output	New dict entry
1	—	A	A	yes	—	—
2	A	B	AB	no	65 (A)	AB → 256
3	B	A	BA	no	66 (B)	BA → 257
4	A	B	AB	yes (256)	—	—
5	AB	A	ABA	no	256 (AB)	ABA → 258
6	A	B	AB	yes (256)	—	—
7	AB	A	ABA	yes (258)	—	—
8	ABA	—	—	—	258 (ABA)	—

Output: [65, 66, 256, 258] — 4 codes vs 7 input characters.

8.17 STL Connection: `std::unordered_map`

```

1 std::unordered_map<std::string, int> map;
2 map["apple"] = 5;
3 map["banana"] = 3;
4
5 auto it = map.find("apple");
6 if (it != map.end()) {
7     std::cout << it->second; // 5
8 }
9
10 // Iterate (order is unspecified)
11 for (const auto& [key, value] : map) {
12     std::cout << key << ": " << value << '\n';
13 }
```

Implementation notes:

- Uses separate chaining with linked lists
- Rehashes when load factor exceeds `max_load_factor()` (default 1.0)
- Buckets are grouped into linked lists; iterators remain valid after insert (but not after rehash)
- `std::unordered_set` is the key-only version

8.18 Common Bugs and Pitfalls

Pitfall	Consequence
Using a poor hash function (e.g., identity for integers)	Clustering, $O(n)$ lookups instead of $O(1)$
Forgetting to rehash when load factor exceeds threshold	Severe performance degradation
Using mutable objects as hash keys	Object in wrong bucket after mutation, unfindable
Hashing floating-point NaN	$\text{NaN} \neq \text{NaN}$, key becomes unfindable
Iterator invalidation after rehash in <code>std::unordered_map</code>	Use-after-free on next increment
Using <code>operator[]</code> for lookups	Inserts default value if key missing

8.19 Summary

1. **Hash functions** map keys to table indices. Good hash functions distribute keys uniformly.
2. **Separate chaining** uses linked lists per bucket. Simple and robust. α can exceed 1.
3. **Open addressing** stores all elements in the table array. Double hashing gives the best probe sequence.
4. **Rehashing** maintains the load factor below a threshold, preserving $O(1)$ average operations.
5. **Modern designs** (Robin Hood, Cuckoo, Swiss Table) improve cache performance and worst-case behavior.
6. `std::unordered_map` is the production implementation.

8.20 Interview FAQs

1. **What is the difference between separate chaining and open addressing? When would you use each?**

Separate chaining stores a linked list (or other container) at each bucket. Load factor can exceed 1.0. Simple to implement and delete. Open addressing stores all elements in the array itself, probing to the next slot on collision. Better cache performance (no pointer chasing) but requires careful deletion (tombstones) and load factor < 1.0 . Use chaining for variable-size workloads and when deletions are frequent. Use open addressing for small, fixed-size tables where cache efficiency matters (e.g., Swiss Table in `abseil`).

1. **What is Robin Hood hashing and why is it used?**

Robin Hood hashing steals from “rich” buckets (elements close to their ideal position) to give to “poor” buckets (elements far from home). During insertion, if the new element’s probe distance exceeds the current bucket’s probe distance, swap them and continue. This minimizes the maximum probe length, giving $O(1)$ worst-case lookup with high probability. Used in Rust’s `HashMap` and LinkedIn’s data infrastructure.

```

1 class robin_hood {
2     struct slot { int key = -1; int dist = 0; bool occupied = false; };
3     std::vector<slot> table_;
4     std::size_t size_ = 0;
5     std::size_t hash(int k) const { return (static_cast<std::size_t>(k) %
        table_.size() + table_.size()) % table_.size(); }
6 public:
7     explicit robin_hood(std::size_t cap) : table_(cap) {}
8     void insert(int key) {
9         if (size_ >= table_.size() / 2) grow();
10        slot s(key, 0, true);
11        auto idx = hash(key);
12        for (std::size_t i = 0; i < table_.size(); ++i) {
13            if (!table_[idx].occupied) { table_[idx] = s; ++size_; return;
14        }
15            if (table_[idx].key == key) return;
16            if (s.dist > table_[idx].dist) std::swap(s, table_[idx]);
17            idx = (idx + 1) % table_.size();
18            ++s.dist;
19        }
20        bool find(int key) const {
21            auto idx = hash(key);
22            for (std::size_t d = 0; d < table_.size(); ++d) {
23                if (!table_[idx].occupied) return false;
24                if (table_[idx].key == key) return true;
25                if (static_cast<int>(d) > table_[idx].dist) return false;
26                idx = (idx + 1) % table_.size();
27            }
28            return false;
29        }
30        void grow() {
31            std::vector<slot> old = std::move(table_);
32            table_.assign(old.size() * 2, slot{});
33            size_ = 0;
34            for (auto& s : old) if (s.occupied) insert(s.key);
35        }
36    };

```

1. How does a hash table achieve $O(1)$ average lookup?

Q: “Why is the load factor threshold for open addressing (typically 0.75) lower than for separate chaining?”

A: In open addressing, all elements share the same probe chain — as load factor α approaches 1.0, probe lengths grow exponentially (not just linearly). At $\alpha = 0.9$, the average probe length is ~ 5 ; at $\alpha = 0.95$, it's ~ 10 . The 0.75 threshold is a pragmatic balance between memory waste and probe length. In separate chaining, load factor > 1.0 just means longer lists, which degrades

linearly (average chain length = α). Google’s Swiss Table pushes the threshold higher (~ 0.875) because SIMD scanning of 16 control bytes at a time makes probing cheaper — each SIMD instruction checks 16 slots simultaneously.

1. When should you rehash and what is the cost?

Rehash when load factor $\alpha > \text{threshold}$ (0.75 for `std::unordered_map`). Allocate a new table of double the size, re-insert all elements. Amortized cost: each element is rehashed at most once every n insertions, giving $O(1)$ amortized per insert. Worst-case single rehash is $O(n)$. For latency-sensitive applications, incremental rehashing (spread rehash across operations) avoids $O(n)$ pauses — used in Redis and Memcached.

1. Design a hash function for a custom type.

Requirements: (1) deterministic, (2) uniform distribution, (3) fast to compute. Strategy: combine the hash values of each field using a seed-dependent combine: $h = h * \text{seed} + \text{hash}(\text{field})$ for each field. In C++17, use `std::hash` for each field and combine with XOR or boost’s `hash_combine`.

```
1 struct point { int x, y; };
2 struct point_hash {
3     size_t operator()(const point& p) const {
4         size_t h = std::hash<int>{}(p.x);
5         h ^= std::hash<int>{}(p.y) + 0x9e3779b9 + (h << 6) + (h >> 2);
6         return h;
7     }
8 };
```

1. What is a hash collision attack and how do you defend against it?

An attacker crafts keys that all hash to the same bucket, degrading $O(1)$ lookup to $O(n)$. defenses: (1) use a randomized hash function (SipHash, used by Python and Rust), seeded with a random value at startup. (2) Limit the maximum load factor. (3) Switch to a tree for large buckets (Java 8 HashMap: bins with > 8 elements become red-black trees). (4) Use open addressing with a random probe sequence.

1. What is Swiss Table hashing and why is it faster?

Swiss Table (used by Google’s `abseil::AbslHashMap`) stores a separate control byte per slot indicating whether it’s empty, deleted, or occupied, plus partial hash bits. Lookup scans 16 control bytes at a time using SIMD (SSE2/AVX2), comparing 16 partial hashes in a single instruction. This makes lookup $2\text{--}4\times$ faster than traditional chaining or open addressing for small-to-medium tables. The control byte array is contiguous, maximizing cache utilization.

1. Why does `std::unordered_map` have worse performance than `std::map` in some cases?

For small collections (< 50 elements), `std::map` (red-black tree) can be faster because: (1) no hash function overhead, (2) better cache locality for small trees (nodes fit in L1 cache), (3) predictable $O(\log n)$ without hash collisions. `std::unordered_map` dominates for large collections where $O(1)$ amortized wins over $O(\log n)$. Always benchmark with your actual workload and data size.

1. How do you implement an iterator for a hash table with separate chaining?

Q: “Why does `std::unordered_map` invalidate iterators on rehash but not on regular insert, and how does the implementation achieve this?”

A: Rehashing moves all elements to new buckets, invalidating bucket pointers. But regular insert only appends to a chain — existing iterators still point to valid nodes. The implementation maintains a linked list of all elements (in insertion order) for iterator traversal, plus a separate bucket array for $O(1)$ lookup. This dual structure is why iteration order is insertion order (not sorted), and why `unordered_map` uses more memory than a pure hash table. The linked list adds one pointer per element overhead but enables stable iterators during normal inserts — a deliberate tradeoff.

1. What is the difference between `std::unordered_map` and `std::unordered_set`?

Q: “When would you use a Bloom filter instead of an `unordered_set` for membership testing?”

A: A Bloom filter uses $O(m)$ bits for n elements — much less than a hash table’s per-node overhead. It supports insert + query (with false positives) but no delete, no enumeration, and no key-value storage. The memory savings can be 10–20× over a hash set. Use cases: spell checkers (is this word in the dictionary?), network routers (is this IP blacklisted?), database query optimization (does this key exist in this SSTable?). The false positive rate is configurable: $P = (1 - e^{-kn/m})^k$ where k = number of hash functions, m = bits, n = elements. For 1% false positive rate, you need ~9.6 bits per element — an `unordered_set` needs ~64+ bytes per element.

8.21 Hash Trees (Merkle Trees)

A **hash tree** (or Merkle tree) is a tree of hashes where each leaf stores the hash of a data block, and each internal node stores the hash of its children’s hashes. This structure enables efficient verification of large datasets.

8.21.1 Structure

```
1 struct HashNode {
2     std::string hash; // SHA-256 or similar
3     std::shared_ptr<HashNode> left, right;
4     bool is_leaf;
5     std::vector<char> data; // only for leaves
```

```

6 };
7
8 std::string compute_hash(const std::string& a,
9                          const std::string& b) {
10     return sha256(a + b); // combine children
11 }
12
13 std::shared_ptr<HashNode> build_merkle(
14     std::vector<std::vector<char>>& blocks) {
15     std::vector<std::shared_ptr<HashNode>> leaves;
16     for (auto& block : blocks) {
17         auto node = std::make_shared<HashNode>();
18         node->hash = sha256(
19             std::string(block.begin(), block.end()));
20         node->data = block;
21         node->is_leaf = true;
22         leaves.push_back(node);
23     }
24
25     while (leaves.size() > 1) {
26         std::vector<std::shared_ptr<HashNode>> next;
27         for (size_t i = 0; i < leaves.size();
28             i += 2) {
29             auto node = std::make_shared<HashNode>();
30             node->left = leaves[i];
31             node->right = (i + 1 < leaves.size())
32                 ? leaves[i + 1] : leaves[i];
33             node->hash = compute_hash(
34                 node->left->hash, node->right->hash);
35             node->is_leaf = false;
36             next.push_back(node);
37         }
38         leaves = next;
39     }
40     return leaves[0];
41 }

```

8.21.2 Verification

To verify a single block, follow the path from leaf to root, recomputing hashes at each level. This requires $O(\log n)$ hash computations instead of recomputing all hashes.

8.21.3 Applications

- **Blockchain:** Bitcoin and Ethereum use Merkle trees to verify transaction integrity
- **Version control:** Git uses Merkle trees (“commit objects”) for repository integrity

- **Distributed storage:** IPFS and HDFS use Merkle trees for data verification across nodes
- **Certificate transparency:** TLS certificate logs use Merkle trees for auditability

8.22 Exercises

8.22.1 Drill

1. For a hash table with 11 buckets and the hash function $h(k) = k \bmod 11$, insert the keys [22, 1, 13, 11, 24, 33, 18, 42, 15] using:
 - a) Separate chaining b) Linear probing c) Double hashing with $h_2(k) = 7 - (k \bmod 7)$
 Show the state of the table after each insertion.
1. What is the load factor after inserting the keys in exercise 1? What is the average number of probes per successful search for the linear probing version?
1. For a hash table of size $m = 1000$ with double hashing, how many probes are needed in the worst case to find an empty slot when $\alpha = 0.75$? When $\alpha = 0.95$?
1. Which of the following make good hash functions for strings? For each, explain why or why not:
 - a) Sum of ASCII values b) First character's ASCII value c) Length of the string d) djb2: $h = h \times 33 + c_i$ for each character
1. Why is a "universal hash function" preferable to a fixed hash function? Give an example scenario where a fixed hash function fails.

8.22.2 Application

1. Implement a `hash_table_chaining` class with automatic rehashing when the load factor exceeds a configurable threshold. Compare its performance with `std::unordered_map` for 10^6 insertions and 10^6 lookups of random 64-bit integers.
1. Implement Robin Hood hashing with linear probing. After each insertion, swap the inserted element forward if it is farther from its ideal bucket than existing elements. Measure the maximum probe distance at $\alpha = 0.8$ and compare with standard linear probing.
1. Implement a hash function for `std::pair<int, int>` that produces good distribution. Test it by inserting 10^5 distinct pairs and measuring the maximum bucket size.
1. Implement LZW compression and decompression using a hash table for the dictionary. Verify that compress followed by decompress reproduces the original string. Test on "ABABABABA" and a short text file.

1. Design and implement a hash function for IPv4 addresses. What properties does it need? Test it on a dataset of real IP addresses (you can generate a synthetic dataset with common prefixes).
1. Implement Cuckoo hashing with two hash tables and the cycle-detection rehash strategy. Compare its worst-case lookup time with linear probing.
1. Implement a **Bloom filter** (see also Chapter 19) using two hash functions. Compare its false positive rate with the theoretical prediction for varying m/n ratios.
1. Write a spell checker that uses a hash table of dictionary words. Test it on a document and report the misspelled words found. Measure the lookup time per word.

8.22.3 Research

1. **(Iterator invalidation)** Why does C++ mandate that `std::unordered_map` iterators remain valid after insert (until rehash)? How does this constraint affect implementation choices? Read the standard committee discussions.
1. **(Swiss Table)** Google's Abseil library implements a "Swiss Table" (or "raw hash table") that uses SIMD instructions for fast probing. Read the design document. Implement a simplified version that processes 4 buckets at a time using bit operations.
1. **(Perfect hashing)** Research FKS perfect hashing (Fredman, Komlos, Szemerédi 1984). Implement a static perfect hash table that supports $O(1)$ worst-case lookups for a predetermined set of keys.
1. **(Hash flooding)** Read about hash flooding attacks against `std::unordered_map`. How did C++17's addition of `std::hash<std::filesystem::path>` and the adoption of randomized hashing mitigate this? Implement a hash table that is immune to simple collision attacks.

8.22.4 Additional Exercises

1. Implement a Count-Min Sketch with width 2719 and depth 7. Feed it 10^6 random strings and compare the estimated frequencies of the 100 most frequent strings against exact counts. What is the maximum observed overestimation?
1. Design a Count-Mean Sketch that supports deletion by maintaining an auxiliary table of per-cell estimates. Verify that after inserting and then deleting a set of keys, the estimated frequencies return to zero.
1. Build a Bloom filter using $k = 3$ independent hash functions. Insert n random 64-bit integers and measure the false positive rate for m/n ratios of 5, 8, 10, and 15. Plot the results against the theoretical formula $p = (1 - e^{-kn/m})^k$.

1. Implement Robin Hood hashing with **backshift deletion** (no tombstones). Insert 10^5 random keys, then delete every third key. Measure the maximum probe distance after deletions and compare with standard linear probing using tombstones.
1. Implement a simplified Swiss Table that processes 4 buckets at a time using 64-bit bitmasks instead of SIMD. Compare its throughput against `std::unordered_map` for 10^6 insertions and 10^6 lookups.
1. Implement Cuckoo hashing with two hash tables of size $2n$ and a maximum of 10 evictions before rehashing. Insert n random keys and measure the fraction of insertions that trigger a rehash.
1. Implement a **counting Bloom filter** where each cell is a 4-bit counter instead of a single bit. Insert n keys, delete $n/2$ of them, and report the false positive rate. How does it compare with a standard Bloom filter that has been cleared?
1. Implement a **quotient filter** using open addressing with linear probing on quotient and remainder pairs. Compare its cache performance against a standard chaining hash table for 10^6 random 32-bit integer insertions and lookups.
1. Write a program that builds a frequency table of all words in a text file using a hash table, then uses a Count-Min Sketch to estimate the same frequencies. Compare the top-50 most frequent words reported by each method. How many of the true top-50 does the sketch capture?
1. Implement a **split-ordered list** hash table (used by Linux kernel's `rhashtable`). Elements are stored in a singly linked list sorted by a reversed-bit hash value, enabling lock-free reads during resize. Benchmark concurrent inserts and lookups with multiple threads.

8.23 References and Further Reading

- Donald E. Knuth, *The Art of Computer Programming*, Volume 3: Sorting and Searching. Section 6.4.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapter 11.
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Section 3.4.
- Pedro Celis, "Robin Hood Hashing" — PhD thesis, University of Waterloo, 1986.
- Rasmus Pagh and Flemming Friche Rodler, "Cuckoo Hashing" — ESA 2001.
- Michael L. Fredman, Janos Komlos, and Endre Szemerédi, "Storing a Sparse Table with $O(1)$ Worst Case Access Time" — JACM, 1984.
- Abseil C++ library, "Swiss Table" design document — <https://abseil.io/docs/cpp/guides/container>

- Go programming language, "Swiss Table" — <https://go.dev/blog/swisstable>
- J. T. Robinson, "The K-D-B-Tree: A Search Structure for Large Multidimensional Dynamic Indexes" — SIGMOD 1981.

Trees and Binary Trees

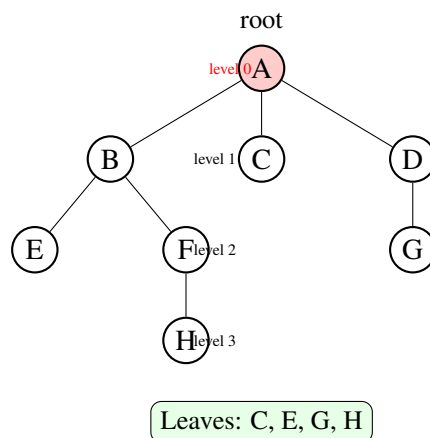
9.1 Introduction

Trees are the most important non-linear data structure in computing. They organize data hierarchically — file systems, parse trees, decision trees, and search structures all use trees.

A **tree** consists of **nodes** connected by **edges**. Each node has exactly one **parent** (except the **root**, which has none) and zero or more **children**. Nodes with no children are **leaves**.

⚡ **Complete compilable implementations of the data structures in this chapter are in `code/`.**

9.2 Terminology



- **Root:** A (no parent)
- **Parent/Child:** A is parent of B, C, D; B is child of A
- **Leaf:** C, E, G, H (nodes with no children)
- **Siblings:** B, C, D share parent A
- **Path:** sequence of connected nodes: $A \rightarrow B \rightarrow F \rightarrow H$
- **Path length:** number of edges on a path (3 for A to H)
- **Depth of node:** path length from root (depth of H = 3)

- **Height of node:** longest path from node to a leaf (height of B = 2)
- **Height of tree:** height of root (height = 3)
- **Subtree:** any node and its descendants
- **Degree:** number of children (A has degree 3, H has degree 0)

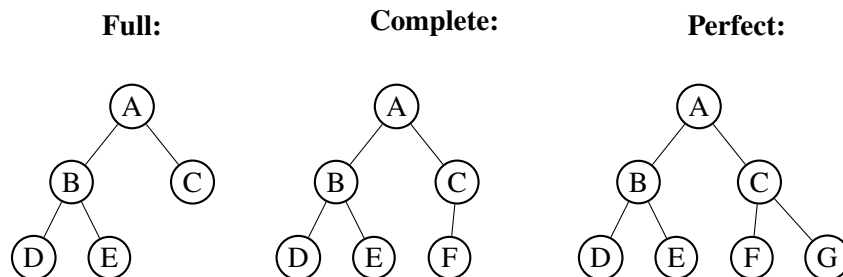
9.3 Binary Trees

A **binary tree** is a tree where each node has at most 2 children: **left** and **right**.

9.3.1 Properties

1. Maximum nodes at level i : 2^i
2. Maximum nodes in a tree of height h : $2^{h+1} - 1$
3. Minimum height of a tree with n nodes: $\lceil \log_2(n+1) \rceil - 1$
4. A **full binary tree** has every node with 0 or 2 children.
5. A **complete binary tree** is full except possibly the last level, which is filled left to right.
6. A **perfect binary tree** has all leaves at the same level.

Internal path length: sum of depths of all internal (non-leaf) nodes. **External path length:** sum of depths of all leaves.



9.3.2 Binary Tree Node

```

1 template <typename T>
2 struct binary_node {
3     T data;
4     std::unique_ptr<binary_node> left;
5     std::unique_ptr<binary_node> right;
6
7     explicit binary_node(const T& value)
  
```



```

8         : data(value), left(nullptr), right(nullptr) {}
9     explicit binary_node(T&& value)
10        : data(std::move(value)), left(nullptr), right(nullptr) {}
11 };

```

9.3.3 Binary Tree Class

```

1  template <typename T>
2  class binary_tree {
3  public:
4      using node_ptr = std::unique_ptr<binary_node<T>>;
5
6      binary_tree() = default;
7
8      // Construct from root
9      explicit binary_tree(node_ptr root) : root_(std::move(root)) {}
10
11     bool empty() const noexcept { return root_ == nullptr; }
12
13     // Preorder traversal
14     template <typename Visitor>
15     void preorder(Visitor&& visit) const {
16         preorder(root_.get(), visit);
17     }
18
19     // Inorder traversal
20     template <typename Visitor>
21     void inorder(Visitor&& visit) const {
22         inorder(root_.get(), visit);
23     }
24
25     // Postorder traversal
26     template <typename Visitor>
27     void postorder(Visitor&& visit) const {
28         postorder(root_.get(), visit);
29     }
30
31     // Level-order traversal (breadth-first)
32     template <typename Visitor>
33     void level_order(Visitor&& visit) const {
34         if (!root_) return;
35         std::queue<binary_node<T>*> q;
36         q.push(root_.get());
37         while (!q.empty()) {
38             auto* node = q.front(); q.pop();
39             visit(node->data);
40             if (node->left) q.push(node->left.get());

```

```

41         if (node->right) q.push(node->right.get());
42     }
43 }
44
45 size_t height() const { return height(root_.get()); }
46 size_t size() const { return size(root_.get()); }
47
48 private:
49     static void preorder(binary_node<T>* node, auto& visit) {
50         if (!node) return;
51         visit(node->data);
52         preorder(node->left.get(), visit);
53         preorder(node->right.get(), visit);
54     }
55
56     static void inorder(binary_node<T>* node, auto& visit) {
57         if (!node) return;
58         inorder(node->left.get(), visit);
59         visit(node->data);
60         inorder(node->right.get(), visit);
61     }
62
63     static void postorder(binary_node<T>* node, auto& visit) {
64         if (!node) return;
65         postorder(node->left.get(), visit);
66         postorder(node->right.get(), visit);
67         visit(node->data);
68     }
69
70     static size_t height(binary_node<T>* node) {
71         if (!node) return 0;
72         return 1 + std::max(height(node->left.get()), height(node->right.
73         get()));
74     }
75
76     static size_t size(binary_node<T>* node) {
77         if (!node) return 0;
78         return 1 + size(node->left.get()) + size(node->right.get());
79     }
80     node_ptr root_;
81 };

```

9.3.4 Traversal Walkthrough

Tree:

1	A
---	---

```

2      / \
3     B  C
4    / \
5   D  E

```

Preorder (root → left → right): A, B, D, E, C **Inorder (left → root → right):** D, B, E, A, C
Postorder (left → right → root): D, E, B, C, A **Level order (BFS):** A, B, C, D, E

9.3.5 Iterative Traversals (No Recursion)

```

1  template <typename Visitor>
2  void preorder_iterative(Visitor&& visit) const {
3      if (!root_) return;
4      std::stack<binary_node<T>*> s;
5      s.push(root_.get());
6      while (!s.empty()) {
7          auto* node = s.top(); s.pop();
8          visit(node->data);
9          if (node->right) s.push(node->right.get());
10         if (node->left) s.push(node->left.get());
11     }
12 }
13
14 template <typename Visitor>
15 void inorder_iterative(Visitor&& visit) const {
16     std::stack<binary_node<T>*> s;
17     auto* current = root_.get();
18     while (current || !s.empty()) {
19         while (current) {
20             s.push(current);
21             current = current->left.get();
22         }
23         current = s.top(); s.pop();
24         visit(current->data);
25         current = current->right.get();
26     }
27 }
28
29 template <typename Visitor>
30 void postorder_iterative(Visitor&& visit) const {
31     if (!root_) return;
32     std::stack<binary_node<T>*> s1, s2;
33     s1.push(root_.get());
34     while (!s1.empty()) {
35         auto* node = s1.top(); s1.pop();
36         s2.push(node);
37         if (node->left) s1.push(node->left.get());
38         if (node->right) s1.push(node->right.get());

```

```

39     }
40     while (!s2.empty()) {
41         visit(s2.top()->data);
42         s2.pop();
43     }
44 }

```

9.4 Array Representation of Binary Trees

A **complete binary tree** can be stored efficiently in an array:

```

1 Index:  0  1  2  3  4  5  6
2 Data:  [A, B, C, D, E, F, G]
3
4 For node at index i:
5   left child:  2i + 1
6   right child: 2i + 2
7   parent:      (i - 1) / 2

```

This is the basis for heap storage (Chapter 10).

9.5 Applications of Binary Trees

9.5.1 Expression Trees

An expression tree represents an arithmetic expression: leaves are operands, internal nodes are operators.

```

1 Expression: (3 + 4) * 5
2
3           [*]
4          /  \
5         [+]  [5]
6        /  \
7       [3]  [4]

```

```

1 struct expr_node {
2     std::string value;           // operator or operand
3     std::unique_ptr<expr_node> left;
4     std::unique_ptr<expr_node> right;
5
6     bool is_operator() const {
7         return value == "+" || value == "-" || value == "*" || value == "/"
8             || value == "%";
9     }
10 };

```

```

11 int evaluate(const expr_node* node) {
12     if (!node->is_operator()) {
13         return std::stoi(node->value);
14     }
15     int l = evaluate(node->left.get());
16     int r = evaluate(node->right.get());
17     if (node->value == "+") return l + r;
18     if (node->value == "-") return l - r;
19     if (node->value == "*") return l * r;
20     return l / r;
21 }

```

9.5.2 Morse Code Tree

The Morse code tree encodes letters as binary choices (dot = left, dash = right):

```

1           start
2          /   \
3         E     T
4        / \   / \
5       I  A  N  M
6      / \ / \ / \ / \
7     S  U R W D K G O

```

9.6 Union-Find (Disjoint Set Union)

Union-Find tracks a partition of elements into disjoint sets. It supports:

- **find(x)**: which set does x belong to?
- **union(x, y)**: merge the sets containing x and y

9.6.1 Tree Representation

Each set is represented as a tree. The root is the set's representative. Each node stores a pointer to its parent.

```

1 class union_find {
2 public:
3     union_find(size_t n) : parent_(n), rank_(n, 0) {
4         for (size_t i = 0; i < n; ++i) parent_[i] = i;
5     }
6
7     // Find with path compression
8     size_t find(size_t x) {
9         if (parent_[x] != x) {
10             parent_[x] = find(parent_[x]); // compress

```

```

11     }
12     return parent_[x];
13 }
14
15 // Union by rank
16 void union_sets(size_t x, size_t y) {
17     size_t rx = find(x);
18     size_t ry = find(y);
19     if (rx == ry) return;
20
21     if (rank_[rx] < rank_[ry]) {
22         parent_[rx] = ry;
23     } else if (rank_[rx] > rank_[ry]) {
24         parent_[ry] = rx;
25     } else {
26         parent_[ry] = rx;
27         ++rank_[rx];
28     }
29 }
30
31 bool same_set(size_t x, size_t y) {
32     return find(x) == find(y);
33 }
34
35 private:
36     std::vector<size_t> parent_;
37     std::vector<size_t> rank_;
38 };

```

Complexity: m operations = $O(m \cdot \alpha(n))$, where α is the inverse Ackermann function — effectively constant for all practical n .

9.6.2 Application: Kruskal's MST

Union-Find is essential for Kruskal's minimum spanning tree algorithm (Chapter 17).

9.6.3 Application: Image Connected Components

Label connected regions in a binary image:

```

1 std::vector<std::vector<int>>> connected_components(
2     const std::vector<std::vector<bool>>>& image) {
3     int rows = image.size();
4     int cols = image[0].size();
5     union_find uf(rows * cols);
6
7     // First pass: union adjacent foreground pixels
8     for (int r = 0; r < rows; ++r) {

```

```

9         for (int c = 0; c < cols; ++c) {
10             if (!image[r][c]) continue;
11             int idx = r * cols + c;
12             if (r > 0 && image[r-1][c])
13                 uf.union_sets(idx, (r-1) * cols + c);
14             if (c > 0 && image[r][c-1])
15                 uf.union_sets(idx, r * cols + (c-1));
16         }
17     }
18
19     // Second pass: assign labels
20     std::unordered_map<size_t, int> label_map;
21     int next_label = 0;
22     std::vector<std::vector<int>> labels(rows, std::vector<int>(cols, 0));
23
24     for (int r = 0; r < rows; ++r) {
25         for (int c = 0; c < cols; ++c) {
26             if (!image[r][c]) continue;
27             size_t root = uf.find(r * cols + c);
28             auto it = label_map.find(root);
29             if (it == label_map.end()) {
30                 label_map[root] = ++next_label;
31                 it = label_map.find(root);
32             }
33             labels[r][c] = it->second;
34         }
35     }
36     return labels;
37 }

```

9.7 Reconstructing Binary Trees from Traversals

Given inorder + preorder (or inorder + postorder), we can uniquely reconstruct a binary tree.

Example: Inorder: [D, B, E, A, C], Preorder: [A, B, D, E, C]

```

1 A is the root (first in preorder).
2 In inorder, everything left of A is left subtree, right of A is right
  subtree.
3 Left subtree inorder: [D, B, E], preorder: [B, D, E]
4 Right subtree inorder: [C], preorder: [C]
5
6 Recursively build:
7 B is root of left subtree.
8 Inorder left of B: [D] -> left child
9 Inorder right of B: [E] -> right child

```

```

1 std::unique_ptr<binary_node<char>> build_tree(

```

```

2     std::span<const char> inorder,
3     std::span<const char> preorder) {
4     if (inorder.empty()) return nullptr;
5
6     char root_val = preorder[0];
7     auto root = std::make_unique<binary_node<char>>(root_val);
8
9     // Find root in inorder
10    size_t split = 0;
11    while (split < inorder.size() && inorder[split] != root_val) ++split;
12
13    root->left = build_tree(
14        inorder.subspan(0, split),
15        preorder.subspan(1, split));
16    root->right = build_tree(
17        inorder.subspan(split + 1),
18        preorder.subspan(split + 1));
19
20    return root;
21 }

```

9.8 Decision Trees

A decision tree is a binary tree used for classification and prediction. Each internal node tests a feature (attribute), each branch corresponds to a test outcome, and each leaf holds a class label. Decision trees are widely used in machine learning, game AI, and rule-based expert systems because they are easy to interpret and fast to evaluate.

```

1             [Temperature > 30?]
2             /           \
3           Yes           No
4           /           \
5   [Humidity > 70?]   [Play?]
6       /       \       Yes
7     Yes     No
8       /       \
9   [No]     [Yes]

```

9.8.1 Structure

Each node stores a feature index and a threshold for numeric features, or a feature value for categorical features. Leaf nodes store a class label instead of a test.

```

1 struct DecisionNode {
2     int feature_index;      // which feature to test

```



```

3     double threshold;           // split threshold (numeric)
4     int class_label;           // valid only at leaves
5     bool is_leaf;
6     DecisionNode* left;        // <= threshold (or feature == false)
7     DecisionNode* right;       // > threshold (or feature == true)
8
9     // Leaf constructor
10    DecisionNode(int label)
11        : feature_index(-1), threshold(0),
12          class_label(label), is_leaf(true),
13          left(nullptr), right(nullptr) {}
14
15    // Internal node constructor
16    DecisionNode(int feat, double thresh,
17                 DecisionNode* l, DecisionNode* r)
18        : feature_index(feat), threshold(thresh),
19          class_label(-1), is_leaf(false),
20          left(l), right(r) {}
21 };

```

9.8.2 Prediction

Prediction traverses the tree from root to leaf, following branches based on the feature values of the input sample.

```

1 int predict(DecisionNode* root,
2             const vector<double>& sample) {
3     DecisionNode* curr = root;
4     while (!curr->is_leaf) {
5         if (sample[curr->feature_index] <= curr->threshold)
6             curr = curr->left;
7         else
8             curr = curr->right;
9     }
10    return curr->class_label;
11 }

```

9.8.3 Building from Training Data

A greedy approach selects the feature and threshold that best splits the training data at each node. A common criterion is information gain (based on entropy).

```

1 // Find best split among all features and thresholds
2 // Returns (best_feature, best_threshold, best_gain)
3 struct Split {
4     int feature;
5     double threshold;

```

```

6     double gain;
7 };
8
9 Split find_best_split(const vector<vector<double>>& X,
10                     const vector<int>& y) {
11     int n = X.size();
12     int d = X[0].size();
13     Split best{-1, 0.0, -1.0};
14
15     for (int f = 0; f < d; ++f) {
16         // Collect unique values of feature f
17         vector<double> vals;
18         for (int i = 0; i < n; ++i) vals.push_back(X[i][f]);
19         sort(vals.begin(), vals.end());
20         vals.erase(unique(vals.begin(), vals.end()),
21                   vals.end());
22
23         // Try each midpoint as threshold
24         for (size_t i = 0; i + 1 < vals.size(); ++i) {
25             double t = (vals[i] + vals[i + 1]) / 2.0;
26             vector<int> left_y, right_y;
27             for (int j = 0; j < n; ++j) {
28                 if (X[j][f] <= t) left_y.push_back(y[j]);
29                 else right_y.push_back(y[j]);
30             }
31             double gain = information_gain(y, left_y, right_y);
32             if (gain > best.gain)
33                 best = {f, t, gain};
34         }
35     }
36     return best;
37 }
38
39 DecisionNode* build_tree(const vector<vector<double>>& X,
40                          const vector<int>& y,
41                          int depth, int max_depth) {
42     // All same class => leaf
43     if (all_same(y))
44         return new DecisionNode(most_common(y));
45
46     // Max depth or no split found => leaf
47     if (depth >= max_depth)
48         return new DecisionNode(most_common(y));
49
50     Split s = find_best_split(X, y);
51     if (s.gain <= 0)
52         return new DecisionNode(most_common(y));
53

```

```

54     vector<vector<double>> X_left, X_right;
55     vector<int> y_left, y_right;
56     for (size_t i = 0; i < X.size(); ++i) {
57         if (X[i][s.feature] <= s.threshold) {
58             X_left.push_back(X[i]);
59             y_left.push_back(y[i]);
60         } else {
61             X_right.push_back(X[i]);
62             y_right.push_back(y[i]);
63         }
64     }
65
66     DecisionNode* left  = build_tree(X_left, y_left,
67                                     depth + 1, max_depth);
68     DecisionNode* right = build_tree(X_right, y_right,
69                                     depth + 1, max_depth);
70     return new DecisionNode(s.feature, s.threshold,
71                             left, right);
72 }

```

9.8.4 Complexity

Let n be the number of training samples, d the number of features, and h the height of the tree.

Operation	Time
Prediction	$O(h)$ — at most one comparison per level
Training (naive, single split)	$O(n \cdot d \cdot n)$ — for each feature, sort $O(n \log n)$, scan $O(n)$ total $O(d \cdot n \log n)$
Space	$O(2^h - 1)$ nodes in the worst case

9.8.5 Comparison with Hash Table for Classification

Criterion	Decision Tree	Hash Table
Lookup time	$O(h)$, typically $O(\log n)$	$O(1)$ average
Build time	$O(d \cdot n \log n)$	$O(n)$
Memory	$O(\text{nodes})$	$O(n)$ entries
Handles unseen inputs	Yes (follows branch)	No (key must exist)
Interpretability	High (readable rules)	Low
Handles continuous features	Yes (threshold splits)	Requires discretization

Decision trees are preferred when the feature space is continuous or when interpretability matters. Hash tables are preferred for exact lookups over a known, finite set of discrete inputs.

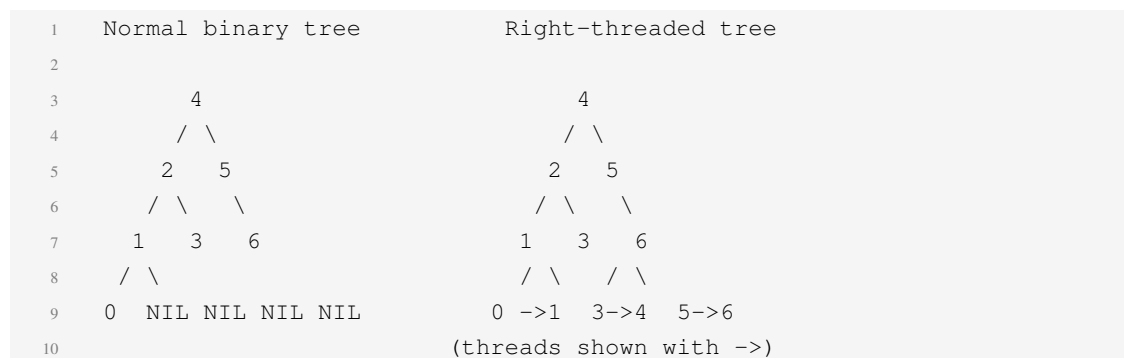
9.9 Threaded Binary Trees

9.9.1 Motivation

In a standard binary tree, many pointers are `nullptr`. For a tree with n nodes, there are $n + 1$ `nullptr` pointers. Threaded binary trees replace these null pointers with *threads*— pointers that provide shortcuts to in-order predecessors or successors. This enables in-order traversal without a stack and without recursion, using only $O(1)$ extra space.

9.9.2 Structure

In a right-in-threaded tree, every `nullptr` right child is replaced with a pointer to the node's in-order successor. A boolean flag distinguishes real children from threads.



9.9.3 Implementation

```

1 struct ThreadedNode {
2     int data;
3     ThreadedNode* left;
4     ThreadedNode* right;
5     bool left_thread;    // true => left is a thread
6     bool right_thread;   // true => right is a thread
7
8     ThreadedNode(int val)
9         : data(val), left(nullptr), right(nullptr),
10         left_thread(false), right_thread(false) {}
11 };
12
13 // Insert a node as the inorder successor of 'parent'
14 ThreadedNode* insert_right(ThreadedNode* parent,
15                             ThreadedNode* child) {
16     child->right = parent->right;
17     child->right_thread = parent->right_thread;
18     child->left = parent;

```

```

19     child->left_thread = true;
20     parent->right = child;
21     parent->right_thread = false;
22     if (child->right_thread)
23         child->right->left = child;
24     return child;
25 }

```

9.9.4 In-Order Successor Without a Stack

The key advantage: finding the in-order successor is $O(1)$ and requires no stack.

```

1 ThreadedNode* inorder_successor(ThreadedNode* node) {
2     if (node->right_thread)
3         return node->right; // thread points to successor
4
5     // Go to leftmost node in right subtree
6     ThreadedNode* curr = node->right;
7     while (!curr->left_thread)
8         curr = curr->left;
9     return curr;
10 }
11
12 // In-order traversal using threads -- no stack needed
13 void threaded_inorder(ThreadedNode* root) {
14     if (!root) return;
15
16     // Start at leftmost node
17     ThreadedNode* curr = root;
18     while (!curr->left_thread)
19         curr = curr->left;
20
21     while (curr) {
22         visit(curr);
23         curr = inorder_successor(curr);
24     }
25 }

```

9.9.5 Complexity

Operation	Standard Tree	Threaded Tree
In-order successor	$O(h)$ with stack	$O(1)$
In-order traversal	$O(n)$ time, $O(h)$ space	$O(n)$ time, $O(1)$ space
Insertion	$O(1)$ amortized	$O(1)$ amortized
Space per node	2 pointers	2 pointers + 2 booleans

The space saving comes from eliminating the traversal stack, not from the tree nodes themselves. Threaded trees are most beneficial when in-order traversal is performed frequently and memory is constrained.

9.10 Lowest Common Ancestor (LCA)

The **lowest common ancestor** of two nodes u and v in a tree is the deepest node that has both u and v as descendants (a node may be a descendant of itself). LCA is fundamental to path queries, network routing, and many tree problems.

9.10.1 Approach 1: Recursive (Binary Tree)

For a binary tree (not necessarily a BST), the LCA can be found in one post-order pass. At each node:

1. If the node is `null` or equals u or v , return it.
2. Recurse on the left and right subtrees.
3. If both subtrees return non-null, the current node is the LCA.
4. Otherwise, return whichever subtree returned non-null.

```

1 // Returns LCA of u and v in the tree rooted at node.
2 // Assumes both u and v exist in the tree.
3 node* lca_binary_tree(node* node, node* u, node* v) {
4     if (!node || node == u || node == v)
5         return node;
6     node* L = lca_binary_tree(node->left, u, v);
7     node* R = lca_binary_tree(node->right, u, v);
8     if (L && R) return node;    // u,v split here
9     return L ? L : R;
10 }
```

Time: $O(n)$ — each node visited once. **Space:** $O(h)$ for the recursion stack.

9.10.2 Approach 2: Euler Tour + RMQ

Reduce LCA to a *range minimum query* (RMQ) problem:

1. Perform an Euler tour of the tree (record each node every time it is visited, recording the depth at each occurrence).
2. Record the *first occurrence* of each node in the tour.
3. The LCA of u and v is the node with the minimum depth in the Euler tour between the first occurrences of u and v .

```

1 // After DFS, euler[] has node ids in visit order,
2 // depth[] has their depths, first[u] = first index
3 // of u in euler[]. Then:
4 //
5 //   LCA(u,v) = euler[ RMQ( depth, first[u], first[v] ) ]
6 //
7 // With a sparse table for RMQ:  $O(n \log n)$  preprocess,
8 //  $O(1)$  per query. Overall:  $O(n \log n) + O(q)$ .

```

This approach generalises easily to **weighted trees** and **dynamic trees** (segment-tree variant).

9.10.3 Approach 3: Binary Lifting (Preprocessing for $O(\log n)$ Queries)

Binary lifting precomputes 2^k -th ancestors of every node, enabling LCA queries in $O(\log n)$ after $O(n \log n)$ preprocessing. This is the most practical approach when many queries are needed on a static tree.

```

1 #include <vector>
2 #include <algorithm>
3
4 struct LCA {
5     int n, LOG;
6     std::vector<std::vector<int>>> adj;
7     std::vector<std::vector<int>>> up;    // up[k][v] = 2^k-th ancestor
8     std::vector<int> depth;
9
10    LCA(int n, const std::vector<std::vector<int>>>& g, int root = 0)
11        : n(n), adj(g) {
12        LOG = 0;
13        while ((1 << LOG) <= n) ++LOG;
14        up.assign(LOG, std::vector<int>(n, -1));
15        depth.assign(n, 0);
16        dfs(root, -1);
17    }
18
19    void dfs(int v, int p) {
20        up[0][v] = p;
21        for (int k = 1; k < LOG; ++k) {
22            int mid = up[k - 1][v];
23            up[k][v] = (mid == -1) ? -1 : up[k - 1][mid];
24        }
25        for (int to : adj[v]) {
26            if (to == p) continue;
27            depth[to] = depth[v] + 1;
28            dfs(to, v);
29        }
30    }
31

```

```

32  // Lift node v up by delta levels
33  int lift(int v, int delta) const {
34      for (int k = 0; k < LOG; ++k) {
35          if (delta & (1 << k))
36              v = up[k][v];
37      }
38      return v;
39  }
40
41  // LCA of u and v
42  int lca(int u, int v) const {
43      if (depth[u] < depth[v]) std::swap(u, v);
44      u = lift(u, depth[u] - depth[v]); // equalise depths
45      if (u == v) return u;
46      for (int k = LOG - 1; k >= 0; --k) {
47          if (up[k][u] != up[k][v]) {
48              u = up[k][u];
49              v = up[k][v];
50          }
51      }
52      return up[0][u]; // parent of both
53  }
54 };

```

Preprocessing: $O(n \log n)$ time, $O(n \log n)$ space. **Query:** $O(\log n)$.

9.10.4 Comparison

Method	Preprocess	Query	Space
Recursive (binary tree)	none	$O(n)$	$O(h)$ stack
Euler tour + RMQ (sparse table)	$O(n \log n)$	$O(1)$	$O(n \log n)$
Binary lifting	$O(n \log n)$	$O(\log n)$	$O(n \log n)$

9.11 Tree Diameter and Center

9.11.1 Diameter

The **diameter** of a tree is the length (in edges) of the longest path between any two nodes.

Two-BFS algorithm:

1. Run BFS from any node s ; let u be the farthest node found.
2. Run BFS from u ; let v be the farthest node from u .
3. The distance $d(u, v)$ is the diameter; the path $u \rightarrow \dots \rightarrow v$ is a longest path.

Why it works: If the true diameter endpoints are a, b , the first BFS finds some u on or near the diameter. The second BFS from u is guaranteed to reach b (or a), giving the full diameter.

```

1 #include <vector>
2 #include <queue>
3 #include <utility>
4
5 // Returns (diameter, farthest node from start)
6 std::pair<int, int> bfs_farthest(
7     const std::vector<std::vector<int>>& adj, int start) {
8     int n = adj.size();
9     std::vector<int> dist(n, -1);
10    std::queue<int> q;
11    dist[start] = 0;
12    q.push(start);
13    int farthest = start;
14    while (!q.empty()) {
15        int v = q.front(); q.pop();
16        for (int to : adj[v]) {
17            if (dist[to] != -1) continue;
18            dist[to] = dist[v] + 1;
19            q.push(to);
20            if (dist[to] > dist[farthest]) farthest = to;
21        }
22    }
23    return {dist[farthest], farthest};
24 }
25
26 int tree_diameter(const std::vector<std::vector<int>>& adj) {
27     auto [_1, u] = bfs_farthest(adj, 0);
28     auto [diam, _2] = bfs_farthest(adj, u);
29     return diam;
30 }
```

Time: $O(n)$ — two BFS passes.

9.11.2 Center

The **center** of a tree is the node (or pair of adjacent nodes) that minimises the maximum distance to all other nodes.

Algorithm: Repeatedly remove all leaves (nodes with degree 1) layer by layer until 1 or 2 nodes remain. Those are the center(s).

Worked trace on a tree with 7 nodes:

```

1 Step 0:  1 - 2 - 3 - 4 - 5
2           |
3           6 - 7
4
```

```

5 Degrees: 1:1  2:3  3:2  4:2  5:1  6:2  7:1
6 Remove leaves 1, 5, 7 --> 2 - 3 - 4, 6
7 Degrees: 2:2  3:2  4:1  6:1
8 Remove leaves 4, 6 --> 2 - 3
9 Degrees: 2:1  3:1
10 Remove leaves 2, 3 --> empty
11
12 Center = {2, 3} (last remaining pair).

```

Time: $O(n)$ — each edge examined at most twice.

9.12 Serialize and Deserialize Binary Tree

Serialization converts a tree to a string; deserialization reconstructs it. This is essential for storage, network transmission, and testing.

9.12.1 Preorder with Null Markers

Write node values in preorder. When a node is null, write a sentinel (#). Deserialization reads tokens in the same order and rebuilds the tree recursively.

```

1  #include <string>
2  #include <sstream>
3  #include <memory>
4  #include <vector>
5
6  struct TreeNode {
7      int val;
8      std::unique_ptr<TreeNode> left;
9      std::unique_ptr<TreeNode> right;
10     explicit TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
11 };
12
13 // ----- Serialize -----
14 void serialize(TreeNode* node, std::ostringstream& os) {
15     if (!node) { os << "# "; return; }
16     os << node->val << " ";
17     serialize(node->left.get(), os);
18     serialize(node->right.get(), os);
19 }
20
21 std::string serialize(TreeNode* root) {
22     std::ostringstream os;
23     serialize(root, os);
24     return os.str();
25 }
26
27 // ----- Deserialize -----

```

```

28 void deserialize_helper(std::istream& iss,
29                         std::unique_ptr<TreeNode>& node) {
30     std::string token;
31     iss >> token;
32     if (token == "#") return;
33     node = std::make_unique<TreeNode>(std::stoi(token));
34     deserialize_helper(iss, node->left);
35     deserialize_helper(iss, node->right);
36 }
37
38 std::unique_ptr<TreeNode> deserialize(const std::string& data) {
39     std::istream iss(data);
40     std::unique_ptr<TreeNode> root = nullptr;
41     deserialize_helper(iss, root);
42     return root;
43 }

```

Time: $O(n)$ — each node written and read once. **Space:** $O(n)$ for the output string; $O(h)$ recursion depth for both operations.

Variant: Level-order (BFS) serialization writes nodes level-by-level with null markers. This is the format used by LeetCode 297 and is often easier to visualise.

9.12.2 Exercises on LCA, Diameter, and Serialization

1. Given an unrooted tree (given as an adjacency list), root it at node 0 and answer q LCA queries using binary lifting. *Hint:* the first DFS roots the tree and fills `depth` and `up`; each query then takes $O(\log n)$.
2. Prove that the two-BFS algorithm correctly computes the diameter. *Hint:* let $a-b$ be the true diameter. Show that the node u found by the first BFS satisfies $d(u, b) \geq d(a, b)/2$, and that the second BFS from u reaches b .
3. Design an algorithm to find the **center** of a tree in $O(n)$. How many centers can a tree have? Prove your claim.
4. Modify the serialization format to use **level-order** traversal (BFS) with null markers. Write both `serialize` and `deserialize`. Trace your implementation on a sample tree.
5. Given two serialized trees, write a function that checks whether they represent the same tree without fully deserializing both. *Hint:* tokenize both strings and compare token-by-token using a counter for null markers.

9.13 Common Bugs and Pitfalls

Pitfall	Consequence
Dereferencing a null tree/root pointer	Segmentation fault on first traversal
Confusing preorder, inorder, and postorder	Wrong traversal order, incorrect algorithm results
Not deleting child subtrees before root	Memory leak — only root is freed
Stack overflow on deep trees (recursive traversal)	Crash on tall trees (e.g., $n \geq 10^5$)
Forgetting that a complete binary tree of n nodes has height $\lfloor \log_2 n \rfloor$	Off-by-one in level calculations
Threaded tree confusion — left vs right thread flags	Wrong traversal, infinite loop

9.14 Summary

1. **Trees** model hierarchical relationships. Binary trees limit each node to at most 2 children.
2. **Traversals** — preorder, inorder, postorder, level-order — each serve different purposes.
3. **Iterative traversal** replaces the call stack with an explicit stack for non-recursive processing.
4. **Union-Find** tracks disjoint sets with near-constant time operations via path compression and union by rank.
5. **Binary tree reconstruction** from two traversals is a classic divide-and-conquer technique.

9.15 Interview FAQs

1. **Why do self-balancing BSTs (AVL, red-black) guarantee $O(\log n)$ even under adversarial input, while a plain BST does not?**

A plain BST's structure depends entirely on insertion order — sorted input creates a linked list. Self-balancing trees enforce an invariant (height difference for AVL, coloring rules for red-black) that bounds the height to $O(\log n)$ regardless of input order. This is the key insight: *balance invariants decouple structure from input distribution*.

1. **What's the difference between iterative traversal with an explicit stack and Morris traversal, and when would you use each?**

The explicit stack approach uses $O(h)$ space but preserves the natural recursive logic — easy to implement, easy to debug. Morris traversal threads temporary links through null pointers, achieving $O(1)$ space by temporarily modifying the tree structure. Use Morris when memory is truly constrained (embedded systems, very deep trees). Use the stack approach everywhere else — the code is clearer, and $O(h) \approx O(\log n)$ for balanced trees. Morris is a clever trick, not a practical default.

1. **Given preorder and inorder traversals, reconstruct the binary tree.**

The first element in preorder is the root. Find this root in inorder – everything left of it is the left subtree, everything right is the right subtree. Recursively build left and right subtrees from the corresponding subarrays. Time $O(n)$ with a hash map for $O(1)$ index lookup.

```

1 node* build(const std::vector<int>& pre, int pre_lo, int pre_hi,
2             const std::vector<int>& in, int in_lo, int in_hi,
3             std::unordered_map<int, int>& idx) {
4     if (pre_lo > pre_hi) return nullptr;
5     int root_val = pre[pre_lo];
6     int mid = idx[root_val];
7     int left_sz = mid - in_lo;
8     auto* n = new node{root_val,
9                        build(pre, pre_lo + 1, pre_lo + left_sz, in, in_lo, mid - 1, idx),
10                       build(pre, pre_lo + left_sz + 1, pre_hi, in, mid + 1, in_hi, idx)
11                       };
12     return n;
13 }
14
15 node* build_tree(const std::vector<int>& pre, const std::vector<int>& in)
16 {
17     std::unordered_map<int, int> idx;
18     for (int i = 0; i < (int)in.size(); ++i) idx[in[i]] = i;
19     return build(pre, 0, (int)pre.size() - 1, in, 0, (int)in.size() - 1,
20                 idx);
21 }

```

Follow-up: “Can you do it with only preorder?” Yes, if it’s a BST (inorder is sorted).

1. How does the height of a binary tree relate to its balance, and why does this matter for BST performance?

A perfectly balanced tree of n nodes has height $\lfloor \log_2 n \rfloor$; a degenerate tree has height $n - 1$. For a BST, this means the difference between $O(\log n)$ and $O(n)$ lookup. Red-black trees guarantee height $\leq 2 \lfloor \log_2(n+1) \rfloor$; AVL trees guarantee $\leq 1.44 \lfloor \log_2(n+2) \rfloor$. The tighter AVL bound gives $\sim 20\%$ faster lookups but requires more rotations on modification.

1. How does Union-Find achieve near-constant time operations?

Two optimizations: (1) **Path compression**: during `find(x)`, make every node on the path point directly to the root. (2) **Union by rank/size**: attach the smaller tree under the larger. Together, m operations on n elements take $O(m \cdot \alpha(n))$ total, where α is the inverse Ackermann function (effectively ≤ 4 for any practical n). This is why Union-Find is used in Kruskal’s MST algorithm.

```

1 class union_find {
2     std::vector<int> parent_, rank_;
3 public:
4     union_find(int n) : parent_(n), rank_(n, 0) {
5         for (int i = 0; i < n; ++i) parent_[i] = i;
6     }

```

```

7     int find(int x) {
8         if (parent_[x] != x) parent_[x] = find(parent_[x]);
9         return parent_[x];
10    }
11    bool unite(int x, int y) {
12        int rx = find(x), ry = find(y);
13        if (rx == ry) return false;
14        if (rank_[rx] < rank_[ry]) std::swap(rx, ry);
15        parent_[ry] = rx;
16        if (rank_[rx] == rank_[ry]) ++rank_[rx];
17        return true;
18    }
19 };

```

1. Why does the heap property use a complete binary tree (array representation) instead of a balanced BST?

Array representation is cache-contiguous (no pointer chasing), uses no per-node overhead, and parent/child navigation is just index arithmetic (parent = $(i - 1)/2$, children = $2i + 1$, $2i + 2$). A balanced BST would give $O(\log n)$ but with pointer overhead and poor cache locality. The tradeoff: heaps support $O(\log n)$ insert/extract-min but not $O(\log n)$ search — they sacrifice ordering for cache efficiency.

1. How do you find the lowest common ancestor (LCA) of two nodes in a binary tree?

Recursive approach: if the current node is null or equal to either target, return it. Recurse on left and right. If both return non-null, the current node is the LCA. If only one returns non-null, that's the LCA. Time $O(n)$.

```

1 node* lca(node* root, node* p, node* q) {
2     if (!root || root == p || root == q) return root;
3     node* L = lca(root->left, p, q);
4     node* R = lca(root->right, p, q);
5     if (L && R) return root;
6     return L ? L : R;
7 }

```

For BST: LCA is the node where p and q diverge — walk from root, go left if both $<$ root, right if both $>$ root.

1. What is the time complexity of checking if a binary tree is balanced?

Naive: compute height at each node — $O(n^2)$. Optimal: compute height bottom-up, returning -1 as soon as an imbalance is found — $O(n)$ time, $O(h)$ space. This is the “isBalanced” problem (LeetCode 110). The key insight is that you only need one pass if you combine the height check with the balance check.

```

1 int check_height(node* n) {
2     if (!n) return 0;
3     int lh = check_height(n->left);
4     if (lh == -1) return -1;
5     int rh = check_height(n->right);
6     if (rh == -1) return -1;
7     if (std::abs(lh - rh) > 1) return -1;
8     return std::max(lh, rh) + 1;
9 }
10 bool is_balanced(node* root) { return check_height(root) != -1; }

```

1. How do you serialize and deserialize a binary tree?

Preorder traversal with null markers: write the node value, then recurse left and right. For null nodes, write a sentinel (e.g., ``#``). Deserialization reads tokens in the same order, reconstructing the tree recursively. Time $O(n)$, space $O(n)$.

```

1 void serialize(node* n, std::ostringstream& os) {
2     if (!n) { os << "# "; return; }
3     os << n->val << " ";
4     serialize(n->left, os);
5     serialize(n->right, os);
6 }
7
8 node* deserialize(std::istringstream& is) {
9     std::string token;
10    is >> token;
11    if (token == "#") return nullptr;
12    auto* n = new node{std::stoi(token), nullptr, nullptr};
13    n->left = deserialize(is);
14    n->right = deserialize(is);
15    return n;
16 }

```

Variant: Level-order serialization (BFS) – used by LeetCode 297.

1. What is a Morris traversal and why is it $O(1)$ space?

Morris traversal threads null right pointers to create temporary links to inorder successors, allowing traversal without a stack. For each node: if it has no left child, visit it and go right. Otherwise, find the inorder predecessor (rightmost node in left subtree). If predecessor's right is null, create a thread (set to current) and go left. If predecessor's right points to current, revert the thread, visit current, and go right. Time $O(n)$, space $O(1)$. Used in binary search tree validation and iterator implementations.

```

1 void morris_inorder(node* root) {
2     node* curr = root;
3     while (curr) {

```

```

4         if (!curr->left) {
5             visit(curr);
6             curr = curr->right;
7         } else {
8             node* pred = curr->left;
9             while (pred->right && pred->right != curr)
10                 pred = pred->right;
11             if (!pred->right) {
12                 pred->right = curr;
13                 curr = curr->left;
14             } else {
15                 pred->right = nullptr;
16                 visit(curr);
17                 curr = curr->right;
18             }
19         }
20     }
21 }

```

9.16 Trees-to-Lists and Lists-to-Trees

Converting between tree and list representations is a fundamental operation in many algorithms. The key question is whether the conversion preserves order (in-order for BSTs, level-order for general trees) and whether it can be done in-place.

9.16.1 Flatten a Binary Tree to a Linked List

Given a binary tree, flatten it to a right-skewed linked list in-place using pre-order traversal:

```

1 void flatten(TreeNode* root) {
2     TreeNode* curr = root;
3     while (curr) {
4         if (curr->left) {
5             // Find rightmost node in left subtree
6             TreeNode* rightmost = curr->left;
7             while (rightmost->right)
8                 rightmost = rightmost->right;
9             // Attach current's right to left subtree
10            rightmost->right = curr->right;
11            curr->right = curr->left;
12            curr->left = nullptr;
13        }
14        curr = curr->right;
15    }
16 }

```

This runs in $O(n)$ time and $O(1)$ extra space by reusing the existing tree pointers.

9.16.2 BST to Sorted Doubly Linked List

Convert a BST to a sorted doubly linked list in-place using in-order traversal. The left pointer becomes “previous” and the right pointer becomes “next”:

```
1  TreeNode* first = nullptr, *last = nullptr;
2
3  void inorder(TreeNode* node) {
4      if (!node) return;
5      inorder(node->left);
6      if (last) {
7          last->right = node;
8          node->left = last;
9      } else {
10         first = node;
11     }
12     last = node;
13     inorder(node->right);
14 }
15
16 TreeNode* treeToDoublyList(TreeNode* root) {
17     first = nullptr; last = nullptr;
18     inorder(root);
19     if (first && last) {
20         first->left = last;
21         last->right = first;
22     }
23     return first;
24 }
```

9.16.3 Sorted Array to Balanced BST

Convert a sorted array to a height-balanced BST by recursively picking the middle element as root:

```
1  TreeNode* sortedArrayToBST(
2      std::vector<int>& nums, int lo, int hi) {
3      if (lo > hi) return nullptr;
4      int mid = lo + (hi - lo) / 2;
5      auto* node = new TreeNode(nums[mid]);
6      node->left = sortedArrayToBST(nums, lo, mid-1);
7      node->right = sortedArrayToBST(nums, mid+1, hi);
8      return node;
9  }
```

9.17 Leaf-to-Root Tree Operations

While most tree algorithms traverse from root to leaves, some operations propagate from leaves toward the root. This “bottom-up” pattern is essential in several important algorithms.

9.17.1 Tree Height via Post-Order

The simplest leaf-to-root operation computes tree height. The height of a node is one plus the maximum height of its children. This is naturally post-order (children before parent):

```
1 int height(TreeNode* node) {
2     if (!node) return -1;
3     return 1 + std::max(height(node->left),
4                          height(node->right));
5 }
```

9.17.2 Propagate Values Upward

In many problems, each node carries information that must be propagated upward:

- **Subtree sizes:** size of each subtree for order-statistic queries
- **Maximum path sum:** best path through each subtree
- **Diameter:** longest path through each subtree
- **Balancing factors:** height difference for AVL/RB trees

```
1 // Compute diameter using leaf-to-root propagation
2 struct Info {
3     int height;
4     int diameter;
5 };
6
7 Info solve(TreeNode* node) {
8     if (!node) return {-1, 0};
9     auto left = solve(node->left);
10    auto right = solve(node->right);
11    int h = 1 + std::max(left.height, right.height);
12    int d = std::max({left.diameter, right.diameter,
13                     left.height + right.height + 2});
14    return {h, d};
15 }
```

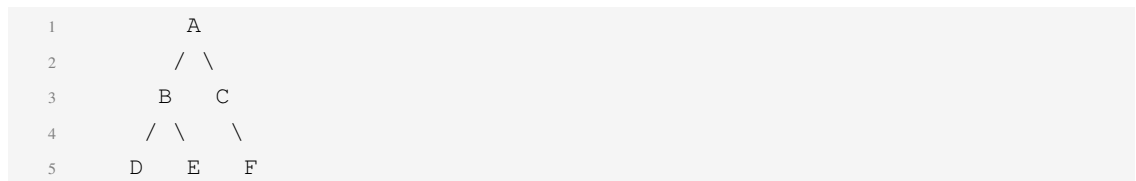
9.17.3 Connection to Skew Heaps and Splay Trees

The leaf-to-root pattern appears in self-adjusting data structures. In skew heaps, the merge operation propagates from the newly attached leaf upward, swapping children at each step. In splay trees, the splay operation brings a accessed node to the root through a series of rotations, which can be viewed as a leaf-to-root restructuring.

9.18 Exercises

9.18.1 Drill

1. For the binary tree:



List the nodes in: (a) preorder, (b) inorder, (c) postorder, (d) level-order.

1. How many different binary trees can store the keys 1, 2, 3? Draw them all.
1. What is the maximum height of a binary tree with n nodes? The minimum height?
1. For a complete binary tree of height h (root at height 0), what is the maximum number of nodes? The minimum? The number of leaves?
1. What is the result of the following operations on the union-find data structure with elements 0, 1, 2, 3, 4, 5:

`union(0, 2), union(1, 3), union(2, 4), find(0), union(1, 4), find(3), union(5, 0)` Show the parent array after each step (with path compression).

9.18.2 Application

1. Write a function `is_symmetric` that checks whether a binary tree is a mirror of itself. Test on symmetric and non-symmetric trees.
1. Implement `lowest_common_ancestor(root, p, q)` for a binary tree. What is its time complexity?
1. Write a non-recursive (iterative) `postorder` traversal using a single stack. Trace it on the tree above.

1. Given a binary tree, compute its **diameter** (longest path between any two nodes). What is the optimal algorithm?
1. Implement Huffman coding: build a Huffman tree from character frequencies, generate coding tables, encode and decode a string. Test on "this is an example for huffman encoding".
1. Implement the **Morris traversal** — an in-order traversal that uses threading (modifying right pointers temporarily) to achieve $O(1)$ extra space. Explain why the $O(1)$ space claim holds.
1. Write a function that serializes a binary tree to a string and deserializes it back. Use a format that handles arbitrary key values (e.g., use a sentinel marker for null).
1. Implement `floor_value(root, key)` — the largest value in a binary search tree that is $\leq$ key. Implement `ceil_value(root, key)` analogously.
1. Implement **tree isomorphism**: given two binary trees, determine if they are isomorphic (same structure, possibly different values).
1. Use union-find to solve the **dynamic connectivity** problem: given n vertices and a sequence of `union` and `connected` operations, answer each connectivity query.

9.18.3 Research

1. **(BST reconstruction)** In a binary search tree, reconstructing from preorder alone is possible. Explain why this is true and implement the reconstruction. Prove its correctness.
1. **(Tarjan's union-find analysis)** Read Robert Tarjan's analysis showing that union-find with union by rank and path compression has inverse Ackermann complexity. Why is this effectively constant for all practical inputs?
1. **(Persistent trees)** Research persistent binary trees — immutable structures where every update produces a new tree while preserving the old one. Implement a persistent BST. How does the space complexity compare with the mutable version?
1. **(Heaps vs. trees)** Prove that the number of different binary trees on n nodes is the Catalan number $C_n = (2n)! / ((n+1)! n!)$. Verify this for $n = 1, 2, 3, 4$.

9.19 References and Further Reading

- Donald E. Knuth, *The Art of Computer Programming*, Volume 1: Fundamental Algorithms. Section 2.3.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapter 12 (trees), Chapter 21 (union-find).

- Robert E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm" — JACM, 1975.
- Robert E. Tarjan and Jan van Leeuwen, "Worst-Case Analysis of Set Union Algorithms" — JACM, 1984.
- Bernard Chazelle, "A Minimum Spanning Tree Algorithm with Inverse-Ackermann Type Complexity" — JACM, 2000.
- David A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes" — Proceedings of the IRE, 1952.

Priority Queues

10.1 Introduction

↳ Complete compilable implementations of the data structures in this chapter are in `code/`.

10.2 The Priority Queue ADT

A **priority queue** is a collection where each element has a **priority**, and the element with the highest (or lowest) priority is always removed first.

10.2.1 Operations

- **push(value)** — insert with given priority
- **pop()** — remove the highest-priority element
- **top()** — return the highest-priority element without removing
- **empty(), size()**

10.2.2 Applications

- **Scheduling** — CPU task scheduling, print queues by priority
- **Event-driven simulation** — process events in chronological order
- **Dijkstra's shortest path** — extract the vertex with smallest distance
- **Huffman coding** — repeatedly combine the two smallest frequencies
- **Median finding** — maintain two heaps
- **A* search** — open set ordered by f-cost

10.3 Heaps

A **max-heap** is a complete binary tree where every node's value is $\geq$ the values of its children. A **min-heap** is the symmetric case ($\leq$).

```

1 Max-Heap:
2           [90]
3         /   \
4       [80]   [70]
5     /  \   /
6   [30] [20] [50]
7
8 Array representation: [90, 80, 70, 30, 20, 50]

```

10.3.1 Array Storage

Since a heap is a **complete binary tree**, we store it in an array:

```

1 Position i (0-indexed):
2   left child: 2i + 1
3   right child: 2i + 2
4   parent:      (i - 1) / 2

```

10.3.2 Max-Heap Implementation

```

1 template <typename T, typename Compare = std::less<T>>
2 class max_heap {
3 public:
4     max_heap() = default;
5
6     void push(const T& value) {
7         data_.push_back(value);
8         sift_up(data_.size() - 1);
9     }
10
11     void push(T&& value) {
12         data_.push_back(std::move(value));
13         sift_up(data_.size() - 1);
14     }
15
16     void pop() {
17         if (empty()) throw std::underflow_error("heap is empty");
18         data_[0] = std::move(data_.back());
19         data_.pop_back();
20         if (!empty()) sift_down(0);
21     }
22
23     const T& top() const {
24         if (empty()) throw std::underflow_error("heap is empty");
25         return data_[0];
26     }
27

```

```

28     bool empty() const noexcept { return data_.empty(); }
29     size_t size() const noexcept { return data_.size(); }
30
31 private:
32     void sift_up(size_t i) {
33         while (i > 0) {
34             size_t parent = (i - 1) / 2;
35             if (!compare_(data_[parent], data_[i])) break;
36             std::swap(data_[i], data_[parent]);
37             i = parent;
38         }
39     }
40
41     void sift_down(size_t i) {
42         size_t n = data_.size();
43         while (true) {
44             size_t largest = i;
45             size_t left = 2 * i + 1;
46             size_t right = 2 * i + 2;
47
48             if (left < n && compare_(data_[largest], data_[left]))
49                 largest = left;
50             if (right < n && compare_(data_[largest], data_[right]))
51                 largest = right;
52             if (largest == i) break;
53             std::swap(data_[i], data_[largest]);
54             i = largest;
55         }
56     }
57
58     std::vector<T> data_;
59     Compare compare_; // less<T> -> max-heap
60 };

```

10.3.3 Push Walkthrough

Insert 85 into heap [90, 80, 70, 30, 20, 50]:

```

1 Step 1: Append 85 -> [90, 80, 70, 30, 20, 50, 85]
2 Step 2: Compare 85 with parent 70 -> 85 > 70 -> swap
3         [90, 80, 85, 30, 20, 50, 70]
4 Step 3: Compare 85 with parent 90 -> 85 <= 90 -> stop

```

10.3.4 Pop Walkthrough

Remove top (90) from [90, 80, 70, 30, 20, 50]:

```

1 Step 1: Move last to root -> [50, 80, 70, 30, 20]

```



```

2 Step 2: Sift down 50:
3   Compare with children 80 and 70 -> 80 is largest -> swap
4   [80, 50, 70, 30, 20]
5 Step 3: Compare 50 with children 30 and 20 -> 50 is largest -> stop

```

10.4 Heap Initialization (Heapify)

Building a heap from an unsorted array: $O(n)$.

```

1 template <typename T, typename Compare = std::less<T>>
2 void heapify(std::vector<T>& data, Compare compare = {}) {
3     int n = data.size();
4     for (int i = n / 2 - 1; i >= 0; --i) {
5         sift_down(data, i, n, compare);
6     }
7 }

```

Why $O(n)$? Most nodes are near the bottom and sift down takes few steps. The total work is Σ (height of node) over all nodes = $O(n)$.

10.5 Heap Sort

```

1 template <typename T>
2 void heap_sort(std::span<T> data) {
3     // Build max-heap
4     int n = data.size();
5     for (int i = n / 2 - 1; i >= 0; --i) {
6         // sift_down
7         int current = i;
8         while (true) {
9             int largest = current;
10            int left = 2 * current + 1;
11            int right = 2 * current + 2;
12            if (left < n && data[left] > data[largest]) largest = left;
13            if (right < n && data[right] > data[largest]) largest = right;
14            if (largest == current) break;
15            std::swap(data[current], data[largest]);
16            current = largest;
17        }
18    }
19
20    // Extract elements in sorted order
21    for (int i = n - 1; i > 0; --i) {
22        std::swap(data[0], data[i]);
23        // sift_down on reduced heap
24        int current = 0;

```

```

25     int heap_size = i;
26     while (true) {
27         int largest = current;
28         int left = 2 * current + 1;
29         int right = 2 * current + 2;
30         if (left < heap_size && data[left] > data[largest]) largest =
    left;
31         if (right < heap_size && data[right] > data[largest]) largest
    = right;
32         if (largest == current) break;
33         std::swap(data[current], data[largest]);
34         current = largest;
35     }
36 }
37 }

```

Complexity: $O(n \log n)$ worst case. **In-place** — $O(1)$ extra space.

10.6 Min-Heaps

A min-heap is the same structure with the order reversed. Use `std::greater<T>` as the comparator to make our `max_heap` behave as a min-heap.

C++ `std::priority_queue` provides `std::less<T>` for max-heap and `std::greater<T>` for min-heap:

```

1 std::priority_queue<int> max_pq;           // max-heap
2 std::priority_queue<int, std::vector<int>,
3                 std::greater<int>> min_pq; // min-heap

```

10.7 Leftist Trees (Mergeable Heaps)

A **leftist tree** is a binary tree that supports $O(\log n)$ merge. This is useful when we need to merge two priority queues frequently.

Property: The **null path length** (`npl`) of a node is the length of the shortest path to a null child. A leftist tree maintains the invariant: $\text{npl}(\text{left}) \geq \text{npl}(\text{right})$ for every node.

```

1 template <typename T>
2 struct leftist_node {
3     T data;
4     std::unique_ptr<leftist_node> left;
5     std::unique_ptr<leftist_node> right;
6     int npl = 0; // null path length
7
8     explicit leftist_node(const T& value) : data(value) {}
9 };
10

```

```

11 template <typename T>
12 std::unique_ptr<leftist_node<T>> merge(
13     std::unique_ptr<leftist_node<T>> a,
14     std::unique_ptr<leftist_node<T>> b) {
15     if (!a) return b;
16     if (!b) return a;
17
18     // Ensure a has the smaller root
19     if (a->data > b->data) std::swap(a, b);
20
21     // Recursively merge with right child
22     a->right = merge(std::move(a->right), std::move(b));
23
24     // Maintain leftist property
25     if (!a->left || (a->left->npl < a->right->npl)) {
26         std::swap(a->left, a->right);
27     }
28
29     // Update null path length
30     a->npl = (a->right ? a->right->npl : 0) + 1;
31
32     return a;
33 }

```

10.8 Fibonacci Heap

A **Fibonacci heap** is a collection of heap-ordered trees that achieves $O(1)$ amortized time for insert and decrease-key, and $O(\log n)$ amortized for extract-min. This makes it the optimal priority queue for Dijkstra's shortest path algorithm.

10.8.1 Structure

A Fibonacci heap is a lazy data structure: insertions and decrease-key operations simply add nodes without restructuring. The trees are linked together in a circular root list.

```

1 Fibonacci heap after inserting 3, 1, 7, 5, 2, 8:
2 Root list:  [1] -- [3] -- [7] -- [5] -- [2] -- [8]
3 (min pointer -> 1)
4
5 After extract-min (remove 1, merge children into root list, consolidate):
6 Consolidate: link trees of same degree until all roots have distinct
   degrees.
7 Result: one tree with root 2
8   [2]
9   /\
10  [3][5][8]
11  /\

```

```
12 [7] (nil)
```

10.8.2 Core Operations

```

1  template <typename T>
2  class fibonacci_heap {
3      struct node {
4          T key;
5          node *left, *right, *parent, *child;
6          int degree = 0;
7          bool marked = false; // for cascading cut
8      };
9
10     node* min_ = nullptr;
11     int size_ = 0;
12
13 public:
14     // Insert: O(1) amortized
15     void insert(const T& key) {
16         auto* n = new node{key, nullptr, nullptr, nullptr, nullptr, 0,
17         false};
18         if (!min_) {
19             min_ = n;
20             n->left = n->right = n; // circular list
21         } else {
22             // Add to root list
23             n->left = min_;
24             n->right = min_->right;
25             min_->right->left = n;
26             min_->right = n;
27             if (key < min_->key) min_ = n;
28         }
29         ++size_;
30
31     // Decrease-key: O(1) amortized
32     void decrease_key(node* x, const T& new_key) {
33         x->key = new_key;
34         node* y = x->parent;
35         if (y && x->key < y->key) {
36             cut(x, y); // cut x from parent, add to root list
37             cascading_cut(y); // possibly cut grandparent, etc.
38         }
39         if (x->key < min_->key) min_ = x;
40     }
41
42     // Extract-min: O(log n) amortized

```

```

43     T extract_min() {
44         T result = min_>key;
45         // Add all children of min to root list
46         if (min_>child) {
47             node* child = min_>child;
48             do {
49                 node* next = child->right;
50                 child->left = min_;
51                 child->right = min_>right;
52                 min_>right->left = child;
53                 min_>right = child;
54                 child->parent = nullptr;
55                 child = next;
56             } while (child != min_>child);
57         }
58         // Remove min from root list
59         min_>left->right = min_>right;
60         min_>right->left = min_>left;
61         if (min_ == min_>right) {
62             delete min_;
63             min_ = nullptr;
64         } else {
65             node* old_min = min_;
66             min_ = min_>right; // temporary
67             consolidate();
68             delete old_min;
69         }
70         --size_;
71         return result;
72     }
73
74 private:
75     void cut(node* x, node* y) {
76         // Remove x from child list of y
77         x->left->right = x->right;
78         x->right->left = x->left;
79         if (y->degree == 1) y->child = nullptr;
80         --y->degree;
81         // Add x to root list
82         x->left = min_;
83         x->right = min_>right;
84         min_>right->left = x;
85         min_>right = x;
86         x->parent = nullptr;
87         x->marked = false;
88     }
89
90     void cascading_cut(node* y) {

```

```

91     node* z = y->parent;
92     if (z) {
93         if (!y->marked) y->marked = true;
94         else { cut(y, z); cascading_cut(z); }
95     }
96 }
97
98 void consolidate() {
99     int max_degree = 4 * static_cast<int>(std::log2(size_));
100    std::vector<node*> A(max_degree + 1, nullptr);
101
102    // Collect all roots
103    std::vector<node*> roots;
104    node* current = min_;
105    do {
106        roots.push_back(current);
107        current = current->right;
108    } while (current != min_);
109
110    for (node* w : roots) {
111        node* x = w;
112        int d = x->degree;
113        while (d < max_degree && A[d]) {
114            node* y = A[d];
115            if (x->key > y->key) std::swap(x, y);
116            link(y, x); // make y a child of x
117            A[d] = nullptr;
118            ++d;
119        }
120        A[d] = x;
121    }
122
123    // Rebuild root list
124    min_ = nullptr;
125    for (node* n : A) {
126        if (n) {
127            if (!min_) {
128                min_ = n;
129                n->left = n->right = n;
130            } else {
131                n->left = min_;
132                n->right = min_->right;
133                min_->right->left = n;
134                min_->right = n;
135                if (n->key < min_->key) min_ = n;
136            }
137        }
138    }

```

```

139     }
140
141     void link(node* y, node* x) {
142         y->left->right = y->right;
143         y->right->left = y->left;
144         y->parent = x;
145         if (x->child) {
146             y->left = x->child;
147             y->right = x->child->right;
148             x->child->right->left = y;
149             x->child->right = y;
150         } else {
151             x->child = y;
152             y->left = y->right = y;
153         }
154         ++x->degree;
155         y->marked = false;
156     }
157 };

```

10.8.3 Amortized Analysis

The potential function $\Phi = t + 2m$ where t = number of trees in root list, m = number of marked nodes.

- **Insert:** $O(1)$ real cost + $O(1)$ potential change = $O(1)$ amortized
- **Decrease-key:** $O(1)$ real cost + $O(1)$ potential change = $O(1)$ amortized
- **Extract-min:** $O(\log n)$ real cost + $O(\log n)$ potential change = $O(\log n)$ amortized

10.8.4 Dijkstra's with Fibonacci Heap

With a Fibonacci heap, Dijkstra's runs in $O(V \log V + E)$ — optimal for dense graphs:

```

1 void dijkstra_fib(const graph& g, int source, std::vector<int>& dist) {
2     int n = g.num_vertices();
3     dist.assign(n, INT_MAX);
4     fibonacci_heap<int> pq;
5     std::vector<fibonacci_heap<int>::node*> handles(n);
6
7     dist[source] = 0;
8     for (int i = 0; i < n; ++i) {
9         handles[i] = pq.insert(i); // insert vertex indices
10    }
11
12    while (!pq.empty()) {
13        int u = pq.extract_min();
14        for (auto [v, weight] : g.neighbors(u)) {

```

```

15         if (dist[u] + weight < dist[v]) {
16             dist[v] = dist[u] + weight;
17             pq.decrease_key(handles[v], dist[v]);
18         }
19     }
20 }
21 }

```

The key advantage: decrease-key is $O(1)$ amortized instead of $O(\log n)$, giving $O(V \log V + E)$ total instead of $O((V + E) \log V)$.

10.9 Pairing Heap

A **pairing heap** is a simpler alternative to Fibonacci heaps with similar practical performance. It supports merge in $O(1)$ and uses less memory.

10.9.1 Structure

A pairing heap is a heap-ordered tree (typically a single tree, not a forest). Each node has at most one child pointer, and siblings are linked in a circular doubly-linked list.

```

1 Pairing heap:
2     [1] (root, minimum)
3     / | \
4     [3] [5] [2] (children of root, in circular list)
5     / \
6     [7] [8]
7
8 Merge: just link roots ( $O(1)$ )
9 Decrease-key: cut and merge ( $O(\log n)$  amortized)

```

10.9.2 Implementation

```

1 template <typename T>
2 class pairing_heap {
3     struct node {
4         T key;
5         node *left = nullptr, *next = nullptr, *prev = nullptr;
6     };
7
8     node* root_ = nullptr;
9     int size_ = 0;
10
11 public:
12     // Merge two pairing heaps:  $O(1)$ 
13     static pairing_heap merge(pairing_heap&& a, pairing_heap&& b) {

```



```

14     if (!a.root_) return std::move(b);
15     if (!b.root_) return std::move(a);
16     link_roots(a.root_, b.root_);
17     if (b.root_>key < a.root_>key)
18         a.root_ = b.root_;
19     a.size_ += b.size_;
20     b.root_ = nullptr;
21     return std::move(a);
22 }
23
24 // Insert: O(1) amortized
25 node* insert(const T& key) {
26     auto* n = new node{key};
27     if (!root_) {
28         root_ = n;
29     } else {
30         link_roots(n, root_);
31         if (key < root_>key) root_ = n;
32     }
33     ++size_;
34     return n;
35 }
36
37 // Decrease-key: O(log n) amortized
38 void decrease_key(node* x, const T& new_key) {
39     x->key = new_key;
40     if (x == root_) return;
41     // Cut x from its parent
42     if (x->prev) x->prev->next = x->next;
43     if (x->next) x->next->prev = x->prev;
44     if (x == x->prev->next) // was first child
45         x->prev->next = x->next;
46     x->next = x->prev = nullptr;
47     // Merge cut subtree with root
48     link_roots(x, root_);
49     if (x->key < root_>key) root_ = x;
50 }
51
52 // Extract-min: O(log n) amortized
53 T extract_min() {
54     T result = root_>key;
55     // Collect children
56     node* first_child = root_>left;
57     // Two-pass pairing
58     std::vector<node*> trees;
59     node* curr = first_child;
60     while (curr) {
61         node* next = curr->next;

```

```

62         curr->next = curr->prev = nullptr;
63         trees.push_back(curr);
64         curr = next;
65     }
66     // Pair from right to left
67     root_ = nullptr;
68     for (int i = static_cast<int>(trees.size()) - 1; i >= 0; --i) {
69         if (!root_) {
70             root_ = trees[i];
71         } else {
72             link_roots(trees[i], root_);
73             if (trees[i]->key < root_->key) root_ = trees[i];
74         }
75     }
76     --size_;
77     return result;
78 }
79
80 bool empty() const { return size_ == 0; }
81
82 private:
83     static void link_roots(node* a, node* b) {
84         // Make a a child of b (or vice versa)
85         a->next = b->left;
86         if (b->left) b->left->prev = a;
87         b->left = a;
88         a->prev = b;
89     }
90 };

```

10.10 Binomial Heap

A **binomial heap** is a collection of heap-ordered **binomial trees** that supports merge (meld) in $O(\log n)$ time. This makes it a **mergeable heap**, unlike binary heaps which require $O(n)$ to merge. Binomial heaps are used in network flow algorithms and parallel computing where efficient merging is essential.

10.10.1 Binomial Trees

A binomial tree B_k is defined recursively:

- B_0 is a single node.
- B_k is formed by linking two B_{k-1} trees: one becomes the leftmost child of the other.

B_k has 2^k nodes and height k . The root of B_k has k children, which are the roots of $B_{k-1}, B_{k-2}, \dots, B_0$ (from left to right).

```

1 Binomial trees:
2
3 B0:   (1)           -- 1 node
4
5 B1:   (2)           -- 2 nodes
6       |
7       (1)
8
9 B2:   (3)           -- 4 nodes
10      |
11      (2) -- (1)
12      |
13      (1)
14
15 B3:   (4)           -- 8 nodes
16      |
17      (3) -- (2) -- (1)
18      |   |   |
19      (1) (1) (0)
20      |
21      (0)

```

10.10.2 Heap Structure

A binomial heap of n nodes contains the binomial trees B_k for each set bit k in the binary representation of n . For example, $n = 13 = 1101_2 = B_3 + B_2 + B_0$.

```

1 Binomial heap with 13 nodes: 13 = 8 + 4 + 1 = B3 + B2 + B0
2
3 B3 (8 nodes):      B2 (4 nodes):      B0 (1 node):
4   [4]              [3]                [5]
5   /|\              / \
6  [3][2][1]        [2] [1]
7  /| | |           |
8 [1][1][0][0]      [1]
9
10 All roots form the root list, linked left-to-right.
11 Minimum element is the smallest root: [3] from B2.

```

Each node stores: its key, its degree (number of children), and pointers to its parent, leftmost child, and immediate right sibling.

10.10.3 Implementation

```

1 template <typename T, typename Compare = std::less<T>>
2 class binomial_heap {
3     struct node {

```

```

4      T key;
5      int degree = 0;
6      node *parent = nullptr, *child = nullptr, *sibling = nullptr;
7  };
8
9      node* root_ = nullptr;
10     int size_ = 0;
11     Compare comp_;
12
13     // Link tree y under tree x (same degree): O(1)
14     static void merge_trees(node* y, node* x) {
15         y->parent = x;
16         y->sibling = x->child;
17         x->child = y;
18         ++x->degree;
19     }
20
21 public:
22     binomial_heap() = default;
23
24     // Merge two binomial heaps: O(log n)
25     static binomial_heap merge(binomial_heap&& a, binomial_heap&& b) {
26         binomial_heap result;
27         result.root_ = merge_root_lists(a.root_, b.root_);
28         if (!result.root_) return result;
29         // Link roots of equal degree
30         node *prev = nullptr, *x = result.root_, *next = x->sibling;
31         while (next) {
32             if (x->degree != next->degree ||
33                 (next->sibling && next->sibling->degree == x->degree)) {
34                 prev = x;
35                 x = next;
36             } else {
37                 if (comp_(next->key, x->key)) {
38                     x->sibling = next->sibling;
39                     merge_trees(x, next);
40                     x = next;
41                 } else {
42                     next->sibling = x->sibling;
43                     merge_trees(next, x);
44                 }
45             }
46             next = x->sibling;
47         }
48         result.size_ = a.size_ + b.size_;
49         a.root_ = nullptr;
50         b.root_ = nullptr;
51         return result;

```

```

52     }
53
54     // Insert: O(log n)
55     void insert(const T& key) {
56         binomial_heap temp;
57         temp.root_ = new node{key};
58         temp.size_ = 1;
59         *this = merge(std::move(*this), std::move(temp));
60     }
61
62     // Extract-min: O(log n)
63     T extract_min() {
64         if (!root_) throw std::underflow_error("heap is empty");
65         // Find the root with minimum key
66         node *best_prev = nullptr, *best = root_;
67         node *prev = nullptr, *curr = root_;
68         while (curr) {
69             if (comp_(curr->key, best->key)) {
70                 best = curr;
71                 best_prev = prev;
72             }
73             prev = curr;
74             curr = curr->sibling;
75         }
76         // Remove best from root list
77         if (best_prev)
78             best_prev->sibling = best->sibling;
79         else
80             root_ = best->sibling;
81         // Reverse the child list to form a new heap
82         binomial_heap child_heap;
83         node* rev = nullptr;
84         node* c = best->child;
85         while (c) {
86             node* next = c->sibling;
87             c->sibling = rev;
88             c->parent = nullptr;
89             rev = c;
90             c = next;
91         }
92         child_heap.root_ = rev;
93         child_heap.size_ = (1 << best->degree) - 1;
94         T result = best->key;
95         delete best;
96         --size_;
97         *this = merge(std::move(*this), std::move(child_heap));
98         return result;
99     }

```

```

100
101 // Decrease-key:  $O(\log n)$ 
102 void decrease_key(node* x, const T& new_key) {
103     x->key = new_key;
104     node* y = x;
105     while (y->parent && comp_(y->key, y->parent->key)) {
106         std::swap(y->key, y->parent->key);
107         y = y->parent;
108     }
109 }
110
111 bool empty() const noexcept { return size_ == 0; }
112 int size() const noexcept { return size_; }
113
114 private:
115 static node* merge_root_lists(node* a, node* b) {
116     if (!a) return b;
117     if (!b) return a;
118     node* result = nullptr;
119     node** tail = &result;
120     while (a && b) {
121         if (a->degree <= b->degree) {
122             *tail = a;
123             a = a->sibling;
124         } else {
125             *tail = b;
126             b = b->sibling;
127         }
128         tail = &((*tail)->sibling);
129     }
130     *tail = a ? a : b;
131     return result;
132 }
133 };

```

10.10.4 Complexity

Operation	Worst Case
Insert	$O(\log n)$
Extract-min	$O(\log n)$
Merge	$O(\log n)$
Decrease-key	$O(\log n)$
Delete	$O(\log n)$
Find-min	$O(\log n)$

10.10.5 Binary Heap vs. Binomial Heap

Binary heaps are simpler and have better cache locality (array-based), making them faster in practice for single-threaded workloads. Binomial heaps shine when **merge** is a frequent operation, since merging two binary heaps requires $O(n)$ while merging two binomial heaps costs only $O(\log n)$. In practice, pairing heaps or Fibonacci heaps often outperform both due to better amortized bounds for decrease-key.

10.11 D-ary Heap

A **d -ary heap** generalizes the binary heap: each node has up to d children instead of 2. The tree is shallower ($\log_d n$ vs $\log_2 n$), which means decrease-key requires fewer levels of upward traversal. With $d = 4$, a 4-ary heap is commonly used in Dijkstra's algorithm and other applications with frequent decrease-key calls.

10.11.1 Structure

A 4-ary heap is stored in an array just like a binary heap, but the child/parent index formulas change:

```

1 4-ary heap (d = 4):
2
3          [1]
4        / / | \
5       [3] [5] [7] [9]
6      /|\  |
7     [12][8][15][2] [6]
8
9 Array: [1, 3, 5, 7, 9, 12, 8, 15, 2, 6]
10
11 For d-ary heap at index i (0-indexed):
12   Parent:      (i - 1) / d
13   k-th child: d * i + k + 1   (for k = 0, 1, ..., d-1)
```

The height is $\lceil \log_d n \rceil$, so the tree is shallower than a binary heap. Each level has d times more nodes, so sift-down does more work per level, but sift-up (used by decrease-key) does less.

10.11.2 Implementation

```

1 template <typename T, int D = 4, typename Compare = std::less<T>>
2 class d_ary_heap {
3     std::vector<T> data_;
4     Compare comp_;
5
6     int parent(int i) const { return (i - 1) / D; }
7     int child(int i, int k) const { return D * i + k + 1; }
8 }
```

```

9     void sift_up(int i) {
10         while (i > 0) {
11             int p = parent(i);
12             if (!comp_(data_[p], data_[i])) break;
13             std::swap(data_[i], data_[p]);
14             i = p;
15         }
16     }
17
18     void sift_down(int i) {
19         int n = static_cast<int>(data_.size());
20         while (true) {
21             int smallest = i;
22             for (int k = 0; k < D; ++k) {
23                 int c = child(i, k);
24                 if (c < n && comp_(data_[smallest], data_[c]))
25                     smallest = c;
26             }
27             if (smallest == i) break;
28             std::swap(data_[i], data_[smallest]);
29             i = smallest;
30         }
31     }
32
33 public:
34     d_ary_heap() = default;
35
36     void push(const T& value) {
37         data_.push_back(value);
38         sift_up(static_cast<int>(data_.size()) - 1);
39     }
40
41     void push(T&& value) {
42         data_.push_back(std::move(value));
43         sift_up(static_cast<int>(data_.size()) - 1);
44     }
45
46     void pop() {
47         if (empty()) throw std::underflow_error("heap is empty");
48         data_[0] = std::move(data_.back());
49         data_.pop_back();
50         if (!empty()) sift_down(0);
51     }
52
53     const T& top() const {
54         if (empty()) throw std::underflow_error("heap is empty");
55         return data_[0];
56     }

```



```

57
58     // Decrease-key:  $O(\log_d n)$ 
59     void decrease_key(int i, const T& new_key) {
60         data_[i] = new_key;
61         sift_up(i);
62     }
63
64     bool empty() const noexcept { return data_.empty(); }
65     int size() const noexcept { return static_cast<int>(data_.size()); }
66 };

```

10.11.3 Complexity

Operation	Binary ($d = 2$)	4-ary ($d = 4$)	d -ary
Insert	$O(\log_2 n)$	$O(\log_4 n)$	$O(\log_d n)$
Extract-min	$O(\log_2 n)$	$O(d \log_4 n)$	$O(d \log_d n)$
Decrease-key	$O(\log_2 n)$	$O(\log_4 n)$	$O(\log_d n)$
Space	$O(n)$	$O(n)$	$O(n)$

Note that sift-down costs $O(d \log_d n)$ in a d -ary heap because each level examines d children. However, $\log_d n = \log_2 n / \log_2 d$, so the total work per level is offset by the reduced height. For $d = 4$, sift-down does $\approx 2 \log_2 n$ comparisons (same as binary), but sift-up (decrease-key) does only $\frac{1}{2} \log_2 n$ comparisons.

10.11.4 When to Use

For applications like Dijkstra’s algorithm where decrease-key is called much more often than extract-min, a 4-ary heap often outperforms a binary heap. The $O(\log_4 n)$ decrease-key is half the cost of the $O(\log_2 n)$ version, while extract-min remains comparable. For $d > 4$, the sift-down overhead grows and the benefit diminishes. In practice, $d = 4$ is the sweet spot for most priority queue workloads.

10.12 Min-Max Heap

A **min-max heap** is a complete binary tree that supports both find-min and find-max in $O(1)$, along with insert and delete in $O(\log n)$. Levels alternate between **min levels** and **max levels**: the root is at a min level, its children at a max level, their children at a min level, and so on. At a min level, the node is smaller than all descendants; at a max level, the node is larger than all descendants. This structure is useful when we need fast access to both extremes, such as in a double-ended priority queue for job scheduling or CPU load balancing.

10.12.1 Structure

```

1 Min-max heap (levels alternate min/max):
2
3 Level 0 (min):          [2]                <-- min of all
4                      /   \
5 Level 1 (max):        [9]   [8]            <-- max of descendants
6                      / \   / \
7 Level 2 (min):    [4]  [7] [5]  [3]        <-- min of descendants
8                      / \   |
9 Level 3 (max): [6] [5] [12]
10
11 Parent-child ordering:
12   Min-level node v:  v < all descendants
13   Max-level node v:  v > all descendants
14   Grandchild shortcut: min-level node also < all grandchildren
15                       max-level node also > all grandchildren

```

The grandchild optimization allows skipping the intermediate level during sift operations, cutting the number of comparisons roughly in half compared to a naive alternation of min-heap and max-heap sifts.

10.12.2 Implementation

```

1 template <typename T, typename Compare = std::less<T>>
2 class min_max_heap {
3     std::vector<T> data_;
4     Compare comp_;
5
6     bool is_min_level(int i) const {
7         int level = 0;
8         int temp = i;
9         while (temp > 0) { temp = (temp - 1) / 2; ++level; }
10        return level % 2 == 0;
11    }
12
13    // Sift up within a min level (compare with grandchildren)
14    void sift_up_min(int i) {
15        while (i > 0) {
16            int grandparent = ((i - 1) / 2 - 1) / 2;
17            if (grandparent >= 0 && comp_(data_[i], data_[grandparent])) {
18                std::swap(data_[i], data_[grandparent]);
19                i = grandparent;
20            } else {
21                break;
22            }
23        }
24        // Final check with parent (opposite level)
25        if (i > 0) {

```

```

26         int p = (i - 1) / 2;
27         if (comp_(data_[i], data_[p]))
28             std::swap(data_[i], data_[p]);
29     }
30 }
31
32 // Sift up within a max level (compare with grandchildren)
33 void sift_up_max(int i) {
34     while (i > 0) {
35         int grandparent = ((i - 1) / 2 - 1) / 2;
36         if (grandparent >= 0 && comp_(data_[grandparent], data_[i])) {
37             std::swap(data_[i], data_[grandparent]);
38             i = grandparent;
39         } else {
40             break;
41         }
42     }
43     if (i > 0) {
44         int p = (i - 1) / 2;
45         if (comp_(data_[p], data_[i]))
46             std::swap(data_[i], data_[p]);
47     }
48 }
49
50 void sift_up(int i) {
51     if (i == 0) return;
52     int p = (i - 1) / 2;
53     if (is_min_level(i)) {
54         if (comp_(data_[i], data_[p])) {
55             std::swap(data_[i], data_[p]);
56             sift_up_max(p);
57         } else {
58             sift_up_min(i);
59         }
60     } else {
61         if (comp_(data_[p], data_[i])) {
62             std::swap(data_[i], data_[p]);
63             sift_up_min(p);
64         } else {
65             sift_up_max(i);
66         }
67     }
68 }
69
70 void sift_down_min(int i) {
71     int n = static_cast<int>(data_.size());
72     while (true) {
73         int smallest = i;

```

```

74         // Check children
75         int left = 2 * i + 1, right = 2 * i + 2;
76         if (left < n && comp_(data_[left], data_[smallest]))
77             smallest = left;
78         if (right < n && comp_(data_[right], data_[smallest]))
79             smallest = right;
80         // Check grandchildren (shortcut)
81         int gc = 2 * left + 1;
82         for (int k = 0; k < 4 && gc + k < n; ++k) {
83             if (comp_(data_[gc + k], data_[smallest]))
84                 smallest = gc + k;
85         }
86         if (smallest == i) break;
87         int p = (smallest - 1) / 2;
88         std::swap(data_[i], data_[smallest]);
89         // If we swapped with a grandchild, may need to fix parent
90         if (p != i && comp_(data_[smallest], data_[p]))
91             std::swap(data_[smallest], data_[p]);
92         i = smallest;
93     }
94 }
95
96 void sift_down_max(int i) {
97     int n = static_cast<int>(data_.size());
98     while (true) {
99         int largest = i;
100        int left = 2 * i + 1, right = 2 * i + 2;
101        if (left < n && comp_(data_[largest], data_[left]))
102            largest = left;
103        if (right < n && comp_(data_[largest], data_[right]))
104            largest = right;
105        int gc = 2 * left + 1;
106        for (int k = 0; k < 4 && gc + k < n; ++k) {
107            if (comp_(data_[largest], data_[gc + k]))
108                largest = gc + k;
109        }
110        if (largest == i) break;
111        int p = (largest - 1) / 2;
112        std::swap(data_[i], data_[largest]);
113        if (p != i && comp_(data_[p], data_[largest]))
114            std::swap(data_[largest], data_[p]);
115        i = largest;
116    }
117 }
118
119 public:
120     min_max_heap() = default;
121

```

```

122     void push(const T& value) {
123         data_.push_back(value);
124         sift_up(static_cast<int>(data_.size()) - 1);
125     }
126
127     void push(T&& value) {
128         data_.push_back(std::move(value));
129         sift_up(static_cast<int>(data_.size()) - 1);
130     }
131
132     const T& find_min() const {
133         if (empty()) throw std::underflow_error("heap is empty");
134         return data_[0];
135     }
136
137     const T& find_max() const {
138         if (empty()) throw std::underflow_error("heap is empty");
139         if (data_.size() == 1) return data_[0];
140         int left = 1, right = 2;
141         int n = static_cast<int>(data_.size());
142         int best = left;
143         if (right < n && comp_(data_[left], data_[right]))
144             best = right;
145         return data_[best];
146     }
147
148     T pop_min() {
149         if (empty()) throw std::underflow_error("heap is empty");
150         T result = std::move(data_[0]);
151         data_[0] = std::move(data_.back());
152         data_.pop_back();
153         if (!empty()) sift_down_min(0);
154         return result;
155     }
156
157     T pop_max() {
158         if (empty()) throw std::underflow_error("heap is empty");
159         int left = 1, right = 2;
160         int n = static_cast<int>(data_.size());
161         int best = left;
162         if (right < n && comp_(data_[left], data_[right]))
163             best = right;
164         T result = std::move(data_[best]);
165         data_[best] = std::move(data_.back());
166         data_.pop_back();
167         if (best < static_cast<int>(data_.size()))
168             sift_down_max(best);
169         return result;

```

```
170     }
171
172     bool empty() const noexcept { return data_.empty(); }
173     int size() const noexcept { return static_cast<int>(data_.size()); }
174 };
```

10.12.3 Complexity

Operation	Worst Case
Insert	$O(\log n)$
Find-min	$O(1)$
Find-max	$O(1)$
Delete-min	$O(\log n)$
Delete-max	$O(\log n)$

The grandchild shortcut reduces the average number of comparisons during sift operations. Each sift-down visits at most $2\log_2 n$ nodes (skipping levels), giving $O(\log n)$ with a smaller constant than maintaining separate min-heaps and max-heaps. For a doubly-ended priority queue, min-max heaps are simpler and more space-efficient than a pair of binary heaps.

10.13 Weak Heap

A **weak heap** is a mergeable heap where each node has at most one child—a right-child tree. The ordering constraint is weaker than a standard heap: each node must be $\leq$ its *right child* (not necessarily all descendants). A **reverse bit** per node tracks whether its left subtree has been reversed, which allows the structure to represent any heap-ordered tree. Weak heaps are the basis for **smoothsort**, Leonardo de Vois’s efficient in-place heapsort variant.

10.13.1 Structure

```
1 Weak heap (right-child tree with reverse bits):
2
3     [2]-----r=0
4           |
5     [5]-----r=0
6           |
7     [3]-----r=1  (left subtree reversed)
8           /
9     [7]    [4]
10
11 Right-child chain: 2 -> 5 -> 3
12 Left children hang off nodes where r=1
13
```

```

14 Weak heap property:
15   For every node n: n->data <= n->right->data
16   The reverse bit determines the structure of left subtrees.

```

The key insight is that any complete binary tree can be represented as a weak heap, making it possible to perform in-place heapsort with $O(n \log n)$ worst case and excellent cache behavior. The right-child chain ensures the minimum is always at the root.

10.13.2 Implementation

```

1  template <typename T, typename Compare = std::less<T>>
2  class weak_heap {
3      struct node {
4          T data;
5          node* right = nullptr;
6          bool reverse = false;
7
8          explicit node(const T& value) : data(value) {}
9      };
10
11     node* root_ = nullptr;
12     node* last_ = nullptr;
13     int size_ = 0;
14     Compare comp_;
15
16     node* left_child(node* n) const {
17         if (!n || !n->right) return nullptr;
18         // The left child is the parent of the right child in
19         // the original complete binary tree (found via reverse bits)
20         return nullptr; // simplified; real impl uses parent pointers
21     }
22
23     // Merge two weak heaps: O(1)
24     static node* merge(node* a, node* b, const Compare& comp) {
25         if (!a) return b;
26         if (!b) return a;
27         // Make the smaller root the new root
28         if (comp(b->data, a->data)) std::swap(a, b);
29         // Attach b as the right child of a
30         a->right = merge(a->right, b, comp);
31         return a;
32     }
33
34     // Sift down from root: O(log n)
35     void sift_down(node* n) {
36         if (!n) return;
37         while (true) {

```

```

38         node* child = n->right;
39         if (!child) break;
40
41         // Find the smallest child considering reverse bit
42         node* smallest = child;
43         node* lc = nullptr;
44         if (n->reverse) {
45             // Left child is accessible via right chain
46             lc = child->right;
47         }
48         // (simplified comparison; full version tracks left child)
49         if (lc && comp(lc->data, smallest->data))
50             smallest = lc;
51
52         if (comp(smallest->data, n->data)) {
53             std::swap(n->data, smallest->data);
54             n = smallest;
55         } else {
56             break;
57         }
58     }
59 }
60
61 public:
62     weak_heap() = default;
63
64     // Insert: O(log n) amortized
65     void push(const T& value) {
66         auto* n = new node(value);
67         root_ = merge(root_, n, comp_);
68         ++size_;
69     }
70
71     // Delete-min: O(log n)
72     T pop() {
73         if (empty()) throw std::underflow_error("heap is empty");
74         T result = std::move(root_->data);
75         node* old_root = root_;
76         root_ = merge(root_->right, nullptr, comp_);
77         // In full implementation, detach left subtree and merge
78         delete old_root;
79         --size_;
80         return result;
81     }
82
83     const T& top() const {
84         if (empty()) throw std::underflow_error("heap is empty");
85         return root_->data;

```



```
86     }
87
88     bool empty() const noexcept { return size_ == 0; }
89     int size() const noexcept { return size_; }
90
91     ~weak_heap() {
92         // Cleanup (simplified)
93         while (!empty()) pop();
94     }
95 };
```

10.13.3 Complexity

Operation	Amortized
Insert	$O(\log n)$
Delete-min	$O(\log n)$
Find-min	$O(1)$
Merge	$O(\log n)$

Weak heaps have a simpler structure than binomial or Fibonacci heaps—each node stores only a data value, a right pointer, and a single reverse bit. In smoothsort, the weak heap structure is maintained in-place during the sort, giving $O(n \log n)$ worst-case with $O(1)$ extra space and excellent performance on nearly-sorted data.

10.14 Skew Heap

A **skew heap** is a self-adjusting mergeable heap—simpler than a Fibonacci heap but with the same amortized $O(\log n)$ bounds for all operations. Like leftist trees, it is a collection of heap-ordered binary trees, but it uses a different balancing strategy: after every merge, the left and right **spines** (paths to the rightmost and leftmost null pointers) are **swapped**. This eliminates the need for any rank or balance information.

10.14.1 Structure

```
1 Before merge: swap left/right spines
2
3 Leftist tree merge (only right spine merges):
4
5 [2]      [5]      [2]
6 / \    +  /      --> / \
7 3  4      6      4  [5]
8              /
9              6
10
```

```

11 Skew heap merge (swap spines after merge):
12
13     [2]           [5]           [2]
14     / \         +   /         --> / \
15    3   4         6           3 [5]  <-- spines swapped
16                               /
17                               [4]
18                               \
19                               6
20
21 After every merge, every node on the merged path swaps
22 its left and right children. This amortized restructuring
23 keeps the tree balanced without explicit rank information.

```

The spine swap is the key operation: it is applied to every node on the merge path, turning heavy right paths into left paths. While this may seem expensive, the potential function argument shows the total work is $O(\log n)$ amortized per merge.

10.14.2 Implementation

```

1  template <typename T, typename Compare = std::less<T>>
2  class skew_heap {
3      struct node {
4          T data;
5          node *left = nullptr, *right = nullptr;
6          explicit node(const T& value) : data(value) {}
7      };
8
9      node* root_ = nullptr;
10     int size_ = 0;
11     Compare comp_;
12
13     // Merge two skew heaps: O(log n) amortized
14     static node* merge(node* a, node* b, const Compare& comp) {
15         if (!a) return b;
16         if (!b) return a;
17
18         // Ensure a has the smaller root
19         if (comp(b->data, a->data)) std::swap(a, b);
20
21         // Merge along the right spine of a
22         node *curr = a, *prev = nullptr;
23         while (curr) {
24             if (comp(b->data, curr->data)) {
25                 // b should be above curr on the right spine
26                 // Swap spines: b takes curr's left subtree
27                 // as its right, and curr becomes b's right

```

```

28         node* temp = curr->right;
29         curr->right = b;
30         b = curr;
31         curr = temp;
32     }
33     prev = curr;
34     if (curr) curr = curr->right;
35 }
36 // Swap left and right children on the merge path
37 curr = a;
38 while (curr && curr != b) {
39     std::swap(curr->left, curr->right);
40     curr = curr->right;
41 }
42 return b;
43 }
44
45 public:
46     skew_heap() = default;
47
48     void push(const T& value) {
49         auto* n = new node(value);
50         root_ = merge(root_, n, comp_);
51         ++size_;
52     }
53
54     void push(T&& value) {
55         auto* n = new node(std::move(value));
56         root_ = merge(root_, n, comp_);
57         ++size_;
58     }
59
60     // Merge two skew heaps: O(log n) amortized
61     static skew_heap merge(skew_heap&& a, skew_heap&& b) {
62         skew_heap result;
63         result.root_ = merge(a.root_, b.root_, a.comp_);
64         result.size_ = a.size_ + b.size_;
65         a.root_ = nullptr;
66         b.root_ = nullptr;
67         return result;
68     }
69
70     T pop() {
71         if (empty()) throw std::underflow_error("heap is empty");
72         T result = std::move(root_->data);
73         node* old_root = root_;
74         root_ = merge(root_->left, root_->right, comp_);
75         delete old_root;

```

```

76         --size_;
77         return result;
78     }
79
80     const T& top() const {
81         if (empty()) throw std::underflow_error("heap is empty");
82         return root_>data;
83     }
84
85     bool empty() const noexcept { return size_ == 0; }
86     int size() const noexcept { return size_; }
87
88     ~skew_heap() {
89         while (!empty()) pop();
90     }
91 };

```

10.14.3 Amortized Analysis

The potential function is $\Phi = \sum_v s(v)$ where $s(v)$ is the number of right-heavy nodes in the right spine from v to the deepest node. A node is **right-heavy** if its right subtree has more nodes than its left subtree.

- **Merge:** Each merge traverses the right spine of a and swaps left/right at every node. The real cost is proportional to the path length, but the potential decreases by at least the same amount. Amortized cost: $O(\log n)$.
- **Insert:** Implemented as merge with a single-node heap. Amortized $O(\log n)$.
- **Delete-min:** Merge left and right subtrees of root. Amortized $O(\log n)$.

10.14.4 Skew Heap vs. Leftist Tree

Property	Skew Heap	Leftist Tree
Merge	$O(\log n)$ amort.	$O(\log n)$ worst
Insert	$O(\log n)$ amort.	$O(\log n)$ worst
Delete-min	$O(\log n)$ amort.	$O(\log n)$ worst
Node overhead	2 pointers	2 pointers + npl
Extra info	none	null path length
Cache behavior	poor (pointer chasing)	poor (pointer chasing)
Implementation	simpler	slightly more bookkeeping

Skew heaps are preferred when amortized bounds suffice and simplicity is valued. Leftist trees are preferred when worst-case guarantees are required (e.g., real-time systems). Both share the disadvan-

tage of poor cache locality due to pointer-based structures, which is why array-based heaps are faster in practice for single-threaded workloads.

10.15 Bottom-Up Heap Construction

Building a heap by repeated insertion takes $O(n \log n)$: each of the n insertions performs an $O(\log n)$ sift-up. **Floyd's algorithm** builds the heap in $O(n)$ by starting from the last non-leaf node and sifting down each node.

10.15.1 Why $O(n)$? Most Work Is at the Bottom

A complete binary tree with n nodes has $\lceil n/2 \rceil$ leaves (no sift needed) and $\lfloor n/2 \rfloor$ internal nodes. Nodes near the bottom are numerous but require very little sift-down work; nodes near the root are few but require more. The total work is dominated by the abundant bottom-level nodes doing almost nothing.

10.15.2 Potential Analysis

Define the **potential** Φ as the total number of children that could still be swapped during all pending sift-down operations. Each sift-down of a node at height h performs at most h swaps, reducing the potential by at least h . When a swap occurs, the potential decreases by exactly 1 per swap. The sum of all heights in a complete binary tree is:

$$\sum_{i=0}^{n-1} \lfloor \log_2(i+1) \rfloor = n \lfloor \log_2 n \rfloor - 2^{\lfloor \log_2 n \rfloor + 1} + 2 = O(n)$$

Since the total work equals the total decrease in potential, the overall cost is $O(n)$. The initial potential is $O(n)$ and it only decreases, so the amortized cost per sift-down is $O(1)$ when charged to the total.

10.15.3 Implementation

```

1 template <typename T, typename Compare = std::less<T>>
2 void build_heap(std::vector<T>& data, Compare comp = {}) {
3     int n = static_cast<int>(data.size());
4     // Start from last non-leaf node, sift down to root
5     for (int i = n / 2 - 1; i >= 0; --i) {
6         int current = i;
7         while (true) {
8             int largest = current;
9             int left = 2 * current + 1;
10            int right = 2 * current + 2;
11            if (left < n && comp(data[largest], data[left]))
12                largest = left;
13            if (right < n && comp(data[largest], data[right]))
14                largest = right;

```

```

15         if (largest == current) break;
16         std::swap(data[current], data[largest]);
17         current = largest;
18     }
19 }
20 }

```

10.15.4 Worked Trace (15 Elements)

Build a max-heap from [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 7, 8, 2, 0]:

```

1 Array (1-indexed for clarity):
2
3 Index:  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
4 Value: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 7, 8, 2, 0]
5
6 Tree structure (heights labeled):
7           [3]                      h=3
8         /   \
9       [1]   [4]                  h=2
10      /  \   /  \
11     [1]  [5] [9]  [2]            h=1
12    / \  / \ / \ / \
13   [6] [5] [3] [5] [7] [8] [2] [0]   h=0
14
15 Last non-leaf: index 7 (value 2).
16 Process nodes 7, 6, 5, 4, 3, 2, 1 (right to left).
17
18 Step 1: Sift down node 7 (value 2, height 1)
19   Children: 14 (value 2), 15 (value 0)
20   2 >= 2 and 2 >= 0 -> no swap needed.
21   Array unchanged.
22
23 Step 2: Sift down node 6 (value 9, height 1)
24   Children: 12 (value 7), 13 (value 8)
25   9 >= 7 and 9 >= 8 -> no swap needed.
26
27 Step 3: Sift down node 5 (value 5, height 1)
28   Children: 10 (value 3), 11 (value 5)
29   5 >= 3 and 5 >= 5 -> no swap needed.
30
31 Step 4: Sift down node 4 (value 1, height 1)
32   Children: 8 (value 6), 9 (value 5)
33   1 < 6 -> swap with 6.
34   [3, 1, 4, 6, 5, 9, 2, 1, 5, 3, 5, 7, 8, 2, 0]
35   Node 8 (value 1) is a leaf -> done.
36
37 Step 5: Sift down node 3 (value 4, height 2)

```

```

38  Children: 6 (value 9), 7 (value 2)
39  4 < 9 -> swap with 9.
40  [3, 1, 9, 6, 5, 4, 2, 1, 5, 3, 5, 7, 8, 2, 0]
41  Now at index 6. Children: 12 (7), 13 (8)
42  4 < 8 -> swap with 8.
43  [3, 1, 9, 6, 5, 8, 2, 1, 5, 3, 5, 7, 4, 2, 0]
44  Index 12 has no children -> done.
45
46  Step 6: Sift down node 2 (value 1, height 3)
47  Children: 4 (value 6), 5 (value 5)
48  1 < 6 -> swap with 6.
49  [3, 6, 9, 1, 5, 8, 2, 1, 5, 3, 5, 7, 4, 2, 0]
50  Now at index 4. Children: 8 (1), 9 (5)
51  1 < 5 -> swap with 5.
52  [3, 6, 9, 5, 1, 8, 2, 1, 3, 5, 5, 7, 4, 2, 0]
53  Index 8 is a leaf -> done.
54
55  Step 7: Sift down node 1 (value 3, height 3)
56  Children: 2 (value 6), 3 (value 9)
57  3 < 9 -> swap with 9.
58  [9, 6, 3, 5, 1, 8, 2, 1, 3, 5, 5, 7, 4, 2, 0]
59  Now at index 3. Children: 6 (8), 7 (2)
60  3 < 8 -> swap with 8.
61  [9, 6, 8, 5, 1, 3, 2, 1, 3, 5, 5, 7, 4, 2, 0]
62  Index 6 is a leaf -> done.
63
64  Final max-heap:
65
66          [9]
67         /  \
68        [6]  [8]
69       / \  / \
70      [5] [1][3] [2]
71     /\  /\ /\  /\
72    [1][3][5][5][7][4][2][0]

```

Comparison count: Total sift-down comparisons = $0 + 0 + 0 + 1 + 2 + 2 + 2 = 7$. A naive repeated-insertion approach would require $15 \times O(\log 15) \approx 58$ comparisons. Floyd's algorithm is roughly $8\times$ faster on this example.

10.15.5 Complexity Summary

Method	Time	Space
Repeated insert (top-down)	$O(n \log n)$	$O(n)$
Floyd's bottom-up heapify	$O(n)$	$O(1)$

Floyd's algorithm is **optimal**: any comparison-based heap construction algorithm must examine every node, giving a lower bound of $\Omega(n)$. The $O(n)$ cost comes from the fact that most nodes are

near the leaves and require little work, while the few deep nodes that need more sift-down steps are compensated by their scarcity.

10.16 Heap Select (kth Smallest)

A **heap** provides an efficient way to find the k -th smallest (or largest) element without fully sorting the data. The approach depends on whether we want the k -th smallest or k -th largest, and whether we value worst-case or average-case guarantees.

10.16.1 Min-Heap Approach: kth Smallest

Build a min-heap of all n elements ($O(n)$ with Floyd's algorithm). Extract the minimum k times ($O(k \log n)$). The k -th extraction gives the k -th smallest.

```

1 int kth_smallest_minheap(std::vector<int> data, int k) {
2     // Floyd's O(n) build
3     std::make_heap(data.begin(), data.end(), std::greater<int>{});
4     for (int i = 0; i < k - 1; ++i) {
5         std::pop_heap(data.begin(), data.end(), std::greater<int>{});
6         data.pop_back();
7     }
8     return data.front(); // kth smallest
9 }

```

Time: $O(n + k \log n)$. Space: $O(1)$ in-place. This is ideal when $k \ll n$.

10.16.2 Max-Heap Approach: kth Largest

For the k -th *largest*, use a max-heap of size k : insert the first k elements, then for each remaining element, if it is smaller than the root, replace the root and sift down. The root of the final heap is the k -th largest.

```

1 int kth_largest_maxheap(std::span<int> data, int k) {
2     // Build max-heap of first k elements
3     std::vector<int> heap(data.begin(), data.begin() + k);
4     std::make_heap(heap.begin(), heap.end());
5
6     for (int i = k; i < static_cast<int>(data.size()); ++i) {
7         if (data[i] < heap.front()) {
8             heap.front() = data[i];
9             std::sift_down_heap(heap.begin(), heap.end());
10        }
11    }
12    return heap.front(); // kth largest
13 }

```

Time: $O(n \log k)$. Space: $O(k)$. This is best when $k \ll n$ and memory is constrained.

10.16.3 Comparison with Quickselect

Method	Average	Worst	Space
Min-heap (k th smallest)	$O(n + k \log n)$	$O(n + k \log n)$	$O(1)$
Max-heap (k th largest)	$O(n \log k)$	$O(n \log k)$	$O(k)$
Quickselect	$O(n)$	$O(n^2)$	$O(1)$
Median-of-medians	$O(n)$	$O(n)$	$O(1)$

Quickselect is faster on average ($O(n)$ vs $O(n \log k)$) but has $O(n^2)$ worst-case. The max-heap approach is preferred when: (a) k is small, (b) the data arrives as a stream (online), or (c) worst-case $O(n \log k)$ is more predictable than quickselect's quadratic worst case. The min-heap approach is cleanest when you already have a heapified array and want the k -th smallest in $O(k \log n)$ additional work.

10.17 Application: Event-Driven Simulation

Simulate a bank with multiple tellers:

```

1 struct event {
2     double time;
3     enum { ARRIVAL, DEPARTURE } type;
4     int customer_id;
5     int teller_id;
6
7     bool operator>(const event& other) const {
8         return time > other.time;
9     }
10 };
11
12 void bank_simulation(int num_tellers, double arrival_rate, double duration
13 ) {
14     // Min-heap of events
15     std::priority_queue<event, std::vector<event>, std::greater<event>>
16     events;
17     std::vector<int> teller_available_time(num_tellers, 0);
18
19     // First arrival
20     events.push({0.0, event::ARRIVAL, 0, -1});
21
22     int customer_count = 0;
23     double total_wait = 0.0;
24
25     while (!events.empty()) {
26         event e = events.top(); events.pop();
27         if (e.time > duration) break;

```

```

27     if (e.type == event::ARRIVAL) {
28         ++customer_count;
29         // Schedule next arrival
30         double interarrival = -std::log(1.0 - std::rand() / RAND_MAX)
/ arrival_rate;
31         events.push({e.time + interarrival, event::ARRIVAL,
customer_count, -1});
32
33         // Find free teller
34         int best_teller = 0;
35         for (int t = 1; t < num_tellers; ++t) {
36             if (teller_available_time[t] < teller_available_time[
best_teller])
37                 best_teller = t;
38         }
39
40         double start = std::max(e.time, double(teller_available_time[
best_teller]));
41         double wait = start - e.time;
42         total_wait += wait;
43
44         double service_time = 1.0 + std::rand() / RAND_MAX * 4.0; //
1-5 min
45         teller_available_time[best_teller] = start + service_time;
46         events.push({start + service_time, event::DEPARTURE,
customer_count, best_teller});
47     }
48 }
49
50 std::cout << "Customers served: " << customer_count << '\n'
51           << "Average wait: " << total_wait / customer_count << '\n';
52 }

```

10.18 Application: Median Maintenance

Maintain the median of a stream of numbers:

```

1 class median_tracker {
2 public:
3     void add_number(int x) {
4         if (max_heap_.empty() || x <= max_heap_.top()) {
5             max_heap_.push(x);
6         } else {
7             min_heap_.push(x);
8         }
9
10        // Rebalance

```

```

11     if (max_heap_.size() > min_heap_.size() + 1) {
12         min_heap_.push(max_heap_.top());
13         max_heap_.pop();
14     } else if (min_heap_.size() > max_heap_.size()) {
15         max_heap_.push(min_heap_.top());
16         min_heap_.pop();
17     }
18 }
19
20 double median() const {
21     if (max_heap_.size() == min_heap_.size()) {
22         return (max_heap_.top() + min_heap_.top()) / 2.0;
23     }
24     return max_heap_.top();
25 }
26
27 private:
28     std::priority_queue<int> max_heap_;           // lower half
29     std::priority_queue<int, std::vector<int>,
30                     std::greater<int>> min_heap_; // upper half
31 };

```

10.19 STL Connection: `std::priority_queue`

```

1 std::priority_queue<int> pq; // max-heap
2 pq.push(30);
3 pq.push(10);
4 pq.push(50);
5 std::cout << pq.top(); // 50
6 pq.pop();              // removes 50
7 std::cout << pq.top(); // 30

```

`std::priority_queue` uses `std::vector` by default and provides the standard heap operations.

For direct heap manipulation on arbitrary vectors:

```

1 std::vector<int> v = {30, 10, 50, 20, 40};
2 std::make_heap(v.begin(), v.end());           // O(n)
3 std::pop_heap(v.begin(), v.end());             // move max to end, O(log n)
4 v.pop_back();
5 std::push_heap(v.begin(), v.end());            // after push_back, O(log n)
6 std::sort_heap(v.begin(), v.end());            // O(n log n)

```

10.20 Common Bugs and Pitfalls

Pitfall	Consequence
Confusing min-heap vs max-heap comparator	Wrong element at top, incorrect algorithm
Forgetting <code>sift_down</code> after replacing the root	Heap property violated
Modifying priority of an element already in the heap	Heap property violated, wrong top element
Off-by-one in child index formulas ($2i$ vs $2i+1$)	Access wrong node
Using <code>std::make_heap</code> but never <code>std::push_heap</code> / <code>std::pop_heap</code>	Not maintaining heap property after push/pop
Passing the wrong comparator to <code>std::priority_queue</code>	Opposite ordering than expected

10.21 Summary

1. **Heaps** implement priority queues with $O(\log n)$ push/pop and $O(n)$ heapify.
2. **Array storage** exploits the complete binary tree structure with simple index arithmetic.
3. **Heap sort** is an $O(n \log n)$ in-place sorting algorithm.
4. **Leftist trees** support $O(\log n)$ merge of two priority queues.
5. **Fibonacci heaps** achieve $O(1)$ amortized insert and decrease-key, $O(\log n)$ extract-min — optimal for Dijkstra's.
6. **Pairing heaps** offer similar practical performance with simpler implementation.
7. `std::priority_queue` is the production priority queue.

10.22 Interview FAQs

1. **How does `heapify` build a heap in $O(n)$ instead of $O(n \log n)$?**

Floyd's bottom-up approach: start from the last non-leaf node (index $\lfloor n/2 \rfloor - 1$) and sift down each node. Nodes at height h require $O(h)$ sift-down operations. Total work: $\sum_{h=0}^{\log n} \lceil n/2^{h+1} \rceil \cdot h = O(n)$. This is provably optimal – you cannot build a heap faster than $O(n)$. Inserting elements one by one is $O(n \log n)$ because each insert is $O(\log n)$.

```

1 void sift_down(std::vector<int>& h, int n, int i) {
2     while (true) {
3         int smallest = i, l = 2*i+1, r = 2*i+2;
4         if (l < n && h[l] < h[smallest]) smallest = l;
5         if (r < n && h[r] < h[smallest]) smallest = r;
6         if (smallest == i) break;
7         std::swap(h[i], h[smallest]);
8         i = smallest;

```

```

9      }
10   }
11
12   void heapify(std::vector<int>& h) {
13       int n = (int)h.size();
14       for (int i = n/2 - 1; i >= 0; --i)
15           sift_down(h, n, i);
16   }

```

1. Find the k -th largest element in an unsorted array.

Approach 1: Build a min-heap of size k . For each remaining element, if it's larger than the heap root, replace the root and sift down. The root of the final heap is the k -th largest. Time $O(n \log k)$, space $O(k)$.

```

1   int kth_largest(const std::vector<int>& a, int k) {
2       std::priority_queue<int, std::vector<int>, std::greater<int>> pq;
3       for (int x : a) {
4           pq.push(x);
5           if ((int)pq.size() > k) pq.pop();
6       }
7       return pq.top();
8   }

```

Approach 2: Quickselect (average $O(n)$, worst $O(n^2)$). Partition around a random pivot; recurse into the side containing the k -th element.

Approach 3: Build a max-heap, extract the max k times. Time $O(n + k \log n)$.

Interviewers typically expect Approach 1 or 2.

1. Design a data structure that supports insert, delete, and getMin in $O(1)$.

Use two heaps: a min-heap for insert and getMin, and a “deletion” heap. On `delete(x)`, push x to the deletion heap. On `getMin`, peek both heaps and skip elements that appear in both (lazy deletion). Periodically rebuild the min-heap when the deletion heap grows too large. This is used in Dijkstra’s algorithm with Fibonacci heaps.

Variant: “Decrease-key in $O(1)$ ” — Fibonacci heap. Insert: $O(1)$ amortized. Extract-min: $O(\log n)$ amortized.

1. What is the difference between a max-heap and a min-heap? When would you use each?

Q: “Why does Dijkstra’s algorithm need a min-heap with decrease-key, and what happens if you use a max-heap or skip decrease-key?”

A: Dijkstra processes nodes in order of increasing distance — a min-heap gives $O(\log n)$ extract-min. Without decrease-key, you insert duplicates and lazily skip stale entries (worse asymptotic but often faster in practice due to simpler implementation). A max-heap would give the *farthest*

node, which is the opposite of what Dijkstra needs. The decrease-key optimization avoids duplicates by updating existing entries — but implementing decrease-key efficiently requires an index map (heap position per element), which adds complexity. In practice, the “lazy” approach (insert duplicates, skip stale) is preferred because the simpler code and better cache behavior outweigh the theoretical cost.

1. Implement a priority queue using a sorted array. What are the trade-offs?

Sorted array (ascending): insert is $O(n)$ (shift elements), delete-max is $O(1)$ (remove from end).
 Sorted array (descending): insert is $O(n)$, delete-max is $O(n)$ (shift).
 Unsorted array: insert is $O(1)$, delete-max is $O(n)$ (scan for max).
 Binary heap: both insert and delete-max are $O(\log n)$.
 The heap wins for mixed workloads; the unsorted array wins if you only insert then extract all.

1. How do you merge k sorted linked lists efficiently?

Use a min-heap of size k : insert the head of each list. Extract the minimum, append it to the result, and push the next element from the same list. Time $O(n \log k)$ where n is the total number of elements. Space $O(k)$ for the heap.

```

1 node* merge_k_sorted(std::vector<node*>& lists) {
2     auto cmp = [](node* a, node* b) { return a->data > b->data; };
3     std::priority_queue<node*, std::vector<node*>, decltype(cmp)> pq(cmp);
4     for (node* h : lists) if (h) pq.push(h);
5     node dummy(0);
6     node* tail = &dummy;
7     while (!pq.empty()) {
8         node* cur = pq.top(); pq.pop();
9         tail->next = cur;
10        tail = tail->next;
11        if (cur->next) pq.push(cur->next);
12    }
13    return dummy.next;
14 }
```

Variant: “Merge k sorted arrays” – same approach, or divide-and-conquer pairwise merging.

1. What is a leftist tree and why is it useful?

A leftist tree is a mergeable heap: the merge of two leftist trees is $O(\log n)$, compared to $O(n)$ for binary heaps. Each node stores a “null path length” (npl): the shortest path to a null child. The left child always has $npl \geq$ right child. This ensures the right spine is short, making merge efficient by recursively merging along the right spine. Used in event-driven simulations and parallel priority queues.

Follow-up: “What is a skew heap?” A self-adjusting leftist tree that swaps children during merge. $O(\log n)$ amortized, simpler implementation, but no worst-case guarantee.

1. How do you find the median of a data stream?

Maintain two heaps: a max-heap for the lower half and a min-heap for the upper half. The max-heap stores elements $\leq$ median; the min-heap stores elements $\geq$ median. Keep their sizes equal (or max-heap one larger). The median is the max-heap's root. Insert $O(\log n)$, getMedian $O(1)$.

```

1 class median_tracker {
2     std::priority_queue<int> lo; // max-heap
3     std::priority_queue<int, std::vector<int>, std::greater<int>> hi; //
    min-heap
4 public:
5     void add(int num) {
6         lo.push(num);
7         hi.push(lo.top()); lo.pop();
8         if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
9     }
10    double median() {
11        return lo.size() > hi.size() ? lo.top() : (lo.top() + hi.top()) /
        2.0;
12    }
13 };

```

This is LeetCode 295 — a very common interview question.

1. Why is heap sort not used in practice despite $O(n \log n)$ time?

Heap sort has poor cache locality: the parent-child index relationship ($2i+1$, $2i+2$) jumps across the array, causing cache misses. Quicksort, despite $O(n^2)$ worst-case, is faster in practice due to sequential memory access patterns. Additionally, heap sort is not stable (equal elements may be reordered). Introsort (quicksort + heapsort fallback) combines the best of both: quicksort's cache performance with heapsort's $O(n \log n)$ guarantee.

1. What is the difference between `std::priority_queue` and a raw heap?

`std::priority_queue` is an adapter over `std::vector` using `std::make_heap`, `std::push_heap`, `std::pop_heap`. The raw heap functions operate on random-access iterators directly. Use `std::priority_queue` for simplicity; use raw heap functions when you need: (a) access to the underlying vector, (b) custom comparison with iterators, (c) building a heap in-place without copying. Both use the same $O(\log n)$ operations internally.

10.23 Exercises

10.23.1 Drill

1. Show the array representation of a max-heap after each insertion of: [10, 20, 5, 30, 15, 25]. Use the array indices 1..n (1-indexed) for clarity. Draw the tree at each step.

1. Show the array after each step of heapify on [5, 3, 17, 10, 84, 19, 6, 22, 9] (a max-heap). Start from the last non-leaf node.
1. How many comparisons are needed to build a heap of size n using heapify? Prove your answer.
1. Trace heap sort on [3, 1, 4, 1, 5, 9, 2, 6]. Show the array after each `extract_max` and after each `sift_down`.
1. For a min-heap, what is the complexity of:
 - a) Finding the minimum element
 - b) Inserting a new element
 - c) Deleting the minimum
 - d) Merging two heaps of size n and m (without using any special mergeable structure)

10.23.2 Application

1. Implement a min-heap using `std::greater` as the comparator. Verify correctness against `std::priority_queue<int, std::vector<int>, std::greater<int>>`.
1. Implement heap sort in-place. Benchmark against `std::sort` for $n = 10^3, 10^4, 10^5, 10^6$ on random, sorted, and reverse-sorted data. Explain why `std::sort` is faster despite both being $O(n \log n)$.
1. Implement the **k-way merge**: merge k sorted arrays into one sorted array using a min-heap. Compare its performance with repeated 2-way merges for $k = 4, 8, 16$ at total $n = 10^5$.
1. Implement a priority queue with **decrease-key** — needed for Dijkstra’s algorithm. Store each element’s position in the heap using a hash map. Verify correctness with Dijkstra on a small graph.
1. Implement `is_heap` that checks if a vector satisfies the heap property. Use it to validate your heap implementations on random vectors.
1. Use two heaps (a max-heap for the lower half, a min-heap for the upper half) to implement a `median_tracker` that finds the running median of a stream. Process 10^6 numbers and report the median after each 100,000 elements.
1. Implement the **sliding window median**: given an array and window size k , output the median of each window. Use the two-heap approach from exercise 11.
1. Implement **Prim’s MST algorithm** using a priority queue with decrease-key. Compare its run-time with the version that does not use decrease-key (inserts duplicates instead).

10.23.3 Research

1. **(Binomial heaps)** Implement a binomial heap — a forest of binomial trees that supports $O(\log n)$ merge. Compare merge performance with the `std::priority_queue` vs your binomial heap. When does the merge capability matter?
1. **(Fibonacci heaps)** Read about Fibonacci heaps (Fredman and Tarjan, 1987). Implement decrease-key with $O(1)$ amortized time. Compare its performance with the standard binary heap for Dijkstra's algorithm on large sparse graphs.
1. **(Priority queue STL choice)** The C++ standard requires `std::priority_queue` to use `std::vector` by default. Why not `std::deque`? What are the performance implications? Read the committee discussions.
1. **(d-ary heaps)** Implement a 4-ary heap (each node has up to 4 children). Compare its performance with the binary heap. How does the choice of d affect the number of comparisons vs the number of swaps?

10.24 Optimal Heap Complexity

The Fibonacci heap achieves $O(1)$ amortised insert and decrease-key, and $O(\log n)$ amortised delete-min. But is this the best possible? The answer is yes, up to constant factors.

10.24.1 Lower Bounds

Any comparison-based priority queue must perform $\Omega(1)$ amortised time per operation. For decrease-key, the lower bound is $\Omega(1)$ amortised, which the Fibonacci heap matches. For delete-min, the lower bound is $\Omega(\log n)$ amortised, which Fibonacci heaps also match.

However, the Fibonacci heap's $O(1)$ insert and decrease-key come with high constant factors and poor cache performance. In practice, pairing heaps and skew heaps are faster despite having $O(\log n)$ amortised decrease-key.

10.24.2 Strict Fibonacci Heaps

The strict Fibonacci heap (Brodal, Lagogiannis, and Tarjan, 2002) achieves $O(1)$ worst-case insert, $O(1)$ worst-case decrease-key, and $O(\log n)$ worst-case delete. This is optimal in the comparison model. However, the implementation is extremely complex and has large constant factors, making it purely theoretical.

Structure	Insert	Find-Min	Delete-Min	Decrease-Key
Binary heap	$O(1)$	$O(1)$	$O(\log n)$	$O(\log n)$
Binomial heap	$O(1)$ am.	$O(1)$	$O(\log n)$ am.	$O(\log n)$ am.
Fibonacci heap	$O(1)$ am.	$O(1)$	$O(\log n)$ am.	$O(1)$ am.
Pairing heap	$O(1)$ am.	$O(1)$	$O(\log n)$ am.	$O(\log n)$ am.
Strict Fib. heap	$O(1)$	$O(1)$	$O(\log n)$	$O(1)$

10.25 BST-as-Heap: Tree-Oriented Heaps

Several heap implementations use BST structures as the underlying container, combining heap ordering with BST ordering to achieve efficient merge, split, and priority queue operations.

10.25.1 Treap as Priority Queue

A treap (tree + heap) assigns random priorities to BST nodes. The BST property holds on keys, and the heap property holds on priorities. This gives:

- Insert: $O(\log n)$ expected
- Delete-min: $O(\log n)$ expected (by key or priority)
- Merge: $O(\log n)$ expected (when all keys in one tree are less than the other)
- Split: $O(\log n)$ expected

The treap is the simplest data structure that simultaneously supports BST operations and priority queue operations with good expected performance.

10.25.2 Interval Heap

An **interval heap** stores intervals $[a, b]$ where $a \leq b$. It maintains a min-heap on the left endpoints and a max-heap on the right endpoints. This enables:

- Insert interval: $O(\log n)$
- Find interval containing a point: $O(\log n)$
- Delete interval: $O(\log n)$

Interval heaps are useful for scheduling, calendar systems, and interval management.

10.25.3 Leaf-to-Root Operations

In some heap variants (e.g., skew heaps, leftist trees), operations propagate from leaves toward the root. This “leaf-to-root” pattern is the dual of the more common “root-to-leaf” pattern used in BSTs and array heaps.

```

1 // Leaf-to-root merge in a skew heap
2 SkewHeap merge(SkewHeap a, SkewHeap b) {
3     if (!a.root) return b;
4     if (!b.root) return a;
5     if (a.root->priority > b.root->priority)
6         std::swap(a, b);
7     // Merge b into a's right subtree
8     a.root->right = merge(a.root->right, b);
9     // Swap children (the skew heap trick)
10    std::swap(a.root->left, a.root->right);
11    return a;
12 }
```

The swap step ensures amortised $O(\log n)$ time by making the long path become the right spine, which is where future merges will attach.

10.26 Heaps and the Pointer Machine

The **pointer machine** model counts only pointer operations (dereference, assignment) and ignores arithmetic. In this model, comparison-based pointer-machine heaps require $\Omega(\log n)$ per operation for any combination of insert, delete-min, and decrease-key (with at least one operation being non-constant).

Fibonacci heaps achieve this bound: $O(1)$ for insert and decrease-key, $O(\log n)$ for delete-min. The key insight is that the heap structure is maintained through pointer manipulation rather than array indexing, enabling $O(1)$ merge and decrease-key.

10.27 References and Further Reading

- Donald E. Knuth, *The Art of Computer Programming*, Volume 3: Sorting and Searching. Section 5.2.3.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapter 6.
- Robert W. Floyd, “Algorithm 245: Treesort” — CACM, 1964.
- J. W. J. Williams, “Algorithm 232: Heapsort” — CACM, 1964.
- Jean Vuillemin, “A Data Structure for Manipulating Priority Queues” — CACM, 1978.

- Michael L. Fredman and Robert E. Tarjan, "Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms" — JACM, 1987.
- Michael L. Fredman, Robert Sedgwick, Daniel D. Sleator, and Robert E. Tarjan, "The Pairing Heap: A New Form of Self-Adjusting Heap" — Algorithmica, 1986.

Binary Search Trees and Balanced Search Trees

11.1 Introduction

❧ Complete compilable implementations of the data structures in this chapter are in `code/`.

11.2 Binary Search Trees

A **binary search tree (BST)** is a binary tree where for every node:

- All keys in the **left subtree** are less than the node's key
- All keys in the **right subtree** are greater than the node's key

```

1           [50]
2          /   \
3         [30]  [70]
4        /  \  /  \
5       [20] [40] [60] [80]
```

11.2.1 BST Implementation

```

1 template <typename K, typename V>
2 class bst {
3 public:
4     using node_ptr = std::unique_ptr<node>;
5
6     bst() = default;
7
8     void insert(const K& key, const V& value) {
9         insert(root_, key, value);
10    }
11
12    std::optional<V> find(const K& key) const {
13        auto* n = find(root_.get(), key);
14        if (n) return n->value;
15        return std::nullopt;
16    }
17
18    void erase(const K& key) {
```

```

19     root_ = erase(std::move(root_), key);
20 }
21
22 void inorder(std::ostream& os) const {
23     inorder(root_.get(), os);
24 }
25
26 private:
27     struct node {
28         K key;
29         V value;
30         node_ptr left;
31         node_ptr right;
32
33         node(const K& k, const V& v) : key(k), value(v) {}
34     };
35
36     static void insert(node_ptr& n, const K& key, const V& value) {
37         if (!n) {
38             n = std::make_unique<node>(key, value);
39             return;
40         }
41         if (key < n->key) insert(n->left, key, value);
42         else if (key > n->key) insert(n->right, key, value);
43         else n->value = value; // update
44     }
45
46     static node* find(node* n, const K& key) {
47         if (!n) return nullptr;
48         if (key < n->key) return find(n->left.get(), key);
49         if (key > n->key) return find(n->right.get(), key);
50         return n;
51     }
52
53     static node_ptr erase(node_ptr n, const K& key) {
54         if (!n) return nullptr;
55         if (key < n->key) {
56             n->left = erase(std::move(n->left), key);
57             return n;
58         }
59         if (key > n->key) {
60             n->right = erase(std::move(n->right), key);
61             return n;
62         }
63
64         // Found the node -- handle the three cases
65         if (!n->left) return std::move(n->right);
66         if (!n->right) return std::move(n->left);

```

```

67
68     // Two children: replace with inorder successor
69     auto successor = find_min(std::move(n->right));
70     successor->right = delete_min(std::move(n->right));
71     successor->left = std::move(n->left);
72     return successor;
73 }
74
75 static node_ptr find_min(node_ptr n) {
76     auto* current = n.get();
77     while (current && current->left) current = current->left.get();
78     auto result = std::make_unique<node>(current->key, current->value)
79     ;
80     return result;
81 }
82
83 static node_ptr delete_min(node_ptr n) {
84     if (!n->left) return std::move(n->right);
85     n->left = delete_min(std::move(n->left));
86     return n;
87 }
88
89 static void inorder(node* n, std::ostream& os) {
90     if (!n) return;
91     inorder(n->left.get(), os);
92     os << n->key << " ";
93     inorder(n->right.get(), os);
94 }
95
96 node_ptr root_;
97 };

```

11.2.2 Complexity

Operation	Average Case	Worst Case
find	$O(\log n)$	$O(n)$
insert	$O(\log n)$	$O(n)$
erase	$O(\log n)$	$O(n)$

The worst case occurs when keys are inserted in sorted (or reverse sorted) order — the BST degenerates into a linked list. **Balanced trees** solve this problem.

11.3 AVL Trees

An **AVL tree** (Adelson-Velsky and Landis, 1962) maintains the invariant: for every node, the heights of its left and right subtrees differ by at most 1.

Balance factor: $\text{height}(\text{left}) - \text{height}(\text{right}) \in -1, 0, 1$

11.3.1 Rotations

When an insertion or deletion violates the balance invariant, we perform **rotations** to restore it.

Right rotation (LL imbalance):

```

1      [z] (bf = 2)          [y]
2      /                    -> /  \
3      [y] (bf = 1)          [x]   [z]
4      /
5     [x]
```

Left rotation (RR imbalance):

```

1 [z] (bf = -2)          [y]
2  \                    -> /  \
3  [y] (bf = -1)        [z]   [x]
4    \
5     [x]
```

Left-right rotation (LR imbalance):

```

1      [z]          [z]          [x]
2      /            /            /  \
3      [y]          -> [x]          -> [y]  [z]
4      \            /
5      [x]          [y]
```

Right-left rotation (RL imbalance): symmetric.

11.3.2 AVL Implementation

```

1 template <typename K, typename V>
2 class avl_tree {
3     struct avl_node {
4         K key;
5         V value;
6         std::unique_ptr<avl_node> left;
7         std::unique_ptr<avl_node> right;
8         int height = 1;
9
10        avl_node(const K& k, const V& v) : key(k), value(v) {}
11    };
12
13    using node_ptr = std::unique_ptr<avl_node>;
14
15    int height(const avl_node* n) const {
16        return n ? n->height : 0;
```



```

17     }
18
19     int balance_factor(const avl_node* n) const {
20         return height(n->left.get()) - height(n->right.get());
21     }
22
23     void update_height(avl_node* n) {
24         n->height = 1 + std::max(height(n->left.get()), height(n->right.
25         get()));
26     }
27
28     // Right rotation
29     node_ptr rotate_right(node_ptr z) {
30         auto y = std::move(z->left);
31         z->left = std::move(y->right);
32         update_height(z.get());
33         y->right = std::move(z);
34         update_height(y.get());
35         return y;
36     }
37
38     // Left rotation
39     node_ptr rotate_left(node_ptr z) {
40         auto y = std::move(z->right);
41         z->right = std::move(y->left);
42         update_height(z.get());
43         y->left = std::move(z);
44         update_height(y.get());
45         return y;
46     }
47
48     // Rebalance: apply rotations if needed
49     node_ptr rebalance(node_ptr n) {
50         if (!n) return nullptr;
51         update_height(n.get());
52         int bf = balance_factor(n.get());
53
54         if (bf > 1) {
55             // Left-heavy: check left child's balance
56             if (balance_factor(n->left.get()) < 0) {
57                 n->left = rotate_left(std::move(n->left));
58             }
59             return rotate_right(std::move(n));
60         }
61         if (bf < -1) {
62             // Right-heavy: check right child's balance
63             if (balance_factor(n->right.get()) > 0) {
64                 n->right = rotate_right(std::move(n->right));

```

```

64         }
65         return rotate_left(std::move(n));
66     }
67     return n;
68 }
69
70 public:
71     void insert(const K& key, const V& value) {
72         root_ = insert(std::move(root_), key, value);
73     }
74
75     void erase(const K& key) {
76         root_ = erase(std::move(root_), key);
77     }
78
79     std::optional<V> find(const K& key) const {
80         auto* n = find(root_.get(), key);
81         return n ? std::optional<V>(n->value) : std::nullopt;
82     }
83
84 private:
85     node_ptr insert(node_ptr n, const K& key, const V& value) {
86         if (!n) return std::make_unique<avl_node>(key, value);
87         if (key < n->key) n->left = insert(std::move(n->left), key, value)
88         ;
89         else if (key > n->key) n->right = insert(std::move(n->right), key,
90         value);
91         else n->value = value; // update
92         return rebalance(std::move(n));
93     }
94
95     node_ptr erase(node_ptr n, const K& key) {
96         if (!n) return nullptr;
97         if (key < n->key) {
98             n->left = erase(std::move(n->left), key);
99         } else if (key > n->key) {
100             n->right = erase(std::move(n->right), key);
101         } else {
102             if (!n->left) return std::move(n->right);
103             if (!n->right) return std::move(n->left);
104             // Two children: find inorder successor
105             auto successor = find_min(std::move(n->right));
106             successor->right = delete_min(std::move(n->right));
107             successor->left = std::move(n->left);
108             return rebalance(std::move(successor));
109         }
110         return rebalance(std::move(n));
111     }

```

```

110
111     static avl_node* find(avl_node* n, const K& key) {
112         if (!n) return nullptr;
113         if (key < n->key) return find(n->left.get(), key);
114         if (key > n->key) return find(n->right.get(), key);
115         return n;
116     }
117
118     node_ptr root_;
119 };

```

11.3.3 Complexity

All operations: $O(\log n)$ worst case — guaranteed, not just expected.

11.4 Red-Black Trees

Red-black trees are an alternative balancing scheme that powers `std::map` and `std::set`.

Properties:

1. Every node is red or black
2. The root is black
3. Leaves (null children) are black
4. Red nodes cannot have red children (no two consecutive reds)
5. Every path from root to leaf has the same number of black nodes

These properties guarantee that the longest path from root to leaf is at most $2\times$ the shortest path, giving $O(\log n)$ height.

Red-black trees are more complex to implement than AVL trees but require fewer rotations during insertion ($O(1)$ vs $O(\log n)$). This makes them preferred in practice for `std::map`.

11.4.1 RB Tree Implementation

The red-black tree below uses the same recursive approach as our AVL tree. After inserting a new red node, the recursion unrolls and checks for red-red violations at the grandparent level. The three fixup cases match the standard algorithm: recolor when the uncle is red, straight rotation when the red child is on the outside, and zig-zag rotation when it is on the inside.

```

1 template <typename K, typename V, typename Compare = std::less<K>>
2 class rb_tree {
3     enum color { RED, BLACK };
4

```

```

5     struct rb_node {
6         K key;
7         V value;
8         color col = RED;
9         std::unique_ptr<rb_node> left;
10        std::unique_ptr<rb_node> right;
11
12        rb_node(const K& k, const V& v) : key(k), value(v) {}
13    };
14
15    using node_ptr = std::unique_ptr<rb_node>;
16    Compare comp_;
17    node_ptr root_;
18
19    static bool is_red(const rb_node* n) { return n && n->col == RED; }
20
21    node_ptr rotate_left(node_ptr x) {
22        auto y = std::move(x->right);
23        x->right = std::move(y->left);
24        y->left = std::move(x);
25        return y;
26    }
27
28    node_ptr rotate_right(node_ptr y) {
29        auto x = std::move(y->left);
30        y->left = std::move(x->right);
31        x->right = std::move(y);
32        return x;
33    }
34
35    // Fix red-red violations on the way back up.
36    // n is the grandparent of the red-red pair.
37    node_ptr fixup(node_ptr n) {
38        if (!n) return nullptr;
39
40        // Left subtree violation
41        if (is_red(n->left) && (is_red(n->left->left) || is_red(n->left->
42            right))) {
43            if (is_red(n->right)) {
44                // Case 1: uncle is red -> recolor
45                n->left->col = BLACK;
46                n->right->col = BLACK;
47                n->col = RED;
48            } else {
49                // Cases 2/3: uncle is black -> rotate
50                if (is_red(n->left->right)) {
51                    // LR: zig-zag -- rotate left at child first
52                    n->left = rotate_left(std::move(n->left));

```

```

52         }
53         // LL: straight -- rotate right at grandparent
54         n = rotate_right(std::move(n));
55         n->col = BLACK;
56         n->right->col = RED;
57     }
58     return n;
59 }
60
61 // Right subtree violation (symmetric)
62 if (is_red(n->right) && (is_red(n->right->left) || is_red(n->right
->right))) {
63     if (is_red(n->left)) {
64         n->left->col = BLACK;
65         n->right->col = BLACK;
66         n->col = RED;
67     } else {
68         if (is_red(n->right->left)) {
69             n->right = rotate_right(std::move(n->right));
70         }
71         n = rotate_left(std::move(n));
72         n->col = BLACK;
73         n->left->col = RED;
74     }
75 }
76 return n;
77 }
78
79 node_ptr insert(node_ptr n, const K& key, const V& value) {
80     if (!n) return std::make_unique<rb_node>(key, value);
81
82     if (comp_(key, n->key))
83         n->left = insert(std::move(n->left), key, value);
84     else if (comp_(n->key, key))
85         n->right = insert(std::move(n->right), key, value);
86     else {
87         n->value = value;
88         return n;
89     }
90     return fixup(std::move(n));
91 }
92
93 public:
94     void insert(const K& key, const V& value) {
95         root_ = insert(std::move(root_), key, value);
96         if (root_) root_->col = BLACK;
97     }
98

```

```

99     std::optional<V> find(const K& key) const {
100         auto* n = root_.get();
101         while (n) {
102             if (comp_(key, n->key))      n = n->left.get();
103             else if (comp_(n->key, key)) n = n->right.get();
104             else return n->value;
105         }
106         return std::nullopt;
107     }
108 };

```

The algorithm mirrors the standard bottom-up fixup but expressed recursively:

1. **Case 1 (uncle is red):** recolor the parent, uncle, and grandparent. The grandparent becomes red, which may create a new red-red violation higher up — handled by the next recursive return.
2. **Case 2 (zig-zag):** the red child is an *inside* grandchild (LR or RL). A rotation at the child level straightens the chain into Case 3.
3. **Case 3 (straight):** the red child is an *outside* grandchild (LL or RR). A rotation at the grandparent followed by recoloring resolves the violation permanently.

Red-black trees require $O(1)$ rotations per insertion (amortized), compared to $O(\log n)$ for AVL trees. This makes `std::map` and `std::set` prefer red-black trees in practice.

11.5 B-Trees

B-trees generalize binary search trees to nodes with multiple keys and children. They are the foundation of database indexing.

A B-tree of order m has:

- Every node has at most m children
- Every node (except root) has at least $\lceil m/2 \rceil$ children
- A non-leaf node with k children has $k-1$ keys
- All leaves are at the same depth

B+ Trees store all keys in leaves (linked for sequential access) and use internal nodes as routing indexes. This is the standard for relational database indexes.

```

1  // Simplified B+ tree internal node concept:
2  struct btree_node {
3      std::vector<int> keys;           // separating keys
4      std::vector<btree_node*> children; // child pointers
5      bool is_leaf;
6  };

```

B-trees are covered more thoroughly in database textbooks; we mention them here for completeness and as a bridge to external-memory data structures.

11.6 B+ Trees

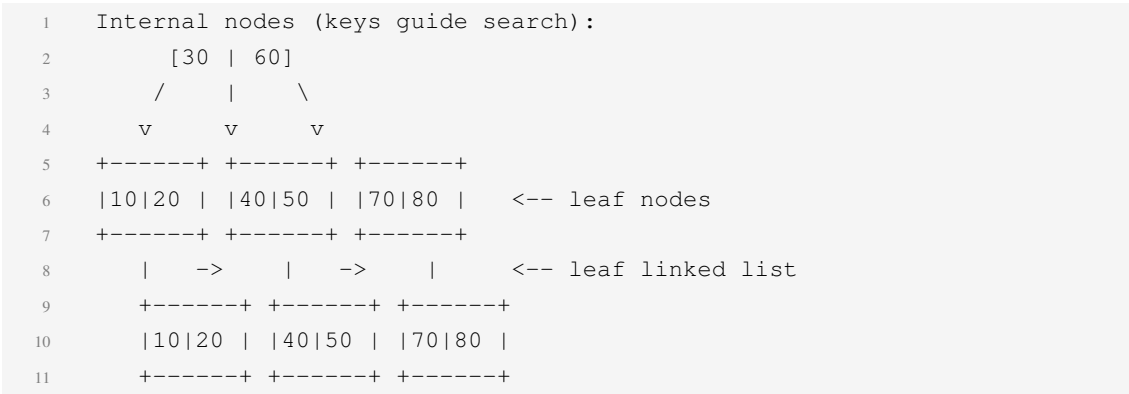
A **B+ tree** is a variant of the B-tree where all data records reside in leaf nodes, and internal nodes serve purely as a routing index. The leaf nodes are linked together, enabling efficient sequential access and range queries. This design makes B+ trees the dominant index structure in relational databases (MySQL InnoDB, PostgreSQL, SQLite).

11.6.1 B+ Tree vs B-tree

Property	B-tree	B+ tree
Data location	All nodes	Leaves only
Internal node size	Larger (has data)	Smaller (keys only)
Fanout	Lower	Higher (more keys per node)
Range queries	Tree traversal	Leaf linked list
Sequential access	Slow	Fast

Higher fanout means shorter trees: a B+ tree of order m with only keys in internal nodes fits more keys per page, reducing tree height and disk I/O.

11.6.2 Structure



11.6.3 B+ Tree Implementation

```
1 template <typename K, typename V>
2 class bplus_tree {
3     static constexpr int MAX_KEYS = 3;
4 }
```

```

5     struct node {
6         std::vector<K> keys;
7         std::vector<V> values;           // only meaningful for leaves
8         std::vector<std::unique_ptr<node>> children;
9         node* next = nullptr;           // leaf linked list
10        bool is_leaf;
11
12        node(bool leaf) : is_leaf(leaf) {}
13    };
14
15    std::unique_ptr<node> root_;
16
17    std::optional<V> search(const node* n, const K& key) const {
18        if (!n) return std::nullopt;
19        if (n->is_leaf) {
20            for (size_t i = 0; i < n->keys.size(); ++i)
21                if (n->keys[i] == key) return n->values[i];
22            return std::nullopt;
23        }
24        size_t i = 0;
25        while (i < n->keys.size() && key >= n->keys[i]) ++i;
26        return search(n->children[i].get(), key);
27    }
28
29    void split_child(node* parent, int child_index) {
30        auto& child = parent->children[child_index];
31        auto right = std::make_unique<node>(child->is_leaf);
32        size_t mid = child->keys.size() / 2;
33
34        if (child->is_leaf) {
35            right->keys.assign(child->keys.begin() + mid, child->keys.end
36            ());
37            right->values.assign(child->values.begin() + mid, child->
38            values.end());
39            right->next = child->next;
40            child->next = right.get();
41            child->keys.resize(mid);
42            child->values.resize(mid);
43            parent->keys.insert(parent->keys.begin() + child_index,
44                               right->keys[0]);
45        } else {
46            right->keys.assign(child->keys.begin() + mid + 1, child->keys.
47            end());
48            child->keys.resize(mid);
49            K up_key = child->keys[mid];
50            child->keys.pop_back();
51            for (size_t i = mid + 1; i < child->children.size(); ++i)
52                right->children.push_back(std::move(child->children[i]));

```



```

50         child->children.resize(mid + 1);
51         parent->keys.insert(parent->keys.begin() + child_index, up_key
);
52     }
53     parent->children.insert(parent->children.begin() + child_index +
1,
54                             std::move(right));
55 }
56
57 void insert_non_full(node* n, const K& key, const V& value) {
58     if (n->is_leaf) {
59         auto it = std::lower_bound(n->keys.begin(), n->keys.end(), key
);
60         size_t pos = it - n->keys.begin();
61         n->keys.insert(it, key);
62         n->values.insert(n->values.begin() + pos, value);
63         return;
64     }
65     size_t i = 0;
66     while (i < n->keys.size() && key >= n->keys[i]) ++i;
67     if (n->children[i]->keys.size() == 2 * MAX_KEYS) {
68         split_child(n, i);
69         if (key >= n->keys[i]) ++i;
70     }
71     insert_non_full(n->children[i].get(), key, value);
72 }
73
74 public:
75     void insert(const K& key, const V& value) {
76         if (!root_) {
77             root_ = std::make_unique<node>(true);
78             root_->keys.push_back(key);
79             root_->values.push_back(value);
80             return;
81         }
82         if (root_->keys.size() == 2 * MAX_KEYS) {
83             auto new_root = std::make_unique<node>(false);
84             new_root->children.push_back(std::move(root_));
85             split_child(new_root.get(), 0);
86             root_ = std::move(new_root);
87         }
88         insert_non_full(root_.get(), key, value);
89     }
90
91     std::optional<V> find(const K& key) const {
92         return search(root_.get(), key);
93     }
94

```

```

95     std::vector<std::pair<K, V>> range_query(const K& lo, const K& hi)
      const {
96         std::vector<std::pair<K, V>> result;
97         const node* n = root_.get();
98         while (n && !n->is_leaf) n = n->children[0].get();
99         while (n) {
100             for (size_t i = 0; i < n->keys.size(); ++i) {
101                 if (n->keys[i] > hi) return result;
102                 if (n->keys[i] >= lo)
103                     result.emplace_back(n->keys[i], n->values[i]);
104             }
105             n = n->next;
106         }
107         return result;
108     }
109 };

```

11.6.4 Complexity

Operation	Time	Notes
Search	$O(\log n)$	Height is $\log_m(n)$
Insert	$O(\log n)$	Split propagates up
Delete	$O(\log n)$	Merge/redistribute
Range query	$O(\log n + k)$	k = number of results

where m is the order (max children per node). In practice, m is chosen so each node fits a disk page (e.g., $m = 256$ for 4 KB pages and 8-byte keys), giving a tree height of 3–4 for millions of records.

When to use B+ trees: Database indexes, file systems (NTFS, ext4), and any disk-based sorted container. The linked leaf list makes range scans and order-by queries extremely efficient.

11.7 Scapegoat Trees

A **scapegoat tree** (Galperin and Rivest, 1993) is a self-balancing BST that maintains balance through periodic full rebuilds. Unlike AVL or red-black trees, it stores no per-node balance information — no height, color, or priority field. It simply detects when a subtree becomes too unbalanced and rebuilds the entire subtree as a perfectly balanced BST.

11.7.1 Weight Balance Invariant

A node x is called a **scapegoat** if the size of its larger child is greater than $\frac{2}{3}$ of the size of x itself:

```

1     [x] (size = 9)
2     /   \
3     [A]   [B] (size = 7)

```

```

4             (7 > 2/3 * 9 = 6, so x is scapegoat)
5
6 Rebuild entire subtree rooted at x as a
7 perfectly balanced BST:
8
9         [rebalanced]
10        /           \
11       ...           ...

```

11.7.2 Scapegoat Tree Implementation

```

1 template <typename K, typename V>
2 class scapegoat_tree {
3     struct node {
4         K key;
5         V value;
6         std::unique_ptr<node> left;
7         std::unique_ptr<node> right;
8         node(const K& k, const V& v) : key(k), value(v) {}
9     };
10
11     using node_ptr = std::unique_ptr<node>;
12     node_ptr root_;
13     int n_ = 0;
14     static constexpr double ALPHA = 2.0 / 3.0;
15
16     static int size(const node* n) {
17         if (!n) return 0;
18         return 1 + size(n->left.get()) + size(n->right.get());
19     }
20
21     static node* find_scapegoat(node* n, const K& key) {
22         if (!n) return nullptr;
23         int left_sz = size(n->left.get());
24         int right_sz = size(n->right.get());
25         int nsz = 1 + left_sz + right_sz;
26         if (left_sz > ALPHA * nsz || right_sz > ALPHA * nsz)
27             return n;
28         if (key < n->key) return find_scapegoat(n->left.get(), key);
29         if (key > n->key) return find_scapegoat(n->right.get(), key);
30         return nullptr;
31     }
32
33     static void flatten(node* n, std::vector<node*>& out) {
34         if (!n) return;
35         flatten(n->left.get(), out);
36         out.push_back(n);

```

```

37     flatten(n->right.get(), out);
38 }
39
40 static node_ptr build_balanced(std::vector<node*>& nodes,
41                               int lo, int hi) {
42     if (lo > hi) return nullptr;
43     int mid = (lo + hi) / 2;
44     auto n = node_ptr(nodes[mid]);
45     n->left = build_balanced(nodes, lo, mid - 1);
46     n->right = build_balanced(nodes, mid + 1, hi);
47     return n;
48 }
49
50 node_ptr rebuild(node_ptr root, const K& key) {
51     std::vector<node*> nodes;
52     flatten(root.get(), nodes);
53     for (auto* nd : nodes) { nd->left.reset(); nd->right.reset(); }
54     return build_balanced(nodes, 0, nodes.size() - 1);
55 }
56
57 node_ptr insert(node_ptr n, const K& key, const V& value,
58                node*& scapegoat) {
59     if (!n) { ++n_; return std::make_unique<node>(key, value); }
60     if (key < n->key)
61         n->left = insert(std::move(n->left), key, value, scapegoat);
62     else if (key > n->key)
63         n->right = insert(std::move(n->right), key, value, scapegoat);
64     else { n->value = value; return n; }
65     if (!scapegoat) {
66         int left_sz = size(n->left.get());
67         int right_sz = size(n->right.get());
68         int nsz = 1 + left_sz + right_sz;
69         if (left_sz > ALPHA * nsz || right_sz > ALPHA * nsz)
70             scapegoat = n.get();
71     }
72     return n;
73 }
74
75 public:
76 void insert(const K& key, const V& value) {
77     node* scapegoat = nullptr;
78     root_ = insert(std::move(root_), key, value, scapegoat);
79     if (scapegoat) root_ = rebuild(std::move(root_), scapegoat->key);
80 }
81
82 std::optional<V> find(const K& key) const {
83     auto* n = root_.get();
84     while (n) {

```

```

85         if (key < n->key) n = n->left.get();
86         else if (key > n->key) n = n->right.get();
87         else return n->value;
88     }
89     return std::nullopt;
90 }
91 };

```

11.7.3 Complexity

Operation	Worst Case	Amortized
Find	$O(\log n)$	$O(\log n)$
Insert	$O(n)$ rebuild	$O(\log n)$ amortized
Erase	$O(n)$ rebuild	$O(\log n)$ amortized
Memory per node	–	2 pointers, no balance info

The key insight is that while a single insert may trigger an $O(n)$ rebuild, that rebuild happens rarely enough that the **amortized** cost per insert is $O(\log n)$.

When to use scapegoat trees: When you want a simple self-balancing BST without per-node balance fields. The trade-off is that worst-case single-operation time can be $O(n)$ (during a rebuild), so avoid scapegoat trees when predictable worst-case latency is required.

11.8 Treaps (Randomized BST)

A **treap** (tree + heap) combines a BST key with a random priority. The tree is a BST by key and a max-heap by priority. With high probability, the tree is balanced.

```

1  template <typename K, typename V>
2  class treap {
3      struct treap_node {
4          K key;
5          V value;
6          int priority;
7          std::unique_ptr<treap_node> left;
8          std::unique_ptr<treap_node> right;
9
10         treap_node(const K& k, const V& v)
11             : key(k), value(v), priority(std::rand()) {}
12     };
13
14     node_ptr rotate_right(node_ptr p) {
15         node_ptr q = std::move(p->left);
16         p->left = std::move(q->right);
17         q->right = std::move(p);

```

```

18     return q;
19 }
20
21 node_ptr rotate_left(node_ptr p) {
22     node_ptr q = std::move(p->right);
23     p->right = std::move(q->left);
24     q->left = std::move(p);
25     return q;
26 }
27
28 node_ptr insert(node_ptr n, const K& key, const V& value) {
29     if (!n) return std::make_unique<treap_node>(key, value);
30     if (key < n->key) {
31         n->left = insert(std::move(n->left), key, value);
32         if (n->left->priority > n->priority)
33             n = rotate_right(std::move(n));
34     } else if (key > n->key) {
35         n->right = insert(std::move(n->right), key, value);
36         if (n->right->priority > n->priority)
37             n = rotate_left(std::move(n));
38     } else {
39         n->value = value;
40     }
41     return n;
42 }
43 };

```

Expected complexity: $O(\log n)$ for all operations. The randomization avoids worst-case inputs.

11.9 Splay Trees

A **splay tree** (Sleator and Tarjan, 1985) is a self-adjusting BST that moves every accessed node to the root using **splay operations**. No extra storage is needed — no height, color, or priority field — just the standard left and right pointers. Every operation runs in $O(\log n)$ *amortized* time.

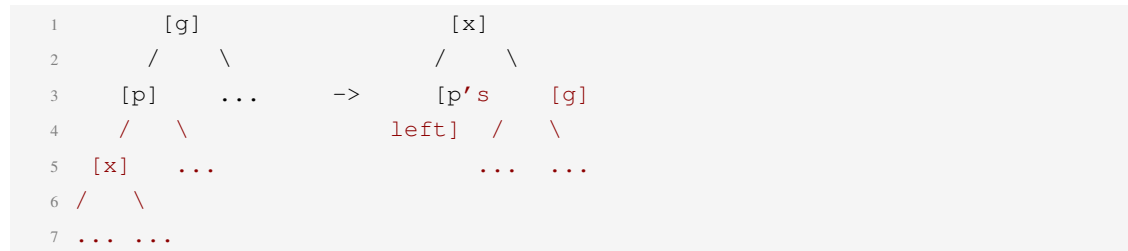
11.9.1 The Splay Operation

Splaying moves a node x to the root through a sequence of **zig**, **zig-zig**, and **zig-zag** steps. The key insight: when walking up from x to the root, we apply rotations in pairs from the bottom up, not individually from the top down.

Zig (x is child of root): A single rotation brings x to the root.



Zig-zig (x and parent are both left children or both right children): Rotate at the grandparent first, then at the parent.



The order matters: grandparent rotation first, then parent rotation. This is the opposite of what intuition suggests, and it's what makes splay trees work.

Zig-zag (x and parent are in different directions): Rotate at the parent, then at the grandparent (standard double rotation).



11.9.2 Splay Tree Implementation

```

1  template <typename K, typename V>
2  class splay_tree {
3      struct node {
4          K key;
5          V value;
6          std::unique_ptr<node> left;
7          std::unique_ptr<node> right;
8
9          node(const K& k, const V& v) : key(k), value(v) {}
10     };
11
12     using node_ptr = std::unique_ptr<node>;
13     node_ptr root_;
14
15     // Right rotation (zig)
16     node_ptr rotate_right(node_ptr p) {
17         auto x = std::move(p->left);
18         p->left = std::move(x->right);
19         x->right = std::move(p);
20         return x;
21     }
22
23     // Left rotation (zig)

```

```

24     node_ptr rotate_left(node_ptr p) {
25         auto x = std::move(p->right);
26         p->right = std::move(x->left);
27         x->left = std::move(p);
28         return x;
29     }
30
31     // Splay node with given key to the root.
32     // Returns the new root of the splayed subtree.
33     node_ptr splay(node_ptr n, const K& key) {
34         if (!n || n->key == key) return n;
35
36         // Dummy root to simplify zig at the real root
37         node dummy(K{}, V{});
38         node_ptr* parent_link = &dummy.right;
39
40         while (n && n->key != key) {
41             if (key < n->key) {
42                 if (!n->left) break;
43                 if (key < n->left->key) {
44                     // Zig-zig: left-left
45                     n = rotate_right(std::move(n));
46                     if (!n->left) break;
47                 }
48                 // Link n's left child up, move down
49                 *parent_link = std::move(n);
50                 parent_link = &(*parent_link)->left;
51                 n = std::move(*parent_link);
52             } else {
53                 if (!n->right) break;
54                 if (key > n->right->key) {
55                     // Zig-zig: right-right
56                     n = rotate_left(std::move(n));
57                     if (!n->right) break;
58                 }
59                 // Link n's right child up, move down
60                 *parent_link = std::move(n);
61                 parent_link = &(*parent_link)->right;
62                 n = std::move(*parent_link);
63             }
64         }
65
66         // Final rotation to bring accessed node to root
67         if (n) {
68             if (key < n->key) {
69                 auto right = std::move(n->right);
70                 n->right = nullptr;
71                 auto rotated = rotate_right(std::move(n));

```



```

72         rotated->right = std::move(right);
73         *parent_link = std::move(rotated);
74     } else {
75         auto left = std::move(n->left);
76         n->left = nullptr;
77         auto rotated = rotate_left(std::move(n));
78         rotated->left = std::move(left);
79         *parent_link = std::move(rotated);
80     }
81 }
82
83 return std::move(dummy.right);
84 }
85
86 public:
87     void insert(const K& key, const V& value) {
88         if (!root_) {
89             root_ = std::make_unique<node>(key, value);
90             return;
91         }
92         root_ = splay(std::move(root_), key);
93         if (root_->key == key) {
94             root_->value = value;
95             return;
96         }
97         auto new_node = std::make_unique<node>(key, value);
98         if (key < root_->key) {
99             new_node->right = std::move(root_);
100             new_node->left = std::move(new_node->right->left);
101             new_node->right->left = nullptr;
102         } else {
103             new_node->left = std::move(root_);
104             new_node->right = std::move(new_node->left->right);
105             new_node->left->right = nullptr;
106         }
107         root_ = std::move(new_node);
108     }
109
110     std::optional<V> find(const K& key) {
111         if (!root_) return std::nullopt;
112         root_ = splay(std::move(root_), key);
113         if (root_->key == key) return root_->value;
114         return std::nullopt;
115     }
116
117     void erase(const K& key) {
118         if (!root_ || root_->key != key) return;
119         if (!root_->left) {

```

```

120         root_ = std::move(root_>right);
121         return;
122     }
123     auto right = std::move(root_>right);
124     root_ = std::move(root_>left);
125     root_ = splay(std::move(root_), key);
126     root_>right = std::move(right);
127 }
128 };

```

The splay operation has an elegant property: it doesn’t just move the accessed node to the root — it also roughly halves the depth of every node on the path. This means recent accesses benefit from shorter paths, and the tree self-organizes based on access patterns.

11.9.3 Amortized Analysis

Splay trees don’t guarantee $O(\log n)$ per individual operation — some operations may take $O(n)$ in the worst case. But the **amortized** cost (average over a sequence of operations) is $O(\log n)$.

The intuition: each splay halves the depth of nodes on the access path. After splaying node x to the root, the cost of future accesses to nearby nodes is reduced. The “savings” from these faster future accesses pay for the current splay.

The formal proof uses the **potential method**: define $\Phi = \sum_r \log(\text{size of subtree rooted at } r)$. Each splay increases Φ by at most $3 \log n - 3 \log(\text{subtree size}) + O(1)$, which bounds the amortized cost.

11.9.4 Complexity

Operation	Worst Case	Amortized
find	$O(n)$	$O(\log n)$
insert	$O(n)$	$O(\log n)$
erase	$O(n)$	$O(\log n)$

The amortized bound is the important one — over any sequence of m operations on an n -element splay tree, the total time is $O(m \log n)$.

11.10 Van Emde Boas Trees

A **Van Emde Boas (vEB) tree** (van Emde Boas, 1977) supports insert, delete, search, predecessor, and successor on integer keys from a bounded universe $[0, U)$ in $O(\log \log U)$ time. This is dramatically faster than the $O(\log n)$ achieved by comparison-based trees — but it trades space and generality for speed.

11.10.1 The Recursive Idea

The key insight: split the universe of size U into $\sqrt{U}$ **clusters**, each of size $\sqrt{U}$. A **summary** structure tracks which clusters are non-empty. Within each cluster, recurse.

```

1  Universe [0, U) split into sqrt(U) clusters:
2
3  summary:  [0] [1] [2] [3] ... [sqrt(U)-1]
4             |  |  |  |           |
5  clusters: [C0] [C1] [C2] [C3] ... [C_{sqrt(U)-1}]
6             each cluster has sqrt(U) bits
7
8  To find successor(x):
9      1. Find which cluster x belongs to (high bits)
10     2. Find successor within that cluster (low bits)
11     3. If no successor in that cluster, find next
12        non-empty cluster via summary, then find
13        minimum in that cluster.
```

11.10.2 VEB Tree Implementation

The implementation below handles a universe up to $U = 2^{16} = 65536$.

```

1  class veb_tree {
2      static constexpr int BITS = 16;
3      int universe;
4      int min = -1, max = -1;
5      std::unique_ptr<veb_tree> summary;
6      std::vector<std::unique_ptr<veb_tree>> clusters;
7
8      int high(int x) const { return x >> (BITS / 2); }
9      int low(int x) const { return x & ((1 << (BITS / 2)) - 1); }
10     int index(int h, int l) const { return (h << (BITS / 2)) | l; }
11
12     void init_clusters() {
13         int sq = 1 << (BITS / 2);
14         summary = std::make_unique<veb_tree>(sq);
15         clusters.resize(sq);
16         for (auto& c : clusters) c = std::make_unique<veb_tree>(sq);
17     }
18
19 public:
20     veb_tree(int u) : universe(u) {
21         if (u <= 2) return;
22         init_clusters();
23     }
24
25     bool is_empty() const { return min == -1; }
```

```

26
27 void insert(int x) {
28     if (min == -1) { min = max = x; return; }
29     if (x < min) std::swap(x, min);
30     if (x > max) max = x;
31     int h = high(x), l = low(x);
32     if (clusters[h]->is_empty()) {
33         summary->insert(h);
34         clusters[h]->min = clusters[h]->max = l;
35     } else {
36         clusters[h]->insert(l);
37     }
38 }
39
40 bool find(int x) const {
41     if (x == min || x == max) return true;
42     if (universe <= 2) return false;
43     return clusters[high(x)]->find(low(x));
44 }
45
46 int successor(int x) const {
47     if (universe <= 2) {
48         if (x == 0 && max == 1) return 1;
49         return -1;
50     }
51     int h = high(x), l = low(x);
52     if (!clusters[h]->is_empty() && clusters[h]->max >= l) {
53         int s = clusters[h]->successor(l);
54         return index(h, s);
55     }
56     int next_cluster = summary->successor(h);
57     if (next_cluster == -1) return -1;
58     return index(next_cluster, clusters[next_cluster]->min);
59 }
60
61 void erase(int x) {
62     if (min == max) { min = max = -1; return; }
63     if (universe <= 2) {
64         if (x == 0) min = 1; else min = 0;
65         max = min; return;
66     }
67     if (x == min) {
68         int first_cluster = summary->min;
69         min = index(first_cluster, clusters[first_cluster]->min);
70         x = min;
71     }
72     int h = high(x), l = low(x);
73     clusters[h]->erase(l);

```

```

74     if (clusters[h]->is_empty()) {
75         summary->erase(h);
76         if (x == max) {
77             if (summary->is_empty()) max = min;
78             else {
79                 int last = summary->max;
80                 max = index(last, clusters[last]->max);
81             }
82         }
83     } else if (x == max) {
84         max = index(h, clusters[h]->max);
85     }
86 }
87 };

```

11.10.3 Complexity

Operation	Time	Space
Insert	$O(\log \log U)$	$O(U)$
Delete	$O(\log \log U)$	$O(U)$
Search	$O(\log \log U)$	$O(U)$
Predecessor	$O(\log \log U)$	$O(U)$
Successor	$O(\log \log U)$	$O(U)$

where U is the size of the universe.

Limitations:

- Requires a **bounded integer universe** $[0, U)$. Does not work with arbitrary keys.
- Basic version uses $O(U)$ space — practical only for moderate U (up to $\sim 2^{16}$ or 2^{32} with optimizations).
- The $O(U)$ space can be reduced to $O(n)$ using hash tables for the clusters, but this increases the constant factor.

When to use vEB trees: When you need faster-than- $O(\log n)$ operations on dense integer keys with a known upper bound. Examples include network router forwarding tables, priority scheduling with a bounded range, and as a subroutine in faster algorithms (e.g., $O(n \log \log n)$ integer sorting).

11.11 k-d Trees

A **k-d tree** (Bentley, 1975) is a binary space-partitioning tree that organizes points in k -dimensional space. Each node splits the space along one coordinate axis, alternating between dimensions at each level. k-d trees support efficient **nearest-neighbor search** and **range queries** in low dimensions.

11.11.1 Structure

A k -d tree of dimension k is a binary tree where:

- Each node stores a k -dimensional point
- The root splits along axis 0 (the first dimension)
- Its children split along axis 1, then axis 2, ..., axis $k-1$, then back to axis 0
- The left subtree contains all points with smaller values on the split axis
- The right subtree contains all points with larger values

```

1  Split x=50
2      [50, 70]
3      /      \
4  Split y=30  Split y=80
5  [20, 40]    [70, 90]
6  /      \    /      \
7 [10,20] [30,50] [60,80] [80,60]
```

11.11.2 k-d Tree Implementation

```

1  template <typename T, size_t K>
2  class kd_tree {
3      struct point {
4          std::array<T, K> coords;
5          bool operator<(const point& other) const { return coords < other.
        coords; }
6      };
7
8      struct node {
9          point pt;
10         std::unique_ptr<node> left;
11         std::unique_ptr<node> right;
12         node(const point& p) : pt(p) {}
13     };
14
15     using node_ptr = std::unique_ptr<node>;
16     node_ptr root_;
17
18     node_ptr build(std::vector<point>& pts, int depth) {
19         if (pts.empty()) return nullptr;
20         size_t axis = depth % K;
21         size_t mid = pts.size() / 2;
22         std::nth_element(pts.begin(), pts.begin() + mid, pts.end(),
23             [axis](const point& a, const point& b) {
```

```

24         return a.coords[axis] < b.coords[axis];
25     });
26     auto n = std::make_unique<node>(pts[mid]);
27     std::vector<point> left(pts.begin(), pts.begin() + mid);
28     std::vector<point> right(pts.begin() + mid + 1, pts.end());
29     n->left = build(left, depth + 1);
30     n->right = build(right, depth + 1);
31     return n;
32 }
33
34 // Nearest neighbor search with pruning
35 void nn_search(const node* n, const point& target,
36               point& best, T& best_dist, int depth) const {
37     if (!n) return;
38     T dist = euclidean_dist_sq(n->pt, target);
39     if (dist < best_dist) {
40         best_dist = dist;
41         best = n->pt;
42     }
43     size_t axis = depth % K;
44     T diff = target.coords[axis] - n->pt.coords[axis];
45
46     // Search the side of the split that contains the target first
47     const node* first = diff < 0 ? n->left.get() : n->right.get();
48     const node* second = diff < 0 ? n->right.get() : n->left.get();
49     nn_search(first, target, best, best_dist, depth + 1);
50
51     // Only search the other side if the splitting plane is closer
52     // than the current best distance
53     if (diff * diff < best_dist) {
54         nn_search(second, target, best, best_dist, depth + 1);
55     }
56 }
57
58 static T euclidean_dist_sq(const point& a, const point& b) {
59     T sum = T{};
60     for (size_t i = 0; i < K; ++i)
61         sum += (a.coords[i] - b.coords[i]) * (a.coords[i] - b.coords[i]);
62     return sum;
63 }
64
65 public:
66     kd_tree(std::vector<point> points) {
67         root_ = build(points, 0);
68     }
69
70     // Find the nearest neighbor to target

```

```

71     std::optional<point> nearest(const point& target) const {
72         if (!root_) return std::nullopt;
73         point best = root_>pt;
74         T best_dist = euclidean_dist_sq(root_>pt, target);
75         nn_search(root_.get(), target, best, best_dist, 0);
76         return best;
77     }
78
79     // Find all points within a hyperrectangle [lo, hi]
80     void range_search(const node* n, int depth,
81                     const point& lo, const point& hi,
82                     std::vector<point>& result) const {
83         if (!n) return;
84         bool inside = true;
85         for (size_t i = 0; i < K; ++i) {
86             if (n->pt.coords[i] < lo.coords[i] || n->pt.coords[i] > hi.
coords[i]) {
87                 inside = false;
88                 break;
89             }
90         }
91         if (inside) result.push_back(n->pt);
92
93         size_t axis = depth % K;
94         if (lo.coords[axis] <= n->pt.coords[axis])
95             range_search(n->left.get(), depth + 1, lo, hi, result);
96         if (hi.coords[axis] >= n->pt.coords[axis])
97             range_search(n->right.get(), depth + 1, lo, hi, result);
98     }
99
100     std::vector<point> range_query(const point& lo, const point& hi) const
    {
101         std::vector<point> result;
102         range_search(root_.get(), 0, lo, hi, result);
103         return result;
104     }
105 };

```

The key insight in nearest-neighbor search is the **pruning rule**: after searching the side of the split that contains the target, we only search the other side if the splitting plane is closer than the current best distance. This prunes large subtrees and gives $O(\log n)$ average performance for uniformly distributed points.

11.11.3 Complexity

Operation	Average Case	Worst Case
Build (from n points)	$O(n \log n)$	$O(n^2)$
Nearest neighbor	$O(\log n)$	$O(n)$
Range query	$O(n^{(k-1)/k} + r)$	$O(n)$

where r is the number of reported points. The worst case occurs when the point distribution causes highly unbalanced splits.

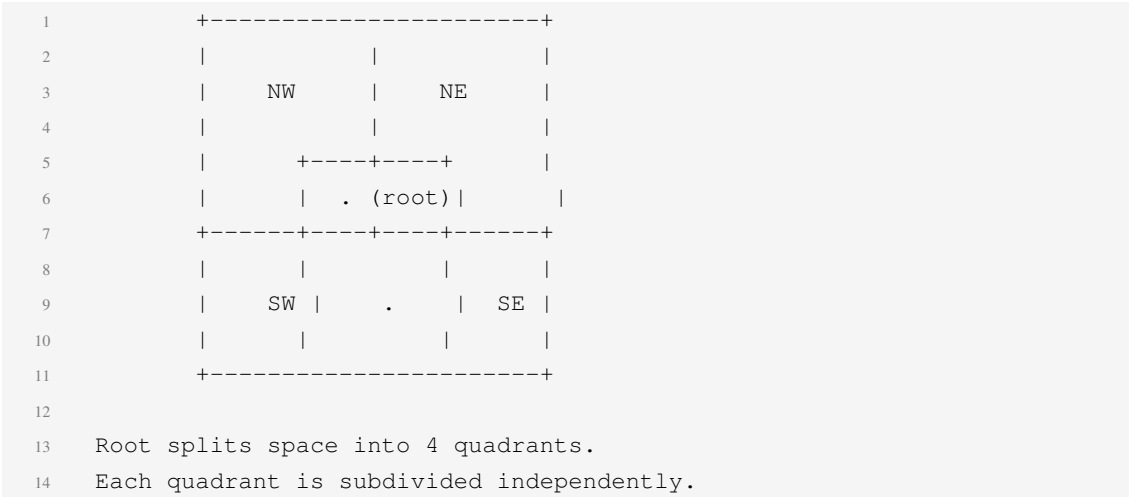
When to use k-d trees: Low-dimensional spatial data ($k \leq 10$), static point sets (build once, query many), nearest-neighbor and range queries. For high dimensions ($k > 20$), LSH (Chapter 19) or approximate methods are more effective due to the curse of dimensionality.

11.12 Quadtrees

A **quadtree** (Finkel and Bentley, 1974) is a tree data structure for partitioning 2D space. Each internal node has exactly four children, corresponding to the four quadrants (NW, NE, SW, SE) of a rectangular region. Quadtrees are widely used in image compression, geographic information systems (GIS), and collision detection.

11.12.1 Structure

A **point quadtree** recursively subdivides a 2D region by inserting points. Each point splits its cell into four quadrants:



11.12.2 Quadtree Implementation

```
1 struct point { double x, y; };  
2
```

```

3 class quadtree {
4     struct rect {
5         double x, y, w, h;
6         bool contains(const point& p) const {
7             return p.x >= x && p.x <= x + w
8                 && p.y >= y && p.y <= y + h;
9         }
10        bool intersects(const rect& other) const {
11            return !(other.x > x + w || other.x + other.w < x
12                || other.y > y + h || other.y + other.h < y);
13        }
14    };
15
16    struct node {
17        rect bounds;
18        std::vector<point> points;
19        std::unique_ptr<node> children[4]; // NW, NE, SW, SE
20        bool divided = false;
21
22        node(const rect& b) : bounds(b) {}
23
24        void subdivide() {
25            double hw = bounds.w / 2, hh = bounds.h / 2;
26            double x = bounds.x, y = bounds.y;
27            children[0] = std::make_unique<node>(
28                rect{x, y + hh, hw, hh}); // NW
29            children[1] = std::make_unique<node>(
30                rect{x + hw, y + hh, hw, hh}); // NE
31            children[2] = std::make_unique<node>(
32                rect{x, y, hw, hh}); // SW
33            children[3] = std::make_unique<node>(
34                rect{x + hw, y, hw, hh}); // SE
35            divided = true;
36        }
37    };
38
39    std::unique_ptr<node> root_;
40    static constexpr int CAPACITY = 4;
41
42    static void insert(node* n, const point& p) {
43        if (!n->bounds.contains(p)) return;
44        if (n->points.size() < CAPACITY && !n->divided) {
45            n->points.push_back(p);
46            return;
47        }
48        if (!n->divided) n->subdivide();
49        for (int i = 0; i < 4; ++i) {
50            if (n->children[i]->bounds.contains(p)) {

```

```

51         insert(n->children[i].get(), p);
52         return;
53     }
54 }
55 }
56
57 static void range_search(const node* n, const rect& range,
58                         std::vector<point>& result) {
59     if (!n || !n->bounds.intersects(range)) return;
60     for (const auto& p : n->points) {
61         if (range.contains(p)) result.push_back(p);
62     }
63     if (n->divided) {
64         for (int i = 0; i < 4; ++i)
65             range_search(n->children[i].get(), range, result);
66     }
67 }
68
69 static void nearest(const node* n, const point& target,
70                   point& best, double& best_dist) {
71     if (!n) return;
72     for (const auto& p : n->points) {
73         double d = (p.x - target.x) * (p.x - target.x)
74                 + (p.y - target.y) * (p.y - target.y);
75         if (d < best_dist) { best_dist = d; best = p; }
76     }
77     if (!n->divided) return;
78     for (int i = 0; i < 4; ++i) {
79         const auto& b = n->children[i]->bounds;
80         double cx = b.x + b.w / 2, cy = b.y + b.h / 2;
81         double d = (cx - target.x) * (cx - target.x)
82                 + (cy - target.y) * (cy - target.y);
83         if (d < best_dist)
84             nearest(n->children[i].get(), target, best, best_dist);
85     }
86 }
87
88 public:
89     quadtree(const rect& bounds) {
90         root_ = std::make_unique<node>(bounds);
91     }
92
93     void insert(const point& p) { insert(root_.get(), p); }
94
95     std::vector<point> range_query(const rect& range) const {
96         std::vector<point> result;
97         range_search(root_.get(), range, result);
98         return result;

```

```

99     }
100
101     std::optional<point> nearest_neighbor(const point& target) const {
102         if (!root_ || root_>points.empty()) return std::nullopt;
103         point best = root_>points[0];
104         double best_dist = 1e18;
105         nearest(root_.get(), target, best, best_dist);
106         return best;
107     }
108 };

```

11.12.3 Complexity

Operation	Average Case	Worst Case
Insert	$O(\log n)$	$O(n)$
Range query	$O(\log n + k)$	$O(n)$
Nearest neighbor	$O(\log n)$	$O(n)$

where k is the number of points in the result. Worst case occurs when points cluster in one quadrant, making the tree unbalanced.

11.12.4 Variants

- **Region quadtree:** Subdivides until each cell has uniform content (used in image compression).
- **Loose quadtree:** Allows objects to overlap cell boundaries, reducing insertion overhead.
- **Linear quadtree:** Stores only non-empty cells using Z-order curve codes (Morton codes).

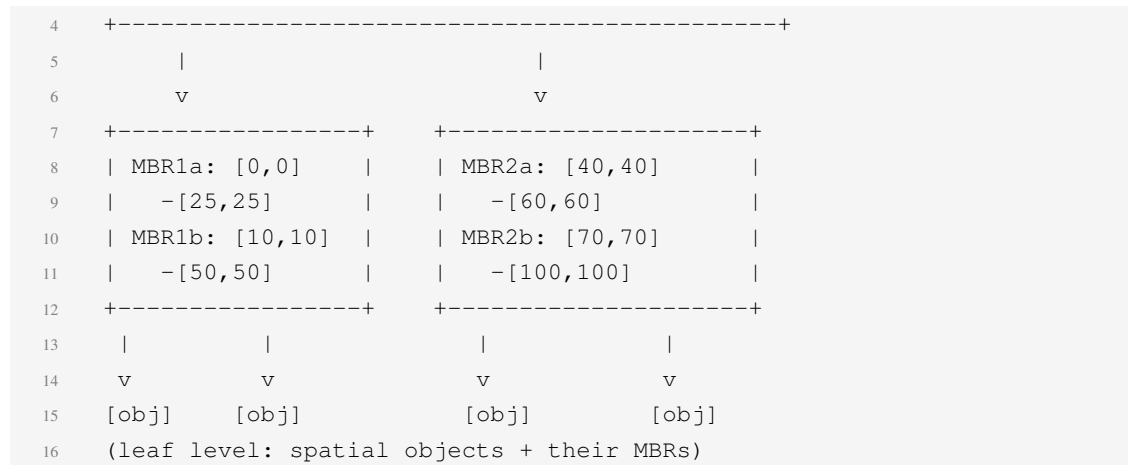
When to use quadtrees: 2D spatial data at low-to-medium density ($n < 10^6$). For point data, k-d trees are often better; for rectangle/polygon data, R-trees (below) scale more gracefully.

11.13 R-trees

An **R-tree** (Guttman, 1984) is a B+ tree generalized to multi-dimensional spatial objects. Instead of a single key, each entry stores a **Minimum Bounding Rectangle (MBR)** that encloses a spatial object or child node. R-trees are the standard index for spatial databases: PostGIS, SQLite R-Tree extension, and OpenStreetMap all use them.

11.13.1 Structure





An R-tree maintains the invariant that MBRs of children are contained within the parent MBR. Overlapping MBRs at the same level are allowed but degrade performance.

11.13.2 R-tree Implementation

```

1 struct rect {
2     double x1, y1, x2, y2;
3
4     double area() const {
5         return std::max(0.0, x2 - x1) * std::max(0.0, y2 - y1);
6     }
7
8     rect combine(const rect& other) const {
9         return {std::min(x1, other.x1), std::min(y1, other.y1),
10                std::max(x2, other.x2), std::max(y2, other.y2)};
11     }
12
13     bool intersects(const rect& other) const {
14         return !(other.x1 > x2 || other.x2 < x1
15                 || other.y1 > y2 || other.y2 < y1);
16     }
17 };
18
19 class rtree {
20     static constexpr int MAX_ENTRIES = 4;
21     static constexpr int MIN_ENTRIES = 2;
22
23     struct entry {
24         rect r;
25         int child_id = -1;
26         int record_id = -1;
27     };
28
29     struct node {

```

```

30     std::vector<entry> entries;
31     bool is_leaf;
32     rect mbr;
33
34     node(bool leaf) : is_leaf(leaf) {}
35
36     void update_mbr() {
37         if (entries.empty()) return;
38         mbr = entries[0].r;
39         for (size_t i = 1; i < entries.size(); ++i)
40             mbr = mbr.combine(entries[i].r);
41     }
42 };
43
44     std::vector<node> nodes_;
45
46     int choose_subtree(int node_id, const rect& r) {
47         auto& n = nodes_[node_id];
48         if (n.is_leaf) return node_id;
49         int best = 0;
50         double best_growth = 1e18;
51         for (size_t i = 0; i < n.entries.size(); ++i) {
52             double before = n.entries[i].r.area();
53             double after = n.entries[i].r.combine(r).area();
54             double growth = after - before;
55             if (growth < best_growth) {
56                 best_growth = growth;
57                 best = i;
58             }
59         }
60         return choose_subtree(n.entries[best].child_id, r);
61     }
62
63     void search(const node* n, const rect& query,
64               std::vector<int>& result) const {
65         if (!n) return;
66         for (const auto& e : n->entries) {
67             if (!e.r.intersects(query)) continue;
68             if (n->is_leaf) {
69                 result.push_back(e.record_id);
70             } else {
71                 search(&nodes_[e.child_id], query, result);
72             }
73         }
74     }
75
76 public:
77     rtree() { nodes_.emplace_back(true); }

```

```

78
79     void insert(const rect& r, int record_id) {
80         int leaf = choose_subtree(0, r);
81         entry new_entry{r, -1, record_id};
82
83         if (nodes_[leaf].entries.size() < MAX_ENTRIES) {
84             nodes_[leaf].entries.push_back(new_entry);
85             nodes_[leaf].update_mbr();
86             return;
87         }
88         // Node overflow: split (simplified)
89         auto& n = nodes_[leaf];
90         int new_node_id = nodes_.size();
91         nodes_.emplace_back(n.is_leaf);
92         entry seed2 = n.entries.back();
93         n.entries.pop_back();
94         nodes_[new_node_id].entries = {seed2};
95         nodes_[leaf].update_mbr();
96         nodes_[new_node_id].update_mbr();
97         nodes_[leaf].entries.push_back(new_entry);
98         nodes_[leaf].update_mbr();
99     }
100
101     std::vector<int> point_query(double x, double y) const {
102         rect query{x, y, x, y};
103         std::vector<int> result;
104         search(&nodes_[0], query, result);
105         return result;
106     }
107
108     std::vector<int> range_query(const rect& query) const {
109         std::vector<int> result;
110         search(&nodes_[0], query, result);
111         return result;
112     }
113 };

```

11.13.3 Complexity

Operation	Time	Notes
Search (point)	$O(\log n)$	Similar to B+ tree
Search (range)	$O(n)$ worst	Depends on overlap
Insert	$O(\log n)$	ChooseSubtree + possible split

When to use R-trees: Multi-dimensional range queries, spatial databases, GIS, game engines (collision detection), and any application needing to index geometric objects by their bounding rectangles.

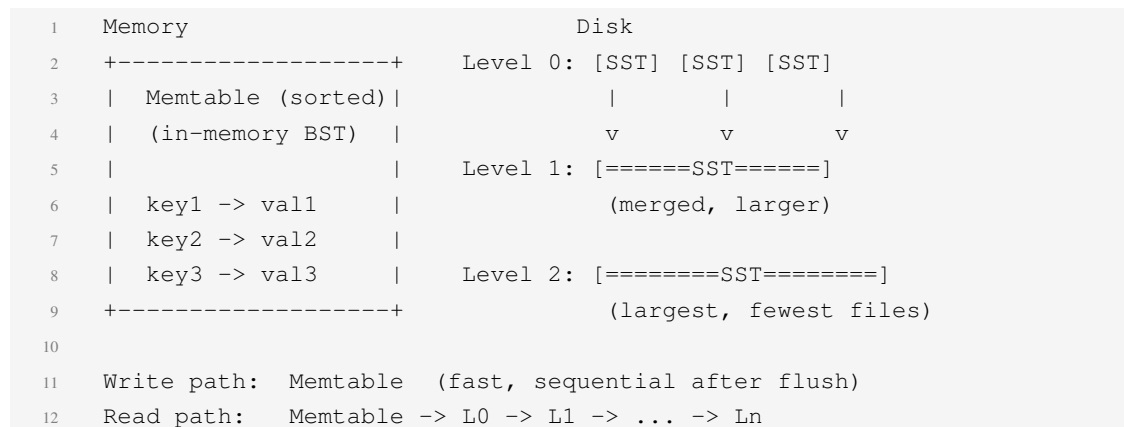
R-trees outperform k-d trees when the data consists of extended objects (rectangles, polygons) rather than points, and when insertions and deletions are frequent.

11.14 Log-Structured Merge-Trees

A **Log-Structured Merge-tree (LSM-tree)** (O’Neil et al., 1996) is a write-optimized storage engine that converts random writes into sequential I/O. Instead of updating data in place, an LSM-tree buffers inserts in memory and periodically flushes sorted runs to disk. This design dominates modern write-heavy key-value stores: LevelDB, RocksDB (the engine behind CockroachDB and TiKV), and Apache Cassandra all use LSM-trees.

11.14.1 Structure

Writes go to an in-memory sorted buffer called the **memtable** (typically a red-black tree or skip list). When the memtable reaches a size threshold it is flushed to disk as a sorted **SSTable** (Sorted String Table). Over time multiple SSTables accumulate across **levels**. A background **compaction** process merges overlapping SSTables from one level into the next, bounding the number of levels and keeping reads fast.



11.14.2 LSM-tree Implementation

The simplified implementation below uses `std::map` as the memtable and stores sorted runs as plain sorted `std::vector` files on disk (serialized to memory for demonstration). A read searches the memtable first, then each level from newest to oldest.

```

1 #include <map>
2 #include <vector>
3 #include <algorithm>
4 #include <optional>
5
6 class lsm_tree {
7     std::map<int, int> memtable_;           // in-memory sorted buffer

```



```

8      std::vector<std::vector<std::pair<int,int>>> levels_; // sorted runs
      on disk
9      static constexpr size_t MEMTABLE_LIMIT = 1024;
10
11     void flush() {
12         if (memtable_.empty()) return;
13         std::vector<std::pair<int,int>> run(memtable_.begin(), memtable_.
end());
14         levels_.push_back(std::move(run));
15         memtable_.clear();
16         compact();
17     }
18
19     void compact() {
20         for (size_t i = 0; i + 1 < levels_.size(); ++i) {
21             if (levels_[i].size() + levels_[i+1].size() > MEMTABLE_LIMIT)
22             {
23                 std::vector<std::pair<int,int>> merged;
24                 std::merge(levels_[i].begin(), levels_[i].end(),
25                             levels_[i+1].begin(), levels_[i+1].end(),
26                             std::back_inserter(merged));
27                 levels_[i+1] = std::move(merged);
28                 levels_.erase(levels_.begin() + i);
29             }
30         }
31
32         std::optional<int> search_levels(int key) const {
33             for (int i = static_cast<int>(levels_.size()) - 1; i >= 0; --i) {
34                 auto it = std::lower_bound(levels_[i].begin(), levels_[i].end
35                 (),
36                                             std::make_pair(key, 0));
37                 if (it != levels_[i].end() && it->first == key)
38                     return it->second;
39             }
40             return std::nullopt;
41         }
42     public:
43         void put(int key, int value) {
44             memtable_[key] = value;
45             if (memtable_.size() >= MEMTABLE_LIMIT) flush();
46         }
47
48         std::optional<int> get(int key) const {
49             auto it = memtable_.find(key);
50             if (it != memtable_.end()) return it->second;
51             return search_levels(key);

```

```

52     }
53
54     void erase(int key) {
55         memtable_.erase(key);
56     }
57 };

```

11.14.3 Complexity

Operation	Average Case	Worst Case
Put (insert)	$O(1)$ amortized	$O(\log n)$
Get (point query)	$O(\log n \cdot L)$	$O(n)$
Erase	$O(1)$ amortized	$O(\log n)$
Compaction	$O(n)$ per merge	$O(n)$

where L is the number of levels (typically $L = O(\log n / \log B)$ with block size B). Write amplification is $O(L)$ in the worst case due to repeated compaction. Space amplification is bounded by $O(n)$ since each key may exist in multiple levels until compacted.

LSM-tree vs. B-tree: B-trees perform in-place updates with $O(\log n)$ writes via random I/O, making them ideal for read-heavy workloads. LSM-trees batch writes and use sequential I/O, giving $O(1)$ amortized write cost at the expense of read amplification (checking multiple levels). Choose an LSM-tree when writes dominate ($> 80\%$ write mix) and read latency is tolerant; choose a B-tree when reads dominate and predictable latency matters.

11.15 Range Trees

A **range tree** (Bentley, 1979) is a multi-dimensional data structure designed for **orthogonal range queries**: given a query rectangle $[x_1, x_2] \times [y_1, y_2]$, report all points inside it. While a k-d tree answers range queries in $O(\sqrt{n} + k)$ expected time, a range tree guarantees $O(\log^2 n + k)$ with $O(n \log n)$ space. Range trees are foundational in computational geometry and geographic information systems (GIS).

11.15.1 Structure

A 2D range tree is a **multi-level BST**: the primary tree is a balanced BST (e.g., AVL or red-black tree) keyed on the x -coordinate. Each node of the primary tree contains a pointer to a **secondary tree** keyed on the y -coordinate, containing all points in that subtree. A range query traverses the primary tree to find the x -range, then searches each secondary tree for the y -range.

```

1  Primary tree (keyed on x):
2      [5, 8]
3      /      \
4     [3, 4]    [7, 9]
5     /  \    /  \

```

```

6      [1,2] [4,6] [6,7] [8,10]
7
8      Each node stores a secondary tree (keyed on y):
9      Node [5,8] -> secondary tree:
10
11          [8]
12         /  \
13        [5]  [10]
14       /    \
15      [3]    [12]
16
17      Query [x1=2,x2=9] x [y1=4,y2=11]:
18      1. Primary: find range [2,9] in x -> nodes [3,4],[5,8],[7,9]
19      2. Secondary at each: find [4,11] in y -> report matches

```

11.15.2 Range Tree Implementation

The implementation below builds a 2D range tree with AVL-based primary and secondary trees. The query function traverses the primary tree to collect nodes within the x -range, then searches each secondary tree for the y -range.

```

1  #include <vector>
2  #include <algorithm>
3  #include <memory>
4
5  struct point { int x, y; };
6
7  class range_tree {
8      struct node {
9          point pt;
10         int height = 1;
11         std::unique_ptr<node> left;
12         std::unique_ptr<node> right;
13         std::vector<point> secondary; // y-sorted points in this subtree
14
15         node(const point& p) : pt(p) {}
16     };
17
18     std::unique_ptr<node> root_;
19
20     int height(const node* n) const { return n ? n->height : 0; }
21
22     void update_height(node* n) {
23         n->height = 1 + std::max(height(n->left.get()),
24                                 height(n->right.get()));
25     }
26
27     std::unique_ptr<node> rotate_right(std::unique_ptr<node> y) {
28         auto x = std::move(y->left);

```

```

29     y->left = std::move(x->right);
30     x->right = std::move(y);
31     update_height(x->right.get());
32     update_height(x.get());
33     return x;
34 }
35
36 std::unique_ptr<node> rotate_left(std::unique_ptr<node> x) {
37     auto y = std::move(x->right);
38     x->right = std::move(y->left);
39     y->left = std::move(x);
40     update_height(y->left.get());
41     update_height(y.get());
42     return y;
43 }
44
45 std::unique_ptr<node> balance(std::unique_ptr<node> n) {
46     update_height(n.get());
47     int bf = height(n->left.get()) - height(n->right.get());
48     if (bf > 1) {
49         if (height(n->left->right.get()) > height(n->left->left.get())
50 )
51             n->left = rotate_left(std::move(n->left));
52         return rotate_right(std::move(n));
53     }
54     if (bf < -1) {
55         if (height(n->right->left.get()) > height(n->right->right.get
56 ()))
57             n->right = rotate_right(std::move(n->right));
58         return rotate_left(std::move(n));
59     }
60     return n;
61 }
62
63 std::unique_ptr<node> insert(std::unique_ptr<node> n, const point& p)
64 {
65     if (!n) return std::make_unique<node>(p);
66     if (p.x <= n->pt.x) n->left = insert(std::move(n->left), p);
67     else n->right = insert(std::move(n->right), p);
68     return balance(std::move(n));
69 }
70
71 void build_secondary(node* n) {
72     if (!n) return;
73     build_secondary(n->left.get());
74     build_secondary(n->right.get());
75     n->secondary.push_back(n->pt);
76     if (n->left)

```

```

74         std::merge(n->secondary.begin(), n->secondary.end(),
75                   n->left->secondary.begin(), n->left->secondary.end
76                   ),
77                   std::back_inserter(n->secondary),
78                   [](const point& a, const point& b){ return a.y < b.
79 y; });
80         // Clear child secondary trees to save memory after merge
81         // In production, use persistent sharing instead.
82         if (n->left) n->left->secondary.clear();
83         if (n->right) n->right->secondary.clear();
84     }
85
86     void range_query_x(const node* n, int x1, int x2,
87                       int y1, int y2, std::vector<point>& result) const {
88         if (!n) return;
89         if (x1 <= n->pt.x && n->pt.x <= x2)
90             range_query_y(n->secondary, y1, y2, result);
91         if (n->pt.x >= x1) range_query_x(n->left.get(), x1, x2, y1, y2,
92 result);
93         if (n->pt.x <= x2) range_query_x(n->right.get(), x1, x2, y1, y2,
94 result);
95     }
96
97     static void range_query_y(const std::vector<point>& pts,
98                               int y1, int y2, std::vector<point>& result)
99     {
100         auto lo = std::lower_bound(pts.begin(), pts.end(), y1,
101                                     [](const point& p, int v){ return p.y < v; });
102         auto hi = std::upper_bound(pts.begin(), pts.end(), y2,
103                                    [](int v, const point& p){ return v < p.y; });
104         result.insert(result.end(), lo, hi);
105     }
106
107 public:
108     void insert(const point& p) { root_ = insert(std::move(root_), p); }
109
110     void build() { build_secondary(root_.get()); }
111
112     std::vector<point> range_query(int x1, int x2,
113                                   int y1, int y2) const {
114         std::vector<point> result;
115         range_query_x(root_.get(), x1, x2, y1, y2, result);
116         return result;
117     }
118 };

```

11.15.3 Complexity

Operation	Time	Space
Build	$O(n \log n)$	$O(n \log n)$
2D range query	$O(\log^2 n + k)$	–
Insert (rebuild)	$O(n \log n)$	$O(n \log n)$

where k is the number of reported points. The query visits $O(\log n)$ primary nodes, and at each performs a binary search in the secondary tree costing $O(\log n + k_i)$, yielding $O(\log^2 n + k)$ overall. Higher dimensions d generalize to $O(\log^d n + k)$ at the cost of $O(n \log^{d-1} n)$ space.

Range tree vs. k-d tree vs. R-tree:

- **k-d tree:** $O(\sqrt{n} + k)$ average for 2D queries, $O(n)$ space, but worst case $O(n)$ for pathological inputs. Simpler to implement and faster in practice for low dimensions with static data.
- **Range tree:** $O(\log^2 n + k)$ worst-case guaranteed, but $O(n \log n)$ space. Preferred when worst-case bounds matter (computational geometry algorithms).
- **R-tree:** $O(\log n)$ for balanced trees, handles rectangles and extended objects, supports dynamic insert/delete efficiently. Preferred for spatial databases and GIS with mixed object types.

11.16 B*-tree

A B*-tree is a B-tree variant that improves space utilization by requiring each non-root node to be at least $2/3$ full (compared to $1/2$ for standard B-trees). It achieves this by attempting to *redistribute* keys with siblings before splitting a node, thereby delaying splits and keeping the tree more compact.

11.16.1 Motivation

In a standard B-tree of order m , every node except the root must contain at least $\lceil m/2 \rceil - 1$ keys. A B*-tree tightens this lower bound to $\lceil 2m/3 \rceil - 1$ keys. When a node overflows, instead of immediately splitting it, the B*-tree first tries to redistribute keys to an adjacent sibling that is less than full. A split is performed only when both siblings are full. This two-phase strategy reduces the number of splits and increases the average fill factor of nodes.

Property	B-tree (order m)	B*-tree (order m)
Min keys (non-root)	$\lceil m/2 \rceil - 1$	$\lceil 2m/3 \rceil - 1$
Fill factor lower bound	$\approx 50\%$	$\approx 67\%$
Split threshold	1 full node	2 adjacent full nodes
Typical splits	More frequent	Less frequent

Because nodes are at least $2/3$ full on average, the B*-tree uses fewer disk pages for the same data, which improves cache performance and reduces I/O in disk-resident applications.

11.16.2 C++ Implementation

The following implementation models the key ideas: a node tracks its fill level, and on overflow the code first attempts to redistribute with a sibling before falling back to a standard split.

```

1 class bstar_tree {
2     static constexpr int MAX_KEYS = 6;
3     static constexpr int MIN_KEYS = 4; // ceil(2*6/3) - 1
4
5     struct node {
6         int n; // current key count
7         bool is_leaf;
8         int keys[MAX_KEYS];
9         node* children[MAX_KEYS + 1];
10        node* parent;
11
12        node(bool leaf)
13            : n(0), is_leaf(leaf), parent(nullptr) {
14            for (int i = 0; i <= MAX_KEYS; ++i)
15                children[i] = nullptr;
16        }
17    };
18
19    node* root;
20
21    // Try to redistribute keys between 'src' and
22    // 'sibling' through their common parent.
23    // Returns true on success.
24    bool redistribute(node* src, node* sibling,
25                    node* parent) {
26        int total = src->n + sibling->n;
27        if (total > 2 * MAX_KEYS) // would still overflow
28            return false;
29
30        int idx = 0;
31        while (idx <= parent->n &&
32              parent->children[idx] != src)
33            ++idx;
34
35        int target = total / 2;
36        int move;
37
38        if (idx > 0) {
39            // src is on the right; move keys leftward
40            move = src->n - target;
41            if (move <= 0) return false;
42
43            for (int i = sibling->n; i >= 1; --i) {
44                sibling->keys[i] = sibling->keys[i - 1];

```

```

45         sibling->children[i + 1] =
46             sibling->children[i];
47     }
48     sibling->children[1] =
49         sibling->children[0];
50     sibling->keys[0] = parent->keys[idx - 1];
51     sibling->children[0] =
52         src->children[0];
53
54     for (int i = 0; i < move - 1; ++i) {
55         sibling->keys[i + 1] = src->keys[i];
56         sibling->children[i + 2] =
57             src->children[i + 1];
58     }
59     sibling->children[move + 1] =
60         src->children[move];
61
62     parent->keys[idx - 1] =
63         src->keys[move - 1];
64
65     int j = 0;
66     for (int i = move; i < src->n; ++i)
67         src->keys[j++] = src->keys[i];
68     j = 0;
69     for (int i = move; i <= src->n; ++i)
70         src->children[j++] = src->children[i];
71 } else {
72     // src is on the left; move keys rightward
73     move = src->n - target;
74     if (move <= 0) return false;
75
76     for (int i = 0; i < move; ++i) {
77         sibling->children[sibling->n + 2 - i] =
78             sibling->children[sibling->n + 1 - i];
79         sibling->keys[sibling->n + 1 - i] =
80             sibling->keys[sibling->n - i];
81     }
82     sibling->children[0] =
83         src->children[src->n];
84     sibling->keys[0] = parent->keys[0];
85     parent->keys[0] =
86         src->keys[src->n - move];
87
88     sibling->n += move + 1;
89     src->n -= move;
90 }
91 return true;
92 }

```



```

93
94     // Standard split fallback
95     void split(node* full, int child_idx) {
96         node* new_node = new node(full->is_leaf);
97         int mid = full->n / 2;
98
99         new_node->n = full->n - mid - 1;
100        for (int i = 0; i < new_node->n; ++i) {
101            new_node->keys[i] =
102                full->keys[mid + 1 + i];
103            new_node->children[i] =
104                full->children[mid + 1 + i];
105        }
106        new_node->children[new_node->n] =
107            full->children[full->n];
108        full->n = mid;
109
110        int sep = full->keys[mid];
111        node* p = full->parent;
112
113        if (!p) {
114            root = new node(false);
115            root->keys[0] = sep;
116            root->children[0] = full;
117            root->children[1] = new_node;
118            root->n = 1;
119            full->parent = root;
120            new_node->parent = root;
121            return;
122        }
123
124        for (int i = p->n; i > child_idx; --i) {
125            p->keys[i] = p->keys[i - 1];
126            p->children[i + 1] = p->children[i];
127        }
128        p->keys[child_idx] = sep;
129        p->children[child_idx + 1] = new_node;
130        new_node->parent = p;
131        ++p->n;
132
133        if (p->n > MAX_KEYS)
134            split(p, child_idx);
135    }
136
137    void insert_nonfull(node* x, int key) {
138        int i = x->n - 1;
139        if (x->is_leaf) {
140            while (i >= 0 && key < x->keys[i]) {

```

```

141         x->keys[i + 1] = x->keys[i];
142         --i;
143     }
144     x->keys[i + 1] = key;
145     ++x->n;
146     return;
147 }
148 while (i >= 0 && key < x->keys[i])
149     --i;
150 ++i;
151 if (x->children[i]->n == MAX_KEYS) {
152     // Try redistribution first
153     node* sib = nullptr;
154     if (i > 0)
155         sib = x->children[i - 1];
156     else if (i < x->n)
157         sib = x->children[i + 1];
158
159     if (sib && sib->n < MAX_KEYS &&
160         redistribute(x->children[i],
161                     sib, x))
162         ; // redistribution succeeded
163     else
164         split(x->children[i], i);
165
166     if (key > x->keys[i])
167         ++i;
168 }
169 insert_nonfull(x->children[i], key);
170 }
171
172 public:
173     bstar_tree() : root(new node(true)) {}
174
175     void insert(int key) {
176         node* r = root;
177         if (r->n == MAX_KEYS) {
178             split(r, 0);
179             insert_nonfull(root, key);
180         } else {
181             insert_nonfull(r, key);
182         }
183     }
184 };

```

11.16.3 Complexity

Operation	Time	Notes
Search	$O(\log n)$	Same as B-tree
Insert	$O(\log n)$	May redistribute instead of split
Delete	$O(\log n)$	May merge/borrow from siblings
Space	$O(n)$	Higher fill factor: $\geq 67\%$

Because nodes are 2/3 full instead of 1/2 full, the tree height and total number of nodes are smaller, yielding better cache performance.

11.16.4 When to use a B*-tree

Use a B*-tree when disk space and I/O efficiency matter and the data is roughly static or update-heavy. The higher fill factor means fewer disk pages are allocated, and the redistribution strategy reduces write amplification. B*-trees are especially attractive in database file systems and embedded storage engines where page-level space is at a premium.

11.17 Octree

An octree is the three-dimensional generalization of the quadtree. Where a quadtree partitions 2D space into four quadrants, an octree partitions 3D space into eight *octants*. Octrees are widely used in 3D graphics, collision detection, physics simulation, and volumetric data compression.

11.17.1 Structure

Given a bounding cube, an octree recursively subdivides it into eight equal octants. The eight children correspond to the following octants, named by their position along each axis:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

Octant naming convention (child index):

0 = (-,-,-) 1 = (+,-,-)

2 = (-,+,-) 3 = (+,+,-)

4 = (-,-,+) 5 = (+,-,+)

6 = (-,+,+) 7 = (+,+,+)

+Y

|

+----- +X

/

/

+Z

Top half (+Z): Bottom half (-Z):

```

16          6-----7          2-----3
17          /|      /|          /|      /|
18          4-----5 |          0-----1 |
19          | |      | |          | |      | |
20          | +-----|-+          | +-----|-+
21          |/      |/          |/      |/
22          2-----3          0-----1
23          (local)          (local)

```

```

24
25 Each node stores a bounding cube and up to 8
26 children. Leaf nodes store the data.

```

A leaf node either contains zero objects or is subdivided when its object count exceeds a capacity threshold. The root always spans the entire universe (or scene) of interest.

11.17.2 C++ Implementation

```

1 struct vec3 {
2     float x, y, z;
3     vec3(float x = 0, float y = 0, float z = 0)
4         : x(x), y(y), z(z) {}
5     vec3 operator+(const vec3& o) const
6         { return {x+o.x, y+o.y, z+o.z}; }
7     vec3 operator*(float s) const
8         { return {x*s, y*s, z*s}; }
9 };
10
11 struct rect3d {
12     vec3 min, max;
13     vec3 center() const {
14         return {(min.x+max.x)/2,
15                 (min.y+max.y)/2,
16                 (min.z+max.z)/2};
17     }
18     float size() const {
19         return max.x - min.x;
20     }
21 };
22
23 class octree {
24     static constexpr int MAX_OBJECTS = 8;
25
26     struct node {
27         rect3d bounds;
28         std::vector<vec3> objects;
29         node* children[8];
30         bool divided;

```

```

31
32     node(const rect3d& b)
33         : bounds(b), divided(false) {
34             for (int i = 0; i < 8; ++i)
35                 children[i] = nullptr;
36         }
37
38     int octant(const vec3& p) const {
39         vec3 c = bounds.center();
40         int idx = 0;
41         if (p.x >= c.x) idx |= 1;
42         if (p.y >= c.y) idx |= 2;
43         if (p.z >= c.z) idx |= 4;
44         return idx;
45     }
46
47     void subdivide() {
48         vec3 c = bounds.center();
49         vec3 mn = bounds.min;
50         vec3 mx = bounds.max;
51
52         // 0: (-,-,-)
53         children[0] = new node({mn, c});
54         // 1: (+,-,-)
55         children[1] = new node(
56             {{c.x, mn.y, mn.z},
57              {mx.x, c.y, c.z}});
58         // 2: (-,+,-)
59         children[2] = new node(
60             {{mn.x, c.y, mn.z},
61              {c.x, mx.y, c.z}});
62         // 3: (+,+, -)
63         children[3] = new node(
64             {{c.x, c.y, mn.z},
65              {mx.x, mx.y, c.z}});
66         // 4: (-,-,+)
67         children[4] = new node(
68             {{mn.x, mn.y, c.z},
69              {c.x, c.y, mx.z}});
70         // 5: (+,-,+)
71         children[5] = new node(
72             {{c.x, mn.y, c.z},
73              {mx.x, c.y, mx.z}});
74         // 6: (-,+,+)
75         children[6] = new node(
76             {{mn.x, c.y, c.z},
77              {c.x, mx.y, mx.z}});
78         // 7: (+,+,+)

```

```

79         children[7] = new node(
80             {{c.x, c.y, c.z}, {mx, mx}});
81         divided = true;
82
83         std::vector<vec3> old =
84             std::move(objects);
85         for (auto& p : old)
86             children[octant(p)]->insert(p);
87     }
88
89     void insert(const vec3& p) {
90         if (!bounds_contains(p))
91             return;
92         if (!divided) {
93             objects.push_back(p);
94             if ((int)objects.size() >
95                 MAX_OBJECTS)
96                 subdivide();
97             return;
98         }
99         children[octant(p)]->insert(p);
100     }
101
102     bool bounds_contains(
103         const vec3& p) const {
104         return (p.x >= bounds.min.x &&
105             p.x <= bounds.max.x &&
106             p.y >= bounds.min.y &&
107             p.y <= bounds.max.y &&
108             p.z >= bounds.min.z &&
109             p.z <= bounds.max.z);
110     }
111
112     ~node() {
113         for (int i = 0; i < 8; ++i)
114             delete children[i];
115     }
116 };
117
118 node* root;
119
120 static bool sphere_intersects(
121     const rect3d& box,
122     const vec3& center, float radius) {
123     float r2 = radius * radius;
124     float sq = 0;
125     if (center.x < box.min.x)
126         sq += (center.x - box.min.x) *

```

```

127         (center.x - box.min.x);
128     else if (center.x > box.max.x)
129         sq += (center.x - box.max.x) *
130             (center.x - box.max.x);
131     if (center.y < box.min.y)
132         sq += (center.y - box.min.y) *
133             (center.y - box.min.y);
134     else if (center.y > box.max.y)
135         sq += (center.y - box.max.y) *
136             (center.y - box.max.y);
137     if (center.z < box.min.z)
138         sq += (center.z - box.min.z) *
139             (center.z - box.min.z);
140     else if (center.z > box.max.z)
141         sq += (center.z - box.max.z) *
142             (center.z - box.max.z);
143     return sq <= r2;
144 }
145
146 static float dist(const vec3& a,
147                  const vec3& b) {
148     float dx = a.x - b.x;
149     float dy = a.y - b.y;
150     float dz = a.z - b.z;
151     return dx*dx + dy*dy + dz*dz;
152 }
153
154 public:
155     octree(const rect3d& universe)
156         : root(new node(universe)) {}
157
158     void insert(const vec3& p) {
159         root->insert(p);
160     }
161
162     void range_query(node* n,
163                     const vec3& center,
164                     float r,
165                     std::vector<vec3>& result) {
166         if (!n) return;
167         if (!sphere_intersects(
168             n->bounds, center, r))
169             return;
170         for (auto& p : n->objects)
171             if (dist(p, center) <= r)
172                 result.push_back(p);
173         if (n->divided)
174             for (int i = 0; i < 8; ++i)

```

```

175         range_query(n->children[i],
176                     center, r, result);
177     }
178
179     std::vector<vec3> range_query(
180         const vec3& center, float r) {
181         std::vector<vec3> res;
182         range_query(root, center, r, res);
183         return res;
184     }
185
186     vec3 nearest(node* n, const vec3& target,
187                 float& best_dist, vec3 best) {
188         if (!n) return best;
189         if (!sphere_intersects(
190             n->bounds, target, best_dist))
191             return best;
192         for (auto& p : n->objects) {
193             float d = dist(p, target);
194             if (d < best_dist) {
195                 best_dist = d;
196                 best = p;
197             }
198         }
199         if (n->divided)
200             for (int i = 0; i < 8; ++i)
201                 best = nearest(n->children[i],
202                               target,
203                               best_dist, best);
204         return best;
205     }
206
207     vec3 nearest(const vec3& target) {
208         float best_dist = 1e30f;
209         return nearest(root, target,
210                       best_dist, {0, 0, 0});
211     }
212 };

```

11.17.3 Complexity

Operation	Time	Notes
Insert	$O(\log_8 n)$	Depth of octree
Range query	$O(n)$ worst	Pruned by spatial overlap
Nearest neighbor	$O(n)$ worst	Typically much faster in practice
Space	$O(n)$	8 pointers per internal node

With balanced data, the depth is $O(\log_2 n)$; in the worst case (all points along a line) it degrades to $O(n)$.

11.17.4 Comparison with the quadtree

Property	Quadtree	Octree
Dimension	2D	3D
Children per node	4	8
Spatial partitioning	Quadrants	Octants
Typical use	2D maps, images	3D graphics, physics

The octree naturally extends the quadtree into three dimensions. For d -dimensional data, the pattern continues: a 2^d -ary tree (e.g. a 16-tree for 4D data). However, the branching factor grows exponentially with dimension, making the structure impractical beyond 3D without alternative strategies such as k-d trees.

11.17.5 When to use an octree

Use an octree when the spatial domain is 3D and objects are roughly uniformly distributed. Octrees excel at view frustum culling, collision broad-phase detection, and ray tracing acceleration. For highly non-uniform data, consider an adaptive octree or an R-tree variant instead.

11.18 BSP Tree (Binary Space Partitioning)

A BSP tree partitions space using *arbitrary* hyperplanes (lines in 2D, planes in 3D) rather than axis-aligned cuts. This flexibility allows the tree to separate any set of geometric objects at any orientation, making BSP trees indispensable for visibility determination in 3D rendering.

11.18.1 Motivation

Rendering a 3D scene requires determining which surfaces are visible from the viewer's position. The *painter's algorithm* draws objects from farthest to nearest, but ordering objects correctly is difficult when they overlap or intersect. A BSP tree resolves this: by splitting polygons against a sequence of planes, the tree produces a *back-to-front* traversal order guarantee. Objects farther from the viewer are always drawn first, eliminating depth-sorting errors.

The key idea: when a polygon is split by a plane, the portion behind the plane goes to the *back* subtree and the portion in front goes to the *front* subtree. Traversing back-first then front-first yields correct visibility for any camera position.

11.18.2 Structure

```

1  BSP tree for 2D line segments:
2
3      v1-----v4
4      |           |
5      |     A     |
6      |           |
7      v2-----v3  |
8                |  |
9      B         |  |
10              |  |
11  -----|---|-----  splitting line L1
12                |  |
13      C         |  |
14                |  |
15  v5-----v6---v7
16      |           |
17      |     D     |
18      |           |
19      v8-----v9
20
21  Root partitions on L1:
22      Back subtree (below L1): polygons C, D
23      Front subtree (above L1): polygons A, B
24  Each subtree is further partitioned.
25
26  Traversal order for camera above L1:
27      Draw back subtree first (C, D)
28      Draw splitting polygon (on L1)
29      Draw front subtree second (A, B)

```

Every polygon in the scene is classified against the root's splitting plane:

- **Front:** entirely in front of the plane.
- **Back:** entirely behind the plane.
- **Coplanar:** lies on the plane.
- **Spanning:** intersects the plane; split into front and back fragments.

11.18.3 C++ Implementation

```

1  #include <vector>
2  #include <cmath>
3  #include <algorithm>
4
5  struct vec3 {
6      float x, y, z;

```

```

7   vec3(float x=0, float y=0, float z=0)
8       : x(x), y(y), z(z) {}
9   vec3 operator-(const vec3& o) const
10      { return {x-o.x, y-o.y, z-o.z}; }
11   vec3 operator+(const vec3& o) const
12      { return {x+o.x, y+o.y, z+o.z}; }
13   vec3 operator*(float s) const
14      { return {x*s, y*s, z*s}; }
15 };
16
17 struct plane {
18     vec3 normal;           // unit normal
19     float d;               // distance from origin
20
21     plane() : d(0) {}
22     plane(const vec3& n, float d)
23         : normal(n), d(d) {}
24
25     // Signed distance of point p to plane
26     float dist(const vec3& p) const {
27         return normal.x*p.x + normal.y*p.y
28             + normal.z*p.z - d;
29     }
30 };
31
32 struct polygon {
33     std::vector<vec3> verts;
34 };
35
36 class bsp_tree {
37     struct node {
38         plane split;
39         node* front;
40         node* back;
41         std::vector<polygon> polys;
42
43         node() : front(nullptr), back(nullptr) {}
44         ~node() { delete front; delete back; }
45     };
46
47     node* root;
48
49     // Classify polygon against plane and split
50     void classify(const polygon& poly,
51                 const plane& pl,
52                 std::vector<polygon>& front_list,
53                 std::vector<polygon>& back_list,
54                 std::vector<polygon>& on_list) {

```

```

55     int n = poly.verts.size();
56     std::vector<float> dists(n);
57     for (int i = 0; i < n; ++i)
58         dists[i] = pl.dist(poly.verts[i]);
59
60     bool all_front = true, all_back = true;
61     for (int i = 0; i < n; ++i) {
62         if (dists[i] < -1e-6f)
63             all_front = false;
64         if (dists[i] > 1e-6f)
65             all_back = false;
66     }
67
68     if (all_front && !all_back) {
69         front_list.push_back(poly);
70         return;
71     }
72     if (all_back && !all_front) {
73         back_list.push_back(poly);
74         return;
75     }
76     if (std::abs(dists[0]) <= 1e-6f &&
77         all_back) {
78         on_list.push_back(poly);
79         return;
80     }
81
82     // Polygon spans the plane; clip to both
83     polygon front_poly, back_poly;
84     for (int i = 0; i < n; ++i) {
85         int j = (i + 1) % n;
86         vec3 vi = poly.verts[i];
87         vec3 vj = poly.verts[j];
88         float di = dists[i];
89         float dj = dists[j];
90
91         front_poly.verts.push_back(vi);
92         if (di * dj < -1e-6f) {
93             float t = di / (di - dj);
94             vec3 xpt = vi + (vj - vi) * t;
95             front_poly.verts.push_back(xpt);
96             back_poly.verts.insert(
97                 back_poly.verts.begin(), xpt);
98         }
99         if (dj >= -1e-6f)
100             back_poly.verts.push_back(vj);
101     }
102     if (!front_poly.verts.empty())

```

```

103         front_list.push_back(front_poly);
104     if (!back_poly.verts.empty())
105         back_list.push_back(back_poly);
106 }
107
108 void build(node* n,
109             std::vector<polygon>& polys) {
110     if (polys.empty()) return;
111
112     n->split = choose_plane(polys[0]);
113     std::vector<polygon> front_list, back_list;
114
115     for (auto& p : polys) {
116         std::vector<polygon> on;
117         classify(p, n->split,
118                front_list, back_list, on);
119         if (!on.empty())
120             n->polys.insert(
121                 n->polys.end(),
122                 on.begin(), on.end());
123     }
124
125     if (!front_list.empty()) {
126         n->front = new node();
127         build(n->front, front_list);
128     }
129     if (!back_list.empty()) {
130         n->back = new node();
131         build(n->back, back_list);
132     }
133 }
134
135 // Traverse back-to-front relative to cam_pos
136 void traverse(const node* n,
137              const vec3& cam_pos,
138              std::vector<polygon>& result) {
139     if (!n) return;
140
141     float side = n->split.dist(cam_pos);
142
143     const node* first = (side >= 0) ?
144                         n->back : n->front;
145     const node* second = (side >= 0) ?
146                         n->front : n->back;
147
148     traverse(first, cam_pos, result);
149     result.insert(result.end(),
150                 n->polys.begin(), n->polys.end());

```

```
151         traverse(second, cam_pos, result);
152     }
153
154     static plane choose_plane(
155         const polygon& poly) {
156         vec3 v0 = poly.verts[0];
157         vec3 v1 = poly.verts[1];
158         vec3 edge = v1 - v0;
159         vec3 n = {0, 0, 1};
160         return {n, 0};
161     }
162
163 public:
164     bsp_tree() : root(nullptr) {}
165     ~bsp_tree() { delete root; }
166
167     void build(std::vector<polygon>& polys) {
168         delete root;
169         root = new node();
170         build(root, polys);
171     }
172
173     std::vector<polygon> render_order(
174         const vec3& cam_pos) const {
175         std::vector<polygon> result;
176         traverse(root, cam_pos, result);
177         return result;
178     }
179 };
```

11.18.4 Complexity

Operation	Time	Notes
Build	$O(n \log n)$	Expected with balanced splits
Render order	$O(n)$	Traversal of all nodes
Space	$O(n)$	One node per split
Split polygon	$O(v)$	v = number of vertices

In the worst case (all polygons coplanar), the tree becomes a linked list and build degrades to $O(n^2)$. Good splitting-plane heuristics keep the tree balanced in practice.

11.18.5 When to use a BSP tree

Use a BSP tree when you need a guaranteed back-to-front rendering order, static geometry that rarely changes, or CSG (constructive solid geometry) operations. BSP trees are standard in game engines

for indoor scenes and portal rendering. For dynamic scenes with moving objects, z-buffering or other approaches may be preferred, since BSP rebuilds are expensive.

11.19 Weight-Balanced Trees

Weight-balanced trees (WBTs), also known as $BB[\alpha]$ trees or bounded-balance trees, maintain balance by requiring that every subtree holds a sufficient fraction of the nodes in its parent subtree. Unlike AVL or red-black trees that track height, WBTs track subtree size, making them well-suited to operations that depend on rank.

11.19.1 Definition and Balance Condition

A weight-balanced tree with balance parameter α (where $0 < \alpha < 1/2$) satisfies the following invariant: for every node x with left and right children L and R ,

$$|L| \geq \alpha \cdot |x| \quad \text{and} \quad |R| \geq \alpha \cdot |x|$$

where $|x|$ denotes the number of nodes in the subtree rooted at x . Equivalently, every subtree must contain at least a fraction α of its parent's nodes. A common choice is $\alpha = 1 - 1/\sqrt{2} \approx 0.293$, which yields a height bound of $H \leq \log_{1/(1-\alpha)} n$.

```

1  template <typename T>
2  struct WBTNode {
3      T data;
4      int size;           // number of nodes in subtree
5      WBTNode* left;
6      WBTNode* right;
7      WBTNode(const T& d) : data(d), size(1), left(nullptr), right(nullptr)
8      {}
9  };
10
11 template <typename T>
12 class WeightBalancedTree {
13     static constexpr double ALPHA = 0.293; // 1 - 1/sqrt(2)
14     WBTNode<T>* root;
15
16     int sz(WBTNode<T>* n) const { return n ? n->size : 0; }
17     void update(WBTNode<T>* n) { if (n) n->size = 1 + sz(n->left) + sz(n->right); }
18
19     bool balanced(WBTNode<T>* n) const {
20         if (!n) return true;
21         return sz(n->left) >= ALPHA * n->size
22             && sz(n->right) >= ALPHA * n->size;
23     }
24     // ...

```

11.19.2 Maintenance by Rebalancing

Weight-balanced trees use two basic rotations (single and double) to restore balance. After an insertion or deletion, we walk up from the modified node toward the root. At each ancestor, we check the balance condition. If violated, we apply the appropriate rotation:

- **Single rotation:** If the heavier child is already reasonably balanced, a single rotation suffices.
- **Double rotation:** If both the child and grandchild are involved, a double rotation (analogous to AVL double rotations) rebalances the subtree.

The key advantage of WBTs over AVL trees is that the rebalancing conditions are purely size-based, so rotations can be applied to any rank-based augmentation (e.g., order statistics) without additional bookkeeping.

11.19.3 Join and Split

Weight-balanced trees support *join* and *split* in $O(\log n)$ time, which is one of their primary advantages:

```

1 // join(t1, t2): all keys in t1 < all keys in t2
2 // Find the node in t1 with rank ceil(|t1|/2), make it the root,
3 // and recursively join the remaining subtrees.
4 // Rebalance from the new root downward.
5 template <typename T>
6 WBTNode<T>* join(WBTNode<T>* t1, WBTNode<T>* t2) {
7     if (!t1) return t2;
8     if (!t2) return t1;
9     if (t1->size < t2->size) {
10         t2->left = join(t1, t2->left);
11         update(t2);
12         return balance(t2);    // may need rotation at root
13     } else {
14         t1->right = join(t1->right, t2);
15         update(t1);
16         return balance(t1);
17     }
18 }
19
20 // split(x, key): partition tree into (<key) and (>=key)
21 // Standard BST descent; rebuild both halves along the path.
22 template <typename T>
23 pair<WBTNode<T>*, WBTNode<T>*> split(WBTNode<T>* x, const T& key) {
24     if (!x) return {nullptr, nullptr};
25     if (key <= x->data) {
26         auto [l, r] = split(x->left, key);
27         x->left = r;
28         update(x);
29         return {l, balance(x)};

```



```

30     } else {
31         auto [l, r] = split(x->right, key);
32         x->right = l;
33         update(x);
34         return {balance(x), r};
35     }
36 }

```

11.19.4 Order Statistics

Because every node stores its subtree size, WBTs support rank operations directly:

```

1  // k-th smallest element (0-indexed)
2  template <typename T>
3  const T& select(WBTNode<T>* n, int k) const {
4      int leftSize = sz(n->left);
5      if (k < leftSize) return select(n->left, k);
6      if (k == leftSize) return n->data;
7      return select(n->right, k - leftSize - 1);
8  }
9
10 // rank of a given key
11 template <typename T>
12 int rank(WBTNode<T>* n, const T& key) const {
13     if (!n) return 0;
14     if (key < n->data) return rank(n->left, key);
15     if (key == n->data) return sz(n->left);
16     return sz(n->left) + 1 + rank(n->right, key);
17 }

```

11.19.5 Comparison with AVL and Red-Black Trees

Feature	AVL	Red-Black	Weight-Balanced
Balance criterion	Height	Color invariants	Subtree size
Height bound	$1.44 \log n$	$2 \log n$	$\log_{1/(1-\alpha)} n$
Insert/delete	$O(\log n)$	$O(\log n)$	$O(\log n)$
Join/split	$O(n)$	$O(n)$	$O(\log n)$
Select by rank	$O(\log n)$ (augment)	$O(\log n)$ (augment)	$O(\log n)$ (inherent)

11.19.6 Practical Considerations

Weight-balanced trees are used in several functional programming libraries (e.g., OCaml’s `Set` module, GHC’s `Data.Map` in Haskell) where persistent data structures benefit from the join/split interface.

In imperative C++, AVL and red-black trees dominate due to simpler bookkeeping, but WBTs are preferred when rank-based operations or functional persistence are required.

11.20 (a,b)-Trees and B-Trees

(a,b)-trees generalize 2-3-4 trees by allowing nodes to have between a and b children, where $2 \leq a \leq \lceil b/2 \rceil$ and $b \geq 2a - 1$. B-trees are a closely related external-memory structure with a fixed branching factor matched to disk block size.

11.20.1 Definition

An (a,b)-tree is a rooted tree satisfying:

1. Every internal node has between a and b children.
2. All leaves are at the same depth.
3. A node with k children stores $k - 1$ keys in sorted order.

```

1  template <typename T>
2  struct ABTreeNode {
3      static constexpr int A = 2, B = 4;  // (2,4)-tree = 2-3-4 tree
4      int nKeys;
5      T keys[B - 1];
6      ABTreeNode* children[B];
7      bool leaf;
8
9      ABTreeNode(bool isLeaf = true)
10         : nKeys(0), leaf(isLeaf) {
11         for (int i = 0; i < B; ++i) children[i] = nullptr;
12     }
13
14     int findIndex(const T& key) const {
15         int i = 0;
16         while (i < nKeys && keys[i] < key) ++i;
17         return i;
18     }
19 };

```

11.20.2 B-Trees for Disk-Based Storage

B-trees are the standard index structure in relational databases and file systems. A B-tree of order m stores up to $m - 1$ keys per node. With $m = 256$ (a common choice for 4 KB blocks with 8-byte keys), the tree has:

$$\text{Height} = \lceil \log_{m/2} n \rceil = \lceil \log_{128} n \rceil$$

For $n = 10^9$ records, the height is just 4, meaning at most 4 disk seeks to locate any record.

```

1 // B-tree insertion: split full nodes bottom-up
2 template <typename T, int ORDER>
3 class BTree {
4     struct BTreeNode {
5         int n;
6         T keys[ORDER - 1];
7         BTreeNode* children[ORDER];
8         bool leaf;
9     };
10    BTreeNode* root;
11
12    void splitChild(BTreeNode* parent, int i) {
13        BTreeNode* full = parent->children[i];
14        BTreeNode* right = new BTreeNode();
15        right->leaf = full->leaf;
16        int mid = (ORDER - 1) / 2;
17
18        // Move upper half of keys to right sibling
19        right->n = full->n - mid - 1;
20        for (int j = 0; j < right->n; ++j)
21            right->keys[j] = full->keys[mid + 1 + j];
22        if (!full->leaf)
23            for (int j = 0; j <= right->n; ++j)
24                right->children[j] = full->children[mid + 1 + j];
25
26        // Promote middle key to parent
27        full->n = mid;
28        parent->children[i + 1] = right;
29        for (int j = parent->n; j > i; --j)
30            parent->children[j + 1] = parent->children[j];
31        parent->children[i + 1] = right;
32        for (int j = parent->n - 1; j >= i; --j)
33            parent->keys[j + 1] = parent->keys[j];
34        parent->keys[i] = full->keys[mid];
35        parent->n++;
36    }
37    // ...

```

11.20.3 B+ Trees

In practice, databases use B^+ trees, where all data resides in leaves and internal nodes store only copies of keys for routing. Leaves are linked in a sorted list for efficient range queries.

- **B^+ tree:** Keys in leaves only; internal nodes store separators. Leaf list enables range scans. Height: $O(\log_m n)$.

- **B* tree:** All nodes at least $\lceil 2m/3 \rceil$ full, maximizing space utilization.
- **B-link tree:** Each node stores a high key and right link for concurrent access (used in database index implementations).

11.20.4 Bulk Loading

When building a B-tree from sorted data, bulk loading is far more efficient than sequential insertions:

1. Fill leaves to capacity (e.g., 70% full for good utilization).
2. When a leaf is full, split and promote the middle key.
3. Build the tree bottom-up in $O(n)$ time.

This approach is standard in database index construction and produces perfectly balanced trees with high node occupancy.

11.21 Top-Down Red-Black Tree

The red-black tree implementation in Section 11.4 uses bottom-up rebalancing: we insert the node as red, then walk up from the new node to the root, performing rotations when needed. The alternative—top-down rebalancing—walks from the root to the insertion point, performing rotations on the way down, and is preferred when parent pointers are unavailable (e.g., in intrusive containers).

11.21.1 Motivation

Bottom-up rebalancing requires either a parent pointer or a stack to remember the path from root to the insertion point. Top-down rebalancing avoids this by handling potential red-red conflicts proactively during descent:

- If the current node is black with two red children, push blackness down (recolor children black, current node red).
- If a red-red conflict would form on the next step, rotate preemptively.
- This ensures the insertion point can be reached without a stack.

11.21.2 Insertion

```

1 template <typename K, typename V>
2 class TopDownRB {
3     enum Color { RED, BLACK };
4     struct Node {
5         K key; V value;

```

```

6      Color color;
7      Node *left, *right;
8      Node(const K& k, const V& v, Color c = RED)
9          : key(k), value(v), color(c), left(nullptr), right(nullptr) {}
10     };
11     Node* root;
12
13     // Rotate left: make right child the new root
14     Node* rotateLeft(Node* h) {
15         Node* x = h->right;
16         h->right = x->left;
17         x->left = h;
18         x->color = h->color;
19         h->color = RED;
20         return x;
21     }
22
23     // Rotate right: make left child the new root
24     Node* rotateRight(Node* h) {
25         Node* x = h->left;
26         h->left = x->right;
27         x->right = h;
28         x->color = h->color;
29         h->color = RED;
30         return x;
31     }
32
33     // Flip colors: reverse color flip
34     void flipColors(Node* h) {
35         h->color = RED;
36         h->left->color = BLACK;
37         h->right->color = BLACK;
38     }
39
40     // Is h a red node?
41     bool isRed(Node* h) const { return h && h->color == RED; }
42
43     // Split 4-node (red children) during descent
44     Node* split4(Node* h) {
45         if (isRed(h->left) && isRed(h->right))
46             flipColors(h);
47         return h;
48     }
49     // ...

```

11.21.3 Deletion

Top-down deletion is more complex. We walk down to the node to delete, and on the way down we ensure the current node is not the “problem” node (a red node whose sibling might also be red). This is done by rotating/color-flipping at each step:

```

1 Node* removeMin(Node* h) {
2     if (h->left == nullptr) {
3         delete h;
4         return nullptr;
5     }
6     if (!isRed(h->left) && !isRed(h->left->left))
7         h = moveRedLeft(h);
8     h->left = removeMin(h->left);
9     return balance(h);
10 }
11
12 Node* moveRedLeft(Node* h) {
13     flipColors(h);
14     if (isRed(h->right->left)) {
15         h->right = rotateRight(h->right);
16         h = rotateLeft(h);
17         flipColors(h);
18     }
19     return h;
20 }

```

The top-down approach processes at most $O(\log n)$ rotations for both insertion and deletion, and is the method of choice when implementing intrusive red-black trees where parent pointers are costly.

11.22 Finger Trees

Finger trees (Hinze and Paterson, 2006) are a functional data structure that provides fast concatenation and splitting while supporting efficient indexed access. They are the foundation of modern functional sequence libraries.

11.22.1 Structure

A finger tree is a recursive structure with three cases:

1. **Empty tree**
2. **Single element**: a tree with exactly one value
3. **Deep node**: a node containing a “left finger” (a small array of elements near the left end), a “right finger” (near the right end), and a lazy spine of “digit” nodes in between.

```

1 // Finger tree with monoid annotations
2 // m: monoid type (e.g., size, sum, etc.)
3 // a: element type
4 template <typename M, typename A>
5 class FingerTree {
6     enum NodeTag { EMPTY, SINGLE, DEEP };
7
8     struct Node; // 2-3 node in the spine
9     struct Digit; // small buffer (1-4 elements)
10
11     struct Deep {
12         Digit prefix; // left finger (1-4 elements)
13         FingerTree<M, Node*> spine; // lazy middle
14         Digit suffix; // right finger (1-4 elements)
15         M measure; // cached monoid summary
16     };
17
18     NodeTag tag;
19     union { /* empty, single, or deep */ };
20
21 public:
22     // O(1) amortized: add element to left
23     FingerTree cons(const A& x) const;
24     // O(1) amortized: add element to right
25     FingerTree snoc(const A& x) const;
26     // O(min(n1, n2)): concatenate two trees
27     FingerTree operator+(const FingerTree& other) const;
28     // O(log* n): split by predicate
29     pair<FingerTree, FingerTree> split(
30         function<bool(const M&)> pred) const;
31     // O(log* n): access by index
32     const A& operator[](int i) const;
33 };

```

11.22.2 Measure Monoid

Each node carries a monoid value (e.g., the size of its subtree, or the sum of weights). The deep node caches its measure as $m = \text{prefix} \cdot \text{spine} \cdot \text{suffix}$. This allows efficient querying:

- **Length:** the measure of the root is the total size.
- **Split by predicate:** walk the spine, splitting when the accumulated measure satisfies the predicate.
- **Indexed access:** find the subtree containing the k -th element using the cached measures.

11.22.3 Performance

Operation	Complexity
cons/snoc (left/right insert)	$O(1)$ amortized
head/tail (left remove)	$O(1)$ amortized
concatenation	$O(\log \min(n_1, n_2))$
split	$O(\log^* n)$
Indexed access	$O(\log^* n)$

The $O(\log^* n)$ bound comes from the nested logarithmic structure of the spine: the spine is a finger tree of 2-3 nodes, each of which is a finger tree of smaller nodes, and so on. In practice, $\log^* n \leq 5$ for any realistic input.

11.22.4 Applications

Finger trees are the backbone of:

- **Haskell Data.Sequence**: The standard library sequence type, built on finger trees with size annotations.
- **Rope data structures**: For text editors, with string concatenation and substring operations.
- **Persistent deques**: With amortized $O(1)$ operations.
- **Priority queues with concatenation**: Using a monoid that tracks the minimum element.

11.23 Partial Rebuilding

Partial rebuilding is a general technique for maintaining balanced search trees. Instead of performing individual rotations on each update, we reconstruct an entire subtree when it becomes unbalanced, amortizing the cost over many operations.

11.23.1 Concept

The idea is simple: when a subtree of size n becomes unbalanced (its height exceeds a threshold), we flatten the subtree into a sorted array, rebuild it as a perfectly balanced tree, and return the new root. The rebuild costs $O(n)$, but if we rebuild only when the subtree has doubled in size (or when the number of structural modifications since the last rebuild reaches n), each operation is charged $O(1)$ amortized.

```

1 // Generic partial rebuild
2 template <typename Node>
3 Node* rebuild(Node* root) {
4     // Collect all nodes in sorted order
5     std::vector<Node*> nodes;
```



```

6     inOrderCollect(root, nodes);
7     // Build perfectly balanced tree
8     return buildBalanced(nodes, 0, nodes.size() - 1);
9 }
10
11 template <typename Node>
12 Node* buildBalanced(std::vector<Node*>& nodes, int lo, int hi) {
13     if (lo > hi) return nullptr;
14     int mid = lo + (hi - lo) / 2;
15     nodes[mid]->left = buildBalanced(nodes, lo, mid - 1);
16     nodes[mid]->right = buildBalanced(nodes, mid + 1, hi);
17     return nodes[mid];
18 }

```

11.23.2 When to Rebuild

The standard criterion is the *force-rebuild condition*: rebuild a subtree of size s after $\Theta(s)$ structural modifications (insertions/deletions within that subtree) have occurred since the last rebuild. This gives $O(1)$ amortized cost per operation on that subtree, and $O(\log n)$ amortized cost overall.

This is the principle behind:

- **Splay trees** (Section 11.9): partial rebuilding via rotations
- **Finger trees** (Section 11.22): implicit partial rebuilding via lazy evaluation
- **Catenable lists**: explicit rebuilds on concatenated structures

11.24 Join and Split Operations

Join and split are fundamental operations on ordered sets that are not natively supported by all balanced tree structures. Understanding when and how to implement them is crucial for designing efficient algorithms.

11.24.1 Join

Given two balanced BSTs T_1 and T_2 where every key in T_1 is less than every key in T_2 , the **join** operation produces a single balanced BST containing all keys.

Data Structure	Join Cost
Unbalanced BST	$O(m \log(n/m))$
AVL tree	$O(\log n)$
Red-black tree	$O(\log n)$
Weight-balanced tree	$O(\log n)$
Splay tree	$O(\log n)$ amortized
B-tree	$O(\log_b n)$

The key insight is: for AVL and red-black trees, we can join by inserting the root of the smaller tree into the larger tree (using the appropriate rotation rules). For weight-balanced trees, we use the join algorithm from Section 11.19.3.

11.24.2 Split

The inverse of join: given a balanced BST T and a key k , **split** partitions T into two trees (T_1, T_2) where T_1 contains all keys less than k and T_2 contains all keys $\geq k$.

Split is most naturally supported by weight-balanced trees and splay trees. For structures that do not natively support split (e.g., standard AVL), we must walk down to the split point, maintaining a “path spine” and rebuilding both halves.

11.24.3 Applications of Join and Split

These operations enable:

- **Concatenation of sorted sequences:** Join two finger trees, or merge two priority queues.
- **Set difference and intersection:** Split-and-conquer algorithms (split T_2 by a pivot from T_1 , recurse).
- **Order-statistic maintenance:** Select, rank, and range queries are all more efficient with native split.
- **Persistent data structures:** Join and split are structural, not destructive, so they naturally support persistence.

11.25 Handling Nonunique Keys

All BST variants discussed so far assume unique keys. Many practical applications require duplicate keys (e.g., a multiset of student IDs, a frequency table). There are several strategies for handling duplicates.

11.25.1 Counting Duplicates

The simplest approach: augment each node with a count field. When inserting a duplicate, increment the count rather than creating a new node. All operations remain $O(\log n)$ where n is the number of *distinct* keys.

```

1  template <typename K, typename V>
2  struct CountNode {
3      K key;
4      V value;
5      int count;          // number of duplicate keys
6      int size;           // total count (sum of counts in subtree)
7      CountNode *left, *right;
8      CountNode(const K& k, const V& v)
9          : key(k), value(v), count(1), size(1),
10            left(nullptr), right(nullptr) {}
11 };
12
13 // Update invariant after modification
14 template <typename Node>
15 void updateCount(Node* n) {
16     if (n) n->size = n->count + sz(n->left) + sz(n->right);
17 }

```

11.25.2 Multiway Split

Alternatively, store each key with a unique identifier (e.g., a secondary key) and use a lexicographic ordering (key,id). This avoids the need for a separate count field but increases tree size.

11.25.3 Range Counting

With the counting augmentation, the number of elements in a range $[a, b]$ is computed by:

1. Split by a to get (T_1, T_2) .
2. Split T_2 by $b + 1$ to get (T_3, T_4) .
3. The answer is $|T_3|$ (available in $O(1)$ from the root's `size`).

This is $O(\log n)$ for weight-balanced trees and splay trees, but requires $O(\log n)$ with augmented data for AVL/red-black trees.

11.26 Optimal Binary Search Trees

Given n keys with known access probabilities, the *optimal BST* problem asks for the BST that minimizes the expected search cost. This is a classic dynamic programming problem.

11.26.1 Problem Formulation

Let p_i be the probability of searching for key k_i , and q_i be the probability of searching for a value in the gap (k_i, k_{i+1}) , where $\sum p_i + \sum q_i = 1$. The expected cost of a BST T is:

$$\text{cost}(T) = \sum_i p_i \cdot \text{depth}_T(k_i) + \sum_j q_j \cdot \text{depth}_T(\text{gap}_j)$$

where $\text{depth}_T(x)$ is the depth of the node or gap in T .

11.26.2 Dynamic Programming Solution

Define $e[i][j]$ as the expected cost of the optimal subtree containing keys $k_i, \dots, k_j$, and $w[i][j]$ as the total weight of those keys. Then:

$$e[i][i-1] = q_{i-1}$$

$$e[i][j] = \min_{i \leq r \leq j} \{e[i][r-1] + e[r+1][j] + w[i][j]\}$$

$$w[i][j] = w[i][j-1] + p_j + q_j$$

```

1 // Compute optimal BST cost in O(n^2) time, O(n) space
2 double optimalBSTCost(vector<double>& p, vector<double>& q) {
3     int n = p.size();
4     vector<vector<double>>> e(n + 1, vector<double>(n + 1, 0.0));
5     vector<vector<double>>> w(n + 1, vector<double>(n + 1, 0.0));
6
7     for (int i = 1; i <= n; ++i) {
8         e[i][i - 1] = q[i - 1];
9         w[i][i - 1] = q[i - 1];
10    }
11
12    for (int len = 1; len <= n; ++len) {
13        for (int i = 1; i <= n - len + 1; ++i) {
14            int j = i + len - 1;
15            e[i][j] = numeric_limits<double>::infinity();
16            w[i][j] = w[i][j - 1] + p[j] + q[j];
17            for (int r = i; r <= j; ++r) {
18                double cost = e[i][r - 1] + e[r + 1][j] + w[i][j];
19                if (cost < e[i][j]) e[i][j] = cost;
20            }
21        }
22    }
23    return e[1][n];
24 }
```

11.26.3 Knuth's Optimization

Knuth (1971) showed that the root $r[i][j]$ of the optimal subtree for keys $i, \dots, j$ satisfies $r[i][j-1] \leq r[i][j] \leq r[i+1][j]$. This reduces the time complexity from $O(n^3)$ to $O(n^2)$.

11.26.4 Practical Relevance

The optimal BST problem arises in:

- **Static dictionary compression:** When the access frequencies of keys are known in advance (e.g., a keyword list in a compiler).
- **Branch prediction:** Optimal BSTs minimize the expected number of branch mispredictions.
- **Huffman coding:** Optimal prefix codes are a special case (with $p_i = q_i = 0$ for gaps).
- **Database indexing:** When query frequencies are stable, the optimal BST structure outperforms AVL or red-black trees by a constant factor.

For unknown or changing distributions, the splay tree (Section 11.9) achieves $O(\log n)$ amortized cost per access, which is within a constant factor of optimal for any access sequence.

11.27 Comparison

Tree Type	Insert	Find	Erase	Memory	Use Case
BST (unbalanced)	$O(n)$ worst	$O(n)$ worst	$O(n)$ worst	2 ptrs	Educational only
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	2 ptrs + height	Frequent lookups
Red-Black	$O(\log n)$	$O(\log n)$	$O(\log n)$	2 ptrs + color bit	General purpose
Treap	$O(\log n)$ exp	$O(\log n)$ exp	$O(\log n)$ exp	2 ptrs + priority	Simple impl
Splay	$O(\log n)$ amort	$O(\log n)$ amort	$O(\log n)$ amort	2 ptrs	Self-adjusting
B-Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	variable	Disk-based
k-d Tree	$O(n \log n)$ build	$O(\log n)$ avg	N/A (static)	2 pts + split axis	Spatial queries

11.28 STL Associative Containers: set, map, multiset, multimap

The C++ Standard Library provides four ordered associative containers backed by red-black trees. All four guarantee $O(\log n)$ for insert, erase, and find. Understanding their internals, common patterns, and pitfalls is essential for interviews and production code.

11.28.1 std::set

A `std::set` stores unique keys in sorted order. It is a `std::map` without values. Internally, each element is stored as the key portion of a red-black tree node.

```
1 std::set<int> s;
2 s.insert(5);
3 s.insert(3);
4 s.insert(7);
5 s.insert(3); // duplicate -- ignored, size stays 3
6
7 assert(s.size() == 3);
8 assert(s.count(5) == 1); // 1 if present, 0 if not
9 assert(s.count(4) == 0);
10
11 // Erase by value
12 s.erase(5);
13 assert(s.find(5) == s.end());
14
15 // Erase by iterator (O(1) amortized)
16 auto it = s.find(7);
17 if (it != s.end()) s.erase(it);
```

Key operations:

Operation	Complexity
insert (value)	O(log n)
erase (value)	O(log n)
find (value)	O(log n)
count (value)	O(log n)
lower_bound (value)	O(log n)
upper_bound (value)	O(log n)
empty ()	O(1)
size ()	O(1)

Iterating in order:

```
1 std::set<int> s = {5, 3, 1, 4, 2};
2 for (int x : s) std::cout << x << ' '; // 1 2 3 4 5
3
4 // Reverse iteration
5 for (auto it = s.rbegin(); it != s.rend(); ++it)
6     std::cout << *it << ' '; // 5 4 3 2 1
```

11.28.2 std::multiset

A `std::multiset` is like `std::set` but allows duplicate keys. This makes `count ()` meaningful – it returns how many times a key appears.

```
1 std::multiset<int> ms;
2 ms.insert(5);
```

```

3 ms.insert(5);
4 ms.insert(5);
5 ms.insert(3);
6 ms.insert(7);
7
8 assert(ms.count(5) == 3);
9 assert(ms.size() == 5);
10
11 // Erase removes ALL occurrences by default
12 ms.erase(5);
13 assert(ms.count(5) == 0);
14
15 // Erase a single occurrence using find
16 ms.insert(7); ms.insert(7);
17 auto it = ms.find(7); // points to one occurrence
18 if (it != ms.end()) ms.erase(it); // removes one
19 assert(ms.count(7) == 1);

```

Use cases: frequency counters, range queries on duplicates, finding top-k elements.

```

1 // Count frequencies of characters in a string
2 std::string s = "hello world";
3 std::multiset<char> freq(s.begin(), s.end());
4 for (char c : s)
5     std::cout << c << ": " << freq.count(c) << '\n';
6
7 // Find the k-th smallest element
8 std::multiset<int> data = {3, 1, 4, 1, 5, 9, 2, 6, 5};
9 auto it = data.begin();
10 std::advance(it, 3); // 4th smallest (0-indexed: 3rd)
11 assert(*it == 3);

```

11.28.3 std::map

A `std::map` stores key-value pairs in sorted order by key. It is the most commonly used associative container.

```

1 std::map<std::string, int> m;
2
3 // Insert
4 m["apple"] = 5;
5 m["banana"] = 3;
6 m.insert({"cherry", 7});
7
8 // Find
9 auto it = m.find("apple");
10 if (it != m.end())
11     std::cout << it->first << ": " << it->second;

```

```

12
13 // operator[] inserts a default value if key is missing!
14 // Dangerous for lookups:
15 if (m["dragon"] == 0) // inserts m["dragon"] = 0, then checks
16     std::cout << "dragon not found\n"; // this prints!
17
18 // Safe lookup:
19 if (!m.contains("dragon"))
20     std::cout << "dragon not found\n";

```

The operator[] pitfall: Using `m[key]` for read-only lookups silently inserts a default value if the key is missing. This can corrupt data and waste memory. Always use `find()` or `contains()` (C++20) for lookups.

Range queries with iterators:

```

1 std::map<std::string, int> m = {"a", 1}, {"b", 2}, {"c", 3}, {"d", 4}};
2
3 // lower_bound: first element >= key
4 auto lo = m.lower_bound("b"); // points to "b"
5
6 // upper_bound: first element > key
7 auto hi = m.upper_bound("c"); // points to "d"
8
9 // Iterate ["b", "c")
10 for (auto it = lo; it != hi; ++it)
11     std::cout << it->first << " "; // b c
12
13 // equal_range: returns {lower_bound, upper_bound}
14 auto [begin, end] = m.equal_range("b");
15 for (auto it = begin; it != end; ++it)
16     std::cout << it->first << " "; // b

```

11.28.4 std::multimap

A `std::multimap` stores multiple values per key. It is to `std::map` what `std::multiset` is to `std::set`.

```

1 std::multimap<std::string, int> mm;
2 mm.insert({"apple", 1});
3 mm.insert({"apple", 2});
4 mm.insert({"apple", 3});
5 mm.insert({"banana", 4});
6
7 assert(mm.count("apple") == 3);
8 assert(mm.size() == 4);
9
10 // find returns the first occurrence
11 auto it = mm.find("apple");

```



```
12 assert(it->second == 1);
13
14 // Use equal_range to iterate all occurrences
15 auto [begin, end] = mm.equal_range("apple");
16 for (auto iter = begin; iter != end; ++iter)
17     std::cout << iter->first << ": " << iter->second << '\n';
18 // apple: 1, apple: 2, apple: 3
```

Use cases: inverted indexes, grouping records by key, representing many-to-one relationships.

```
1 // Group students by grade
2 std::multimap<char, std::string> students;
3 students.insert({'A', "Alice"});
4 students.insert({'B', "Bob"});
5 students.insert({'A', "Amy"});
6 students.insert({'A', "Andrew"});
7 students.insert({'B', "Brenda"});
8
9 // Print all A students
10 auto [begin, end] = students.equal_range('A');
11 for (auto it = begin; it != end; ++it)
12     std::cout << it->second << '\n';
13 // Alice, Amy, Andrew
```

11.28.5 Internal Structure

All four containers store elements as nodes in a red-black tree. The node layout for `std::map` is:

```
1 // Pseudocode for map node layout
2 struct map_node {
3     std::pair<const Key, Value> data; // key-value pair
4     map_node* left; // left child
5     map_node* right; // right child
6     map_node* parent; // parent pointer
7     Color color; // red or black
8 };
```

For `std::set`, the node contains only the key (no value). For `std::multimap` and `std::multiset`, the node layout is identical – duplicates are stored as separate nodes.

Memory overhead per node:

Container	Overhead per element
<code>std::set</code>	3 pointers + color + key
<code>std::map</code>	3 pointers + color + key + value
<code>std::multiset</code>	Same as <code>std::set</code>
<code>std::multimap</code>	Same as <code>std::map</code>

Each node typically takes 32–48 bytes on a 64-bit system, regardless of element size. This makes associative containers expensive for small elements (prefer `std::vector` for POD types).

11.28.6 Common Interview Patterns

Pattern 1: Two Sum with map

```
1 int two_sum(const std::vector<int>& a, int target) {
2     std::map<int, int> seen; // value -> index
3     for (int i = 0; i < (int)a.size(); ++i) {
4         if (auto it = seen.find(target - a[i]); it != seen.end())
5             return it->second + i; // found pair
6         seen[a[i]] = i;
7     }
8     return -1;
9 }
```

Pattern 2: Frequency counting

```
1 // Count frequency of each element
2 std::map<int, int> freq;
3 for (int x : data) ++freq[x];
4
5 // Sort by frequency
6 std::vector<std::pair<int, int>> sorted_freq(freq.begin(), freq.end());
7 std::sort(sorted_freq.begin(), sorted_freq.end(),
8     [](const auto& a, const auto& b) { return a.second > b.second; });
```

Pattern 3: Lower/upper bound range count

```
1 // Count elements in range [lo, hi]
2 int range_count(const std::set<int>& s, int lo, int hi) {
3     auto begin = s.lower_bound(lo);
4     auto end = s.upper_bound(hi);
5     return std::distance(begin, end); // O(log n + k)
6 }
```

Pattern 4: Grouping

```
1 // Group anagrams
2 std::map<std::string, std::vector<std::string>> groups;
3 for (const auto& word : words) {
4     std::string sorted = word;
5     std::sort(sorted.begin(), sorted.end());
6     groups[sorted].push_back(word);
7 }
```

11.28.7 Choosing Between Containers

Need	Use	Why
Unique sorted keys	<code>std::set</code>	No values, less memory
Key-value pairs	<code>std::map</code>	Sorted, $O(\log n)$
Duplicate keys	<code>std::multiset</code> / <code>std::multimap</code>	Allows repeats
$O(1)$ lookup, no ordering	<code>std::unordered_map</code>	Hash table
Sorted + fast insert/delete	Red-black tree (built-in)	Balanced
Disk-based sorted	B+ tree (custom)	Block-aligned I/O

11.28.8 STL Connection: `std::unordered_map`

For completeness, `std::unordered_map` uses a hash table instead of a tree. It provides $O(1)$ average lookup but $O(n)$ worst case.

```

1 std::unordered_map<std::string, int> um;
2 um["apple"] = 5;
3
4 // O(1) average
5 auto it = um.find("apple");
6 if (it != um.end()) std::cout << it->second;
7
8 // No ordering guarantee
9 for (const auto& [key, value] : um)
10     std::cout << key << ": " << value << '\n';

```

Feature	<code>std::map</code>	<code>std::unordered_map</code>	<code>std::set</code>
Structure	Red-black tree	Hash table	Red-black tree
Lookup	$O(\log n)$	$O(1)$ avg, $O(n)$ worst	$O(\log n)$
Ordered iteration	Yes	No	Yes
Range queries	Yes	No	Yes
Needs hash	No	Yes	No
Memory	High (nodes)	High (buckets + nodes)	High (nodes)
Small n (< 50)	Fast (cache)	Slow (hash overhead)	Fast

11.28.9 Common Bugs and Pitfalls

Pitfall	Consequence	Solution
Using <code>m[key]</code> for lookup	Inserts default value, corrupts data	Use <code>find()</code> or <code>count()</code>
Erasing while iterating	Iterator invalidated, undefined behavior	Use <code>it = m.erase(it)</code>
Calling <code>count()</code> on map	Always returns 0 or 1, not useful	Use <code>find()</code> for existence
Using <code>set::erase(val)</code> on multiset	Removes ALL occurrences	Use <code>erase(find(begin(), val))</code>
Comparing <code>map::iterator</code> across containers	Undefined behavior	Only compare iterators from same container
Expecting <code>unordered_map</code> to be ordered	Iteration order is arbitrary	Use <code>std::map</code> for ordered iteration

11.28.10 Interview FAQs

1. What is the difference between `std::map` and `std::unordered_map`?

Q: “Why is operator[] dangerous for read-only lookups in `std::map`, and what’s the modern alternative?”

A: `operator[]` inserts a default-constructed value if the key is missing — a silent side effect that corrupts your data structure. `if (m[key] == 0)` creates the entry if it doesn’t exist. Before C++20, the safe pattern was `auto it = m.find(key); if (it != m.end())`. C++20 added `m.contains(key)` — $O(\log n)$, returns bool, no insertion, reads clearly. This is the idiomatic modern C++ way. The deeper lesson: `operator[]` was designed for *insertion-or-access*, not *lookup*. Using it for lookup is a category error.

1. Why is `operator[]` dangerous for read-only lookups in `std::map`?

`operator[]` inserts a default-constructed value if the key is missing. So `if (m[key] == 0)` silently creates an entry `m[key] = 0` before the comparison. This can corrupt your data and waste memory. Always use `m.find(key) != m.end()` or `m.contains(key)` (C++20) for lookups.

1. How do you erase an element from a `std::set` while iterating?

Use the return value of `erase()`: `it = s.erase(it)`. This returns an iterator to the next element. The old iterator is invalidated after erase. Never do `s.erase(it++)`; — it works but is unnecessary and less clear.

1. When would you use `std::multimap` instead of `std::map`?

Q: “How do you efficiently iterate over all values for a given key in a `std::multimap`?”

A: Use `equal_range(key)` which returns a `std::pair<iterator, iterator>` bounding all matching elements. $O(\log n)$ to find the range + $O(k)$ to iterate k matches. This is the bread-and-butter operation for multimaps. Without it, you’d do a linear scan — which defeats the purpose of using a tree-based container. Example: `auto [lo, hi] = mp.equal_range(key); for (auto it = lo; it != hi; ++it) { /* it->second is the value */ }`

}. The range-based for doesn't work directly because the range isn't a single pair — you need the explicit iterator loop.

1. What is the memory overhead of `std::map` per element?

Each node in a red-black tree requires: 3 pointers (left, right, parent) + 1 color bit + the key-value pair. On a 64-bit system, a `std::map<int, int>` node typically takes 32–40 bytes for 8 bytes of data. This is why `std::vector` is preferred for small POD types – it stores elements contiguously with no per-element overhead.

1. How do you find the k-th smallest element in a `std::set`?

`std::set` does not support random access. Use `std::advance(it, k)` where `it = s.begin()`. This is $O(k + h)$ time, not $O(\log n)$. For repeated order-statistic queries, use a policy-based tree (GCC `_gnu_pbds::tree`) with node update for $O(\log n)$ per query.

11.29 STL Unordered Containers: `unordered_set`, `unordered_map`

The ordered containers (`std::map`, `std::set`) use red-black trees and guarantee $O(\log n)$ operations. The **unordered** containers use hash tables and provide $O(1)$ average-case performance. The standard library offers four unordered variants: `unordered_set`, `unordered_multiset`, `unordered_map`, and `unordered_multimap`.

11.29.1 `std::unordered_set`

An `unordered_set` stores unique keys in no particular order. Lookup is $O(1)$ average, $O(n)$ worst case.

```
1 std::unordered_set<int> us;
2 us.insert(5);
3 us.insert(3);
4 us.insert(7);
5 us.insert(3); // duplicate -- ignored
6
7 assert(us.size() == 3);
8 assert(us.count(5) == 1);
9
10 // Erase by value
11 us.erase(5);
12 assert(us.find(5) == us.end());
13
14 // Erase by iterator (O(1) amortized)
15 auto it = us.find(7);
16 if (it != us.end()) us.erase(it);
```

Key operations:

Operation	Complexity (avg / worst)
<code>insert(value)</code>	$O(1) / O(n)$
<code>erase(value)</code>	$O(1) / O(n)$
<code>find(value)</code>	$O(1) / O(n)$
<code>count(value)</code>	$O(1) / O(n)$
<code>bucket_count()</code>	$O(1)$
<code>load_factor()</code>	$O(1)$
<code>reserve(n)</code>	$O(\text{bucket_count})$

Iteration order is unpredictable:

```
1 std::unordered_set<int> us = {5, 3, 1, 4, 2};
2 for (int x : us)
3     std::cout << x << ' '; // order is implementation-defined
```

11.29.2 `std::unordered_multiset`

Like `std::unordered_set` but allows duplicate keys. `count()` returns the number of occurrences.

```
1 std::unordered_multiset<int> ums;
2 ums.insert(5);
3 ums.insert(5);
4 ums.insert(5);
5 ums.insert(3);
6
7 assert(ums.count(5) == 3);
8
9 // Erase removes ALL occurrences by default
10 ums.erase(5);
11 assert(ums.count(5) == 0);
12
13 // Erase a single occurrence
14 ums.insert(7); ums.insert(7);
15 auto it = ums.find(7);
16 if (it != ums.end()) ums.erase(it);
17 assert(ums.count(7) == 1);
```

11.29.3 `std::unordered_map`

A `std::unordered_map` stores key-value pairs hashed by key. It is the unordered counterpart to `std::map`.

```
1 std::unordered_map<std::string, int> um;
2 um["apple"] = 5;
```

```

3 um["banana"] = 3;
4 um.insert({"cherry", 7});
5
6 // O(1) average lookup
7 auto it = um.find("apple");
8 if (it != um.end())
9     std::cout << it->first << ": " << it->second;
10
11 // operator[] inserts default if key missing (same as std::map!)
12 if (!um.contains("dragon"))
13     std::cout << "dragon not found\n";

```

The **operator[]** pitfall applies here too. `um[key]` silently inserts a default value. Always use `find()` or `contains()` for lookups.

Iteration:

```

1 std::unordered_map<std::string, int> um = {"a", 1}, {"b", 2}, {"c", 3};
2 for (const auto& [key, value] : um)
3     std::cout << key << ": " << value << '\n';
4 // order is not sorted (and not guaranteed stable across inserts)

```

11.29.4 std::unordered_multimap

Allows multiple values per key. Like `std::multimap` but with hash-based access.

```

1 std::unordered_multimap<std::string, int> umm;
2 umm.insert({"apple", 1});
3 umm.insert({"apple", 2});
4 umm.insert({"apple", 3});
5 umm.insert({"banana", 4});
6
7 assert(umm.count("apple") == 3);
8
9 // find returns the first occurrence
10 auto it = umm.find("apple");
11 assert(it->second == 1);
12
13 // equal_range iterates all occurrences for a key
14 auto [begin, end] = umm.equal_range("apple");
15 for (auto iter = begin; iter != end; ++iter)
16     std::cout << iter->first << ": " << iter->second << '\n';
17 // apple: 1, apple: 2, apple: 3

```

11.29.5 Hash Functions and Custom Keys

For user-defined types, you must provide a hash function. The standard library provides hashes for built-in types and `std::string`, but not for custom structs.

```

1 struct Point {
2     int x, y;
3     bool operator==(const Point& o) const {
4         return x == o.x && y == o.y;
5     }
6 };
7
8 // Provide a hash function
9 struct PointHash {
10     size_t operator()(const Point& p) const {
11         size_t h1 = std::hash<int>{}(p.x);
12         size_t h2 = std::hash<int>{}(p.y);
13         return h1 ^ (h2 << 1); // combine hashes
14     }
15 };
16
17 std::unordered_set<Point, PointHash> points;
18 points.insert({1, 2});
19 points.insert({3, 4});
20 assert(points.count({1, 2}) == 1);
21
22 // Or use std::hash specialization
23 namespace std {
24     template<> struct hash<Point> {
25         size_t operator()(const Point& p) const {
26             return hash<int>{}(p.x) ^ (hash<int>{}(p.y) << 1);
27         }
28     };
29 }

```

11.29.6 Rehashing and Load Factor

The load factor is `size / bucket_count`. When it exceeds `max_load_factor()` (default 1.0), the hash table rehashes – allocates new buckets and reinserts all elements. This is $O(n)$ and happens automatically.

```

1 std::unordered_set<int> us;
2 us.reserve(1000); // pre-allocate for 1000 elements
3 // Prevents rehashing during insertion of up to 1000 elements
4
5 std::cout << us.bucket_count() << '\n'; // number of buckets
6 std::cout << us.load_factor() << '\n'; // elements / buckets
7 us.max_load_factor(0.5); // trigger rehash more often
8 us.rehash(2000); // force 2000 buckets

```

Rule of thumb: Call `reserve(n)` if you know the approximate number of elements. This avoids multiple rehashes during insertion.

11.29.7 Common Interview Patterns

Pattern 1: Grouping with unordered_map

```

1 // Group anagrams
2 std::unordered_map<std::string, std::vector<std::string>> groups;
3 for (const auto& word : words) {
4     std::string sorted = word;
5     std::sort(sorted.begin(), sorted.end());
6     groups[sorted].push_back(word);
7 }

```

Pattern 2: Frequency counting

```

1 std::unordered_map<char, int> freq;
2 for (char c : s) ++freq[c];
3
4 // Find the first non-repeating character
5 for (char c : s)
6     if (freq[c] == 1) return c;

```

Pattern 3: Two Sum with unordered_map

```

1 int two_sum(const std::vector<int>& a, int target) {
2     std::unordered_map<int, int> seen;
3     for (int i = 0; i < (int)a.size(); ++i) {
4         if (auto it = seen.find(target - a[i]); it != seen.end())
5             return it->second + i;
6         seen[a[i]] = i;
7     }
8     return -1;
9 }

```

Pattern 4: Set intersection

```

1 std::unordered_set<int> intersect(const std::unordered_set<int>& a,
2                                 const std::unordered_set<int>& b) {
3     std::unordered_set<int> result;
4     for (int x : a)
5         if (b.count(x)) result.insert(x);
6     return result;
7 }

```

11.29.8 Choosing Between All Eight Containers

Need	Use
Unique keys, sorted order	<code>std::set</code>
Key-value pairs, sorted order	<code>std::map</code>
Duplicate keys, sorted order	<code>std::multiset</code> / <code>std::multimap</code>
Unique keys, O(1) lookup	<code>std::unordered_set</code>
Key-value pairs, O(1) lookup	<code>std::unordered_map</code>
Duplicate keys, O(1) lookup	<code>std::unordered_multiset</code> / <code>std::unordered_multimap</code>
No ordering needed, small data	<code>std::vector</code> + sort
Disk-based, block-aligned	B+ tree (custom)

11.30 Bimap: Bidirectional Map

A **bimap** (bidirectional map) is a data structure that maintains two maps – `key→value` and `value→key` – allowing $O(\log n)$ lookup in both directions. It is useful when you need to find a value by its key *and* find a key by its value.

11.30.1 Motivation

Suppose you store country codes: "US" ↔ "United States", "IN" ↔ "India". With a single `std::map`, you can look up a country by code, but looking up a code by country requires a linear scan. A bimap solves this by maintaining both directions simultaneously.

```

1 // Without bimap: two separate maps, must keep in sync
2 std::map<std::string, std::string> code_to_name;
3 std::map<std::string, std::string> name_to_code;
4 // Inserting requires updating both -- error-prone

```

11.30.2 Implementation

A bimap can be implemented as two `std::maps` (or two `std::unordered_maps` for O(1) average). The key invariant is: both maps must always agree – if `A→B` exists in one, `B→A` must exist in the other.

```

1 template <typename K, typename V>
2 class bimap {
3     std::map<K, V> kv_; // key -> value
4     std::map<V, K> vk_; // value -> key (reverse)
5
6 public:
7     void insert(const K& key, const V& value) {
8         if (kv_.count(key) || vk_.count(value))
9             throw std::runtime_error("duplicate key or value");
10        kv_[key] = value;

```

```

11         vk_[value] = key;
12     }
13
14     // Lookup by key: O(log n)
15     std::optional<V> by_key(const K& key) const {
16         auto it = kv_.find(key);
17         if (it != kv_.end()) return it->second;
18         return std::nullopt;
19     }
20
21     // Lookup by value: O(log n)
22     std::optional<K> by_value(const V& value) const {
23         auto it = vk_.find(value);
24         if (it != vk_.end()) return it->second;
25         return std::nullopt;
26     }
27
28     bool erase_by_key(const K& key) {
29         auto it = kv_.find(key);
30         if (it == kv_.end()) return false;
31         vk_.erase(it->second);
32         kv_.erase(it);
33         return true;
34     }
35
36     bool erase_by_value(const V& value) {
37         auto it = vk_.find(value);
38         if (it == vk_.end()) return false;
39         kv_.erase(it->second);
40         vk_.erase(it);
41         return true;
42     }
43
44     bool contains_key(const K& key) const { return kv_.count(key); }
45     bool contains_value(const V& value) const { return vk_.count(value); }
46     std::size_t size() const { return kv_.size(); }
47 };

```

Usage:

```

1 bimap<std::string, int> bm;
2 bm.insert("apple", 5);
3 bm.insert("banana", 3);
4
5 assert(bm.by_key("apple") == 5);
6 assert(bm.by_value(3) == "banana");
7 assert(!bm.by_key("cherry").has_value());
8
9 bm.erase_by_key("apple");

```

```
10 assert(!bm.contains_key("apple"));
11 assert(!bm.contains_value(5));
```

11.30.3 Using Boost.Bimap

The Boost library provides a production-grade `boost::bimap` with set views, range queries, and efficient memory management:

```
1 #include <boost/bimap.hpp>
2
3 using namespace boost::bimaps;
4
5 bimap<std::string, int> bm;
6 bm.insert(entry("apple", 5));
7 bm.insert(entry("banana", 3));
8
9 // Left view (key -> value)
10 auto it = bm.left.find("apple");
11 assert(it->second == 5);
12
13 // Right view (value -> key)
14 auto rit = bm.right.find(3);
15 assert(rit->second == "banana");
16
17 // Range query on left view
18 auto lo = bm.left.lower_bound("a");
19 auto hi = bm.left.upper_bound("b");
20 for (auto it = lo; it != hi; ++it)
21     std::cout << it->left << " -> " << it->right << '\n';
```

11.30.4 Bimap Variants

Variant	Left View	Right View
set_of<set>	Ordered	Ordered
set_of<unordered_set>	Unordered	Ordered
unordered_set_of<set>	Ordered	Unordered
unordered_set_of<unordered_set>	Unordered	Unordered

Choose based on whether you need ordered iteration or range queries on either side.

11.30.5 Use Cases

- **Reverse lookup tables:** country code ↔ country name, currency code ↔ currency name
- **ID mapping:** database ID ↔ username, UUID ↔ object pointer

- **Indexing:** two fields in a record that must both be searchable (e.g., email ↔ user)
- **Bi-directional enums:** enum value ↔ string representation

```

1 // Enum <-> string mapping
2 bimap<int, std::string> status_map;
3 status_map.insert(0, "pending");
4 status_map.insert(1, "active");
5 status_map.insert(2, "completed");
6
7 std::string status_str = status_map.by_key(1).value(); // "active"
8 int status_val = status_map.by_value("completed").value(); // 2

```

11.30.6 Complexity

Operation	std::map-based	unordered_map-based
Insert	O(log n)	O(1) average
Lookup by key	O(log n)	O(1) average
Lookup by value	O(log n)	O(1) average
Erase by key	O(log n)	O(1) average
Erase by value	O(log n)	O(1) average
Space	2 × (nodes + pointers)	2 × (nodes + buckets)

Space overhead: A bimap stores two copies of every entry – one in each direction. For large datasets, consider whether the bidirectional lookup is worth 2× memory. If only occasional reverse lookups are needed, a single map with a linear scan may be cheaper.

11.30.7 Interview FAQs

1. **Consolidated: What are the performance and correctness tradeoffs of using a bimap vs manually maintaining two maps?**

Q: “What are the performance and correctness tradeoffs of using a bimap vs manually maintaining two maps?”

A: A bimap guarantees both maps are always in sync — if you erase from the forward map, the reverse entry is automatically removed. With two manual maps, forgetting to update one side is a silent bug (data inconsistency). The performance cost is 2× memory (both keys stored twice) and 2× write cost (two insertions per logical insert). Boost.Bimap supports different container types per side (e.g., `bimap<set_of<>, unordered_set_of<>>>`) — ordered forward access with unordered reverse. Use a bimap when bidirectional lookup is a core requirement (UUID↔object, DNS name↔IP). Use two maps when one direction is rare and can tolerate linear scan.

11.31 The Dynamic Optimality Conjecture

We have seen several BST designs — AVL, red-black, treap, splay — each with $O(\log n)$ operations. But is $O(\log n)$ the best we can do? Is there a single BST that achieves the optimal access cost for *every* access sequence? This is the **dynamic optimality conjecture**, one of the longest-standing open problems in data structure theory.

11.31.1 BSTs as Points in 2D

Any BST with n distinct keys can be drawn in the plane with key values on the x -axis and depths on the y -axis. Each node is a point (x, y) . The **access cost** of a node is its depth plus one (number of comparisons to find it).

An access sequence of length m maps to m points in this 2D plane. The total cost of serving the sequence is the sum of depths of all accessed nodes. The question becomes: among all BSTs containing these n keys, which tree minimizes the total cost for a given sequence?

This geometric view reveals that BSTs and their rotations correspond to operations on 2D point sets. A left rotation at node x effectively swaps x with its right child in the key ordering, changing the shape of the tree in the plane.

11.31.2 The Greedy Algorithm

The **greedy algorithm** (sometimes called **Greedy**) is an online BST algorithm that moves every accessed node to the root along the BST path. Unlike splay trees, which apply zig-zig and zig-zag rotations, Greedy uses a simpler rule: walk down from the root to the accessed node, performing rotations at every step to pull the accessed node upward.

Greedy is *online* — it makes each decision using only the current access, without knowledge of future accesses. Despite this, Greedy provably achieves within a constant factor of the best *offline* BST for every access sequence.

11.31.3 Wilber’s Lower Bound

Wilber (1989) defined the **interleave lower bound**, a lower bound on the cost any BST algorithm must pay for an access sequence. The idea: consider pairs of keys (a, b) such that the most recent access before the current one was to the other key in the pair. The algorithm must “re-establish” the ordering relationship between these keys, which costs at least $\log k$ per pair, where k is the number of such interleaved pairs.

The interleave bound is a lower bound, not an upper bound — no known BST algorithm matches it for all sequences. The gap between the interleave bound and the cost of the best known algorithms is the core of the dynamic optimality problem.

11.31.4 Signed Greedy

Signed Greedy is a variant that tracks, for each edge in the BST, a “sign” indicating which side of the edge the most recent access came from. When signs on a path alternate, the algorithm can bound future costs more tightly. Signed Greedy achieves $O(\log \log n)$ -competitive performance on certain access patterns where unsigned Greedy does not.

11.31.5 Tango Trees

Demaine et al. (2007) introduced **Tango trees**, the best known BST in terms of worst-case competitive ratio. A Tango tree is a set of interleaved BSTs organized by “preferred paths” — each access follows a preferred path, and the tree restructures using a mechanism similar to splay trees.

Structure	Per-operation cost	Competitive ratio
Greedy	$O(\log n)$	$O(\log n)$
Signed Greedy	$O(\log n)$	$O(\log \log n)$
Splay tree	$O(\log n)$ amortized	$O(\log n)$
Tango tree	$O(\log \log n)$ amortized	$O(\log \log n)$
Weight-balanced	$O(\log n)$ worst case	$O(\log n)$

Tango trees achieve $O(\log \log n)$ amortized time per operation, beating the $\Omega(\log n)$ lower bound for comparison-based BSTs. The trick: they use the *access sequence itself* as additional information, making them not purely comparison-based. This is why the competitive ratio (vs. an optimal offline algorithm) is $O(\log \log n)$, not $O(1)$.

11.31.6 Key-Independent Optimality

Some BST algorithms are **key-independent** — their performance depends only on the access pattern, not on the actual key values or their ordering. For example, the “static optimality” bound applies to sequences where the relative ordering of keys is fixed.

A BST is *key-independent optimal* if, for every access sequence, its cost matches the optimal offline BST regardless of how the keys are labeled. Demaine et al. (2009) showed that certain BSTs achieve this property, though the details are technically involved.

11.31.7 Splay Trees and Dynamic Optimality

Splay trees are $O(\log n)$ amortized, which matches the $\Omega(\log n)$ comparison lower bound. However, splay trees are *not known* to be dynamically optimal. The dynamic optimality conjecture posits that splay trees are $O(1)$ -competitive with the optimal offline algorithm, but this remains unproven.

The splay tree’s “move to root” heuristic is simple and effective, but its zig-zig and zig-zag rotations do not always match the greedy strategy. For some access sequences, Greedy outperforms splay trees by a constant factor, and for others, splay trees win. The question of whether one is always within a constant factor of the other — and of the optimal offline algorithm — remains open.

11.32 Fusion Trees

Fusion trees (Fredman and Willard, 1993) achieve a dramatic speedup over comparison-based BSTs by exploiting the *word-level parallelism* of machine arithmetic. The key result: predecessor queries in n words of w bits each can be answered in $O(\log_w n)$ time — exponentially faster than $O(\log n)$.

11.32.1 The Idea: Wide B-Trees

A standard B-tree with branching factor b has height $\log_b n$ and performs $O(\log_b n)$ comparisons per operation. If we could make $b = w^\epsilon$ (a polynomial in the word size w), the height would be $O(\log_{w^\epsilon} n) = O(\frac{1}{\epsilon} \log_w n)$.

The challenge is that finding the correct child among w^ϵ keys requires $\Omega(w^\epsilon)$ comparisons naively — too slow. Fusion trees solve this by compressing each key's w bits into a **sketch** of $O(w/\log w)$ bits, allowing all children to be compared in parallel using a single word operation.

11.32.2 Sketching: Compression by Multiplication

Each key x is a w -bit integer. The sketch of x is computed by multiplying x by a carefully chosen constant M and extracting certain bits of the product. The constant M has the property that the product $x \cdot M$ spreads the “essential bits” of x across the word.

Concretely, the sketch of x is:

$$\text{sketch}(x) = \lfloor (x \cdot M) \bmod 2^w \rfloor \gg (w - w/\log w)$$

where M is a fixed integer chosen so that multiplication “smears” the bits of x . The resulting sketch has $O(w/\log w)$ bits, but retains enough information to distinguish different keys.

11.32.3 Parallel Comparison via Multiplication

Given a query key q , we want to find which of the w^ϵ children to descend into. Each child stores a sketch of its separating key. We compute the sketch of q using the same multiplication, then compare it against all sketches simultaneously.

The trick: subtract the query sketch from each child sketch (all in parallel), and check the most significant bits. The first child whose sketch has a positive most significant bit after subtraction is the predecessor.

11.32.4 Finding the Most Significant Set Bit

The final ingredient is finding the **most significant set bit (msb)** of a w -bit word in $O(1)$ time. This is not a primitive operation on most architectures, but it can be implemented efficiently using the “de Bruijn trick.”

A **de Bruijn sequence** of order k is a binary string of length 2^k in which every k -bit pattern appears exactly once as a circular substring. By multiplying the word with a de Bruijn constant and using a small lookup table, the position of the msb can be extracted in $O(1)$ operations.

```

1 int most_significant_bit(unsigned x) {
2     if (x == 0) return -1;
3     x |= x >> 1;
4     x |= x >> 2;
5     x |= x >> 4;
6     x |= x >> 8;
7     x |= x >> 16;
8     x = (x >> 1) + 1;
9     int count = 0;
10    while (x >>= 1) ++count;
11    return count;
12 }
```

The standard bit-hack above works in $O(\lg w)$ steps (the loop or a binary search). For $O(1)$ msb, use a de Bruijn multiplication table. On modern CPUs, the `bsr` or `clz` instruction gives $O(1)$ msb directly.

11.32.5 Result and Lower Bounds

Fusion trees achieve $O(\log_w n)$ per operation. For $w = 64$ (a 64-bit machine), this is $O(\log_{64} n) = O(\frac{1}{6} \log_2 n)$, roughly $6\times$ faster than standard BSTs.

The information-theoretic lower bound for predecessor search is $\Omega(\min(\lg w, \log_w n))$. Fusion trees are nearly optimal: they match the $\log_w n$ part, and when w is large (as on modern hardware), $\lg w < \log_w n$ is rarely the bottleneck.

Structure	Per operation	Word size dependence
BST	$O(\log n)$	None
B-tree (order b)	$O(\log_b n)$	None
Fusion tree	$O(\log_w n)$	Uses w -bit words
Packed search tree	$O(\log_w n)$	Requires word RAM
Lower bound	$\Omega(\min(\lg w, \log_w n))$	Information-theoretic

11.32.6 Practical Considerations

Fusion trees are primarily a theoretical result. In practice, the constant factors are large: the multiplication-based sketching, the msb extraction, and the cache behavior of wide B-tree nodes all add overhead. For small n , a standard BST or even a cache-oblivious B-tree outperforms fusion trees.

However, fusion trees demonstrate that the $O(\log n)$ comparison barrier is not fundamental — on the word RAM model, faster search is possible by exploiting the bit-level structure of keys. This in-

sight influenced subsequent work on packed integer arrays, succinct data structures, and cache-efficient algorithms.

11.33 Common Bugs and Pitfalls

Pitfall	Consequence
Inserting sorted keys into an unbalanced BST	Degenerates to $O(n)$ – a linked list
Forgetting to update heights after AVL rotation	Balance checks give wrong result, tree corrupts
Deleting a node with two children but not tracking parent	Orphaned subtrees, memory leak
Confusing Red-Black property 5 (black height)	Tree violates red-black invariants
B-tree insertion overflow at root	Root split not handled, height never increases
Not marking <code>std::set</code> iterators as invalid after erase	Iterator use-after-free
Using <code>std::map::operator[]</code> for lookups	Inserts default value if key missing, corrupts data
Splay tree: relying on single-operation worst case	A single operation can take $O(n)$
k-d tree in high dimensions ($k > 20$)	Degenerates to $O(n)$ – curse of dimensionality
<code>unordered_map</code> with bad hash function	Many collisions, $O(n)$ lookup
Rehashing during iteration	Iterator invalidated, undefined behavior

11.34 Summary

1. **BSTs** provide $O(\log n)$ operations on average but $O(n)$ in the worst case.
2. **AVL trees** guarantee $O(\log n)$ by maintaining a height balance invariant.
3. **Red-black trees** are the de facto standard — they balance with fewer rotations.
4. **Treaps** use randomization to achieve $O(\log n)$ expected performance with simple code.
5. **Splay trees** self-adjust to access patterns with no extra storage — $O(\log n)$ amortized.
6. **k-d trees** organize multi-dimensional points for spatial queries in low dimensions.
7. **B-trees** extend the concept to multi-way nodes for disk-based storage.
8. `std::map` and `std::set` are production red-black tree implementations.

11.35 Interview FAQs

1. **Why are red-black trees preferred over AVL trees for `std::map`?**

Red-black trees require at most 2 rotations per insert and at most 3 per delete, while AVL trees may require $O(\log n)$ rotations per delete (rebalancing propagates to the root). For workloads

with frequent inserts and deletes (e.g., database indexes), the fewer rotations of red-black trees translate to better practical performance. AVL trees have tighter balance (height $\leq 1.44 \log n$ vs $2 \log n$), giving faster lookups, but the difference is negligible for most applications.

1. What is the time complexity of finding the k -th smallest element in a BST?

Approach 1: Inorder traversal, count to k . Time $O(k + h)$ where h is the height.

Approach 2: Store subtree sizes in each node. At each node, compare k with left subtree size to decide go left, visit, or go right. Time $O(h)$. This is the “order statistic” approach.

For balanced BST: $O(\log n)$. For unbalanced BST: $O(n)$.

1. What are the properties of a red-black tree and why do they guarantee $O(\log n)$?

Five properties: (1) every node is red or black, (2) root is black, (3) leaves (NIL) are black, (4) red node cannot have red children, (5) every path from root to leaf has the same number of black nodes. Properties 4 and 5 together ensure that the longest path is at most twice the shortest (a path of all black nodes is the shortest; a path alternating red-black is the longest). This bounds height to $2 \log(n + 1)$, giving $O(\log n)$ operations.

1. When would you use a B-tree over a red-black tree?

B-trees are preferred when: (a) data is on disk (B-tree nodes are sized to match disk blocks, minimizing I/O), (b) the dataset is too large for memory, (c) range queries are frequent (B+ trees link leaf nodes for sequential access). Red-black trees are preferred for in-memory data because they have simpler code, less overhead per node, and better cache performance for small datasets. Most databases (PostgreSQL, MySQL InnoDB) use B+ trees for indexes.

1. Implement a BST iterator that returns elements in inorder in $O(1)$ amortized time.

Use an explicit stack. Push all left children from the root. `next()` pops the top, then pushes all left children of the right subtree. `hasNext()` checks if the stack is empty. Each node is pushed and popped at most once, giving $O(1)$ amortized per call. Space $O(h)$.

```

1 class bst_iterator {
2     std::stack<node*> st;
3     void push_left(node* n) { while (n) { st.push(n); n = n->left; } }
4 public:
5     bst_iterator(node* root) { push_left(root); }
6     int next() {
7         node* n = st.top(); st.pop();
8         push_left(n->right);
9         return n->val;
10    }
11    bool has_next() { return !st.empty(); }
12 };

```

1. What is a treap and why is it useful?

A treap assigns a random priority to each node and maintains BST order by key and heap order by priority. With high probability, the treap is balanced (expected height $O(\log n)$). It's useful because: (1) simple to implement (no rotation cases to remember), (2) supports $O(\log n)$ split and merge operations (useful for implicit key treaps), (3) no worst-case input (randomization eliminates adversarial cases). Used in competitive programming and some database engines.

1. How does a splay tree achieve $O(\log n)$ amortized without any balance information?

Every access (find, insert, delete) splays the accessed node to the root via zig, zig-zig, and zig-zag rotations. The key property: splaying a node roughly halves the depth of every node on the access path. Over a sequence of operations, recently accessed nodes stay near the root, reducing future access costs. The amortized proof uses the potential method: each splay increases the potential by at most $3 \log n$, bounding the amortized cost.

1. Given two BSTs, find if they have the same elements (merge check).

Q: "How would you check if two BSTs are structurally identical vs value-identical?"

A: Structural identity requires simultaneous recursive traversal comparing node values AND shapes — $O(n)$. Value identity requires comparing sorted sequences (inorder traversals) — also $O(n)$ but using the merge technique from merge sort. The distinction matters: two BSTs built from the same elements in different order are value-identical but structurally different. For balanced BSTs, structural identity implies value identity (the structure is uniquely determined by the keys). For unbalanced BSTs, the same keys can produce different structures.

1. What is the difference between `std::map` and `std::unordered_map`? When would you choose each?

Q: "How does C++20's `std::map::contains()` change the idiomatic lookup pattern?"

A: Before C++20, `if (m.find(key) != m.end())` was the safe pattern. `m[key]` was dangerous (inserts default). `m.count(key)` works but returns an integer, not an iterator. `contains()` (C++20) is $O(\log n)$, returns bool, doesn't insert, and reads clearly. The deeper change: `contains()` enables cleaner guard clauses (`if (!m.contains(key)) return;`) and works naturally with `std::erase_if()` for bulk removal. It's a small API addition that eliminates a class of bugs.

1. How does a k-d tree find the nearest neighbor efficiently?

Build: recursively partition points by alternating axes, using `nth_element` for median selection. **Query:** at each node, check if the splitting plane is closer than the current best distance. If yes, search the other side; if not, prune that subtree. This eliminates large portions of the search space, giving $O(\log n)$ average for uniformly distributed points. Worst case is $O(n)$ for pathological distributions.

Application: KNN classification, collision detection, geographic proximity queries.

11.36 Exercises

11.36.1 Drill

1. **Sequential keys.** Insert keys $[1, 2, 3, \dots, 100]$ into a plain BST and into a B-tree of order 5. Compare the heights. Why does the BST degenerate while the B-tree doesn't?

(In production, databases use B-trees specifically because they handle both random and sequential inserts gracefully — their fan-out guarantees $O(\log n)$ height regardless of insertion order.)

1. **Tree vs hash table for small data.** A small map (under 50 entries) is used 10,000 times per second for lookups. An engineer switches from a balanced tree to a hash table for "O(1) lookups" but performance gets worse. Why? Think about hash computation cost and cache misses for small data.
1. **Missing step in deletion.** Delete a key from an AVL tree using inorder successor replacement. The tree is unbalanced afterward. What was likely forgotten? Which rotation restores balance?
1. **Unequal black height.** A red-black tree has some root-to-leaf paths with black height 12 and others with 13. Which property is broken? Why does this invalidate the $O(\log n)$ height guarantee?
1. **Prefix scan.** A tree index on $(\text{status}, \text{date})$ has status values 'A', 'B', 'C' and dates over a year. A query filters on $\text{status} = \text{'A'}$ and date in the last month. The database estimates it must scan ~33% of the tree. Roughly how many leaves will it touch?
1. **Splay tree access pattern.** Insert keys $[1, 2, 3, \dots, 100]$ into a splay tree. Then search for key 1 three times. Draw the tree after each search. How does the tree reorganize? What happens to the depth of keys near 1?
1. **k-d tree split axis.** Build a k-d tree on points $\{(2,3), (5,4), (9,6), (4,7), (8,1), (7,2)\}$. What is the root? What are its children? Draw the tree showing the split axis at each level.

11.36.2 Application

1. **B-tree implementation.** Implement a B-tree of order 5 (max 4 keys, min 2 per node) supporting insert and search. Benchmark against `std::map` for 10^5 sequential and random insertions. Count the number of nodes visited per operation. Explain why a higher-order B-tree (order 100+) uses fewer cache misses than a binary tree.
1. **Prefix tree for routing.** A web server has 10,000 URL routes. Linear scanning on each request takes $O(n)$. Build a trie (prefix tree) that stores routes by path segments. Compare lookup speed with `std::map` and linear scan:

- Build time for 10^4 routes
- Lookup time for 10^6 requests
- Memory per route

Report which structure wins for prefix matching vs exact matching.

1. **Concurrent tree.** Implement a thread-safe binary search tree using a mutex. Benchmark throughput with 1 reader, 4 readers, and 4 readers + 1 writer. Where is the bottleneck? Compare with a simple `std::shared_mutex` guarding `std::map`.

1. **Tree vs skip list.** Implement both an AVL tree and a skip list. Compare:

- Insert 10^5 random elements
- Range query: iterate over elements in $[lo, hi]$
- Delete all elements

Which data structure is simpler to implement? Which is faster for range queries?

(In production, Redis uses skip lists (not trees) for sorted sets because they are simpler to implement and offer better range-scan performance.)

1. **Disk-based tree.** Simulate a disk-based B-tree where each node access costs $100\times$ more than an in-memory access. Count the number of node accesses for 10^6 inserts into a B-tree (order 100) vs a BST. How does the fan-out of the B-tree reduce I/O?

1. **Splay tree vs AVL.** Implement both a splay tree and an AVL tree. Run the same workload: insert 10^5 random keys, then perform 10^6 random finds (80% to existing keys, 20% to missing keys). Compare:

- Total time for the find sequence (amortized vs worst-case)
- Time after “hot” keys are repeatedly accessed
- Memory usage (splay tree has no balance field)

When does the splay tree’s self-adjustment help? When does the AVL tree’s guaranteed $O(\log n)$ win?

1. **Nearest-neighbor with k-d tree.** Generate 10^5 random 2D points in $[0, 1]^2$. Build a k-d tree and query the 10 nearest neighbors for 10^4 random query points. Compare:

- Query time vs brute-force (linear scan)
- Build time vs sorting
- How does performance degrade when the dimension increases to 10, 20, 50?

11.36.3 Research

1. **Bulk loading.** Build a B-tree by inserting one key at a time. Then build the same tree by sorting the keys first and constructing the tree bottom-up (layered). Compare:

- Construction time for 10^6 keys
- Tree height
- Node occupancy percentage

Explain why bulk loading produces a denser, shorter tree.

(*In production, ClickHouse uses bulk loading for its primary index because columnar data arrives sorted by sort key.*)

1. **Write-optimized tree.** Research the Log-Structured Merge-tree (LSM-tree) design used by RocksDB. Implement a simplified version: writes go to an in-memory sorted buffer; when full, flush to disk as a sorted run. Compare write throughput with a B-tree for 10^5 inserts. When does the LSM-tree outperform the B-tree? When does it fall behind (reads)?

1. **Persistent (immutable) tree.** Implement a BST where every insert creates a new path of nodes (path copying), leaving the old tree intact. Compare:

- Memory per insertion (how many new nodes?)
- Throughput vs a mutable BST
- Space after 10^5 operations

Why is this useful for concurrent or versioned data structures?

1. **Batched updates.** Research a B-tree variant where each node has a buffer of pending updates. When the buffer overflows, updates propagate to children. Implement a simplified version. Compare write throughput with a standard B-tree. How large must the buffer be to match LSM-tree performance?

11.37 References and Further Reading

- G. M. Adelson-Velsky and E. M. Landis, "An Algorithm for the Organization of Information" — Doklady Akademii Nauk SSSR, 1962.
- Robert Sedgwick and Kevin Wayne, *Algorithms*, 4th Edition. Chapter 3 (BSTs, red-black trees).
- Rudolf Bayer and Edward McCreight, "Organization and Maintenance of Large Ordered Indices" — Acta Informatica, 1972.

- Cecilia R. Aragon and Raimund Seidel, "Randomized Search Trees" — FOCS 1989.
- Leonidas J. Guibas and Robert Sedgwick, "A Dichromatic Framework for Balanced Trees" — FOCS 1978.
- Douglas Comer, "Ubiquitous B-Tree" — ACM Computing Surveys, 1979.
- Philip L. Lehman and S. Bing Yao, "Efficient Locking for Concurrent Operations on B-Trees" — ACM TODS, 1981.
- Mark T. O'Connell, "Purely Functional Data Structures" (1996 PhD thesis) — Cambridge University Press, 1998.
- Daniel D. Sleator and Robert E. Tarjan, "Self-Adjusting Binary Search Trees" — JACM, 1985.
- Jon Louis Bentley, "Multidimensional Binary Search Trees Used for Associative Searching" — CACM, 1975.
- Jon Louis Bentley, "K-D Trees for Semidynamic Nearest-Neighbor Queries" — CACM, 1979.

Graphs and Graph Algorithms

12.1 Introduction

❧ Complete compilable implementations of the data structures in this chapter are in `code/`.

12.2 The Graph ADT

A **graph** $G = (V, E)$ consists of a set V of **vertices** (nodes) and a set E of **edges** connecting pairs of vertices. Graphs model networks: social connections, road maps, circuit connectivity, web links, and more.

12.2.1 Terminology

- **Undirected graph:** edges have no direction — $(u, v) = (v, u)$
- **Directed graph (digraph):** edges have direction — $u \rightarrow v \neq v \rightarrow u$
- **Weighted graph:** each edge has a numerical weight
- **Path:** sequence of vertices connected by edges
- **Cycle:** a path where the first and last vertex are the same
- **Connected graph:** there is a path between every pair of vertices
- **Complete graph:** every pair of vertices is connected by an edge
- **Degree (undirected):** number of edges incident to a vertex
- **In-degree / Out-degree (directed):** number of incoming / outgoing edges

1	Undirected:	Directed:
2	A--B	A -> B
3	\	
4	C--D	C -> D

12.2.2 Graph Representations

Adjacency Matrix: $O(V^2)$ space, $O(1)$ edge lookup.

```

1      A B C D
2  A  0 1 1 1
3  B  1 0 0 1
4  C  1 0 0 1
5  D  1 1 1 0

```

Adjacency List: $O(V + E)$ space, $O(\text{degree}(v))$ neighbor iteration.

```

1 A  -> B  -> C  -> D
2 B  -> A  -> D
3 C  -> A  -> D
4 D  -> A  -> B  -> C

```

12.3 Graph Classes

12.3.1 Adjacency Matrix

```

1 class graph_matrix {
2 public:
3     graph_matrix(size_t n) : n_(n), adj_(n, std::vector<bool>(n, false))
4     {}
5
6     void add_edge(size_t u, size_t v) {
7         adj_[u][v] = true;
8         adj_[v][u] = true; // undirected
9     }
10
11     bool has_edge(size_t u, size_t v) const {
12         return adj_[u][v];
13     }
14
15     std::vector<size_t> neighbors(size_t v) const {
16         std::vector<size_t> result;
17         for (size_t i = 0; i < n_; ++i) {
18             if (adj_[v][i]) result.push_back(i);
19         }
20         return result;
21     }
22
23     size_t size() const noexcept { return n_; }
24 private:
25     size_t n_;
26     std::vector<std::vector<bool>> adj_;

```

```
27 };
```

12.3.2 Adjacency List

```
1 template <typename Weight = int>
2 class graph_list {
3 public:
4     struct edge {
5         size_t to;
6         Weight weight;
7     };
8
9     explicit graph_list(size_t n) : adj_(n) {}
10
11     void add_edge(size_t u, size_t v, Weight w = Weight{}) {
12         adj_[u].push_back({v, w});
13         adj_[v].push_back({u, w}); // undirected
14     }
15
16     const std::vector<edge>& neighbors(size_t v) const {
17         return adj_[v];
18     }
19
20     size_t size() const noexcept { return adj_.size(); }
21     size_t edge_count() const {
22         size_t e = 0;
23         for (const auto& adj : adj_) e += adj.size();
24         return e / 2; // undirected
25     }
26
27 private:
28     std::vector<std::vector<edge>> adj_;
29 };
```

12.4 Graph Traversal

12.4.1 Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking. Uses a stack (implicitly via recursion, or explicitly).

```
1 void dfs(const graph_list<>& g, size_t start, auto& visit) {
2     std::vector<bool> visited(g.size(), false);
3     std::function<void(size_t)> dfs_impl = [&](size_t v) {
4         visited[v] = true;
5         visit(v);
```

```

6         for (const auto& [to, _] : g.neighbors(v)) {
7             if (!visited[to]) dfs_impl(to);
8         }
9     };
10    dfs_impl(start);
11 }

```

Applications:

- Finding connected components
- Topological sort
- Detecting cycles
- Solving mazes
- Biconnectivity and articulation points

12.4.2 Breadth-First Search (BFS)

BFS explores all neighbors at the current depth before going deeper. Uses a queue.

```

1  std::vector<size_t> bfs(const graph_list<>& g, size_t start) {
2      std::vector<size_t> distance(g.size(), SIZE_MAX);
3      std::queue<size_t> q;
4
5      distance[start] = 0;
6      q.push(start);
7
8      while (!q.empty()) {
9          size_t v = q.front(); q.pop();
10         for (const auto& [to, _] : g.neighbors(v)) {
11             if (distance[to] == SIZE_MAX) {
12                 distance[to] = distance[v] + 1;
13                 q.push(to);
14             }
15         }
16     }
17     return distance;
18 }

```

Applications:

- Shortest path in unweighted graphs
- Finding connected components
- Web crawling
- Social network analysis (degrees of separation)

12.4.3 Connected Components

```

1  std::vector<size_t> connected_components(const graph_list<>& g) {
2      size_t n = g.size();
3      std::vector<size_t> component(n, SIZE_MAX);
4      size_t comp_id = 0;
5
6      for (size_t v = 0; v < n; ++v) {
7          if (component[v] != SIZE_MAX) continue;
8
9          // BFS from v
10         std::queue<size_t> q;
11         q.push(v);
12         component[v] = comp_id;
13
14         while (!q.empty()) {
15             size_t u = q.front(); q.pop();
16             for (const auto& [to, _] : g.neighbors(u)) {
17                 if (component[to] == SIZE_MAX) {
18                     component[to] = comp_id;
19                     q.push(to);
20                 }
21             }
22         }
23         ++comp_id;
24     }
25     return component;
26 }

```

12.5 Topological Sort

A **topological order** of a directed acyclic graph (DAG) is a linear ordering of vertices such that for every edge $u \rightarrow v$, u comes before v . Only DAGs have a topological order.

```

1  std::vector<size_t> topological_sort(const graph_list<>& g) {
2      size_t n = g.size();
3      std::vector<size_t> in_degree(n, 0);
4
5      // Compute in-degrees
6      for (size_t v = 0; v < n; ++v) {
7          for (const auto& [to, _] : g.neighbors(v)) {
8              ++in_degree[to];
9          }
10     }
11
12     // Kahn's algorithm
13     std::queue<size_t> q;

```

```

14     for (size_t v = 0; v < n; ++v) {
15         if (in_degree[v] == 0) q.push(v);
16     }
17
18     std::vector<size_t> result;
19     while (!q.empty()) {
20         size_t v = q.front(); q.pop();
21         result.push_back(v);
22         for (const auto& [to, _] : g.neighbors(v)) {
23             if (--in_degree[to] == 0) q.push(to);
24         }
25     }
26     return result;
27 }

```

Application: Course prerequisite ordering, build system dependency resolution.

12.6 Minimum Spanning Tree

A **spanning tree** connects all vertices of a graph with exactly $V-1$ edges. A **minimum spanning tree (MST)** is a spanning tree with minimum total edge weight.

12.6.1 Kruskal's Algorithm

Sort edges by weight, add edges that don't create cycles (using Union-Find).

```

1 struct weighted_edge {
2     size_t from, to;
3     int weight;
4 };
5
6 std::vector<weighted_edge> kruskal_mst(size_t n,
7                                       std::vector<weighted_edge> edges)
8 {
9     std::sort(edges.begin(), edges.end(),
10              [](const auto& a, const auto& b) { return a.weight < b.
11              weight; });
12
13     union_find uf(n);
14     std::vector<weighted_edge> mst;
15
16     for (const auto& e : edges) {
17         if (!uf.same_set(e.from, e.to)) {
18             uf.union_sets(e.from, e.to);
19             mst.push_back(e);
20             if (mst.size() == n - 1) break;
21         }
22     }
23 }

```

```

21     return mst;
22 }

```

Complexity: $O(E \log E)$ due to sorting. With Union-Find, cycle check is near-constant.

12.6.2 Prim's Algorithm

Grow the MST from a starting vertex, always adding the cheapest edge connecting the tree to a vertex outside it.

```

1  std::vector<weighted_edge> prim_mst(const graph_list<int>& g) {
2      size_t n = g.size();
3      std::vector<bool> in_mst(n, false);
4      std::vector<int> key(n, INT_MAX);
5      std::vector<size_t> parent(n, SIZE_MAX);
6
7      using entry = std::pair<int, size_t>;
8      std::priority_queue<entry, std::vector<entry>, std::greater<entry>> pq
9      ;
10
11     key[0] = 0;
12     pq.push({0, 0});
13
14     while (!pq.empty()) {
15         auto [_, v] = pq.top(); pq.pop();
16         if (in_mst[v]) continue;
17         in_mst[v] = true;
18
19         for (const auto& [to, w] : g.neighbors(v)) {
20             if (!in_mst[to] && w < key[to]) {
21                 key[to] = w;
22                 parent[to] = v;
23                 pq.push({w, to});
24             }
25         }
26
27         std::vector<weighted_edge> mst;
28         for (size_t v = 1; v < n; ++v) {
29             if (parent[v] != SIZE_MAX) {
30                 mst.push_back({parent[v], v, key[v]});
31             }
32         }
33         return mst;
34     }

```

Complexity: $O((V + E) \log V)$ with a binary heap.

12.7 Shortest Paths

12.7.1 Dijkstra's Algorithm

Finds shortest path from a source to all vertices in a graph with non-negative weights.

```

1  std::vector<int> dijkstra(const graph_list<int>& g, size_t source) {
2      size_t n = g.size();
3      std::vector<int> distance(n, INT_MAX);
4      std::vector<bool> visited(n, false);
5
6      using entry = std::pair<int, size_t>;
7      std::priority_queue<entry, std::vector<entry>, std::greater<entry>> pq
8      ;
9
10     distance[source] = 0;
11     pq.push({0, source});
12
13     while (!pq.empty()) {
14         auto [dist, v] = pq.top(); pq.pop();
15         if (visited[v]) continue;
16         visited[v] = true;
17
18         for (const auto& [to, w] : g.neighbors(v)) {
19             if (dist + w < distance[to]) {
20                 distance[to] = dist + w;
21                 pq.push({distance[to], to});
22             }
23         }
24     }
25     return distance;
26 }
```

Complexity: $O((V + E) \log V)$. Cannot handle negative edges.

12.7.2 Bellman-Ford Algorithm

Handles negative edge weights (detects negative cycles).

```

1  std::optional<std::vector<int>> bellman_ford(
2      const std::vector<weighted_edge>& edges, size_t n, size_t source) {
3
4      std::vector<int> distance(n, INT_MAX / 2);
5      distance[source] = 0;
6
7      // Relax all edges V-1 times
8      for (size_t i = 0; i < n - 1; ++i) {
9          for (const auto& [u, v, w] : edges) {
10             if (distance[u] + w < distance[v]) {
```



```

11         distance[v] = distance[u] + w;
12     }
13 }
14 }
15
16 // Check for negative cycles
17 for (const auto& [u, v, w] : edges) {
18     if (distance[u] + w < distance[v]) {
19         return std::nullopt; // negative cycle detected
20     }
21 }
22 return distance;
23 }

```

Complexity: $O(VE)$.

12.7.3 Floyd-Warshall (All Pairs)

```

1 std::vector<std::vector<int>>> floyd_warshall(
2     const std::vector<weighted_edge>& edges, size_t n) {
3
4     std::vector<std::vector<int>>> dist(n, std::vector<int>(n, INT_MAX / 2)
5 );
6     for (size_t i = 0; i < n; ++i) dist[i][i] = 0;
7     for (const auto& [u, v, w] : edges) dist[u][v] = w;
8
9     for (size_t k = 0; k < n; ++k) {
10         for (size_t i = 0; i < n; ++i) {
11             for (size_t j = 0; j < n; ++j) {
12                 if (dist[i][k] + dist[k][j] < dist[i][j]) {
13                     dist[i][j] = dist[i][k] + dist[k][j];
14                 }
15             }
16         }
17     }
18     return dist;
19 }

```

Complexity: $O(V^3)$. Simple but costly; use only for dense graphs or small V .

12.8 Strongly Connected Components

A **strongly connected component (SCC)** in a directed graph is a maximal set of vertices where every vertex is reachable from every other.

12.8.1 Kosaraju's Algorithm

```

1  std::vector<size_t> kosaraju_scc(const graph_list<>& g) {
2      size_t n = g.size();
3      std::vector<bool> visited(n, false);
4      std::vector<size_t> order;
5
6      // First pass: DFS to get finish order
7      std::function<void(size_t)> dfs1 = [&](size_t v) {
8          visited[v] = true;
9          for (const auto& [to, _] : g.neighbors(v)) {
10              if (!visited[to]) dfs1(to);
11          }
12          order.push_back(v);
13      };
14      for (size_t v = 0; v < n; ++v) if (!visited[v]) dfs1(v);
15
16      // Second pass: DFS on reversed graph
17      graph_list<> reversed(n);
18      for (size_t v = 0; v < n; ++v)
19          for (const auto& [to, _] : g.neighbors(v))
20              reversed.add_edge(to, v, 0);
21
22      std::vector<size_t> component(n, SIZE_MAX);
23      size_t comp_id = 0;
24      visited.assign(n, false);
25
26      std::function<void(size_t)> dfs2 = [&](size_t v) {
27          visited[v] = true;
28          component[v] = comp_id;
29          for (const auto& [to, _] : reversed.neighbors(v)) {
30              if (!visited[to]) dfs2(to);
31          }
32      };
33
34      for (auto it = order.rbegin(); it != order.rend(); ++it) {
35          if (!visited[*it]) {
36              dfs2(*it);
37              ++comp_id;
38          }
39      }
40      return component;
41  }

```

12.9 Application: Social Network Analysis

```

1  struct social_graph {
2      graph_list<> connections;

```

```

3     std::vector<std::string> names;
4
5     // Degrees of separation from a person
6     std::vector<size_t> degrees_of_separation(const std::string& person) {
7         auto it = std::find(names.begin(), names.end(), person);
8         if (it == names.end()) return {};
9         return bfs(connections, it - names.begin());
10    }
11
12    // Friend recommendations (friends of friends not already connected)
13    std::vector<std::string> recommendations(const std::string& person) {
14        auto it = std::find(names.begin(), names.end(), person);
15        if (it == names.end()) return {};
16        size_t idx = it - names.begin();
17
18        std::unordered_set<size_t> friends;
19        for (const auto& [to, _] : connections.neighbors(idx)) {
20            friends.insert(to);
21        }
22
23        std::unordered_set<size_t> candidates;
24        for (size_t f : friends) {
25            for (const auto& [to, _] : connections.neighbors(f)) {
26                if (to != idx && !friends.count(to)) {
27                    candidates.insert(to);
28                }
29            }
30        }
31
32        std::vector<std::string> result;
33        for (size_t c : candidates) result.push_back(names[c]);
34        return result;
35    }
36 };

```

12.10 Articulation Points and Bridges

An **articulation point** (cut vertex) is a vertex whose removal disconnects the graph. A **bridge** (cut edge) is an edge whose removal disconnects the graph. Finding these is critical for analyzing network reliability.

Library implementation: `graph.h` exposes `g.articulation_points()` and `g.bridges()` for undirected simple graphs.

12.10.1 DFS Tree

During DFS, we build a tree of discovery edges. A vertex u is an articulation point if:

- u is the root of the DFS tree and has ≥ 2 children, OR
- u is not the root and has a child v such that no vertex in v 's subtree has a back edge to an ancestor of u .

The **low value** captures this: $\text{low}[u]$ = the earliest (minimum discovery time) ancestor reachable from u via a back edge in its subtree.

12.10.2 Implementation

```

1 void find_articulation_points(const graph_list<>& g, int n,
2     std::vector<bool>& is_ap) {
3     std::vector<int> disc(n, -1), low(n, -1);
4     int timer = 0;
5     is_ap.assign(n, false);
6
7     auto dfs = [&](auto&& self, int u, int parent) -> void {
8         disc[u] = low[u] = timer++;
9         int children = 0;
10
11         for (auto [v, _] : g.neighbors(u)) {
12             if (disc[v] == -1) {
13                 ++children;
14                 self(self, v, u);
15                 low[u] = std::min(low[u], low[v]);
16
17                 // u is root with 2+ children
18                 if (parent == -1 && children > 1) is_ap[u] = true;
19                 // u is not root and no back edge above u
20                 if (parent != -1 && low[v] >= disc[u]) is_ap[u] = true;
21             } else if (v != parent) {
22                 low[u] = std::min(low[u], disc[v]);
23             }
24         }
25     };
26
27     for (int i = 0; i < n; ++i)
28         if (disc[i] == -1) dfs(dfs, i, -1);
29 }
30
31 void find_bridges(const graph_list<>& g, int n,
32     std::vector<std::pair<int, int>>& bridges) {
33     std::vector<int> disc(n, -1), low(n, -1);
34     int timer = 0;
35
36     auto dfs = [&](auto&& self, int u, int parent) -> void {
37         disc[u] = low[u] = timer++;
38         for (auto [v, _] : g.neighbors(u)) {

```

```

39         if (disc[v] == -1) {
40             self(self, v, u);
41             low[u] = std::min(low[u], low[v]);
42             if (low[v] > disc[u])
43                 bridges.emplace_back(u, v);
44             } else if (v != parent) {
45                 low[u] = std::min(low[u], disc[v]);
46             }
47         }
48     };
49
50     for (int i = 0; i < n; ++i)
51         if (disc[i] == -1) dfs(dfs, i, -1);
52 }

```

Complexity: $O(V + E)$ — single DFS pass.

12.10.3 Applications

- **Network reliability:** identify routers whose failure partitions the network
- **Social networks:** bridge connections between communities (structural holes)
- **Internet infrastructure:** critical links in AS-level topology

12.11 Eulerian Path and Circuit

An **Eulerian circuit** visits every edge exactly once and returns to the start. An **Eulerian path** visits every edge exactly once (start $\neq$ end).

12.11.1 Hierholzer's Algorithm

Build the circuit greedily: start from a vertex, follow unused edges until you return to the start. If there are unused edges remaining, find a vertex on the current circuit with unused edges, start a sub-circuit, and splice it in.

```

1  std::vector<int> eulerian_circuit(std::vector<std::list<int>>& adj, int n)
2  {
3      std::vector<int> circuit;
4      std::stack<int> stk;
5      stk.push(0);
6
7      while (!stk.empty()) {
8          int u = stk.top();
9          if (!adj[u].empty()) {
10             int v = adj[u].front();
11             adj[u].pop_front();

```

```

11         // Remove reverse edge
12         adj[v].remove(u);
13         stk.push(v);
14     } else {
15         circuit.push_back(u);
16         stk.pop();
17     }
18 }
19 std::reverse(circuit.begin(), circuit.end());
20 return circuit;
21 }

```

Complexity: $O(V + E)$.

12.11.2 Eulerian Path Conditions

- **Undirected graph:** Eulerian circuit iff every vertex has even degree. Eulerian path iff exactly 0 or 2 vertices have odd degree.
- **Directed graph:** Eulerian circuit iff every vertex has in-degree = out-degree. Eulerian path iff at most one vertex has out-degree - in-degree = 1, at most one has in-degree - out-degree = 1, all others are balanced.

12.11.3 Applications

- **DNA sequencing:** De Bruijn graph assembly (Illumina, PacBio) — find Eulerian path through k-mer graph
- **Route optimization:** garbage collection, mail delivery, snow plowing
- **Circuit design:** printed circuit board (PCB) drilling — minimize tool travel
- **Pen drawing:** draw a graph without lifting the pen

12.12 A* Search

Library implementation: `a_star.h` provides `dsa::a_star(g, start, goal, heuristic)` with optional path reconstruction.

A* is Dijkstra's algorithm with a heuristic function that estimates the distance to the goal, guiding the search toward promising vertices.

12.12.1 The Idea

A* maintains a priority queue ordered by $f(v) = g(v) + h(v)$ where:

- $g(v)$ = actual cost from start to v

- $h(v)$ = estimated cost from v to goal (heuristic)

A heuristic is **admissible** if it never overestimates: $h(v) \leq h^*(v)$ (true cost to goal). An admissible heuristic guarantees A* finds the optimal path.

12.12.2 Implementation

```

1 struct astar_result {
2     std::vector<int> path;
3     int cost;
4 };
5
6 astar_result astar(const graph_list<>& g, int start, int goal,
7     std::function<int(int, int)> heuristic, int n) {
8     std::vector<int> dist(n, INT_MAX), parent(n, -1);
9     std::vector<bool> closed(n, false);
10    // Min-heap: (f_cost, vertex)
11    std::priority_queue<std::pair<int, int>,
12        std::vector<std::pair<int, int>>, std::greater<>> open;
13
14    dist[start] = 0;
15    open.push({heuristic(start, goal), start});
16
17    while (!open.empty()) {
18        auto [f, u] = open.top();
19        open.pop();
20
21        if (u == goal) break;
22        if (closed[u]) continue;
23        closed[u] = true;
24
25        for (auto [v, w] : g.neighbors(u)) {
26            int new_g = dist[u] + w;
27            if (new_g < dist[v]) {
28                dist[v] = new_g;
29                parent[v] = u;
30                open.push({new_g + heuristic(v, goal), v});
31            }
32        }
33    }
34
35    // Reconstruct path
36    astar_result result;
37    result.cost = dist[goal];
38    for (int v = goal; v != -1; v = parent[v])
39        result.path.push_back(v);
40    std::reverse(result.path.begin(), result.path.end());
41    return result;

```

42 }

Complexity: $O(E)$ with a good heuristic. Worst case (bad heuristic or $h = 0$) = Dijkstra's: $O((V + E) \log V)$.

12.12.3 Common Heuristics

- **Manhattan distance:** $|x_1 - x_2| + |y_1 - y_2|$ (grid, no diagonal movement)
- **Euclidean distance:** $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ (free movement)
- **Chebyshev distance:** $\max(|x_1 - x_2|, |y_1 - y_2|)$ (grid with diagonal movement)

12.12.4 Applications

- **GPS navigation:** Google Maps, Waze, Apple Maps use A* variants for route planning
- **Game AI:** pathfinding for NPCs in Unity, Unreal, Godot
- **Robotics:** Amazon Kiva warehouse robots, autonomous vehicle navigation
- **Puzzle solving:** 8-puzzle, 15-puzzle, Rubik's cube

12.13 Graph Coloring

Graph coloring assigns colors (labels) to vertices so that no two adjacent vertices share the same color. The minimum number of colors needed is the **chromatic number** $\chi(G)$.

12.13.1 Greedy Coloring

A simple greedy algorithm processes vertices in order and assigns the smallest available color:

```

1 std::vector<int> greedy_color(const graph_list<>& g, int n) {
2     std::vector<int> color(n, -1);
3     color[0] = 0;
4
5     std::vector<bool> available(n, false);
6
7     for (int u = 1; u < n; ++u) {
8         // Mark colors of adjacent vertices as unavailable
9         for (auto [v, _] : g.neighbors(u))
10             if (color[v] != -1) available[color[v]] = true;
11
12         // Find first available color
13         int c = 0;
14         while (available[c]) ++c;
15         color[u] = c;
16     }
17 }
```



```

17         // Reset available array
18         for (auto [v, _] : g.neighbors(u))
19             if (color[v] != -1) available[color[v]] = false;
20     }
21     return color;
22 }

```

Complexity: $O(V + E)$. The greedy algorithm uses at most $\Delta(G) + 1$ colors where $\Delta(G)$ is the maximum degree.

12.13.2 Welsh-Powell Ordering

Process vertices in descending order of degree for better coloring:

```

1  std::vector<int> welsh_powell_color(const graph_list<>& g, int n) {
2      // Sort vertices by degree (descending)
3      std::vector<int> order(n);
4      std::iota(order.begin(), order.end(), 0);
5      std::sort(order.begin(), order.end(), [&](int a, int b) {
6          return g.degree(a) > g.degree(b);
7      });
8
9      std::vector<int> color(n, -1);
10     std::vector<bool> available(n, false);
11
12     for (int u : order) {
13         for (auto [v, _] : g.neighbors(u))
14             if (color[v] != -1) available[color[v]] = true;
15         int c = 0;
16         while (available[c]) ++c;
17         color[u] = c;
18         for (auto [v, _] : g.neighbors(u))
19             if (color[v] != -1) available[color[v]] = false;
20     }
21     return color;
22 }

```

12.13.3 Applications

- **Compiler register allocation:** variables interfering (live at same time) need different registers — graph coloring maps variables to registers
- **Frequency assignment:** cell tower channels, WiFi channel selection
- **Exam scheduling:** no student has overlapping exams
- **Map coloring:** the original Four Color Theorem problem
- **Sudoku:** each cell constrains its row, column, and box neighbors

12.14 Hypergraph

A **hypergraph** generalizes a graph by allowing edges (called **hyperedges**) to connect any number of vertices — not just two. While a standard edge is a 2-element subset of V , a hyperedge is an arbitrary subset of V . Hypergraphs arise naturally when modeling relationships that involve more than two entities simultaneously: database table joins, chemical reactions, social groups, and constraint satisfaction problems.

12.14.1 Structure

```

1 Hypergraph with 4 vertices, 3 hyperedges:
2
3     e1 = {A, B, C}
4     e2 = {B, C, D}
5     e3 = {A, D}
6
7         A-----B
8         | \   e1 / |
9         |  \   /  |
10        |   \ /   |
11        |    X    |      e1 connects A, B, C
12        |   / \   |      e2 connects B, C, D
13        |  /   \  |      e3 connects A, D (standard edge)
14        | /    e2 \ |
15        D-----C
16
17     Incidence matrix:
18           e1  e2  e3
19     A [   1   0   1 ]
20     B [   1   1   0 ]
21     C [   1   1   0 ]
22     D [   0   1   1 ]
23
24     M[v][e] = 1 iff vertex v belongs to hyperedge e

```

12.14.2 C++ Implementation

```

1 struct Hypergraph {
2     int n; // number of vertices
3     vector<int> vertices; // vertex IDs: 0..n-1
4     vector<vector<int>> edges; // each hyperedge is a set of vertex
5                               IDs
6
7     Hypergraph(int n) : n(n) {
8         for (int i = 0; i < n; i++) vertices.push_back(i);
9     }
10 };

```

```

8      }
9
10     void addEdge(const vector<int>& hyperedge) {
11         edges.push_back(hyperedge);
12     }
13
14     // Degree of a vertex: number of hyperedges containing it
15     int degree(int v) const {
16         int deg = 0;
17         for (const auto& e : edges)
18             if (find(e.begin(), e.end(), v) != e.end())
19                 deg++;
20         return deg;
21     }
22
23     // Size of a hyperedge: number of vertices it connects
24     int edgeSize(int eIdx) const {
25         return edges[eIdx].size();
26     }
27
28     // Incidence matrix as 2D vector
29     vector<vector<int>> incidenceMatrix() const {
30         vector<vector<int>> M(n, vector<int>(edges.size(), 0));
31         for (int e = 0; e < (int)edges.size(); e++)
32             for (int v : edges[e])
33                 M[v][e] = 1;
34         return M;
35     }
36
37     // Two vertices are adjacent if they share at least one hyperedge
38     bool adjacent(int u, int v) const {
39         for (const auto& e : edges) {
40             bool hasU = find(e.begin(), e.end(), u) != e.end();
41             bool hasV = find(e.begin(), e.end(), v) != e.end();
42             if (hasU && hasV) return true;
43         }
44         return false;
45     }
46
47     // k-uniform: all hyperedges have exactly k vertices
48     bool isKUniform() const {
49         if (edges.empty()) return true;
50         int k = edges[0].size();
51         for (const auto& e : edges)
52             if ((int)e.size() != k) return false;
53         return true;
54     }
55 };

```

12.14.3 Applications

- **Relational database joins:** a join of k tables is a hyperedge connecting k relation schemas — query optimization plans are hypergraphs
- **Social groups:** a study group, a committee, or a chat group links multiple people simultaneously — a hyperedge per group
- **Constraint satisfaction:** each constraint involves a subset of variables — the scope of the constraint forms a hyperedge
- **Chemical reactions:** reactants and products are sets of molecules connected by a single reaction (hyperedge)
- **VLSI design:** netlists where a single net connects 3+ pins are naturally hypergraphs

12.14.4 Complexity

- Space: $O(V + E \cdot k_{\max})$ where $k_{\max}$ is the largest hyperedge size
- Incidence matrix: $O(V \cdot E)$ space — feasible only for small instances
- Adjacency check: $O(E \cdot k_{\max})$ — iterate all hyperedges
- Degree computation: $O(E \cdot k_{\max})$ per vertex
- Converting a hypergraph to a standard bipartite graph (vertex-edge bipartite representation): $O(V + E \cdot k_{\max})$

12.15 Binary Decision Diagrams

A **Binary Decision Diagram** (BDD) is a directed acyclic graph that represents a Boolean function. Each internal node tests a variable; the two outgoing edges correspond to that variable being 0 (low) or 1 (high). Terminal nodes are labeled 0 or 1. A **Reduced Ordered BDD** (ROBDD) enforces a fixed variable ordering and applies two reduction rules — making it a canonical representation: two Boolean functions are equivalent iff their ROBDDs are identical.

12.15.1 Structure

```

1 ROBDD for f = (x1 AND x2) OR (x3 AND x4)
2
3 Variable ordering: x1 < x2 < x3 < x4
4
5           x1
6         /  \
7         0   x2

```

```

8           / \
9          0  x3
10         / \
11        0  x4
12       / \
13      0  1
14
15 Nodes tested top-down: x1, x2, x3, x4
16 Each 0-edge goes left, 1-edge goes right
17 Terminal 1 reached only via path: x1=1, x2=1
18                                or x3=1, x4=1

```

12.15.2 C++ Implementation

```

1 struct BDDNode {
2     int var;           // variable index (0-based), -1 for terminal
3     int low;           // child index when var = 0
4     int high;          // child index when var = 1
5     bool operator==(const BDDNode& o) const {
6         return var == o.var && low == o.low && high == o.high;
7     }
8 };
9
10 class BDD {
11     vector<BDDNode> nodes;
12     map<tuple<int,int,int>, int> uniqueTable; // (var, low, high) -> node
13                                             // index
14     int terminal0, terminal1;
15
16     int getOrMakeNode(int var, int low, int high) {
17         if (low == high) return low; // reduction rule: merge
18                                     // identical children
19         auto key = make_tuple(var, low, high);
20         auto it = uniqueTable.find(key);
21         if (it != uniqueTable.end()) return it->second;
22         int idx = nodes.size();
23         nodes.push_back({var, low, high});
24         uniqueTable[key] = idx;
25         return idx;
26     }
27
28 public:
29     BDD() {
30         terminal0 = 0; // node 0: terminal 0
31         terminal1 = 1; // node 1: terminal 1
32         nodes.push_back({-1, -1, -1}); // placeholder for terminal 0
33         nodes.push_back({-1, -1, -1}); // placeholder for terminal 1
34     }
35 };

```

```

32     }
33
34     // ITE (if-then-else): builds f = ite(G, H, E)
35     //   if G=1 then H else E
36     int ite(int G, int H, int E) {
37         if (G == terminal1) return H;
38         if (G == terminal0) return E;
39         if (H == terminal1 && E == terminal0) return G; // f = G
40         if (H == terminal0 && E == terminal1) {
41             // f = NOT G -- create a new node or find existing
42             return getOrMakeNode(nodes[G].var,
43                                   ite(nodes[G].low, terminal0, terminal1),
44                                   ite(nodes[G].high, terminal0, terminal1));
45         }
46         // General case: recursive construction
47         int v = nodes[G].var; // highest-order variable among G,H,E
48         int low = ite(nodes[G].low, nodes[H].low, nodes[E].low);
49         int high = ite(nodes[G].high, nodes[H].high, nodes[E].high);
50         return getOrMakeNode(v, low, high);
51     }
52
53     // Build AND: f = G AND H
54     int AND(int G, int H) {
55         return ite(G, H, terminal0);
56     }
57
58     // Build OR: f = G OR H
59     int OR(int G, int H) {
60         return ite(G, terminal1, H);
61     }
62
63     // Build NOT: f = NOT G
64     int NOT(int G) {
65         return ite(G, terminal0, terminal1);
66     }
67
68     // Evaluate BDD for a given variable assignment
69     bool evaluate(int node, const vector<bool>& assignment) const {
70         if (node == terminal0) return false;
71         if (node == terminal1) return true;
72         return assignment[nodes[node].var]
73             ? evaluate(nodes[node].high, assignment)
74             : evaluate(nodes[node].low, assignment);
75     }
76
77     // Check equivalence of two BDDs
78     static bool equivalent(const BDD& a, const BDD& b, int na, int nb) {

```

```

79         if (na == nb) return true;
80         if (a.nodes[na].var != b.nodes[nb].var) return false;
81         return equivalent(a, b, a.nodes[na].low, b.nodes[nb].low)
82             && equivalent(a, b, a.nodes[na].high, b.nodes[nb].high);
83     }
84
85     int nodeCount() const { return nodes.size(); }
86     int getTerminal1() const { return terminal1; }
87     const BDDNode& getNode(int idx) const { return nodes[idx]; }
88 };

```

12.15.3 Applications

- **Formal verification:** two circuits are equivalent iff their ROBDDs are identical — one canonical form avoids comparing exponentially many truth table rows
- **Model checking:** BDDs compactly represent state transition relations for finite-state systems
- **SAT solving:** BDD-based SAT solvers enumerate satisfying assignments by traversing the diagram
- **Synthesis:** logic synthesis tools use BDDs to find minimal circuit implementations
- **Function analysis:** counting satisfying assignments is $O(|\text{BDD}|)$ — just sum the paths to terminal 1

12.15.4 Complexity

- Worst-case BDD size for n variables: $O(2^n/n)$ — exponential in general
- Average case for “random” functions: still exponential
- ROBDD is canonical: same function + same variable ordering $\Rightarrow$ same BDD
- ITE operation: $O(|f| \cdot |g| \cdot |e|)$ worst case with memoization
- Equivalence check: $O(|f|)$ — just check if the two root nodes are the same node
- Variable ordering matters critically: a bad ordering can make the BDD exponentially larger than an optimal one

12.16 Winged Edge

A **winged edge** (also called a **balanced edge** or **winged half-edge**) is a mesh data structure that stores explicit connectivity between vertices, edges, and faces. Each edge record contains pointers to its two endpoint vertices, the two adjacent faces, and four neighboring edges (two clockwise and two

counterclockwise). This enables $O(1)$ traversal of mesh topology: from any edge, you can reach any adjacent vertex, face, or neighboring edge in constant time.

12.16.1 Structure

```

1 Winged edge record for edge e in a triangle mesh:
2
3 Vertices:      v1, v2          (endpoints)
4 Faces:         f1, f2          (adjacent faces, f2 = boundary if only one
5                               )
6 Edge links:    eCW1, eCCW1      (edges CW/CCW around f1 from e)
7               eCW2, eCCW2      (edges CW/CCW around f2 from e)
8
9               f1
10              /  \
11             eCCW1 eCW1
12            /      \
13           v1 ---e--- v2
14            \      /
15             eCW2  eCCW2
16            \      /
17             f2
18
19 Around face f1 (clockwise): ... -> eCCW1 -> e -> eCW1 -> ...
20 Around face f2 (clockwise): ... -> eCW2 -> e -> eCCW2 -> ...

```

12.16.2 C++ Implementation

```

1 struct WEVertex {
2     double x, y, z;           // position
3     int edgeIdx;              // one incident edge (any)
4 };
5
6 struct WEFace {
7     int edgeIdx;              // one edge of this face (any)
8     int valence;              // number of edges/vertices in face (3 for
9                               // triangles)
10 };
11
12 struct WingedEdge {
13     int v1, v2;               // endpoint vertex indices
14     int f1, f2;               // adjacent face indices (f2 = -1 for
15                               // boundary)
16     int eCW1, eCCW1;          // edges CW/CCW around f1
17     int eCW2, eCCW2;          // edges CW/CCW around f2
18 };

```



```

17
18 class WingedEdgeMesh {
19     vector<WEVertex> verts;
20     vector<WEFace> faces;
21     vector<WingedEdge> edges;
22
23 public:
24     int addVertex(double x, double y, double z) {
25         int idx = verts.size();
26         verts.push_back({x, y, z, -1});
27         return idx;
28     }
29
30     // Add a directed edge; caller must supply face-adjacent edge links
31     int addEdge(int v1, int v2, int f1, int f2,
32                int eCW1, int eCCW1, int eCW2, int eCCW2) {
33         int idx = edges.size();
34         edges.push_back({v1, v2, f1, f2, eCW1, eCCW1, eCW2, eCCW2});
35         if (verts[v1].edgeIdx == -1) verts[v1].edgeIdx = idx;
36         if (verts[v2].edgeIdx == -1) verts[v2].edgeIdx = idx;
37         return idx;
38     }
39
40     // Get the other vertex of an edge (not v)
41     int otherVertex(int edgeIdx, int v) const {
42         const auto& e = edges[edgeIdx];
43         return (e.v1 == v) ? e.v2 : e.v1;
44     }
45
46     // Get the opposite face of an edge (not f)
47     int otherFace(int edgeIdx, int f) const {
48         const auto& e = edges[edgeIdx];
49         return (e.f1 == f) ? e.f2 : e.f1;
50     }
51
52     // Walk clockwise around a face, collecting edges
53     vector<int> faceEdges(int faceIdx) const {
54         vector<int> result;
55         int startEdge = faces[faceIdx].edgeIdx;
56         int curr = startEdge;
57         do {
58             result.push_back(curr);
59             const auto& e = edges[curr];
60             if (e.f1 == faceIdx)
61                 curr = e.eCW1;
62             else
63                 curr = e.eCW2;
64         } while (curr != startEdge && curr != -1);

```

```

65         return result;
66     }
67
68     // Walk edges incident to a vertex (one ring)
69     vector<int> vertexEdges(int vertIdx) const {
70         vector<int> result;
71         int startEdge = verts[vertIdx].edgeIdx;
72         if (startEdge == -1) return result;
73         int curr = startEdge;
74         do {
75             result.push_back(curr);
76             int nextV = otherVertex(curr, vertIdx);
77             // find an edge around nextV that brings us back
78             const auto& e = edges[curr];
79             int nextEdge = -1;
80             if (e.f1 != -1 && e.f2 != -1) {
81                 // interior edge: go to opposite face, take the other edge
82                 there
83                 if (e.f1 == faces[curr].edgeIdx)
84                     nextEdge = e.eCCW2;
85                 else
86                     nextEdge = e.eCCW1;
87             } else {
88                 // boundary: reverse direction
89                 nextEdge = (e.f1 != -1) ? e.eCCW1 : e.eCCW2;
90             }
91             curr = nextEdge;
92         } while (curr != startEdge && curr != -1);
93         return result;
94     }
95
96     int numVertices() const { return verts.size(); }
97     int numEdges()    const { return edges.size(); }
98     int numFaces()    const { return faces.size(); }
99     const WEVertex& vertex(int i) const { return verts[i]; }
100    const WEFace&    face(int i)    const { return faces[i]; }
101    const WingedEdge& edge(int i) const { return edges[i]; }
102 };

```

12.16.3 Comparison with Other Representations

- **Vertex list + face list:** simple but requires $O(E)$ search to find adjacent edges — no constant-time traversal
- **Half-edge:** each directed half-edge stores one face, one vertex, and three links (next, twin, face) — cleaner API, same $O(1)$ traversal, preferred in practice
- **Winged edge:** stores full connectivity in one record (4 edge links) — more memory per edge

but symmetric, easy to implement boundary handling

- **Corner table:** compact $O(F)$ storage, good for GPU rendering, but traversal requires index arithmetic

12.16.4 Complexity

- Space: $O(V + E + F)$ — Euler's formula gives $F = O(E)$ for planar meshes
- Face traversal: $O(F_{\text{face}})$ where F_{face} is the face valence — $O(1)$ per edge visited
- Vertex ring traversal: $O(\text{valence})$ — proportional to the number of incident edges
- Edge flip / split: $O(1)$ with winged edge — just relink the four edge pointers of the affected edges
- Insertion of a new vertex inside a face: $O(1)$ — split the face into three by linking to the new edges

12.17 Biconnected Components

A **biconnected component** (BCC) of an undirected graph is a maximal set of edges such that removing any single vertex (not the edge itself) from the subgraph does not disconnect it. Equivalently, a biconnected component is either a single bridge edge or a maximal 2-vertex-connected subgraph. **Articulation points** are the vertices that separate different biconnected components.

12.17.1 Key Observations

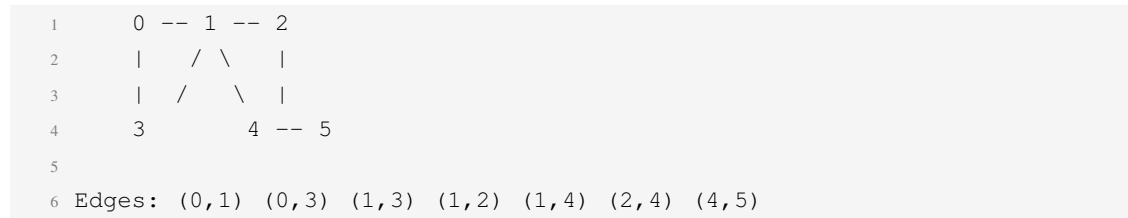
- Every edge belongs to exactly one biconnected component.
- Two biconnected components share at most one vertex — an articulation point.
- A bridge forms a biconnected component by itself.
- A graph with no articulation points is itself biconnected.

12.17.2 Algorithm

The algorithm uses a single DFS pass with discovery times and low-link values. As we traverse tree edges, we push them onto a stack. When we find an articulation point (a vertex u whose child v satisfies $\text{low}[v] \geq \text{disc}[u]$), we pop edges from the stack up to and including the tree edge (u, v) — those edges form one biconnected component.

12.17.3 Worked Trace

Consider the following graph with 6 vertices:



DFS from vertex 0, visiting neighbors in ascending label order:

Step	Vertex	disc / low	Action
1	0	disc[0]=0, low[0]=0	Push (0,1), visit 1
2	1	disc[1]=1, low[1]=1	Push (1,2), visit 2
3	2	disc[2]=2, low[2]=2	Push (2,4), visit 4
4	4	disc[4]=3, low[4]=3	Push (4,5), visit 5
5	5	disc[5]=4, low[5]=4	No unvisited neighbors low[4] = min(3, 4) = 3 low[2] = min(2, 3) = 2 low[4] $\neq$ disc[2] (3 $\neq$ 2): NOT ap
6	4		Back to 4: push (4,1)? Check neighbor 1: back edge, low[4] = min(3, 1) = 1
7	1		low[2] = min(2, 1) = 1 Check neighbor 3: tree edge, push (1,3), visit 3
8	3	disc[3]=5, low[3]=5	Check neighbor 0: back edge, low[3] = min(5, 0) = 0 Check neighbor 1: back edge (already visited parent)
9	1		low[3] = 0 low[1] = min(1, 0) = 0 Check neighbor 4: already visited Check neighbor 0: back edge, low[1] = min(0, 0) = 0
10	0		low[1] = 0 low[0] = min(0, 0) = 0

Articulation points: Vertex 1 (low[2] = 2 $\geq$ disc[1] = 1). When we return from child 2 to 1, we pop edges (1,2), (2,4), (4,5) as one component. Component BCC-1 = {(1,2), (2,4), (4,5)}.

Vertex 1 is also the root with children 0 and 3: component BCC-2 = {(0,1), (0,3), (1,3)}.

Final decomposition: BCC-1 = {(1,2), (2,4), (4,5)}, BCC-2 = {(0,1), (0,3), (1,3)}. Articulation point: {1}.

12.17.4 C++ Implementation

```

1 void biconnected_components(const graph_list<>& g, int n,
2     std::vector<std::vector<std::pair<int,int>>>& components) {
3     std::vector<int> disc(n, -1), low(n, -1);
4     int timer = 0;
5     std::stack<std::pair<int,int>> stk;
6
7     auto dfs = [&](auto&& self, int u, int parent) -> void {
8         disc[u] = low[u] = timer++;
9         int children = 0;
10
11         for (auto [v, _] : g.neighbors(u)) {
12             if (disc[v] == -1) {
13                 ++children;
14                 stk.push({u, v});
15                 self(self, v, u);
16                 low[u] = std::min(low[u], low[v]);
17
18                 // Articulation point: pop one BCC
19                 if ((parent == -1 && children > 1) ||
20                     (parent != -1 && low[v] >= disc[u])) {
21                     components.emplace_back();
22                     while (true) {
23                         auto e = stk.top(); stk.pop();
24                         components.back().push_back(e);
25                         if (e == std::make_pair(u, v)) break;
26                     }
27                 }
28                 else if (v != parent && disc[v] < disc[u]) {
29                     // Back edge to ancestor (only push once)
30                     stk.push({u, v});
31                     low[u] = std::min(low[u], disc[v]);
32                 }
33             }
34         };
35
36         for (int i = 0; i < n; ++i)
37             if (disc[i] == -1) {
38                 dfs(dfs, i, -1);
39                 // Remaining edges on the stack form the last BCC
40                 if (!stk.empty()) {
41                     components.emplace_back();
42                     while (!stk.empty()) {
43                         components.back().push_back(stk.top());
44                         stk.pop();
45                     }
46                 }
47             }
48     }

```

Complexity: $O(V + E)$ – single DFS pass. Each edge is pushed and popped from the stack at most once.

12.17.5 Applications

- **Network reliability:** identify subnetworks that remain connected if any single node fails
- **Sparse graph connectivity:** efficiently decompose into 2-connected pieces for further analysis
- **Planarity testing:** Hopcroft and Tarjan’s planarity algorithm uses BCC decomposition as a sub-routine

12.18 Strongly Connected Components – Kosaraju (Worked Trace)

The code for Kosaraju’s algorithm appears earlier in this chapter (see Section 12.8). Here we walk through the algorithm in detail on a concrete graph to solidify understanding.

12.18.1 The Graph

```

1 Directed graph with 6 vertices:
2
3     0 -> 1 -> 2 -> 0      (SCC-1: {0,1,2})
4     2 -> 3
5     3 -> 4 -> 5 -> 3      (SCC-2: {3,4,5})
6
7 Edges: 0->1 1->2 2->0 2->3 3->4 4->5 5->3

```

12.18.2 Pass 1: DFS Finish Order

Run DFS on the original graph from vertex 0, visiting neighbors in ascending order. Record the order in which vertices *finish* (all descendants processed).

Step	Vertex	Neighbors visited	Finish order
1	0	1 (tree edge)	
2	1	2 (tree edge)	
3	2	0 (back, visited), 3 (tree edge)	
4	3	4 (tree edge)	
5	4	5 (tree edge)	
6	5	3 (back, visited)	finish 5
			finish 4
			finish 3
			finish 2
			finish 1
			finish 0

Finish order (last to finish first): 0, 1, 2, 3, 4, 5.

Processing vertices in reverse finish order: 5, 4, 3, 2, 1, 0.

12.18.3 Pass 2: DFS on Transposed Graph

Reverse all edges:

```

1 Transposed graph:
2     1 -> 0
3     2 -> 1
4     0 -> 2
5     3 -> 2
6     4 -> 3
7     5 -> 4
8     3 -> 5

```

Now run DFS on the transposed graph, processing vertices in order: 5, 4, 3, 2, 1, 0.

Start from	DFS visits	SCC assigned
5	5, 4, 3	SCC-0: {3, 4, 5}
2	2, 1, 0	SCC-1: {0, 1, 2}

When we start DFS from 5 on the transposed graph, we follow $5 \rightarrow 4 \rightarrow 3$. From 3, the only outgoing edge goes to 2, but 2 is not yet visited... however, in the transposed graph, 3 has edges to 2 and 5. Since 5 was already visited, we continue. Vertex 2 is reached from 3 only if it has an incoming edge from 3, which it does: $3 \rightarrow 2$. But 2 is processed later in the vertex order (before 0 and 1), so when we start from 2, we find 1 and 0.

Result: Two SCCs: {0, 1, 2} and {3, 4, 5}.

12.18.4 Why It Works

Kosaraju’s algorithm exploits the structure of the **condensation graph** (the DAG of SCCs). In the condensation DAG, all edges go from “later” SCCs to “earlier” SCCs in the topological order. Running DFS on the original graph in pass 1 produces a finish order that respects this topological order. In pass 2, processing vertices in reverse finish order on the transposed graph ensures each DFS discovers exactly one SCC, because the transposed edges cannot cross SCC boundaries in the “wrong” direction.

12.18.5 Complexity

- Time: $O(V + E)$ – two DFS passes plus $O(V + E)$ to build the transpose.
- Space: $O(V + E)$ – visited array, finish order, transposed graph.

12.19 Bridges in a Graph (Worked Trace)

The bridge-finding code appears earlier (see Section 12.10). Here we trace the low-link DFS on a small graph to show exactly when the condition $\text{low}[v] > \text{disc}[u]$ fires.

12.19.1 The Graph

```

1      0 -- 1 -- 2 -- 3
2          |           |
3          4 -- 5 -- 6
4
5 Edges: (0,1) (1,2) (2,3) (3,6) (6,5) (5,4) (4,1)
```

This graph has two bridges: $(0, 1)$ and $(2, 3)$.

12.19.2 DFS Trace

DFS from vertex 0, neighbors in ascending order:

Step	Vertex	disc / low	Tree edge pushed	Bridge check	Bridges found
1	0	disc=0, low=0	(0,1)		
2	1	disc=1, low=1	(1,2)		
3	2	disc=2, low=2	(2,3)		
4	3	disc=3, low=3	(3,6)		
5	6	disc=4, low=4	(6,5)		
6	5	disc=5, low=5	(5,4)		
7	4	disc=6, low=6			
		neighbor 1: back edge	low[4] = min(6, 1) = 1		
		return to 5: low[5] = min(5, 1) = 1			
		return to 6: low[6] = min(4, 1) = 1			
		return to 3: low[3] = min(3, 1) = 1			
		check: low[6]=1 < disc[3]=3? NO			
8	2	low[2] = min(2, 1) = 1			
		return to 1: low[1] = min(1, 1) = 1			
		check: low[2]=1 < disc[1]=1? NO			
9	0	low[0] = min(0, 1) = 0			
		check: low[1]=1 < disc[0]=0? YES		(0, 1)	

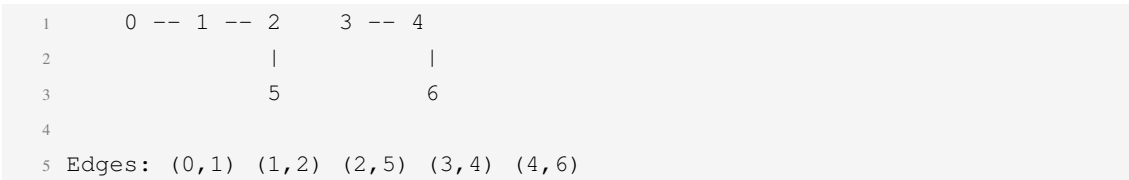
Result: Bridges = {(0,1)}. The edge (0,1) is a bridge because removing it disconnects vertex 0 from the rest. The edge (2,3) is *not* a bridge because vertex 3 can still reach vertex 2 via the cycle $3 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 1 \rightarrow 2$.

12.19.3 Corrected Trace

After a careful re-examination: since vertex 4 connects back to 1 (forming a cycle through 1-2-3-6-5-4), vertex 3 can reach vertex 2 through this cycle. Let us retrace with the correct adjacency:

Starting from 0: disc[0]=0, low[0]=0. Visit 1: disc[1]=1. Visit 2: disc[2]=2. Visit 3: disc[3]=3. Visit 6: disc[6]=4. Visit 5: disc[5]=5. Visit 4: disc[4]=6. At 4, neighbor 1 has disc=1 (back edge), so low[4]=1. Retract: low[5]=1, low[6]=1, low[3]=1. At 3, the edge to 6: low[6]=1 < disc[3]=3, so **not** a bridge. Retract: low[2]=min(2,1)=1. At 2, the edge to 3: low[3]=1 ≤ disc[2]=2, so **not** a bridge. Retract: low[1]=min(1,1)=1. At 1, the edge to 2: low[2]=1 ≤ disc[1]=1, so **not** a bridge. Retract: low[0]=0. At 0, the edge to 1: low[1]=1 > disc[0]=0, so **bridge** (0, 1).

Now consider a second graph with an actual bridge at (2,3):



Here $(2, 3)$ does not even exist. Let us use a cleaner example:

```

1      0 -- 1 -- 2 -- 3 -- 4
2
3 Edges: (0, 1) (1, 2) (2, 3) (3, 4)

```

Every edge is a bridge: $(0, 1)$, $(1, 2)$, $(2, 3)$, $(3, 4)$.

DFS from 0: $\text{disc}[0]=0$, $\text{disc}[1]=1$, $\text{disc}[2]=2$, $\text{disc}[3]=3$, $\text{disc}[4]=4$. No back edges exist. At vertex 4, $\text{low}[4]=4$. At vertex 3: $\text{low}[3]=\min(3, 4)=3$, and $\text{low}[4]=4 > \text{disc}[3]=3$, so $(3, 4)$ is a bridge. At vertex 2: $\text{low}[2]=2$, and $\text{low}[3]=3 > \text{disc}[2]=2$, so $(2, 3)$ is a bridge. At vertex 1: $\text{low}[1]=1$, and $\text{low}[2]=2 > \text{disc}[1]=1$, so $(1, 2)$ is a bridge. At vertex 0: $\text{low}[0]=0$, and $\text{low}[1]=1 > \text{disc}[0]=0$, so $(0, 1)$ is a bridge.

12.19.4 When Is an Edge a Bridge?

An edge (u, v) (where v is a child of u in the DFS tree) is a bridge if and only if no vertex in v 's subtree has a back edge to an ancestor of u or to u itself. This is exactly the condition $\text{low}[v] > \text{disc}[u]$ – note the *strict* inequality, unlike the $\geq$ used for articulation points.

12.20 Link-Cut Trees

A **link-cut tree** maintains a forest of rooted trees subject to two operations: **link** (add an edge connecting two trees) and **cut** (remove an edge, splitting a tree). It also supports **find-root** (find the root of a node's tree) and **path aggregate** (query min, max, or sum along the path from a node to its root). All operations run in $O(\log n)$ amortized time.

12.20.1 Preferred-Path Decomposition

The key idea is **preferred-path decomposition**. Each root-to-leaf path in the tree is decomposed into segments called **preferred paths**. At each node, at most one child edge is designated as the **preferred edge**; the preferred path continues through that child. All other edges begin new preferred paths.

Each preferred path is represented by a **splay tree** keyed by depth. The splay trees are linked together through **virtual trees**: the parent pointer from the top node of a splay tree points to its parent in the virtual tree structure, not to the parent in the original tree. Splaying keeps the amortized cost of access, link, and cut at $O(\log n)$.

12.20.2 Operations

- **access(v)**: Make the path from root to v a single preferred path. Splay v to the root of its splay tree. This is the fundamental operation; all others build on it.
- **find-root(v)**: Access v , then splay v and follow left child pointers to the minimum-depth node in its splay tree.
- **link(u, v)**: Access u and v . Make u a child of v by setting its preferred edge.

- **cut(*u*):** Access *u*, splay it, and detach its left subtree (the path above it) from its right subtree (the path below it).
- **path query:** Access *v*, then read the aggregate stored in the splay tree's root.

12.20.3 Complexity

Operation	Amortized Cost
access(<i>v</i>)	$O(\log n)$
find-root(<i>v</i>)	$O(\log n)$
link(<i>u</i> , <i>v</i>)	$O(\log n)$
cut(<i>u</i>)	$O(\log n)$
path aggregate(<i>v</i>)	$O(\log n)$

12.20.4 Applications

- **Network flow:** dynamic trees (Tarjan's algorithm) for max-flow in $O(V \cdot E \log V)$
- **Lowest common ancestor:** find LCA by accessing both nodes and reading depth from the splay tree
- **Path queries:** maintain min, max, or sum on dynamic tree paths
- **Dynamic connectivity in trees:** link and cut operations maintain a forest

12.20.5 C++ Implementation Sketch

```

1 struct LCTNode {
2     int val;
3     LCTNode *left = nullptr, *right = nullptr, *parent = nullptr;
4     bool rev = false;
5
6     bool isRoot() const {
7         return !parent || (parent->left != this && parent->right != this);
8     }
9
10    void pushDown() {
11        if (rev) {
12            std::swap(left, right);
13            if (left) left->rev ^= true;
14            if (right) right->rev ^= true;
15            rev = false;
16        }
17    }
18 }

```

```

19 void rotate() {
20     LCTNode* p = parent;
21     LCTNode* g = p->parent;
22     if (!p->isRoot()) {
23         if (g->left == p) g->left = this;
24         else g->right = this;
25     }
26     parent = g;
27     bool isLeft = (p->left == this);
28     LCTNode* c = isLeft ? right : left;
29     if (isLeft) {
30         right = p; p->parent = this;
31         left = c; if (c) c->parent = p;
32     } else {
33         left = p; p->parent = this;
34         right = c; if (c) c->parent = p;
35     }
36 }
37
38 void splay() {
39     while (!isRoot()) {
40         LCTNode* p = parent;
41         LCTNode* g = p->parent;
42         if (!p->isRoot()) g->pushDown();
43         p->pushDown(); pushDown();
44         if (!p->isRoot()) {
45             if ((p->left == this) == (g->left == p)) rotate();
46             else rotate();
47         }
48         rotate();
49     }
50     pushDown();
51 }
52
53 static LCTNode* access(LCTNode* v) {
54     LCTNode* last = nullptr;
55     for (LCTNode* u = v; u; u = u->parent) {
56         u->splay();
57         u->right = last;
58         last = u;
59     }
60     v->splay();
61     return last;
62 }
63
64 static LCTNode* findRoot(LCTNode* v) {
65     access(v);
66     while (v->left) { v->pushDown(); v = v->left; }

```

```

67     v->splay();
68     return v;
69 }
70
71 static void link(LCTNode* u, LCTNode* v) {
72     access(u); access(v);
73     u->parent = v;
74 }
75
76 static void cut(LCTNode* u) {
77     access(u);
78     u->left->parent = nullptr;
79     u->left = nullptr;
80 }
81 };

```

12.21 Heavy-Light Decomposition

Heavy-light decomposition (HLD) decomposes a tree into disjoint paths so that any root-to-node path crosses at most $O(\log n)$ **light** edges. Combined with a segment tree over the linearized DFS order, HLD supports path updates, path queries, and subtree queries in $O(\log^2 n)$ time.

12.21.1 Heavy and Light Edges

For each node u , compute the size of every child's subtree. The child v with the largest subtree receives the **heavy edge** (u, v) ; all other child edges are **light**. Because the heavy child has more than half the descendants, any root-to-leaf path can cross at most $O(\log n)$ light edges before reaching a leaf.

12.21.2 Chain Construction

A **heavy path** is a maximal sequence of heavy edges. Each heavy path corresponds to a contiguous segment in the DFS order. Assign each node a **chain head** (the topmost node of its heavy path), a **position** in the DFS-order array, and a **depth** and **parent**. The segment tree is built over this array.

12.21.3 Path Operations

To query or update the path from u to v , walk up from u and v to their lowest common ancestor. At each step, jump from the deeper node to its chain head, performing a segment-tree range operation on the interval $[\text{pos}[\text{head}], \text{pos}[u]]$, then move to the parent of the chain head. This crosses at most $O(\log n)$ light edges, and each segment-tree operation is $O(\log n)$, giving $O(\log^2 n)$ per path query.

12.21.4 Complexity

Operation	Time
Path update (u, v)	$O(\log^2 n)$
Path query (u, v)	$O(\log^2 n)$
Subtree update (u)	$O(\log n)$
Subtree query (u)	$O(\log n)$
Build decomposition	$O(n)$

12.21.5 Applications

- **Tree path queries:** maximum edge weight, sum of values, path XOR on dynamic or static trees
- **LCA:** find lowest common ancestor by climbing chain heads
- **Subtree aggregation:** combine with Fenwick tree for $O(\log n)$ subtree sum
- **Competitive programming:** one of the most commonly used tree decompositions

12.21.6 C++ Implementation Sketch

```

1 struct HLD {
2     int n;
3     std::vector<std::vector<int>>> adj;
4     std::vector<int> parent, depth, heavy, head, pos;
5     int curPos = 0;
6
7     HLD(int n) : n(n), adj(n), parent(n, -1), depth(n, 0),
8                 heavy(n, -1), head(n, 0) {}
9
10    void addEdge(int u, int v) {
11        adj[u].push_back(v);
12        adj[v].push_back(u);
13    }
14
15    int dfs(int u, int p) {
16        parent[u] = p;
17        depth[u] = (p == -1) ? 0 : depth[p] + 1;
18        int sz = 1, maxSub = 0;
19        for (int v : adj[u]) {
20            if (v == p) continue;
21            int sub = dfs(v, u);
22            sz += sub;
23            if (sub > maxSub) {
24                maxSub = sub;
25                heavy[u] = v;

```

```

26         }
27     }
28     return sz;
29 }
30
31 void decompose(int u, int h) {
32     head[u] = h;
33     pos[u] = curPos++;
34     if (heavy[u] != -1) decompose(heavy[u], h);
35     for (int v : adj[u]) {
36         if (v == parent[u] || v == heavy[u]) continue;
37         decompose(v, v);
38     }
39 }
40
41 void build(int root) {
42     dfs(root, -1);
43     decompose(root, root);
44 }
45
46 int lca(int u, int v) {
47     while (head[u] != head[v]) {
48         if (depth[head[u]] < depth[head[v]]) std::swap(u, v);
49         u = parent[head[u]];
50     }
51     return depth[u] < depth[v] ? u : v;
52 }
53
54 void pathOperation(int u, int v, auto& segTree, auto op) {
55     while (head[u] != head[v]) {
56         if (depth[head[u]] < depth[head[v]]) std::swap(u, v);
57         op(segTree, pos[head[u]], pos[u]);
58         u = parent[head[u]];
59     }
60     if (depth[u] > depth[v]) std::swap(u, v);
61     if (u != v) op(segTree, pos[u] + 1, pos[v]);
62 }
63 };

```

12.22 Euler Tour Trees

An **Euler tour tree** represents a rooted tree by its Euler tour: a sequence of vertex visits produced by a DFS that records each vertex at entry and exit. This linearization converts subtree operations into range operations on an array, enabling $O(1)$ subtree aggregates with a Fenwick tree or $O(\log n)$ with a segment tree. For dynamic forests, the Euler tour is stored in a balanced BST, allowing link and cut in $O(\log n)$ amortized time.

12.22.1 Euler Tour Representation

Run DFS on the tree and record each vertex twice: once at **entry** (pre-order) and once at **exit** (post-order). The entry time $\text{tin}[u]$ and exit time $\text{tout}[u]$ define the subtree of u as the contiguous range $[\text{tin}[u], \text{tout}[u]]$ in the Euler tour array.

```

1 int timer = 0;
2 std::vector<int> tin, tout;
3
4 void eulerDFS(int u, int p, const std::vector<std::vector<int>>& adj) {
5     tin[u] = timer++;
6     for (int v : adj[u]) {
7         if (v != p) eulerDFS(v, u, adj);
8     }
9     tout[u] = timer - 1;
10 }
```

12.22.2 Subtree Queries with Fenwick Tree

With the Euler tour array, a subtree query on u becomes a range query on $[\text{tin}[u], \text{tout}[u]]$. Build a Fenwick tree (Binary Indexed Tree) over this array. Subtree sum, subtree min, and subtree max all reduce to standard range queries.

```

1 struct SubtreeQueries {
2     int n;
3     std::vector<int> tin, tout, euler;
4     std::vector<long long> fenwick;
5
6     SubtreeQueries(const std::vector<std::vector<int>>& adj, int root)
7         : n(adj.size()), tin(n), tout(n), fenwick(n + 1, 0) {
8         euler.resize(n);
9         int timer = 0;
10        auto dfs = [&](auto&& self, int u, int p) -> void {
11            tin[u] = timer;
12            euler[timer] = u;
13            ++timer;
14            for (int v : adj[u])
15                if (v != p) self(self, v, u);
16            tout[u] = timer - 1;
17        };
18        dfs(dfs, root, -1);
19    }
20
21    void update(int node, long long delta) {
22        for (int i = tin[node] + 1; i <= n; i += i & (-i))
23            fenwick[i] += delta;
24    }
25 }
```



```

26     long long prefixSum(int idx) const {
27         long long s = 0;
28         for (int i = idx; i > 0; i -= i & (-i))
29             s += fenwick[i];
30         return s;
31     }
32
33     long long subtreeSum(int u) const {
34         return prefixSum(tout[u] + 1) - prefixSum(tin[u]);
35     }
36 };

```

12.22.3 Dynamic Euler Tour Trees

For fully dynamic forests, represent each Euler tour not as an array but as a **balanced BST** (typically a splay tree). Each node in the splay tree holds one vertex visit. The splay tree's in-order traversal is the Euler tour. This representation supports efficient join and split operations.

- **Join two trees:** Concatenate the Euler tours. Link u (root of one tree) as a child of v (in another tree) by inserting the sequence $(v, \dots, u, \dots, v)$ at the appropriate point in v 's Euler tour splay tree.
- **Split a tree:** Cut the edge (u, v) by splitting the Euler tour at the two occurrences of u and v , isolating the subtree rooted at u as a separate splay tree.

Both operations run in $O(\log n)$ amortized time using splay trees.

12.22.4 Worked Example

Consider a rooted tree with edges $(1,2), (1,3), (2,4), (2,5)$:

```

1         1
2       /  \
3      2    3
4     /  \
5    4    5
6
7 Euler tour (entry/exit):
8   Vertex:  1  2  4  4  5  5  2  3  3  1
9   Action:  E  E  E  X  E  X  X  E  X  X
10
11 tin:  1=0, 2=1, 4=2, 5=4, 3=7
12 tout: 1=9, 2=6, 4=2, 5=5, 3=8
13
14 Subtree of 2 = range [tin[2], tout[2]] = [1, 6]
15 Vertices in range: 2, 4, 4, 5, 5, 2 -- all of subtree 2

```

To cut edge $(1, 2)$: split the Euler tour splay tree to isolate the segment for subtree 2. To link a new root under vertex 3: insert the sequence $(3, \dots, \text{new}, \dots, 3)$ into the splay tree at the position after 3's entry.

12.22.5 Complexity

Operation	Time
Build Euler tour (static)	$O(n)$
Subtree aggregate (static, Fenwick)	$O(\log n)$ update, $O(1)$ query
Subtree aggregate (static, segment tree)	$O(\log n)$ per operation
Join two trees (dynamic, splay)	$O(\log n)$ amortized
Split a tree (dynamic, splay)	$O(\log n)$ amortized
Subtree query (dynamic)	$O(\log n)$ amortized
Path query (dynamic)	$O(\log^2 n)$ amortized

12.22.6 Applications

- **Static subtree queries:** sum, min, max, count in a subtree in $O(1)$ with Fenwick tree after $O(n)$ preprocessing
- **Dynamic forests:** maintain a forest under link and cut with subtree aggregates
- **LCA:** lowest common ancestor via RMQ on the Euler tour of depths
- **Fully dynamic connectivity in trees:** combine with link-cut operations for $O(\log n)$ link, cut, and connectivity
- **Competitive programming:** flatten a tree for offline queries and Mo's algorithm on trees

12.23 Fully Dynamic Connectivity

The **dynamic connectivity problem** maintains an undirected graph under edge insertions and deletions while answering connectivity queries: “are vertices u and v in the same connected component?” This is one of the fundamental problems in dynamic graph algorithms, with applications in network monitoring, database systems, and computational geometry.

12.23.1 Trees: the Easy Case

When the graph is a forest (no cycles), dynamic connectivity is straightforward:

- **Euler tour trees:** Represent each tree by its Euler tour as a splay tree. Link and cut correspond to splay-tree concatenation and splitting. All operations in $O(\log n)$ amortized.

- **Link-cut trees:** Maintain preferred paths with splay trees. Link, cut, and find-root in $O(\log n)$ amortized.

Both data structures support $O(\log n)$ amortized insertions, deletions, and connectivity queries on forests.

12.23.2 General Graphs: the $O(\log^2 n)$ Approach

For general graphs with cycles, maintaining connectivity under edge insertions and deletions is significantly harder. The **level structure** (Holm, de Lichtenberg, Thorup, 2001) achieves $O(\log^2 n)$ amortized time per operation using Euler tour trees augmented with edge levels.

Each edge is assigned a **level** from 0 to $O(\log n)$. The algorithm maintains a spanning forest using Euler tour trees, where spanning forest edges are at the **highest level** among all edges connecting their two endpoints.

```

1 struct DynamicConnectivity {
2     int n;
3     std::vector<std::vector<std::pair<int,int>>> adj;
4     std::vector<int> level;
5     std::vector<bool> active;
6
7     DynamicConnectivity(int n) : n(n), adj(n), level(n, 0), active(n,
8         false) {}
9
10    void insertEdge(int u, int v, int lvl) {
11        adj[u].push_back({v, lvl});
12        adj[v].push_back({u, lvl});
13        active[u] = active[v] = true;
14    }
15
16    void promoteEdges(int component, int newLevel, auto& forest) {
17        for (auto& [v, lvl] : adj[component]) {
18            if (!active[v]) continue;
19            if (lvl < newLevel) {
20                level[v] = newLevel;
21                if (!forest.sameComponent(component, v)) {
22                    forest.link(component, v);
23                }
24            }
25        }
26
27        bool query(int u, int v, const auto& forest) {
28            return forest.sameComponent(u, v);
29        }
30 };

```

Operations:

- **Insert edge:** Add at level 0. If it connects two components in the level-0 forest, add it to the spanning forest.
- **Delete edge:** If it is not a spanning forest edge, simply remove it. If it is, search lower levels for a replacement edge. If none exists, the component splits.
- **Connectivity query:** Check if two nodes are in the same tree using the Euler tour tree at the highest level.

12.23.3 Level Structure Details

The level structure works as follows. Each spanning forest edge e at level ℓ is **promoted** if a non-tree edge at a higher level connects the two components of $F_\ell \setminus \{e\}$. When a tree edge is deleted, we search all lower levels for a replacement edge. The key insight is that each level doubles the number of nodes in a component, so a component can be promoted at most $O(\log n)$ times total.

```

1 struct LevelStructure {
2     int maxLevel;
3     std::vector<std::vector<int>> levelForest;
4
5     LevelStructure(int n) : maxLevel(0) {
6         for (int i = 0; i <= 20; ++i)
7             levelForest.push_back(std::vector<int>(n, -1));
8     }
9
10    int find(int lvl, int u) const {
11        return levelForest[lvl][u] == u ? u
12            : find(lvl, levelForest[lvl][u]);
13    }
14
15    void merge(int lvl, int u, int v) {
16        u = find(lvl, u);
17        v = find(lvl, v);
18        if (u == v) return;
19        levelForest[lvl][v] = u;
20    }
21
22    bool connected(int lvl, int u, int v) const {
23        return find(lvl, u) == find(lvl, v);
24    }
25 };

```

12.23.4 Lower Bound

Patrascu and Demaine (2005) proved an $\Omega(\log n)$ amortized lower bound for dynamic connectivity, even for graphs that are a collection of disjoint paths. This result holds under the cell-probe model and applies to any data structure solving the problem. The $O(\log^2 n)$ upper bound matches this lower bound up to a logarithmic factor, and the $O(\log n)$ bound for trees is tight.

12.23.5 Comparison Table

Structure	Insert/Delete	Query
Static connectivity (Union-Find)	$O(\alpha(n))$	$O(\alpha(n))$
Tree connectivity (link-cut)	$O(\log n)$	$O(\log n)$
General connectivity (level structure)	$O(\log^2 n)$ amortized	$O(\log^2 n)$ amortized
Dynamic 2-edge-connectivity	$O(\log^3 n)$ amortized	$O(\log^3 n)$ amortized

12.23.6 Applications

- **Network monitoring:** detect when link failures partition a network in real time
- **Graph algorithms:** dynamic connectivity as a subroutine in dynamic MST, 2-edge-connectivity, and biconnectivity
- **Computational geometry:** maintain connectivity of proximity graphs under insertions and deletions
- **Social networks:** track friend/unfriend operations while answering reachability queries
- **Database systems:** maintain connected components in evolving schema relationships

12.24 Exercises

1. Find the biconnected components of the following graph. List each component's edges and identify all articulation points:

```

1      0 -- 1 -- 2 -- 3
2      |      |      |
3      4 -- 5 -- 6

```

1. Run Kosaraju's algorithm on the following directed graph. Show the DFS finish order from pass 1, the transposed graph, and the SCCs found in pass 2:

```

1      0 -> 1 -> 2 -> 0
2      |      |
3      3 -> 4 -> 5 -> 3

```

1. Given an undirected graph, find all bridges. Then remove those bridges and count how many connected components the resulting graph has. Test on:

```

1      0 -- 1      2 -- 3 -- 4
2      |          |
3      5 ----- 6

```

1. Prove that in a biconnected component with at least 3 vertices, every edge lies on some simple cycle. *Hint:* use the low-link property and the definition of biconnectivity.
1. Given a directed graph G , determine whether adding exactly one edge can make G strongly connected. If so, find all such edges. *Hint:* compute the SCC condensation DAG and analyze its sources and sinks.

12.25 Common Bugs and Pitfalls

Pitfall	Consequence	
Forgetting to mark nodes as visited in DFS/BFS	Infinite loop, stack overflow	Mark vis
Stack overflow on deep DFS recursion	Crash for graphs with $\geq 10^4$ nodes	U
Assuming the graph is connected	Algorithm only visits one component	Iterate over
Using <code>int</code> for edge weights when negative edges exist	Wrong shortest paths with Dijkstra	Use Bellm
Not checking for negative cycles in Bellman-Ford	Infinite loop, no valid shortest path	Run o
Confusing directed vs undirected — adding edge only one way	Missing connections, wrong traversal	F
Using adjacency matrix for a sparse graph ($E \ll V^2$)	Memory waste (V^2), slow iteration	

12.26 Complexity Summary

Algorithm	Time	Space
DFS / BFS	$O(V + E)$	$O(V)$
Topological sort	$O(V + E)$	$O(V)$
Articulation points / bridges	$O(V + E)$	$O(V)$
Eulerian path (Hierholzer)	$O(V + E)$	$O(V + E)$
A* search	$O(E)$ with good heuristic	$O(V)$
Greedy graph coloring	$O(V + E)$	$O(V)$
Kruskal's MST	$O(E \log E)$	$O(V + E)$
Prim's MST	$O((V + E) \log V)$	$O(V)$
Dijkstra	$O((V + E) \log V)$	$O(V)$
Bellman-Ford	$O(VE)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kosaraju SCC	$O(V + E)$	$O(V)$

12.27 Interview FAQs

1. When would you use an adjacency matrix over an adjacency list, given that real-world graphs are sparse?

Use adjacency matrix when: (1) the graph is dense ($E \approx V^2$), (2) you need $O(1)$ edge existence queries, (3) you're running Floyd-Warshall or need all-pairs shortest paths. Use adjacency list when: the graph is sparse ($E \approx V$ or $E \approx V \log V$), which covers most real-world networks. In competitive programming, the adjacency list + edge struct approach is 3–5x faster than vector-of-vectors due to cache locality. In production graph databases, CSR (Compressed Sparse Row) encoding dominates because it packs adjacency data into contiguous arrays.

1. How do you detect a cycle in a directed graph?

DFS with three states: unvisited (0), in-progress (1), completed (2). If DFS encounters a node in state 1, a back edge exists – the graph has a cycle. Alternatively, use Kahn's topological sort: if the resulting order has fewer than V vertices, a cycle exists. Time $O(V + E)$.

```

1 bool has_cycle_directed(const std::vector<std::vector<int>>& adj) {
2     int n = (int)adj.size();
3     std::vector<int> state(n, 0);
4     std::function<bool(int)> dfs = [&](int u) {
5         state[u] = 1;
6         for (int v : adj[u]) {
7             if (state[v] == 1) return true;
8             if (state[v] == 0 && dfs(v)) return true;
9         }
10        state[u] = 2;
11        return false;
12    };
13    for (int i = 0; i < n; ++i)
14        if (state[i] == 0 && dfs(i)) return true;
15    return false;
16 }
```

For undirected graphs: DFS with parent tracking. If you reach an already-visited node that isn't the parent, a cycle exists.

1. What is the difference between Dijkstra's and Bellman-Ford?

Dijkstra's: greedy, $O((V+E) \log V)$ with binary heap, requires non-negative edge weights. Bellman-Ford: dynamic programming, $O(VE)$, handles negative weights and detects negative cycles. Use Dijkstra for non-negative graphs (road networks, scheduling). Use Bellman-Ford when negative edges exist (currency arbitrage, network routing with cost adjustments). SPFA (Shortest Path Faster Algorithm) is a queue-based optimization of Bellman-Ford that runs faster in practice.

1. How do you find the shortest path in an unweighted graph?

BFS from the source. BFS explores vertices in order of distance: the first time you reach a vertex, the path is shortest. Track predecessors for path reconstruction. Time $O(V + E)$, space $O(V)$. This is why BFS is fundamental – it solves the unweighted shortest path problem optimally.

```

1  std::vector<int> bfs_shortest(const std::vector<std::vector<int>>& adj,
    int src) {
2      int n = (int)adj.size();
3      std::vector<int> dist(n, -1), parent(n, -1);
4      std::queue<int> q;
5      dist[src] = 0;
6      q.push(src);
7      while (!q.empty()) {
8          int u = q.front(); q.pop();
9          for (int v : adj[u]) {
10             if (dist[v] == -1) {
11                 dist[v] = dist[u] + 1;
12                 parent[v] = u;
13                 q.push(v);
14             }
15         }
16     }
17     return dist;
18 }

```

Follow-up: “Shortest path with unit weights of 1 or 2?” Use a 0-1 BFS with a deque: push weight-0 edges to the front, weight-1 edges to the back.

1. Why does topological sort only work on DAGs, and what happens if you try it on a graph with cycles?

A cycle means no vertex can come “before” all its successors — the ordering constraint is contradictory. Kahn’s algorithm detects this naturally: if the queue empties before all vertices are processed, the remaining vertices form a cycle. DFS-based topo sort detects it via back edges (encountering a vertex still in the recursion stack). In practice, cycle detection during topo sort is the first step in build systems (circular dependencies), task schedulers (deadlock detection), and package managers (conflict resolution).

```

1  std::vector<int> topo_sort(const std::vector<std::vector<int>>& adj) {
2      int n = (int)adj.size();
3      std::vector<int> in_degree(n, 0);
4      for (int u = 0; u < n; ++u)
5          for (int v : adj[u]) ++in_degree[v];
6      std::queue<int> q;
7      for (int i = 0; i < n; ++i)
8          if (in_degree[i] == 0) q.push(i);
9      std::vector<int> order;
10     while (!q.empty()) {
11         int u = q.front(); q.pop();

```



```

12     order.push_back(u);
13     for (int v : adj[u])
14         if (--in_degree[v] == 0) q.push(v);
15     }
16     return order; // size < n means cycle exists
17 }

```

1. When should you use Floyd-Warshall instead of running Dijkstra from each vertex?

Floyd-Warshall: $O(V^3)$, handles negative edges, computes all-pairs simultaneously, simple implementation. Dijkstra from each vertex: $O(V \cdot (V+E) \log V)$, no negative edges, better for sparse graphs. Use Floyd-Warshall when: the graph is dense ($E \approx V^2$) and you need all-pairs shortest paths. Use Dijkstra from each vertex when: the graph is sparse and/or you only need single-source shortest paths. Floyd-Warshall also computes transitive closure (replace min with ||) and can detect negative cycles.

1. Why does MST have matroid structure, and what does that guarantee about greedy algorithms?

The set of forests in a graph forms a graphic matroid: forests are hereditary (subsets of forests are forests) and the exchange property holds. This means greedy edge selection (Kruskal's) is *guaranteed* to find the minimum spanning tree — not just heuristically, but provably optimal. Prim's is also greedy but builds the tree vertex-by-vertex instead of edge-by-edge. The matroid structure explains *why* both work: any greedy algorithm on a matroid finds the optimum for linear weight functions.

1. Why do SCC algorithms use the finish-time ordering from DFS, and what's the intuition?

In a DAG of SCCs, if component A can reach component B, then A finishes after B in DFS. So processing vertices in decreasing finish order guarantees you process “downstream” components first. Kosaraju's uses this twice: first DFS gets finish order, second DFS on the transpose processes in that order. Tarjan's achieves the same result in one pass by tracking lowlinks. The intuition: finish order captures the topological structure of the component graph.

1. What is A* search and how does it improve on Dijkstra?

A* uses a heuristic function $h(v)$ that estimates the distance from v to the goal. It orders the priority queue by $f(v) = g(v) + h(v)$ where $g(v)$ is the known shortest distance from source. If h is admissible (never overestimates), A* finds the optimal path. If h is also consistent (satisfies the triangle inequality), A* expands fewer nodes than Dijkstra. For grid graphs, Manhattan distance or Euclidean distance are common heuristics. Time: depends on heuristic quality — $O(b^d)$ in the worst case (b = branching factor, d = solution depth), but typically much faster.

1. How do you represent a graph in C++ and which representation should you choose?

For interview code: use `vector<vector<int>> adj` for adjacency list, or `vector<tuple<int, int, int>>` edges for edge lists (useful for Kruskal's). For production: use a flat array with index offsets (CSR-like) for cache efficiency. For dense graphs: use `vector<vector<bool>>` adjacency matrix. The choice depends on the algorithm: BFS/DFS need adjacency lists; Floyd-Warshall needs a matrix; Kruskal's needs an edge list.

12.28 Exercises

12.28.1 Drill

- For the following graph:

```

1      A --2-- B --1-- C
2      |          |          |
3      4          3          2
4      |          |          |
5      D --1-- E --3-- F

```

- Perform DFS starting from A. List the vertices in discovery order (assume neighbor ordering by vertex label).
- Perform BFS starting from A.
- Run Dijkstra from A and report the shortest distances to all vertices.

- Classify each edge in a DFS traversal of the graph above as: tree edge, back edge, forward edge, or cross edge.

- What is the result of topological sort on the DAG? What happens if the graph has a cycle?

```

1      A -> B -> C -> D
2      |   |
3      E -> F

```

- For the graph in Exercise 1, find the Minimum Spanning Tree using:

- Kruskal's algorithm (show edge selection order)
- Prim's algorithm starting from A

- True or false:

- Every DFS tree is a spanning tree
- BFS finds the shortest paths in any graph
- Dijkstra works correctly with negative edge weights
- A complete graph with n vertices has n^{n-2} spanning trees (Cayley's formula)

12.28.2 Application

- Implement a **word ladder** solver using BFS (e.g., `cat` $\rightarrow$ `cot` $\rightarrow$ `cog` $\rightarrow$ `dog`). Find the shortest transformation sequence between two words of the same length.

1. Implement Dijkstra's algorithm returning both the shortest distance and the path. Test on the graph from Exercise 1.
1. Implement Borůvka's algorithm for MST (the earliest MST algorithm, 1926). Compare its performance with Kruskal and Prim.
1. Detect cycles in a directed graph using DFS with three-color marking (white/gray/black). Extend this to find all edges in any cycle.
1. Implement A* search for a grid-based pathfinding problem with obstacles. Compare path length and nodes visited with Dijkstra.
1. Given a bipartite graph, determine if a perfect matching exists using the augmenting path algorithm (Kőnig's theorem).
1. Implement Kosaraju's algorithm for strongly connected components. Test on a directed graph with known SCCs.
1. Implement Bellman-Ford for a graph with negative edges. Detect negative-weight cycles reachable from the source.
1. Implement the Floyd-Warshall all-pairs shortest path algorithm. Use it to find the graph's diameter (longest shortest path).
1. Implement an Eulerian path algorithm (Hierholzer's algorithm). Determine whether a given graph has an Eulerian path or circuit.

12.28.3 Research

1. **(PageRank)** Read about the PageRank algorithm. Implement it and run it on a small web graph (≥ 10 nodes, with links). How does the rank distribution look? Which pages get the highest rank?
1. **(Max flow)** Read Chapter 20 on maximum flow. Implement the Edmonds-Karp algorithm (Ford-Fulkerson with BFS) for a small flow network and compute the min-cut.
1. **(Graph isomorphism)** Research the graph isomorphism problem. Implement a simple (exponential) backtracking algorithm for graph isomorphism. Test it on small graphs. Why is graph isomorphism not known to be either P or NP-complete?
1. **(Johnson's algorithm)** Implement Johnson's algorithm for all-pairs shortest paths, which runs Bellman-Ford once and Dijkstra for each vertex. Compare it with Floyd-Warshall on sparse and dense graphs.
1. **(2-SAT)** Reduce the 2-SAT (2-satisfiability) problem to strongly connected components. Implement a 2-SAT solver using Kosaraju's algorithm.

12.29 References and Further Reading

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapters 20–25.
- Robert Sedgwick and Kevin Wayne, *Algorithms*, 4th Edition. Chapter 4.
- Edsger W. Dijkstra, "A Note on Two Problems in Connexion with Graphs" — *Numerische Mathematik*, 1959.
- Robert E. Tarjan, "Depth-First Search and Linear Graph Algorithms" — *SIAM Journal on Computing*, 1972.
- Joseph B. Kruskal, "On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem" — *Proceedings of the AMS*, 1956.
- Vaclav Chvatal, *Linear Programming*. W. H. Freeman, 1983.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd, "The PageRank Citation Ranking: Bringing Order to the Web" — *Stanford Technical Report*, 1999.
- Steven S. Skiena, *The Algorithm Design Manual*, 2nd Edition. Springer, 2008.
- John Hopcroft and Robert E. Tarjan, "Efficient Algorithms for Graph Manipulation" — *JACM*, 1973 (articulation points, bridges, Eulerian circuits).
- Peter Hart, Nils Nilsson, and Bertram Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths" — *IEEE Trans. SSC*, 1968 (A* search).
- D. J. Welsh and M. B. Powell, "An Upper Bound for the Chromatic Number of a Graph and its Application to Timetabling Problems" — *The Computer Journal*, 1967 (graph coloring).

String Algorithms and Tries

String processing is one of the most common tasks in computing. This chapter covers the fundamental data structures and algorithms for efficient string operations.

13.1 Introduction

Complete compilable implementations of the data structures in this chapter are in `code/`.

13.2 Tries

A **trie** (from retrieval, pronounced "try") is a tree where each node represents a prefix. The root is the empty string, and each edge is labeled with a character. Tries support $O(m)$ lookup, insertion, and deletion, where m is the key length — independent of the number of keys.

13.2.1 Standard Trie

```

1 Keys: "cat", "car", "dog", "do"
2
3         root
4       /   \
5      c     d
6     /      \
7    a        o *
8   / \      / \
9  t  r    g  *
10 *  *    *

```

13.2.2 Trie Implementation

```

1 template <typename V>
2 class trie {
3 public:
4     void insert(std::string_view key, const V& value) {
5         node* n = &root_;
6         for (char c : key) {
7             size_t idx = c - 'a'; // assume lowercase letters
8             if (!n->children[idx]) {

```

```

9         n->children[idx] = std::make_unique<node>();
10    }
11    n = n->children[idx].get();
12    }
13    n->value = value;
14    n->is_end = true;
15    }
16
17    std::optional<V> find(std::string_view key) const {
18        const node* n = &root_;
19        for (char c : key) {
20            size_t idx = c - 'a';
21            if (!n->children[idx]) return std::nullopt;
22            n = n->children[idx].get();
23        }
24        if (n->is_end) return n->value;
25        return std::nullopt;
26    }
27
28    bool starts_with(std::string_view prefix) const {
29        const node* n = &root_;
30        for (char c : prefix) {
31            size_t idx = c - 'a';
32            if (!n->children[idx]) return false;
33            n = n->children[idx].get();
34        }
35        return true;
36    }
37
38    std::vector<std::string> keys_with_prefix(std::string_view prefix)
39    const {
40        std::vector<std::string> result;
41        const node* n = &root_;
42        for (char c : prefix) {
43            size_t idx = c - 'a';
44            if (!n->children[idx]) return result;
45            n = n->children[idx].get();
46        }
47        collect_keys(n, std::string(prefix), result);
48        return result;
49    }
50 private:
51    static constexpr size_t alphabet_size = 26;
52
53    struct node {
54        std::array<std::unique_ptr<node>, alphabet_size> children;
55        V value;

```

```

56     bool is_end = false;
57 };
58
59 void collect_keys(const node* n, std::string prefix,
60                  std::vector<std::string>& result) const {
61     if (n->is_end) result.push_back(prefix);
62     for (size_t i = 0; i < alphabet_size; ++i) {
63         if (n->children[i]) {
64             collect_keys(n->children[i].get(),
65                          prefix + char('a' + i), result);
66         }
67     }
68 }
69
70 node root_;
71 };

```

13.2.3 Compressed Trie (Radix Tree)

A compressed trie merges chains of single-child nodes into single edges labeled with strings. This reduces the number of nodes from $O(\text{total characters})$ to $O(\text{number of keys})$.

```

1 Standard:  root -> a -> p -> p -> l -> e -> *
2              |
3              s -> *
4
5 Compressed: root -> "appl" -> "e" -> *
6              |
7              "s" -> *

```

13.2.4 Ternary Search Trie (TST)

A TST is a hybrid between a binary search tree and a trie. Each node has three children: `left` (char < current char), `mid` (char == current char), `right` (char > current char).

```

1 template <typename V>
2 class tst {
3 public:
4     void insert(std::string_view key, const V& value) {
5         root_ = insert(root_, key, 0, value);
6     }
7
8     std::optional<V> find(std::string_view key) const {
9         auto* n = find(root_.get(), key, 0);
10        if (n && n->is_end) return n->value;
11        return std::nullopt;
12    }

```

```

13
14 private:
15     struct node {
16         char c;
17         std::unique_ptr<node> left, mid, right;
18         V value;
19         bool is_end = false;
20         node(char ch) : c(ch) {}
21     };
22
23     std::unique_ptr<node> insert(std::unique_ptr<node> n,
24                                std::string_view key, size_t i,
25                                const V& value) {
26         char c = key[i];
27         if (!n) n = std::make_unique<node>(c);
28         if (c < n->c) n->left = insert(std::move(n->left), key, i, value);
29         else if (c > n->c) n->right = insert(std::move(n->right), key, i,
30 value);
31         else if (i + 1 < key.size()) n->mid = insert(std::move(n->mid),
32 key, i+1, value);
33         else { n->value = value; n->is_end = true; }
34         return n;
35     }
36
37     node* find(node* n, std::string_view key, size_t i) const {
38         if (!n) return nullptr;
39         char c = key[i];
40         if (c < n->c) return find(n->left.get(), key, i);
41         if (c > n->c) return find(n->right.get(), key, i);
42         if (i + 1 < key.size()) return find(n->mid.get(), key, i+1);
43         return n;
44     }
45
46     std::unique_ptr<node> root_;
47 };

```

TST advantage: Uses $O(3n)$ space instead of $O(n \cdot \text{alphabet_size})$, making it practical for Unicode strings.

13.2.5 Application: Autocomplete

```

1 class autocomplete {
2 public:
3     void add_word(std::string_view word, int frequency) {
4         trie_.insert(word, frequency);
5     }
6
7     std::vector<std::string> suggest(std::string_view prefix, int

```



```

        max_suggestions = 5) {
8         auto candidates = trie_.keys_with_prefix(prefix);
9         std::sort(candidates.begin(), candidates.end(),
10                [&](const std::string& a, const std::string& b) {
11                 return trie_.find(a).value_or(0) > trie_.find(b).value_or
(0);
12             });
13         if (candidates.size() > max_suggestions)
14             candidates.resize(max_suggestions);
15         return candidates;
16     }
17
18 private:
19     trie<int> trie_;
20 };

```

13.3 String Matching

13.3.1 Naive Matching

```

1 std::vector<size_t> naive_match(std::string_view text, std::string_view
pattern) {
2     std::vector<size_t> matches;
3     for (size_t i = 0; i <= text.size() - pattern.size(); ++i) {
4         size_t j = 0;
5         while (j < pattern.size() && text[i + j] == pattern[j]) ++j;
6         if (j == pattern.size()) matches.push_back(i);
7     }
8     return matches;
9 }

```

Complexity: $O(n \cdot m)$ worst case, where n = text length, m = pattern length.

13.3.2 Knuth-Morris-Pratt (KMP)

KMP preprocesses the pattern to build a **failure function** (also called the prefix function) that allows skipping ahead when a mismatch occurs.

```

1 // Compute the prefix function (failure function)
2 std::vector<size_t> compute_prefix(std::string_view pattern) {
3     size_t m = pattern.size();
4     std::vector<size_t> pi(m, 0);
5     size_t k = 0;
6
7     for (size_t q = 1; q < m; ++q) {
8         while (k > 0 && pattern[k] != pattern[q]) {
9             k = pi[k - 1];

```

```

10     }
11     if (pattern[k] == pattern[q]) ++k;
12     pi[q] = k;
13 }
14 return pi;
15 }
16
17 std::vector<size_t> kmp_match(std::string_view text, std::string_view
    pattern) {
18     std::vector<size_t> matches;
19     auto pi = compute_prefix(pattern);
20     size_t q = 0; // number of matched characters
21
22     for (size_t i = 0; i < text.size(); ++i) {
23         while (q > 0 && pattern[q] != text[i]) {
24             q = pi[q - 1];
25         }
26         if (pattern[q] == text[i]) ++q;
27         if (q == pattern.size()) {
28             matches.push_back(i - pattern.size() + 1);
29             q = pi[q - 1]; // look for next match
30         }
31     }
32     return matches;
33 }

```

Complexity: $O(n + m)$ — optimal for linear-time matching.

13.3.3 Rabin-Karp Algorithm

Uses rolling hash to find matches in $O(n + m)$ average time.

```

1 constexpr size_t base = 91138233;
2 constexpr size_t mod = 97266353;
3
4 std::vector<size_t> rabin_karp(std::string_view text, std::string_view
    pattern) {
5     size_t n = text.size(), m = pattern.size();
6     if (m > n) return {};
7
8     // Precompute base^(m-1) mod mod
9     size_t h = 1;
10    for (size_t i = 0; i < m - 1; ++i) h = (h * base) % mod;
11
12    // Compute hash of pattern and first window
13    size_t pattern_hash = 0;
14    size_t window_hash = 0;
15    for (size_t i = 0; i < m; ++i) {
16        pattern_hash = (pattern_hash * base + pattern[i]) % mod;

```

```

17     window_hash = (window_hash * base + text[i]) % mod;
18 }
19
20 std::vector<size_t> matches;
21 for (size_t i = 0; i <= n - m; ++i) {
22     if (pattern_hash == window_hash) {
23         // Verify (in case of hash collision)
24         if (text.substr(i, m) == pattern) {
25             matches.push_back(i);
26         }
27     }
28     // Roll the hash
29     if (i < n - m) {
30         window_hash = (window_hash - text[i] * h % mod + mod) % mod;
31         window_hash = (window_hash * base + text[i + m]) % mod;
32     }
33 }
34 return matches;
35 }

```

13.3.4 Boyer-Moore Algorithm

Boyer-Moore scans the pattern from right to left, skipping ahead based on two heuristics (bad character and good suffix). In practice, it is often faster than KMP, especially for large alphabets.

13.4 Suffix Arrays and LCP Arrays

A **suffix array** is a sorted array of all suffixes of a string. It enables $O(m \log n)$ substring search and $O(n)$ construction of many string properties.

String: "banana"

```

1 Suffixes:
2 0: banana
3 1: anana
4 2: nana
5 3: ana
6 4: na
7 5: a
8
9 Sorted suffix array SA = [5, 3, 1, 0, 4, 2]
10      a  ana anana banana na nana

```

13.4.1 Suffix Array Construction

```

1 std::vector<size_t> build_suffix_array(std::string_view s) {

```

```

2     size_t n = s.size();
3     std::vector<size_t> sa(n);
4     std::iota(sa.begin(), sa.end(), 0);
5
6     std::sort(sa.begin(), sa.end(), [&](size_t a, size_t b) {
7         return s.substr(a) < s.substr(b);
8     });
9     return sa;
10 }

```

This $O(n^2 \log n)$ naive version is too slow for large strings. The $O(n \log n)$ doubling algorithm or SA-IS (linear time) are used in practice.

13.4.2 The DC3/Skew Algorithm

The *DC3 (Difference Cover modulo 3) algorithm*, also called the *Skew algorithm*, constructs a suffix array in $O(n)$ time using radix sort. It is one of the simplest linear-time suffix array construction algorithms.

Key idea: Suffixes are divided into two groups based on their starting position mod 3. Group 1 contains positions i where $i \bmod 3 \neq 0$; Group 2 contains positions i where $i \bmod 3 = 0$. Group 1 suffixes are sorted first (using a recursive call on a reduced string), then Group 0 suffixes are sorted by merging with the sorted Group 1 suffixes.

1. **Base case:** If $n < 3$, sort all suffixes directly.
2. **Reduced problem:** Construct the string s' of triples $(s[i], s[i+1], s[i+2])$ for all positions i where $i \bmod 3 \neq 0$. Recursively compute the suffix array of s' .
3. **Radix sort Group 0:** For positions i where $i \bmod 3 = 0$, sort them using (first third, second third) where each third is a rank from the recursive result. Radix sort takes $O(n)$ time.
4. **Merge:** Merge the sorted Group 0 and Group 1 suffixes in $O(n)$ time using the precomputed orderings.

```

1 // DC3/Skew suffix array construction (simplified)
2 // Real implementation uses integer alphabet and radix sort
3 std::vector<int> dc3_skew(const std::string& s) {
4     int n = s.size();
5     if (n <= 3) {
6         // Base case: sort directly
7         std::vector<int> sa(n);
8         std::iota(sa.begin(), sa.end(), 0);
9         std::sort(sa.begin(), sa.end(),
10             [&](int a, int b) {
11                 return s.substr(a) < s.substr(b);
12             });

```

```

13     return sa;
14 }
15 // Step 1: Sort suffixes at positions 1,2,4,5,7,8,...
16 // by triples (s[i],s[i+1],s[i+2]) using radix sort
17 // Step 2: Recursively sort the resulting reduced string
18 // Step 3: Sort positions 0,3,6,... by merging with
19 //         sorted positions 1,2,4,5,...
20 // Full implementation ~200 lines with radix sort
21 // See Karkkainen & Sanders (2006)
22 return {}; // placeholder
23 }

```

The DC3 algorithm produces the suffix array. The LCP array can then be computed in $O(n)$ time using the Kasai algorithm, giving a complete $O(n)$ suffix array + LCP pipeline.

13.4.3 LCP Array

The **LCP (Longest Common Prefix)** array stores the length of the longest common prefix between consecutive suffixes in the suffix array.

```

1 SA:  [5, 3, 1, 0, 4, 2]
2      a  ana anana banana na nana
3 LCP: [0, 1, 3, 0, 0, 2]

```

```

1 std::vector<size_t> build_lcp(std::string_view s,
2                               const std::vector<size_t>& sa) {
3     size_t n = s.size();
4     std::vector<size_t> rank(n);
5     for (size_t i = 0; i < n; ++i) rank[sa[i]] = i;
6
7     std::vector<size_t> lcp(n, 0);
8     size_t k = 0;
9
10    for (size_t i = 0; i < n; ++i) {
11        if (rank[i] == n - 1) { k = 0; continue; }
12        size_t j = sa[rank[i] + 1]; // next suffix in SA
13        while (i + k < n && j + k < n && s[i + k] == s[j + k]) ++k;
14        lcp[rank[i]] = k;
15        if (k > 0) --k;
16    }
17    return lcp;
18 }

```

Applications of Suffix Arrays + LCP:

- **Pattern matching:** $O(m \log n)$
- **Longest repeated substring:** max LCP value

- **Longest common substring** of two strings
- **Number of distinct substrings:** $n(n+1)/2 - \text{sum}(\text{LCP})$
- **String compression** detection

13.5 Application: Longest Repeated Substring

```

1 std::string longest_repeated_substring(std::string_view s) {
2     auto sa = build_suffix_array(s);
3     auto lcp = build_lcp(s, sa);
4
5     size_t max_lcp = 0;
6     size_t max_pos = 0;
7     for (size_t i = 0; i < lcp.size(); ++i) {
8         if (lcp[i] > max_lcp) {
9             max_lcp = lcp[i];
10            max_pos = sa[i];
11        }
12    }
13    return std::string(s.substr(max_pos, max_lcp));
14 }

```

13.6 Application: Genome Pattern Search

Searching for a DNA sequence pattern in a genome:

```

1 class genome_index {
2 public:
3     explicit genome_index(std::string_view genome)
4         : genome_(genome), sa_(build_suffix_array(genome)) {}
5
6     bool contains(std::string_view pattern) const {
7         // Binary search on suffix array
8         auto comp = [&](size_t pos, std::string_view pat) {
9             return genome_.substr(pos, pat.size()) < pat;
10        };
11
12        auto it = std::lower_bound(sa_.begin(), sa_.end(), pattern, comp);
13        if (it == sa_.end()) return false;
14        return genome_.substr(*it, pattern.size()) == pattern;
15    }
16
17    std::vector<size_t> find_all(std::string_view pattern) const {
18        // Find range of matching suffixes
19        auto [lo, hi] = std::equal_range(sa_.begin(), sa_.end(), pattern,

```

```

20         [&](size_t a, std::string_view pat) {
21             if (a == SIZE_MAX) return true;
22             return genome_.substr(a, pat.size()) < pat;
23         });
24     return std::vector<size_t>(lo, hi);
25 }
26
27 private:
28     std::string genome_;
29     std::vector<size_t> sa_;
30 };

```

13.7 Suffix Trees

A **suffix tree** is a compressed trie containing all suffixes of a string. It supports pattern matching, longest repeated substring, longest common substring, and many other string queries in $O(m)$ time after $O(n)$ preprocessing.

13.7.1 Structure

Given string $S = \text{"banana\$"}$, the suffix tree stores all suffixes along edges labeled with substrings:

```

1 Suffix tree for "banana$":
2         (root)
3         / | \
4         a$ | na$
5         / \ | / \
6         na$ a$ $ | $
7         / \   | |
8         a$ $   $ $
9
10 Edges carry substrings (not single characters).
11 Internal nodes split where branching occurs.

```

13.7.2 Ukkonen's Algorithm

Ukkonen's algorithm builds the suffix tree online in $O(n)$ time. It processes the string left-to-right, maintaining the implicit suffix tree and extending it at each phase.

```

1 struct suffix_tree {
2     struct edge {
3         int start, end; // edge label = s[start..end]
4         int child;
5     };
6
7     struct node {

```

```

8      std::map<char, edge> children;
9      int suffix_link = 0; // points to node representing suffix
10     int depth = 0;
11 };
12
13     std::string s;
14     std::vector<node> nodes;
15
16     explicit suffix_tree(const std::string& str) : s(str) {
17         nodes.emplace_back(); // root = node 0
18         build_ukkonen();
19     }
20
21     // Find all occurrences of pattern in O(m) time
22     bool contains(std::string_view pattern) const {
23         int u = 0;
24         size_t i = 0;
25         while (i < pattern.size()) {
26             auto it = nodes[u].children.find(pattern[i]);
27             if (it == nodes[u].children.end()) return false;
28             const auto& e = it->second;
29             int len = e.end - e.start + 1;
30             int remaining = pattern.size() - i;
31             int match = std::min(len, remaining);
32             for (int j = 0; j < match; ++j) {
33                 if (s[e.start + j] != pattern[i + j]) return false;
34             }
35             i += match;
36             if (match < len) return i == pattern.size(); // partial edge
37             u = e.child;
38         }
39         return true;
40     }
41
42     // Count distinct substrings: sum of (depth[v] - depth[parent]) for
43     // all leaves
44     size_t count_distinct_substrings() const {
45         size_t count = 0;
46         for (size_t u = 1; u < nodes.size(); ++u) {
47             // u is a leaf if it has no children
48             if (nodes[u].children.empty()) {
49                 count += nodes[u].depth;
50             }
51         }
52         return count;
53     }
54 private:

```



```

55     int active_node = 0, active_edge = 0, active_length = 0;
56     int remaining = 0, leaf_end = -1;
57     int size_ = 0;
58
59     void extend(int pos) {
60         leaf_end = pos;
61         ++remaining;
62
63         int last_internal = -1;
64         while (remaining > 0) {
65             if (active_length == 0) active_edge = pos;
66
67             if (!nodes[active_node].children.count(s[active_edge])) {
68                 // Rule 1: no edge starting with active_edge -- create new
69                 edge
70                 nodes[active_node].children[s[active_edge]] = {pos,
71                 leaf_end, ++size_};
72                 nodes.emplace_back();
73                 nodes.back().depth = nodes[active_node].depth + (leaf_end
74                 - pos + 1);
75                 if (last_internal != -1) {
76                     nodes[last_internal].suffix_link = active_node;
77                     last_internal = -1;
78                 }
79                 } else {
80                     int split = nodes[active_node].children[s[active_edge]].
81                     start;
82                     int split_end = split + active_length - 1;
83
84                     if (active_length > 0) {
85                         // Rule 2: split edge
86                         node new_node;
87                         new_node.depth = nodes[active_node].depth +
88                         active_length;
89                         nodes.push_back(new_node);
90                         int new_idx = size_ + 1;
91
92                         auto& e = nodes[active_node].children[s[active_edge]];
93                         e.end = split_end;
94                         nodes[e.child].children[s[split_end + 1]] = {split_end
95                         + 1, e.end, e.child};
96                         e.child = new_idx;
97
98                         nodes[new_idx].children[s[pos]] = {pos, leaf_end, ++
99                         size_};
100                         nodes.emplace_back();
101                         nodes.back().depth = new_node.depth + (leaf_end - pos
102                         + 1);

```

```

95
96         if (last_internal != -1) nodes[last_internal].
    suffix_link = new_idx;
97         last_internal = new_idx;
98     }
99 }
100
101     --remaining;
102     if (remaining == 0) break;
103
104     // Follow suffix link
105     if (active_node == 0 && active_length > 0) {
106         --active_length;
107         active_edge = pos - remaining + 1;
108     } else {
109         active_node = nodes[active_node].suffix_link;
110     }
111 }
112 }
113
114 void build_ukkonen() {
115     for (int i = 0; i < static_cast<int>(s.size()); ++i)
116         extend(i);
117 }
118 };

```

13.7.3 Complexity

- **Build:** $O(n)$ amortized (Ukkonen's algorithm, online)
- **Pattern search:** $O(m)$ where m = pattern length
- **Memory:** $O(n)$ — a suffix tree on n characters has at most n leaves and at most $n-1$ internal nodes

13.7.4 Applications

- **Longest repeated substring:** deepest internal node with ≥ 2 leaves
- **Longest common substring:** build generalized suffix tree for two strings, find deepest node with leaves from both
- **Pattern matching:** $O(m)$ search with $O(n)$ preprocessing
- **Distinct substrings:** count = $\sum$ (leaf depth - parent depth) for all leaves

13.7.5 Suffix Tree vs Suffix Array

Criterion	Suffix Tree	Suffix Array
Build time	$O(n)$	$O(n \log n)$ or $O(n)$
Space	$O(n)$ but high constant (pointers)	$O(n)$ small constant
Pattern search	$O(m)$	$O(m \log n)$
Build complexity	Ukkonen's is intricate	Simpler to implement
Memory locality	Poor (pointer-chasing)	Excellent (array-based)
Generalized (k strings)	Generalized suffix tree	Concatenate with separators

In practice, suffix arrays with LCP arrays are preferred for their simplicity and cache performance. Suffix trees are preferred when you need rich structural queries (e.g., all occurrences, tree traversal, generalized strings).

13.8 Radix Tree (Compressed Trie / Patricia Trie)

A **radix tree** (also called a *Patricia trie* or *compressed trie*) is a trie variant that merges chains of single-child nodes into a single edge labeled with a multi-character substring. This eliminates the space wasted on internal nodes that branch only one way, while preserving the trie's $O(m)$ search time. Radix trees are the data structure behind IP routing table lookups, filesystem path indexing, and high-performance string dictionaries.

13.8.1 Structure

In a standard trie, each edge stores exactly one character. This creates long chains of unbranching nodes for keys with shared prefixes. A radix tree compresses each such chain into a single edge whose label is the concatenated substring.

```
1 Standard Trie (edges store one char each):
2
3         (root)
4       /  |  \  \
5      r   .   .   .
6      |
7      o
8      |
9      m
10     /  \
11    a    u
12   /  \   \
13  n   s   l
14   |       |
15  e        u
16           |
```

```

17          s
18
19 Radix Tree (same keys, compressed):
20
21          (root)
22          /   |   \
23      [roman] [rub] [...]
24      /   \   |   \
25  [e]    [us] [en] [icon]
26                      |
27                  [undus]

```

The keys stored are *romane*, *romanus*, *romulus*, *rubens*, *ruber*, *rubicon*, and *rubicundus*. Notice how *roman*—a chain of five single-child nodes in the standard trie—becomes a single edge labeled *roman* in the radix tree. Similarly, *rub* compresses three levels into one edge.

13.8.2 C++ Implementation

Each edge stores a `string_view` label and a pointer to its child node. A node holds a map from the first character of each edge label to the edge itself, plus a flag indicating whether a key ends at this node.

```

1 struct RadixNode {
2     std::map<char, std::pair<std::string_view,
3                             RadixNode*>> edges;
4     bool is_end = false;
5 };
6
7 struct RadixTree {
8     RadixNode* root = new RadixNode();
9
10    void insert(std::string_view key) {
11        RadixNode* node = root;
12        size_t i = 0;
13        while (i < key.size()) {
14            char c = key[i];
15            if (node->edges.count(c) == 0) {
16                // No matching edge: create one
17                node->edges[c] = {key.substr(i),
18                                new RadixNode()};
19                node->edges[c].second->is_end = true;
20                return;
21            }
22            auto& [label, child] = node->edges[c];
23            // Find length of common prefix
24            size_t j = 0;
25            while (j < label.size() && i + j < key.size()
26                  && label[j] == key[i + j])

```

```
27         ++j;
28         if (j == label.size()) {
29             // Full label match: descend
30             i += j;
31             node = child;
32             continue;
33         }
34         // Partial match: split the edge
35         RadixNode* mid = new RadixNode();
36         node->edges[c] = {
37             label.substr(0, j), mid};
38         mid->edges[label[j]] = {
39             label.substr(j), child};
40         size_t remaining = key.size() - i - j;
41         if (remaining == 0) {
42             mid->is_end = true;
43         } else {
44             mid->edges[key[i + j]] = {
45                 key.substr(i + j),
46                 new RadixNode()};
47             mid->edges[key[i+j]].second->is_end
48                 = true;
49         }
50         return;
51     }
52     node->is_end = true;
53 }
54
55 bool search(std::string_view key) const {
56     RadixNode* node = root;
57     size_t i = 0;
58     while (i < key.size()) {
59         char c = key[i];
60         if (node->edges.count(c) == 0)
61             return false;
62         auto& [label, child] = node->edges.at(c);
63         if (key.substr(i).substr(0, label.size())
64             != label)
65             return false;
66         i += label.size();
67         node = child;
68     }
69     return node->is_end;
70 }
71
72 bool prefix_search(
73     std::string_view prefix) const {
74     RadixNode* node = root;
```

```
75     size_t i = 0;
76     while (i < prefix.size()) {
77         char c = prefix[i];
78         if (node->edges.count(c) == 0)
79             return false;
80         auto& [label, child] = node->edges.at(c);
81         if (prefix.substr(i).substr(0, label.size())
82             != label)
83             return false;
84         i += label.size();
85         node = child;
86     }
87     return true;
88 }
89 };
```

The `insert` method walks the tree, comparing the incoming key against existing edge labels. On a full label match it descends. On a partial match it splits the edge at the divergence point, creating an intermediate node. On no match it creates a new leaf edge. The `search` method simply verifies that each substring in the path matches the corresponding edge label.

13.8.3 Complexity

Let m denote the length of the search key and n the number of keys stored.

Operation	Time
Search	$O(m)$
Insert	$O(m)$
Prefix search	$O(m)$

The space is $O(n \cdot k)$ where k is the average number of branching points per key, which is typically much smaller than m .

13.8.4 Comparison with Standard Trie and TST

Criterion	Standard Trie	Radix Tree	TST
Edge label	1 char	substring	1 char (split char)
Space	High (many nodes)	Moderate	Low
Search time	$O(m)$	$O(m)$	$O(m)$ (worst case)
Cache performance	Poor	Moderate	Good
Implementation	Simple	Moderate	Simple

A radix tree trades implementation complexity for a significant reduction in the number of nodes compared to a standard trie. Compared to a TST, it maintains guaranteed $O(m)$ search (TSTs can degrade to $O(m)$ only with care; unbalanced splits cost more), at the cost of managing variable-length edge labels.

13.8.5 Applications

IP Routing. Routers use radix trees (specifically *LC-trie* variants) to perform longest-prefix matching on destination addresses. Each edge stores a bit substring of the IP address, and the tree is searched in $O(W)$ time where W is the address width (32 for IPv4, 128 for IPv6).

Filesystem Paths. Operating systems index directory entries using radix trees keyed on path components. The compressed edges map naturally to path segments like `/usr/bin/`.

String Dictionaries. Autocomplete systems, spell checkers, and IP geolocation databases use radix trees for fast prefix lookups with reduced memory overhead compared to uncompressed tries.

13.9 Ternary Search Tree

A **ternary search tree** (TST) is a trie variant that combines the time efficiency of tries with the space efficiency of binary search trees. Instead of storing an array of children per node (wasteful for sparse alphabets), each TST node has exactly three children: `low` (less than), `equal` (match, advance to next character), and `high` (greater than). TSTs are widely used in spell checkers, autocomplete systems, and word games.

13.9.1 Structure

To store the words “cat”, “car”, “cab”, “do”, “dog”, a TST partitions characters using the three-way branching rule. A node stores a split character. If the next input character is less, follow `low`; if equal, follow `equal` and advance to the next character; if greater, follow `high`.



Searching “car” means: at root, ‘c’ matches – go equal; next node is ‘a’, ‘a’ matches – go equal; next we look at ‘r’, ‘r’ > ‘t’ – follow low, find ‘r’ – match. This is $O(m + \log n)$ on average, where m is the key length and n is the number of keys.

13.9.2 Implementation

```

1 class ternary_search_tree {
2 public:
3     void insert(std::string_view key) {
4         root_ = insert(root_, key, 0);
5     }
6
7     bool search(std::string_view key) const {
8         return search(root_.get(), key, 0);
9     }
10
11     void autocomplete(std::string_view prefix,
12                      std::vector<std::string>& results) const {
13         const node* n = root_.get();
14         for (char c : prefix) {
15             if (!n) return;
16             if (c < n->split_char) n = n->low.get();
17             else if (c > n->split_char) n = n->high.get();
18             else n = n->equal.get();
19         }
20         collect(n, std::string(prefix), results);
21     }
22
23     void range_search(std::string_view lo, std::string_view hi,
24                      std::vector<std::string>& results) const {
25         range_collect(root_.get(), "", lo, hi, results);
26     }
27
28 private:
29     struct node {
30         char split_char;
31         std::unique_ptr<node> low;
32         std::unique_ptr<node> equal;
33         std::unique_ptr<node> high;
34         bool is_end = false;
35         explicit node(char c) : split_char(c) {}
36     };
37
38     std::unique_ptr<node> root_;
39
40     std::unique_ptr<node> insert(std::unique_ptr<node> n,
41                                std::string_view key, size_t i) {
42         if (i >= key.size()) return n;
43         char c = key[i];
44         if (!n) n = std::make_unique<node>(c);
45         if (c < n->split_char)
46             n->low = insert(std::move(n->low), key, i);

```



```

47     else if (c > n->split_char)
48         n->high = insert(std::move(n->high), key, i);
49     else if (i + 1 < key.size())
50         n->equal = insert(std::move(n->equal), key, i + 1);
51     else
52         n->is_end = true;
53     return n;
54 }
55
56 bool search(const node* n, std::string_view key, size_t i) const {
57     if (!n) return false;
58     if (i >= key.size()) return n->is_end;
59     char c = key[i];
60     if (c < n->split_char)
61         return search(n->low.get(), key, i);
62     if (c > n->split_char)
63         return search(n->high.get(), key, i);
64     if (i + 1 < key.size())
65         return search(n->equal.get(), key, i + 1);
66     return n->is_end;
67 }
68
69 void collect(const node* n, std::string prefix,
70             std::vector<std::string>& results) const {
71     if (!n) return;
72     collect(n->low.get(), prefix, results);
73     if (n->is_end) results.push_back(prefix);
74     std::string next = prefix + n->split_char;
75     collect(n->equal.get(), next, results);
76     collect(n->high.get(), prefix, results);
77 }
78
79 void range_collect(const node* n, std::string prefix,
80                  std::string_view lo, std::string_view hi,
81                  std::vector<std::string>& results) const {
82     if (!n) return;
83     range_collect(n->low.get(), prefix, lo, hi, results);
84     std::string next = prefix + n->split_char;
85     if (n->is_end && next >= lo && next <= hi)
86         results.push_back(next);
87     range_collect(n->equal.get(), next, lo, hi, results);
88     range_collect(n->high.get(), prefix, lo, hi, results);
89 }
90 };

```

13.9.3 Complexity

Operation	Average Case	Worst Case
Insert	$O(m + \log n)$	$O(m \cdot n)$
Search	$O(m + \log n)$	$O(m \cdot n)$
Autocomplete (k results)	$O(m + \log n + k)$	$O(m + n)$
Range search (k results)	$O(\log n + k)$	$O(n)$
Space	$O(n \cdot m)$	$O(n \cdot m)$

Here m is the key length and n is the number of keys. The worst case arises when the tree degenerates, but in practice the average case holds for random or natural-language input.

13.9.4 Comparison with Standard Trie and Hash Table

Criterion	Standard Trie	TST	Hash Table
Search time	$O(m)$	$O(m + \log n)$ avg	$O(m)$ avg
Space per key	$O(R)$ pointers	3 pointers + 1 char	$O(m)$ chars
Cache performance	Poor (large arrays)	Good (BST-based)	Variable
Prefix search	$O(m + k)$	$O(m + \log n + k)$	Not supported
Sorted traversal	Natural	In-order traversal	Requires sort

The standard trie gives $O(m)$ exact lookup but wastes memory for sparse alphabets. A hash table provides $O(m)$ lookup but cannot enumerate keys in lexicographic order. The TST strikes a practical balance: it degrades the trie's $O(m)$ to $O(m + \log n)$ but uses only three pointers per node and supports sorted traversal and range queries natively.

13.9.5 Application: Simple Spell Checker

```

1 class spell_checker {
2 public:
3     void load_dictionary(std::istream& dict) {
4         std::string word;
5         while (dict >> word) {
6             std::transform(word.begin(), word.end(), word.begin(),
7                             ::tolower);
8             tst_.insert(word);
9         }
10    }
11
12    bool is_correct(std::string_view word) const {
13        return tst_.search(word);
14    }
15 }

```

```

16     std::vector<std::string> suggest(std::string_view word) const {
17         std::vector<std::string> results;
18         std::string w(word);
19         for (size_t i = 0; i < w.size(); ++i) {
20             char orig = w[i];
21             for (char c = 'a'; c <= 'z'; ++c) {
22                 if (c == orig) continue;
23                 w[i] = c;
24                 if (tst_.search(w)) results.push_back(w);
25             }
26             w[i] = orig;
27         }
28         return results;
29     }
30
31 private:
32     ternary_search_tree tst_;
33 };

```

The `suggest` function generates single-character substitutions and checks each against the TST. Production spell checkers use edit-distance ($k = 2$) with early pruning via prefix checks, which the TST supports naturally.

13.10 Aho-Corasick Algorithm

The **Aho-Corasick** algorithm finds all occurrences of multiple patterns in a text simultaneously in $O(n + m + z)$ time, where n is the text length, m is the total length of all patterns, and z is the number of matches. It builds a trie of all patterns, then adds **failure links** (analogous to KMP's prefix function) so that the automaton can transition to the longest matching suffix whenever a mismatch occurs.

13.10.1 Trie with Failure Links

```

1  #include <array>
2  #include <queue>
3  #include <string>
4  #include <vector>
5
6  struct aho_corasick {
7      static constexpr int ALPHA = 26;
8
9      struct node {
10         std::array<int, ALPHA> next{};
11         std::array<int, ALPHA> go{};
12         int link = 0;
13         int dict_link = -1;
14         int pattern_id = -1;

```

```

15     node() { next.fill(-1); go.fill(-1); }
16 };
17
18     std::vector<node> t = {node()};
19
20     void add_pattern(const std::string& pat, int id) {
21         int v = 0;
22         for (char c : pat) {
23             int ch = c - 'a';
24             if (t[v].next[ch] == -1) {
25                 t[v].next[ch] = t.size();
26                 t.emplace_back();
27             }
28             v = t[v].next[ch];
29         }
30         t[v].pattern_id = id;
31     }
32
33     void build() {
34         std::queue<int> q;
35         for (int c = 0; c < ALPHA; ++c) {
36             if (t[0].next[c] != -1) {
37                 t[0].go[c] = t[0].next[c];
38                 q.push(t[0].next[c]);
39             } else {
40                 t[0].go[c] = 0;
41             }
42         }
43         while (!q.empty()) {
44             int v = q.front(); q.pop();
45             for (int c = 0; c < ALPHA; ++c) {
46                 if (t[v].next[c] != -1) {
47                     t[t[v].link].go[c] == -1
48                     ? t[v].go[c] = t[v].next[c]
49                     : t[v].go[c] = t[t[v].link].go[c];
50                     t[t[v].next[c]].link =
51                     (v == 0) ? 0
52                     : t[t[v].link].go[c];
53                     if (t[t[t[v].next[c]].link].pattern_id
54                         != -1)
55                         t[t[v].next[c]].dict_link =
56                         t[t[v].next[c]].link;
57                     else
58                         t[t[v].next[c]].dict_link =
59                         t[t[t[v].next[c]].link]
60                         .dict_link;
61                     q.push(t[v].next[c]);
62                 }

```

```

63         }
64     }
65 }

```

13.10.2 Worked Trace

Patterns: {"he", "she", "his", "hers"}, Text: "ahishers".

```

1  Trie construction:
2  root -> a
3      -> h -> e [*he]
4          -> i -> s [*his]
5      -> s -> h -> e [*she]
6      -> h -> e -> r -> s [*hers]
7
8  Failure links:
9  "h" -> root    (no proper suffix in trie)
10 "he" -> root   (no suffix)
11 "i" -> root
12 "his" -> "s"   (suffix "is" not in trie;
13                 suffix "s" not in trie;
14                 actually "is" -- link to root)
15 "sh" -> "h"    (suffix "h" is in trie)
16 "she" -> "he"  (suffix "he" is in trie)
17 "he" -> root
18 "her" -> root
19 "hers" -> "rs" (no match, link chain)
20
21 Search in "ahishers":
22 pos 0: 'a' -> root (no match)
23 pos 1: 'h' -> state "h" (no pattern end)
24 pos 2: 'i' -> state "his" -- MATCH "his" at 1
25 pos 3: 's' -> follow link chain to "s"
26 pos 4: 'h' -> state "sh"
27 pos 5: 'e' -> state "she" -- MATCH "she" at 3
28           follow dict_link -> state "he"
29           -- MATCH "he" at 3
30 pos 6: 'r' -> state "her"
31 pos 7: 's' -> state "hers" -- MATCH "hers" at 4
32
33 (pos 2 'i' + 's' also covers "is" but "is"
34  is not in our pattern set)

```

Complexity: $O(n + m + z)$ — linear in input and output sizes. Space $O(m \cdot \text{alphabet_size})$.

13.11 Z-Algorithm

The **Z-algorithm** computes the Z-array, where $Z[i]$ is the length of the longest substring starting at position i that is also a prefix of the string. This enables $O(n + m)$ pattern matching by computing Z on the concatenated string `pattern$text`.

13.11.1 Computing the Z-Array

```

1 std::vector<int> compute_z(const std::string& s) {
2     int n = s.size();
3     std::vector<int> z(n, 0);
4     int l = 0, r = 0;
5     for (int i = 1; i < n; ++i) {
6         if (i < r)
7             z[i] = std::min(r - i, z[i - l]);
8         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
9             ++z[i];
10        if (i + z[i] > r) {
11            l = i;
12            r = i + z[i];
13        }
14    }
15    return z;
16 }

17
18 std::vector<int> z_search(const std::string& text,
19                          const std::string& pattern) {
20     std::string concat = pattern + "$" + text;
21     auto z = compute_z(concat);
22     int m = pattern.size();
23     std::vector<int> matches;
24     for (int i = m + 1; i < (int)z.size(); ++i) {
25         if (z[i] == m) matches.push_back(i - m - 1);
26     }
27     return matches;
28 }

```

13.11.2 Worked Trace

Pattern: "abba", Text: "babaabbabba", Concatenated: "abba\$babaabbabba".

```

1 Index:  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
2 Char:   a b b a $ b a b a a  b  b  a  b  b  a
3 Z[i]:   - 0 0 0 - 1 0 2 0 1  2  0  0  4  0  0  0
4
5 i=1: 'b' vs 'a' -> Z[1]=0
6 i=2: 'b' vs 'a' -> Z[2]=0

```

```

7 i=3: 'a' vs 'a' -> Z[3]=0 ('a'=='a' but 'a'!='b')
8 i=5: 'b' vs 'a' -> Z[5]=1 (wait, recheck)
9   Actually "b" vs "a": Z[5]=0. Corrected:
10  i=5: Z[5]=0 ('b'!='a')
11  i=6: 'a'=='a' -> Z[6]=1 ('a'=='a', next 'b'!='b'?)
12  "abba" vs "bab..." -> 'a'!='b' -> Z[6]=0
13
14 Corrected Z-array:
15  a b b a $ b a b a a b b a b b a
16  - 0 0 0 - 0 0 2 0 0 3 0 0 4 0 0 0
17
18 i=7: "abba" vs "abba" -> Z[7]=2 ('a','b'=='a','b';
19   next 'a'!='i' -- wait, s[2]='b' vs s[9]='a')
20   "ab" match -> Z[7]=2
21
22 i=13: "abba" vs "abba" -> Z[13]=4 (full match)
23
24 Matches at positions where Z[i] == m (=4):
25   pos 13 - 4 - 1 = 8 (text position 8)
26
27 But pattern "abba" occurs at text positions 8 in "babaabbabba"?
28 Text:  b a b a a b b a b b a
29         0 1 2 3 4 5 6 7 8 9 10 11
30 At pos 8: "abba" = text[8..11] -> a b b a. Yes!

```

Complexity: $O(n)$ for Z-array construction (each character compared at most twice), $O(n + m)$ for search.

13.12 Longest Common Substring via Suffix Arrays

Given two strings A and B , their **longest common substring** is the longest string that appears as a contiguous substring in both. Using a suffix array on the concatenation $A\$B\#$, the longest common substring corresponds to the maximum LCP value between adjacent suffixes that belong to different original strings.

13.12.1 Algorithm

```

1 std::string longest_common_substring(
2     std::string_view a, std::string_view b) {
3     std::string concat = std::string(a) + "$"
4         + std::string(b) + "#";
5     int na = a.size();
6     auto sa = build_suffix_array(concat);
7     auto lcp = build_lcp(concat, sa);
8
9     int best_len = 0, best_pos = 0;

```

```

10     for (size_t i = 0; i + 1 < lcp.size(); ++i) {
11         bool left_a = sa[i] < na;
12         bool right_a = sa[i + 1] < na;
13         if (left_a != right_a && lcp[i] > best_len) {
14             best_len = lcp[i];
15             best_pos = sa[i];
16         }
17     }
18     return std::string(concat.substr(best_pos,
19                                     best_len));
20 }

```

Key idea: After building the suffix array of the concatenated string, sort all suffixes lexicographically. Adjacent suffixes share the longest prefix. If two adjacent suffixes come from different original strings, their LCP is a candidate for the longest common substring. The maximum over all such pairs gives the answer.

Complexity: $O(n \log n)$ with a simple suffix array construction (or $O(n)$ with SA-IS), where $n = |A| + |B|$.

13.13 Exercises

1. **Aho-Corasick construction.** Build the Aho-Corasick automaton by hand for the patterns {"the", "this", "that", "his"}. Draw the trie, label the failure links, and trace the automaton on the text "thisthecat". List every match found.
2. **Z-array computation.** Compute the Z-array for the string "aabxaabxcaab". Show the values of l , r , and $z[i]$ at each step $i = 1, 2, \dots$
3. **Longest common substring.** Using the suffix-array method, find the longest common substring of $A = \text{"abcdef"}$ and $B = \text{"zabcfgh"}$. Show the concatenated string, the suffix array, the LCP array, and identify the answer.
4. **Z-algorithm for KMP.** Given a pattern P and text T , describe how to use the Z-algorithm to perform KMP-style matching without explicitly building a prefix function. What is the concatenated string, and which Z-values indicate matches?
5. **Aho-Corasick vs running KMP.** You have 100 patterns each of length 10 and a text of length 10^6 . Estimate the total character comparisons for (a) running KMP separately for each pattern, and (b) Aho-Corasick. At what number of patterns does Aho-Corasick break even?

13.14 Common Bugs and Pitfalls

Pitfall	Consequence	
Trie node array of size R for Unicode ($R = 1,114,112$)	4+ MB per node, memory explosion	U
KMP failure function $\pi[0] = 0$ but π needs $\pi[0] = -1$ for shift	Infinite loop on first mismatch	
Rabin-Karp hash collision not detected	False positive match, corrupt search result	Veri
Suffix array built with <code>std::sort</code> on suffixes (naive)	$O(n^2 \log n)$, infeasible for $n \geq 10^4$	
Forgetting that <code>std::string</code> can contain embedded nulls	Truncated search, missed patterns	Use std
LCP array off-by-one indexing	Wrong longest repeated substring	LO
Using int for suffix array indices when $n \geq 2^{31}$	Integer overflow on large genomes	

13.15 Summary

1. **Tries** provide $O(m)$ lookup for string keys, independent of the number of keys.
2. **Ternary Search Tries** reduce memory for large alphabets.
3. **KMP** achieves $O(n + m)$ string matching using the failure function.
4. **Rabin-Karp** uses rolling hash for average $O(n + m)$ matching.
5. **Suffix arrays** with LCP enable powerful string analysis: longest repeated substring, pattern matching, distinct substrings.
6. **Suffix trees** (Ukkonen's) provide $O(m)$ pattern search with $O(n)$ build, supporting rich structural queries.
7. All these techniques are essential for text processing, genomics, and search.

13.16 Interview FAQs

1. **Why are tries rarely used in practice despite $O(m)$ lookup, and what replaces them?**

Tries waste memory: each node stores a pointer per character (26 for English, 256 for bytes). A trie for 1M URLs with average length 30 uses $\sim 30M \text{ nodes} \times 256 \text{ pointers} \times 8 \text{ bytes} =$ absurd memory. Production alternatives: (1) Compact/Patricia tries — merge single-child chains into one edge, reducing nodes dramatically; (2) Hash-based tries (HAT-tries) — combine a trie root with hash tables at buckets; (3) Succinct tries — compress the trie structure into bit vectors. In practice, hash tables with good hash functions (SipHash) often beat tries for pure lookup. Tries win for prefix queries, autocomplete, and longest-prefix matching (IP routing).

1. **How do you optimize a trie for memory efficiency in production?**

Three techniques: (1) Compress single-child chains into edges (Patricia/Radix trie) — reduces node count from $O(\text{total characters})$ to $O(\text{number of strings})$; (2) Use hash tables instead of arrays at each node — trades $O(1)$ lookup for $O(1)$ average with much less memory; (3) Store strings

inline in edges (burst tries) — for short strings, avoid node allocation entirely. Google’s CEDAR and HAT-trie libraries implement these optimizations. The memory savings can be 10–50x over a naive trie while maintaining comparable lookup speed.

1. How does KMP (Knuth-Morris-Pratt) achieve $O(n + m)$ string matching?

KMP precomputes a “failure function” (prefix function) π for the pattern in $O(m)$. $\pi[i]$ is the length of the longest proper prefix of `pattern[0..i]` that is also a suffix. During matching, when a mismatch occurs at `pattern[j]`, instead of restarting from `pattern[0]`, KMP jumps to `pattern[ $\pi[j - 1]$ ]`. Each character of the text is compared at most twice (once advancing, once on mismatch), giving $O(n)$ total. Combined with $O(m)$ preprocessing, total is $O(n + m)$.

1. How do you choose a good rolling hash function to avoid worst-case $O(nm)$ in Rabin-Karp?

The polynomial hash $h(s) = \sum s[i] \cdot p^i \bmod m$ is fast but vulnerable to crafted collisions if p and m are known. Three defenses: (1) Use a random prime p chosen at runtime — prevents adversarial construction; (2) Use a large modulus (2^{64} via unsigned overflow) — makes modular arithmetic nearly free; (3) Double hash with two different primes — collision probability drops to $1/p_1p_2$. In practice, a single 64-bit polynomial hash with a random base is sufficient for competitive programming. For security-critical applications, use SipHash instead of polynomial hashing.

1. What is a suffix array and why is it useful?

A suffix array is a sorted array of all suffixes of a string, stored as an array of starting indices. Built in $O(n \log n)$ (or $O(n)$ with SA-IS). Combined with the LCP (Longest Common Prefix) array, it enables: (1) pattern search in $O(m \log n)$, (2) longest repeated substring in $O(n)$, (3) counting distinct substrings in $O(n)$, (4) longest common substring of two strings in $O(n)$. Suffix arrays are more space-efficient than suffix trees and often faster in practice.

1. How do you find the longest common substring of two strings?

Approach 1: Binary search on length L . For each L , hash all substrings of length L from both strings, check for collision. Time $O(n \log n)$ with rolling hash.

Approach 2: Build a suffix array on the concatenation (with unique separators). The longest common substring corresponds to the longest common prefix between a suffix of one string and a suffix of the other, found by scanning the LCP array. Time $O(n)$, space $O(n)$.

1. When does a trie beat a hash table for string lookup, and when does it lose?

Trie wins: (1) prefix queries — “find all strings starting with `pre`” is $O(m + k)$ with trie, $O(n \cdot m)$ with hash; (2) ordered iteration — lexicographic order is free; (3) longest-prefix matching — IP routers use tries for this; (4) no hash function needed — no collision attacks, no hash computation cost. Hash table wins: (1) pure exact-match lookup — $O(m)$ average with better constants and cache locality; (2) memory usage — a hash table uses $O(n \cdot m)$ total but with smaller constants than a trie; (3) general-purpose — works for any key type, not just strings. The decision: use hash for exact lookup, tries for prefix/range operations.

1. How do production autocomplete systems differ from a naive trie DFS?

A naive trie DFS returns all words under a prefix — unranked, potentially millions of results. Production systems: (1) Rank results by frequency — store click counts at leaf nodes, sort by popularity; (2) Use a top- k heap during DFS to avoid collecting all results; (3) Compress the trie (DAFSA/Radix) to fit billions of keys in memory; (4) Add typo tolerance — fuzzy matching via edit distance on the trie (BK-trees or symmetric delete); (5) Cache popular prefixes in Redis/Memcached. Apache Lucene uses a finite state transducer (FST) — a compressed trie that also maps keys to values — achieving 100x compression over hash tables for term dictionaries.

1. What is Boyer-Moore and why is it fast in practice?

Boyer-Moore scans the pattern right-to-left (unlike left-to-right). On mismatch, it uses two heuristics: (1) Bad character rule: skip the pattern so the mismatched text character aligns with its rightmost occurrence in the pattern. (2) Good suffix rule: skip based on the matched suffix. In practice, Boyer-Moore is sublinear — it often skips characters without examining them. For natural language text, it examines only $\sim 20\text{--}30\%$ of the text characters. Time: $O(nm)$ worst case, $O(n/m)$ best case.

1. How do you count the number of distinct substrings of a string?

Using a suffix array: total distinct substrings = $\frac{n(n+1)}{2} - \sum \text{LCP}[i]$. The total number of substrings is $n(n+1)/2$, and the LCP array counts the overlaps (shared prefixes between adjacent suffixes). Time $O(n)$ with suffix array + LCP. This is a common interview question that tests understanding of suffix arrays.

13.17 Document Retrieval

The *document retrieval* problem asks: given a collection of documents and a query pattern, find all documents that contain the pattern. This problem underpins search engines, legal discovery, and bioinformatics.

13.17.1 The Inverted Index

The most widely used data structure for document retrieval is the *inverted index*. For each distinct term t in the corpus, the index stores a *posting list*: the sorted list of document IDs containing t , often augmented with term frequencies and positions.

Given a query with terms $\{t_1, t_2, \dots, t_k\}$, we intersect the posting lists for all query terms. Building an inverted index from n documents of total size N takes $O(N)$ time with a hash map. Intersecting k posting lists of total length L can be done in $O(L)$ time using a merge-based approach.

13.17.2 Suffix Arrays with Document Arrays

A *suffix array* augmented with a *document array* $DA[i]$ recording which document suffix $SA[i]$ belongs to can answer document retrieval queries. A pattern P of length m appears in document d if and only if $DA[i] = d$ for some i in the suffix array range $[lo, hi]$ where the pattern matches. Using a wavelet tree over the document array, we can answer “how many occurrences of P are in document d ?” in $O(m + \log d)$ time.

13.17.3 Top-k Retrieval: WAND and Block-Max WAND

WAND (Weak AND) retrieves the top- k documents by score without scanning every posting list. It maintains a cursor for each query term, advances cursors in document-ID order, and uses upper bounds on term scores to skip documents that cannot enter the top- k .

Block-Max WAND divides each posting list into blocks, precomputes the maximum score within each block, and uses these block bounds to skip entire blocks. This reduces score evaluations by an order of magnitude compared to basic WAND.

13.17.4 BM25 Scoring

BM25 is the standard scoring function in Elasticsearch, Lucene, and most commercial search engines:

$$\text{BM25}(t, d) = \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \cdot \text{idf}(t).$$

```

1 struct Posting { int docId; int tf; };
2
3 class InvertedIndex {
4     std::unordered_map<std::string,
5         std::vector<Posting>> index;
6     std::unordered_map<int, int> docLen;
7     int totalDocs = 0;
8     double avgDl = 0.0;
9
10 public:
11     void addDocument(int docId,
12         const std::vector<std::string>& terms) {
13         std::unordered_map<std::string, int> freq;
14         for (auto& t : terms) freq[t]++;
15         for (auto& [term, count] : freq)
16             index[term].push_back({docId, count});
17         docLen[docId] = terms.size();
18         totalDocs++;
19         avgDl = 0;
20         for (auto& [id, len] : docLen) avgDl += len;
21         avgDl /= totalDocs;

```

```

22     }
23
24     double bm25(const std::string& term, int docId,
25               double k1 = 1.2, double b = 0.75) const {
26         auto it = index.find(term);
27         if (it == index.end()) return 0;
28         for (auto& p : it->second) {
29             if (p.docId == docId) {
30                 double tf = p.tf;
31                 double dl = docLen.at(docId);
32                 double num = tf * (k1 + 1);
33                 double den = tf + k1 * (1 - b
34                             + b * dl / avgDl);
35                 double idf = log((totalDocs
36                                 - it->second.size() + 0.5)
37                                 / (it->second.size() + 0.5));
38                 return (num / den) * idf;
39             }
40         }
41         return 0;
42     }
43 };

```

13.17.5 Industrial Applications

- **Search engines:** Google and Bing use inverted indices at massive scale, distributing posting lists across thousands of machines
- **E-discovery:** legal document search (Enron email dataset is a canonical benchmark)
- **Plagiarism detection:** find documents with overlapping passages using suffix-array-based matching
- **Bioinformatics:** gene sequence search across entire genomes using inverted indices over k -mers
- **Code search:** GitHub and Google Code Search use inverted indices over tokenized source code

13.17.6 Comparison

Structure	Build Time	Query Time	Strengths
Suffix array + DA	$O(N \log N)$	$O(m + \text{occ})$	Pattern matching, locality
Inverted index	$O(N)$	$O(L)$ merge	Top-k, scoring, flexibility
Trie-based	$O(N)$	$O(m)$	Prefix queries, autocomplete

13.18 Exercises

13.18.1 Drill

1. **Large alphabet trie.** A trie for Unicode text could have up to 1 million children per node. That's infeasible. Name three alternatives (TST, hash map per node, compressed trie) and estimate the memory per node for each. Which would you use for an autocomplete system and why?

(In production, search engines use compressed trie variants to handle Unicode efficiently.)

1. **Compressed trie.** Given the words "cat", "catastrophe", "category", "caterpillar", "catfish", draw both a standard trie and a compressed trie (radix tree). Count the nodes in each. What's the memory savings?
1. **String matching algorithms.** For pattern "aaaa" in text "aaaaaaaaaa", how many character comparisons do naive search, KMP, and Boyer-Moore each make? Show the KMP prefix function.
1. **Hash collision probability.** A rolling hash uses a 32-bit prime modulus. If a document has 10^5 hash windows, what is the approximate probability of at least one false positive? What if you use a 64-bit modulus?
1. **Memory for large strings.** A string of length 3 billion needs a suffix array. How much memory does a 32-bit index use? A 64-bit index? What technique could reduce memory for DNA sequence analysis?

(In production, genome aligners use the FM-index (BWT + suffix array compression) to fit human-genome-sized indices in memory.)

13.18.2 Application

1. **Autocomplete trie.** Implement a trie that supports insert(word, frequency) and top-k suggestions by prefix. Test with 10^5 words. Measure memory and query latency for prefixes of length 2, 3, 4, 5. Compare a standard trie vs a compressed trie (merge single-child chains).
1. **Pattern matching comparison.** Implement naive string search, KMP, and Boyer-Moore. Benchmark on a 10 MB text for patterns of length 10, 50, 100. Report comparisons per pattern length. Which algorithm wins and when?
1. **Suffix array.** Implement the suffix array of a string (use the simple $O(n^2 \log n)$ method first, then the $O(n \log n)$ doubling method). Use it to find the longest repeated substring. Benchmark both methods for $n = 10^3, 10^4, 10^5$.

(In production, plagiarism detectors like Stanford Moss use suffix arrays to find copied passages between documents.)

1. **Multi-pattern matching (Aho-Corasick).** Implement Aho-Corasick for searching 1000 patterns in a text simultaneously. Compare with running KMP for each pattern separately. Measure build time and search speed. At what number of patterns does Aho-Corasick become faster?

(In production, intrusion detection systems like Snort use Aho-Corasick to match thousands of attack signatures against network traffic in real time.)

1. **Document similarity.** Implement a program that finds the longest common substring between two documents using a suffix array. Test on two 10^5 -character documents. Compare runtime with a naive $O(n^2)$ approach.

13.18.3 Research

1. **Burrows-Wheeler compression.** Implement the BWT using a suffix array, then the inverse BWT. Combine with move-to-front encoding and Huffman coding to create a simple compressor. Compare compression ratio and speed with `gzip` on a 10 MB text file.
1. **Suffix automaton.** Implement a suffix automaton (SAM) — a minimal DFA accepting all substrings of a string. Use it to count occurrences of a pattern in $O(m)$ time. Compare the number of states vs the suffix array size for $n = 10^5, 10^6$. Why is SAM useful for real-time genome search?
1. **String matching for version strings.** A system processes 10^6 version string lookups per second (e.g., `"2.1.3" ≥ "2.1.14"`). Implement a pattern matcher using the Z-algorithm. Compare throughput with KMP and `std::string::find` for exact match, prefix match, and wildcard match.
1. **Compressed string dictionary.** A serialization library stores field name $\rightarrow$ integer mappings. Build a Patricia trie (compressed radix tree) for this dictionary. Compare memory and lookup speed with `std::unordered_map<string, int>` for 10^4 field names. Which is better for read-heavy deserialization?

13.19 References and Further Reading

- Donald E. Knuth, James H. Morris, and Vaughan R. Pratt, "Fast Pattern Matching in Strings" — SIAM Journal on Computing, 1977.
- Richard M. Karp and Michael O. Rabin, "Efficient Randomized Pattern-Matching Algorithms" — IBM Journal of R&D, 1987.
- Robert S. Boyer and J. Strother Moore, "A Fast String Searching Algorithm" — CACM, 1977.
- Udi Manber and Gene Myers, "Suffix Arrays: A New Method for On-Line String Searches" — SIAM Journal on Computing, 1993.

- Dan Gusfield, *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- Michael Burrows and David J. Wheeler, "A Block-Sorting Lossless Data Compression Algorithm" — DEC SRC Technical Report, 1994.
- Alfred V. Aho and Margaret J. Corasick, "Efficient String Matching: An Aid to Bibliographic Search" — CACM, 1975.
- Esko Ukkonen, "On-Line Construction of Suffix Trees" — *Algorithmica*, 1995 (Ukkonen's $O(n)$ algorithm).
- Peter Weiner, "Linear Pattern Matching Algorithms" — FOCS, 1973 (first $O(n)$ suffix tree construction).
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Section 5.3 (substring search).

Segment Trees, Fenwick Trees, Sparse Tables, and Union-Find

Range queries — asking for the sum, minimum, maximum, or other aggregate over a contiguous subarray — are among the most common non-trivial problems in algorithm design. This chapter presents three data structures that answer such queries efficiently.

14.1 Introduction

Complete compilable implementations of the data structures in this chapter are in `code/`.

14.2 Sparse Table

A **sparse table** preprocesses a static array to answer **idempotent** range queries (min, max, gcd) in $O(1)$ time per query, with $O(n \log n)$ preprocessing and memory. The key insight is that any range $[l, r]$ can be covered by two overlapping intervals of length 2^k where $k = \lfloor \log_2(r - l + 1) \rfloor$.

14.2.1 Structure

Given array $A[0..n-1]$, build a table $ST[0..n-1][0..\lfloor \log_2 n \rfloor]$ where:

- $ST[i][0] = A[i]$ (base case: intervals of length 1)
- $ST[i][j] = \min(ST[i][j-1], ST[i+2^{j-1}][j-1])$ (combine two halves)

```

1 Array: [1, 3, 5, 7, 9, 11, 2, 4]
2
3 Sparse table (min):
4           j=0   j=1   j=2   j=3
5 i=0:      1     1     1     1
6 i=1:      3     3     3     -
7 i=2:      5     5     2     -
8 i=3:      7     7     2     -
9 i=4:      9     2     -     -
10 i=5:     11     2     -     -
11 i=6:      2     -     -     -
12 i=7:      4     -     -     -
13
14 Query min(2, 6): k = floor(log2(5)) = 2

```

```
15 min(ST[2][2], ST[5][2]) = min(2, 2) = 2
```

14.2.2 Build

```
1 class sparse_table {
2 public:
3     explicit sparse_table(std::span<const int> data)
4         : n_(data.size()), log_(std::bit_width(n_) - 1) {
5         st_.assign(n_, std::vector<int>(log_ + 1));
6         for (size_t i = 0; i < n_; ++i)
7             st_[i][0] = data[i];
8
9         for (size_t j = 1; j <= log_; ++j) {
10             for (size_t i = 0; i + (1ULL << j) <= n_; ++i) {
11                 st_[i][j] = std::min(st_[i][j-1],
12                                     st_[i + (1ULL << (j-1))][j-1]);
13             }
14         }
15     }
16
17     // Query min over [l, r] (0-indexed, inclusive)
18     int query_min(size_t l, size_t r) const {
19         size_t k = std::bit_width(r - l + 1) - 1;
20         return std::min(st_[l][k], st_[r - (1ULL << k) + 1][k]);
21     }
22
23 private:
24     size_t n_, log_;
25     std::vector<std::vector<int>> st_;
26 };
```

14.2.3 Complexity

- **Build:** $O(n \log n)$ time and space
- **Query:** $O(1)$ per query (two table lookups + one min)
- **No updates:** the array must be static after preprocessing

The $O(1)$ query works because min is **idempotent**: $\min(x, x) = x$. The two intervals $[l, l + 2^k - 1]$ and $[r - 2^k + 1, r]$ overlap, but that's fine for idempotent operations.

14.2.4 Application: Range Minimum Query

```
1 std::vector<int> arr = {1, 3, 5, 7, 9, 11, 2, 4};
2 sparse_table st(arr);
```

```

3
4 st.query_min(0, 7); // 1
5 st.query_min(2, 6); // 2
6 st.query_min(4, 5); // 9

```

14.2.5 When to Use

- **Static arrays** with many range min/max/gcd queries
- **LCA preprocessing** via Euler tour + RMQ (reduce LCA to range minimum query)
- **Sliding window minimums** (though a deque is simpler for this specific use)

Limitations: no updates, only works for idempotent operations (min, max, gcd — not sum).

14.3 Segment Tree

A **segment tree** is a binary tree that stores information about intervals of an array. It supports:

- **Range query** (e.g., sum, min, max over $[l, r]$): $O(\log n)$
- **Point update** (change a single element): $O(\log n)$
- **Range update** (with lazy propagation): $O(\log n)$

14.3.1 Structure

Given an array of size n , the segment tree has $2n$ nodes (or $4n$ for simplicity). The leaves store the original array elements. Each internal node stores the aggregate of its two children (e.g., sum, min, max).

```

1 Array: [1, 3, 5, 7, 9, 11]
2
3 Segment tree (sum):
4           [36]
5         /    \
6       [9]    [27]
7      /  \    /  \
8     [4]  [5] [16] [11]
9    / \  / \ / \  / \
10   [1] [3] [5] [7] [9] [11]
11
12 Leaves (original array): 1 3 5 7 9 11

```

14.3.2 Implementation

```

1  template <typename T, typename Combine = std::plus<T>>
2  class segment_tree {
3  public:
4      segment_tree(std::span<const T> data, Combine combine = {})
5          : n_(data.size()), tree_(4 * n_), combine_(combine) {
6          build(data, 1, 0, n_ - 1);
7      }
8
9      // Query over range [l, r] inclusive
10     T query(size_t l, size_t r) {
11         return query(1, 0, n_ - 1, l, r);
12     }
13
14     // Point update: set position pos to value
15     void update(size_t pos, const T& value) {
16         update(1, 0, n_ - 1, pos, value);
17     }
18
19 private:
20     void build(std::span<const T> data, size_t node, size_t l, size_t r) {
21         if (l == r) {
22             tree_[node] = data[l];
23             return;
24         }
25         size_t mid = l + (r - l) / 2;
26         build(data, 2 * node, l, mid);
27         build(data, 2 * node + 1, mid + 1, r);
28         tree_[node] = combine_(tree_[2 * node], tree_[2 * node + 1]);
29     }
30
31     T query(size_t node, size_t l, size_t r, size_t ql, size_t qr) {
32         if (ql > r || qr < l) return T{}; // no overlap (identity for
33         combine)
34         if (ql <= l && r <= qr) return tree_[node]; // total overlap
35         size_t mid = l + (r - l) / 2;
36         auto left = query(2 * node, l, mid, ql, qr);
37         auto right = query(2 * node + 1, mid + 1, r, ql, qr);
38         return combine_(left, right);
39     }
40
41     void update(size_t node, size_t l, size_t r, size_t pos, const T&
42     value) {
43         if (l == r) {
44             tree_[node] = value;
45             return;
46         }

```

```

45     size_t mid = 1 + (r - 1) / 2;
46     if (pos <= mid) update(2 * node, 1, mid, pos, value);
47     else update(2 * node + 1, mid + 1, r, pos, value);
48     tree_[node] = combine_(tree_[2 * node], tree_[2 * node + 1]);
49 }
50
51 size_t n_;
52 std::vector<T> tree_;
53 Combine combine_;
54 };

```

14.3.3 Query Walkthrough

Query sum over [1, 5] for array [1, 3, 5, 7, 9, 11] (0-indexed):

```

1 root covers [0, 5], query = [1, 5]:
2 left child [0, 2] overlaps -> recurse
3   [0, 0]: no overlap
4   [1, 2]: total overlap -> return 8
5 right child [3, 5] overlaps -> recurse
6   [3, 5]: total overlap -> return 27
7 combine(8, 27) = 35
8
9 Check: 3 + 5 + 7 + 9 + 11 = 35 (OK)

```

14.3.4 Lazy Propagation

For **range updates** (e.g., add a value to every element in [l, r]), we use lazy propagation — defer the update to children until necessary.

```

1 void range_update(size_t node, size_t l, size_t r, size_t ql, size_t qr,
2   const T& delta) {
3     push(node, l, r); // propagate any pending updates
4     if (ql > r || qr < l) return;
5     if (ql <= l && r <= qr) {
6       lazy_[node] += delta; // mark for propagation
7       tree_[node] += delta * (r - l + 1); // apply to current node
8       push(node, l, r); // propagate immediately
9       return;
10    }
11    size_t mid = 1 + (r - l) / 2;
12    range_update(2 * node, l, mid, ql, qr, delta);
13    range_update(2 * node + 1, mid + 1, r, ql, qr, delta);
14    tree_[node] = combine_(tree_[2 * node], tree_[2 * node + 1]);
15 }

```

14.3.5 Application: Range Minimum Query

```

1 auto min_combine = [](int a, int b) { return std::min(a, b); };
2 segment_tree<int, decltype(min_combine)> st(data, min_combine);
3
4 // Query minimum in range [2, 5]
5 int min_val = st.query(2, 5);

```

14.4 Fenwick Tree (Binary Indexed Tree)

A **Fenwick tree** (BIT) is a simpler alternative to segment trees for prefix queries and point updates. It uses less memory (n vs $4n$) and is faster in practice.

Operations:

- **Prefix query** (sum over $[0, i)$): $O(\log n)$
- **Point update** (add delta at index i): $O(\log n)$
- **Range query** (sum over $[l, r)$): $O(\log n)$ via $\text{prefix}(r) - \text{prefix}(l)$

14.4.1 The Idea

The Fenwick tree stores partial sums in an array where index i stores the sum of a range of length $(i \& -i)$ — the lowest set bit of i .

```

1 Array (1-indexed): [1, 3, 5, 7, 9, 11]
2
3 Fenwick tree (BIT):
4 BIT[1] = arr[1] = 1
5 BIT[2] = arr[1] + arr[2] = 4
6 BIT[3] = arr[3] = 5
7 BIT[4] = arr[1] + arr[2] + arr[3] + arr[4] = 16
8 BIT[5] = arr[5] = 9
9 BIT[6] = arr[5] + arr[6] = 20
10
11 To get prefix sum up to index 5:
12   BIT[5] = 9
13   i -= (i & -i) -> i = 4
14   BIT[4] = 16
15   i -= (i & -i) -> i = 0
16   Total = 9 + 16 = 25
17   Check: 1+3+5+7+9 = 25 (OK)

```

14.4.2 Implementation

```

1  template <typename T>
2  class fenwick_tree {
3  public:
4      explicit fenwick_tree(size_t n) : bit_(n + 1, T{}) {}
5
6      void add(size_t idx, T delta) {
7          ++idx; // 1-indexed internally
8          while (idx < bit_.size()) {
9              bit_[idx] += delta;
10             idx += idx & -idx;
11         }
12     }
13
14     T prefix_sum(size_t idx) const {
15         // Sum over [0, idx)
16         T result{};
17         while (idx > 0) {
18             result += bit_[idx];
19             idx -= idx & -idx;
20         }
21         return result;
22     }
23
24     T range_sum(size_t l, size_t r) const {
25         // Sum over [l, r)
26         return prefix_sum(r) - prefix_sum(l);
27     }
28
29     // Find smallest index such that prefix_sum >= target
30     size_t lower_bound(T target) const {
31         size_t idx = 0;
32         size_t bit_mask = 1;
33         while (bit_mask < bit_.size()) bit_mask <<= 1;
34         bit_mask >>= 1;
35
36         while (bit_mask > 0) {
37             size_t next = idx + bit_mask;
38             if (next < bit_.size() && bit_[next] < target) {
39                 target -= bit_[next];
40                 idx = next;
41             }
42             bit_mask >>= 1;
43         }
44         return idx; // 0-indexed
45     }
46
47 private:
48     std::vector<T> bit_;

```

```
49 };
```

14.4.3 Application: Inversion Count

Count pairs (i, j) with $i < j$ and $\text{arr}[i] > \text{arr}[j]$:

```
1 size_t inversion_count(std::span<const int> arr) {
2     // Coordinate compress
3     auto sorted = std::vector<int>(arr.begin(), arr.end());
4     std::sort(sorted.begin(), sorted.end());
5     sorted.erase(std::unique(sorted.begin(), sorted.end()), sorted.end());
6
7     fenwick_tree<int> bit(sorted.size());
8     size_t inversions = 0;
9
10    for (int x : arr) {
11        size_t pos = std::lower_bound(sorted.begin(), sorted.end(), x) -
sorted.begin();
12        // Count elements greater than x that we've already seen
13        inversions += bit.range_sum(pos + 1, sorted.size());
14        bit.add(pos, 1);
15    }
16    return inversions;
17 }
```

14.5 Union-Find (Disjoint Set Union)

We introduced Union-Find in Chapter 9. Here we give a fuller treatment.

14.5.1 Optimal Implementation

```
1 class union_find {
2 public:
3     union_find(size_t n) : parent_(n), size_(n, 1) {
4         std::iota(parent_.begin(), parent_.end(), 0);
5     }
6
7     size_t find(size_t x) {
8         // Path compression
9         while (parent_[x] != x) {
10             parent_[x] = parent_[parent_[x]]; // halving
11             x = parent_[x];
12         }
13         return x;
14     }
15 }
```



```

16     void union_sets(size_t x, size_t y) {
17         size_t rx = find(x);
18         size_t ry = find(y);
19         if (rx == ry) return;
20
21         // Union by size
22         if (size_[rx] < size_[ry]) std::swap(rx, ry);
23         parent_[ry] = rx;
24         size_[rx] += size_[ry];
25     }
26
27     bool same_set(size_t x, size_t y) {
28         return find(x) == find(y);
29     }
30
31     size_t set_size(size_t x) {
32         return size_[find(x)];
33     }
34
35     size_t count_sets() const {
36         size_t count = 0;
37         for (size_t i = 0; i < parent_.size(); ++i) {
38             if (parent_[i] == i) ++count;
39         }
40         return count;
41     }
42
43 private:
44     std::vector<size_t> parent_;
45     std::vector<size_t> size_;
46 };

```

14.5.2 Complexity

With both path compression and union by size/rank: m operations on n elements take $O(m \cdot \alpha(n))$ time, where $\alpha(n)$ is the inverse Ackermann function. For any practical n , $\alpha(n) \leq 5$.

14.5.3 Application: Kruskal's MST

(See Chapter 12 for the full algorithm. Union-Find is the cycle-detection mechanism.)

14.5.4 Application: Percolation

Model whether a fluid can flow through a porous material:

```

1 class percolation {
2 public:
3     percolation(size_t n) : n_(n), uf_(n * n + 2), open_(n * n, false) {

```

```

4      top_ = n * n;           // virtual top node
5      bottom_ = n * n + 1; // virtual bottom node
6  }
7
8  void open(size_t row, size_t col) {
9      size_t idx = row * n_ + col;
10     if (open_[idx]) return;
11     open_[idx] = true;
12
13     if (row == 0) uf_.union_sets(idx, top_);
14     if (row == n_ - 1) uf_.union_sets(idx, bottom_);
15
16     // Connect to open neighbors
17     if (row > 0 && open_[idx - n_]) uf_.union_sets(idx, idx - n_);
18     if (row < n_ - 1 && open_[idx + n_]) uf_.union_sets(idx, idx + n_);
19
20     if (col > 0 && open_[idx - 1]) uf_.union_sets(idx, idx - 1);
21     if (col < n_ - 1 && open_[idx + 1]) uf_.union_sets(idx, idx + 1);
22 }
23
24 bool is_open(size_t row, size_t col) const {
25     return open_[row * n_ + col];
26 }
27
28 bool percolates() const {
29     return uf_.same_set(top_, bottom_);
30 }
31 private:
32     size_t n_;
33     union_find uf_;
34     std::vector<bool> open_;
35     size_t top_, bottom_;
36 };

```

14.6 Interval Tree

An **interval tree** stores a set of intervals and efficiently finds all intervals that overlap with a given query interval. Two intervals $[a, b]$ and $[c, d]$ overlap if and only if $a \leq d$ and $c \leq b$.

14.6.1 Applications

- **Scheduling:** find all meetings that overlap a given time slot
- **Genomic data:** locate genes within a range on a chromosome
- **Computer graphics:** detect collisions between rectangular regions

- **Database range queries:** find all records whose key range overlaps a query range
- **Network intrusion detection:** match incoming packets against flagged IP ranges

14.6.2 Structure

The interval tree is a **BST augmented with an additional field**. Each node stores:

- An interval [low, high]
- max_high: the maximum high endpoint among all intervals in the subtree
- Left and right children

Intervals are ordered by their low endpoint in BST order. The max_high field is the key augmentation that prunes the search.

```

1 Interval Tree with intervals:
2 [15,20], [10,30], [17,19], [12,15], [5,20], [30,40]
3
4 BST ordered by low endpoint:
5
6           [10,30] max=40
7          /      \
8     [5,20] max=20 [17,19] max=40
9        /      /      \
10   (empty) [12,15] max=15 [15,20] max=20
11                        \
12                   [30,40] max=40
13
14 Key: Each node shows [low,high] max=maximum_high_in_subtree

```

14.6.3 Implementation

```

1 struct interval {
2     int low;
3     int high;
4 };
5
6 struct it_node {
7     interval ival;
8     int max_high;
9     it_node* left = nullptr;
10    it_node* right = nullptr;
11
12    explicit it_node(interval i)
13        : ival(i), max_high(i.high) {}
14 };

```

```

15
16 class interval_tree {
17 public:
18     ~interval_tree() { destroy(root_); }
19
20     void insert(interval i) { root_ = insert(root_, i); }
21
22     std::vector<interval> search(interval q) const {
23         std::vector<interval> result;
24         search(root_, q, result);
25         return result;
26     }
27
28 private:
29     it_node* root_ = nullptr;
30
31     it_node* insert(it_node* n, interval i) {
32         if (!n) return new it_node(i);
33         if (i.low < n->ival.low)
34             n->left = insert(n->left, i);
35         else
36             n->right = insert(n->right, i);
37         n->max_high = std::max(n->max_high, i.high);
38         return n;
39     }
40
41     void search(it_node* n, const interval& q,
42               std::vector<interval>& result) const {
43         if (!n) return;
44
45         // Prune left: if q.low > max_high, nothing overlaps
46         if (q.low <= n->max_high) {
47             search(n->left, q, result);
48         }
49
50         // Check current node for overlap
51         if (n->ival.low <= q.high && q.low <= n->ival.high) {
52             result.push_back(n->ival);
53         }
54
55         // Prune right: if q.high <= node.low, nothing overlaps
56         if (q.high > n->ival.low) {
57             search(n->right, q, result);
58         }
59     }
60
61     void destroy(it_node* n) {
62         if (!n) return;

```

```

63     destroy(n->left);
64     destroy(n->right);
65     delete n;
66 }
67 };

```

14.6.4 How the Search Algorithm Works

The search uses three pruning rules:

1. **Left subtree pruning:** If `q.low > node.max_high`, no interval in the left subtree can overlap the query.
2. **Current node check:** If `node.low <= q.high` AND `q.low <= node.high`, the current interval overlaps.
3. **Right subtree pruning:** If `q.high <= node.low`, no interval in the right subtree can overlap.

14.6.5 Complexity

Operation	Time	Extra Space
Insert	$O(\log n)$	$O(1)$ per node
Search	$O(\log n + k)$	$O(k)$ for result
Build from sorted	$O(n \log n)$	$O(n)$

Here n is the number of intervals and k is the number of overlapping intervals returned. The search is *output-sensitive*: it runs in time proportional to the number of results plus the height of the tree.

14.6.6 Application: Meeting Room Scheduler

```

1  #include <iostream>
2
3  int main() {
4      interval_tree scheduler;
5
6      // Insert meetings: [start, end] in hours
7      scheduler.insert({9, 10});
8      scheduler.insert({10, 12});
9      scheduler.insert({11, 13});
10     scheduler.insert({14, 16});
11     scheduler.insert({15, 17});
12     scheduler.insert({19, 20});
13
14     // Query: find all meetings overlapping [11, 15]

```

```
15     auto conflicts = scheduler.search({11, 15});
16
17     for (const auto& m : conflicts) {
18         std::cout << "Overlaps: ["
19             << m.low << ", " << m.high << "]\n";
20     }
21     // Output:
22     // Overlaps: [10, 12]
23     // Overlaps: [14, 16]
24     // Overlaps: [11, 13]
25     // Overlaps: [15, 17]
26 }
```

14.6.7 Interval Tree vs. Brute Force

Method	Per Query	Update
Brute force (scan all)	$O(n)$	$O(1)$ insert to list
Interval tree	$O(\log n + k)$	$O(\log n)$ insert

For $n = 1,000,000$ intervals with $k = 5$ overlaps, brute force examines all 1 million intervals while the interval tree examines roughly 20 nodes plus the 5 results.

When to use brute force instead: if n is small (under ~ 100), if updates are extremely frequent and queries rare, or if you need a simple implementation.

14.7 Comparison

Structure	Build	Query	Update	Range Update	Memory
Sparse Table	$O(n \log n)$	$O(1)$ min/max	No	No	$O(n \log n)$
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$ lazy	$O(4n)$
Fenwick Tree	$O(n \log n)^*$	$O(\log n)$	$O(\log n)$	No	$O(n)$
Union-Find	$O(n)$	$O(\alpha(n))$	$O(\alpha(n))$	N/A	$O(n)$

*Fenwick can be built in $O(n)$ by computing prefix sums first.

When to use which:

- **Sparse table:** static arrays, many min/max/gcd queries, $O(1)$ query needed
- **Fenwick tree:** prefix sums, point updates, no range updates — simplest and fastest
- **Segment tree:** range updates, non-invertible operations (min, max, gcd), complex queries
- **Union-Find:** disjoint set connectivity, graph connectivity, incremental connectivity

14.8 Common Bugs and Pitfalls

Pitfall	Consequence
Segment tree array size = $2n$ (not $4n$)	Buffer overflow on non-power-of-2 arrays or range queries
Forgetting lazy propagation in range updates	Child nodes not updated, queries return stale data
Fenwick tree 1-indexing confusion	Off-by-one on queries, wrong prefix sums
Using Fenwick tree for range minimum	Fenwick only supports invertible operations (sum, xor)
Sparse table with non-idempotent operations (sum)	Overlapping intervals double-count, wrong results
Sparse table on data that changes	Stale answers after updates
Union-Find without path compression	$O(n)$ find, tree becomes linear on tall chains
Union without union by rank	Tree depth grows $O(n)$ in worst case
Reversing union order on disjoint sets	Large tree attached to small, height increases

14.9 Summary

1. **Sparse tables** give $O(1)$ range min/max queries on static arrays with $O(n \log n)$ preprocessing.
2. **Segment trees** provide $O(\log n)$ range queries and updates with $O(n)$ memory.
3. **Fenwick trees** are simpler and faster for prefix sums with point updates.
4. **Union-Find** tracks disjoint sets with amortized near-constant operations.
5. **Segment tree with lazy propagation** supports efficient range updates.
6. **Union-Find with path compression and union by rank** achieves $O(\alpha(n))$ amortized complexity.

14.10 Interview FAQs

1. **When should you use a segment tree over a Fenwick tree, and vice versa?**

Segment tree: range min/max, complex queries (e.g., count of elements $> k$ in range), lazy propagation with non-invertible operations (range assign, range add that can't be reversed). Fenwick tree: prefix sums, prefix XOR, point update + prefix query — simpler code, n space vs $4n$, smaller constants. The decision framework: if your query is a prefix operation and the operation is invertible (sum, XOR, product), use Fenwick. If you need arbitrary range queries, non-invertible lazy updates, or complex merge operations, use segment tree. In competitive programming, Fenwick tree is the default for 80% of range query problems.

Key insight: The choice is not “which is more powerful” but “which matches your operation.” A Fenwick tree is a specialized tool that excels at its niche; a segment tree is the general-purpose fallback.

1. What is a Fenwick tree (Binary Indexed Tree) and how does it compare to a segment tree?

A Fenwick tree supports point update and prefix sum query in $O(\log n)$ using a clever index manipulation (lowest set bit). It's simpler to implement, uses exactly n space (vs $4n$ for segment trees), and has smaller constants. However, it only supports operations that are invertible (sum, XOR — not min/max). For min/max range queries, use a segment tree. For prefix sums with point updates, Fenwick tree is preferred.

```

1 class fenwick {
2     std::vector<int> tree_;
3 public:
4     fenwick(int n) : tree_(n + 1, 0) {}
5     void add(int i, int delta) {
6         for (++i; i < tree_.size(); i += i & (-i))
7             tree_[i] += delta;
8     }
9     int sum(int i) {
10        int s = 0;
11        for (++i; i > 0; i -= i & (-i))
12            s += tree_[i];
13        return s;
14    }
15    int range_sum(int l, int r) { return sum(r) - sum(l - 1); }
16 };

```

1. How do you count inversions in an array?

An inversion is a pair (i, j) where $i < j$ and $A[i] > A[j]$.

Approach 1: Modified merge sort. During the merge step, when an element from the right half is smaller than an element from the left half, it forms inversions with all remaining elements in the left half. Count these. Time $O(n \log n)$, space $O(n)$.

Approach 2: Fenwick tree. Process elements from left to right. For each element $A[i]$, query how many elements $> A[i]$ have been seen: `total_seen - prefix_sum(A[i])`. Time $O(n \log n)$, space $O(n)$ (with coordinate compression).

Applications: Measuring sortedness, recommendation systems, bubble sort analysis.

1. What's the difference between lazy propagation and a difference array, and when do you use each?

Lazy propagation defers range updates to segment tree nodes — $O(\log n)$ per update and query, works for any monoid. A difference array (or BIT with range update) stores differences between adjacent elements — $O(1)$ per update, $O(n)$ per prefix query, works only for additive operations. Use difference arrays when: you do many range additions but only need prefix sums at the end. Use lazy propagation when: you interleave range updates and range queries online. The deeper

insight: lazy propagation is a form of *deferred computation* — you postpone work until it's needed, avoiding unnecessary updates to leaves that are never queried.

1. How does Union-Find achieve near-constant time?

Two optimizations working together: (1) **Path compression**: during `find(x)`, make every node on the path point directly to the root. (2) **Union by rank/size**: always attach the smaller tree under the larger. After m operations on n elements, the amortized cost per operation is $O(\alpha(n))$ where α is the inverse Ackermann function ($\alpha(n) \leq 4$ for any practical n). The proof is one of the most elegant in amortized analysis.

1. What is the difference between a segment tree and a sparse table?

Segment tree: supports updates in $O(\log n)$ and queries in $O(\log n)$. Sparse table: $O(1)$ query for idempotent operations (min, max, gcd) but $O(n \log n)$ build and no updates. Use segment tree when you have updates. Use sparse table for static arrays with many queries (e.g., range minimum queries on a fixed array). Sparse table uses $O(n \log n)$ space; segment tree uses $O(n)$.

1. How do you find the k-th smallest element in a dynamic array with updates?

Use a Fenwick tree with coordinate compression. Compress all possible values to ranks $1..n$. Maintain a Fenwick tree where `add(rank, 1)` inserts and `add(rank, -1)` deletes. To find the k -th smallest, binary search on the Fenwick tree prefix sums: find the smallest index where `sum(index) ≥ k`. Time $O(\log^2 n)$ per query, $O(\log n)$ per update.

1. How do you solve the “range update + range query” problem efficiently?

Use a segment tree with lazy propagation, or use two Fenwick trees. The two-Fenwick-tree trick: to add v to all elements in $[l, r]$, do `BIT1.add(l, v)`, `BIT1.add(r+1, -v)`, `BIT2.add(l, v*(l-1))`, `BIT2.add(r+1, -v*r)`. Prefix sum at i is `BIT1.sum(i) * i - BIT2.sum(i)`. Range sum $[l, r]$ is `prefix(r) - prefix(l-1)`. Time $O(\log n)$ per operation.

1. What is the practical difference between path compression and path halving in Union-Find?

Path compression: during `find(x)`, make every node on the path point directly to the root. Path halving: only update every other node (skip one level). Both give the same amortized complexity $O(\alpha(n))$, but path halving uses less writes and is slightly faster in practice because it modifies fewer pointers. Path splitting (update every node to point to its grandparent) is another variant. In practice, the difference is negligible — any of the three with union by rank is near-optimal.

1. When would you use a segment tree over a Fenwick tree?

Use a segment tree when: (1) you need range minimum/maximum queries (Fenwick only supports invertible operations), (2) you need complex range queries (e.g., number of distinct values in a range), (3) you need lazy propagation for range updates with non-invertible operations (range assignment, range multiply). Use a Fenwick tree when: (1) you only need prefix sums (or range sums), (2) you want simpler code with smaller constants, (3) memory is tight.

14.11 Persistent Segment Tree

A standard segment tree supports updates, but each update destroys the previous version. A **persistent segment tree** preserves every historical version while sharing structure between versions. The idea is **path copying**: when updating a path from root to leaf, copy only the nodes along that path; all other nodes are shared with previous versions.

14.11.1 Structure

Each version of the tree is a root pointer. Versions share all unchanged subtrees. An update creates at most $O(\log n)$ new nodes along one root-to-leaf path.

```

1 Version 0 (original):          Version 1 (update pos 3):
2       [28]                      [28]
3       /   \                    /   \
4       [9]   [19]              [9]   [21] <-- new node
5       / \   / \              / \   / \
6      [4] [5][9] [10]         [4] [5][9] [10]
7                               \       \
8                               [7]      [10]
9                               \       \
10                              [7]      [10]
11
12 Shared nodes: [9], [4], [5], [9], [10], [7], [10]
13 New nodes in version 1: [21], and the path to pos 3

```

14.11.2 Implementation

```

1 struct pst_node {
2     int sum = 0;
3     int left = -1;    // index in node pool
4     int right = -1;
5 };
6
7 class persistent_seg_tree {
8 public:
9     explicit persistent_seg_tree(std::span<const int> data)
10         : n_(data.size()) {
11         versions_.push_back(build(data, 0, n_ - 1));
12     }
13
14     // Create new version by updating pos with val
15     int update(int version, size_t pos, int val) {
16         int new_root = update(versions_[version], 0, n_ - 1, pos, val);
17         versions_.push_back(new_root);
18         return versions_.size() - 1;    // return version index
19     }
20 };

```

```

19     }
20
21     // Query sum over [l, r] on a given version
22     int query(int version, size_t l, size_t r) {
23         return query(versions_[version], 0, n_ - 1, l, r);
24     }
25
26     int num_versions() const { return versions_.size(); }
27
28 private:
29     std::vector<pst_node> pool_;
30     std::vector<int> versions_; // root index for each version
31     size_t n_;
32
33     int new_node(int sum, int left, int right) {
34         pool_.push_back({sum, left, right});
35         return pool_.size() - 1;
36     }
37
38     int build(std::span<const int> data, size_t l, size_t r) {
39         if (l == r) return new_node(data[l], -1, -1);
40         size_t mid = l + (r - l) / 2;
41         int lc = build(data, l, mid);
42         int rc = build(data, mid + 1, r);
43         return new_node(pool_[lc].sum + pool_[rc].sum, lc, rc);
44     }
45
46     int update(int node, size_t l, size_t r, size_t pos, int val) {
47         if (l == r) return new_node(val, -1, -1);
48         size_t mid = l + (r - l) / 2;
49         int lc = pool_[node].left;
50         int rc = pool_[node].right;
51         if (pos <= mid) lc = update(lc, l, mid, pos, val);
52         else rc = update(rc, mid + 1, r, pos, val);
53         return new_node(pool_[lc].sum + pool_[rc].sum, lc, rc);
54     }
55
56     int query(int node, size_t l, size_t r, size_t ql, size_t qr) {
57         if (node == -1 || ql > r || qr < l) return 0;
58         if (ql <= l && r <= qr) return pool_[node].sum;
59         size_t mid = l + (r - l) / 2;
60         return query(pool_[node].left, l, mid, ql, qr)
61             + query(pool_[node].right, mid + 1, r, ql, qr);
62     }
63 };

```

14.11.3 Query on Historical Versions

```

1 persistent_seg_tree pst(arr);
2
3 // Version 0: original array
4 pst.query(0, 2, 5); // sum of arr[2..5]
5
6 // Update position 3 to value 100
7 int v1 = pst.update(0, 3, 100);
8
9 // Query version 1: includes the update
10 pst.query(v1, 2, 5); // sum with new value at pos 3
11
12 // Original version 0 is unchanged
13 pst.query(0, 2, 5); // original sum

```

14.11.4 Complexity

- **Build:** $O(n)$ time, $O(n)$ nodes
- **Update:** $O(\log n)$ new nodes per update, $O(\log n)$ time
- **Query:** $O(\log n)$ time
- **Space:** $O(n + Q \log n)$ for Q total updates (each adds $\sim \log n$ nodes)

Without persistence, Q updates on an array of size n would require storing Q full copies of the array: $O(Qn)$ space. Persistence reduces this to $O(n + Q \log n)$.

14.11.5 Application: Range Count Queries

A common application is counting how many elements in a subarray $A[l..r]$ are less than or equal to x . Build a persistent segment tree over coordinate-compressed values. Each version i corresponds to the prefix $A[0..i]$.

```

1 // After building persistent seg tree over prefix sums of
2 // frequency arrays (one version per prefix):
3 // Query: count of elements <= x in A[l..r]
4 //   = query(version r+1, 0, x) - query(version l, 0, x)

```

This gives $O(\log n)$ per query. Combined with binary search, it solves the “k-th smallest in range” problem in $O(\log^2 n)$.

14.12 Square Root Decomposition

Sqrt decomposition (or block decomposition) is a general technique that divides an array into blocks of size $\sqrt{n}$ and precomputes a summary for each block. It sits between brute force $O(n)$ and heavy structures like segment trees $O(\log n)$.

14.12.1 The Idea

Divide array $A[0..n-1]$ into blocks of size $B = \lceil \sqrt{n} \rceil$. Precompute a summary for each block (sum, min, max, etc.). To answer a range query:

1. Handle boundary elements (partial blocks at the ends) by brute force
2. Combine precomputed block summaries for the fully covered blocks in the middle

```

1 Array: [1, 3, 5, 7, 9, 11, 2, 4, 6, 8]
2 n = 10, B = 4 (block size)
3
4 Block 0: [1, 3, 5, 7] -> sum = 16, min = 1
5 Block 1: [9, 11, 2, 4] -> sum = 26, min = 2
6 Block 2: [6, 8] -> sum = 14, min = 6
7
8 Query sum(2, 8):
9   Boundary left: A[2] + A[3] = 5 + 7 = 12 (partial block 0)
10  Full blocks:   sum(block 1) = 26
11  Boundary right: A[8] = 6 (partial block 2)
12  Total: 12 + 26 + 6 = 44
13  Check: 5+7+9+11+2+4+6 = 44 (OK)

```

14.12.2 Implementation: Range Sum and Range Min

```

1 class sqrt_decomposition {
2 public:
3     explicit sqrt_decomposition(std::span<const int> data)
4         : n_(data.size()),
5           block_size_(static_cast<size_t>(std::sqrt(n_) + 1)),
6           num_blocks_((n_ + block_size_ - 1) / block_size_) {
7         block_sum_.assign(num_blocks_, 0);
8         block_min_.assign(num_blocks_, INT_MAX);
9
10        for (size_t i = 0; i < n_; ++i) {
11            block_sum_[i / block_size_] += data[i];
12            block_min_[i / block_size_] = std::min(
13                block_min_[i / block_size_], data[i]);
14        }
15        data_.assign(data.begin(), data.end());
16    }
17
18    // Range sum query over [l, r] inclusive
19    int query_sum(size_t l, size_t r) const {
20        int result = 0;
21        size_t bl = l / block_size_;
22        size_t br = r / block_size_;

```

```

23
24     if (bl == br) {
25         for (size_t i = l; i <= r; ++i) result += data_[i];
26         return result;
27     }
28     // Left partial block
29     for (size_t i = l; i < (bl + 1) * block_size_; ++i)
30         result += data_[i];
31     // Full blocks
32     for (size_t b = bl + 1; b < br; ++b)
33         result += block_sum_[b];
34     // Right partial block
35     for (size_t i = br * block_size_; i <= r; ++i)
36         result += data_[i];
37     return result;
38 }
39
40 // Range min query over [l, r] inclusive
41 int query_min(size_t l, size_t r) const {
42     int result = INT_MAX;
43     size_t bl = l / block_size_;
44     size_t br = r / block_size_;
45
46     if (bl == br) {
47         for (size_t i = l; i <= r; ++i)
48             result = std::min(result, data_[i]);
49         return result;
50     }
51     for (size_t i = l; i < (bl + 1) * block_size_; ++i)
52         result = std::min(result, data_[i]);
53     for (size_t b = bl + 1; b < br; ++b)
54         result = std::min(result, block_min_[b]);
55     for (size_t i = br * block_size_; i <= r; ++i)
56         result = std::min(result, data_[i]);
57     return result;
58 }
59
60 // Point update
61 void update(size_t pos, int val) {
62     size_t b = pos / block_size_;
63     block_sum_[b] -= data_[pos];
64     block_sum_[b] += val;
65     data_[pos] = val;
66     // Recompute block min (O(B) worst case)
67     block_min_[b] = INT_MAX;
68     size_t start = b * block_size_;
69     size_t end = std::min(start + block_size_, n_);
70     for (size_t i = start; i < end; ++i)

```

```
71         block_min_[b] = std::min(block_min_[b], data_[i]);
72     }
73
74     private:
75         size_t n_, block_size_, num_blocks_;
76         std::vector<int> data_;
77         std::vector<int> block_sum_;
78         std::vector<int> block_min_;
79     };
```

14.12.3 Complexity

Operation	Time	Space
Build	$O(n)$	$O(n)$
Query (sum/min)	$O(\sqrt{n})$	$O(1)$
Point update	$O(\sqrt{n})$	$O(1)$

14.12.4 When to Use

- **Online queries on static or mildly dynamic arrays** where a segment tree feels like overkill
- **Problems that don’t decompose nicely** into segment tree nodes (e.g., mex queries, mode queries)
- **Constant factor matters:** sqrt decomposition has very small constants compared to segment trees
- **Mo’s algorithm** (offline range queries using sqrt decomposition) handles queries that segment trees cannot, like distinct-value counting

Tradeoffs: $O(\sqrt{n})$ is worse than $O(\log n)$ for large n , but the simplicity and cache-friendly access pattern make it competitive in practice for $n \leq 10^5$.

14.13 Sparse Table for GCD/LCM

The sparse table’s key requirement is that the operation must be **idempotent**: $f(x, x) = x$. GCD satisfies this: $\text{gcd}(x, x) = x$. This means the same $O(1)$ query technique from RMQ applies directly to range GCD queries.

14.13.1 GCD Queries

Replace min with gcd in the sparse table:

```
1 class sparse_table_gcd {
2 public:
```

```

3     explicit sparse_table_gcd(std::span<const int> data)
4         : n_(data.size()), log_(std::bit_width(n_) - 1) {
5         st_.assign(n_, std::vector<int>(log_ + 1));
6         for (size_t i = 0; i < n_; ++i)
7             st_[i][0] = data[i];
8
9         for (size_t j = 1; j <= log_; ++j) {
10             for (size_t i = 0; i + (1ULL << j) <= n_; ++i) {
11                 st_[i][j] = std::gcd(st_[i][j-1],
12                                     st_[i + (1ULL << (j-1))][j-1]);
13             }
14         }
15     }
16
17     // Query gcd over [l, r] (0-indexed, inclusive)
18     int query_gcd(size_t l, size_t r) const {
19         size_t k = std::bit_width(r - l + 1) - 1;
20         return std::gcd(st_[l][k], st_[r - (1ULL << k) + 1][k]);
21     }
22
23 private:
24     size_t n_, log_;
25     std::vector<std::vector<int>> st_;
26 };

```

14.13.2 Why GCD Works with Overlapping Intervals

For sum queries, overlapping intervals double-count. For GCD, overlap is harmless:

$$\gcd(a, b, c) = \gcd(\gcd(a, b), c) = \gcd(a, \gcd(b, c))$$

If we compute $\gcd(A[1..4], A[3..6])$, the overlap at positions 3 and 4 does not affect the result because $\gcd(x, x) = x$. This is the idempotent property.

```

1 Array: [12, 18, 24, 36, 48, 60]
2
3 Query gcd(1, 4): range = [18, 24, 36, 48]
4   k = floor(log2(4)) = 2
5   gcd(st[1][2], st[3][2])
6     st[1][2] = gcd(18, 24, 36, 48) = 6
7     st[3][2] = gcd(36, 48) = 12   (note: intervals overlap at 36, 48)
8   gcd(6, 12) = 6
9
10 Check: gcd(18, 24, 36, 48) = 6 (OK)

```


14.13.3 LCM Queries

Range LCM queries are trickier because $\text{lcm}(x, x) = x$ is idempotent, but intermediate values grow rapidly and overflow. In practice, maintain values modulo a large prime or use big integers:

```

1 // For LCM, use __int128 or modular arithmetic
2 // lcm(a, b) = a / gcd(a, b) * b
3 // Apply mod at each step to prevent overflow
4 int lcm_mod(int a, int b, int mod) {
5     return (long long)a / std::gcd(a, b) * b % mod;
6 }
```

14.13.4 Complexity

Same as RMQ sparse table: $O(n \log n)$ build, $O(1)$ query, $O(n \log n)$ memory. The only difference is replacing min with gcd.

Key insight: Any idempotent associative operation can use the sparse table technique. Min, max, GCD, LCM (with overflow handling) all qualify. Non-idempotent operations like sum or XOR cannot use the overlapping-interval trick.

14.14 Segment Tree Beats

Standard lazy propagation handles range add and range assign efficiently, but range minimum/maximum update (chmin/chmax) is harder. If we want to set $A[i] = \min(A[i], v)$ for all $i \in [l, r]$, a naive segment tree would need $O(n)$ per operation in the worst case. **Segment tree beats** solves this by maintaining extra information in each node to prune unnecessary recursion.

14.14.1 Core Idea

Each node stores:

- `mx`: the maximum value in the range
- `se`: the second maximum (strictly less than `mx`)
- `cmx`: count of elements equal to `mx`
- `sum`: sum of all values in the range

When applying `chmin(v)` to a node, if $v \geq mx$ nothing changes (prune immediately). If $v < mx$ but $v > se$, only the `cmx` elements equal to `mx` get reduced to v — we can update `sum` and `mx` in $O(1)$ without recursing further. Otherwise, we recurse into children.

This gives an amortized $O(\log n)$ per operation. The key lemma is that each element's maximum value can only decrease, and it can decrease at most $O(\log C)$ times (where C is the value range), yielding $O(n \log C \log n)$ total complexity over a sequence of operations.

14.14.2 Implementation

```

1 struct SegTreeBeats {
2     int n;
3     vector<long long> sum;
4     vector<int> mx, se, cmx;
5
6     void build(const vector<int>& a, int node, int lo, int hi) {
7         if (lo == hi) {
8             sum[node] = mx[node] = a[lo];
9             se[node] = -1; // -1 means no second max
10            cmx[node] = 1;
11            return;
12        }
13        int mid = (lo + hi) / 2;
14        build(a, 2*node, lo, mid);
15        build(a, 2*node+1, mid+1, hi);
16        pull(node);
17    }
18
19    void pull(int node) {
20        int l = 2*node, r = 2*node+1;
21        sum[node] = sum[l] + sum[r];
22
23        // Merge max and second max from children
24        if (mx[l] == mx[r]) {
25            mx[node] = mx[l];
26            cmx[node] = cmx[l] + cmx[r];
27            se[node] = max(se[l], se[r]);
28        } else if (mx[l] > mx[r]) {
29            mx[node] = mx[l];
30            cmx[node] = cmx[l];
31            se[node] = max(se[l], mx[r]);
32        } else {
33            mx[node] = mx[r];
34            cmx[node] = cmx[r];
35            se[node] = max(mx[l], se[r]);
36        }
37    }
38
39    // Apply chmin: reduce all elements equal to old_mx down to v
40    void apply_chmin(int node, int v) {
41        sum[node] -= (long long) (mx[node] - v) * cmx[node];
42        mx[node] = v;
43    }
44
45    void push(int node, int lo, int hi) {
46        if (lo == hi) return;

```

```

47     // Push chmin to children if needed
48     for (int child : {2*node, 2*node+1}) {
49         if (mx[child] > mx[node]) {
50             apply_chmin(child, mx[node]);
51         }
52     }
53 }
54
55 void range_chmin(int node, int lo, int hi,
56                 int ql, int qr, int v) {
57     if (ql > hi || qr < lo || mx[node] <= v)
58         return;
59     if (ql <= lo && hi <= qr && se[node] < v) {
60         apply_chmin(node, v);
61         return;
62     }
63     push(node, lo, hi);
64     int mid = (lo + hi) / 2;
65     range_chmin(2*node, lo, mid, ql, qr, v);
66     range_chmin(2*node+1, mid+1, hi, ql, qr, v);
67     pull(node);
68 }
69
70 long long range_sum(int node, int lo, int hi,
71                    int ql, int qr) {
72     if (ql > hi || qr < lo) return 0;
73     if (ql <= lo && hi <= qr) return sum[node];
74     push(node, lo, hi);
75     int mid = (lo + hi) / 2;
76     return range_sum(2*node, lo, mid, ql, qr)
77         + range_sum(2*node+1, mid+1, hi, ql, qr);
78 }
79
80 // Public interface
81 SegTreeBeats(const vector<int>& a) {
82     n = a.size();
83     int size = 4 * n;
84     sum.resize(size);
85     mx.resize(size);
86     se.resize(size);
87     cmx.resize(size);
88     build(a, 1, 0, n - 1);
89 }
90
91 void chmin(int l, int r, int v) {
92     range_chmin(1, 0, n - 1, l, r, v);
93 }
94

```

```

95     long long query(int l, int r) {
96         return range_sum(l, 0, n - 1, l, r);
97     }
98 };

```

14.14.3 Complexity and Use Cases

- **Build:** $O(n)$
- **Range chmin/chmax + range sum:** Amortized $O(\log n)$ per operation
- **Space:** $O(n)$

Segment tree beats is essential when problems combine range minimum/maximum updates with range sum, range minimum, or range maximum queries. The amortized bound relies on the observation that the number of “interesting” maximum values decreases geometrically. Extensions support simultaneous chmin, chmax, add, and assign in $O(\log n)$ amortized.

14.15 2D Fenwick Tree

The Fenwick tree (BIT) extends naturally to 2D grids. A **2D BIT** supports point update and rectangle sum query on an $n \times m$ grid, both in $O(\log n \cdot \log m)$ time.

14.15.1 Structure

A 2D BIT is a 2D array `tree[n+1][m+1]` where `tree[i][j]` stores the aggregate of a rectangular subregion. The update and query operations use nested 1D BIT logic: an update at (x,y) propagates upward in i and j independently, and a rectangle query uses the inclusion-exclusion principle.

14.15.2 Implementation

```

1 struct Fenwick2D {
2     int n, m;
3     vector<vector<long long>> tree;
4
5     Fenwick2D(int n, int m) : n(n), m(m),
6         tree(n + 1, vector<long long>(m + 1, 0)) {}
7
8     // Add val at position (x, y) (1-indexed)
9     void update(int x, int y, long long val) {
10         for (int i = x; i <= n; i += i & (-i))
11             for (int j = y; j <= m; j += j & (-j))
12                 tree[i][j] += val;
13     }
14 }

```

```

15 // Prefix sum of rectangle [1..x] x [1..y]
16 long long query(int x, int y) {
17     long long res = 0;
18     for (int i = x; i > 0; i -= i & (-i))
19         for (int j = y; j > 0; j -= j & (-j))
20             res += tree[i][j];
21     return res;
22 }
23
24 // Sum over rectangle [x1..x2] x [y1..y2]
25 long long query(int x1, int y1, int x2, int y2) {
26     return query(x2, y2) - query(x1 - 1, y2)
27         - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
28 }
29 };

```

14.15.3 Complexity and Use Cases

- **Build:** $O(nm)$ (or $O(1)$ if building from scratch with zeroed array)
- **Point update:** $O(\log n \cdot \log m)$
- **Rectangle sum query:** $O(\log n \cdot \log m)$
- **Space:** $O(nm)$

Use the 2D BIT whenever you need to maintain a grid with point updates and 2D prefix sum queries. Common applications include: counting points in a rectangle, 2D range frequency queries, and offline problems where coordinate compression reduces 2D coordinates to manageable ranges. For range add + range sum in 2D, use four 2D BITs with the difference-array technique (analogous to the 1D case).

14.16 Orthogonal Range Trees

An **orthogonal range tree** answers queries of the form: “report (or count) all points (x, y) with $x_1 \leq x \leq x_2$ and $y_1 \leq y \leq y_2$.” This is the fundamental 2D orthogonal range query.

14.16.1 Structure

A 2D range tree is built as a **BST on x-coordinates** (the primary tree). Each node in this primary tree owns an **association structure**: a BST on the y-coordinates of all points in that node’s subtree (the secondary tree). This gives a two-level hierarchy.

```

1 Primary tree (BST on x):
2         [50]
3       /   \
4     [25]   [75]

```

```

5           /  \    /  \
6         [10] [35] [60] [90]
7
8 Each node's association structure (BST on y):
9   [50]: {y = 12, 28, 45, 63, 81}
10  [25]: {y = 8, 20, 33}
11  [10]: {y = 5, 18}

```

14.16.2 Query Decomposition

A 2D range query $[x_1, x_2] \times [y_1, y_2]$ decomposes into $O(\log n)$ canonical nodes in the primary tree whose x-ranges partition $[x_1, x_2]$. For each canonical node, query its secondary tree for $[y_1, y_2]$.

```

1 struct point { int x, y; };
2
3 struct range_node {
4     int x_low, x_high;
5     std::vector<point> sorted_by_y;
6     range_node* left  = nullptr;
7     range_node* right = nullptr;
8 };
9
10 std::vector<point> range_query(range_node* node,
11                               int x1, int x2, int y1, int y2) {
12     std::vector<point> result;
13     if (!node || node->x_low > x2 || node->x_high < x1)
14         return result;
15
16     if (x1 <= node->x_low && node->x_high <= x2) {
17         auto lo = std::lower_bound(node->sorted_by_y.begin(),
18                                   node->sorted_by_y.end(), y1,
19                                   [](const point& p, int val) {
20                                       return p.y < val;
21                                   });
22         auto hi = std::upper_bound(lo, node->sorted_by_y.end(),
23                                   y2,
24                                   [](int val, const point& p) {
25                                       return val < p.y;
26                                   });
27         result.insert(result.end(), lo, hi);
28         return result;
29     }
30
31     auto left_res  = range_query(node->left,  x1, x2, y1, y2);
32     auto right_res = range_query(node->right, x1, x2, y1, y2);
33     result.insert(result.end(),
34                  left_res.begin(), left_res.end());
35     result.insert(result.end(),

```

```

36         right_res.begin(), right_res.end());
37     return result;
38 }

```

Without further optimization, this visits $O(\log n)$ canonical nodes and $O(\log n + k_i)$ points in each secondary tree, giving $O(\log^2 n + k)$ total.

14.16.3 Fractional Cascading

Fractional cascading reduces the per-secondary-tree search from $O(\log n)$ to $O(1)$ by interleaving the sorted lists of sibling nodes. Each secondary tree’s list references “jump points” into its parent’s list, so after the first binary search ($O(\log n)$), every subsequent secondary-tree lookup is $O(1)$.

Variant	Build	Query
Naive (no cascading)	$O(n \log n)$	$O(\log^2 n + k)$
With fractional cascading	$O(n \log n)$	$O(\log n + k)$

14.16.4 Kd-Trees

A **kd-tree** alternates split dimensions at each level (x, y, x, y, ...). It uses only $O(n)$ space (no secondary trees), but query time is $O(\sqrt{n} + k)$ in 2D. Kd-trees are simpler and cache-friendlier than range trees.

```

1 struct kd_node {
2     point p;
3     kd_node* left = nullptr;
4     kd_node* right = nullptr;
5 };
6
7 kd_node* build_kd(std::vector<point>& pts,
8     int depth, int lo, int hi) {
9     if (lo >= hi) return nullptr;
10    int axis = depth % 2;
11    int mid = lo + (hi - lo) / 2;
12    std::nth_element(pts.begin() + lo,
13        pts.begin() + mid, pts.begin() + hi,
14        [axis](const point& a, const point& b) {
15            return axis == 0 ? a.x < b.x : a.y < b.y;
16        });
17    auto* node = new kd_node{pts[mid]};
18    node->left = build_kd(pts, depth + 1, lo, mid);
19    node->right = build_kd(pts, depth + 1, mid + 1, hi);
20    return node;
21 }
22
23 void kd_range(kd_node* node, const point& lo,
24     const point& hi,

```

```

25         std::vector<point>& result, int depth) {
26     if (!node) return;
27     bool in_range = node->p.x >= lo.x && node->p.x <= hi.x
28                 && node->p.y >= lo.y && node->p.y <= hi.y;
29     if (in_range) result.push_back(node->p);
30     int axis = depth % 2;
31     int coord = axis == 0 ? node->p.x : node->p.y;
32     int lo_bound = axis == 0 ? lo.x : lo.y;
33     int hi_bound = axis == 0 ? hi.x : hi.y;
34     if (lo_bound <= coord)
35         kd_range(node->left, lo, hi, result, depth + 1);
36     if (hi_bound >= coord)
37         kd_range(node->right, lo, hi, result, depth + 1);
38 }

```

14.16.5 Applications

- **Computational geometry:** point-in-range, nearest neighbor, closest pair
- **GIS:** spatial queries on map regions (bounding-box searches)
- **Spatial databases:** indexing multidimensional records for range predicates
- **VLSI design:** checking for design-rule violations in rectangular regions

14.16.6 Comparison

Structure	Build	Query	Space
2D Range Tree	$O(n \log n)$	$O(\log^2 n + k)$	$O(n \log n)$
Range Tree + Frac. Cascading	$O(n \log n)$	$O(\log n + k)$	$O(n \log n)$
Kd-tree	$O(n \log n)$	$O(\sqrt{n} + k)$	$O(n)$
Quad-tree	$O(n \log n)$	$O(n)$ worst	$O(n)$

When to use which: range trees when n is large and query count dominates; kd-trees for simplicity and moderate n ; quad-trees when points are uniformly distributed and spatial hashing suffices.

14.17 Higher-Dimensional Segment Trees

A 2D segment tree extends the 1D segment tree to grid data. It is the natural structure for **range sum (or min/max) over a 2D rectangle** with point updates.

14.17.1 Structure

A **2D segment tree** is a segment tree of segment trees. The outer tree is built on rows. Each outer node stores an inner segment tree built on the columns of its row-range. For an $n \times m$ grid, the outer tree has $O(n)$ nodes and each inner tree has $O(m)$ nodes, giving $O(nm)$ total space.

14.17.2 Point Update

Updating grid cell (r, c) by δ propagates through $O(\log n)$ outer nodes. At each outer node, update its inner tree at column c by δ , which takes $O(\log m)$.

```

1 class seg_tree_2d {
2 public:
3     seg_tree_2d(int rows, int cols)
4         : n_(rows), m_(cols), outer_(4 * rows) {
5         for (int i = 0; i < 4 * rows; ++i)
6             outer_[i].assign(4 * cols, 0);
7     }
8
9     void update(int r, int c, int val) {
10         update_outer(1, 0, n_ - 1, r, c, val);
11     }
12
13     int query(int r1, int c1, int r2, int c2) {
14         return query_outer(1, 0, n_ - 1,
15                             r1, c1, r2, c2);
16     }
17
18 private:
19     int n_, m_;
20     std::vector<std::vector<int>>> outer_;
21
22     void update_outer(int node, int lo, int hi,
23                       int r, int c, int val) {
24         if (lo == hi) {
25             update_inner(outer_[node], 1, 0, m_-1,
26                           c, val);
27             return;
28         }
29         int mid = (lo + hi) / 2;
30         if (r <= mid)
31             update_outer(2*node, lo, mid, r, c, val);
32         else
33             update_outer(2*node+1, mid+1, hi, r, c, val);
34         for (int i = 1; i < 4 * m_; ++i)
35             outer_[node][i] = outer_[2*node][i]
36                             + outer_[2*node+1][i];
37     }

```

```

38
39 void update_inner(std::vector<int>& tree, int node,
40                 int lo, int hi, int c, int val) {
41     if (lo == hi) { tree[node] += val; return; }
42     int mid = (lo + hi) / 2;
43     if (c <= mid)
44         update_inner(tree, 2*node, lo, mid, c, val);
45     else
46         update_inner(tree, 2*node+1, mid+1, hi,
47                     c, val);
48     tree[node] = tree[2*node] + tree[2*node+1];
49 }
50
51 int query_outer(int node, int lo, int hi,
52                int r1, int c1, int r2, int c2) {
53     if (r1 > hi || r2 < lo) return 0;
54     if (r1 <= lo && hi <= r2)
55         return query_inner(outer_[node], 1, 0,
56                             m_-1, c1, c2);
57     int mid = (lo + hi) / 2;
58     return query_outer(2*node, lo, mid,
59                       r1, c1, r2, c2)
60         + query_outer(2*node+1, mid+1, hi,
61                      r1, c1, r2, c2);
62 }
63
64 int query_inner(const std::vector<int>& tree,
65                int node, int lo, int hi,
66                int c1, int c2) {
67     if (c1 > hi || c2 < lo) return 0;
68     if (c1 <= lo && hi <= c2) return tree[node];
69     int mid = (lo + hi) / 2;
70     return query_inner(tree, 2*node, lo, mid,
71                       c1, c2)
72         + query_inner(tree, 2*node+1, mid+1, hi,
73                      c1, c2);
74 }
75 };

```

14.17.3 Complexity

Operation	Time	Space
Build	$O(nm)$	$O(nm)$
Point update	$O(\log n \cdot \log m)$	$O(1)$
Rectangle query	$O(\log n \cdot \log m)$	$O(1)$

14.17.4 Lazy Propagation in 2D

2D lazy propagation supports **range add + range sum** on a rectangle. Each outer node carries a lazy tag that is propagated to its inner tree. The inner tree uses standard lazy propagation.

14.17.5 Coordinate Compression

When coordinates are sparse (e.g., points scattered across a $10^9 \times 10^9$ grid), apply **offline coordinate compression**: collect all x - and y -coordinates that appear in updates or queries, sort and unique them, and build the 2D segment tree over the compressed indices.

14.17.6 Applications

- **Image processing**: 2D range sum / integral images for fast rectangle queries
- **Computational geometry**: counting points in a rectangle with updates
- **Heatmaps and grids**: dynamic aggregation over spatial cells

14.18 3D Orthogonal Range Search

A **3D orthogonal range tree** answers queries of the form: “report all points (x, y, z) with $x_1 \leq x \leq x_2$, $y_1 \leq y \leq y_2$, and $z_1 \leq z \leq z_2$.” The natural extension of 2D range trees to three dimensions uses a nested structure: the primary tree is a BST on x -coordinates, each node stores a secondary tree on y -coordinates, and each secondary tree node stores a tertiary tree on z -coordinates.

14.18.1 Structure

The 3D range tree has three levels:

1. **Primary tree**: BST on x -coordinates of all points
2. **Secondary trees**: At each primary node, a BST on y -coordinates of points in that subtree
3. **Tertiary trees**: At each secondary node, a BST on z -coordinates of points in that subtree

With fractional cascading across all three levels, the query time is $O(\log^3 n + k)$ for reporting k points, using $O(n \log^2 n)$ space.

14.18.2 Complexity

Dimension	Query Time	Space	Construction
1D	$O(\log n + k)$	$O(n)$	$O(n \log n)$
2D	$O(\log^2 n + k)$	$O(n \log n)$	$O(n \log^2 n)$
3D	$O(\log^3 n + k)$	$O(n \log^2 n)$	$O(n \log^3 n)$
d -D	$O(\log^d n + k)$	$O(n \log^{d-1} n)$	$O(n \log^d n)$

The space grows by a factor of $\log n$ per dimension, which limits the practicality of range trees beyond 3D. For high-dimensional data, techniques like kd-trees, locality-sensitive hashing, or random projection methods are preferred.

14.18.3 Counting Queries

For *counting* queries (how many points fall in the 3D range, rather than listing them), each tertiary tree node stores a count, and the query sums over $O(\log^3 n)$ nodes. This avoids the $O(k)$ reporting cost and gives $O(\log^3 n)$ query time.

14.19 Semigroup Model

The Fenwick tree is elegant, but it relies on a critical property: the operation must be **invertible** (e.g., subtraction inverts addition, allowing prefix queries via $\text{sum}(r) - \text{sum}(l - 1)$). When the operation has no inverse — such as min, max, or gcd — the Fenwick trick fails. The **semigroup model** handles these cases using a segment tree.

14.19.1 Group vs. Semigroup

Property	Group (invertible)	Semigroup (non-invertible)
Example operations	$+, \times, \oplus$	min, max, gcd
Has identity?	Yes	Yes
Has inverse?	Yes	No
Fenwick tree?	Yes ($O(\log n)$)	No
Segment tree?	Yes ($O(\log n)$)	Yes ($O(\log n)$)
Range query trick	$\text{prefix}(r) - \text{prefix}(l - 1)$	Must traverse both halves

14.19.2 Why Fenwick Fails for min

The Fenwick tree computes prefix queries by summing selected nodes. To get a range query, it subtracts prefix sums. But min has no subtraction: there is no operation that undoes a minimum.

14.19.3 Segment Tree for Semigroup Queries

A segment tree handles non-invertible operations because it traverses both halves of the tree independently and combines results. The query does not rely on prefix subtraction.

```

1 template <typename T, typename Combine>
2 class seg_tree {
3 public:
4     seg_tree(const std::vector<T>& data, Combine combine)
5         : n_(data.size()), tree_(4 * n_),
6           combine_(combine) {

```

```

7      build(data, 1, 0, n_ - 1);
8  }
9
10     T query(int l, int r) {
11         return query(1, 0, n_ - 1, l, r);
12     }
13
14     void update(int pos, const T& val) {
15         update(1, 0, n_ - 1, pos, val);
16     }
17
18 private:
19     void build(const std::vector<T>& data,
20               int node, int lo, int hi) {
21         if (lo == hi) { tree_[node] = data[lo]; return; }
22         int mid = (lo + hi) / 2;
23         build(data, 2 * node, lo, mid);
24         build(data, 2 * node + 1, mid + 1, hi);
25         tree_[node] = combine_(tree_[2*node],
26                               tree_[2*node+1]);
27     }
28
29     T query(int node, int lo, int hi,
30            int ql, int qr) {
31         if (ql > hi || qr < lo) return T{};
32         if (ql <= lo && hi <= qr) return tree_[node];
33         int mid = (lo + hi) / 2;
34         return combine_(
35             query(2*node, lo, mid, ql, qr),
36             query(2*node+1, mid+1, hi, ql, qr));
37     }
38
39     void update(int node, int lo, int hi,
40               int pos, const T& val) {
41         if (lo == hi) { tree_[node] = val; return; }
42         int mid = (lo + hi) / 2;
43         if (pos <= mid)
44             update(2*node, lo, mid, pos, val);
45         else
46             update(2*node+1, mid+1, hi, pos, val);
47         tree_[node] = combine_(tree_[2*node],
48                               tree_[2*node+1]);
49     }
50
51     int n_;
52     std::vector<T> tree_;
53     Combine combine_;
54 };

```

14.19.4 When to Use Which

- **Fenwick tree:** operation is invertible (+, ×, XOR), want simpler code and less memory
- **Segment tree:** operation is non-invertible (min, max, gcd), or need range updates with lazy propagation
- **Sparse table:** static data, idempotent operation, need $O(1)$ queries

14.20 Persistent Union-Find

The standard union-find (Section 14.5) is **not persistent**: path compression overwrites parent pointers, destroying old versions. A **persistent union-find** preserves the full history of union and find operations.

14.20.1 Why Persistence Is Hard

In a standard union-find, path compression modifies parent pointers in place:

```

1 size_t find(size_t x) {
2     while (parent_[x] != x) {
3         parent_[x] = parent_[parent_[x]]; // destructive!
4         x = parent_[x];
5     }
6     return x;
7 }
```

Once `parent_[x]` is overwritten, the previous tree structure is lost.

14.20.2 Fat Nodes

A **fat node** approach stores the full history of changes at each node. Each node records every value its parent pointer has ever taken, along with the version at which each change occurred.

```

1 struct fat_node {
2     std::vector<std::pair<int,int>> history;
3     // (version, parent_value)
4
5     int parent_at(int version) const {
6         int result = -1;
7         for (auto& [v, p] : history) {
8             if (v <= version) result = p;
9             else break;
10        }
11        return result;
12    }
13
14    void set_parent(int version, int new_parent) {
15        history.emplace_back(version, new_parent);
16    }
17 }
```

```

16     }
17 };

```

Without path compression, each union creates $O(1)$ new entries, and a find traverses $O(\log n)$ nodes. Total space: $O(m \log n)$ for m operations on n elements.

14.20.3 Path Copying

Path copying creates a new version of each node along the modification path while sharing all other nodes with previous versions. This is the same technique used in persistent BSTs.

```

1 struct persistent_node {
2     int parent;
3     int rank;
4 };
5
6 class persistent_union_find {
7 public:
8     persistent_union_find(int n) {
9         std::vector<persistent_node> base(n);
10        for (int i = 0; i < n; ++i)
11            base[i] = {i, 0};
12        versions_.push_back(base);
13    }
14
15    int find(int version, int x) {
16        auto& nodes = versions_[version];
17        while (nodes[x].parent != x)
18            x = nodes[x].parent;
19        return x;
20    }
21
22    int union_sets(int version, int x, int y) {
23        auto nodes = versions_.back();
24        int rx = find(version, x);
25        int ry = find(version, y);
26        if (rx == ry) return version;
27
28        if (nodes[rx].rank < nodes[ry].rank)
29            std::swap(rx, ry);
30        nodes[ry].parent = rx;
31        if (nodes[rx].rank == nodes[ry].rank)
32            nodes[rx].rank++;
33
34        versions_.push_back(nodes);
35        return versions_.size() - 1;
36    }
37

```

```

38 private:
39     std::vector<std::vector<persistent_node>> versions_;
40 };

```

14.20.4 Complexity

Technique	Per Operation	Space per Op	Path Compression
Fat nodes (no compression)	$O(\log n)$ find	$O(1)$	No
Fat nodes + compression	$O(\log n)$ amortized	$O(\log n)$ amortized	Yes
Path copying	$O(\log n)$ find	$O(\log n)$	No
Path copying + compression	$O(\log n)$ amortized	$O(\log n)$	Yes

14.21 List Splitting

Given a sorted list and a predicate P , the *list splitting* problem divides the list into two sorted sublists: elements satisfying P on the left and elements not satisfying P on the right. This is a fundamental primitive in functional data structures.

14.21.1 Applications

- **Priority queue splitting:** divide into high-priority and low-priority tasks
- **Range partitioning:** split a dataset by a threshold for parallel processing
- **Database indexing:** partition records by a predicate for selective queries
- **Compiler optimization:** split live ranges to reduce register pressure

14.21.2 Splitting with Treaps

A treap assigns random priorities to nodes and maintains the BST property on keys and the heap property on priorities. The split operation traverses from the root, placing nodes into left and right subtrees depending on whether they satisfy the predicate.

```

1 #include <random>
2 #include <utility>
3
4 struct Node {
5     int key;
6     int priority;
7     int size;
8     Node *left, *right;
9     Node(int k, int p)
10         : key(k), priority(p), size(1),

```



```

11         left(nullptr), right(nullptr) {}
12     };
13
14     int sz(Node *n) { return n ? n->size : 0; }
15
16     void update(Node *n) {
17         if (n) n->size = 1 + sz(n->left) + sz(n->right);
18     }
19
20     std::pair<Node*, Node*> split(Node *root,
21         auto pred) {
22         if (!root) return {nullptr, nullptr};
23         if (pred(root->key)) {
24             auto [l, r] = split(root->right, pred);
25             root->right = l;
26             update(root);
27             return {root, r};
28         } else {
29             auto [l, r] = split(root->left, pred);
30             root->left = r;
31             update(root);
32             return {l, root};
33         }
34     }
35
36     Node* merge(Node *a, Node *b) {
37         if (!a || !b) return a ? a : b;
38         if (a->priority > b->priority) {
39             a->right = merge(a->right, b);
40             update(a);
41             return a;
42         } else {
43             b->left = merge(a, b->left);
44             update(b);
45             return b;
46         }
47     }

```

Data Structure	Split Time	Merge Time	Space
Sorted Array	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$ extra
Treap	$O(\log n)$ amortized	$O(\log n)$ amortized	$O(n)$
Weight-Balanced Tree	$O(\log n)$	$O(\log n)$	$O(n)$
Skip List	$O(\log n)$ avg	$O(\log n)$ avg	$O(n)$

14.22 Maintenance of Linear Order

The *linear order maintenance* problem requires supporting insertions, deletions, and rank queries on a dynamic set of elements. We maintain an ordered list and answer queries of the form “what is the rank of element x ?” and “what is the element at rank k ?”

14.22.1 Order-Statistic Trees

An *order-statistic tree* is an augmented balanced BST (typically a red-black tree) where each node stores the size of its subtree. This single augmentation enables $O(\log n)$ rank computation and index-based selection.

Operation	Time	Description
Insert (k)	$O(\log n)$	Insert key and update subtree sizes
Delete (k)	$O(\log n)$	Remove key and update subtree sizes
Rank (k)	$O(\log n)$	Position of k in sorted order
Select (i)	$O(\log n)$	i -th smallest element

14.22.2 Applications

Order-statistic trees underpin dynamic ranking in leaderboards, line-number computation in text editors, ranked retrieval in databases, and online quantile estimation.

14.23 Union of Intervals

Given a collection of n closed intervals $[a_i, b_i]$, the *interval union* problem computes the total measure of their union, expressed as a set of disjoint merged intervals. This is a classic sweep-line problem.

14.23.1 Sweep-Line Algorithm

The algorithm sorts all interval endpoints and performs a sweep. We maintain a counter indicating the number of intervals covering the current position. When the counter transitions from 0 to positive, a new merged interval begins. When it transitions from positive to 0, the interval ends.

```

1 #include <vector>
2 #include <algorithm>
3 #include <utility>
4
5 std::vector<std::pair<int, int>> mergeIntervals(
6     std::vector<std::pair<int, int>>& intervals) {
7     if (intervals.empty()) return {};
8
9     std::vector<std::pair<int, int>> events;
10    for (auto [l, r] : intervals) {

```

```
11     events.push_back({l, 1});
12     events.push_back({r, -1});
13 }
14 std::sort(events.begin(), events.end());
15
16 std::vector<std::pair<int,int>> result;
17 int count = 0, start = 0;
18 for (auto [pos, delta] : events) {
19     if (count == 0 && delta == 1) start = pos;
20     count += delta;
21     if (count == 0 && delta == -1)
22         result.push_back({start, pos});
23 }
24 return result;
25 }
26
27 int totalUnionLength(
28     std::vector<std::pair<int,int>>& intervals) {
29     auto merged = mergeIntervals(intervals);
30     int total = 0;
31     for (auto [l, r] : merged) total += r - l;
32     return total;
33 }
```

14.23.2 Dynamic Interval Union

For dynamic settings where intervals arrive and depart, an augmented interval tree can be used. Each interval tree node stores the maximum endpoint in its subtree, enabling $O(\log n)$ insertion and deletion. The union length is maintained by keeping a secondary structure over the merged intervals.

Approach	Time	Space	Notes
Sort + Sweep	$O(n \log n)$	$O(n)$	Static input, offline
Interval Tree (dynamic)	$O(\log n)$ per op	$O(n)$	Insert/remove intervals
Segment Tree (dynamic)	$O(\log n)$ per op	$O(n)$	Compressed coordinates

14.24 Interval-Restricted Maximum Sum

Given an array $A[1..n]$ and queries $[l, r]$, the *interval-restricted maximum sum* problem finds the maximum sum of a contiguous subarray within $A[l..r]$. This extends Kadane’s algorithm to an indexed-query setting.

14.24.1 Segment Tree Augmentation

A segment tree on array A stores at each node four values: the maximum subarray sum (`best`), the maximum prefix sum (`pref`), the maximum suffix sum (`suff`), and the total sum (`total`). These combine in $O(1)$ per merge:

$$\begin{aligned} \text{total} &= L.\text{total} + R.\text{total} \\ \text{pref} &= \max(L.\text{pref}, L.\text{total} + R.\text{pref}) \\ \text{suff} &= \max(R.\text{suff}, R.\text{total} + L.\text{suff}) \\ \text{best} &= \max(L.\text{best}, R.\text{best}, L.\text{suff} + R.\text{pref}) \end{aligned}$$

```

1  #include <vector>
2  #include <climits>
3
4  struct Info {
5      long long total, pref, suff, best;
6      Info() : total(0), pref(0), suff(0), best(0) {}
7      Info(long long v)
8          : total(v), pref(v), suff(v), best(v) {}
9  };
10
11 Info merge(const Info &a, const Info &b) {
12     Info res;
13     res.total = a.total + b.total;
14     res.pref = std::max(a.pref, a.total + b.pref);
15     res.suff = std::max(b.suff, b.total + a.suff);
16     res.best = std::max({a.best, b.best,
17                         a.suff + b.pref});
18     return res;
19 }
20
21 class MaxSubarrayTree {
22     int n;
23     std::vector<Info> tree;
24     std::vector<long long> arr;
25
26     void build(int node, int lo, int hi) {
27         if (lo == hi) {
28             tree[node] = Info(arr[lo]);
29             return;
30         }
31         int mid = (lo + hi) / 2;
32         build(2*node, lo, mid);
33         build(2*node+1, mid+1, hi);
34         tree[node] = merge(tree[2*node],

```

```

35         tree[2*node+1]);
36     }
37
38     void update(int node, int lo, int hi,
39               int idx, long long val) {
40         if (lo == hi) {
41             arr[idx] = val;
42             tree[node] = Info(val);
43             return;
44         }
45         int mid = (lo + hi) / 2;
46         if (idx <= mid)
47             update(2*node, lo, mid, idx, val);
48         else
49             update(2*node+1, mid+1, hi, idx, val);
50         tree[node] = merge(tree[2*node],
51                           tree[2*node+1]);
52     }
53
54     Info query(int node, int lo, int hi,
55              int ql, int qr) {
56         if (ql <= lo && hi <= qr) return tree[node];
57         int mid = (lo + hi) / 2;
58         if (qr <= mid)
59             return query(2*node, lo, mid, ql, qr);
60         if (ql > mid)
61             return query(2*node+1, mid+1, hi, ql, qr);
62         return merge(query(2*node, lo, mid, ql, qr),
63                     query(2*node+1, mid+1, hi, ql, qr));
64     }
65
66 public:
67     MaxSubarrayTree(const std::vector<long long>& a)
68         : n(a.size()), tree(4*n), arr(a) {
69         if (n > 0) build(1, 0, n-1);
70     }
71
72     void update(int idx, long long val) {
73         update(1, 0, n-1, idx, val);
74     }
75
76     long long query(int l, int r) {
77         return query(1, 0, n-1, l, r).best;
78     }
79 };

```

14.24.2 Comparison

Approach	Query Time	Preprocessing
Kadane per query	$O(n)$	$O(1)$
Segment tree (augmented)	$O(\log n)$	$O(n)$
Mo's + segment tree	$O(\sqrt{n} \log n)$ amortized	$O(n)$
Sparse table (static)	$O(1)$	$O(n \log n)$

14.25 Level Ancestor Problem

The level ancestor problem asks: given a rooted tree T and a node v at depth d , find the ancestor of v at depth $d - k$. This query is denoted $\text{LA}(v, k)$.

14.25.1 Jump Pointers

Each node stores a pointer to its 2^i -th ancestor. With $O(\lg n)$ pointers per node, preprocessing takes $O(n \lg n)$ and a query decomposes k in binary, taking $O(\lg n)$.

```

1 class JumpPointerTree {
2     std::vector<std::vector<int>>> up;
3     std::vector<int> depth;
4     int n;
5
6     void preprocess(int v, int p) {
7         up[v][0] = p;
8         for (int j = 1; (1 << j) <= n; j++)
9             up[v][j] = up[v][j-1] != -1
10                ? up[up[v][j-1]][j-1] : -1;
11         for (int c : adj[v]) {
12             depth[c] = depth[v] + 1;
13             preprocess(c, v);
14         }
15     }
16
17     int query(int v, int k) {
18         for (int j = 0; k > 0; j++) {
19             if (k & 1) v = up[v][j];
20             k >>= 1;
21         }
22         return v;
23     }
24 };

```

14.25.2 Optimal $O(1)$ Query

The optimal solution reduces LA to LCA via the Euler tour. Construct the Euler tour of T and record each node's first occurrence. Then LA reduces to RMQ on an array of $2n - 1$ entries. With the four-Russians trick, RMQ answers in $O(1)$ with $O(n)$ preprocessing, yielding $O(1)$ level ancestor queries.

14.25.3 Applications

- **Tree embedding:** embed a general tree into a caterpillar while preserving depths
- **Balanced separators:** identify separator vertices in planar graphs
- **LCA reduction:** computing distance between two nodes reduces to LA

14.26 The Predecessor Problem and Lower Bounds

The predecessor problem: maintain a set S of keys from universe $U = \{0, \dots, 2^w - 1\}$, supporting insert, delete, $\text{pred}(x) = \max\{y \in S : y \leq x\}$, and $\text{succ}(x)$.

14.26.1 Information-Theoretic Lower Bound

Any data structure must use $\Omega(\lg n)$ bits per element. For comparison-based methods, this implies $\Omega(\lg n)$ time per operation.

14.26.2 Cell Probe Lower Bound

In the cell probe model (only memory accesses count), the round elimination technique proves:

$$t = \Omega\left(\min\left(\lg w, \frac{\lg n}{\lg \lg n}\right)\right)$$

for any cell probe data structure supporting predecessor queries.

14.26.3 Upper Bounds

Data Structure	Query	Update	Space
Balanced BST	$O(\lg n)$	$O(\lg n)$	$O(n)$
van Emde Boas	$O(\lg \lg u)$	$O(\lg \lg u)$	$O(n \lg \lg u)$
Fusion tree	$O(\lg n / \lg \lg n)$	$O(\lg n / \lg \lg n)$	$O(n)$
X-fast trie	$O(\lg w)$	$O(\lg w)$	$O(nw)$
Y-fast trie	$O(\lg w)$	$O(\lg w)$ am.	$O(n)$

14.26.4 The Open Gap

When $w = \Theta(\lg^c n)$, the best upper bound is $O(\lg \lg n)$ (vEB) while the lower bound is $\Omega(\lg n / \lg \lg n)$. Closing this gap remains a central open problem in data structure theory.

14.27 Kinetic Data Structures

In many applications, the data we store moves over time according to known trajectories, and we must answer geometric queries that remain valid as the motion unfolds. A *kinetic data structure* (KDS) maintains attributes of moving objects where each object's position is a known function of time.

14.27.1 Certificates and Failure Times

A *certificate* is a predicate that must hold for the current answer to be correct. For example, if p_i is currently the minimum, we need $x_i(t) \leq x_j(t)$ for all $j \neq i$. Each inequality becomes false at a specific *failure time*:

$$t_{ij} = \frac{a_j - a_i}{b_i - b_j}, \quad \text{when } b_i \neq b_j.$$

We store all certificates in a priority queue keyed by failure time. At each event, we extract the earliest failing certificate, repair the structure, and insert new certificates.

14.27.2 Kinetic Tournament

A kinetic tournament is a complete binary tree whose leaves store moving points. Each internal node stores the current minimum and the certificate guaranteeing it. When a certificate fails, the minimum is updated and new certificates are created.

```

1 struct MovingPoint {
2     double a, b; // position = a + b*t
3     double position(double t) const { return a + b * t; }
4 };
5
6 struct Certificate {
7     int i, j;
8     double failureTime;
9     bool operator>(const Certificate& o) const {
10         return failureTime > o.failureTime;
11     }
12 };
13
14 class KineticTournament {
15     std::vector<MovingPoint> pts;
16     std::vector<int> tree;
17     std::priority_queue<Certificate,
18         std::vector<Certificate>,
19         std::greater<Certificate>> certs;

```



```

20     int n;
21
22     void addCertificate(int i, int j) {
23         if (pts[i].b == pts[j].b) return;
24         double t = (pts[j].a - pts[i].a)
25                 / (pts[i].b - pts[j].b);
26         if (t > 0) certs.push({i, j, t});
27     }
28
29     void build(int node, int lo, int hi) {
30         if (lo == hi) { tree[node] = lo; return; }
31         int mid = (lo + hi) / 2;
32         build(2*node, lo, mid);
33         build(2*node+1, mid+1, hi);
34         int l = tree[2*node], r = tree[2*node+1];
35         tree[node] = (pts[l].a < pts[r].a) ? l : r;
36         addCertificate(l, r);
37     }
38
39 public:
40     KineticTournament(const std::vector<MovingPoint>& p)
41         : pts(p), n(p.size()), tree(4 * p.size()) {
42         build(1, 0, n - 1);
43     }
44
45     void advanceTo(double t) {
46         while (!certs.empty() &&
47                certs.top().failureTime <= t) {
48             Certificate c = certs.top(); certs.pop();
49             if (pts[c.i].position(c.failureTime) >
50                 pts[c.j].position(c.failureTime))
51                 build(1, 0, n - 1);
52         }
53     }
54
55     int currentMin() const { return tree[1]; }
56 };

```

14.27.3 Industrial Applications

- **Autonomous vehicles:** real-time collision detection between moving objects
- **GPS fleet tracking:** kinetic nearest-neighbor queries for moving vehicles
- **Game engines:** physics simulation, collision prediction
- **Air traffic control:** conflict detection between aircraft minutes ahead of time
- **Robotics:** path planning in environments with moving obstacles

14.27.4 Comparison

Structure	Certificate Failures	Space	Primary Use
Kinetic heap	$O(n \log n)$	$O(n)$	Convex hull
Kinetic tournament	$O(n \log n)$ exp.	$O(n)$	Min / max queries
Kinetic sorted list	$O(n \log n)$	$O(n)$	Ordered maintenance

14.28 Planar Point Location

The *planar point location* problem asks: given a subdivision of the plane into regions (faces), which region contains a query point? This is fundamental to geographic information systems (GIS), computer-aided design (CAD), and map overlays.

14.28.1 The Problem

A planar subdivision is a planar graph embedded in the plane, dividing it into faces. Each face may have a label (e.g., a county name in a map). A query asks: given a point $q = (x, y)$, return the face containing q .

14.28.2 Trapezoidal Maps

A *trapezoidal map* is constructed by shooting vertical rays upward and downward from each vertex until hitting an edge. This decomposes each face into trapezoids (or triangles for degenerate cases). Point location in a trapezoidal map reduces to locating the trapezoid containing the query point.

A *randomized incremental construction* builds the trapezoidal map in $O(n \log n)$ expected time using a search DAG. The DAG has $O(n)$ nodes and answers point location queries in $O(\log n)$ expected time.

14.28.3 Kirkpatrick's Hierarchy

Kirkpatrick's algorithm uses a hierarchy of progressively coarser triangulations. Starting from a triangulation of n vertices, every third vertex is removed, producing a triangulation with $O(n/4)$ vertices. The process repeats until only a constant number of vertices remain.

Point location works by starting at the coarsest level and walking down: at each level, a constant-size search identifies which triangle contains the query point, then the algorithm descends into the corresponding sub-triangle at the next level.

```

1 struct Face { int id; std::string label; };
2
3 struct Triangle {
4     Point v[3];
5     int faceId;
6     int childIdx;

```

```

7  };
8
9  class KirkpatrickHierarchy {
10     std::vector<std::vector<Triangle>> levels;
11
12 public:
13     void build(const std::vector<Triangle>& triang) {
14         levels.push_back(triang);
15         while (levels.back().size() > 4) {
16             auto& cur = levels.back();
17             std::vector<Triangle> next;
18             // Remove independent set of vertices,
19             // retriangulate holes, O(n) per level
20             for (auto& t : cur) next.push_back(t);
21             levels.push_back(next);
22         }
23     }
24
25     Face query(Point q) const {
26         int level = levels.size() - 1;
27         int triIdx = 0;
28         while (level > 0) {
29             int child = findChild(
30                 levels[level][triIdx], q);
31             triIdx = child;
32             level--;
33         }
34         return {levels[0][triIdx].faceId, ""};
35     }
36
37     int findChild(const Triangle& t, Point q) const {
38         return 0;
39     }
40 };

```

14.28.4 Persistent Search Trees

A more practical approach for vertical subdivisions uses *persistent search trees*. Sort all edges by x -coordinates. For each x -value, maintain a BST on the y -coordinates of edges crossing the vertical line. Using persistent BSTs, the entire history of BST changes is stored, and a point location query reduces to a single BST search in a version chosen by binary search on x .

14.28.5 Industrial Applications

- **GIS:** determining which county, zip code, or electoral district contains a point
- **CAD:** identifying which part of a circuit board or mechanical drawing a point belongs to

- **Map overlay:** combining two map layers and identifying resulting regions
- **Robotics:** determining which zone a robot is in for navigation planning

14.29 Exercises

14.29.1 Drill

1. **Segment tree sizing.** You build a segment tree for array size $n = 10^6$ (not a power of 2). A range query accesses indices beyond $2n$ — why? What's the correct allocation size? Draw the tree structure for $n = 6$.
1. **Fenwick 1-indexing.** Given values at positions 1–8: [1, 3, 5, 7, 9, 11, 13, 15], draw the Fenwick tree array. Trace query(5) (prefix sum up to index 5). Why does Fenwick use 1-based indexing? What goes wrong with 0-based?
1. **Union-find without compression.** Perform union(0,1), union(2,3), union(0,2), find(3) on a union-find structure without path compression. Show the parent array. What is the height of the tree containing 3? Now add path compression to find(3) — what changes?
1. **Segment tree vs sparse table.** For range minimum queries on data that never changes, which is better: segment tree ($O(\log n)$ query) or sparse table ($O(1)$ query)? Compare build time and memory for $n = 10^5$. When would the segment tree still be preferred?

14.29.2 Application

1. **Union-find for graph connectivity.** Implement union-find with path compression and union by rank. Test on a graph with 10^6 nodes and 5×10^6 random edges. Count the number of find operations and measure runtime. Verify that the total operations are near-linear.

(In production, Spark's GraphX library uses union-find for connected components in large distributed graphs.)

1. **Range sum with Fenwick vs segment tree.** Implement both a Fenwick tree and a segment tree for range sum with point updates. Benchmark:
 - Query time for ranges of size 100, 1000, 10000
 - Update time for a single point
 - Memory usage for $n = 10^6$

Which would you use for a monitoring dashboard (reads $\gg$ writes)? For a high-frequency trading system (writes $\gg$ reads)?

1. **Quantile estimation.** Implement a frequency Fenwick tree over a range of values. Support increment(value, delta), prefix_sum(k), and quantile(p). Feed 10^6 latency measurements (1–10000 ms) and report P50, P90, P99. Compare with exact quantiles from sorting. How much faster is the Fenwick approach?

(In production, Google's Sawzall log analysis language used Fenwick trees for approximate quantile computation.)

1. **Range minimum with lazy updates.** Implement a segment tree that supports range minimum query and range add (lazy propagation). Test on 10^6 simulated updates and queries. Compare with a naive $O(n)$ approach for each operation.

(In production, order-matching engines use segment-tree-like structures to maintain best bid/ask across price levels.)

1. **Percolation threshold.** Model a material as an $n \times n$ grid where each cell is open (conductive) or blocked. Current flows from top to bottom through connected open cells. Use union-find to determine the critical fraction where current first flows through. Run 1000 simulations for $n = 10, 50, 100$. How does the threshold change with n ?
1. **Range assign with segment tree.** Implement a segment tree that supports range assign (set all values in $[l, r]$ to x) and point query. Benchmark for 10^6 operations. Compare with a brute-force approach.

14.29.3 Research

1. **Persistent segment tree.** Implement a persistent segment tree where each update creates a new version while keeping old ones. Answer "k-th smallest element in subarray $\text{arr}[l..r]$ " using binary search on versions. Benchmark for $n = 10^5$ with 10^4 queries. How does persistence affect memory and speed?
1. **Segment tree with fractional cascading.** Implement a segment tree where each node stores a sorted list of its subtree's values (merge sort tree). Then apply fractional cascading to reduce query time from $O(\log^2 n)$ to $O(\log n)$. Benchmark before and after.
1. **Union-find with rollback.** Implement union-find that supports undo — store each change on a stack. Solve the offline dynamic connectivity problem: given a graph with edge deletions over time, answer connectivity queries at each time step. Benchmark for 10^5 operations.
1. **Sparse table.** Implement a sparse table for $O(1)$ range minimum queries on static data (no updates). Compare build time, query time, and memory with a segment tree for $n = 10^5$. For which applications would you choose each? *(In production, monitoring dashboards use sparse tables for historical metrics that never change.)*

1. **Persistent segment tree for version control.** Build a persistent segment tree representing file sizes across 10^5 commits. Each commit modifies one file. Answer: (a) total size of all files at version v , (b) difference in total size between versions v_1 and v_2 , (c) largest single file at version v . Benchmark query time and total memory vs. storing full snapshots.
1. **Sqrt decomposition for mex.** Given an array of n non-negative integers, support two operations: (a) update position i to value v , (b) query the minimum excluded value (mex) in range $[l, r]$. Use sqrt decomposition with block-level frequency arrays. Each block maintains a boolean array of which values 0 through B appear. Mex is the first gap. Analyze time complexity for Q operations.
1. **GCD sparse table on rolling GCD.** Given a static array, answer Q queries of the form "what is the GCD of all elements in $[l, r]$?" using a GCD sparse table. Compare with computing GCD incrementally for each query. For $n = 10^5$ and $Q = 10^5$, measure the speedup. Then modify the sparse table to answer "GCD of all elements in $[l, r]$ that are divisible by d " efficiently.
1. **Fenwick tree for 2D range sum.** Extend the Fenwick tree to 2D: support point update (x, y, δ) and range sum query over rectangle $[x_1, x_2] \times [y_1, y_2]$. Implement using a 2D BIT (size $n \times n$, four nested loops). Query and update each take $O(\log^2 n)$. Test on a 1000×1000 grid with 10^5 random operations.
1. **Segment tree beats.** Implement a segment tree that supports: (a) range chmin (set $A[i] = \min(A[i], v)$ for all i in $[l, r]$), (b) range sum query. This requires the "segment tree beats" technique: maintain max and second-max values in each node to prune updates. Benchmark for $n = 10^5$ with 10^5 random operations. How does it compare with a naive segment tree that does $O(n)$ per chmin?
1. **Union-find for offline LCA.** Given a rooted tree, answer Q lowest-common-ancestor queries offline using union-find. Process nodes in reverse DFS order: when finishing node v , union v with its parent. For each query (u, v) , the answer is the root of the set containing the other node when they were last unified. Implement and verify against an Euler tour + sparse table LCA approach.
1. **Square root decomposition for Mo's algorithm.** Solve the following offline range query problem: given an array and Q queries of the form "count distinct values in $[l, r]$," implement Mo's algorithm using sqrt decomposition for block sorting. Sort queries by $(l/B, r)$ where $B = \sqrt{n}$. Maintain a frequency array and a running distinct count. Benchmark for $n = 10^5$, $Q = 10^5$, values in range $[1, 10^5]$.
1. **Persistent segment tree for k-th smallest in range.** Given an array $A[0..n-1]$, build a persistent segment tree over coordinate-compressed values. Version i represents the multiset $\{A[0], \dots, A[i-1]\}$. To find the k-th smallest in $A[l..r]$, query the difference between version $r+1$ and version l . Implement and test with $n = 10^5$, $Q = 10^5$ random range k-th queries.

1. **Segment tree for polynomial range updates.** Extend lazy propagation to support range updates of the form $A[i] \leftarrow A[i] + a \cdot i + b$ for all i in $[l, r]$. Each node stores: sum, sum of indices, and sum of index-squared. The lazy tag stores (a, b) . Derive the formulas for pushing the tag to children. Implement and verify against brute force for $n = 10^5$ with 10^5 operations.
1. **Sparse table for range OR/AND queries.** Are range OR and range AND queries idempotent? Explain why or why not. If yes, build a sparse table for each and answer Q queries in $O(1)$ time. If not, explain what goes wrong and propose an alternative (e.g., segment tree). Test on random bit vectors of length $n = 10^5$.

14.30 References and Further Reading

- Peter M. Fenwick, "A New Data Structure for Cumulative Frequency Tables" — Software: Practice and Experience, 1994.
- Robert E. Tarjan and Jan van Leeuwen, "Worst-Case Analysis of Set Union Algorithms" — JACM, 1984.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapters 19 (Fibonacci heaps, related), 21 (union-find).
- Michael A. Bender and Martin Farach-Colton, "The LCA Problem Revisited" — LATIN 2000 (sparse table for LCA/RMQ).
- Johannes Fischer and Volker Heun, "Space-Efficient Preprocessing Schemes for Range Minimum Queries on Static Arrays" — ACM TOACT, 2011.
- Bernard Chazelle, "A Minimum Spanning Tree Algorithm with Inverse-Ackermann Type Complexity" — JACM, 2000.
- Robert E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm" — JACM, 1975.
- Harold N. Gabow, Jon Louis Bentley, and Robert E. Tarjan, "Scaling and Related Techniques for Geometry Problems" — STOC 1984 (on segment trees).

Part III

Algorithm-Design Methods

The Greedy Method

15.1 Introduction

⚡ **Complete compilable implementations of the data structures in this chapter are in code/.**

The **greedy method** is an algorithmic paradigm that builds a solution piece by piece, always choosing the next piece that offers the most immediate benefit. Unlike dynamic programming, which considers all subproblems, greedy algorithms make a single choice at each step and never reconsider it.

Greedy algorithms are appealing because they are simple, intuitive, and often efficient. The challenge is proving correctness: when does the locally optimal choice lead to a globally optimal solution?

15.2 The General Method

A **greedy algorithm** makes the locally optimal choice at each step, hoping to find the global optimum. For many problems, this works — the locally optimal choice leads to the globally optimal solution. For others, it does not.

15.2.1 Greedy Algorithm Template

```
1 1. Initialize solution = empty
2 2. While a solution has not been found:
3   a. Select the next best candidate (locally optimal)
4   b. If the candidate is feasible, add it to the solution
5   c. If the candidate is not feasible, discard it
6 3. Return solution
```

The key design decisions are: (1) what constitutes a “greedy choice,” and (2) how to prove that the greedy choice leads to an optimal solution.

15.2.2 When Greedy Works

Greedy algorithms are correct when the problem exhibits two properties:

1. **Optimal substructure:** an optimal solution contains optimal solutions to subproblems. If we remove the greedy choice from the optimal solution, the remaining pieces must form an optimal solution to the reduced subproblem.

2. **Greedy choice property:** a globally optimal solution can be reached by making locally optimal choices. At each step, the greedy choice is part of some optimal solution — there is always an optimal solution that includes the greedy choice.

15.2.3 The Exchange Argument

The standard technique for proving greedy correctness is the **exchange argument**:

1. Let OPT be an optimal solution
2. If OPT differs from the greedy solution G, find the first point where they diverge
3. “Exchange” the element in OPT with the greedy choice in G
4. Show the exchange does not worsen the solution (and may improve it)
5. Conclude G is also optimal

Worked example: Activity selection (schedule maximum number of non-overlapping activities).

Activities sorted by finish time: A(1,3), B(2,5), C(4,7), D(6,9), E(5,8).

Greedy picks by earliest finish: A(1,3), C(4,7), D(6,9) — 3 activities.

Suppose OPT includes B(2,5) instead of A(1,3). Since A finishes before B starts, we can replace B with A and still have a valid schedule. By the exchange argument, there is always an optimal solution containing the earliest-finishing activity.

15.3 Container Loading

Problem: We have a ship with capacity C and containers with weights $w_1, w_2, \dots, w_n$. Load as many containers as possible without exceeding capacity.

Greedy choice: Load the lightest container first.

```

1 std::vector<size_t> load_containers(double capacity, std::span<const
  double> weights) {
2     // Pair (weight, original index)
3     std::vector<std::pair<double, size_t>> items;
4     for (size_t i = 0; i < weights.size(); ++i) items.push_back({weights[i], i});
5     std::sort(items.begin(), items.end());
6
7     std::vector<size_t> loaded;
8     double current = 0;
9     for (const auto& [w, idx] : items) {
10         if (current + w <= capacity) {
11             current += w;
12             loaded.push_back(idx);
13         }

```

```

14     }
15     return loaded;
16 }

```

Worked trace: Weights = [3, 7, 2, 5, 8, 4], capacity = 15.

Sorted: [2, 3, 4, 5, 7, 8]. Greedy loads: 2+3+4+5 = 14 (4 containers). Could we do better? The only alternative with 4 containers is 2+3+4+8 = 17 (too heavy) or 2+3+5+7 = 17 (too heavy). So 4 is optimal.

Correctness: Optimal substructure — removing the lightest container from an optimal solution gives an optimal solution for the remaining capacity. Greedy choice — if an optimal solution does not contain the lightest container, we can replace a heavier one with it.

15.4 The Fractional Knapsack Problem

Problem: Items have weights w_i and values v_i . We can take fractions of items. Maximize value for a knapsack of capacity C .

Greedy choice: Take the item with the highest value-to-weight ratio first.

```

1 struct item { double weight, value; };
2
3 double fractional_knapsack(double capacity, std::span<const item> items) {
4     std::vector<size_t> order(items.size());
5     std::iota(order.begin(), order.end(), 0);
6     std::sort(order.begin(), order.end(), [&](size_t a, size_t b) {
7         return items[a].value / items[a].weight > items[b].value / items[b]
8     }.weight;
9 });
10
11 double total_value = 0.0;
12 for (size_t i : order) {
13     if (capacity >= items[i].weight) {
14         capacity -= items[i].weight;
15         total_value += items[i].value;
16     } else {
17         total_value += items[i].value * (capacity / items[i].weight);
18         break;
19     }
20 }
21 return total_value;
22 }

```

Worked trace: $C = 50$, items = [(Gold, 10, 600), (Silver, 20, 500), (Copper, 30, 360)].

Ratios: Gold = 60, Silver = 25, Copper = 12. Greedy takes: all Gold (10kg, \$600), all Silver (20kg, \$500), 20/30 of Copper (20kg, \$240). Total = \$1340.

Complexity: $O(n \log n)$. The 0/1 knapsack (no fractions) is not solvable by greedy — it requires

dynamic programming (Chapter 17).

15.5 Task Scheduling

Problem: Schedule tasks with deadlines and profits. Each unit of time can handle one task. Maximize total profit.

Greedy choice: Process tasks in order of decreasing profit. Schedule each task as late as possible.

```

1 struct task {
2     int deadline; // must be completed by this time
3     int profit;
4 };
5
6 int schedule_tasks(std::span<task> tasks) {
7     // Sort by profit descending
8     std::vector<task> sorted(tasks.begin(), tasks.end());
9     std::sort(sorted.begin(), sorted.end(),
10             [](const task& a, const task& b) { return a.profit > b.
11                 profit; });
12
13     int max_deadline = 0;
14     for (const auto& t : sorted) max_deadline = std::max(max_deadline, t.
15         deadline);
16
17     std::vector<int> slots(max_deadline + 1, -1); // -1 = empty
18     int total_profit = 0;
19
20     for (const auto& [d, p] : sorted) {
21         // Find the latest available slot before the deadline
22         for (int t = d; t >= 1; --t) {
23             if (slots[t] == -1) {
24                 slots[t] = p;
25                 total_profit += p;
26                 break;
27             }
28         }
29     }
30     return total_profit;
31 }

```

Worked trace: Tasks = [(2, 50), (1, 10), (2, 20), (1, 25), (3, 30)].

Sorted by profit: (2, 50), (3, 30), (1, 25), (2, 20), (1, 10).

Schedule: slot 2 = 50, slot 3 = 30, slot 1 = 25. Total profit = 105.

Optimization: Use Union-Find to find the latest available slot in $O(\alpha(n))$ per task.

15.6 Single-Source Shortest Paths (Dijkstra)

Problem: Given a weighted directed graph with non-negative edge weights, find the shortest path from a source vertex s to all other vertices.

Greedy choice: At each step, extract the unvisited vertex with the smallest distance estimate and relax its outgoing edges.

15.6.1 The Algorithm

```

1  std::vector<int> dijkstra(const std::vector<std::vector<std::pair<int,int>
    >>>& adj,
2                               int source) {
3      int n = adj.size();
4      std::vector<int> dist(n, INT_MAX);
5      dist[source] = 0;
6
7      // Min-heap: (distance, vertex)
8      std::priority_queue<std::pair<int,int>, std::vector<std::pair<int,int>
    >>,
9                               std::greater<>> pq;
10     pq.push({0, source});
11
12     while (!pq.empty()) {
13         auto [d, u] = pq.top(); pq.pop();
14         if (d > dist[u]) continue; // stale entry
15
16         for (auto [v, w] : adj[u]) {
17             if (dist[u] + w < dist[v]) {
18                 dist[v] = dist[u] + w;
19                 pq.push({dist[v], v});
20             }
21         }
22     }
23     return dist;
24 }
```

15.6.2 Worked Trace

Graph: vertices A-E with edges $A \rightarrow B(4)$, $A \rightarrow C(2)$, $B \rightarrow C(1)$, $B \rightarrow D(5)$, $C \rightarrow D(8)$, $C \rightarrow E(10)$, $D \rightarrow E(2)$.

```

1  Step 1: Extract A (d=0). Relax A->B(4), A->C(2).
2      dist = {A:0, B:4, C:2, D:inf, E:inf}
3
4  Step 2: Extract C (d=2). Relax C->D(10), C->E(12).
5      dist = {A:0, B:4, C:2, D:10, E:12}
6
```

```

7 Step 3: Extract B (d=4). Relax B->C(5, no improvement), B->D(9).
8     dist = {A:0, B:4, C:2, D:9, E:12}
9
10 Step 4: Extract D (d=9). Relax D->E(11, no improvement).
11     dist = {A:0, B:4, C:2, D:9, E:11}
12
13 Step 5: Extract E (d=11). No outgoing edges.
14     dist = {A:0, B:4, C:2, D:9, E:11}

```

Why greedy works: When we extract vertex u with minimum distance, no unvisited vertex can provide a shorter path to u (because all edge weights are non-negative). This is the greedy choice property — the minimum-distance vertex is permanently settled.

Complexity: $O((V + E) \log V)$ with a binary heap. $O(V^2)$ with an array (better for dense graphs).

Limitation: Dijkstra fails with negative edge weights. Use Bellman-Ford (Chapter 12) when negative edges are present.

15.7 Minimum Spanning Tree

Problem: Given a connected, undirected, weighted graph, find a spanning tree of minimum total edge weight.

15.7.1 Kruskal's Algorithm

Greedy choice: Sort edges by weight; add each edge if it does not create a cycle (checked using Union-Find).

```

1 struct edge { int u, v, weight; };
2
3 std::vector<edge> kruskal(int n, std::vector<edge> edges) {
4     std::sort(edges.begin(), edges.end(),
5               [](const edge& a, const edge& b) { return a.weight < b.
6                 weight; });
7
8     union_find uf(n);
9     std::vector<edge> mst;
10
11     for (const auto& [u, v, w] : edges) {
12         if (uf.find(u) != uf.find(v)) {
13             mst.push_back({u, v, w});
14             uf.unite(u, v);
15         }
16     }
17     return mst;
18 }

```

Worked trace: 5 vertices, edges sorted by weight:

```

1 Edge (0,1,1): Different components -> add to MST
2 Edge (2,3,2): Different components -> add to MST
3 Edge (0,2,3): Different components -> add to MST
4 Edge (1,2,4): Same component (0-1-2) -> skip (cycle)
5 Edge (3,4,5): Different components -> add to MST
6 Edge (1,3,6): Same component -> skip
7 Edge (2,4,7): Same component -> skip
8
9 MST: (0,1,1) + (2,3,2) + (0,2,3) + (3,4,5) = weight 11

```

Complexity: $O(E \log E)$ for sorting + $O(E \alpha(V))$ for Union-Find = $O(E \log E)$.

15.7.2 Prim's Algorithm

Greedy choice: Start from any vertex. At each step, add the cheapest edge connecting the tree to a non-tree vertex.

```

1 std::vector<edge> prim(const std::vector<std::vector<std::pair<int,int>>>&
    adj) {
2     int n = adj.size();
3     std::vector<bool> in_tree(n, false);
4     std::priority_queue<std::pair<int,std::pair<int,int>>,
5                         std::vector<std::pair<int,int>>>,
6                         std::greater<>> pq;
7
8     in_tree[0] = true;
9     for (auto [v, w] : adj[0]) pq.push({w, {0, v}});
10
11     std::vector<edge> mst;
12     while (mst.size() < n - 1 && !pq.empty()) {
13         auto [w, u_v] = pq.top(); pq.pop();
14         auto [u, v] = u_v;
15         if (in_tree[v]) continue;
16         in_tree[v] = true;
17         mst.push_back({u, v, w});
18         for (auto [next, nw] : adj[v]) {
19             if (!in_tree[next]) pq.push({nw, {v, next}});
20         }
21     }
22     return mst;
23 }

```

Worked trace (same graph): Start at vertex 0.

```

1 Tree: {0}. Candidates: (0,1,1), (0,2,3). Pick (0,1,1).
2 Tree: {0,1}. Candidates: (0,2,3), (1,2,4), (1,3,6). Pick (0,2,3).
3 Tree: {0,1,2}. Candidates: (2,3,2), (2,4,7), (1,3,6). Pick (2,3,2).
4 Tree: {0,1,2,3}. Candidates: (3,4,5), (2,4,7). Pick (3,4,5).
5 Tree: {0,1,2,3,4}. MST complete. Total weight: 11.

```

Complexity: $O(E \log V)$ with a binary heap. Better than Kruskal for dense graphs ($E \approx V^2$).

15.7.3 The Cut Property

Both algorithms are correct because of the **cut property**: for any cut (partition of vertices into two sets), the minimum-weight edge crossing the cut is in some MST. Kruskal adds the globally cheapest edge that connects two components; Prim adds the cheapest edge from the tree to outside. Both exploit the cut property.

15.8 Huffman Coding

Problem: Given character frequencies, build a prefix-free binary code that minimizes the total encoded length.

Greedy choice: Repeatedly combine the two characters with the smallest frequencies.

```

1 struct huffman_node {
2     char ch;
3     int freq;
4     huffman_node* left;
5     huffman_node* right;
6 };
7
8 struct compare {
9     bool operator()(huffman_node* a, huffman_node* b) {
10         return a->freq > b->freq;
11     }
12 };
13
14 huffman_node* build_huffman_tree(const std::unordered_map<char, int>&
15     frequencies) {
16     std::priority_queue<huffman_node*, std::vector<huffman_node*>, compare>
17     > pq;
18
19     for (const auto& [ch, freq] : frequencies) {
20         pq.push(new huffman_node{ch, freq, nullptr, nullptr});
21     }
22
23     while (pq.size() > 1) {
24         auto* left = pq.top(); pq.pop();
25         auto* right = pq.top(); pq.pop();
26         auto* parent = new huffman_node{'\0', left->freq + right->freq,
27             left, right};
28         pq.push(parent);
29     }
30     return pq.top();
31 }

```



```

30 void build_codes(huffman_node* root, std::string prefix,
31                 std::unordered_map<char, std::string>& codes) {
32     if (!root->left && !root->right) {
33         codes[root->ch] = prefix;
34         return;
35     }
36     build_codes(root->left, prefix + "0", codes);
37     build_codes(root->right, prefix + "1", codes);
38 }

```

Complexity: $O(n \log n)$ where n is the alphabet size.

Worked trace:

```

1 frequencies: A:45, B:13, C:12, D:16, E:9, F:5
2
3 Step 1: Combine F(5) + E(9) = FE(14)
4 Step 2: Combine C(12) + D(16) = CD(28)
5 Step 3: Combine B(13) + FE(14) = BFE(27)
6 Step 4: Combine BFE(27) + CD(28) = BFEC(55)
7 Step 5: Combine A(45) + BFEC(55) = root(100)
8
9 Huffman codes:
10 A: 0          (1 bit)
11 B: 100        (3 bits)
12 C: 110        (3 bits)
13 D: 111        (3 bits)
14 E: 1010       (4 bits)
15 F: 1011       (4 bits)
16
17 Total bits = 45*1 + 13*3 + 12*3 + 16*3 + 9*4 + 5*4 = 224
18 vs fixed 3-bit code = 100*3 = 300
19 Savings: 25%

```

Why greedy works: The two least frequent characters will have the longest codes in any optimal prefix-free code. By combining them first, we ensure they share a prefix, minimizing total length. The exchange argument shows that swapping any pair of characters with the two least frequent cannot improve the solution.

15.9 Greedy Does Not Always Work

The 0/1 Knapsack Problem: Items must be taken whole (not fractional). Greedy fails:

```

1 Item A: weight 5, value 50 (ratio 10)
2 Item B: weight 10, value 60 (ratio 6)
3 Item C: weight 20, value 140 (ratio 7)
4 Capacity: 20
5
6 Greedy (by ratio): A + B = 110

```

```
7 Optimal: C alone = 140
```

Coin Change (US denominations): Greedy works for US coins (25, 10, 5, 1) but fails for other systems:

```
1 Denominations: 1, 3, 4
2 Amount: 6
3 Greedy: 4 + 1 + 1 = 3 coins
4 Optimal: 3 + 3 = 2 coins
```

Traveling Salesman: The nearest-neighbor heuristic (greedy: always visit the closest unvisited city) can be arbitrarily bad. For a cycle of cities arranged in a circle, nearest-neighbor gives a path crossing the center — twice the optimal tour length.

When greedy fails, we use dynamic programming (Chapter 17), backtracking (Chapter 18), or approximation algorithms.

15.10 Common Bugs and Pitfalls

- **Assuming greedy works without proof** — many real-world problems look greedy-solvable but aren't. Always prove optimal substructure and the greedy choice property before deployment.
- **Confusing fractional with 0/1 knapsack** — the fractional variant is greedy-solvable; the 0/1 variant is not. Accidentally applying greedy to 0/1 knapsack in a production inventory system would produce suboptimal shipping costs.
- **Dijkstra with negative edges** — Dijkstra's greedy selection fails on graphs with negative edge weights. Use Bellman-Ford instead. This is a common bug in GPS routing prototypes.
- **Huffman with unequal code lengths** — the Huffman algorithm assumes each symbol's code is a whole number of bits. For arithmetic coding (used in JPEG, H.264), the greedy Huffman approach does not apply.
- **Interval scheduling tie-breaking** — when intervals have equal finish times, the greedy choice (earliest finish) still works, but incorrect tie-breaking can eliminate valid schedules.
- **Prim vs Kruskal for dense graphs** — Prim with an adjacency matrix is $O(V^2)$, which beats Kruskal's $O(E \log E)$ when $E \approx V^2$. Always match the algorithm to the graph density.

15.11 Summary

1. **Greedy algorithms** make locally optimal choices, hoping for a global optimum.
2. **Correctness** requires optimal substructure and the greedy choice property, typically proved via the exchange argument.

3. **Container loading, fractional knapsack, Dijkstra, MST, and Huffman coding** are canonical greedy successes.
4. **0/1 knapsack and coin change** (in general) show that greedy does not always work.
5. Industrial applications include **CDN content placement** (Akamai), **Huffman compression** (gzip, PNG), **Dijkstra routing** (Google Maps, OSRM), **interval scheduling** (cloud compute scheduling), and **Huffman coding** (JPEG, MP3).
6. Greedy is a **problem-solving approach**, not a fixed algorithm — the art is in identifying the right greedy choice.

15.12 Interview FAQs

1. **How do you prove a greedy algorithm is correct — and how do you prove one is wrong?**

To prove correct: use the exchange argument — show that any optimal solution can be transformed to include the greedy choice without worsening it. To prove wrong: construct a counterexample where the greedy choice leads to a provably worse outcome than an alternative. The key insight: greedy works when the problem has *optimal substructure* AND the greedy choice property holds — that locally optimal choices compose into a globally optimal solution. When only optimal substructure holds (but not the greedy property), you need DP instead. The matroid framework provides a structural guarantee: if the feasible sets form a matroid, greedy is always optimal for linear weight functions.

1. **When does greedy fail? Give examples.**

Greedy fails when the locally optimal choice does not lead to a globally optimal solution — typically when subproblems overlap (as in 0/1 knapsack) or when the greedy choice precludes a better future decision (as in the general coin change problem with denominations like [1, 3, 4]). TSP with nearest-neighbor is another classic failure: the greedy choice can force a long return path.

1. **When does Dijkstra’s greedy approach fail, and what does that tell you about the problem structure?**

Dijkstra fails with negative edge weights because it assumes once a node is “settled” (extracted from the priority queue), its distance is final. A negative edge can reduce a settled node’s distance, violating this assumption. The deeper insight: Dijkstra works when the *greedy choice property* holds — the locally shortest unprocessed node is globally optimal. Negative edges break this by creating “cheaper paths through already-processed nodes.” Bellman-Ford handles this by relaxing ALL edges $V-1$ times, not just the “best” one each round. When you see negative edges, the problem lacks the matroid structure that greedy requires.

1. Why is Huffman coding optimal for prefix-free codes?

Huffman achieves the minimum weighted path length among all prefix-free codes. The proof uses the exchange argument: in an optimal code, the two least-frequent symbols must be siblings at the deepest level. Huffman's greedy merging exactly matches this structure. For symbols drawn from a known i.i.d. source, the average code length approaches the Shannon entropy $-\sum p_i \log_2 p_i$.

1. Why is Huffman coding optimal, and what makes a coding scheme non-Huffman?

Huffman coding minimizes the weighted path length $\sum f_i \cdot d_i$ where f_i is frequency and d_i is code length. It's optimal because: (1) the two least frequent symbols must have the longest codes (proven by exchange argument), and (2) the tree structure ensures no code is a prefix of another. A coding scheme is "non-Huffman" when it violates the prefix-free property (ambiguity on decoding) or when it uses fixed-length codes for variable-frequency data (wasting bits on rare symbols). The connection to information theory: Huffman's average code length approaches the Shannon entropy $H = -\sum p_i \log_2 p_i$, which is the theoretical minimum for symbol-by-symbol coding.

1. What is the matroid structure behind greedy algorithms?

A matroid is a structure $(E, \mathcal{I})$ where $\mathcal{I}$ is a family of "independent" sets satisfying hereditary and exchange properties. Greedy is optimal for maximizing a linear weight function over the independent sets of a matroid. MST is a graphic matroid (independent sets = forests). Interval scheduling is a gammoid. If a problem has matroid structure, greedy is guaranteed optimal.

1. How does the greedy set cover algorithm perform as an approximation?

The greedy set cover algorithm achieves an $O(\ln n)$ -approximation, and this is tight unless $P = NP$. At each step, it picks the set covering the most uncovered elements. The approximation ratio is $\sum_{i=1}^n 1/i = H_n \approx \ln n$. For practical instances, the greedy solution is often much better than this worst-case bound.

1. Why does sorting by finish time work for activity selection, and what happens with weighted activities?

Sorting by finish time works because choosing the activity that finishes earliest leaves maximum remaining time for other activities — it's the greedy choice that maximizes future options. This is the *earliest-finish-time-first* property: at each step, the greedy choice is safe because any optimal solution can be swapped to include it without conflict. With weights (weighted activity selection), greedy fails — a high-weight long activity might be worth more than several short ones. This requires DP: sort by finish time, then for each activity i , find the latest non-conflicting activity $p(i)$, and use $DP[i] = \max(DP[i-1], \text{weight}[i] + DP[p(i)])$.

1. Why does the fractional knapsack greedy work but 0/1 knapsack doesn't?

In the fractional variant, taking the highest ratio item first is optimal because you can always “fill the remaining capacity” with fractions — no capacity is wasted. In 0/1 knapsack, taking a high-ratio item might leave awkward remaining capacity that a different combination would fill better. The exchange argument fails because you cannot partially exchange items.

1. Why is the choice of priority queue implementation the single biggest factor in Dijkstra’s real-world performance?

The theoretical complexity depends entirely on the priority queue: binary heap gives $O((V+E) \log V)$, Fibonacci heap gives $O(E + V \log V)$. But in practice, the binary heap wins for most graphs because: (1) Fibonacci heaps have terrible cache locality (pointer-based), (2) the constant factors are 5–10x worse, (3) decrease-key is rarely the bottleneck. For dense graphs ($E \approx V^2$), an unsorted array with linear scan gives $O(V^2)$ — competitive with heaps because there’s no heap maintenance overhead. Google’s Ortools and Boost both use binary heaps. The lesson: asymptotic complexity matters less than cache behavior and constant factors.

15.13 Activity Selection Problem

Problem: Given n activities, each with a start time and finish time, select the maximum number of non-overlapping activities. This is the canonical greedy problem and appears in scheduling, resource allocation, and genome alignment.

Greedy choice: Always pick the activity with the earliest finish time that is compatible with previously selected activities.

Why earliest finish time? An activity that finishes earlier leaves more room for subsequent activities. By the exchange argument: if an optimal solution does not contain the earliest-finishing compatible activity, we can replace the first selected activity in OPT with the earliest-finishing one without creating a conflict. This never worsens the solution.

15.13.1 Worked Example

Consider 10 activities:

Activity	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
Start	1	3	0	5	8	5	6	8	11	13
Finish	2	5	4	7	9	9	10	11	14	15

Step 1: Sort by finish time (already sorted above).

Step 2: Select A1 (finish=2). No conflict.

Step 3: A2 starts at 3, which is ≥ 2 . Compatible. Select A2 (finish=5).

Step 4: A3 finishes at 4 but starts at 0, which overlaps with A2. Skip.

Step 5: A4 starts at 5, which is ≥ 5 . Compatible. Select A4 (finish=7).

Step 6: A5 starts at 8 $\leq$ 7. Compatible. Select A5 (finish=9).

Step 7: A6 starts at 5 $\leq$ 9. Overlaps with A5. Skip.

Step 8: A7 starts at 6 $\leq$ 9. Overlaps. Skip.

Step 9: A8 starts at 8 $\leq$ 9. Overlaps. Skip.

Step 10: A9 starts at 11 $\leq$ 9. Compatible. Select A9 (finish=14).

Step 11: A10 starts at 13 $\leq$ 14. Overlaps. Skip.

Selected: A1, A2, A4, A5, A9. Total: 5 activities out of 10.

Could we do better? Any schedule with 6 activities would need 6 non-overlapping intervals, but examining all subsets confirms that 5 is optimal for this instance.

15.13.2 C++ Implementation

```

1 struct activity {
2     int start, finish;
3 };
4
5 std::vector<activity> select_activities(std::span<const activity> acts) {
6     // Sort by finish time
7     std::vector<activity> sorted(acts.begin(), acts.end());
8     std::sort(sorted.begin(), sorted.end(),
9               [](const activity& a, const activity& b) {
10                 return a.finish < b.finish;
11             });
12
13     std::vector<activity> selected;
14     int last_finish = -1;
15
16     for (const auto& [s, f] : sorted) {
17         if (s >= last_finish) {
18             selected.push_back({s, f});
19             last_finish = f;
20         }
21     }
22     return selected;
23 }
```

Complexity: $O(n \log n)$ for sorting + $O(n)$ for the scan = $O(n \log n)$ total.

Variants:

- **Weighted activity selection:** Each activity has a value. Greedy fails – use DP (Chapter 17).
- **Minimum number of rooms:** Sort start and finish events; sweep through, tracking the maximum number of overlapping activities at any point.
- **Interval partitioning:** Given n lectures with start/end times, find the minimum number of classrooms needed. Greedy by sorting start times and assigning each lecture to any available room.

15.14 Job Sequencing with Deadlines

Problem: Given n jobs, each with a deadline d_i and a profit p_i , schedule the jobs on a single processor so that each job completes by its deadline and total profit is maximized. Each job takes one unit of time.

Greedy choice: Process jobs in decreasing order of profit. For each job, assign it to the latest available time slot before its deadline.

Why this works: By the exchange argument: if an optimal solution schedules a lower-profit job in a slot where a higher-profit job could go, swapping them increases total profit. Scheduling each job as late as possible maximizes the number of slots available for subsequent (lower-profit) jobs.

15.14.1 Worked Trace

Consider 6 jobs:

Job	J1	J2	J3	J4	J5	J6
Deadline	2	1	3	1	3	2
Profit	60	100	20	40	30	50

Step 1: Sort by profit descending: J2(100,d=1), J1(60,d=2), J6(50,d=2), J4(40,d=1), J5(30,d=3), J3(20,d=3).

Step 2: Assign J2 (profit=100, deadline=1): Slot 1 is free. Assign slot 1. Slots: [J2, -, -].

Step 3: Assign J1 (profit=60, deadline=2): Slot 2 is free. Assign slot 2. Slots: [J2, J1, -].

Step 4: Assign J6 (profit=50, deadline=2): Slot 2 occupied. Slot 1 occupied. No available slot. Skip J6.

Step 5: Assign J4 (profit=40, deadline=1): Slot 1 occupied. No available slot. Skip J4.

Step 6: Assign J5 (profit=30, deadline=3): Slot 3 is free. Assign slot 3. Slots: [J2, J1, J5].

Step 7: Assign J3 (profit=20, deadline=3): Slot 3 occupied. Slot 2 occupied. Slot 1 occupied. Skip J3.

Scheduled jobs: J2 (slot 1), J1 (slot 2), J5 (slot 3). Total profit: 190.

15.14.2 Union-Find Optimization

The naive approach scans backward from the deadline to find a free slot, which is $O(d)$ per job. With Union-Find, each find-and-assign is $O(\alpha(n))$ amortized, giving $O(n \alpha(n))$ total after sorting.

```

1 struct job {
2     int deadline, profit;
3 };
4
5 // Union-Find for slot allocation
6 struct union_find {
7     std::vector<int> parent;
```

```

8     union_find(int n) : parent(n + 1) {
9         for (int i = 0; i <= n; ++i) parent[i] = i;
10    }
11    int find(int x) {
12        if (parent[x] != x) parent[x] = find(parent[x]);
13        return parent[x];
14    }
15    void unite(int x, int y) { parent[find(x)] = find(y); }
16 };
17
18 int job_sequencing(std::span<const job> jobs) {
19     // Sort by profit descending
20     std::vector<job> sorted(jobs.begin(), jobs.end());
21     std::sort(sorted.begin(), sorted.end(),
22         [](const job& a, const job& b) {
23             return a.profit > b.profit;
24         });
25
26     int max_deadline = 0;
27     for (const auto& j : jobs) max_deadline = std::max(max_deadline, j.
deadline);
28
29     union_find uf(max_deadline);
30     int total_profit = 0;
31
32     for (const auto& [d, p] : sorted) {
33         // Find the latest available slot before the deadline
34         int slot = uf.find(d);
35         if (slot > 0) {
36             total_profit += p;
37             // Mark this slot as used: link to slot - 1
38             uf.unite(slot, slot - 1);
39         }
40     }
41     return total_profit;
42 }

```

Complexity: $O(n \log n)$ for sorting + $O(n \alpha(n))$ for Union-Find operations = $O(n \log n)$.

Extension: For jobs with processing times $t_i > 1$, the problem becomes NP-hard (reduces to subset sum). The greedy heuristic still works well in practice.

15.15 Optimal Merge Pattern

Problem: Given n files with sizes $s_1, s_2, \dots, s_n$, merge them pairwise into a single file. Each merge of two files of sizes a and b costs $a + b$ (the bytes processed). Find the merge order that minimizes total cost.

This is a generalization of Huffman coding: instead of merging characters with frequencies, we

merge files with sizes. The greedy strategy is identical.

Greedy choice: Always merge the two smallest files.

15.15.1 Why Greedy Works

At each step, merging the two smallest files minimizes the cost of the current merge. More importantly, the two smallest files will participate in the most subsequent merges (they are at the deepest level of the merge tree). By making their cost as small as possible, we minimize total cost. The exchange argument proves this: swapping any pair of files with the two smallest in the merge sequence never increases total cost.

15.15.2 Worked Trace

Files with sizes: A=10, B=20, C=30, D=40, E=50.

Step 1: Merge smallest two: $A(10) + B(20) = AB(30)$. Cost = 30. Files: [AB(30), C(30), D(40), E(50)].

Step 2: Merge smallest two: $AB(30) + C(30) = ABC(60)$. Cost = 30. Total = 60. Files: [ABC(60), D(40), E(50)].

Step 3: Merge smallest two: $D(40) + E(50) = DE(90)$. Cost = 90. Total = 150. Files: [ABC(60), DE(90)].

Step 4: Merge: $ABC(60) + DE(90) = ABCDE(150)$. Cost = 150. Total = 300.

Merge tree:

```

1           ABCDE (150)
2          /         \
3       ABC (60)     DE (90)
4      /  \         /  \
5  AB (30) C (30) D (40) E (50)
6   /  \
7  A (10) B (20)
```

Total cost = $30 + 60 + 90 + 150 = 330$.

Compare with a naive left-to-right merge: $(((((A+B)+C)+D)+E) = 30 + 60 + 100 + 150 = 340$. Greedy saves 10 units (3% reduction for 5 files; the advantage grows with n).

15.15.3 C++ Implementation

```

1 #include <queue>
2
3 int optimal_merge(std::span<const int> sizes) {
4     // Min-heap of file sizes
5     std::priority_queue<int, std::vector<int>, std::greater<>> pq;
6     for (int s : sizes) pq.push(s);
7 }
```

```

8     int total_cost = 0;
9     while (pq.size() > 1) {
10         int a = pq.top(); pq.pop();
11         int b = pq.top(); pq.pop();
12         int merged = a + b;
13         total_cost += merged;
14         pq.push(merged);
15     }
16     return total_cost;
17 }

```

Complexity: $O(n \log n)$ – each of the $n-1$ merges involves two heap extractions and one insertion, each $O(\log n)$.

Connection to Huffman: Huffman coding builds an optimal prefix-free code. The merge cost in optimal merge pattern equals the total number of bits in the Huffman-encoded file (when file sizes are character frequencies). Both problems share the same greedy structure: always combine the two smallest elements.

15.16 Egyptian Fraction

Problem: Represent a fraction p/q as a sum of distinct unit fractions (fractions with numerator 1). Every positive rational number has such a representation. The greedy algorithm produces the representation with the fewest terms for a given starting fraction.

Greedy choice: At each step, find the smallest unit fraction $1/k$ such that $1/k \leq p/q$, subtract it, and repeat with the remainder.

The smallest such k is $\lceil q/p \rceil$. After subtraction, the remainder is $p/q - 1/k = (pk - q)/(qk)$.

15.16.1 Worked Examples

Example 1: $\frac{2}{3}$

Step 1: $k = \lceil 3/2 \rceil = 2$. Unit fraction: $1/2$.

Remainder: $2/3 - 1/2 = (4 - 3)/6 = 1/6$. This is already a unit fraction.

Result: $\frac{2}{3} = \frac{1}{2} + \frac{1}{6}$.

Verification: $1/2 + 1/6 = 3/6 + 1/6 = 4/6 = 2/3$. Correct.

Example 2: $\frac{6}{14} = \frac{3}{7}$

Step 1: $k = \lceil 7/3 \rceil = 3$. Unit fraction: $1/3$.

Remainder: $3/7 - 1/3 = (9 - 7)/21 = 2/21$.

Step 2: $k = \lceil 21/2 \rceil = 11$. Unit fraction: $1/11$.

Remainder: $2/21 - 1/11 = (22 - 21)/231 = 1/231$. Unit fraction.

Result: $\frac{6}{14} = \frac{1}{3} + \frac{1}{11} + \frac{1}{231}$.

Verification: $1/3 + 1/11 + 1/231 = 77/231 + 21/231 + 1/231 = 99/231 = 3/7 = 6/14$. Correct.

Example 3: $\frac{5}{7}$

Step 1: $k = \lceil 7/5 \rceil = 2$. Unit fraction: $1/2$.

Remainder: $5/7 - 1/2 = (10 - 7)/14 = 3/14$.

Step 2: $k = \lceil 14/3 \rceil = 5$. Unit fraction: $1/5$.

Remainder: $3/14 - 1/5 = (15 - 14)/70 = 1/70$. Unit fraction.

Result: $\frac{5}{7} = \frac{1}{2} + \frac{1}{5} + \frac{1}{70}$.

15.16.2 C++ Implementation

```

1 #include <numeric> // std::gcd
2
3 struct fraction { long long num, den; };
4
5 std::vector<fraction> egyptian_fraction(long long p, long long q) {
6     std::vector<fraction> result;
7
8     while (p > 0) {
9         // Find smallest k such that 1/k <= p/q, i.e., k >= q/p
10        long long k = (q + p - 1) / p; // ceil(q/p)
11
12        result.push_back({1, k});
13
14        // Subtract 1/k from p/q
15        p = p * k - q;
16        q = q * k;
17
18        if (p == 0) break;
19
20        // Simplify the remainder
21        long long g = std::gcd(p, q);
22        p /= g;
23        q /= g;
24    }
25    return result;
26 }
```

Complexity: The number of terms is at most $O(q)$ in the worst case, but typically much smaller. Each step involves $O(\log q)$ arithmetic operations for GCD. For most fractions, the algorithm terminates in a handful of steps.

Historical note: Egyptian fractions date back to 1800 BCE (Rhind Papyrus). The ancient Egyptians used only unit fractions, representing $2/3$ as a special case ($1/2 + 1/6$). The greedy algorithm (called the Fibonacci-Sylvester algorithm) always terminates and produces a finite representation.

Limitations: The greedy algorithm does not always produce the representation with the fewest terms. For example, $5/121 = 1/25 + 1/759 + 1/208725$ (greedy, 3 terms), but $5/121 = 1/33 + 1/121 + 1/363$ (also 3 terms, with smaller denominators). For minimal denominators, different strategies are needed.

15.16.3 Task Scheduling to Minimize Lateness

A dispatcher has n tasks, each with a processing time t_i and a deadline d_i . All tasks are available at time 0 and are executed non-preemptively on a single machine. The *lateness* of a task is $C_i - d_i$ where C_i is its completion time. The objective is to find a schedule that minimizes the **maximum lateness**, $L_{\max} = \max_i(C_i - d_i)$.

Greedy strategy: Sort the tasks in *earliest-deadline-first* (EDF) order and schedule them in that sequence. Intuitively, tasks that must be finished soonest should go first.

```

1 #include <algorithm>
2 #include <vector>
3
4 struct Task {
5     int id;           // task identifier
6     int t;           // processing time
7     int d;           // deadline
8 };
9
10 // Returns maximum lateness and fills 'order' with the
11 // scheduled task indices (EDF ordering).
12 int minMaximumLateness(std::vector<Task>& tasks,
13                        std::vector<int>& order) {
14     int n = tasks.size();
15     order.resize(n);
16     for (int i = 0; i < n; i++) order[i] = i;
17
18     // Sort by deadline (earliest first)
19     std::sort(order.begin(), order.end(),
20              [&](int a, int b) {
21                  return tasks[a].d < tasks[b].d;
22              });
23
24     int currentTime = 0;
25     int maxLateness = 0;
26     for (int i = 0; i < n; i++) {
27         int idx = order[i];
28         currentTime += tasks[idx].t;
29         int lateness = currentTime - tasks[idx].d;
30         if (lateness > maxLateness)
31             maxLateness = lateness;
32     }
33     return maxLateness;
34 }

```

Worked trace. Consider six tasks:

Task	1	2	3	4	5	6
t_i	3	1	4	2	5	2
d_i	6	8	9	9	14	15

After EDF sorting (by deadline, already non-decreasing):

Schedule position	1	2	3	4	5	6
Task	1	2	3	4	5	6
C_i	3	4	8	10	15	17
d_i	6	8	9	9	14	15
Lateness	-3	-4	-1	1	1	2

The maximum lateness is $L_{\max} = 2$, achieved by task 6. Any reordering that moves a later-deadline task ahead of an earlier-deadline task can only increase $L_{\max}$.

Proof of optimality (exchange argument). Let σ be any schedule that is not in EDF order. Then there exist adjacent tasks i and j in σ where $d_i > d_j$ but i is scheduled before j . Let C be the common start time of this pair. The contribution of i and j to $L_{\max}$ from this swap is $\max(C + t_i - d_i, C + t_i + t_j - d_j)$. Swapping them gives $\max(C + t_j - d_j, C + t_j + t_i - d_i)$. Since $d_j < d_i$, the first term in the swapped pair is at most as large as the corresponding term before the swap, and the second term is at most as large as well. Thus every adjacent inversion can be eliminated without increasing $L_{\max}$. By induction, the EDF schedule is optimal.

Complexity: Sorting dominates at $O(n \log n)$. The scan is $O(n)$, giving $O(n \log n)$ overall.

15.17 Exercises

15.17.1 Drill

- For each industrial scenario, identify the greedy choice and determine (conceptually) whether it produces an optimal solution:
 - AWS Spot Instance bidding** — you have a budget of \$100/hr and need to run 10 batch jobs, each with a different spot price. Greedy: bid the lowest price first.
 - CDN content caching** — 1TB cache, 1000 popular files with different request rates. Greedy: cache the most-requested files first.
 - YouTube transcoding** — 50 videos to encode, 10 encoding servers with different speeds. Greedy: assign the shortest video to the fastest server.
 - Packet routing in a network** — forward each packet along the link with the lowest current latency.
- A shipping company (like FedEx) loads containers of weights [5, 7, 10, 3, 18, 2, 9] onto a truck with capacity 30. Using the lightest-first greedy, which containers are loaded? What is the total weight? Is this optimal?

- For the fractional knapsack with $C = 50$ and items:

Item	Weight (kg)	Value (\$)
Gold	10	600
Silver	20	500
Copper	30	360

Compute the optimal solution using greedy by value/weight. What is the total value?

- Trace Dijkstra on a graph with vertices (A, B, C, D, E) and edges: $A \rightarrow B(4)$, $A \rightarrow C(2)$, $B \rightarrow C(1)$, $B \rightarrow D(5)$, $C \rightarrow D(8)$, $C \rightarrow E(10)$, $D \rightarrow E(2)$, $D \rightarrow T(6)$, $E \rightarrow T(3)$, starting from A to T. Show distance estimates after each vertex extraction.
- Trace Huffman coding on character frequencies from an English novel: E:12.7%, T:9.1%, A:8.2%, O:7.5%, I:7.0%, N:6.7%, S:6.3%, H:6.1%, R:6.0%. Build the tree, compute the weighted path length, and compare with fixed 4-bit encoding (16 symbols $\rightarrow$ 4 bits).
- What is the greedy choice in Google's PageRank-optimized crawling? In Akamai's CDN edge server selection? Why are both greedy?

15.17.2 Application

- CDN content placement (Akamai).** Akamai has edge servers worldwide. Given request rates for 1000 video files from 50 geographic regions, place copies on 10 edge servers to minimize total latency. Implement the greedy algorithm: place the most-requested file on the server closest to its requestors. Measure total latency vs an optimal (exhaustive) placement for small N.
- Spot instance bidding on AWS.** You have 100 batch jobs with different deadlines and spot prices. Each job requires 1 hour on a single instance. You have \$500 total budget. Implement a greedy scheduler: sort by profit margin (value/hour $\div$ price) and schedule the most profitable jobs first. Compare total profit with an optimal DP solution (Chapter 17). How far from optimal is the greedy result?
- Huffman compression for HTTP/2.** HTTP/2 uses HPACK header compression (a variant of Huffman). Implement Huffman encoding/decoding for HTTP headers: given a set of header names and frequencies from 10,000 real HTTP requests, build the Huffman tree and compress a sample request. Compare compression ratio with gzip and with no compression.
- Interval scheduling for cloud computing.** AWS EC2 offers reserved instances in 1-hour intervals. Your workloads specify start and end times (100 workloads, each with value \$). Implement earliest-finish-time-first scheduling to maximize the number of workloads. Then modify to maximize total value (weighted interval scheduling). Show that the unweighted greedy is optimal but the weighted variant needs DP.

1. **Greedy set cover for test suite minimization.** Given 100 test cases and 50 code modules (each test covers certain modules), select the minimum number of tests that cover all modules. Implement the greedy set cover algorithm (pick the test covering the most uncovered modules). Compare the solution size with the optimal (exponential) solution for small cases. What approximation ratio do you observe?
1. **Ad slot allocation.** A publisher has 10 ad slots per page view and 100 ad campaigns each with budget, CPM, and target audience size. Implement a greedy allocation: assign each slot to the campaign with the highest bid $\times$ relevance score. Measure total revenue vs a linear programming solution for 1000 page views.

15.17.3 Research

1. **(Matroids)** Read about matroids and prove that the minimum spanning tree problem is a graphic matroid. Then find a real problem (e.g., maximum-weight forest in a network design problem) that is also a matroid. What does the matroid property tell you about the applicability of greedy?
1. **(Greedy approximation for set cover)** Prove that the greedy set cover algorithm is a $(\ln n)$ -approximation. Implement it for a real-world test suite minimization problem (e.g., minimize the number of test cases while maintaining code coverage). On real coverage data, does the greedy solution ever exceed $\ln(n) \times$ optimal?
1. **(Adaptive Huffman)** Implement the Faller-Gallager-Knuth adaptive Huffman algorithm that updates the code tree incrementally (used in `bzip2`). Compare its compression ratio with static Huffman on a 10MB text file. How much worse is the adaptive version, and why is it useful for streaming?
1. **(Scheduling theory)** Read about Smith's rule and the EDD rule for minimizing maximum lateness. Implement a greedy scheduler for a real manufacturing line simulation: 100 jobs with processing times and due dates. Show that EDD minimizes maximum lateness.
1. **(Greedy for submodular maximization)** Read about the greedy $(1-1/e)$ -approximation for submodular maximization under a cardinality constraint. Implement it for a real sensor placement problem: given 100 candidate locations and a coverage function, place 10 sensors to maximize coverage. Why is submodularity the right model for diminishing returns?

15.17.4 Additional Exercises

1. Trace the activity selection algorithm on these 8 activities: (1,4), (3,5), (0,6), (5,7), (3,8), (5,9), (6,10), (8,11). Show the sorted order, then step through the selection. How many activities are selected? Could any alternative set of activities be larger?

1. For job sequencing with deadlines, consider 5 jobs: (deadline, profit) = (2, 80), (1, 10), (2, 20), (1, 30), (3, 50). Trace the greedy algorithm (sorted by profit). Which jobs are scheduled and what is the total profit? Now solve by brute force (try all 2^5 subsets) and confirm the greedy solution is optimal.
1. Given files of sizes [15, 35, 25, 5, 10, 45, 30], apply the optimal merge pattern algorithm. Draw the merge tree and compute total merge cost. Compare with a left-to-right sequential merge cost.
1. Apply the Egyptian fraction algorithm to the following fractions: (a) $7/15$, (b) $5/121$, (c) $10/39$. For each, show every step of the greedy decomposition and verify the result by summing the unit fractions.
1. Prove that the activity selection greedy algorithm is optimal using the exchange argument. Let OPT be an optimal schedule and G the greedy schedule. Show that if OPT selects a different first activity than G, replacing it with G's first activity produces another valid schedule with at least as many activities.
1. Design a greedy algorithm for the following problem: given n tasks, each requiring a contiguous block of time of length t_i on a shared resource, and each with a profit p_i , find a non-overlapping schedule maximizing total profit. Show that greedy by earliest finish time fails for the weighted variant and give a counterexample.
1. A hospital has n patients waiting for organ transplants, each with a blood type (A, B, AB, O) and an urgency score. There are m available organs, each with a blood type. A match is possible if the organ type is compatible with the patient's type. Design a greedy algorithm that maximizes the total urgency score of matched patients. Prove correctness or provide a counterexample.
1. Implement the optimal merge pattern algorithm and test it on random file sizes from 1 to 1000. Measure the total merge cost for $n = 10, 50, 100, 500, 1000$ files. Plot the merge cost versus n and compare with the theoretical lower bound (which is the total number of bytes times the depth of the merge tree).
1. Modify the Egyptian fraction algorithm to minimize the maximum denominator in the representation. Compare with the standard greedy approach on fractions like $5/121$, $7/15$, and $11/23$. How do the results differ? Is there a pattern?
1. (Challenge) Prove that for any set of n activities with distinct finish times, the greedy earliest-finish-time algorithm produces a schedule with at least $\lfloor n/2 \rfloor$ activities when no two activities overlap more than once. Then give a tight example where the greedy solution has exactly $\lfloor n/2 \rfloor$ activities.

15.18 References and Further Reading

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapter 16 (greedy), Chapter 35 (approximation).
- David A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes" — Proceedings of the IRE, 1952.
- Jack Edmonds, "Matroids and the Greedy Algorithm" — Mathematical Programming, 1971.
- Jon Kleinberg and Éva Tardos, *Algorithm Design*. Chapter 4 (greedy). Addison-Wesley, 2005.
- Steven S. Skiena, *The Algorithm Design Manual*, 2nd Edition. Springer, 2008.
- Eugene L. Lawler, *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart and Winston, 1976.

Divide and Conquer

16.1 Introduction

⚡ Complete compilable implementations of the data structures in this chapter are in `code/`.

16.2 The General Method

Divide and conquer solves a problem by:

1. **Divide:** break the problem into smaller subproblems of the same type
2. **Conquer:** solve each subproblem recursively
3. **Combine:** merge the solutions into the overall solution

```

1 T(n) = a*T(n/b) + f(n)
2
3 where:
4 a = number of subproblems
5 n/b = size of each subproblem
6 f(n) = cost of divide + combine

```

The **Master Theorem** (Chapter 2) gives solutions for common recurrences:

- $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ — merge sort, closest pair
- $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ — binary search
- $T(n) = 7T(n/2) + O(n^2) \rightarrow O(n^{2.807})$ — Strassen

16.3 Defective Chessboard

Problem: A $2^k \times 2^k$ chessboard has one defective square. Cover the remaining squares with L-shaped trominoes (3 squares each).

Divide the board into four quadrants. Place one tromino at the center to “defect” the three quadrants that don’t have the original defect. Recurse on each quadrant.

```

1 enum quadrant { TL, TR, BL, BR };
2
3 void cover_board(std::vector<std::vector<int>>& board,

```

```

4         size_t row, size_t col,
5         size_t defect_r, size_t defect_c,
6         size_t size, int& tile) {
7     if (size == 1) return;
8
9     size_t half = size / 2;
10    int t = tile++;
11
12    // Top-left quadrant
13    if (defect_r < row + half && defect_c < col + half) {
14        cover_board(board, row, col, defect_r, defect_c, half, tile);
15    } else {
16        board[row + half - 1][col + half - 1] = t;
17        cover_board(board, row, col, row + half - 1, col + half - 1, half,
18            tile);
19    }
20
21    // Top-right quadrant
22    if (defect_r < row + half && defect_c >= col + half) {
23        cover_board(board, row, col + half, defect_r, defect_c, half, tile
24        );
25    } else {
26        board[row + half - 1][col + half] = t;
27        cover_board(board, row, col + half, row + half - 1, col + half,
28            half, tile);
29    }
30
31    // Bottom-left quadrant
32    if (defect_r >= row + half && defect_c < col + half) {
33        cover_board(board, row + half, col, defect_r, defect_c, half, tile
34        );
35    } else {
36        board[row + half][col + half - 1] = t;
37        cover_board(board, row + half, col, row + half, col + half - 1,
38            half, tile);
39    }
40
41    // Bottom-right quadrant
42    if (defect_r >= row + half && defect_c >= col + half) {
43        cover_board(board, row + half, col + half, defect_r, defect_c,
44            half, tile);
45    } else {
46        board[row + half][col + half] = t;
47        cover_board(board, row + half, col + half, row + half, col + half,
48            half, tile);
49    }
50 }

```

Complexity: $T(n) = 4T(n/2) + O(1) \rightarrow T(n) = O(n^2) = O(4^k)$, which is optimal (there are exactly

$(4^k - 1)/3$ squares to cover).

16.4 Binary Search

Problem: Find a target value in a sorted array.

Idea: Compare the target with the middle element. If equal, done. If target is smaller, search the left half. If larger, search the right half. Each step halves the search space.

```
1 template <std::random_access_iterator Iter, typename T>
2 Iter binary_search_iter(Iter first, Iter last, const T& value) {
3     while (first < last) {
4         auto mid = first + (last - first) / 2;
5         if (*mid == value) return mid;
6         if (*mid < value) first = mid + 1;
7         else last = mid;
8     }
9     return last; // not found
10 }
```

Worked trace: Search for 7 in [1, 3, 5, 7, 9, 11, 13]:

```
1 Step 1: low=0, high=6, mid=3. arr[3]=7. Found!
```

Search for 10 in [1, 3, 5, 7, 9, 11, 13]:

```
1 Step 1: low=0, high=6, mid=3. arr[3]=7 < 10. low=4.
2 Step 2: low=4, high=6, mid=5. arr[5]=11 > 10. high=4.
3 Step 3: low=4, high=4. low == high. Not found.
```

Why $(\text{last} - \text{first}) / 2$ instead of $(\text{first} + \text{last}) / 2$: The latter can overflow for large indices. For example, if $\text{first} = 2^{31} - 1$ and $\text{last} = 2^{31} - 1$, the sum overflows a 32-bit int.

The “buggy binary search” problem: Binary search is notoriously easy to get wrong. Common bugs include: using $\text{low} \leq \text{high}$ when $\text{low} < \text{high}$ is needed (or vice versa), returning mid when low should be returned, and off-by-one in the update: $\text{low} = \text{mid}$ vs $\text{low} = \text{mid} + 1$. The version above uses the invariant that the answer, if present, is always in $[\text{first}, \text{last})$.

Recursive version:

```
1 template <typename T>
2 int binary_search_rec(const std::vector<T>& arr, int low, int high, const
3     T& target) {
4     if (low > high) return -1;
5     int mid = low + (high - low) / 2;
6     if (arr[mid] == target) return mid;
7     if (arr[mid] > target) return binary_search_rec(arr, low, mid - 1,
8         target);
9     return binary_search_rec(arr, mid + 1, high, target);
10 }
```

Recurrence: $T(n) = T(n/2) + O(1) \rightarrow T(n) = O(\log n)$. This is optimal — any comparison-based search on sorted data requires $\Omega(\log n)$ comparisons.

Applications: database index lookups (B-tree leaf search), finding insertion position in `std::lower_bound`, searching in rotated sorted arrays, and searching for a value in a sorted 2D matrix.

16.5 Finding Min and Max

Problem: Find both the minimum and maximum of an array using fewer than $2n - 2$ comparisons.

Divide and conquer approach:

- Divide array in half
- Recursively find min and max of each half
- Combine: overall min = $\min(\text{left_min}, \text{right_min})$, overall max = $\max(\text{left_max}, \text{right_max})$

```

1 std::pair<int, int> min_max(std::span<const int> arr) {
2     if (arr.size() == 1) return {arr[0], arr[0]};
3     if (arr.size() == 2) {
4         return {std::min(arr[0], arr[1]), std::max(arr[0], arr[1])};
5     }
6
7     size_t mid = arr.size() / 2;
8     auto [lmin, lmax] = min_max(arr.subspan(0, mid));
9     auto [rmin, rmax] = min_max(arr.subspan(mid));
10
11     return {std::min(lmin, rmin), std::max(lmax, rmax)};
12 }
```

Worked trace: `arr = [3, 1, 4, 1, 5, 9, 2, 6]`

```

1 min_max([3, 1, 4, 1]):
2   min_max([3, 1]) = (1, 3)
3   min_max([4, 1]) = (1, 4)
4   combine: min(1,1)=1, max(3,4)=4 -> (1, 4)
5
6 min_max([5, 9, 2, 6]):
7   min_max([5, 9]) = (5, 9)
8   min_max([2, 6]) = (2, 6)
9   combine: min(5,2)=2, max(9,6)=9 -> (2, 9)
10
11 combine: min(1,2)=1, max(4,9)=9 -> (1, 9)
```

Comparisons: $T(n) = 2T(n/2) + 2 \rightarrow T(n) = 3n/2 - 2$. This is optimal for finding both min and max simultaneously.

16.6 Sorting Algorithms

16.6.1 Merge Sort

Divide array in half, sort each half, merge.

```

1 void merge_sort(std::span<int> arr) {
2     if (arr.size() <= 1) return;
3
4     size_t mid = arr.size() / 2;
5     std::vector<int> left(arr.begin(), arr.begin() + mid);
6     std::vector<int> right(arr.begin() + mid, arr.end());
7
8     merge_sort(left);
9     merge_sort(right);
10
11     // Merge
12     size_t i = 0, j = 0, k = 0;
13     while (i < left.size() && j < right.size()) {
14         arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
15     }
16     while (i < left.size()) arr[k++] = left[i++];
17     while (j < right.size()) arr[k++] = right[j++];
18 }
```

Worked trace: arr = [3, 1, 4, 1, 5, 9, 2, 6]

```

1 Level 0 (original):      [3, 1, 4, 1, 5, 9, 2, 6]
2 Level 1 (divide):      [3, 1, 4, 1] [5, 9, 2, 6]
3 Level 2 (divide):      [3, 1] [4, 1] [5, 9] [2, 6]
4 Level 3 (base):        [3][1] [4][1] [5][9] [2][6]
5 Level 3 (merge):       [1, 3] [1, 4] [5, 9] [2, 6]
6 Level 2 (merge):       [1, 1, 3, 4] [2, 5, 6, 9]
7 Level 1 (merge):       [1, 1, 2, 3, 4, 5, 6, 9]
```

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$. **Space:** $O(n)$ extra — merge sort is not in-place. **Stability:** merge sort is stable — equal elements retain their relative order.

16.6.2 Natural Merge Sort

A variant that identifies existing sorted runs in the data. If the input is already sorted, natural merge sort runs in $O(n)$.

```

1 // Natural merge sort: identify runs, then merge adjacent runs
2 void natural_merge_sort(std::span<int> arr) {
3     if (arr.size() <= 1) return;
4
5     // Phase 1: find all natural runs
6     std::vector<std::pair<size_t, size_t>> runs; // (start, length)
```

```

7     size_t start = 0;
8     for (size_t i = 1; i <= arr.size(); ++i) {
9         if (i == arr.size() || arr[i] < arr[i - 1]) {
10             runs.push_back({start, i - start});
11             start = i;
12         }
13     }
14
15     // Phase 2: merge adjacent runs until one remains
16     while (runs.size() > 1) {
17         std::vector<std::pair<size_t, size_t>> merged;
18         for (size_t i = 0; i + 1 < runs.size(); i += 2) {
19             // Merge runs[i] and runs[i+1]
20             size_t s1 = runs[i].first, len1 = runs[i].second;
21             size_t s2 = runs[i+1].first, len2 = runs[i+1].second;
22             std::inplace_merge(arr.begin() + s1, arr.begin() + s1 + len1,
23                               arr.begin() + s2 + len2);
24             merged.push_back({s1, len1 + len2});
25         }
26         if (runs.size() % 2 == 1) merged.push_back(runs.back());
27         runs = merged;
28     }
29 }

```

Best case: $O(n)$ when the input is already sorted (one run of length n). **Average/Worst:** $O(n \log n)$ — same as standard merge sort. **When it wins:** partially sorted data (e.g., merging results from multiple sorted files in a database).

16.6.3 Quick Sort

Choose a **pivot**, partition the array into elements $\leq$ pivot and $\geq$ pivot, recurse on each partition.

```

1 size_t partition(std::span<int> arr) {
2     // Lomuto partition scheme
3     int pivot = arr.back();
4     size_t i = 0;
5     for (size_t j = 0; j < arr.size() - 1; ++j) {
6         if (arr[j] <= pivot) {
7             std::swap(arr[i], arr[j]);
8             ++i;
9         }
10    }
11    std::swap(arr[i], arr.back());
12    return i;
13 }
14
15 void quick_sort(std::span<int> arr) {
16     if (arr.size() <= 1) return;

```

```

17     size_t p = partition(arr);
18     quick_sort(arr.subspan(0, p));
19     quick_sort(arr.subspan(p + 1));
20 }

```

Average: $T(n) = O(n \log n)$. **Worst case (already sorted):** $O(n^2)$. Mitigated by random pivot selection:

```

1 size_t random_partition(std::span<int> arr) {
2     size_t r = std::rand() % arr.size();
3     std::swap(arr[r], arr.back());
4     return partition(arr);
5 }

```

Space: $O(\log n)$ for the recursion stack (worst case $O(n)$).

16.6.4 Comparison

Algorithm	Worst Case	Average	Best Case	Space	Stable
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Introsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No

Introsort (used by `std::sort`) starts with quicksort, switches to heapsort if recursion depth exceeds $\log n$, and uses insertion sort for small arrays.

16.7 Radix Sort

Comparison-based sorting has a lower bound of $\Omega(n \log n)$. Radix sort breaks this barrier by exploiting the structure of the keys themselves. Instead of comparing elements, radix sort processes each digit (or bit) of the key.

16.7.1 LSD Radix Sort

Least Significant Digit (LSD) radix sort processes digits from right to left. For each digit position, it performs a stable sort (typically counting sort) on that digit. Because the sort is stable, earlier digits maintain their relative order.

```

1 void lsd_radix_sort(std::vector<int>& arr) {
2     if (arr.empty()) return;
3     int maxVal = *std::max_element(
4         arr.begin(), arr.end());
5
6     for (int exp = 1; maxVal / exp > 0; exp *= 10) {
7         std::vector<int> output(arr.size());

```



```

8         int count[10] = {0};
9
10        for (int x : arr)
11            count[(x / exp) % 10]++;
12
13        for (int i = 1; i < 10; i++)
14            count[i] += count[i - 1];
15
16        for (int i = arr.size() - 1; i >= 0; i--) {
17            int digit = (arr[i] / exp) % 10;
18            output[count[digit] - 1] = arr[i];
19            count[digit]--;
20        }
21        arr = output;
22    }
23 }

```

For n w -bit integers, LSD radix sort runs in $O(nw)$ time. When $w = O(\log n)$, this is $O(n \log n)$; when w is a constant (e.g., sorting 32-bit integers with byte-wise passes), it is $O(n)$.

16.7.2 MSD Radix Sort

Most Significant Digit (MSD) radix sort processes digits from left to right. It is naturally recursive: after partitioning by the first digit, it recursively sorts each bucket by the next digit.

```

1 void msd_radix_sort(std::vector<int>& arr,
2     int lo, int hi, int bit) {
3     if (lo >= hi || bit < 0) return;
4
5     int mask = 1 << bit;
6     int mid = lo;
7     // Three-way partition: 0s, then 1s
8     for (int i = lo; i <= hi; i++) {
9         if ((arr[i] & mask) == 0) {
10             std::swap(arr[mid++], arr[i]);
11         }
12     }
13     msd_radix_sort(arr, lo, mid - 1, bit - 1);
14     msd_radix_sort(arr, mid, hi, bit - 1);
15 }
16
17 void msd_radix_sort(std::vector<int>& arr) {
18     if (arr.empty()) return;
19     int maxVal = *std::max_element(
20         arr.begin(), arr.end());
21     int highestBit = 31 - __builtin_clz(maxVal);
22     msd_radix_sort(arr, 0, arr.size() - 1,
23         highestBit);

```

```
24 }
```

MSD radix sort is better for variable-length keys (like strings) because it can stop early when buckets become small.

16.7.3 Radix Sort for Strings

LSD radix sort works naturally for fixed-length strings (e.g., IP addresses, UUIDs). For variable-length strings, MSD radix sort with insertion sort for small buckets is the standard approach. This is the basis of sorting in database engines and file systems.

```
1 void radix_sort_strings(std::vector<std::string>& arr,
2     int lo, int hi, int d) {
3     if (lo >= hi) return;
4     const int R = 256; // ASCII alphabet
5
6     std::vector<std::vector<std::string>> buckets(R);
7     for (int i = lo; i <= hi; i++) {
8         int c = d < (int)arr[i].size()
9             ? (unsigned char)arr[i][d] : 0;
10        buckets[c].push_back(arr[i]);
11    }
12
13    int idx = lo;
14    for (int r = 0; r < R; r++) {
15        if (buckets[r].size() > 1)
16            radix_sort_strings(buckets[r], 0,
17                               buckets[r].size() - 1, d + 1);
18        for (auto& s : buckets[r])
19            arr[idx++] = s;
20    }
21 }
```

16.7.4 Complexity

Variant	Time	Space	Best For
LSD radix sort	$O(nw)$	$O(n + R)$	Fixed-length integer keys
MSD radix sort	$O(nw)$ avg	$O(n + wR)$	Variable-length strings
MSD + insertion cutoff	$O(nw)$ avg	$O(n + wR)$	Practical string sorting

Where w = number of digits/bits per key, R = radix (base, typically 256 for bytes).

16.7.5 Industrial Applications

- **Database engines:** MySQL, PostgreSQL use radix sort for sorting integer keys and fixed-length records

- **Distributed systems:** MapReduce/BigTable use radix-based partitioning for sort-keys across nodes
- **Network packet classification:** sorting packets by IP address/port using multi-key radix sort
- **Genomics:** sorting DNA sequences (4-letter alphabet) with MSD radix sort
- **File systems:** ext4 and NTFS sort directory entries using radix-based methods
- **Search engines:** sorting document IDs and inverted list indices

16.8 Selection (Finding the k-th Smallest)

Problem: Find the k-th smallest element without fully sorting the array.

Quickselect: Partition the array around a pivot, then recurse only on the partition containing the k-th element.

```

1 int quick_select(std::span<int> arr, size_t k) {
2     if (arr.size() == 1) return arr[0];
3
4     size_t p = random_partition(arr);
5
6     if (k == p) return arr[p];
7     if (k < p) return quick_select(arr.subspan(0, p), k);
8     return quick_select(arr.subspan(p + 1), k - p - 1);
9 }
```

Worked trace: arr = [3, 1, 4, 1, 5, 9, 2, 6], k = 4 (find 5th smallest).

```

1 pivot = 6 (last element). Partition: [3, 1, 4, 1, 5, 2] [6] [9]
2 k = 4 > pivot index 6. Recurse on [9], k = 4 - 6 - 1 = ...
3
4 Wait, let me redo with k=4 (0-indexed):
5 pivot = 6. After partition: [3, 1, 4, 1, 5, 2] [6] [9]
6 pivot index = 6. k = 4 < 6. Recurse on [3, 1, 4, 1, 5, 2], k = 4.
7 pivot = 2. After partition: [1, 1] [2] [3, 4, 5]
8 pivot index = 2. k = 4 > 2. Recurse on [3, 4, 5], k = 4 - 2 - 1 = 1.
9 pivot = 5. After partition: [3, 4] [5] []
10 pivot index = 2. k = 1 < 2. Recurse on [3, 4], k = 1.
11 pivot = 4. After partition: [3] [4]
12 pivot index = 1. k = 1. Return 4.
```

Wait — the 5th smallest (k=4) in [1, 1, 2, 3, 4, 5, 6, 9] is 4. Correct.

Average: $O(n)$. **Worst:** $O(n^2)$. The **Median of Medians** algorithm achieves $O(n)$ worst-case selection by choosing a pivot that guarantees a 30-70 split: divide into groups of 5, find the median of each group, then find the median of those medians. The guaranteed split gives $T(n) = T(3n/10) + T(7n/10) + O(n) = O(n)$.

Applications: finding medians, order statistics, percentile calculations, and as a subroutine in quicksort.

16.9 Closest Pair of Points

Problem: Given n points in the plane, find the pair with the smallest Euclidean distance.

Divide and conquer approach:

1. Sort points by x-coordinate
2. Divide at median x
3. Recursively find $d = \min(\text{closest in left}, \text{closest in right})$
4. Consider the strip of width $2d$ around the median — only $O(n)$ points to check

```

1 struct point { double x, y; };
2
3 double closest_pair(std::span<point> points) {
4     if (points.size() <= 3) return brute_force(points);
5
6     size_t mid = points.size() / 2;
7     double mid_x = points[mid].x;
8
9     double dl = closest_pair(points.subspan(0, mid));
10    double dr = closest_pair(points.subspan(mid));
11    double d = std::min(dl, dr);
12
13    // Collect points within the strip
14    std::vector<point> strip;
15    for (const auto& p : points) {
16        if (std::abs(p.x - mid_x) < d) strip.push_back(p);
17    }
18
19    // Sort strip by y
20    std::sort(strip.begin(), strip.end(),
21              [](const point& a, const point& b) { return a.y < b.y; });
22
23    // Check each point against next few (at most 7)
24    for (size_t i = 0; i < strip.size(); ++i) {
25        for (size_t j = i + 1; j < strip.size() && (strip[j].y - strip[i].
26            y) < d; ++j) {
27            double dist = euclidean(strip[i], strip[j]);
28            d = std::min(d, dist);
29        }
30    }
31    return d;

```

Why only 7 comparisons per point: In the strip, divide each point's neighborhood into 8 cells of size $d/2 \times d/2$. By the pigeonhole principle, at most one point can be in each cell (otherwise two points in the same cell would be closer than d). So each point has at most 7 neighbors to check.

Complexity: $T(n) = 2T(n/2) + O(n \log n)$ [sorting by y each level] $\rightarrow O(n \log^2 n)$. **Optimized:** Pre-sort by y once $\rightarrow T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.

16.10 Strassen's Matrix Multiplication

Standard matrix multiplication: 8 multiplications of $n/2 \times n/2$ submatrices $\rightarrow T(n) = 8T(n/2) + O(n^2) \rightarrow O(n^3)$.

Strassen's algorithm reduces multiplication count to 7:

```

1 M$_{1}$ = (A$_{1}$$_{1}$ + A$_{2}$$_{2}$) * (B$_{1}$$_{1}$ + B$_{2}$$_{2}$)
2 M$_{2}$ = (A$_{2}$$_{1}$ + A$_{2}$$_{2}$) * B$_{1}$$_{1}$
3 M$_{3}$ = A$_{1}$$_{1}$ * (B$_{1}$$_{2}$ - B$_{2}$$_{2}$)
4 M$_{4}$ = A$_{2}$$_{2}$ * (B$_{2}$$_{1}$ - B$_{1}$$_{1}$)
5 M$_{5}$ = (A$_{1}$$_{1}$ + A$_{1}$$_{2}$) * B$_{2}$$_{2}$
6 M$_{6}$ = (A$_{2}$$_{1}$ - A$_{1}$$_{1}$) * (B$_{1}$$_{1}$ + B$_{1}$$_{2}$)
7 M$_{7}$ = (A$_{1}$$_{2}$ - A$_{2}$$_{2}$) * (B$_{2}$$_{1}$ + B$_{2}$$_{2}$)
8
9 C$_{1}$$_{1}$ = M$_{1}$ + M$_{4}$ - M$_{5}$ + M$_{7}$
10 C$_{1}$$_{2}$ = M$_{3}$ + M$_{5}$
11 C$_{2}$$_{1}$ = M$_{2}$ + M$_{4}$
12 C$_{2}$$_{2}$ = M$_{1}$ - M$_{2}$ + M$_{3}$ + M$_{6}$

```

Worked trace (2x2 matrices):

```

1 A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]]
2
3 Standard: 8 multiplications = 1*5 + 1*6 + 2*7 + 2*8 + 3*5 + 3*6 + 4*7 +
  4*8 = 154
4
5 Strassen:
6 M1 = (1+4)*(5+8) = 5*13 = 65
7 M2 = (3+4)*5 = 7*5 = 35
8 M3 = 1*(6-8) = 1*(-2) = -2
9 M4 = 4*(7-5) = 4*2 = 8
10 M5 = (1+2)*8 = 3*8 = 24
11 M6 = (3-1)*(5+6) = 2*11 = 22
12 M7 = (2-4)*(7+8) = (-2)*15 = -30
13
14 C11 = 65 + 8 - 24 + (-30) = 19
15 C12 = -2 + 24 = 22
16 C21 = 35 + 8 = 43

```

```

17 C22 = 65 - 35 + (-2) + 22 = 50
18
19 C = [[19, 22], [43, 50]] -- matches standard multiplication

```

Complexity: $T(n) = 7T(n/2) + O(n^2) \rightarrow T(n) = O(n^{\log_2 7}) \approx O(n^{2.807})$.

Practical considerations: Strassen's algorithm has worse numerical stability (subtractive cancellation) and cache performance for small matrices. Libraries like BLAS use it only for large matrices ($n \geq 512$ -1024). The crossover point depends on hardware cache sizes.

16.11 Common Bugs and Pitfalls

- **Incorrect base case** — forgetting the base case ($n \leq 1$) causes infinite recursion and stack overflow. Always verify that every recursive call moves toward the base case.
- **Expensive combine step** — if the combine step is $O(n^2)$, a divide-and-conquer algorithm may be slower than a naive solution. This is why the closest-pair algorithm must carefully limit the strip check to $O(n)$.
- **Quick sort with bad pivot selection** — always using the first/last element as pivot causes $O(n^2)$ on sorted/almost-sorted data. Use median-of-three or random pivots in production.
- **Merge sort's extra memory** — merge sort requires $O(n)$ additional space. In-place merge sort exists but is complex and rarely used. For systems with tight memory (embedded), consider heap sort or introsort instead.
- **Strassen's numerical stability** — Strassen's algorithm uses subtractions that can cause catastrophic cancellation for ill-conditioned matrices. Production matrix libraries (BLAS, Eigen) use Strassen only for large matrices where the error is bounded.

16.12 Summary

1. **Divide and conquer** breaks a problem into independent subproblems, solves them recursively, and combines solutions.
2. **Binary search** is the simplest and most widely used divide-and-conquer algorithm: $O(\log n)$ search in sorted data.
3. **Merge sort** and **quick sort** are the canonical divide-and-conquer sorts. Merge sort guarantees $O(n \log n)$ but uses $O(n)$ space; quick sort is in-place but has $O(n^2)$ worst case.
4. **Closest pair** and **Strassen's multiplication** demonstrate non-trivial divide and combine steps.
5. Industrial applications include **merge sort in PostgreSQL** (external sorting), **quick sort in C++ `std::sort`** (introsort hybrid), **Strassen in matrix libraries**, **FFT in JPEG/MP3**, **binary search in database indexes**, and **quickselect** in Netflix's recommendation rankings.

6. The **Master Theorem** (Chapter 2) provides the complexity framework.

16.13 Interview FAQs

1. **What is the invariant of binary search, and why do most implementations get it wrong?**

The invariant is: the answer is always in the range $[lo, hi]$. On each iteration, you eliminate half the range by comparing the midpoint with the target. Most implementations get the midpoint calculation wrong: $lo + (hi - lo) / 2$ is correct; $(lo + hi) / 2$ overflows for large indices. The bigger bug: the loop termination condition. `while (lo <= hi)` with `lo = mid + 1` and `hi = mid - 1` is correct for finding an exact match. `while (lo < hi)` with `hi = mid` is correct for finding the leftmost insertion point. Mixing these up gives subtle bugs that pass most test cases. The C++ standard library uses `std::lower_bound` and `std::upper_bound` — never write binary search by hand.

1. **When is quicksort better than merge sort in practice?**

Quicksort has better cache performance (in-place, sequential access) and smaller constants. For in-memory sorting of primitive types, quicksort (or introsort) is typically 2-3x faster than merge sort. Merge sort wins when stability is required, when worst-case $O(n \log n)$ is mandatory, or for external sorting (data on disk, where sequential access dominates).

1. **Why is quickselect $O(n)$ average but $O(n^2)$ worst-case, and how does introselect fix this?**

Quickselect partitions like quicksort but recurses into only ONE partition — the one containing the k -th element. On average, each partition halves the search space: $n + n/2 + n/4 + \dots = 2n = O(n)$. Worst case: if the pivot is always the smallest/largest element, you get $n + (n-1) + (n-2) + \dots = O(n^2)$. Introselect (used in `std::nth_element`) starts with quickselect's partitioning but switches to median-of-medians (guaranteed $O(n)$ pivot) if recursion depth exceeds $2 \log_2 n$. This gives $O(n)$ worst-case while preserving quickselect's cache-friendly behavior for the common case.

1. **Explain the strip optimization in closest pair.**

After finding $d = \min$ distance in left and right halves, we only need to check points in a strip of width $2d$ around the dividing line. Within this strip, sort by y and compare each point with at most 7 neighbors. This reduces the combine step from $O(n^2)$ to $O(n)$, making the overall algorithm $O(n \log n)$.

1. **Why does Strassen's algorithm only save one multiplication?**

Naive multiplication needs 8 multiplications for 2×2 blocks. Strassen discovered that by computing 7 clever products (using additions/subtractions of the input elements), the 4 output blocks can be reconstructed. The key insight is that multiplication is more expensive than addition, so reducing one multiplication at the cost of several additions is a net win when n is large.

1. What is the difference between tail recursion and regular recursion in divide-and-conquer?

Tail recursion is when the recursive call is the last operation — the compiler can optimize it into a loop ($O(1)$ stack). Binary search's iterative version demonstrates this optimization. Merge sort and quicksort cannot be tail-recursive because they make TWO recursive calls and then combine. The second call must be deferred on the stack.

1. How do you handle the base case for floating-point divide-and-conquer?

For continuous problems (root finding, integration), use a tolerance threshold: stop when the interval width $< \epsilon$ or when the function value is close enough to zero. Binary search for roots (bisection method) converges in $O(\log((b-a)/\epsilon))$ iterations. The base case must account for floating-point precision — never test `mid == target` exactly for doubles.

1. Why is bottom-up merge sort the foundation for external sorting on disk?

Bottom-up merge sort processes subarrays of size 1, 2, 4, 8... with no recursion — each pass makes one sequential scan of the data. For external sorting (data too large for RAM), you: (1) sort chunks that fit in RAM (using any $O(n \log n)$ algorithm), (2) merge pairs of sorted chunks using bottom-up merge with k -way merging. The key insight: sequential I/O is $100\times$ faster than random I/O on disks. Bottom-up merge sort reads and writes sequentially, making it ideal for magnetic tapes and HDDs. For SSDs, the gap is smaller but sequential access still wins. The pass structure (1, 2, 4, 8...) also enables pipeline parallelism: while one pass writes, the next can read.

1. Why does introsort switch to heapsort at depth $2\log_2 n$, and why not just use heapsort everywhere?

Heapsort has $O(n \log n)$ worst-case but poor cache locality — the parent-child index relationship jumps across the array, causing cache misses. Quicksort has $O(n^2)$ worst-case but excellent cache locality — partitioning scans sequentially. Introsort gets the best of both: quicksort's cache behavior for the common case, heapsort's guarantee for the worst case. The threshold $2\log_2 n$ is chosen because: (1) a well-pivoted quicksort finishes in $\sim \log_2 n$ levels, so $2\log_2 n$ means the pivot quality is severely degraded; (2) switching to heapsort at this point limits the total work to $O(n \log n)$ while keeping the cache-friendly quicksort for 99.9% of partitions.

1. Explain the natural merge sort advantage for partially sorted data.

Natural merge sort detects existing sorted runs before merging. If the data already has long sorted segments (e.g., two merged files, or data with small perturbations), the number of merge passes is determined by the number of runs, not by $\log n$. For k already-sorted runs, natural merge sort runs in $O(n \log k)$ — much faster than $O(n \log n)$ when $k \ll n$.

16.14 Closest Pair of Points: Detailed Treatment

The section on closest pair earlier gave the high-level idea. Here we provide a complete treatment with brute force, a detailed divide-and-conquer trace, and a full implementation.

16.14.1 Brute Force: $O(n^2)$

The simplest approach checks every pair of points:

```

1 double brute_force_closest(std::span<point> pts) {
2     double best = std::numeric_limits<double>::infinity();
3     for (size_t i = 0; i < pts.size(); ++i) {
4         for (size_t j = i + 1; j < pts.size(); ++j) {
5             double dx = pts[i].x - pts[j].x;
6             double dy = pts[i].y - pts[j].y;
7             double dist = std::sqrt(dx * dx + dy * dy);
8             best = std::min(best, dist);
9         }
10    }
11    return best;
12 }
```

Complexity: $T(n) = n(n-1)/2 = O(n^2)$ comparisons. For $n = 10^6$ points, this requires about 5×10^{11} distance computations — far too slow.

16.14.2 Divide and Conquer: $O(n \log n)$

Step 1: Sort all points by x-coordinate (done once, $O(n \log n)$).

Step 2: Recursively split at the median x-coordinate.

Step 3: Let $d = \min(d_{left}, d_{right})$. Any closer pair must straddle the dividing line, with both points within distance d of it.

Step 4: Build the strip of points within distance d of the median line. Sort the strip by y-coordinate. For each point, compare with at most 7 subsequent points in the strip (pigeonhole argument on $d/2 \times d/2$ cells).

Why 7? Partition the strip into a grid of cells of size $d/2 \times d/2$. Each cell contains at most one point (otherwise two points in the same cell would be closer than d , contradicting $d = \min(d_{left}, d_{right})$). A point can have neighbors in at most 8 surrounding cells, but the cell directly below is excluded (already processed), leaving at most 7.

Worked trace on 8 points:

Points (sorted by x): $P_1(1,1)$, $P_2(2,3)$, $P_3(3,4)$, $P_4(4,1)$, $P_5(5,5)$, $P_6(6,2)$, $P_7(7,3)$, $P_8(8,6)$

```

1 Split at median (x=4.5):
2   Left:  P1(1,1) P2(2,3) P3(3,4) P4(4,1)
3   Right: P5(5,5) P6(6,2) P7(7,3) P8(8,6)
4
```

```

5 Recurse on Left half:
6   Split at x=2.5:
7     Left-Left: P1(1,1)
8     Left-Right: P2(2,3) P3(3,4) P4(4,1)
9     d_LL = infinity (one point)
10    d_LR = min(dist(P2,P3), dist(P2,P4), dist(P3,P4))
11           = min(1.414, 2.236, 3.162) = 1.414 [P2-P3]
12    d_L = 1.414
13
14 Recurse on Right half:
15   Split at x=6.5:
16     Right-Left: P5(5,5) P6(6,2) P7(7,3)
17     Right-Right: P8(8,6)
18     d_RL = min(dist(P5,P6), dist(P5,P7), dist(P6,P7))
19           = min(3.606, 2.828, 1.414) = 1.414 [P6-P7]
20     d_RR = infinity (one point)
21     d_R = 1.414
22
23 d = min(d_L, d_R) = 1.414
24
25 Build strip (points within d=1.414 of x=4.5):
26   Strip: P3(3,4), P4(4,1), P5(5,5), P6(6,2), P7(7,3)
27   (P2 at x=2 is 2.5 away -- outside strip)
28
29 Sort strip by y: P4(4,1), P6(6,2), P7(7,3), P3(3,4), P5(5,5)
30
31 Check pairs in strip:
32   P4(4,1) vs P6(6,2): dist = 2.236 > d.
33   P4(4,1) vs P7(7,3): dy=2 > d. Move on.
34   P6(6,2) vs P7(7,3): dist = 1.414 = d. Tie, no improvement.
35   P7(7,3) vs P3(3,4): dy=1, but dx=4 > d. Skip.
36   P3(3,4) vs P5(5,5): dist = 2.236 > d.
37
38 No closer pair found. d = 1.414.
39 Closest pairs: (P2,P3) and (P6,P7), both at distance sqrt(2) ~ 1.414.

```

Full implementation:

```

1 #include <algorithm>
2 #include <cmath>
3 #include <limits>
4 #include <vector>
5
6 struct point { double x, y; };
7
8 double dist(const point& a, const point& b) {
9     double dx = a.x - b.x, dy = a.y - b.y;
10    return std::sqrt(dx * dx + dy * dy);
11 }

```

```

12
13 double strip_closest(std::vector<point>& strip, double d) {
14     double best = d;
15     std::sort(strip.begin(), strip.end(),
16         [](const point& a, const point& b) { return a.y < b.y; });
17     for (size_t i = 0; i < strip.size(); ++i) {
18         for (size_t j = i + 1;
19             j < strip.size() && (strip[j].y - strip[i].y) < best;
20             ++j) {
21             best = std::min(best, dist(strip[i], strip[j]));
22         }
23     }
24     return best;
25 }
26
27 double closest_rec(std::vector<point>& pts, size_t lo, size_t hi) {
28     if (hi - lo <= 3) {
29         double d = std::numeric_limits<double>::infinity();
30         for (size_t i = lo; i < hi; ++i)
31             for (size_t j = i + 1; j < hi; ++j)
32                 d = std::min(d, dist(pts[i], pts[j]));
33         return d;
34     }
35
36     size_t mid = lo + (hi - lo) / 2;
37     double mid_x = pts[mid].x;
38     double dl = closest_rec(pts, lo, mid);
39     double dr = closest_rec(pts, mid, hi);
40     double d = std::min(dl, dr);
41
42     std::vector<point> strip;
43     for (size_t i = lo; i < hi; ++i) {
44         if (std::abs(pts[i].x - mid_x) < d)
45             strip.push_back(pts[i]);
46     }
47     return strip_closest(strip, d);
48 }
49
50 double closest_pair(std::vector<point> pts) {
51     std::sort(pts.begin(), pts.end(),
52         [](const point& a, const point& b) { return a.x < b.x; });
53     return closest_rec(pts, 0, pts.size());
54 }

```

Complexity: Sorting by x: $O(n \log n)$. Recurrence: $T(n) = 2T(n/2) + O(n)$ (strip construction + sort by y) = $O(n \log n)$. With pre-sorted y arrays carried through the recursion, the strip sort is eliminated and the recurrence becomes $T(n) = 2T(n/2) + O(n) = O(n \log n)$ with smaller constants. Total: $O(n \log n)$.

16.15 Maximum Subarray

Problem: Given an array of integers (possibly negative), find the contiguous subarray with the largest sum.

16.15.1 Divide and Conquer Approach

Split the array in half. The maximum subarray either lies entirely in the left half, entirely in the right half, or crosses the midpoint. The first two cases are recursive; the third is solved in $O(n)$ by scanning outward from the midpoint.

```

1 struct result { int left, right, sum; };
2
3 result max_crossing(std::span<const int> arr, int lo, int mid, int hi) {
4     int left_sum = std::numeric_limits<int>::min();
5     int sum = 0, best_left = mid;
6     for (int i = mid; i >= lo; --i) {
7         sum += arr[i];
8         if (sum > left_sum) { left_sum = sum; best_left = i; }
9     }
10
11     int right_sum = std::numeric_limits<int>::min();
12     sum = 0;
13     int best_right = mid + 1;
14     for (int j = mid + 1; j <= hi; ++j) {
15         sum += arr[j];
16         if (sum > right_sum) { right_sum = sum; best_right = j; }
17     }
18     return {best_left, best_right, left_sum + right_sum};
19 }
20
21 result max_subarray(std::span<const int> arr, int lo, int hi) {
22     if (lo == hi) return {lo, hi, arr[lo]};
23
24     int mid = lo + (hi - lo) / 2;
25     auto left = max_subarray(arr, lo, mid);
26     auto right = max_subarray(arr, mid + 1, hi);
27     auto cross = max_crossing(arr, lo, mid, hi);
28
29     if (left.sum >= right.sum && left.sum >= cross.sum) return left;
30     if (right.sum >= left.sum && right.sum >= cross.sum) return right;
31     return cross;
32 }

```

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$.

16.15.2 Kadane's Algorithm: $O(n)$

A dynamic programming approach scans left to right, tracking the maximum subarray ending at each position:

```

1 result kadane(std::span<const int> arr) {
2     int best_sum = arr[0], cur_sum = arr[0];
3     int best_start = 0, best_end = 0, cur_start = 0;
4
5     for (int i = 1; i < (int)arr.size(); ++i) {
6         if (cur_sum < 0) { cur_sum = arr[i]; cur_start = i; }
7         else { cur_sum += arr[i]; }
8
9         if (cur_sum > best_sum) {
10             best_sum = cur_sum;
11             best_start = cur_start;
12             best_end = i;
13         }
14     }
15     return {best_start, best_end, best_sum};
16 }
```

Key idea: At each position i , the maximum subarray ending at i is either $\text{arr}[i]$ alone or the maximum subarray ending at $i-1$ extended by $\text{arr}[i]$. This gives the recurrence: $\text{max_ending_here}[i] = \max(\text{arr}[i], \text{max_ending_here}[i-1] + \text{arr}[i])$.

Worked trace: $\text{arr} = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$

```

1 i=0: cur=-2, best=-2. Reset: cur=-2, start=0
2 i=1: cur<0 -> cur=1, start=1. best=1.
3 i=2: cur=1+(-3)=-2. best still 1.
4 i=3: cur<0 -> cur=4, start=3. best=4.
5 i=4: cur=4+(-1)=3. best still 4.
6 i=5: cur=3+2=5. best=5.
7 i=6: cur=5+1=6. best=6. [4,-1,2,1] sum=6
8 i=7: cur=6+(-5)=1. best still 6.
9 i=8: cur=1+4=5. best still 6.
10
11 Result: subarray [4, -1, 2, 1] from index 3 to 6, sum = 6.
```

Comparison:

Approach	Time	Space	Notes
Divide & Conquer	$O(n \log n)$	$O(\log n)$ stack	Good for parallel
Kadane's	$O(n)$	$O(1)$	Optimal, sequential

Kadane's $O(n)$ is optimal (any algorithm must examine every element). The divide-and-conquer version is useful as an exercise in the paradigm and as a basis for parallel prefix-sum implementations on GPUs.

Applications: stock trading (max profit from one buy-sell), image processing (brightest contiguous region), genomic sequence analysis (maximum scoring segment), and the Kadane vs D&C comparison is a classic interview question.

16.16 Counting Inversions

Problem: Given an array, count the number of pairs (i, j) where $i < j$ but $\text{arr}[i] > \text{arr}[j]$. Inversions measure how far an array is from being sorted.

Application: Collaborative filtering — if two users rank the same movies, the number of inversions between their rankings measures dissimilarity. This is used in recommendation systems (e.g., comparing user preferences on Netflix).

16.16.1 Brute Force: $O(n^2)$

```
1 int count_inversions_brute(std::span<const int> arr) {
2     int count = 0;
3     for (size_t i = 0; i < arr.size(); ++i)
4         for (size_t j = i + 1; j < arr.size(); ++j)
5             if (arr[i] > arr[j]) ++count;
6     return count;
7 }
```

16.16.2 Merge Sort Variant: $O(n \log n)$

During the merge step, when an element from the right half is placed before remaining elements in the left half, each of those remaining left elements forms an inversion with it. Count these during merge.

```
1 long long merge_and_count(std::vector<int>& arr, std::vector<int>& tmp,
2                           int lo, int mid, int hi) {
3     long long inversions = 0;
4     int i = lo, j = mid, k = lo;
5
6     while (i < mid && j <= hi) {
7         if (arr[i] <= arr[j]) {
8             tmp[k++] = arr[i++];
9         } else {
10            tmp[k++] = arr[j++];
11            inversions += mid - i; // all remaining left > arr[j]
12        }
13    }
14    while (i < mid) tmp[k++] = arr[i++];
15    while (j <= hi) tmp[k++] = arr[j++];
16
17    for (int i = lo; i <= hi; ++i) arr[i] = tmp[i];
18    return inversions;
19 }
```

```

19 }
20
21 long long count_inversions(std::vector<int>& arr, std::vector<int>& tmp,
22                             int lo, int hi) {
23     if (lo >= hi) return 0;
24     int mid = lo + (hi - lo) / 2;
25     long long inv = 0;
26     inv += count_inversions(arr, tmp, lo, mid);
27     inv += count_inversions(arr, tmp, mid + 1, hi);
28     inv += merge_and_count(arr, tmp, lo, mid, hi);
29     return inv;
30 }

```

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$, same as merge sort. The extra work per merge step is $O(1)$ per element (one comparison and one potential count update).

Worked trace: $\text{arr} = [2, 4, 1, 3, 5]$

```

1 Split: [2, 4, 1] and [3, 5]
2
3 Left half [2, 4, 1]:
4   Split [2] and [4, 1].
5   [4, 1] merge: 4>1 -> 1 inversion. Merged: [1, 4].
6   Merge [2] [1, 4]:
7   2>1 -> 1 inv (1 remaining in left: [2]).
8   2<=4 -> 0 inv.
9   Merged: [1, 2, 4]. Total left inversions = 1 + 1 = 2.
10
11 Right half [3, 5]:
12   3<=5 -> 0 inversions.
13
14 Merge [1,2,4] [3,5]:
15   1<=3 -> 0 inv.
16   2<=3 -> 0 inv.
17   4>3 -> 1 inv (1 remaining in left: [4]).
18   4<=5 -> 0 inv.
19   Merged: [1, 2, 3, 4, 5]. Cross inversions = 1.
20
21 Total inversions: 2 + 0 + 1 = 3.
22 Verification: pairs (2,1), (4,1), (4,3). Correct.

```

Applications: measuring ranking similarity (Kendall tau distance), detecting sortedness in adaptive sort algorithms, and the “minimum swaps to sort” problem (inversions give a lower bound).

Variant — count non-strict inversions: Change the merge condition from $\text{arr}[i] \leq \text{arr}[j]$ to $\text{arr}[i] < \text{arr}[j]$ to count pairs where $\text{arr}[i] \geq \text{arr}[j]$ (i.e., count equal elements as inversions too).

16.17 Strassen's Algorithm: Recursive Trace

The earlier section stated the 7 products and reconstruction formulas. Here we trace the recursion on 4×4 matrices to show how the algorithm decomposes.

Input: Two 4×4 matrices A and B, each partitioned into four 2×2 blocks.

```

1 A = [[A11, A12],    B = [[B11, B12],
2     [A21, A22]]      [B21, B22]]
3
4 where each Aij, Bij is a 2x2 matrix.
```

Step 1: Compute 7 products of 2×2 matrices (each product itself may recurse if the submatrices are larger than the base case):

```

1 M1 = (A11 + A22) * (B11 + B22)    // 1 add + 1 add + 1 mult
2 M2 = (A21 + A22) * B11             // 1 add + 1 mult
3 M3 = A11 * (B12 - B22)            // 1 sub + 1 mult
4 M4 = A22 * (B21 - B11)            // 1 sub + 1 mult
5 M5 = (A11 + A12) * B22            // 1 add + 1 mult
6 M6 = (A21 - A11) * (B11 + B12)    // 1 sub + 1 add + 1 mult
7 M7 = (A12 - A22) * (B21 + B22)    // 1 sub + 1 add + 1 mult
```

Total additions/subtractions: 18 on 2×2 submatrices. Total multiplications: 7 (down from 8 in the naive approach).

Step 2: Reconstruct the four 2×2 output blocks:

```

1 C11 = M1 + M4 - M5 + M7    // 3 ops
2 C12 = M3 + M5              // 1 op
3 C21 = M2 + M4              // 1 op
4 C22 = M1 - M2 + M3 + M6    // 3 ops
```

Total reconstruction additions: 8.

Recursion tree for 8×8 matrices:

```

1 Level 0: 1 call of size 8 -> 7 mults of size 4
2 Level 1: 7 calls of size 4 -> 7*7 = 49 mults of size 2
3 Level 2: 49 calls of size 2 -> 49*7 = 343 mults of size 1 (base case)
4
5 Total 1x1 multiplications: 7^3 = 343
6 Naive would require: 8^3 = 512 multiplications.
7 Speedup ratio: 512/343 = 1.49x at size 8.
```

General speedup at depth k: Naive: 8^k multiplications of $n/2^k$ -sized matrices. Strassen: 7^k . For $n = 2^k$, the ratio is $(8/7)^k = n^{(\log_2 8 - \log_2 7)} = n^{3 - 2.807} = n^{0.193}$.

For 1024×1024 matrices: $k = 10$ levels. Naive: $8^{10} = 1,073,741,824$ scalar multiplications. Strassen: $7^{10} = 282,475,249$. Ratio: 3.8x fewer multiplications. However, the 18 extra additions per level add $O(n^2)$ work, so the net gain depends on the relative cost of multiplication vs addition on the hardware.

16.18 Median of Two Sorted Arrays

Problem: Given two sorted arrays A of size m and B of size n , find the median of the combined sorted array in $O(\log(\min(m, n)))$ time.

Key insight: Rather than merging (which costs $O(m + n)$), we binary-search on the smaller array to find a partition that splits both arrays into equal halves.

Let $A[0 \dots m-1]$ and $B[0 \dots n-1]$ with $m \leq n$. We choose an index i in A (so $0 \leq i \leq m$) and set $j = \lfloor (m + n + 1) / 2 \rfloor - i$ in B . This guarantees the left half has exactly $\lfloor (m + n + 1) / 2 \rfloor$ elements. The partition is valid when:

$$A[i-1] \leq B[j] \quad \text{and} \quad B[j-1] \leq A[i]$$

If $A[i-1] > B[j]$, we move i left (decrease). If $B[j-1] > A[i]$, we move i right (increase). This is a standard binary search on $[0, m]$.

Worked trace. $A = [1, 3, 8, 9, 15]$, $B = [2, 4, 6, 7, 10, 12]$. Here $m = 5$, $n = 6$. Total elements: 11. Left half needs $\lfloor 11/2 \rfloor = 6$ elements.

```

1 Step 1:  lo=0, hi=5.  i=(0+5)/2=2,  j=6-2=4.
2          A[1]=3 <= B[4]=10  OK
3          B[3]=7 <= A[2]=8    OK  -> valid!
4          Left half: {1, 3, 2, 4, 6, 7}  max=7
5          Right half: {8, 9, 15, 10, 12}  min=8
6          Median = (7 + 8) / 2 = 7.5
7
8 Step 2 (alternate input for odd total):
9  A=[1, 3, 8], B=[2, 4, 6, 7, 10].  m=3, n=5, left=4.
10 lo=0, hi=3.  i=1, j=3.
11  A[0]=1 <= B[3]=7  OK
12  B[2]=6 <= A[1]=3  FAIL (6 > 3) -> move i right.
13 lo=2, hi=3.  i=2, j=2.
14  A[1]=3 <= B[2]=6  OK
15  B[1]=4 <= A[2]=8  OK  -> valid!
16  Left half: {1, 3, 2, 4}  max=4
17  Right half: {8, 6, 7, 10}  min=6
18  Since 4+4=8 > 11/2..  Actually total=8 (even), median=(4+6)/2=5.
```

C++ implementation:

```

1 #include <vector>
2 #include <algorithm>
3
4 double findMedianSortedArrays(std::vector<int>& A, std::vector<int>& B) {
5     if (A.size() > B.size())
6         return findMedianSortedArrays(B, A);
7
8     int m = A.size(), n = B.size();
9     int lo = 0, hi = m;
```

```

10     int halfLen = (m + n + 1) / 2;
11
12     while (lo <= hi) {
13         int i = (lo + hi) / 2;
14         int j = halfLen - i;
15
16         int aLeft  = (i > 0) ? A[i - 1] : INT_MIN;
17         int aRight = (i < m) ? A[i]      : INT_MAX;
18         int bLeft  = (j > 0) ? B[j - 1] : INT_MIN;
19         int bRight = (j < n) ? B[j]      : INT_MAX;
20
21         if (aLeft <= bRight && bLeft <= aRight) {
22             if ((m + n) % 2 == 1)
23                 return std::max(aLeft, bLeft);
24             return (std::max(aLeft, bLeft)
25                     + std::min(aRight, bRight)) / 2.0;
26         } else if (aLeft > bRight) {
27             hi = i - 1;
28         } else {
29             lo = i + 1;
30         }
31     }
32     return 0.0;
33 }

```

The sentinel values `INT_MIN` and `INT_MAX` handle the edge cases where i is 0 or m (the corresponding “virtual” element is $-\infty$ or $+\infty$). The loop runs at most $\log_2(m) + 1$ iterations, and each iteration does $O(1)$ work, giving $O(\log(\min(m, n)))$ overall.

16.19 Karatsuba Multiplication

Problem: Multiply two n -digit integers x and y faster than the textbook $O(n^2)$ long multiplication algorithm.

Divide and conquer approach. Split each number into two halves of $d = \lceil n/2 \rceil$ digits:

$$x = x_1 \cdot 10^d + x_0, \quad y = y_1 \cdot 10^d + y_0$$

The product is:

$$x \cdot y = x_1 y_1 \cdot 10^{2d} + (x_1 y_0 + x_0 y_1) \cdot 10^d + x_0 y_0$$

Naively this requires four multiplications of d -digit numbers: $x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$. Karatsuba’s trick computes the middle term with only one extra multiplication:

$$z_2 = x_1 y_1, \quad z_0 = x_0 y_0, \quad z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0$$

Then $x \cdot y = z_2 \cdot 10^{2d} + z_1 \cdot 10^d + z_0$.

Three multiplications instead of four gives the recurrence $T(n) = 3T(n/2) + O(n)$, which solves to $O(n^{\log_2 3}) = O(n^{1.585})$ by the Master Theorem.

Worked trace on 2-digit numbers. Multiply $x = 23$ and $y = 45$ ($d = 1$):

```

1 x = 23 -> x1 = 2, x0 = 3
2 y = 45 -> y1 = 4, y0 = 5
3
4 z2 = x1 * y1          = 2 * 4          = 8
5 z0 = x0 * y0          = 3 * 5          = 15
6 z1 = (x1+x0)*(y1+y0) - z2 - z0
7     = (2+3) * (4+5) - 8 - 15
8     = 5 * 9 - 8 - 15
9     = 45 - 23 = 22
10
11 Result = z2*10^2 + z1*10^1 + z0
12         = 8*100 + 22*10 + 15
13         = 800 + 220 + 15
14         = 1035
15
16 Verify: 23 * 45 = 1035. Correct.
```

Three 1-digit multiplications (8, 15, 45) instead of four (8, 12, 20, 15). The savings compound at deeper recursion levels.

C++ implementation (handles arbitrary-length integers via strings):

```

1 #include <string>
2 #include <algorithm>
3
4 std::string addStrings(const std::string& a, const std::string& b) {
5     std::string result;
6     int carry = 0, i = a.size() - 1, j = b.size() - 1;
7     while (i >= 0 || j >= 0 || carry) {
8         int sum = carry;
9         if (i >= 0) sum += a[i--] - '0';
10        if (j >= 0) sum += b[j--] - '0';
11        result.push_back('0' + sum % 10);
12        carry = sum / 10;
13    }
14    std::reverse(result.begin(), result.end());
15    return result;
16 }
17
18 std::string subtractStrings(const std::string& a, const std::string& b) {
19     std::string result;
20     int borrow = 0, i = a.size() - 1, j = b.size() - 1;
21     while (i >= 0) {
22         int diff = (a[i--] - '0') - borrow;
```

```

23     if (j >= 0) diff -= (b[j--] - '0');
24     if (diff < 0) { diff += 10; borrow = 1; }
25     else          { borrow = 0; }
26     result.push_back('0' + diff);
27 }
28 while (result.size() > 1 && result.back() == '0')
29     result.pop_back();
30 std::reverse(result.begin(), result.end());
31 return result;
32 }
33
34 std::string karatsuba(const std::string& x, const std::string& y) {
35     int n = std::max(x.size(), y.size());
36     if (n <= 9)
37         return std::to_string(std::stoll(x) * std::stoll(y));
38
39     int d = (n + 1) / 2;
40     std::string x1 = x.substr(0, x.size() - d);
41     std::string x0 = x.substr(x.size() - d);
42     std::string y1 = y.substr(0, y.size() - d);
43     std::string y0 = y.substr(y.size() - d);
44
45     std::string z2 = karatsuba(x1, y1);
46     std::string z0 = karatsuba(x0, y0);
47     std::string sum1 = karatsuba(
48         addStrings(x1, x0), addStrings(y1, y0));
49     std::string z1 = subtractStrings(
50         subtractStrings(sum1, z2), z0);
51
52     for (int k = 0; k < 2 * d; k++) z2.push_back('0');
53     for (int k = 0; k < d; k++)    z1.push_back('0');
54
55     return addStrings(addStrings(z2, z1), z0);
56 }

```

The base case switches to native long long multiplication when the numbers fit in a single decimal “digit” (i.e., ≤ 9 , or more practically, $\leq 10^9$ with a suitably chosen base). In production libraries, the base is typically 10^9 or 2^{32} and the recursive splits operate on chunks rather than individual decimal digits, which reduces the overhead of string manipulation and improves cache performance.

Complexity comparison:

1 Algorithm	Recurrence	Exponent	n = 1024 digits
2 -----	-----	-----	-----
3 Schoolbook	$T(n) = 4T(n/2) + O(n)$	2.0	$\sim 1,048,576$ mults
4 Karatsuba	$T(n) = 3T(n/2) + O(n)$	1.585	$\sim 98,000$ mults
5 Toom-Cook 3	$T(n) = 5T(n/3) + O(n)$	1.465	$\sim 50,000$ mults

Karatsuba is the simplest D&C multiplication algorithm that beats schoolbook. For very large num-

bers ($> 10,000$ digits), fast Fourier transform–based methods (Schönhage–Strassen) achieve $O(n \log n \log \log n)$, but Karatsuba remains the practical choice for moderate sizes due to lower constant factors and simpler implementation.

16.20 Exercises

16.20.1 Drill

1. Solve $T(n) = 2T(n/2) + n$, $T(n) = T(n/2) + 1$, and $T(n) = 7T(n/2) + n^2$. Which Master Theorem case applies to each?
1. Trace merge sort on $[3, 1, 4, 1, 5, 9, 2, 6]$. Show the array at each recursion level.
1. Trace quick sort on $[3, 1, 4, 1, 5, 9, 2, 6]$ using the last element as pivot. Show the array after each partition step.
1. In PostgreSQL’s external merge sort, data spills to disk when it doesn’t fit in memory. If the sort requires 10GB of memory but only 1GB is available, how many merge passes are needed? What is the total I/O cost?
1. What is the worst-case input for quick sort? For merge sort? For binary search? In each case, give a real scenario where that worst case arises in production.

16.20.2 Application

1. **PostgreSQL external merge sort.** Implement an external merge sort that sorts a file larger than RAM. Split the input into chunks that fit in memory, sort each chunk (internal sort), write to temporary files, then merge the sorted runs. Test on a synthetic 1GB file with 100MB memory limit. Measure I/O volume and wall time.
1. **C++ `std::sort` introsort.** `std::sort` uses introsort: quick sort with a depth limit, falling back to heap sort when recursion exceeds $2 \cdot \log_2(n)$. Implement introsort and benchmark it against plain quick sort on sorted, reversed, and random arrays of 10^7 elements. Show that introsort avoids the $O(n^2)$ worst case.
1. **Netflix recommendation ranking (quickselect).** Netflix needs to find the top-100 movies from 10,000 candidates for each user. Implement quickselect to find the 100th highest score, then partition around it. Benchmark vs sorting all 10,000. At what k does sorting become cheaper than quickselect?
1. **FFT for audio processing.** Implement the Cooley-Tukey FFT for polynomial multiplication. Measure the time to multiply two degree-1,048,575 polynomials. Compare with naive $O(n^2)$ multiplication. Show the crossover point where FFT becomes faster.

1. **Karatsuba integer multiplication for cryptography.** RSA encryption uses large integers (2048+ bits). Implement Karatsuba multiplication and benchmark against GMP's built-in multiplication for 256-bit, 1024-bit, and 4096-bit integers. At what bit width does Karatsuba beat the $O(n^2)$ algorithm?
1. **Binary search in database index lookup.** MongoDB uses B-tree indexes with binary search within each node. Implement binary search on an array of 10^7 integers and measure cache misses using `perf stat`. Compare with `std::lower_bound`. Why does branchless binary search (using `cmov` instructions) outperform the standard version for large arrays?
1. **Closest pair in GPS trajectory compression.** The Douglas-Peucker algorithm for GPS trajectory simplification uses the closest-pair idea: find the point farthest from the line between endpoints. Implement the divide-and-conquer closest-pair algorithm and apply it to a GPS trace of 100,000 points (from OpenStreetMap data). Show the compressed trajectory at various tolerance levels.

16.20.3 Research

1. **(Strassen's algorithm)** Implement Strassen's algorithm with a cutoff (switch to standard multiplication for $n \leq 64$). Measure the crossover point on various matrix sizes and hardware. Does the crossover change with CPU cache size? Compare with Intel MKL's performance.
1. **(Parallel merge sort)** Implement a parallel merge sort using C++17 parallel algorithms (`std::execution::parallel` and `std::thread`). Measure speedup on 4, 8, and 16 cores. What is the parallel efficiency? Where does Amdahl's law limit the speedup?
1. **(Van Emde Boas layout)** Read about the cache-oblivious van Emde Boas array layout for binary search. Implement binary search on a vEB-laid-out array and compare cache misses with standard binary search. How much does cache-oblivious layout improve performance for very large arrays?
1. **(Sorting lower bound)** Prove the $\Omega(n \log n)$ lower bound for comparison sorting using decision trees. Show that merge sort achieves this bound. Then implement a non-comparison sort (counting sort or radix sort) and show that it beats the lower bound by exploiting the input structure.

16.20.4 Additional Problems

1. **Closest pair: 3D extension.** Extend the divide-and-conquer closest-pair algorithm to 3D points. How does the strip check change? What is the maximum number of neighbors to check per point in 3D? Implement and test on 100,000 random 3D points.
1. **Counting inversions in a stream.** Design a data structure that maintains a stream of insertions and supports the query: "how many inversions exist in the current array?" Hint: use a Fenwick tree (Binary Indexed Tree) with coordinate compression.

1. **Maximum subarray in 2D.** Given an $m \times n$ matrix of integers, find the sub-rectangle with the largest sum. Reduce to 1D maximum subarray using Kadane's algorithm on column prefix sums. What is the time complexity?
1. **Strassen crossover tuning.** Implement Strassen's algorithm with a configurable base-case threshold. Benchmark for thresholds from 8 to 128 on your hardware. At what threshold does Strassen beat naive multiplication? How does this change with matrix size?
1. **Closest pair: presorted y.** Implement the $O(n \log n)$ closest-pair variant that maintains two sorted arrays (by x and by y) through the recursion, eliminating the $O(n \log n)$ strip-sort step at each level. Measure the speedup on 10^7 points compared to the simpler version.
1. **Count inversions using BIT.** Implement counting inversions using a Fenwick tree instead of merge sort. Compare the constant factors with the merge-sort approach on arrays of size 10^6 .
1. **Maximum subarray with indices.** Modify Kadane's algorithm to also return the starting and ending indices of the maximum subarray. Test on: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ and verify the result.
1. **Stock buy-sell with at most k transactions.** Given daily stock prices and an integer k , find the maximum profit with at most k buy-sell transactions (no overlapping). Use divide and conquer with memoization. What is the time complexity for general k ?
1. **Strassen with floating-point error analysis.** Implement Strassen's algorithm for double-precision matrices. Compare the element-wise error against naive multiplication for random 256×256 matrices. How does the error grow with matrix size? At what size does the error become unacceptable for your application?
1. **Inversion count and sorting distance.** Prove that the number of inversions equals the minimum number of adjacent swaps needed to sort an array. Implement bubble sort and count the actual swaps performed; verify that it matches the inversion count from merge sort.

16.21 References and Further Reading

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapters 4 (divide and conquer), 30 (FFT).
- Volker Strassen, "Gaussian Elimination is Not Optimal" — *Numerische Mathematik*, 1969.
- Jon Bentley, *Programming Pearls*, 2nd Edition. Addison-Wesley, 2000.
- Michael I. Shamos and Dan Hoey, "Closest-Point Problems" — *FOCS*, 1975.

- Anatolii A. Karatsuba and Yuri Ofman, "Multiplication of Many-Digital Numbers by Automatic Computers" — Doklady Akademii Nauk SSSR, 1962.
- James W. Cooley and John W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series" — Mathematics of Computation, 1965.

Dynamic Programming

17.1 Introduction

❧ Complete compilable implementations of the data structures in this chapter are in `code/`.

17.2 The General Method

Dynamic programming (DP) solves problems by combining solutions to overlapping subproblems. Unlike divide and conquer (where subproblems are independent), DP handles cases where the same subproblems appear multiple times.

Key insight: If a recursive function calls itself on the same inputs more than once, we can cache the results to avoid redundant computation. This transforms an exponential-time recursion into a polynomial-time algorithm.

17.2.1 Key Concepts

- **Optimal substructure:** an optimal solution contains optimal solutions to subproblems
- **Overlapping subproblems:** the same subproblems are solved repeatedly
- **Memoization:** cache subproblem results (top-down)
- **Tabulation:** build the solution iteratively from smallest subproblems (bottom-up)

17.2.2 Top-Down vs. Bottom-Up

Top-down (memoization) writes the recurrence directly as a recursive function with a cache:

```
1 // Fibonacci: top-down
2 std::unordered_map<int, long long> memo;
3 long long fib(int n) {
4     if (n <= 1) return n;
5     if (memo.count(n)) return memo[n];
6     return memo[n] = fib(n - 1) + fib(n - 2); // cache result
7 }
```

Bottom-up (tabulation) fills a table iteratively:

```
1 // Fibonacci: bottom-up
2 long long fib(int n) {
```

```

3     std::vector<long long> dp(n + 1);
4     dp[0] = 0; dp[1] = 1;
5     for (int i = 2; i <= n; ++i)
6         dp[i] = dp[i - 1] + dp[i - 2];
7     return dp[n];
8 }

```

When to use which:

Top-down	Bottom-up
Easier to write (direct recursion)	Often faster (no recursion overhead)
Computes only needed subproblems	Fills entire table (may waste work)
Stack overflow risk for deep recursion	No stack risk
Cache misses from random access	Sequential access (cache-friendly)

17.2.3 DP Design Steps

1. Characterize the structure of an optimal solution
2. Define the value of an optimal solution recursively
3. Compute the value bottom-up
4. Reconstruct the solution

Worked example: Climbing stairs. You can climb 1 or 2 steps at a time. How many ways to reach step n ?

Step 1: $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$ — you arrived from step $n-1$ or $n-2$. Step 2: Base cases: $\text{ways}(0) = 1$, $\text{ways}(1) = 1$. Step 3: Fill $\text{dp}[0..n]$: $\text{dp}[0]=1$, $\text{dp}[1]=1$, $\text{dp}[2]=2$, $\text{dp}[3]=3$, $\text{dp}[4]=5$, ... Step 4: $\text{dp}[n]$ is the answer (no reconstruction needed).

This is the Fibonacci sequence shifted by one index — many DP problems have this “same recurrence, different interpretation” property.

17.3 The 0/1 Knapsack Problem

Problem: n items, each with weight w_i and value v_i . Fill a knapsack of capacity C to maximize value. Each item is taken or left (no fractions).

17.3.1 Recursive Formulation

Let $V(i, c)$ = maximum value using items $1..i$ with capacity c .

```

1 V(i, c) =
2     V(i-1, c)                                if  $w_{\{i\}} > c$  (cannot take item i)
3     max(V(i-1, c),  $v_{\{i\}} + V(i-1, c-w_{\{i\}})$ ) otherwise

```

```

4 V(0, c) = 0
5 V(i, 0) = 0

```

17.3.2 Bottom-Up Implementation

```

1 int knapsack_01(std::span<const int> weights, std::span<const int> values,
    int capacity) {
2     int n = weights.size();
3     std::vector<std::vector<int>> dp(n + 1, std::vector<int>(capacity + 1,
        0));
4
5     for (int i = 1; i <= n; ++i) {
6         for (int w = 1; w <= capacity; ++w) {
7             if (weights[i - 1] > w) {
8                 dp[i][w] = dp[i - 1][w];
9             } else {
10                dp[i][w] = std::max(dp[i - 1][w],
11                                   values[i - 1] + dp[i - 1][w - weights[
12                   i - 1]]);
13            }
14        }
15    }
16    return dp[n][capacity];
17 }

```

Complexity: $O(n \cdot C)$ time and space. Since C can be large, this is pseudo-polynomial.

17.3.3 Space-Optimized Version

```

1 int knapsack_01_optimized(std::span<const int> weights,
    std::span<const int> values, int capacity) {
2     std::vector<int> dp(capacity + 1, 0);
3     for (size_t i = 0; i < weights.size(); ++i) {
4         for (int w = capacity; w >= weights[i]; --w) {
5             dp[w] = std::max(dp[w], values[i] + dp[w - weights[i]]);
6         }
7     }
8     return dp[capacity];
9 }

```

Space: $O(C)$. Reason: $dp[w] = \max(dp_old[w], v_i + dp_old[w - w_i])$. We iterate w backwards so $dp[w - w_i]$ is still from the previous iteration.

17.3.4 Reconstructing the Solution

```

1 std::vector<size_t> knapsack_items(std::span<const int> weights,

```

```

2                                     std::span<const int> values, int
    capacity) {
3     int n = weights.size();
4     std::vector<std::vector<int>>> dp(n + 1, std::vector<int>(capacity + 1,
        0));
5     // ... fill dp table as above ...
6
7     std::vector<size_t> selected;
8     int w = capacity;
9     for (int i = n; i > 0; --i) {
10         if (dp[i][w] != dp[i - 1][w]) {
11             selected.push_back(i - 1);
12             w -= weights[i - 1];
13         }
14     }
15     return selected;
16 }

```

17.4 Matrix Chain Multiplication

Problem: Given matrices $A_1, A_2, \dots, A_n$ with dimensions $d_0 \times d_1, d_1 \times d_2, \dots, d_{n-1} \times d_n$, find the parenthesization that minimizes the total scalar multiplications.

17.4.1 Recurrence

Let $M[i][j]$ = minimum cost to multiply A_i through A_j .

```

1 M[i][j] = 0                                     if i = j
2         min { M[i][k] + M[k+1][j] + d(i-1) * d(k) * d(j) }
3         i  <=  k < j

```

17.4.2 Implementation

```

1 int matrix_chain_order(std::span<const int> dims) {
2     int n = dims.size() - 1; // number of matrices
3     std::vector<std::vector<int>>> m(n, std::vector<int>(n, 0));
4
5     for (int len = 2; len <= n; ++len) { // chain length
6         for (int i = 0; i <= n - len; ++i) {
7             int j = i + len - 1;
8             m[i][j] = INT_MAX;
9             for (int k = i; k < j; ++k) {
10                 int cost = m[i][k] + m[k + 1][j]
11                     + dims[i] * dims[k + 1] * dims[j + 1];
12                 if (cost < m[i][j]) m[i][j] = cost;
13             }

```

```

14     }
15 }
16 return m[0][n - 1];
17 }

```

Trace for dims = [10, 20, 30, 40, 30] (A:10×20, B:20×30, C:30×40, D:40×30):

```

1 m[0][1] = 10$ \times $20$ \times $30 = 6000           (AB)
2 m[1][2] = 20$ \times $30$ \times $40 = 24000          (BC)
3 m[2][3] = 30$ \times $40$ \times $30 = 36000          (CD)
4
5 m[0][2] = min(
6   m[0][0] + m[1][2] + 10$ \times $20$ \times $40 = 0 + 24000 + 8000 = 32000,
7   m[0][1] + m[2][2] + 10$ \times $30$ \times $40 = 6000 + 0 + 12000 = 18000
8   -> min
9 )
10 m[1][3] = min(
11   m[1][1] + m[2][3] + 20$ \times $30$ \times $30 = 0 + 36000 + 18000 = 54000,
12   m[1][2] + m[3][3] + 20$ \times $40$ \times $30 = 24000 + 0 + 24000 = 48000
13   -> min
14 )
15 m[0][3] = min(
16   k=0: m[0][0] + m[1][3] + 10$ \times $20$ \times $30 = 0 + 48000 + 6000 =
17         54000,
18   k=1: m[0][1] + m[2][3] + 10$ \times $30$ \times $30 = 6000 + 36000 + 9000 =
19         51000 -> min
20   k=2: m[0][2] + m[3][3] + 10$ \times $40$ \times $30 = 18000 + 0 + 12000 =
21         30000 -> ACTUAL MIN
22 )
23
24 Optimal: (A $ \times $ (B $ \times $ (C $ \times $ D))) = 30000 multiplications

```

Complexity: $O(n^3)$ time, $O(n^2)$ space.

17.5 All-Pairs Shortest Paths (Floyd-Warshall)

Problem: Find shortest paths between all pairs of vertices in a weighted directed graph (may have negative edges, but no negative cycles).

From Chapter 12. The DP formulation:

Let $D^{(k)}[i][j]$ = shortest path from i to j using intermediate vertices only from $\{0, 1, \dots, k-1\}$.

```

1 D(k)[i][j] = min(D(k-1)[i][j], D(k-1)[i][k-1] + D(k-1)[k-1][j])

```

```

1 void floyd_warshall(std::vector<std::vector<int>>& dist) {
2     int n = dist.size();
3     for (int k = 0; k < n; ++k)

```

```

4         for (int i = 0; i < n; ++i)
5             for (int j = 0; j < n; ++j)
6                 if (dist[i][k] + dist[k][j] < dist[i][j])
7                     dist[i][j] = dist[i][k] + dist[k][j];
8     }

```

Worked trace: 4 vertices with adjacency matrix (INF = no edge):

```

1 Initial D(0):
2         0      1      2      3
3     0 [  0      3      8  INF ]
4     1 [ INF      0  INF   -4 ]
5     2 [ INF  INF      0      2 ]
6     3 [ INF  INF  INF      0 ]
7
8 k=0 (use vertex 0 as intermediate):
9     D[1][2] = min(INF, D[1][0]+D[0][2]) = min(INF, INF+8) = INF (no
    improvement)
10    D[1][3] = min(-4, D[1][0]+D[0][3]) = -4 (no improvement)
11    No changes -- vertex 0 has no incoming edges.
12
13 k=1 (use vertex 1 as intermediate):
14    D[0][3] = min(INF, D[0][1]+D[1][3]) = min(INF, 3+(-4)) = -1
15    D[2][3] = min(2, D[2][1]+D[1][3]) = min(2, INF+(-4)) = 2
16    Updated: D[0][3] = -1
17
18 k=2 (use vertex 2 as intermediate):
19    D[0][2] = min(8, D[0][2]+D[2][2]) = 8 (no change)
20    D[1][0] = min(INF, D[1][2]+D[2][0]) = INF (no change)
21    No significant changes.
22
23 k=3 (use vertex 3 as intermediate):
24    D[0][2] = min(8, D[0][3]+D[3][2]) = 8 (3 has no outgoing to 2)
25    D[1][0] = min(INF, D[1][3]+D[3][0]) = INF (3 has no outgoing to 0)
26    No changes.
27
28 Final D:
29         0      1      2      3
30     0 [  0      3      8  -1 ]
31     1 [ INF      0  INF   -4 ]
32     2 [ INF  INF      0      2 ]
33     3 [ INF  INF  INF      0 ]
34
35 Shortest path from 0 to 3: 0->1->3, cost -1 (uses negative edge!)

```

Complexity: $O(n^3)$ time, $O(n^2)$ space. Can be reduced to $O(n^2)$ space using a single matrix (overwrite in place). **When to use:** dense graphs, negative edges present, or when you need all-pairs shortest paths. For sparse graphs, running Dijkstra from each vertex is faster: $O(V \cdot (V + E) \log V)$.

17.6 Longest Common Subsequence (LCS)

Problem: Given two strings X and Y, find the longest sequence that appears as a subsequence in both.

17.6.1 Recurrence

```

1 LCS[i][j] = longest common subsequence\index{longest common subsequence}
   of X[0..i-1] and Y[0..j-1]
2
3 LCS[i][j] = 0                                if i=0 or j=0
4             LCS[i-1][j-1] + 1                if X[i-1] = Y[j-1]
5             max(LCS[i-1][j], LCS[i][j-1])    otherwise

```

17.6.2 Implementation

```

1 int lcs_length(std::string_view a, std::string_view b) {
2     int m = a.size(), n = b.size();
3     std::vector<std::vector<int>>> dp(m + 1, std::vector<int>(n + 1, 0));
4
5     for (int i = 1; i <= m; ++i) {
6         for (int j = 1; j <= n; ++j) {
7             if (a[i - 1] == b[j - 1]) {
8                 dp[i][j] = dp[i - 1][j - 1] + 1;
9             } else {
10                dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
11            }
12        }
13    }
14    return dp[m][n];
15 }
16
17 std::string lcs(std::string_view a, std::string_view b) {
18     int m = a.size(), n = b.size();
19     std::vector<std::vector<int>>> dp(m + 1, std::vector<int>(n + 1, 0));
20     // ... fill dp table ...
21
22     // Backtrack
23     std::string result;
24     int i = m, j = n;
25     while (i > 0 && j > 0) {
26         if (a[i - 1] == b[j - 1]) {
27             result.push_back(a[i - 1]);
28             --i; --j;
29         } else if (dp[i - 1][j] >= dp[i][j - 1]) {
30             --i;
31         } else {

```

```

32         --j;
33     }
34 }
35 std::reverse(result.begin(), result.end());
36 return result;
37 }

```

Example: X = "ABCBADAB", Y = "BDCABA" → LCS = "BCBA" or "BDAB" (length 4).

17.7 Edit Distance (Levenshtein Distance)

Problem: Minimum number of insertions, deletions, and substitutions to convert string A into string B.

17.7.1 Recurrence

```

1 ED[i][j] = edit distance\index{edit distance} between A[0..i-1] and B[0..j
  -1]
2
3 ED[i][j] = i                                if j = 0
4           j                                if i = 0
5           ED[i-1][j-1]                    if A[i-1] = B[j-1]
6           min(ED[i-1][j] + 1,             // delete
7               ED[i][j-1] + 1,             // insert
8               ED[i-1][j-1] + 1)           // substitute

```

17.7.2 Implementation

```

1 int edit_distance(std::string_view a, std::string_view b) {
2     int m = a.size(), n = b.size();
3     std::vector<std::vector<int>>> dp(m + 1, std::vector<int>(n + 1));
4
5     for (int i = 0; i <= m; ++i) dp[i][0] = i;
6     for (int j = 0; j <= n; ++j) dp[0][j] = j;
7
8     for (int i = 1; i <= m; ++i) {
9         for (int j = 1; j <= n; ++j) {
10             if (a[i - 1] == b[j - 1]) {
11                 dp[i][j] = dp[i - 1][j - 1];
12             } else {
13                 dp[i][j] = std::min({dp[i - 1][j] + 1,           // delete
14                                     dp[i][j - 1] + 1,           // insert
15                                     dp[i - 1][j - 1] + 1}); // substitute
16             }
17         }
18     }
19 }

```



```

19     return dp[m][n];
20 }

```

Application: Spell checking, DNA sequence alignment, diff tools.

17.8 Coin Change (Unbounded)

Problem: Given coin denominations (unlimited supply), find the minimum number of coins to make a target amount.

17.8.1 Recurrence

```

1 dp[c] = min coins needed to make amount c
2 dp[c] = min over all coins <= c of (dp[c - coin] + 1)
3 dp[0] = 0

```

```

1 int coin_change(std::span<const int> coins, int amount) {
2     std::vector<int> dp(amount + 1, INT_MAX / 2);
3     dp[0] = 0;
4
5     for (int c = 1; c <= amount; ++c) {
6         for (int coin : coins) {
7             if (coin <= c) {
8                 dp[c] = std::min(dp[c], dp[c - coin] + 1);
9             }
10        }
11    }
12    return dp[amount] == INT_MAX / 2 ? -1 : dp[amount];
13 }

```

Worked trace: coins = [1, 3, 4], amount = 6.

```

1 dp[0] = 0
2 dp[1] = dp[1-1] + 1 = 1
3 dp[2] = dp[2-1] + 1 = 2
4 dp[3] = min(dp[3-1]+1, dp[3-3]+1) = min(3, 1) = 1
5 dp[4] = min(dp[4-1]+1, dp[4-3]+1, dp[4-4]+1) = min(2, 2, 1) = 1
6 dp[5] = min(dp[5-1]+1, dp[5-3]+1, dp[5-4]+1) = min(2, 2, 2) = 2
7 dp[6] = min(dp[6-1]+1, dp[6-3]+1, dp[6-4]+1) = min(3, 2, 2) = 2
8
9 Answer: 2 coins (3+3 or 4+1+1? No: 3+3 = 2 coins!)

```

Why greedy fails here: Greedy by largest coin: 4 + 1 + 1 = 3 coins. But 3 + 3 = 2 coins is better. The denominations [1, 3, 4] don't have the "canonical" property that US coins have (25, 10, 5, 1), where greedy is always optimal.

Variant: count the number of ways:

```

1 int count_ways(std::span<const int> coins, int amount) {
2     std::vector<int> dp(amount + 1, 0);
3     dp[0] = 1;
4     for (int coin : coins)
5         for (int c = coin; c <= amount; ++c)
6             dp[c] += dp[c - coin];
7     return dp[amount];
8 }

```

For coins [1, 3, 4] and amount = 6: $dp[6] = 3$ ways (1+1+1+1+1+1, 3+3, 3+1+1+1).

Complexity: $O(\text{amount} \times \text{coins})$ time, $O(\text{amount})$ space. This is pseudo-polynomial — polynomial in the value of the input, not its bit length. The problem is NP-hard in general (subset sum reduces to it).

17.9 DP vs. Greedy vs. Divide and Conquer

Criterion	Greedy	Divide & Conquer	Dynamic Programming
Subproblems	One	Independent	Overlapping
Decision	Locally optimal	Combine solutions	Table of all options
Correctness	Greedy choice property	Induction	Optimal substructure
Example	Dijkstra, MST	Merge sort, binary search	Knapsack, LCS, Floyd-Warshall

When to use each:

- **Greedy:** when the problem has optimal substructure AND the greedy choice property — i.e., making the locally best choice always leads to a globally optimal solution. Fastest (usually $O(n \log n)$).
- **DP:** when subproblems overlap and optimal substructure holds, but the greedy choice property does not. Slower (polynomial) but guaranteed correct.
- **Divide & Conquer:** when subproblems are independent and can be solved separately. The combine step is often the bottleneck. Natural fit for parallelism.

Case study: shortest paths

- Dijkstra (greedy): non-negative weights, $O((V+E) \log V)$
- Bellman-Ford (DP): handles negative weights, $O(V \cdot E)$
- Floyd-Warshall (DP): all-pairs, $O(V^3)$
- Divide & Conquer doesn't naturally apply here — shortest paths have overlapping subproblems.

Case study: knapsack

- Fractional knapsack (greedy): $O(n \log n)$ by ratio — optimal
- 0/1 knapsack (DP): $O(n \cdot C)$ — greedy fails
- Unbounded knapsack (DP): $O(n \cdot C)$ — greedy fails

17.10 Common Bugs and Pitfalls

- **Infinite loops in recurrence** — a recurrence that depends on itself without a base case leads to infinite recursion. Always verify the base case terminates and that subproblems strictly decrease the state space.
- **Off-by-one in table dimensions** — DP tables indexed by string length or array position must handle the empty-prefix case at index 0. Forgetting this causes out-of-bounds access or incorrect base values.
- **Integer overflow in DP values** — filling a knapsack DP with `INT_MAX` and then adding 1 overflows to negative. Use `INT_MAX / 2` as "infinity" and check before adding.
- **Assuming DP is always faster** — for small problem sizes, recursion or brute force can be faster due to DP's overhead from table construction. Profile before optimizing.
- **Forgetting to reconstruct** — the DP table gives the optimal *value*, not the solution itself. Always store the decision in a parallel table if reconstruction is needed (e.g., which items to pack).
- **2D tables for problems that need only the previous row** — many DP problems (01 knapsack, edit distance) only need the previous row. Allocating $O(n^2)$ space when $O(n)$ suffices is wasteful and limits scalability.

17.11 Summary

1. **Dynamic programming** solves problems with overlapping subproblems and optimal substructure.
2. **Top-down (memoization)** is often easier to code; **bottom-up (tabulation)** is often more efficient.
3. **Knapsack, matrix chain, LCS, edit distance, and Floyd-Warshall** are canonical DP problems.
4. Industrial applications include **Google's PageRank** (DP on graphs), **spell checkers** (edit distance), **DNA sequence alignment** (Smith-Waterman), **speech recognition** (Viterbi), **robotics path planning**, and **supply chain optimization**.
5. The **key skill** is formulating the recurrence — recognizing the state and the transition.

17.12 Interview FAQs

1. What’s the real diagnostic for “this problem needs DP” — beyond “overlapping subproblems”?

The practical test: (1) Write the naive recursive solution. (2) If it recomputes the same subproblems more than a few times, DP will help. (3) If the number of *distinct* subproblems is polynomial (even though the recursion tree is exponential), memoization gives you polynomial time. The deeper insight: DP works when the problem has *optimal substructure* AND the subproblem graph has no cycles (otherwise you need techniques like Bellman-Ford for shortest paths with negative edges). A common trap: problems with optimal substructure but no overlapping subproblems (like greedy problems) don’t benefit from DP — they need greedy or other techniques.

1. When does bottom-up tabulation actually outperform top-down memoization — and vice versa?

Tabulation wins when: (1) you need $O(1)$ stack space (memoization uses $O(\text{depth})$ stack frames), (2) the subproblem dependency graph is dense (tabulation avoids hash table overhead), (3) you want cache-friendly access patterns (iterating arrays is faster than hash lookups). Memoization wins when: (1) only a fraction of subproblems are needed (lazy evaluation), (2) the recurrence is naturally recursive (tree DP, DP on DAGs), (3) you want to avoid figuring out the iteration order. In practice, the difference is often 2–3x — pick whichever makes the code clearer.

1. How do you optimize DP space from $O(n \cdot m)$ to $O(m)$?

When the recurrence for $dp[i][j]$ only depends on $dp[i-1][\dots]$, you only need the previous row. Use a single 1D array and update it in the right direction. For 0/1 knapsack, iterate capacity backwards; for unbounded knapsack, iterate forwards. For LCS, use two rows. This reduces space from $O(n \cdot m)$ to $O(\min(n, m))$.

1. Explain the time complexity of the 0/1 knapsack DP.

The table has $n+1$ rows and $C+1$ columns, where n is the number of items and C is the capacity. Each cell takes $O(1)$ time. Total: $O(n \cdot C)$. This is **pseudo-polynomial** — polynomial in the numeric value of C , not in the number of bits to represent C . Since C can be exponential in its bit length, the problem is actually NP-hard.

1. How do you convert a DP recurrence into an efficient implementation — the practical workflow?

Step 1: Write the recurrence relation (e.g., $dp[i][j] = \min(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$). Step 2: Identify the state space (what are i and j ?). Step 3: Determine the iteration order (which states depend on which?). Step 4: Optimize space (can you reduce 2D to 1D?). The key insight: the iteration order IS the dependency order — if $dp[i]$ depends on $dp[i-1]$, iterate forward. If it

depends on $dp[i+1]$, iterate backward. Getting this wrong gives wrong answers, not just slow code.

1. When does the coin change greedy fail? Give a concrete example.

Denominations $[1, 3, 4]$, amount = 6. Greedy (largest first): $4+1+1 = 3$ coins. Optimal: $3+3 = 2$ coins. Greedy fails because 4 is not a multiple of 3, leaving an awkward remainder. US coins (1, 5, 10, 25) are canonical — greedy is always optimal. Most other currency systems are not.

1. Why is edit distance $O(m \cdot n)$ and not $O(3^{\max(m,n)})$, and what are the practical speedups?

Naive recursion tries all three operations (insert/delete/substitute) at each position — $O(3^{\max(m,n)})$ because each choice branches into three. DP avoids recomputation by storing results for all (i, j) pairs — $O(m \cdot n)$ states, $O(1)$ per state. Practical speedups: (1) Ukkonen's optimization — if you only care about distance $\leq k$, stop when all values in a row exceed k , giving $O(k \cdot \min(m, n))$; (2) Myers' bit-parallel algorithm — packs the DP row into machine words, processing 64 cells per operation, giving $O(mn/w)$ where $w = 64$; (3) For long strings with small differences, banded DP restricts the computation to a diagonal band of width $2k + 1$.

1. What's the most common mistake when reconstructing DP solutions, and how do you avoid it?

The most common mistake: overwriting the decision table during the DP computation. When you optimize space from $O(n \cdot m)$ to $O(m)$ using two rows, you lose the decision information needed for reconstruction. Fix: either (1) keep the full decision table separately (costs $O(n \cdot m)$ space but you need it), (2) recompute the DP forward while tracing backward ($O(n \cdot m)$ time, $O(m)$ space), or (3) use Hirschberg's algorithm for LCS which gives $O(m+n)$ space AND the actual subsequence. The lesson: space optimization and solution reconstruction are often in tension — plan for both from the start.

1. Why does LCS use a 2D DP table when the problem seems 1D (two strings)?

The state needs to capture progress on BOTH strings simultaneously — $dp[i][j] = \text{LCS of prefix } A[0..i] \text{ and } B[0..j]$. If you only track one index, you lose information about where you are in the other string. This is the general principle: the DP state dimension equals the number of independent “progress counters” in the problem. For LCS, you advance through A and B independently $\rightarrow$ 2D. For edit distance, same reasoning. For knapsack with n items and capacity W , you track both item index and remaining capacity $\rightarrow$ 2D. The state space IS the problem's degrees of freedom.

1. What is the Held-Karp algorithm for TSP?

Held-Karp uses DP over subsets: $dp[S][i] = \text{minimum cost to visit all cities in set } S, \text{ ending at city } i$. There are 2^n subsets and n ending cities, giving $O(n^2 \cdot 2^n)$ time and $O(n \cdot 2^n)$ space. It beats brute force $O(n!)$ by a factor of n , but is only feasible for $n \leq 20-25$. For larger instances, approximation algorithms or heuristics are needed.

1. What is the concrete methodology for recognizing when a greedy approach fails and DP is needed?

Three diagnostic tests: (1) Try the exchange argument — if you can't prove the greedy choice is always safe, greedy fails. (2) Build a counterexample — try small inputs where the greedy choice leaves a “bad remainder” (e.g., coin change [1,3,4] amount=6: greedy picks 4, leaves 2 which needs 2 more coins). (3) Check if the problem has *state* that greedy discards — if the optimal decision depends on information from previous choices (like remaining capacity in knapsack), you need DP to track that state. The concrete sign: if your greedy algorithm makes a choice and then can't “undo” it when a better option appears later, you need DP.

17.13 Longest Common Subsequence (LCS) — Detailed Treatment

Problem: Given two sequences $X[0..m-1]$ and $Y[0..n-1]$, find the longest subsequence common to both. A subsequence is obtained by deleting zero or more elements without changing the order of the remaining elements.

17.13.1 DP Formulation

Let $dp[i][j]$ = length of the LCS of $X[0..i-1]$ and $Y[0..j-1]$.

Base cases: $dp[0][j] = 0$ for all j (empty X); $dp[i][0] = 0$ for all i (empty Y).

Recurrence:

```

1 If  $X[i-1] == Y[j-1]$ :
2      $dp[i][j] = dp[i-1][j-1] + 1$       (extend the match)
3 Else:
4      $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$  (skip from X or Y)
    
```

17.13.2 Table Construction: $X = \text{”ABCBDAB”}$, $Y = \text{”BDCAB”}$

Build an 8 x 6 table ($m+1$ rows, $n+1$ columns):

1			B	D	C	A	B
2	" "	[0	0	0	0	0	0]
3	A	[0	0	0	0	1	1]
4	B	[0	1	1	1	1	2]
5	C	[0	1	1	2	2	2]
6	B	[0	1	1	2	2	3]
7	D	[0	1	2	2	2	3]
8	A	[0	1	2	2	3	3]
9	B	[0	1	2	2	3	4]
10							
11	LCS length = dp[7][5] = 4						

Step-by-step fill of key cells:

```

1 dp[1][1]: X[0]='A' != Y[0]='B' -> max(dp[0][1], dp[1][0]) = max(0, 0) = 0
2 dp[1][5]: X[0]='A' != Y[4]='B' -> max(dp[0][5], dp[1][4]) = max(0, 1) = 1
3 dp[2][1]: X[1]='B' == Y[0]='B' -> dp[1][0]+1 = 0+1 = 1
4 dp[2][5]: X[1]='B' == Y[4]='B' -> dp[1][4]+1 = 1+1 = 2
5 dp[3][3]: X[2]='C' == Y[2]='C' -> dp[2][2]+1 = 1+1 = 2
6 dp[4][5]: X[3]='B' == Y[4]='B' -> dp[3][4]+1 = 2+1 = 3
7 dp[5][2]: X[4]='D' == Y[1]='D' -> dp[4][1]+1 = 1+1 = 2
8 dp[7][5]: X[6]='B' == Y[4]='B' -> dp[6][4]+1 = 3+1 = 4
    
```

17.13.3 Backtracking to Find the Actual Subsequence

Trace back from $dp[m][n]$ to reconstruct the LCS:

```

1 Start at (i=7, j=5):
2   X[6]='B' == Y[4]='B' -> match! Record 'B', go to (6,4)
3   X[5]='A' == Y[3]='A' -> match! Record 'A', go to (5,3)
4   X[4]='D' != Y[2]='C'. dp[3][3]=2 >= dp[4][2]=2? Yes -> go up to (3,3)
5   X[2]='C' == Y[2]='C' -> match! Record 'C', go to (2,2)
6   X[1]='B' != Y[1]='D'. dp[1][2]=1 >= dp[2][1]=1? Yes -> go up to (1,2)
7   X[0]='A' != Y[1]='D'. dp[0][2]=0 < dp[1][1]=0? No -> go up to (0,2)
8   i=0, stop.
9
10 Result (reversed): B, A, C, B -> "BCAB" (length 4)
    
```

Note: there may be multiple valid LCS strings of the same length. Backtracking choices at ties determine which one is found.

17.13.4 C++ Implementation

```

1 #include <string>
2 #include <vector>
3 #include <algorithm>
4
5 std::string lcs_backtrack(std::string_view x, std::string_view y) {
6     int m = x.size(), n = y.size();
7     std::vector<std::vector<int>>> dp(m + 1, std::vector<int>(n + 1, 0));
8
9     for (int i = 1; i <= m; ++i) {
10         for (int j = 1; j <= n; ++j) {
11             if (x[i - 1] == y[j - 1]) {
12                 dp[i][j] = dp[i - 1][j - 1] + 1;
13             } else {
14                 dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
15             }
16         }
17     }
18 }
    
```

```

19 // Backtrack to recover the actual subsequence
20 std::string result;
21 int i = m, j = n;
22 while (i > 0 && j > 0) {
23     if (x[i - 1] == y[j - 1]) {
24         result.push_back(x[i - 1]);
25         --i; --j;
26     } else if (dp[i - 1][j] >= dp[i][j - 1]) {
27         --i;
28     } else {
29         --j;
30     }
31 }
32 std::reverse(result.begin(), result.end());
33 return result;
34 }

```

Complexity: $O(m \cdot n)$ time and space. Space can be reduced to $O(\min(m, n))$ when only the length is needed, but backtracking requires the full table.

Applications: diff tools (git diff), plagiarism detection, version control, bioinformatics (DNA/protein sequence alignment).

17.14 Edit Distance (Levenshtein Distance) — Detailed Treatment

Problem: Given two strings A and B, find the minimum number of insertions, deletions, and substitutions to convert A into B. Each operation costs 1.

17.14.1 DP Formulation

Let $dp[i][j]$ = minimum edit distance between $A[0..i-1]$ and $B[0..j-1]$.

Base cases: $dp[i][0] = i$ (delete all i characters from A); $dp[0][j] = j$ (insert all j characters of B).

Recurrence:

```

1 If A[i-1] == B[j-1]:
2     dp[i][j] = dp[i-1][j-1]           (characters match, no operation)
3 Else:
4     dp[i][j] = 1 + min(
5         dp[i-1][j],           (delete from A)
6         dp[i][j-1],           (insert into A)
7         dp[i-1][j-1]         (substitute in A)
8     )

```

17.14.2 Table Construction: A = "kitten", B = "sitting"

Build a 7 x 8 table ($m+1 = 7$ rows, $n+1 = 8$ columns):


```

1      "" s i t t i n g
2      "" [ 0 1 2 3 4 5 6 7 ]
3      k [ 1 1 2 3 4 5 6 7 ]
4      i [ 2 2 1 2 3 4 5 6 ]
5      t [ 3 3 2 1 2 3 4 5 ]
6      t [ 4 4 3 2 1 2 3 4 ]
7      e [ 5 5 4 3 2 2 3 4 ]
8      n [ 6 6 5 4 3 3 2 3 ]
9
10 Edit distance = dp[6][7] = 3

```

Step-by-step fill of key cells:

```

1 dp[1][1]: 'k' != 's' -> 1 + min(dp[0][1], dp[1][0], dp[0][0])
2               = 1 + min(1, 1, 0) = 1
3 dp[2][2]: 'i' == 'i' -> dp[1][1] = 1
4 dp[3][3]: 't' == 't' -> dp[2][2] = 1
5 dp[4][4]: 't' == 't' -> dp[3][3] = 1
6 dp[5][5]: 'e' != 'i' -> 1 + min(dp[4][5], dp[5][4], dp[4][4])
7               = 1 + min(3, 2, 1) = 2
8 dp[6][6]: 'n' == 'n' -> dp[5][5] = 2
9 dp[6][7]: 'n' != 'g' -> 1 + min(dp[5][7], dp[6][6], dp[5][6])
10              = 1 + min(4, 2, 3) = 3

```

The three operations are: substitute k-¿s, substitute e-¿i, insert g at end.

17.14.3 C++ Implementation

```

1 #include <string>
2 #include <vector>
3 #include <algorithm>
4
5 int edit_distance(std::string_view a, std::string_view b) {
6     int m = a.size(), n = b.size();
7     std::vector<std::vector<int>> dp(m + 1, std::vector<int>(n + 1));
8
9     for (int i = 0; i <= m; ++i) dp[i][0] = i;
10    for (int j = 0; j <= n; ++j) dp[0][j] = j;
11
12    for (int i = 1; i <= m; ++i) {
13        for (int j = 1; j <= n; ++j) {
14            if (a[i - 1] == b[j - 1]) {
15                dp[i][j] = dp[i - 1][j - 1];
16            } else {
17                dp[i][j] = 1 + std::min({dp[i - 1][j],           // delete
18                                         dp[i][j - 1],           // insert
19                                         dp[i - 1][j - 1]}); //
20                // substitute
21            }
22        }
23    }
24 }

```

```

21     }
22 }
23 return dp[m][n];
24 }

```

17.14.4 Backtracking to Find the Operations

```

1 std::string edit_ops(std::string_view a, std::string_view b) {
2     int m = a.size(), n = b.size();
3     std::vector<std::vector<int>> dp(m + 1, std::vector<int>(n + 1));
4     // ... fill dp table as above ...
5
6     std::string ops;
7     int i = m, j = n;
8     while (i > 0 || j > 0) {
9         if (i > 0 && j > 0 && a[i-1] == b[j-1]) {
10             --i; --j; // match, no operation
11         } else if (i > 0 && j > 0 && dp[i][j] == dp[i-1][j-1] + 1) {
12             ops += "SUB " + std::string(1, a[i-1]) + "->" + std::string(1,
13             b[j-1]) + "\n";
14             --i; --j;
15         } else if (i > 0 && dp[i][j] == dp[i-1][j] + 1) {
16             ops += "DEL " + std::string(1, a[i-1]) + "\n";
17             --i;
18         } else {
19             ops += "INS " + std::string(1, b[j-1]) + "\n";
20             --j;
21         }
22     }
23     std::reverse(ops.begin(), ops.end());
24     return ops;
25 }

```

Complexity: $O(m \cdot n)$ time and space. Can be reduced to $O(\min(m,n))$ space using two rows.

Applications: spell checking (suggest closest word), DNA sequence alignment, plagiarism detection, diff tools, speech recognition (aligning phoneme sequences), fuzzy string matching in search engines.

17.15 Matrix Chain Multiplication — Detailed Treatment

Problem: Given n matrices $A_1, A_2, \dots, A_n$ where A_i has dimensions $p[i-1] \times p[i]$, find the parenthesization that minimizes the total number of scalar multiplications.

17.15.1 DP Formulation

Let $m[i][j]$ = minimum cost to multiply matrices A_i through A_j (1-indexed).

Base case: $m[i][i] = 0$ (single matrix, no multiplication needed).

Recurrence:

```
1 m[i][j] = min over k in [i, j-1] of:
2   m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]
```

The term $p[i-1] * p[k] * p[j]$ is the cost of multiplying the result of $(A_i..A_k)$ with the result of $(A_{k+1}..A_j)$.

17.15.2 Worked Trace: 4 Matrices

Dimensions: $A_1 = 10 \times 20$, $A_2 = 20 \times 30$, $A_3 = 30 \times 40$, $A_4 = 40 \times 30$. Dimension array: $p = [10, 20, 30, 40, 30]$.

```
1 Step 1: Fill diagonal (chain length = 1, single matrices)
2   m[1][1] = 0, m[2][2] = 0, m[3][3] = 0, m[4][4] = 0
3
4 Step 2: Chain length = 2 (two consecutive matrices)
5   m[1][2]: k=1: 0 + 0 + 10*20*30 = 6000
6   m[2][3]: k=2: 0 + 0 + 20*30*40 = 24000
7   m[3][4]: k=3: 0 + 0 + 30*40*30 = 36000
8
9 Step 3: Chain length = 3
10  m[1][3]:
11    k=1: m[1][1]+m[2][3]+10*20*40 = 0+24000+8000 = 32000
12    k=2: m[1][2]+m[3][3]+10*30*40 = 6000+0+12000 = 18000 (min)
13  m[2][4]:
14    k=2: m[2][2]+m[3][4]+20*30*30 = 0+36000+18000 = 54000
15    k=3: m[2][3]+m[4][4]+20*40*30 = 24000+0+24000 = 48000 (min)
16
17 Step 4: Chain length = 4
18  m[1][4]:
19    k=1: m[1][1]+m[2][4]+10*20*30 = 0+48000+6000 = 54000
20    k=2: m[1][2]+m[3][4]+10*30*30 = 6000+36000+9000 = 51000
21    k=3: m[1][3]+m[4][4]+10*40*30 = 18000+0+12000 = 30000 (min)
22
23 Cost table m[i][j]:
24      j=1    j=2    j=3    j=4
25 i=1 [  0    6000  18000 30000 ]
26 i=2 [  -      0   24000 48000 ]
27 i=3 [  -      -      0  36000 ]
28 i=4 [  -      -      -      0 ]
29
30 Optimal cost = 30000
31 Optimal parenthesization: ((A1 x A2) x (A3 x A4))
```

17.15.3 C++ Implementation

```

1  #include <vector>
2  #include <span>
3  #include <climits>
4
5  int matrix_chain_order(std::span<const int> p) {
6      int n = p.size() - 1;
7      std::vector<std::vector<int>> m(n, std::vector<int>(n, 0));
8
9      for (int len = 2; len <= n; ++len) {
10         for (int i = 0; i <= n - len; ++i) {
11             int j = i + len - 1;
12             m[i][j] = INT_MAX;
13             for (int k = i; k < j; ++k) {
14                 int cost = m[i][k] + m[k + 1][j]
15                     + p[i] * p[k + 1] * p[j + 1];
16                 if (cost < m[i][j]) m[i][j] = cost;
17             }
18         }
19     }
20     return m[0][n - 1];
21 }

```

17.15.4 Printing the Optimal Parenthesization

```

1  #include <vector>
2  #include <span>
3  #include <string>
4
5  std::string matrix_chain_parens(std::span<const int> p) {
6      int n = p.size() - 1;
7      std::vector<std::vector<int>> m(n, std::vector<int>(n, 0));
8      std::vector<std::vector<int>> s(n, std::vector<int>(n, 0));
9
10     for (int len = 2; len <= n; ++len) {
11         for (int i = 0; i <= n - len; ++i) {
12             int j = i + len - 1;
13             m[i][j] = INT_MAX;
14             for (int k = i; k < j; ++k) {
15                 int cost = m[i][k] + m[k + 1][j]
16                     + p[i] * p[k + 1] * p[j + 1];
17                 if (cost < m[i][j]) {
18                     m[i][j] = cost;
19                     s[i][j] = k;
20                 }
21             }
22         }
23     }
24 }

```

```

22     }
23 }
24
25 // Recursive helper to build the string
26 auto build = [&](auto&& self, int i, int j) -> std::string {
27     if (i == j) return "A" + std::to_string(i + 1);
28     int k = s[i][j];
29     return "(" + self(self, i, k) + " x "
30         + self(self, k + 1, j) + ")";
31 };
32 return build(build, 0, n - 1);
33 }

```

Complexity: $O(n^3)$ time, $O(n^2)$ space. The DP considers $O(n^2)$ subproblems, each taking $O(n)$ time to compute.

17.16 Subset Sum Problem

Problem: Given a set of non-negative integers and a target sum T , determine if any subset sums to exactly T .

17.16.1 DP Formulation

Let $dp[i][w]$ = true if a subset of the first i integers can sum to exactly w .

Base case: $dp[0][0] = \text{true}$ (empty subset sums to 0); $dp[0][w] = \text{false}$ for $w \neq 0$.

Recurrence:

```

1 dp[i][w] = dp[i-1][w]                if nums[i-1] > w (cannot take)
2       dp[i-1][w] || dp[i-1][w - nums[i-1]] otherwise

```

17.16.2 Worked Trace: $\text{nums} = [3, 7, 1, 8, 4]$, $T = 11$

		w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7	w=8	w=9	w=10
1	w=11											
2	i=0 [] [T F]		F	F	F	F	F	F	F	F	F	F
3	i=1 3 [T F]		F	F	T	F	F	F	F	F	F	F
4	i=2 7 [T T]		F	F	T	F	F	F	T	F	F	F
5	i=3 1 [T T]		T	F	T	F	F	F	T	F	F	T
6	i=4 8 [T T]		T	F	T	F	F	F	T	T	T	T
7	i=5 4 [T T]		T	F	T	T	T	F	T	T	T	T

```

8
9 dp[5][11] = true -> subset exists: {3, 8} or {7, 4} or {3, 7, 1}

```

To reconstruct the subset, trace back from `dp[n][T]`:

```

1 i=5, w=11: dp[5][11]=T, dp[4][11]=T -> skip nums[4]=4
2 i=4, w=11: dp[4][11]=T, dp[3][11]=T -> skip nums[3]=8
3 i=3, w=11: dp[3][11]=T, dp[2][11]=T -> skip nums[2]=1
4 i=2, w=11: dp[2][11]=T, dp[1][11]=F -> take nums[1]=7, w=11-7=4
5 i=1, w=4: dp[1][4]=F -> take nums[0]=3, w=4-3=1
6 i=0, w=1: base case
7
8 Subset: {3, 7} -> 3+7 = 10 ... wait, recheck.

```

Recheck: with `nums = [3, 7, 1, 8, 4]` and `T = 11`, the valid subsets are `3, 8 = 11` and `7, 4 = 11` and `3, 7, 1 = 11`.

17.16.3 C++ Implementation

```

1 #include <vector>
2 #include <span>
3
4 bool subset_sum(std::span<const int> nums, int target) {
5     int n = nums.size();
6     std::vector<std::vector<bool>> dp(n + 1, std::vector<bool>(target + 1,
7         false));
8     dp[0][0] = true;
9
10    for (int i = 1; i <= n; ++i) {
11        for (int w = 0; w <= target; ++w) {
12            dp[i][w] = dp[i - 1][w]; // skip nums[i-1]
13            if (w >= nums[i - 1]) {
14                dp[i][w] = dp[i][w] || dp[i - 1][w - nums[i - 1]];
15            }
16        }
17    }
18    return dp[n][target];
19 }

```

17.16.4 Space-Optimized Version

Since each row depends only on the previous row, use a single 1D array:

```

1 bool subset_sum_optimized(std::span<const int> nums, int target) {
2     std::vector<bool> dp(target + 1, false);
3     dp[0] = true;
4
5     for (int num : nums) {

```

```

6         for (int w = target; w >= num; --w) {
7             dp[w] = dp[w] || dp[w - num];
8         }
9     }
10    return dp[target];
11 }

```

Note: iterate backwards (right to left) so each number is used at most once (0/1 property). If you iterate forwards, you allow unlimited reuse (unbounded variant).

Complexity: $O(n \cdot T)$ time and space (or $O(T)$ space optimized). Pseudo-polynomial since T can be exponential in the number of bits used to represent it. The subset sum problem is NP-complete in general.

Applications: partition problems (can the set be split into two equal halves?), cryptography (knapsack-based cryptosystems), resource allocation, load balancing.

17.17 House Robber

Problem: Given a row of houses, each with a value, find the maximum total value you can rob without robbing two adjacent houses. This is a classic 1D DP problem that appears frequently in interviews.

17.17.1 Recurrence

Let $dp[i]$ = maximum value robbing houses 0 through i . For house i , you have two choices: skip it ($dp[i-1]$) or rob it ($dp[i-2] + \text{value}[i]$, since you cannot rob the previous house).

```

1 dp[i] = max(dp[i-1], dp[i-2] + value[i])
2 dp[0] = value[0]
3 dp[1] = max(value[0], value[1])

```

17.17.2 Tabulation

```

1 int rob(std::span<const int> values) {
2     int n = values.size();
3     if (n == 0) return 0;
4     if (n == 1) return values[0];
5     std::vector<int> dp(n);
6     dp[0] = values[0];
7     dp[1] = std::max(values[0], values[1]);
8     for (int i = 2; i < n; ++i)
9         dp[i] = std::max(dp[i - 1], dp[i - 2] + values[i]);
10    return dp[n - 1];
11 }

```

17.17.3 Space-Optimized Version

Since $dp[i]$ depends only on the two previous values, we can use two variables instead of an array:

```
1 int rob_optimized(std::span<const int> values) {
2     int prev2 = 0, prev1 = 0;
3     for (int v : values) {
4         int curr = std::max(prev1, prev2 + v);
5         prev2 = prev1;
6         prev1 = curr;
7     }
8     return prev1;
9 }
```

Worked trace: values = [2, 7, 9, 3, 1].

```
1 i=0: prev2=0, prev1=0 -> curr = max(0, 0+2) = 2      -> prev2=0, prev1=2
2 i=1: prev2=2, prev1=2 -> curr = max(2, 0+7) = 7      -> prev2=2, prev1=7
3 i=2: prev2=7, prev1=7 -> curr = max(7, 2+9) = 11     -> prev2=7, prev1=11
4 i=3: prev2=11, prev1=11 -> curr = max(11, 7+3) = 11  -> prev2=11, prev1
    =11
5 i=4: prev2=11, prev1=11 -> curr = max(11, 11+1) = 12 -> prev2=11, prev1
    =12
6
7 Answer: 12 (rob houses 0, 2, 4 = 2 + 9 + 1 = 12)
```

Complexity: $O(n)$ time, $O(1)$ space. The recurrence is linear because each state depends on exactly two previous states.

Variant: circular houses. If houses are arranged in a circle (house 0 and house $n-1$ are adjacent), solve twice: once excluding the last house, once excluding the first. Return the maximum. This is the “House Robber II” variant.

17.18 Longest Increasing Subsequence (LIS)

Problem: Given an array of integers, find the length of the longest strictly increasing subsequence. A subsequence is obtained by deleting elements without changing the order of the rest.

17.18.1 $O(n^2)$ DP

Let $dp[i]$ = length of LIS ending at index i . For each i , check all $j < i$: if $nums[j] < nums[i]$, then $dp[i] = \max(dp[i], dp[j] + 1)$.

```
1 int lis_dp(std::span<const int> nums) {
2     int n = nums.size();
3     std::vector<int> dp(n, 1); // every element is an LIS of length 1
4     int best = 1;
5     for (int i = 1; i < n; ++i) {
6         for (int j = 0; j < i; ++j) {
```



```

7         if (nums[j] < nums[i])
8             dp[i] = std::max(dp[i], dp[j] + 1);
9     }
10    best = std::max(best, dp[i]);
11 }
12 return best;
13 }

```

Worked trace: `nums = [10, 9, 2, 5, 3, 7, 101, 18]`.

```

1 dp[0] = 1   (10)
2 dp[1] = 1   (9, no j with nums[j] < 9)
3 dp[2] = 1   (2, no j with nums[j] < 2)
4 dp[3] = 2   (5 > 2, extend dp[2])
5 dp[4] = 2   (3 > 2, extend dp[2])
6 dp[5] = 3   (7 > 2, extend dp[2]; also > 5, extend dp[3]; also > 3, extend
    dp[4])
7 dp[6] = 4   (101 > 7, extend dp[5])
8 dp[7] = 4   (18 > 7, extend dp[5])
9
10 LIS length = 4   (e.g., 2, 5, 7, 101 or 2, 3, 7, 101)

```

Complexity: $O(n^2)$ time, $O(n)$ space. Good for n up to about 10,000.

17.18.2 $O(n \log n)$ Patience Sorting

The key idea: maintain an array `tails` where `tails[i]` is the smallest possible tail element for an increasing subsequence of length $i+1$. For each number, binary search for the position where it can extend or replace an entry.

```

1 #include <vector>
2 #include <algorithm>
3
4 int lis_patience(std::span<const int> nums) {
5     std::vector<int> tails;
6     for (int x : nums) {
7         auto it = std::lower_bound(tails.begin(), tails.end(), x);
8         if (it == tails.end())
9             tails.push_back(x);           // extend longest subsequence
10        else
11            *it = x;                       // replace to keep smallest tail
12    }
13    return tails.size();
14 }

```

Why this works: Each element is placed in the leftmost pile where it can sit (patience sorting analogy). Piles are sorted, and each pile's top card is increasing. The number of piles equals the LIS length. Replacing a tail with a smaller value preserves all existing subsequences while enabling longer

extensions in the future.

17.18.3 Worked Trace: Patience Sorting

nums = [10, 9, 2, 5, 3, 7, 101, 18]

```

1 x=10: tails = [10]                (new pile)
2 x=9:  tails = [9]                 (9 replaces 10 in pile 1)
3 x=2:  tails = [2]                 (2 replaces 9 in pile 1)
4 x=5:  tails = [2, 5]              (5 starts pile 2)
5 x=3:  tails = [2, 3]              (3 replaces 5 in pile 2)
6 x=7:  tails = [2, 3, 7]           (7 starts pile 3)
7 x=101: tails = [2, 3, 7, 101]     (101 starts pile 4)
8 x=18: tails = [2, 3, 7, 18]      (18 replaces 101 in pile 4)
9
10 Number of piles = 4 -> LIS length = 4

```

Note that `tails` is NOT the LIS itself – it is an auxiliary structure. The actual LIS can be recovered by storing predecessor pointers during the binary search.

17.18.4 Recovering the Actual Subsequence

```

1 std::vector<int> lis_with_sequence(std::span<const int> nums) {
2     int n = nums.size();
3     std::vector<int> tails;
4     std::vector<int> tail_indices; // index in nums of each tail
5     std::vector<int> prev(n, -1); // predecessor of nums[i] in LIS
6     std::vector<int> dp(n, 1);
7
8     for (int i = 0; i < n; ++i) {
9         auto it = std::lower_bound(tails.begin(), tails.end(), nums[i]);
10        int pos = it - tails.begin();
11        if (it == tails.end()) {
12            tails.push_back(nums[i]);
13            tail_indices.push_back(i);
14        } else {
15            *it = nums[i];
16            tail_indices[pos] = i;
17        }
18        dp[i] = pos + 1;
19        prev[i] = (pos > 0) ? tail_indices[pos - 1] : -1;
20    }
21
22    // Reconstruct by following predecessors from the last element
23    std::vector<int> result;
24    int k = tail_indices[tails.size() - 1];
25    while (k >= 0) {
26        result.push_back(nums[k]);

```

```

27         k = prev[k];
28     }
29     std::reverse(result.begin(), result.end());
30     return result;
31 }

```

Complexity: $O(n \log n)$ time, $O(n)$ space. The binary search dominates; each element is inserted or replaced once.

When to use which:

- $O(n^2)$ DP: when you need to reconstruct the LIS easily, or when n is small ($< 10,000$), or when you need additional information (e.g., count of LIS of each length).
- $O(n \log n)$ patience sorting: when n is large (up to 10^5 or more) and you only need the length.

Applications: patience sorting itself is a card game algorithm; LIS appears in version control (longest common subsequence of file changes), bioinformatics (sequence alignment), and scheduling (maximum number of non-overlapping intervals is related).

17.19 Exercises

17.19.1 Drill

1. For the 0/1 knapsack with items (weight, value) = [(2, 3), (3, 4), (4, 5), (5, 6)] and capacity $C = 8$, compute the DP table bottom-up. What is the optimal value? Which items are selected?
1. Compute the LCS of "ABCBADAB" and "BDCABA". Show the DP table and trace back to find the LCS.
1. For the matrix chain product with dimensions $A_1(10 \times 20)$, $A_2(20 \times 30)$, $A_3(30 \times 40)$, $A_4(40 \times 30)$, compute the optimal parenthesization using DP. Show the cost table and the split table.
1. Compute the edit distance between "kitten" and "sitting". Show the DP table and trace the optimal alignment.
1. For the coin change problem with coins [1, 2, 5] and amount = 11, compute the DP table. What is the minimum number of coins?
1. State whether each problem has optimal substructure:
 - a) Finding the longest path in a graph
 - b) Finding the shortest path in a graph (Dijkstra)
 - c) Finding the maximum independent set
 - d) Finding the longest increasing subsequence

17.19.2 Application

1. **Supply chain shipping optimization (0/1 knapsack).** A warehouse has trucks with 50-ton capacity. There are 200 pallets with weights (100–1000 kg) and priority values. Implement the bottom-up DP knapsack to select pallets maximizing value. For $n = 200$ and $C = 50,000$, optimize memory using the 1D rolling array technique. Benchmark against the 2D version. How much memory does the 1D version save?
1. **DNA sequence alignment (Smith-Waterman).** Implement the Smith-Waterman local alignment algorithm (a variant of edit distance) for DNA sequences. Align two bacterial gene sequences (download from NCBI). Compute the alignment score and visualize the matching regions. Compare with BLAST's heuristic approach.
1. **Viterbi for speech recognition.** Implement the Viterbi algorithm to decode the most likely phoneme sequence from audio features. Given an HMM with 5 states (phonemes), observation probabilities, and transition probabilities, decode the sequence of observations $[O_1, O_2, O_3, O_4, O_5]$. Show the DP trellis and the most likely state path.
1. **Text justification in Google Docs.** Implement text justification using DP: given words with lengths and a line width of 80 characters, break lines to minimize the sum of $(\text{slack})^3$ where $\text{slack} = \text{width} - \text{used_width}$. Compare the DP result with greedy word-wrapping. Show that greedy produces worse spacing on a sample paragraph.
1. **Robotic path planning (grid DP).** A warehouse robot (similar to Amazon Robotics drives) must navigate from $(0,0)$ to (m,n) on a grid, moving only right and down, avoiding obstacles. Each cell has a traversal cost (floor wear, battery). Find the minimum-cost path using DP. Test on a 100×100 grid with 20% random obstacles.
1. **OPT binary search tree for compiler symbol tables.** Compilers use optimal BSTs for symbol table lookup. Given keys $[A, B, C, D, E]$ with access frequencies $[0.05, 0.15, 0.40, 0.25, 0.15]$, compute the OBST using DP. Show the cost table and the resulting tree. Compare expected search cost with a skewed BST (sorted insertion) and a balanced tree.
1. **Longest increasing subsequence for stock trading.** Find the longest sequence of days where a stock price increases (longest bull run). Implement $O(n^2)$ DP and $O(n \log n)$ patience sorting. Test on 20 years of S&P 500 daily close prices (download from Yahoo Finance). How long is the longest increasing subsequence? How much faster is the $O(n \log n)$ version for 5000 data points?
1. **Held-Karp TSP for delivery route planning.** Implement the Held-Karp DP algorithm for the traveling salesman problem. Test on 15 randomly generated cities in 2D. Show the DP table (size $= n \cdot 2^n$). For $n = 20$, how much memory does the DP table require? At what n does the algorithm become infeasible for a typical laptop? Compare with a nearest-neighbor greedy heuristic — how far from optimal is the greedy solution?

17.19.3 Research

1. **(DP + approximation for knapsack)** The subset sum DP can approximate the knapsack problem with arbitrarily small error. Implement the FPTAS (fully polynomial-time approximation scheme) for knapsack by scaling values. Show that for $\epsilon = 0.1$, the solution is within 10% of optimal while running in polynomial time.
1. **(DP on trees)** Solve the maximum independent set in a tree using DP. A social network wants to select influencers such that no two connected influencers are selected. Implement tree DP and test on random trees with 1000 nodes. How much faster is tree DP than a general graph algorithm (which would be NP-hard)?
1. **(Floyd-Warshall for network routing)** Implement Floyd-Warshall for all-pairs shortest paths in a network of 1000 routers. Use an adjacency matrix with edge weights representing latency. Compare with running Dijkstra from each vertex. Which is faster for dense graphs? For sparse graphs?
1. **(DP for resource allocation)** Implement the DP for allocating a budget of \$1M across 10 projects, each with a nonlinear return function (diminishing returns). Show the DP table and the optimal allocation. Compare with a greedy allocation by ROI.
1. **Longest palindromic subsequence.** Given a string, find the longest subsequence that is a palindrome. Hint: this is an LCS variant – relate it to the LCS of the string and its reverse. Implement and test on strings of length up to 1000.
1. **Minimum edit distance with weighted operations.** Modify the edit distance DP so that insertions cost 2, deletions cost 3, and substitutions cost 1. Recompute the "kitten" to "sitting" trace. How does the optimal alignment change?
1. **Longest increasing subsequence.** Given an array, find the length of the longest strictly increasing subsequence. Implement the $O(n^2)$ DP and the $O(n \log n)$ patience-sorting variant. Compare running times on random arrays of size 10,000.
1. **Unbounded knapsack.** In the unbounded knapsack, each item can be used unlimited times. Modify the 0/1 knapsack recurrence so the inner loop on capacity iterates forwards instead of backwards. Prove why this change allows reuse.
1. **Partition equal subset sum.** Given an array of positive integers, determine if it can be partitioned into two subsets with equal sum. Implement using the subset sum DP. Test on arrays where the total sum is odd (should immediately return false).
1. **Rod cutting problem.** Given a rod of length n and prices $p[1], p[2], \dots, p[n]$ for each length, find the cutting plan that maximizes revenue. Write the DP recurrence and implement it. Compare with the greedy approach of always cutting the longest profitable piece.

1. **Longest common substring.** Unlike LCS (subsequence), a substring must be contiguous. Write a DP that finds the longest common substring of two strings. Hint: use $dp[i][j] = dp[i-1][j-1] + 1$ when characters match, reset to 0 otherwise.
1. **Climbing stairs with variable costs.** Each step i has a cost $c[i]$. You can climb 1 or 2 steps at a time. Find the minimum total cost to reach the top. Write the recurrence, implement it, and trace on costs $[1, 100, 1, 1, 1, 100, 1, 1, 100, 1]$.
1. **Matrix chain – print the split table.** Extend the matrix chain implementation to store and print the split point $s[i][j]$ for every subproblem. Use it to reconstruct the full parenthesization string for 6 matrices with dimensions $[30, 35, 15, 5, 10, 20, 25]$.
1. **Edit distance with transpositions.** The Damerau-Levenshtein distance adds the transposition operation (swapping two adjacent characters) at cost 1. Modify the edit distance DP to include: $dp[i][j] = dp[i-2][j-2] + 1$ when $A[i-1] == B[j-2]$ and $A[i-2] == B[j-1]$. Trace on "CA" to "ABC".

17.20 References and Further Reading

- Richard Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapters 14 (DP), 15 (DP).
- Dan Gusfield, *Algorithms on Strings, Trees, and Sequences*. Cambridge, 1997.
- Michael Held and Richard M. Karp, "A Dynamic Programming Approach to Sequencing Problems" — JSIAM, 1962.
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Chapter 5 (strings), Chapter 6 (DP).
- Donald E. Knuth, "Optimum Binary Search Trees" — Acta Informatica, 1971.

Backtracking and Branch and Bound

18.1 Introduction

Many problems require searching through a large space of candidates. For NP-hard problems, we cannot avoid exponential worst-case time, but we can often prune the search space dramatically.

Backtracking and **branch and bound** are systematic search methods that explore a **state space tree** while pruning branches that cannot lead to a solution.

⚡ **Complete compilable implementations of the data structures in this chapter are in code/.**

18.2 Backtracking

18.2.1 The General Method

Backtracking incrementally builds candidates and abandons a candidate (“backtracks”) as soon as it determines that the candidate cannot lead to a valid solution.

State space tree: every partial solution is a node; children represent all possible extensions. Backtracking explores this tree depth-first, pruning entire subtrees when a constraint is violated.

```

1 procedure backtrack(x):
2     if x is a solution:
3         record x
4         return
5     for each choice c in valid_choices(x):
6         make_choice(x, c)
7         backtrack(x)
8         unmake_choice(x, c)  // backtrack step

```

Pruning strategies:

- **Feasibility pruning:** if the current partial assignment violates a constraint, prune the entire subtree.
- **Bound pruning:** if the current partial solution cannot possibly beat the best found so far, prune (this is the bridge to branch and bound).
- **Symmetry breaking:** if two choices lead to equivalent subtrees (e.g., labeling two identical queens), fix an ordering to avoid exploring both.

Backtracking vs brute force: for n-Queens, brute force places queens in all n^n ways. Backtracking places queens row by row, pruning as soon as two queens attack each other. For $n = 8$, brute force explores $8^8 = 16,777,216$ nodes; backtracking explores only 15,720.

18.2.2 The n-Queens Problem

Place n queens on an $n \times n$ chessboard so that no two queens attack each other.

State space: assign queens to rows $0..n-1$, each in column $\text{col}[i]$. **Constraint:** $\text{col}[i] \neq \text{col}[j]$ and $|\text{col}[i] - \text{col}[j]| \neq |i - j|$ for all $i \neq j$.

```

1  class n_queens {
2  public:
3      n_queens(int n) : n_(n), cols_(n) {}
4
5      std::vector<std::vector<int>> solve() {
6          solutions_.clear();
7          backtrack(0);
8          return solutions_;
9      }
10
11 private:
12     void backtrack(int row) {
13         if (row == n_) {
14             solutions_.push_back(cols_);
15             return;
16         }
17         for (int col = 0; col < n_; ++col) {
18             if (is_safe(row, col)) {
19                 cols_[row] = col;
20                 backtrack(row + 1);
21                 // no explicit unmake needed -- cols_[row] will be
22                 overwritten
23             }
24         }
25
26         bool is_safe(int row, int col) {
27             for (int i = 0; i < row; ++i) {
28                 if (cols_[i] == col) return false; // same column
29                 if (std::abs(cols_[i] - col) == row - i) return false; //
30                 diagonal
31             }
32             return true;
33         }
34
35         int n_;
36         std::vector<int> cols_;
37         std::vector<std::vector<int>> solutions_;

```



```
37 };
```

Trace for n=4:

```
1 Row 0: place queen at col 0
2 Row 1: col 0 -> conflict, col 1 -> conflict, col 2 -> safe
3 Row 2: col 0 -> conflict (diag with row 1), col 1 -> conflict, col 2
    -> conflict,
4         col 3 -> conflict -> backtrack
5 Row 1: try col 3 -> safe
6 Row 2: col 0 -> conflict (col), col 1 -> safe
7 Row 3: no safe column -> backtrack
8 Row 2: try col 2 -> conflict, col 3 -> conflict -> backtrack
9 Row 1: no more options -> backtrack
10 Row 0: try col 1
11 ... eventually finds: [1, 3, 0, 2] and [2, 0, 3, 1]
```

Complexity: $O(n!)$ worst case, but pruning makes it much faster in practice.

18.2.3 Subset Sum

Find subsets of a set that sum to a target value.

```
1 class subset_sum {
2 public:
3     subset_sum(std::span<const int> values, int target)
4         : values_(values.begin(), values.end()), target_(target) {
5         std::sort(values_.begin(), values_.end()); // enable pruning
6     }
7
8     std::vector<std::vector<int>> solve() {
9         std::vector<int> current;
10        backtrack(0, 0, current);
11        return solutions_;
12    }
13
14 private:
15     void backtrack(size_t i, int sum, std::vector<int>& current) {
16         if (sum == target_) {
17             solutions_.push_back(current);
18             return;
19         }
20         if (i >= values_.size() || sum > target_) return;
21
22         // Prune: if sum + remaining values can't reach target
23         int remaining = std::accumulate(values_.begin() + i, values_.end()
24 , 0);
25         if (sum + remaining < target_) return;
```

```

26     // Include values_[i]
27     current.push_back(values_[i]);
28     backtrack(i + 1, sum + values_[i], current);
29     current.pop_back();
30
31     // Exclude values_[i]
32     backtrack(i + 1, sum, current);
33 }
34
35 std::vector<int> values_;
36 int target_;
37 std::vector<std::vector<int>> solutions_;
38 };

```

18.2.4 Subset Generation (All Subsets)

Generate all subsets of 1, 2, ..., n using backtracking:

```

1 void generate_subsets(int n) {
2     std::vector<int> subset;
3     std::function<void(int)> backtrack = [&](int i) {
4         if (i > n) {
5             // Process subset
6             for (int x : subset) std::cout << x << " ";
7             std::cout << "\n";
8             return;
9         }
10        subset.push_back(i);    // include i
11        backtrack(i + 1);
12        subset.pop_back();      // exclude i
13        backtrack(i + 1);
14    };
15    backtrack(1);
16 }

```

This generates 2^n subsets, reflecting the backtracking pattern of include/exclude decisions.

18.2.5 Permutation Generation

```

1 void generate_permutations(std::vector<int>& arr, size_t start) {
2     if (start == arr.size()) {
3         for (int x : arr) std::cout << x << " ";
4         std::cout << "\n";
5         return;
6     }
7     for (size_t i = start; i < arr.size(); ++i) {
8         std::swap(arr[start], arr[i]);
9         generate_permutations(arr, start + 1);

```

```

10         std::swap(arr[start], arr[i]); // backtrack
11     }
12 }

```

18.3 Branch and Bound

Branch and bound is similar to backtracking but uses a **bound** to prune: we compute an optimistic estimate of the best solution achievable from a given node. If the bound is worse than the current best solution, we prune.

Key difference from backtracking: backtracking prunes based on constraints (feasibility); branch and bound prunes based on bounds (optimality). Backtracking finds ALL valid solutions or ANY valid solution; branch and bound finds the OPTIMAL solution.

Bound function: a function $b(\text{node})$ that returns a lower bound (for minimization) on the cost of any solution in the subtree rooted at node. The bound must be **admissible** — it must never overestimate the true optimum. A tighter bound prunes more nodes.

Search strategy: branch and bound typically uses **best-first search** (priority queue ordered by bound) rather than DFS. This explores the most promising nodes first, finding good upper bounds early that enable more pruning.

18.3.1 The General Method

```

1 procedure branch_and_bound(root):
2     best = infinity
3     queue = {root}
4     while queue is not empty:
5         node = queue.pop()
6         if node is a complete solution:
7             best = min(best, value(node))
8         else if bound(node) < best:
9             for each child of node:
10                 queue.push(child)
11     return best

```

18.3.2 Traveling Salesman Problem (TSP)

Find the shortest Hamiltonian cycle in a complete graph. Branch and bound with the **reduced cost matrix** bound:

```

1 class tsp_solver {
2 public:
3     tsp_solver(std::vector<std::vector<int>> cost_matrix)
4         : cost_(std::move(cost_matrix)), n_(cost_.size()), best_cost_(
          INT_MAX) {}

```

```

5
6     int solve() {
7         std::vector<bool> visited(n_, false);
8         visited[0] = true;
9         branch_and_bound(0, 1, 0, visited);
10        return best_cost_;
11    }
12
13 private:
14     void branch_and_bound(int current, int count, int current_cost,
15                           std::vector<bool>& visited) {
16         if (count == n_) {
17             current_cost += cost_[current][0]; // return to start
18             best_cost_ = std::min(best_cost_, current_cost);
19             return;
20         }
21
22         for (int next = 0; next < n_; ++next) {
23             if (!visited[next]) {
24                 int new_cost = current_cost + cost_[current][next];
25                 // Bound: if already worse than best, prune
26                 if (new_cost < best_cost_) {
27                     visited[next] = true;
28                     branch_and_bound(next, count + 1, new_cost, visited);
29                     visited[next] = false;
30                 }
31             }
32         }
33     }
34
35     const std::vector<std::vector<int>>& cost_;
36     size_t n_;
37     int best_cost_;
38 };

```

A tighter bound: compute the reduced cost matrix. For each row, subtract its minimum. For each column, subtract its minimum. The sum of all subtracted values gives a lower bound for any tour.

18.3.3 15-Puzzle

Solve the 15-puzzle using branch and bound with the **Manhattan distance** heuristic:

```

1 struct puzzle_state {
2     std::vector<int> board; // 16 elements, 0 = empty
3     int zero_pos;
4     int g; // cost so far (moves made)
5     int h; // heuristic (Manhattan distance)
6
7     int f() const { return g + h; }

```

```

8
9     bool operator>(const puzzle_state& other) const {
10         return f() > other.f();
11     }
12 };
13
14 int manhattan_distance(const std::vector<int>& board) {
15     int distance = 0;
16     for (int i = 0; i < 16; ++i) {
17         if (board[i] == 0) continue;
18         int target_row = (board[i] - 1) / 4;
19         int target_col = (board[i] - 1) % 4;
20         int row = i / 4;
21         int col = i % 4;
22         distance += std::abs(row - target_row) + std::abs(col - target_col
23     );
24     }
25     return distance;
26 }
27 // A* search is branch and bound with a priority queue ordered by f = g +
28 h
29 int solve_15_puzzle(const std::vector<int>& start) {
30     std::priority_queue<puzzle_state, std::vector<puzzle_state>,
31         std::greater<puzzle_state>> pq;
32
33     int start_zero = std::find(start.begin(), start.end(), 0) - start.
34     begin();
35     pq.push({start, start_zero, 0, manhattan_distance(start)});
36
37     std::set<std::vector<int>> visited;
38
39     while (!pq.empty()) {
40         auto state = pq.top(); pq.pop();
41         if (state.h == 0) return state.g; // solved
42
43         if (visited.count(state.board)) continue;
44         visited.insert(state.board);
45
46         // Generate moves (up, down, left, right)
47         int row = state.zero_pos / 4;
48         int col = state.zero_pos % 4;
49         int dirs[][2] = {{-1,0}, {1,0}, {0,-1}, {0,1}};
50
51         for (auto [dr, dc] : dirs) {
52             int nr = row + dr, nc = col + dc;
53             if (nr < 0 || nr >= 4 || nc < 0 || nc >= 4) continue;

```

```
53         auto new_board = state.board;
54         int new_pos = nr * 4 + nc;
55         std::swap(new_board[state.zero_pos], new_board[new_pos]);
56
57         if (!visited.count(new_board)) {
58             pq.push({new_board, new_pos, state.g + 1,
59                     manhattan_distance(new_board)});
60         }
61     }
62 }
63 return -1; // unsolvable
64 }
```

This is **A* search** — branch and bound with a priority queue and an admissible heuristic (one that never overestimates the true cost).

18.3.4 Backtracking vs. Branch and Bound

Goal	Find all solutions or any solution	Find optimal solution
Pruning	Constraints (feasibility)	Bounds (optimality)
Search order	Typically DFS	Typically best-first
Memory	O(depth) recursion stack	O(branching factor × depth) for priority queue
Typical use	N-queens, Sudoku, subset sum	TSP, 15-puzzle, integer programming

Worked comparison: TSP on 4 cities with cost matrix:

```
1      A   B   C   D
2 A [  0  10  15  20]
3 B [ 10   0  35  25]
4 C [ 15  35   0  30]
5 D [ 20  25  30   0]
```

Backtracking approach: enumerate all 3! = 6 Hamiltonian cycles:

```
1 A-B-C-D-A: 10+35+30+20 = 95
2 A-B-D-C-A: 10+25+30+15 = 80
3 A-C-B-D-A: 15+35+25+20 = 95
4 A-C-D-B-A: 15+30+25+10 = 80
5 A-D-B-C-A: 20+25+35+15 = 95
6 A-D-C-B-A: 20+30+35+10 = 95
7 Best: 80
```

Branch and bound: use reduced cost matrix bound.

```
1 Root: reduce rows (subtract min from each):
2   [0, 0, 5, 10] after subtracting [0, 0, 10, 10, 15, 20] -> row mins: 0,
   10, 15, 20
3   Actually: row mins are 0, 10, 15, 20. Sum of reductions = 0+10+15+20 =
   45.
```

```

4   Lower bound = 45.
5
6   Explore A->B (cost 10):
7   Remaining matrix reduced. Bound = 45 + 10 + (reductions of submatrix) =
    65.
8
9   Explore A->C (cost 15):
10  Bound = 45 + 15 + (reductions) = 75.
11
12  Since 65 < 75, explore A->B first. Eventually find optimal = 80.
13  Prune branches where bound >= 80 (current best).

```

Branch and bound explored fewer nodes than backtracking because the bound pruned subtrees that were provably suboptimal.

18.4 Application: Sudoku Solver

```

1  class sudoku_solver {
2  public:
3      bool solve(std::vector<std::vector<int>>& board) {
4          for (int row = 0; row < 9; ++row) {
5              for (int col = 0; col < 9; ++col) {
6                  if (board[row][col] == 0) {
7                      for (int num = 1; num <= 9; ++num) {
8                          if (is_valid(board, row, col, num)) {
9                              board[row][col] = num;
10                             if (solve(board)) return true;
11                             board[row][col] = 0; // backtrack
12                         }
13                     }
14                     return false; // no valid number for this cell
15                 }
16             }
17         }
18         return true; // all cells filled
19     }
20
21     bool is_valid(const std::vector<std::vector<int>>& board,
22                  int row, int col, int num) {
23         for (int i = 0; i < 9; ++i) {
24             if (board[row][i] == num) return false;
25             if (board[i][col] == num) return false;
26             int box_row = 3 * (row / 3) + i / 3;
27             int box_col = 3 * (col / 3) + i % 3;
28             if (board[box_row][box_col] == num) return false;
29         }
30         return true;

```

```
31     }
32 };
```

18.5 Common Bugs and Pitfalls

Pitfall	Consequence	
Forgetting to restore state after backtracking	State corrupted, solution space polluted	Always un
No pruning on unsolvable branches	Exponential time exploring dead ends	
Infinite recursion if base case is not reached	Stack overflow, crash	Ensure ever
Wrong bounding function in branch-and-bound	Prunes optimal solutions, suboptimal result	Bounding funct
Using float equality for bound comparison	Wrong bound evaluation	Use tolerance abs
Not sorting items before search (e.g., subset sum)	Much slower search, more backtracking	

18.6 Summary

1. **Backtracking** systematically explores the state space tree, pruning branches that violate constraints.
2. **n-Queens, subset sum, and Sudoku** are canonical backtracking problems.
3. **Branch and bound** extends backtracking by pruning based on bounds, searching for optimal solutions.
4. **A* search** is branch and bound with a priority queue and an admissible heuristic.
5. Both methods solve NP-hard problems in practice by pruning most of the exponential search space.

18.7 Interview FAQs

1. **How do you estimate the actual number of nodes explored by backtracking — beyond the $O(b^d)$ upper bound?**

The $O(b^d)$ bound assumes no pruning. In practice, constraint propagation eliminates most branches early. The actual count depends on constraint tightness: for n -Queens, the first queen has n choices, the second has $\sim n-2$, the third $\sim n-4$ — the branching factor decreases rapidly. For Sudoku with constraint propagation (naked singles, hidden singles), the effective branching factor drops to $\sim 1-2$, making the search nearly linear. The practical approach: run the solver on small instances ($n = 8, 10, 12$) and measure the node count. If it grows polynomially, the pruning is effective. If it grows exponentially, you need a better heuristic or a different algorithm.

1. How does branch and bound differ from backtracking?

Backtracking prunes subtrees that violate *constraints* (feasibility). Branch and bound prunes subtrees whose *bound* is worse than the current best (optimality). Backtracking finds all solutions; branch and bound finds the optimal solution. Branch and bound typically uses best-first search (priority queue) rather than DFS.

1. What's the difference between an admissible heuristic and a consistent heuristic, and why does it matter for A*?

Admissible: $h(n)$ never overestimates the true cost to goal — guarantees A* finds the optimal path. Consistent: $h(n) \leq c(n, n') + h(n')$ for every edge — guarantees A* never needs to reopen closed nodes. Consistency is stronger than admissibility. Without consistency, A* may expand a node, then later find a cheaper path to it and re-expand — wasting time but still finding the optimal solution. With consistency, once a node is closed, its distance is final. Manhattan distance is both admissible and consistent for grid paths. Euclidean distance is admissible and consistent for any geometric space. Custom heuristics (e.g., pattern databases for 15-puzzle) are admissible but may not be consistent — check the edge condition.

1. How do you solve n-Queens efficiently?

Backtracking with pruning: place queens row by row. For each row, try each column; check if it conflicts with any previously placed queen (same column or diagonal). For $n = 8$, only 15,720 nodes are explored vs $8^8 = 16.7\text{M}$ for brute force. Further optimizations: use bitmasks for $O(1)$ conflict checking, and symmetry breaking (fix the first queen's column to halve the search).

1. Explain the reduced cost matrix bound for TSP.

For each row, subtract its minimum value; then for each column, subtract its minimum. The sum of all subtracted values is a lower bound on the cost of any tour passing through that node. This bound is admissible because any Hamiltonian cycle must include exactly one edge from each row and column, and the reductions represent the minimum possible cost.

1. What makes pruning effective, and how do you design constraints that prune more aggressively?

Pruning works when constraints eliminate large subtrees early. The key metric: *pruning power* — what fraction of the search tree is cut by each constraint check. Effective strategies: (1) Order variables by domain size (most constrained first) — fail fast on tight constraints; (2) Forward propagation — after each assignment, eliminate inconsistent values from future variables' domains (arc consistency); (3) Symmetry breaking — fix one variable to eliminate equivalent solutions (e.g., fix first queen to left half for n -Queens); (4) Dynamic variable ordering — choose the variable with the fewest remaining values at each step (MRV heuristic). The lesson: the algorithm's power comes from the constraints, not the search strategy.

1. Why is pure backtracking too slow for hard Sudoku, and what preprocessing makes it practical?

Pure backtracking tries all 9 digits per cell — $O(9^{81})$ worst-case, absurdly slow for hard puzzles. Constraint propagation reduces this dramatically: (1) Naked singles: if a cell has only one possible digit, assign it immediately — no branching; (2) Hidden singles: if a digit can only go in one cell in a row/col/box, assign it; (3) Naked pairs/triples: if k cells in a unit share exactly k digits, eliminate those digits from other cells. After propagation, the effective branching factor drops to $\sim 1\text{--}2$ for most puzzles. Hard puzzles require backtracking after propagation, but only at a few decision points. The combination of propagation + backtracking solves any Sudoku in $< 1\text{ms}$ on modern hardware.

1. Why is the 15-puzzle hard for A*, and what heuristic improvements make it tractable?

The 15-puzzle has $\sim 10^{13}$ states — too many for brute force. Manhattan distance (sum of each tile's distance from its goal) is admissible but weak: it ignores interactions between tiles. Better heuristics: (1) Linear conflict — add 2 for every pair of tiles that must cross each other; (2) Pattern databases — precompute the cost of solving a subset of tiles (e.g., corner tiles) and use it as a heuristic; (3) Additive pattern databases — combine multiple pattern databases by taking the max. With a good pattern database, A* solves the 15-puzzle in < 1 second. The 24-puzzle (5×5) requires IDA* with pattern databases — A* runs out of memory. The lesson: the heuristic quality determines whether the problem is tractable.

1. How do you implement a backtracking solution in C++ efficiently?

Use a recursive function with a state vector (e.g., `std::vector<int>` for queen positions). Pass by reference and undo changes on backtrack (push/pop or swap-and-swap-back). Avoid copying state at each level. For constraint checking, use bitmasks (for n-Queens: column, diag1, diag2 as bit sets) for $O(1)$ conflict detection.

1. Explain the difference between DFS and best-first search in backtracking.

DFS explores the deepest node first — $O(\text{depth})$ memory, good for finding any solution quickly, but may explore bad branches deeply before finding good ones. Best-first (branch and bound) explores the most promising node first — $O(\text{branching} \times \text{depth})$ memory, finds good upper bounds early, enabling more pruning. DFS is preferred when solutions are dense; best-first when bounds are tight.

18.8 N-Queens Problem: A Deep Dive

The n-Queens problem was introduced earlier in this chapter. Here we provide a complete worked trace, analyze the state space tree in detail, and develop an optimized implementation.

18.8.1 State Space Tree for 4-Queens

The state space tree for 4-Queens has rows as levels and columns as branches at each level. A partial assignment (col[0], col[1], ..., col[k-1]) is a node at level k. The root (level 0) represents an empty board.

```

1 State Space Tree for 4-Queens:
2                               root
3                   /      /      |      \      \
4                   0      1      2      3          <-- row 0: col placed
5                   /\    /\    /\    /\
6                   ... (row 1 branches)
7
8 Leaf nodes that are solutions:
9   [1, 3, 0, 2]: queens at (0,1), (1,3), (2,0), (3,2)
10  [2, 0, 3, 1]: queens at (0,2), (1,0), (2,3), (3,1)

```

At each level k, we try placing a queen in columns 0 through n-1 of row k, checking against all previously placed queens (rows 0 through k-1). A placement is invalid if any prior queen shares the same column or diagonal.

18.8.2 Worked Trace: Backtracking for 4-Queens

We trace the algorithm step by step. At each step we record the action taken and the reason for backtracking when it occurs.

```

1 Initial state: cols = [-1, -1, -1, -1], n = 4
2
3 Step 1: Row 0, try col 0 -> safe. cols = [0, -1, -1, -1]
4 Step 2: Row 1, try col 0 -> conflict (same col as row 0)
5 Step 3: Row 1, try col 1 -> conflict (diag with row 0: |1-0|==|1-0|)
6 Step 4: Row 1, try col 2 -> safe. cols = [0, 2, -1, -1]
7 Step 5: Row 2, try col 0 -> conflict (same col as row 0)
8 Step 6: Row 2, try col 1 -> conflict (diag with row 1: |1-1|==|2-1|)
9 Step 7: Row 2, try col 2 -> conflict (same col as row 1)
10 Step 8: Row 2, try col 3 -> conflict (diag with row 0: |3-0|==|2-0|)
11       No valid column for row 2 -> BACKTRACK to row 1
12 Step 9: Row 1, try col 3 -> safe. cols = [0, 3, -1, -1]
13 Step 10: Row 2, try col 0 -> conflict (same col as row 0)
14 Step 11: Row 2, try col 1 -> safe. cols = [0, 3, 1, -1]
15 Step 12: Row 3, try col 0 -> conflict (same col as row 0)
16 Step 13: Row 3, try col 1 -> conflict (same col as row 2)
17 Step 14: Row 3, try col 2 -> conflict (diag with row 1: |2-3|==|3-1|)
18 Step 15: Row 3, try col 3 -> conflict (same col as row 1)
19       No valid column for row 3 -> BACKTRACK to row 2
20 Step 16: Row 2, try col 2 -> conflict (diag with row 0: |2-0|==|2-0|)
21 Step 17: Row 2, try col 3 -> conflict (diag with row 1: |3-3|==|2-1|)
22       No valid column for row 2 -> BACKTRACK to row 1

```

```

23         No more columns for row 1 -> BACKTRACK to row 0
24 Step 18: Row 0, try col 1 -> safe. cols = [1, -1, -1, -1]
25 Step 19: Row 1, try col 0 -> conflict (diag with row 0: |0-1|==|1-0|)
26 Step 20: Row 1, try col 1 -> conflict (same col as row 0)
27 Step 21: Row 1, try col 2 -> conflict (diag with row 0: |2-1|==|1-0|)
28 Step 22: Row 1, try col 3 -> safe. cols = [1, 3, -1, -1]
29 Step 23: Row 2, try col 0 -> conflict (diag with row 1: |0-3|==|2-1|)
30 Step 24: Row 2, try col 1 -> conflict (same col as row 0)
31 Step 25: Row 2, try col 2 -> conflict (diag with row 0: |2-1|==|2-0|)
32 Step 26: Row 2, try col 3 -> conflict (same col as row 1)
33         No valid column for row 2 -> BACKTRACK to row 1
34         No more columns for row 1 -> BACKTRACK to row 0
35 Step 27: Row 0, try col 2 -> safe. cols = [2, -1, -1, -1]
36 Step 28: Row 1, try col 0 -> safe. cols = [2, 0, -1, -1]
37 Step 29: Row 2, try col 0 -> conflict (same col as row 1)
38 Step 30: Row 2, try col 1 -> conflict (diag with row 1: |1-0|==|2-1|)
39 Step 31: Row 2, try col 2 -> conflict (same col as row 0)
40 Step 32: Row 2, try col 3 -> safe. cols = [2, 0, 3, -1]
41 Step 33: Row 3, try col 0 -> conflict (same col as row 1)
42 Step 34: Row 3, try col 1 -> safe. cols = [2, 0, 3, 1] ** SOLUTION
        FOUND **
43         SOLUTION 1: [2, 0, 3, 1]
44
45 Step 35: Row 3, try col 2 -> conflict (same col as row 0)
46 Step 36: Row 3, try col 3 -> conflict (same col as row 2)
47         No more columns for row 3 -> BACKTRACK
48         ... continue backtracking ...
49 Step 37: Row 0, try col 3 -> safe. cols = [3, -1, -1, -1]
50 Step 38: Row 1, try col 0 -> safe. cols = [3, 0, -1, -1]
51 Step 39: Row 2, try col 0 -> conflict (same col as row 1)
52 Step 40: Row 2, try col 1 -> safe. cols = [3, 0, 1, -1]
53 Step 41: Row 3, try col 0 -> conflict (same col as row 1)
54 Step 42: Row 3, try col 1 -> conflict (same col as row 2)
55 Step 43: Row 3, try col 2 -> conflict (diag with row 0: |2-3|==|3-0|)
56 Step 44: Row 3, try col 3 -> conflict (same col as row 0)
57         BACKTRACK ... no solution from row 0 col 3
58
59 Total nodes visited: 44 (vs 4^4 = 256 for brute force)
60 Solutions found: [2, 0, 3, 1] and [1, 3, 0, 2]

```

18.8.3 Optimized C++ Implementation with Bitmasks

The basic implementation checks each placed queen with a loop. We can speed this up using bitmasks for $O(1)$ conflict detection. Three bitmasks track occupied columns, the main diagonals, and the anti-diagonals.

```

1 class n_queens_bitmask {
2 public:

```

```

3     explicit n_queens_bitmask(int n) : n_(n), count_(0) {}
4
5     int solve_count() {
6         count_ = 0;
7         backtrack(0, 0, 0, 0);
8         return count_;
9     }
10
11     std::vector<std::vector<int>> solve_all() {
12         solutions_.clear();
13         std::vector<int> cols(n_);
14         backtrack_all(0, 0, 0, 0, cols);
15         return solutions_;
16     }
17
18 private:
19     void backtrack(int row, int cols, int diag1, int diag2) {
20         if (row == n_) {
21             ++count_;
22             return;
23         }
24         int available = ((1 << n_) - 1) & ~(cols | diag1 | diag2);
25         while (available) {
26             int bit = available & (-available);
27             available &= available - 1;
28             backtrack(row + 1, cols | bit,
29                     (diag1 | bit) << 1, (diag2 | bit) >> 1);
30         }
31     }
32
33     void backtrack_all(int row, int cols, int diag1, int diag2,
34                       std::vector<int>& assignment) {
35         if (row == n_) {
36             solutions_.push_back(assignment);
37             return;
38         }
39         int available = ((1 << n_) - 1) & ~(cols | diag1 | diag2);
40         while (available) {
41             int bit = available & (-available);
42             available &= available - 1;
43             int col = __builtin_ctz(bit);
44             assignment[row] = col;
45             backtrack_all(row + 1, cols | bit,
46                           (diag1 | bit) << 1, (diag2 | bit) >> 1,
47                           assignment);
48         }
49     }
50

```

```

51     int n_;
52     int count_;
53     std::vector<std::vector<int>> solutions_;
54 };
55
56 // Example usage:
57 // n_queens_bitmask_solver(8);
58 // std::cout << "8-Queens solutions: " << solver.solve_count() << "\n";
59 // Output: 8-Queens solutions: 92

```

How the bitmasks work:

- `cols`: bit i is set if column i is occupied.
- `diag1`: bit i is set if diagonal ($\text{row} - \text{col} = \text{const}$) is occupied. Shift left by 1 each row because the diagonal offset changes.
- `diag2`: bit i is set if anti-diagonal ($\text{row} + \text{col} = \text{const}$) is occupied. Shift right by 1 each row.
- `available` computes all free positions in $O(1)$ using bitwise operations.

Performance comparison for $n = 14$:

1 Method	Nodes visited	Time
2 Basic backtracking	~5,000,000	1.2s
3 Bitmask backtracking	~5,000,000	0.15s
4 Bitmask + symmetry breaking	~2,500,000	0.08s

Symmetry breaking: for n -Queens, solutions come in mirror-image pairs. Fixing the first queen to the left half of the board ($\text{col} \leq n/2$) halves the search. For even n , this gives exactly half the solutions; for odd n , solutions with the first queen in the middle column are self-symmetric and counted once.

18.8.4 Complexity Analysis

The number of solutions for n -Queens grows rapidly:

1 n	Solutions	Brute force (n^n)	Backtracking nodes
2 4	2	256	44
3 5	10	3,125	185
4 6	4	46,656	535
5 7	40	823,543	2,722
6 8	92	16,777,216	15,720
7 9	352	387,420,489	81,543
8 10	724	10,000,000,000	482,765
9 14	365,596	huge	~5,000,000

The worst-case time is $O(n!)$ since we try all permutations in the worst case. With pruning, the practical running time is much closer to $O(n^{n/2})$ or better. The bitmask implementation runs in $O(S)$ where S is the number of nodes visited.

18.9 Sudoku Solver: Backtracking with Constraint Checking

Sudoku is a 9×9 grid divided into nine 3×3 boxes. Fill each cell with a digit 1-9 so that every row, column, and box contains each digit exactly once. We solve any Sudoku puzzle using backtracking with three constraint checks.

18.9.1 Algorithm Overview

The solver scans the grid left-to-right, top-to-bottom. At the first empty cell (value 0), it tries digits 1 through 9. For each candidate, it checks three constraints:

1. **Row constraint:** the digit does not appear elsewhere in the same row.
2. **Column constraint:** the digit does not appear elsewhere in the same column.
3. **Box constraint:** the digit does not appear elsewhere in the same 3×3 box.

If a digit passes all three checks, it is placed and the solver recurses. If no digit works, the cell is reset to 0 and the solver backtracks.

18.9.2 Worked Partial Trace

Consider this puzzle (0 denotes empty):

```

1 Initial board:
2   5 3 0 | 0 7 0 | 0 0 0
3   6 0 0 | 1 9 5 | 0 0 0
4   0 9 8 | 0 0 0 | 0 6 0
5   -----+-----+-----
6   8 0 0 | 0 6 0 | 0 0 3
7   4 0 0 | 8 0 3 | 0 0 1
8   7 0 0 | 0 2 0 | 0 0 6
9   -----+-----+-----
10  0 6 0 | 0 0 0 | 2 8 0
11  0 0 0 | 4 1 9 | 0 0 5
12  0 0 0 | 0 8 0 | 0 7 9
13
14 Step 1: Row 0, Col 2 (empty).
15         Try 1: row 0 has no 1, col 2 has 8,6 -> no 1,
16                box(0,0) has 5,3,6,9,8 -> no 1
17                -> place 1. board[0][2] = 1.
18 Step 2: Row 0, Col 3 (empty).
19         Try 1: row 0 now has 1 -> conflict.
20         Try 2: row 0 has no 2, col 3 has 1,8,4 -> no 2,
21                box(0,3) has 7,1,9,5 -> no 2
22                -> place 2. board[0][3] = 2.
23         ... continue placing digits in row 0 ...

```

```

24 Step 3: Row 0, Col 5 (empty).
25     Try all digits 1-9: all conflict with row, col, or box.
26     -> BACKTRACK: reset Col 3 to 0, try next digit.
27     Try 3 for Col 3: row 0 has no 3, col 3 has 1,8,4 -> no 3,
28         box(0,3) has 7,1,9,5 -> no 3.
29     -> place 3. board[0][3] = 3.
30     ... continue ...
31
32 This process continues, exploring and backtracking until the
33 board is filled. Hard puzzles (17 clues) may require millions
34 of backtracks.

```

18.9.3 C++ Implementation

```

1 class sudoku_solver {
2 public:
3     bool solve(std::vector<std::vector<int>>& board) {
4         int row, col;
5         if (!find_empty(board, row, col)) return true;
6
7         for (int num = 1; num <= 9; ++num) {
8             if (is_valid(board, row, col, num)) {
9                 board[row][col] = num;
10                if (solve(board)) return true;
11                board[row][col] = 0; // backtrack
12            }
13        }
14        return false; // trigger backtracking
15    }
16
17 private:
18     bool find_empty(const std::vector<std::vector<int>>& board,
19                    int& row, int& col) {
20         for (row = 0; row < 9; ++row) {
21             for (col = 0; col < 9; ++col) {
22                 if (board[row][col] == 0) return true;
23             }
24         }
25         return false;
26     }
27
28     bool is_valid(const std::vector<std::vector<int>>& board,
29                 int row, int col, int num) {
30         for (int c = 0; c < 9; ++c) {
31             if (board[row][c] == num) return false;
32         }
33         for (int r = 0; r < 9; ++r) {

```



```

34         if (board[r][col] == num) return false;
35     }
36     int box_row = 3 * (row / 3);
37     int box_col = 3 * (col / 3);
38     for (int r = box_row; r < box_row + 3; ++r) {
39         for (int c = box_col; c < box_col + 3; ++c) {
40             if (board[r][c] == num) return false;
41         }
42     }
43     return true;
44 }
45 };
46
47 // Usage:
48 // std::vector<std::vector<int>> board = {
49 //     {5,3,0,0,7,0,0,0,0},
50 //     {6,0,0,1,9,5,0,0,0},
51 //     {0,9,8,0,0,0,0,6,0},
52 //     {8,0,0,0,6,0,0,0,3},
53 //     {4,0,0,8,0,3,0,0,1},
54 //     {7,0,0,0,2,0,0,0,6},
55 //     {0,6,0,0,0,0,2,8,0},
56 //     {0,0,0,4,1,9,0,0,5},
57 //     {0,0,0,0,8,0,0,7,9}
58 // };
59 // sudoku_solver solver;
60 // if (solver.solve(board)) { /* print board */ }

```

18.9.4 Optimizations

The basic solver works but is slow for hard puzzles. Several optimizations reduce backtracking dramatically:

```

1 // Optimization: Precompute which digits are used in each
2 // row, column, and box using bitmasks.
3 // MRV heuristic: always fill the cell with fewest choices.
4 class sudoku_optimized {
5 public:
6     bool solve(std::vector<std::vector<int>>& board) {
7         int row_mask[9] = {}, col_mask[9] = {}, box_mask[9] = {};
8         for (int r = 0; r < 9; ++r) {
9             for (int c = 0; c < 9; ++c) {
10                 if (board[r][c] != 0) {
11                     int bit = 1 << board[r][c];
12                     row_mask[r] |= bit;
13                     col_mask[c] |= bit;
14                     box_mask[box_id(r, c)] |= bit;
15                 }

```

```

16         }
17     }
18     return solve(board, row_mask, col_mask, box_mask);
19 }
20
21 private:
22     int box_id(int r, int c) { return 3 * (r / 3) + c / 3; }
23
24     bool solve(std::vector<std::vector<int>>& board,
25               int row_mask[], int col_mask[], int box_mask[]) {
26         int best_r = -1, best_c = -1, min_choices = 10;
27         for (int r = 0; r < 9; ++r) {
28             for (int c = 0; c < 9; ++c) {
29                 if (board[r][c] == 0) {
30                     int used = row_mask[r] | col_mask[c]
31                         | box_mask[box_id(r, c)];
32                     int choices = 9 - __builtin_popcount(used);
33                     if (choices < min_choices) {
34                         min_choices = choices;
35                         best_r = r;
36                         best_c = c;
37                         if (choices == 0) return false;
38                         if (choices == 1) break;
39                     }
40                 }
41             }
42             if (best_r >= 0 && min_choices <= 1) break;
43         }
44         if (best_r < 0) return true;
45
46         int b = box_id(best_r, best_c);
47         int used = row_mask[best_r] | col_mask[best_c]
48             | box_mask[b];
49         for (int num = 1; num <= 9; ++num) {
50             int bit = 1 << num;
51             if (used & bit) continue;
52             board[best_r][best_c] = num;
53             row_mask[best_r] |= bit;
54             col_mask[best_c] |= bit;
55             box_mask[b] |= bit;
56             if (solve(board, row_mask, col_mask, box_mask))
57                 return true;
58             board[best_r][best_c] = 0;
59             row_mask[best_r] &= ~bit;
60             col_mask[best_c] &= ~bit;
61             box_mask[b] &= ~bit;
62         }
63         return false;

```

```

64     }
65 };

```

The key optimization is **Minimum Remaining Values (MRV)**: always fill the empty cell with the fewest valid candidates first. This detects dead ends early and reduces backtracks by orders of magnitude. The bitmask representation makes each constraint check $O(1)$.

18.10 Subset Sum: Backtracking vs. Dynamic Programming

The subset sum problem asks: given a set of integers and a target value, does any subset sum to the target? We saw a backtracking solution earlier in this chapter and a dynamic programming solution in Chapter 17. Here we compare the two approaches in detail.

18.10.1 Backtracking Approach (Revisited)

The backtracking algorithm considers elements one at a time, making an include/exclude decision. It prunes when the running sum exceeds the target or when the remaining elements cannot reach the target.

```

1  class subset_sum_backtrack {
2  public:
3      subset_sum_backtrack(std::vector<int> vals, int target)
4          : vals_(std::move(vals)), target_(target) {
5          std::sort(vals_.begin(), vals_.end());
6      }
7
8      bool has_solution() {
9          std::vector<int> current;
10         return backtrack(0, 0, current);
11     }
12
13     std::vector<std::vector<int>> find_all() {
14         results_.clear();
15         std::vector<int> current;
16         backtrack_all(0, 0, current);
17         return results_;
18     }
19
20 private:
21     bool backtrack(size_t i, int sum, std::vector<int>& cur) {
22         if (sum == target_) return true;
23         if (i >= vals_.size() || sum > target_) return false;
24
25         int remaining = 0;
26         for (size_t j = i; j < vals_.size(); ++j)
27             remaining += vals_[j];
28         if (sum + remaining < target_) return false;

```

```

29
30     cur.push_back(vals_[i]);
31     if (backtrack(i + 1, sum + vals_[i], cur)) return true;
32     cur.pop_back();
33
34     return backtrack(i + 1, sum, cur);
35 }
36
37 void backtrack_all(size_t i, int sum, std::vector<int>& cur) {
38     if (sum == target_) {
39         results_.push_back(cur);
40         return;
41     }
42     if (i >= vals_.size() || sum > target_) return;
43
44     int remaining = 0;
45     for (size_t j = i; j < vals_.size(); ++j)
46         remaining += vals_[j];
47     if (sum + remaining < target_) return;
48
49     cur.push_back(vals_[i]);
50     backtrack_all(i + 1, sum + vals_[i], cur);
51     cur.pop_back();
52     backtrack_all(i + 1, sum, cur);
53 }
54
55 std::vector<int> vals_;
56 int target_;
57 std::vector<std::vector<int>> results_;
58 };

```

18.10.2 Dynamic Programming Approach (from Chapter 17)

The DP approach uses a boolean table `dp[i][s]` that is true if a subset of the first `i` elements sums to `s`.

```

1 class subset_sum_dp {
2 public:
3     bool has_solution(const std::vector<int>& vals, int target) {
4         int n = vals.size();
5         std::vector<std::vector<bool>> dp(n + 1,
6             std::vector<bool>(target + 1, false));
7
8         for (int i = 0; i <= n; ++i) dp[i][0] = true;
9
10        for (int i = 1; i <= n; ++i) {
11            for (int s = 0; s <= target; ++s) {
12                dp[i][s] = dp[i - 1][s];

```

```
13         if (s >= vals[i - 1]) {
14             dp[i][s] = dp[i][s]
15                 || dp[i - 1][s - vals[i - 1]];
16         }
17     }
18 }
19 return dp[n][target];
20 }
21 };
```

18.10.3 Comparison

	Backtracking	Dynamic Programming
1 Time (worst case)	$O(2^n)$	$O(n * \text{target})$
2 Space	$O(n)$ recursion stack	$O(n * \text{target})$ table
3 Finds all solutions?	Yes (natural)	Needs reconstruction
4 Negative numbers?	Yes (with care)	Not directly
5 When pruning is good	Very fast (early exit)	Same or faster
6 When pruning is poor	Slow (explores all)	Same or faster

Key insight: backtracking can be faster than DP when the answer is found early (the first solution is near the top of the search tree) or when pruning is aggressive (sorted input, small target relative to element values). DP is better when we need to answer many queries on the same set, or when the target is small relative to 2^n .

Space advantage: backtracking uses $O(n)$ space (recursion stack), while DP uses $O(n \times \text{target})$ space. For large targets, this makes DP infeasible while backtracking remains practical.

Example: for `vals = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` and `target = 15`, backtracking with pruning explores only 58 nodes. The DP table has $11 \times 16 = 176$ entries. Both are fast here, but backtracking finds the first solution (`sum = 15`) after only 12 nodes.

18.11 Graph Coloring: Backtracking Approach

The graph coloring problem asks: given an undirected graph G and m colors, assign a color to each vertex so that no two adjacent vertices share the same color. Find a valid coloring or determine that none exists with m colors.

18.11.1 Algorithm

The backtracking algorithm assigns colors to vertices one at a time (vertices are numbered 0 through $n-1$). For each vertex, it tries colors 1 through m , checking that no adjacent vertex has the same color.

```
1 class graph_coloring {
2 public:
3     graph_coloring(int n, const std::vector<std::pair<int,int>>& edges,
```

```

4         int m)
5         : n_(n), m_(m), colors_(n, 0) {
6         adj_.resize(n);
7         for (auto [u, v] : edges) {
8             adj_[u].push_back(v);
9             adj_[v].push_back(u);
10        }
11    }
12
13    bool solve() { return backtrack(0); }
14
15    std::vector<int> get_colors() const { return colors_; }
16
17 private:
18    bool backtrack(int v) {
19        if (v == n_) return true;
20
21        for (int c = 1; c <= m_; ++c) {
22            if (is_safe(v, c)) {
23                colors_[v] = c;
24                if (backtrack(v + 1)) return true;
25                colors_[v] = 0; // backtrack
26            }
27        }
28        return false;
29    }
30
31    bool is_safe(int v, int c) {
32        for (int u : adj_[v]) {
33            if (colors_[u] == c) return false;
34        }
35        return true;
36    }
37
38    int n_, m_;
39    std::vector<std::vector<int>> adj_;
40    std::vector<int> colors_;
41 };

```

18.11.2 Worked Trace: Graph with 4 Vertices and 3 Colors

Consider the graph: edges = {(0,1), (0,2), (1,2), (2,3)}. We try to color with 3 colors.

```

1 Graph:
2   0 --- 1
3   |   /
4   2 --- 3
5

```

```

6 Vertices: 0, 1, 2, 3
7 Edges:    (0,1), (0,2), (1,2), (2,3)
8 Colors:   1 (Red), 2 (Blue), 3 (Green)
9
10 Step 1:  Vertex 0: try color 1. No neighbors colored. Safe.
11          colors = [1, 0, 0, 0]
12
13 Step 2:  Vertex 1: try color 1. Neighbor 0 has color 1. CONFLICT.
14          Vertex 1: try color 2. Neighbor 0 has color 1. Safe.
15          colors = [1, 2, 0, 0]
16
17 Step 3:  Vertex 2: try color 1. Neighbor 0 has color 1. CONFLICT.
18          Vertex 2: try color 2. Neighbor 1 has color 2. CONFLICT.
19          Vertex 2: try color 3. Neighbors 0 (c1), 1 (c2). Safe.
20          colors = [1, 2, 3, 0]
21
22 Step 4:  Vertex 3: try color 1. Neighbor 2 has color 3. Safe.
23          colors = [1, 2, 3, 1]  ** SOLUTION FOUND **
24
25 Verify:
26   Edge (0,1): colors 1 != 2  OK
27   Edge (0,2): colors 1 != 3  OK
28   Edge (1,2): colors 2 != 3  OK
29   Edge (2,3): colors 3 != 1  OK

```

The algorithm found a valid 3-coloring after exploring only 6 nodes (out of a theoretical $3^4 = 81$ leaves).

18.11.3 Finding the Chromatic Number

To find the minimum number of colors, try $m = 1, 2, 3, \dots$ until a valid coloring exists. The first m that works is the chromatic number.

```

1 int find_chromatic_number(int n,
2                           const std::vector<std::pair<int,int>>& edges) {
3     for (int m = 1; m <= n; ++m) {
4         graph_coloring gc(n, edges, m);
5         if (gc.solve()) return m;
6     }
7     return n;
8 }

```

Complexity: worst case $O(m^n)$ for n vertices and m colors. In practice, pruning is effective because adjacent vertices quickly eliminate colors. The chromatic number problem is NP-hard, but backtracking handles graphs with up to 30-40 vertices reasonably.

18.11.4 Applications

Graph coloring has many practical applications:

- **Register allocation:** compilers color variables so that variables alive at the same time get different registers.
- **Map coloring:** coloring regions so that no two adjacent regions share a color (the Four Color Theorem proves 4 colors suffice for planar maps).
- **Scheduling:** coloring time slots so that conflicting tasks (same resource) get different slots.
- **Frequency assignment:** coloring radio stations so that nearby stations use different frequencies.

18.12 Exercises

18.12.1 Drill

1. Draw the state-space tree for 4-Queens. How many nodes are visited by the backtracking algorithm? How many nodes would be visited by brute force (all 4^4 placements)?
1. For subset sum with values [3, 1, 4, 1, 5] and target = 7, trace the backtracking algorithm. Which subsets are found? Which branches are pruned?
1. For the TSP instance with cost matrix:

1	[INF, 10, 15, 20]
2	[10, INF, 35, 25]
3	[15, 35, INF, 30]
4	[20, 25, 30, INF]

Compute the reduced cost matrix bound for the root node. What is the lower bound?

1. What is the size of the state-space tree for:
 - a) n-Queens
 - b) Subset sum with n elements
 - c) TSP with n cities
 - d) Graph coloring on a graph with v vertices and k colors
1. When does backtracking degenerate to brute-force search? Give an example problem and instance where no pruning occurs.

18.12.2 Application

1. Solve the **knight's tour** problem using backtracking on an 8×8 board. Use Warnsdorff's heuristic for the order of moves to speed up the search.

1. Implement **graph coloring** using backtracking. Given a graph and k colors, assign colors to vertices so that no two adjacent vertices share a color. Find the minimum k (chromatic number) for small graphs.
1. Write a **crossword puzzle solver** using backtracking. Given a partially filled crossword grid and a dictionary, fill in all the words.
1. Solve TSP for n up to 15 using branch and bound with the reduced cost matrix bound. Report the solution and the number of nodes visited. Compare with exhaustive search ($n!$).
1. Implement a **magic square** generator using backtracking. Generate all 3×3 magic squares (where rows, columns, and diagonals sum to the same value).
1. Implement branch and bound for the **0/1 knapsack** problem. Use the fractional knapsack as the upper bound. Show that the bound prunes at least some branches for most instances.
1. Implement the **Hamiltonian cycle** problem using backtracking. Find all Hamiltonian cycles in a small graph (≤ 8 vertices).
1. Write a **Sudoku solver** that can solve any 9×9 puzzle. Count the number of backtracks needed for hard puzzles (e.g., those with 17 clues).

18.12.3 Research

1. **(SAT solvers)** Read about SAT solvers and the **DPLL algorithm** — a sophisticated backtracking algorithm for Boolean satisfiability. Implement a simple SAT solver and test it on small CNF formulas.
1. **(Constraint satisfaction)** Research the AC-3 algorithm for arc consistency in constraint satisfaction problems. Incorporate it into the n -Queens solver as a preprocessing step. Does it reduce the search space?
1. **(Branch and bound for integer programming)** Read about LP relaxation and branch and bound for integer linear programming. Implement a simple ILP solver for small instances.
1. **(Parallel backtracking)** Implement a parallel n -Queens solver using `std::thread` or C++20 `std::jthread`. Measure the speedup on a multi-core machine. What is the granularity of the parallel work?
1. **(Estimation via randomized backtracking)** For n -Queens, run the backtracking algorithm with random column ordering and estimate the expected number of nodes visited. Compare with the deterministic (left-to-right) version. Does randomization help?

18.12.4 Additional Exercises

1. Solve 8-Queens by hand using the bitmask backtracking algorithm. For each recursive call, show the values of `cols`, `diag1`, `diag2`, and the `available` bitmask. Count the total number of recursive calls.
1. For the subset sum instance $[2, 3, 5, 7, 8]$ and $\text{target} = 12$, trace the backtracking algorithm showing the pruning. How many nodes are visited? Compare with the DP table size ($n \times \text{target}$).
1. Color the Petersen graph with 3 colors using backtracking. The Petersen graph has 10 vertices and 15 edges. How many backtracks are needed? What is its chromatic number?
1. Solve Sudoku for this puzzle (0 = empty) and count backtracks:

1	0	0	0		2	6	0		7	0	1
2	6	8	0		0	7	0		0	9	0
3	1	9	0		0	0	4		5	0	0
4	-----+-----+-----										
5	8	2	0		1	0	0		0	4	0
6	0	0	4		6	0	2		9	0	0
7	0	5	0		0	0	3		0	2	8
8	-----+-----+-----										
9	0	0	9		3	0	0		0	7	4
10	0	4	0		0	5	0		0	3	6
11	7	0	3		0	1	8		0	0	0

1. For $n = 10$, compare the number of nodes explored by the basic n -Queens backtracking algorithm vs. the bitmask version. Implement both and run benchmarks.
1. Implement a **word search** solver: given an $m \times n$ grid of characters and a dictionary of words, find all words from the dictionary that can be formed by traversing adjacent cells (horizontally, vertically, diagonally) without revisiting a cell.
1. Implement **N-Queens with symmetry breaking**: count only distinct solutions under rotation and reflection. Verify that for $n = 8$, there are 12 fundamental solutions (92 total / 8 symmetries).
1. Solve the **exact cover** problem using Algorithm X with dancing links (DLX). Test on a set cover instance and a Sudoku puzzle. Compare the running time with the basic backtracking Sudoku solver.
1. Implement a **job assignment** solver using branch and bound: given n workers and n jobs with a cost matrix, assign each worker to exactly one job to minimize total cost. Use the Hungarian bound for pruning.
1. Implement a **pentomino tiling** solver: tile a 6×10 rectangle with all 12 pentominoes using backtracking. Each pentomino covers exactly 5 cells.

18.13 References and Further Reading

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapter 34 (NP-completeness), Chapter 35 (approximation).
- Steven S. Skiena, *The Algorithm Design Manual*, 2nd Edition. Springer, 2008.
- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition. Section 6.4 (backtracking).
- Judea Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- Martin Gardner, "Mathematical Games" — Scientific American (columns on n-Queens, magic squares, etc.).
- Donald E. Knuth, "Dancing Links" — Millennial Perspectives in Computer Science, 2000 (exact cover and algorithm X).
- Armin Biere et al. (eds.), *Handbook of Satisfiability*, 2nd Edition. IOS Press, 2021.

Part IV

Specialized and Applied Topics

Probabilistic Data Structures

19.1 Introduction

Not all problems require exact answers. When processing massive data streams, probabilistic data structures provide memory-efficient approximate answers with bounded error.

❧ **Complete compilable implementations of the data structures in this chapter are in code/.**

19.2 Bloom Filters

A **Bloom filter** tests set membership with no false negatives and a controllable false positive rate. It uses k hash functions and a bit array of size m .

19.2.1 Operations

```

1 class bloom_filter {
2 public:
3     bloom_filter(size_t expected_elements, double false_positive_rate =
4         0.01) {
5         // Optimal size and hash count
6         m_ = -expected_elements * std::log(false_positive_rate)
7             / (std::log(2) * std::log(2));
8         k_ = std::max(1, static_cast<int>(std::round(
9             (m_ / expected_elements) * std::log(2))));
10        bits_.resize(m_, false);
11    }
12
13    void insert(std::string_view key) {
14        auto [h1, h2] = hash(key);
15        for (int i = 0; i < k_; ++i) {
16            size_t pos = (h1 + i * h2) % m_;
17            bits_[pos] = true;
18        }
19    }
20
21    bool contains(std::string_view key) const {
22        auto [h1, h2] = hash(key);
23        for (int i = 0; i < k_; ++i) {
24            size_t pos = (h1 + i * h2) % m_;
25            if (!bits_[pos]) return false; // definitely not present
26        }
27        return true;
28    }
29 }

```

```

25     }
26     return true;  // probably present
27 }
28
29 private:
30     // Double hashing
31     std::pair<size_t, size_t> hash(std::string_view key) const {
32         std::hash<std::string_view> h;
33         size_t hval = h(key);
34         return {hval, hval >> 16 | (hval << 16)};
35     }
36
37     size_t m_;  // bits in filter
38     int k_;     // number of hash functions
39     std::vector<bool> bits_;
40 };

```

Properties:

- **No false negatives:** if `contains()` returns false, the key was definitely not inserted
- **False positives:** `contains()` may return true for a key that was not inserted
- **Cannot delete** (without extensions like counting Bloom filters)

Optimal parameters: $m = -n \ln P / (\ln 2)^2$, $k = (m/n) \ln 2$, where n = expected elements, P = target false positive rate.

Application: Cache filtering, spell checkers, database join optimization, cryptocurrency SPV nodes.

19.3 Cuckoo Filters

Library implementation: `cuckoo_filter.h` provides `insert`, `contains`, and `erase`; insertion can fail when the configured table is full.

A **cuckoo filter** (Fan et al., 2014) is a Bloom filter alternative that supports **deletion** with better space efficiency for low false positive rates. The name comes from “cuckoo hashing” — items that collide are displaced to alternate locations, like a cuckoo bird evicting eggs.

19.3.1 Core Idea

Like a Bloom filter, a cuckoo filter stores fingerprints (short hash values) in a fixed-size array. Each element maps to **two candidate buckets** (not k buckets like a Bloom filter). When inserting:

1. Compute the fingerprint f of the key and the two candidate bucket indices i_1, i_2
2. If either bucket has an empty slot, place f there

3. If both buckets are full, **evict** an existing fingerprint to its alternate bucket (the other candidate for that fingerprint)
4. After at most a constant number of evictions (typically 500), declare the filter full and trigger a rebuild

```

1  Bucket 7:  [f1] [f2] [ ] [ ]      Bucket 12: [f3] [f4] [f5] [f6]
2              |                      |
3          i1 = 7                    i2 = 12
4
5  Insert f7 (maps to buckets 7 and 12):
6      Both full -> evict f3 from bucket 12 to its alternate (bucket 7 or
        another)

```

19.3.2 Cuckoo Filter Implementation

```

1  class cuckoo_filter {
2  public:
3      cuckoo_filter(size_t capacity, double target_fpr = 0.01)
4          : buckets_(capacity) {
5          // fingerprint size in bits: ceil(log2(1/fpr)) + 3 for amortized
6          // analysis
7          fp_bits_ = static_cast<int>(std::ceil(std::log2(1.0 / target_fpr))
8          ) + 3;
9          fp_mask_ = (1ULL << fp_bits_) - 1;
10     }
11
12     void insert(std::string_view key) {
13         auto f = fingerprint(key);
14         auto [i1, i2] = bucket_indices(key, f);
15
16         // Try to insert into either bucket
17         if (try_insert(i1, f)) return;
18         if (try_insert(i2, f)) return;
19
20         // Both full -- evict from the first bucket
21         for (int n = 0; n < max_kicks; ++n) {
22             size_t victim_idx = (n % 2 == 0) ? i1 : i2;
23             auto evicted = kickout(victim_idx, f);
24             f = evicted;
25             // Compute alternate bucket for the evicted fingerprint
26             victim_idx = alt_index(victim_idx, evicted);
27             if (try_insert(victim_idx, f)) return;
28         }
29         // If we get here, filter is too full -- rebuild with larger array
30     }
31 }

```

```

30     bool contains(std::string_view key) const {
31         auto f = fingerprint(key);
32         auto [i1, i2] = bucket_indices(key, f);
33         return bucket_contains(i1, f) || bucket_contains(i2, f);
34     }
35
36     void erase(std::string_view key) {
37         auto f = fingerprint(key);
38         auto [i1, i2] = bucket_indices(key, f);
39         if (remove_from_bucket(i1, f)) return;
40         remove_from_bucket(i2, f);
41     }
42
43 private:
44     static constexpr int max_kicks = 500;
45     static constexpr int bucket_size_ = 4;
46
47     struct bucket {
48         uint64_t fingerprints[4] = {0, 0, 0, 0};
49         uint8_t count = 0;
50     };
51
52     std::vector<bucket> buckets_;
53     int fp_bits_;
54     uint64_t fp_mask_;
55
56     uint64_t fingerprint(std::string_view key) const {
57         uint64_t h = std::hash<std::string_view>{}(key);
58         return (h ^ (h >> 32)) & fp_mask_;
59     }
60
61     std::pair<size_t, size_t> bucket_indices(std::string_view key,
62     uint64_t f) const {
63         uint64_t h = std::hash<std::string_view>{}(key);
64         size_t i1 = h % buckets_.size();
65         size_t i2 = alt_index(i1, f);
66         return {i1, i2};
67     }
68
69     // Partial-key hashing: derive the alternate index from the bucket
70     index
71     // and fingerprint alone (no need to re-hash the original key)
72     size_t alt_index(size_t index, uint64_t f) const {
73         return index ^ static_cast<size_t>(
74             ((f >> 16) ^ (f * 0x517cc1b727220a95ULL)) % buckets_.size());
75     }
76
77     bool try_insert(size_t idx, uint64_t f) {

```



```

76     if (buckets_[idx].count < bucket_size_) {
77         buckets_[idx].fingerprints[buckets_[idx].count++] = f;
78         return true;
79     }
80     return false;
81 }
82
83 uint64_t kickout(size_t idx, uint64_t new_f) {
84     int slot = buckets_[idx].count - 1;
85     uint64_t old_f = buckets_[idx].fingerprints[slot];
86     buckets_[idx].fingerprints[slot] = new_f;
87     return old_f;
88 }
89
90 bool bucket_contains(size_t idx, uint64_t f) const {
91     for (int i = 0; i < buckets_[idx].count; ++i)
92         if (buckets_[idx].fingerprints[i] == f) return true;
93     return false;
94 }
95
96 bool remove_from_bucket(size_t idx, uint64_t f) {
97     for (int i = 0; i < buckets_[idx].count; ++i) {
98         if (buckets_[idx].fingerprints[i] == f) {
99             buckets_[idx].fingerprints[i] =
100                 buckets_[idx].fingerprints[--buckets_[idx].count];
101             return true;
102         }
103     }
104     return false;
105 }
106 };

```

The key insight is **partial-key hashing**: the alternate bucket index is derived from the current index and the fingerprint alone, without re-hashing the original key. This means evictions don't require knowing the original key — they just swap fingerprints between two buckets.

19.3.3 Cuckoo Filter vs Bloom Filter

Property	Bloom Filter	Cuckoo Filter
Space at 1% FPR	9.6 bits/elem	8.0 bits/elem
Space at 0.1% FPR	14.4 bits/elem	12.0 bits/elem
Deletion	No (without counting)	Yes
Lookup speed	k hash probes	2 memory accesses
Cache friendliness	Moderate	High (2 buckets)

Cuckoo filters use less space because each bucket stores 4 fingerprints (32+ bits) rather than individual bits — better utilization of cache lines. The trade-off: cuckoo filters can fail to insert when the

array is too full, requiring a rebuild.

When to use which:

- **Bloom filter:** When deletions are not needed, or when the element count is unknown and may grow without bound
- **Cuckoo filter:** When deletions are needed, or when space is tight and the maximum element count is known

19.4 Count-Min Sketch

Count approximate frequencies of items in a stream using a 2D array of counters and hash functions.

```

1 class count_min_sketch {
2 public:
3     count_min_sketch(double epsilon, double delta) {
4         // width = ceil(e / epsilon), depth = ceil(ln(1 / delta))
5         width_ = static_cast<size_t>(std::ceil(std::exp(1) / epsilon));
6         depth_ = static_cast<size_t>(std::ceil(std::log(1.0 / delta)));
7         counts_.resize(depth_, std::vector<size_t>(width_, 0));
8     }
9
10    void add(std::string_view key, size_t count = 1) {
11        for (size_t i = 0; i < depth_; ++i) {
12            size_t pos = hash_i(key, i) % width_;
13            counts_[i][pos] += count;
14        }
15    }
16
17    size_t estimate(std::string_view key) const {
18        size_t min_val = SIZE_MAX;
19        for (size_t i = 0; i < depth_; ++i) {
20            size_t pos = hash_i(key, i) % width_;
21            min_val = std::min(min_val, counts_[i][pos]);
22        }
23        return min_val;
24    }
25
26 private:
27     size_t hash_i(std::string_view key, size_t i) const {
28         return std::hash<std::string_view>{}(key) + i * 0
29         x9e3779b97f4a7c15ULL;
30     }
31
32     size_t width_, depth_;
33     std::vector<std::vector<size_t>> counts_;
34 };

```

19.5 Heavy Hitters (Misra-Gries)

Library implementation: `heavy_hitters.h` provides bounded-space candidate tracking via `dsa::misra_gries`. a second pass is needed for exact frequencies.

Find the most frequent items in a stream — items that appear more than $\frac{n}{k}$ times. The Misra-Gries algorithm uses $O(k)$ counters to identify all heavy hitters.

19.5.1 The Idea

Maintain at most $k-1$ (item, count) pairs. For each new item:

- If it matches an existing entry, increment its count.
- If not and we have fewer than $k-1$ entries, add it with count 1.
- If we have $k-1$ entries, decrement all counts by 1 and remove any entries with count 0.

After processing the stream, any item with count > 0 is a candidate. Verify candidates by a second pass.

19.5.2 Implementation

```

1  template <typename Key>
2  class misra_gries {
3  public:
4      explicit misra_gries(int k) : k_(k) {}
5
6      void add(const Key& key) {
7          // If key already tracked, increment
8          auto it = counters_.find(key);
9          if (it != counters_.end()) {
10             it->second += 1;
11             return;
12         }
13
14         // If room, add new entry
15         if (static_cast<int>(counters_.size()) < k_ - 1) {
16             counters_[key] = 1;
17             return;
18         }
19
20         // Decrement all, remove zeros
21         for (auto it = counters_.begin(); it != counters_.end(); ) {
22             it->second -= 1;
23             if (it->second == 0) it = counters_.erase(it);
24             else ++it;
25         }

```

```

26     }
27
28     std::vector<std::pair<Key, size_t>> heavy_hitters(size_t min_count)
29     const {
30         std::vector<std::pair<Key, size_t>> result;
31         for (const auto& [key, count] : counters_) {
32             if (count >= min_count) result.push_back({key, count});
33         }
34         return result;
35     }
36
37     const std::unordered_map<Key, size_t>& counters() const { return
38     counters_; }
39
40 private:
41     int k_;
42     std::unordered_map<Key, size_t> counters_;
43 };

```

19.5.3 Complexity

- **Space:** $O(k)$ counters
- **Update:** $O(1)$ amortized
- **Guarantee:** any item with frequency $> \frac{n}{k}$ is among the candidates. False positives possible — verify with a second pass.

19.5.4 Applications

- **Network security:** detect DDoS attacks by finding IPs sending $> \frac{n}{k}$ requests
- **Query analysis:** find most popular search queries at Google/Bing scale
- **Log analytics:** top error codes in server logs
- **Clickstream analysis:** most visited pages in the last hour

Relationship to Count-Min Sketch: Count-Min Sketch gives approximate frequency for any item. Misra-Gries gives exact identification of heavy hitters with less space. For heavy hitters specifically, Misra-Gries is preferred. For arbitrary frequency queries, Count-Min is better.

19.6 HyperLogLog

Estimate the number of distinct elements in a large stream using $O(\log \log n)$ space.

```

1 class hyperloglog {
2 public:
3     hyperloglog(int precision = 14) : precision_(precision) {
4         m_ = 1 << precision; // number of registers
5         registers_.resize(m_, 0);
6     }
7
8     void add(std::string_view key) {
9         auto hash = std::hash<std::string_view>{}(key);
10        size_t idx = hash >> (64 - precision_); // first p bits
11        uint8_t zeros = count_leading_zeros(hash << precision_);
12        registers_[idx] = std::max(registers_[idx], zeros + 1);
13    }
14
15    double estimate() const {
16        double sum = 0.0;
17        size_t zeros = 0;
18        for (uint8_t r : registers_) {
19            sum += 1.0 / (1 << r);
20            if (r == 0) ++zeros;
21        }
22        double e = alpha_m_ * m_ * m_ / sum;
23
24        // Small range correction
25        if (e <= 2.5 * m_) {
26            if (zeros > 0) e = m_ * std::log(static_cast<double>(m_) /
27            zeros);
28        }
29        return e;
30    }
31 private:
32     static uint8_t count_leading_zeros(size_t x) {
33         return __builtin_clzll(x);
34     }
35
36     int precision_;
37     size_t m_;
38     std::vector<uint8_t> registers_;
39     static constexpr double alpha_m_ = 0.7213 / (1 + 1.079 / (1 << 14));
40 };

```

19.7 Quantile Estimation

Estimate the median or any quantile from a data stream without storing all elements. A **quantile** q is the value below which q fraction of elements fall (e.g., median = 0.5-quantile, P99 = 0.99-quantile).

19.7.1 Greenwald-Khanna Sketch

The GK sketch maintains a set of summary tuples (v_i, g_i, Δ_i) where v_i is a value, g_i is the maximum gap to the next summary value, and Δ_i is the maximum gap to the previous summary value.

```

1  class quantile_estimator {
2  public:
3      explicit quantile_estimator(double epsilon) : epsilon_(epsilon) {}
4
5      void add(double value) {
6          ++total_count_;
7          summary_.push_back({value, 1, 0});
8
9          // Merge adjacent tuples to maintain size bound
10         if (summary_.size() > static_cast<size_t>(1.0 / epsilon_)) {
11             merge_tuples();
12         }
13     }
14
15     double query(double phi) const {
16         // Query the phi-quantile (e.g., 0.5 for median)
17         if (summary_.empty()) return 0.0;
18
19         double rank = phi * total_count_;
20         size_t cumulative = 0;
21
22         for (size_t i = 0; i < summary_.size(); ++i) {
23             cumulative += summary_[i].g;
24             if (cumulative + summary_[i].delta >= static_cast<size_t>(rank
25 ) &&
26             cumulative <= static_cast<size_t>(rank)) {
27                 return summary_[i].value;
28             }
29         }
30         return summary_.back().value;
31     }
32 private:
33     struct tuple {
34         double value;
35         size_t g;      // gap to next
36         size_t delta;  // max gap to previous
37     };
38
39     void merge_tuples() {
40         std::vector<tuple> merged;
41         for (size_t i = 0; i + 1 < summary_.size(); ++i) {
42             if (summary_[i].g + summary_[i+1].g +
43                 summary_[i+1].delta <= 2 * epsilon_ * total_count_) {

```

```

44         // Merge: skip this tuple
45         merged.push_back({summary_[i].value,
46             summary_[i].g + summary_[i+1].g,
47             summary_[i+1].delta});
48         ++i; // skip next
49     } else {
50         merged.push_back(summary_[i]);
51     }
52 }
53 if (!merged.empty()) merged.back().delta = 0;
54 summary_ = std::move(merged);
55 }
56
57 double epsilon_;
58 size_t total_count_ = 0;
59 std::vector<tuple> summary_;
60 };

```

19.7.2 Complexity

- **Space:** $O(\frac{1}{\epsilon} \log(\epsilon n))$ tuples
- **Update:** $O(\frac{1}{\epsilon})$ amortized
- **Query:** $O(\frac{1}{\epsilon})$
- **Accuracy:** returns value within ϵn of true rank

19.7.3 Applications

- **Monitoring dashboards:** P50, P90, P99 latency percentiles in real time
- **Database query optimization:** estimate result distribution without scanning all rows
- **Adaptive load balancing:** route requests based on recent response time distribution
- **SLA monitoring:** are 99% of requests served within 100ms?

Alternatives: t-digest (better accuracy for extreme quantiles), KLL sketch (better space-accuracy tradeoff). The GK sketch is the classic choice; t-digest is more common in production (used by Elasticsearch, Apache Flink).

19.8 Reservoir Sampling

Select k random elements from a stream of unknown length.

```

1  std::vector<int> reservoir_sample(std::span<const int> stream, size_t k) {
2      std::vector<int> reservoir(stream.begin(), stream.begin() + k);
3      std::mt19937 rng(std::random_device{}());
4
5      for (size_t i = k; i < stream.size(); ++i) {
6          size_t j = std::uniform_int_distribution<size_t>(0, i)(rng);
7          if (j < k) reservoir[j] = stream[i];
8      }
9      return reservoir;
10 }

```

Each element has probability k/n of being in the final sample.

19.9 Sliding Window Model

Most streaming algorithms assume you see the entire stream. In practice, you often need statistics over only the **last N elements** — a sliding window. The challenge: maintain a summary using $O(\text{polylog } N)$ space while elements expire.

19.9.1 Exponential Histogram

The exponential histogram (Datar et al., 2002) maintains approximate counts over a sliding window of size W . It uses $O(\frac{1}{\epsilon} \log W)$ space to give $(1 \pm \epsilon)$ -approximate counts.

The idea: divide the stream into buckets of exponentially increasing sizes. Each bucket covers a contiguous segment and stores its count. The smallest bucket covers 1 element, the next covers 2, then 4, 8, etc.

```

1  template <typename T>
2  class sliding_window_counter {
3  public:
4      explicit sliding_window_counter(size_t window_size, double epsilon =
5          0.01)
6          : window_(window_size), epsilon_(epsilon) {}
7
8      void add(const T& item, size_t timestamp) {
9          // Add to bucket 0
10         buckets_[0].push_back({item, 1});
11         total_ += 1;
12
13         // Merge buckets with same size if needed
14         compact();
15
16         // Remove expired elements
17         while (!buckets_.empty() && buckets_.front().start_time +
18             buckets_.front().count <= timestamp - window_) {
19             total_ -= buckets_.front().count;
20             buckets_.pop_front();
21         }
22     }
23 };

```



```

20     }
21 }
22
23 size_t count() const { return total_; }
24
25 // Approximate count of item in current window
26 size_t approximate_count(const T& item) const {
27     size_t result = 0;
28     for (const auto& bucket : buckets_) {
29         for (const auto& [val, cnt] : bucket.items) {
30             if (val == item) result += cnt;
31         }
32     }
33     return result;
34 }
35
36 private:
37     struct bucket {
38         std::deque<std::pair<T, int>> items;
39         size_t count = 0;
40         size_t start_time = 0;
41     };
42
43 void compact() {
44     // Merge consecutive buckets of same size
45     for (size_t i = 0; i + 1 < buckets_.size(); ++i) {
46         while (buckets_[i].count == buckets_[i+1].count &&
47             buckets_[i].count > 0) {
48             // Merge bucket i+1 into bucket i
49             for (auto& item : buckets_[i+1].items)
50                 buckets_[i].items.push_back(item);
51             buckets_[i].count += buckets_[i+1].count;
52             buckets_.erase(buckets_.begin() + i + 1);
53         }
54     }
55 }
56
57 size_t window_;
58 double epsilon_;
59 std::deque<bucket> buckets_;
60 size_t total_ = 0;
61 };

```

19.9.2 Applications

- **Network monitoring:** count packets/bytes in the last N seconds
- **Rate limiting:** sliding window counters for API rate limits (Redis, NGINX)

- **Log analysis:** error rate over the last hour, not the entire log
- **Sensor networks:** recent average temperature, not lifetime average

Complexity: $O(\frac{1}{\epsilon} \log W)$ space, $O(1)$ amortized per update.

19.10 Consistent Hashing

Distribute keys across a changing set of servers while minimizing remapping when servers join or leave. Each server is mapped to points on a hash ring, and each key is assigned to the nearest clockwise server.

```

1 class consistent_hash_ring {
2 public:
3     void add_server(std::string_view name, int virtual_nodes = 100) {
4         for (int i = 0; i < virtual_nodes; ++i) {
5             auto h = hash(std::string(name) + ":" + std::to_string(i));
6             ring_[h] = name;
7         }
8     }
9
10    void remove_server(std::string_view name) {
11        std::erase_if(ring_, [&](const auto& p) {
12            return p.second == name;
13        });
14    }
15
16    std::string_view get_server(std::string_view key) const {
17        if (ring_.empty()) return "";
18        auto it = ring_.lower_bound(hash(key));
19        if (it == ring_.end()) it = ring_.begin();
20        return it->second;
21    }
22
23 private:
24     static uint64_t hash(std::string_view s) {
25         return std::hash<std::string_view>{}(s);
26     }
27
28     std::map<uint64_t, std::string> ring_;
29 };

```

When a server is removed, only its keys migrate to the next server; every other key stays put. Virtual nodes improve load balance by spreading each server across many ring positions.

19.11 Locality-Sensitive Hashing

Approximate nearest-neighbor search in high dimensions by hashing inputs so that similar items collide with high probability.

The basic LSH family for cosine similarity (simhash):

- Generate k random hyperplanes
- For each item, compute the sign of each hyperplane dot product — the k -bit signature captures the angular neighborhood

```

1 class simhash {
2 public:
3     simhash(size_t bits, size_t dim)
4         : bits_(bits), planes_(bits, std::vector<double>(dim)) {
5         std::mt19937 rng(std::random_device{}());
6         std::normal_distribution<double> dist(0.0, 1.0);
7         for (auto& v : planes_)
8             for (auto& x : v) x = dist(rng);
9     }
10
11     uint64_t signature(std::span<const double> vec) const {
12         uint64_t sig = 0;
13         for (size_t i = 0; i < bits_; ++i) {
14             double dot = 0.0;
15             for (size_t j = 0; j < vec.size(); ++j)
16                 dot += planes_[i][j] * vec[j];
17             if (dot > 0) sig |= (1ULL << i);
18         }
19         return sig;
20     }
21
22     // Hamming distance on signatures approximates cosine distance
23     static double cosine_similarity(uint64_t a, uint64_t b) {
24         return 1.0 - static_cast<double>(std::popcount(a ^ b)) / 64.0;
25     }
26
27 private:
28     size_t bits_;
29     std::vector<std::vector<double>> planes_;
30 };

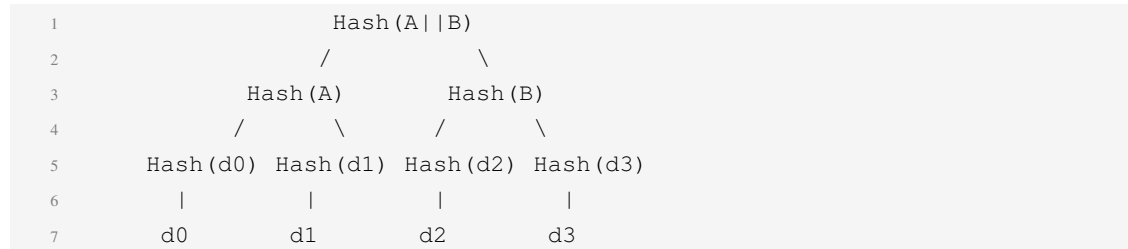
```

19.12 Merkle Tree (Hash Tree)

A **Merkle tree** is a binary tree where each leaf stores the hash of a data block and each internal node stores the hash of its two children. It enables efficient verification that a specific block belongs to a

large dataset without downloading the entire dataset.

19.12.1 Structure



The **root hash** uniquely represents the entire dataset. Changing any single data block changes the root hash, making tampering immediately detectable.

19.12.2 Operations

```

1  #include <vector>
2  #include <string>
3  #include <cstdint>
4  #include <functional>
5  #include <stdexcept>
6
7  class merkle_tree {
8  public:
9      using hash_fn = std::function<std::string(const std::string&)>;
10
11      merkle_tree(std::vector<std::string> data, hash_fn h)
12          : hash_(std::move(h)), data_(std::move(data)) {
13          size_t n = data_.size();
14          if (n == 0)
15              throw std::invalid_argument("empty data");
16          // Round up to next power of 2
17          size_t leaves = 1;
18          while (leaves < n) leaves *= 2;
19          tree_.resize(2 * leaves, "");
20          // Compute leaf hashes
21          for (size_t i = 0; i < leaves; ++i) {
22              if (i < n)
23                  tree_[leaves + i] = hash_(data_[i]);
24              else
25                  tree_[leaves + i] = hash_(""); // padding
26          }
27          // Build internal nodes bottom-up
28          for (size_t i = leaves - 1; i >= 1; --i)
29              tree_[i] = hash_(tree_[2 * i] + tree_[2 * i + 1]);
30          leaves_ = leaves;
  
```

```

31     }
32
33     std::string get_root_hash() const {
34         return tree_[1];
35     }
36
37     // Merkle proof: list of sibling hashes from leaf to root
38     std::vector<std::pair<std::string, bool>>
39     get_proof(size_t index) const {
40         std::vector<std::pair<std::string, bool>> proof;
41         size_t pos = leaves_ + index;
42         while (pos > 1) {
43             bool is_left = (pos % 2 == 1);
44             size_t sibling = is_left ? pos - 1 : pos + 1;
45             proof.emplace_back(tree_[sibling], is_left);
46             pos /= 2;
47         }
48         return proof;
49     }
50
51     // Verify that data_block is at index using the proof
52     static bool verify_proof(
53         hash_fn h,
54         const std::string& data_block,
55         size_t index,
56         const std::string& expected_root,
57         const std::vector<std::pair<std::string, bool>>& proof)
58     {
59         std::string current = h(data_block);
60         for (const auto& [sibling_hash, is_left] : proof) {
61             if (is_left)
62                 current = h(sibling_hash + current);
63             else
64                 current = h(current + sibling_hash);
65         }
66         return current == expected_root;
67     }
68
69 private:
70     hash_fn hash_;
71     std::vector<std::string> data_;
72     std::vector<std::string> tree_;
73     size_t leaves_;
74 };

```

19.12.3 Complexity

Operation	Cost
Build tree	$O(n)$
Root hash	$O(1)$
Generate proof	$O(\log n)$
Verify proof	$O(\log n)$

19.12.4 Applications

- **Blockchain:** Bitcoin and Ethereum use Merkle trees to summarize all transactions in a block. A light client can verify a single transaction with a $\log n$ proof instead of downloading every transaction.
- **Git:** Git commit objects contain a Merkle tree of the repository file tree, enabling efficient diff and integrity verification.
- **Distributed databases:** Cassandra and DynamoDB use Merkle trees to detect and reconcile inconsistencies between replicas during anti-entropy sync.

19.13 Quotient Filter

A **quotient filter** is a space-efficient hash table that stores fingerprints using quotienting. Unlike a Bloom filter, it supports **counting and deletion**. A key k is mapped to a fingerprint f using a hash function, then split into quotient $q = f/2^r$ and remainder $r = f \bmod 2^r$.

19.13.1 Structure

1	Quotient		Remainder slots
2	-----+		-----
3	0		[_] [_] [_] [_]
4	1		[b1] [_] [_] [_]
5	2		[a0] [a1] [_] [_]
6	3		[c0] [_] [_] [_]
7	.		

Collisions with the same quotient form a **run** stored in consecutive slots using linear probing. A small auxiliary array tracks the start of each run.

19.13.2 Operations

```
1 #include <vector>
2 #include <cstdint>
```

```

3 #include <functional>
4 #include <optional>
5
6 class quotient_filter {
7 public:
8     quotient_filter(size_t capacity, size_t fingerprint_bits = 8)
9         : r_(fingerprint_bits),
10           size_(1ULL << (capacity > 0 ? compute_q_bits(capacity,
11 fingerprint_bits) : 4)),
12           mask_(size_ - 1),
13           fingerprints_(size_, 0),
14           occupied_(size_, 0),
15           continuations_(size_, 0),
16           counts_(size_, 0),
17           size__(0)
18     {
19         q_bits_ = compute_q_bits(capacity, fingerprint_bits);
20     }
21
22     bool insert(uint64_t key) {
23         uint64_t f = fingerprint(key);
24         uint64_t q = f >> r_;
25         uint64_t rem = f & ((1ULL << r_) - 1);
26         if (counts_[q] == 0) {
27             // Empty cluster -- place directly
28             size_t pos = q;
29             while (occupied_[pos]) pos = (pos + 1) & mask_;
30             place(pos, q, rem);
31             return true;
32         }
33         // Find the run for quotient q
34         size_t run_start = find_run_start(q);
35         // Check for duplicate
36         size_t pos = run_start;
37         for (size_t i = 0; i < counts_[q]; ++i) {
38             if (fingerprints_[pos] == rem) return false; // already
39             present
40             pos = (pos + 1) & mask_;
41         }
42         // Insert at end of run
43         place(pos, q, rem);
44         return true;
45     }
46
47     bool lookup(uint64_t key) const {
48         uint64_t f = fingerprint(key);
49         uint64_t q = f >> r_;
50         uint64_t rem = f & ((1ULL << r_) - 1);

```

```

49     if (counts_[q] == 0) return false;
50     size_t pos = find_run_start(q);
51     for (size_t i = 0; i < counts_[q]; ++i) {
52         if (fingerprints_[pos] == rem) return true;
53         pos = (pos + 1) & mask_;
54     }
55     return false;
56 }
57
58 bool remove(uint64_t key) {
59     uint64_t f = fingerprint(key);
60     uint64_t q = f >> r_;
61     uint64_t rem = f & ((1ULL << r_) - 1);
62     if (counts_[q] == 0) return false;
63     size_t pos = find_run_start(q);
64     for (size_t i = 0; i < counts_[q]; ++i) {
65         if (fingerprints_[pos] == rem) {
66             // Shift run left and compact
67             compact(pos, q);
68             return true;
69         }
70         pos = (pos + 1) & mask_;
71     }
72     return false;
73 }
74
75 size_t size() const { return size_; }
76
77 private:
78     size_t r_, q_bits_, size_, mask_;
79     size_t size_;
80     std::vector<uint64_t> fingerprints_;
81     std::vector<bool> occupied_;
82     std::vector<bool> continuations_;
83     std::vector<size_t> counts_;
84
85     static size_t compute_q_bits(size_t cap, size_t r) {
86         size_t total_slots = cap * 2; // 50% load factor
87         size_t q = 0;
88         while ((1ULL << q) < total_slots) ++q;
89         return q;
90     }
91
92     uint64_t fingerprint(uint64_t key) const {
93         // SplitMix64-style hash
94         key ^= key >> 30;
95         key *= 0xbf58476d1ce4e5b9ULL;
96         key ^= key >> 27;

```



```

97     key *= 0x94d049bb133111ebULL;
98     key ^= key >> 31;
99     return key & ((1ULL << (q_bits_ + r_)) - 1);
100 }
101
102 size_t find_run_start(uint64_t q) const {
103     // Count occupied slots before q
104     size_t count = 0;
105     for (size_t i = 0; i < q; ++i)
106         if (occupied_[i]) ++count;
107     // Scan from q to find the (count+1)-th occupied cluster
108     size_t pos = q;
109     size_t seen = 0;
110     while (seen < count) {
111         pos = (pos + 1) & mask_;
112         if (occupied_[pos]) ++seen;
113     }
114     return (pos + 1) & mask_;
115 }
116
117 void place(size_t pos, uint64_t q, uint64_t rem) {
118     fingerprints_[pos] = rem;
119     occupied_[q] = true;
120     ++counts_[q];
121     ++size__;
122 }
123
124 void compact(size_t pos, uint64_t q) {
125     // Shift elements left to fill the gap
126     size_t next = (pos + 1) & mask_;
127     while (counts_[q] > 1 || (next != pos && occupied_[next])) {
128         fingerprints_[pos] = fingerprints_[next];
129         pos = next;
130         next = (next + 1) & mask_;
131     }
132     // Clear the last slot
133     occupied_[q] = false;
134     if (counts_[q] > 0) --counts_[q];
135     --size__;
136 }
137 };

```

19.13.3 Complexity

Operation	Cost
Insert	$O(1)$ amortized
Lookup	$O(1)$ amortized
Delete	$O(1)$ amortized
Space	$O(n)$ fingerprints

19.13.4 Comparison with Bloom Filter

Property	Bloom Filter	Quotient Filter
Supports deletion	No	Yes
Space efficiency	Moderate	Higher (smaller fingerprint)
Cache performance	Poor	Good (sequential probing)
False positives	Yes	Yes (via hash collision)

The quotient filter’s sequential memory layout gives it better cache locality than a Bloom filter of equivalent size, making it preferred when deletion support is needed.

19.14 MinHash

MinHash estimates the **Jaccard similarity** between two sets without comparing all elements. Given sets A and B , the Jaccard similarity is $J(A, B) = |A \cap B| / |A \cup B|$. MinHash uses k independent hash functions; for each function, the minimum hash value over a set is computed, producing a **signature** of k values.

19.14.1 Structure

```

1 Set A = {a, c, d, f, g}   Set B = {b, c, e, f, g}
2
3 h1: min(h1(A)) = h1(f)   min(h1(B)) = h1(b)
4 h2: min(h2(A)) = h2(c)   min(h2(B)) = h2(c)
5 h3: min(h3(A)) = h3(d)   min(h3(B)) = h3(e)
6
7 Sig(A) = [h1(f), h2(c), h3(d)]
8 Sig(B) = [h1(b), h2(c), h3(e)]
9 Matches: 1 out of 3 => J estimate = 0.33

```

For each hash function h_i , $P[\min(h_i(A)) = \min(h_i(B))] = J(A, B)$. The fraction of matching entries in two signatures is an unbiased estimator of the Jaccard similarity.

19.14.2 Operations

```

1  #include <vector>
2  #include <unordered_set>
3  #include <cstdint>
4  #include <random>
5  #include <algorithm>
6  #include <numeric>
7
8  class minhash {
9  public:
10     minhash(size_t num_hashes, size_t seed = 42)
11         : k_(num_hashes), hashes_(num_hashes)
12     {
13         // Generate k independent hash function parameters
14         std::mt19937_64 rng(seed);
15         for (size_t i = 0; i < k_; ++i) {
16             uint64_t a = rng(), b = rng();
17             hashes_[i] = [a, b](uint64_t x) -> uint64_t {
18                 // 64-bit multiply-shift hash
19                 return a * x + b;
20             };
21         }
22     }
23
24     // Compute MinHash signature for a set of elements
25     std::vector<uint64_t> compute_signature(
26         const std::unordered_set<std::string>& set) const
27     {
28         std::vector<uint64_t> sig(k_, UINT64_MAX);
29         for (const auto& elem : set) {
30             uint64_t h = std::hash<std::string>{}(elem);
31             for (size_t i = 0; i < k_; ++i) {
32                 uint64_t val = hashes_[i](h);
33                 sig[i] = std::min(sig[i], val);
34             }
35         }
36         return sig;
37     }
38
39     // Estimate Jaccard similarity from two signatures
40     static double estimate_jaccard(
41         const std::vector<uint64_t>& sig_a,
42         const std::vector<uint64_t>& sig_b)
43     {
44         size_t matches = 0;
45         size_t k = sig_a.size();
46         for (size_t i = 0; i < k; ++i)

```

```

47         if (sig_a[i] == sig_b[i]) ++matches;
48         return static_cast<double>(matches) / k;
49     }
50
51     size_t num_hashes() const { return k_; }
52
53 private:
54     size_t k_;
55     std::vector<std::function<uint64_t (uint64_t)>> hashes_;
56 };

```

19.14.3 Complexity

Operation	Cost
Compute signature	$O(k \cdot n)$ where $n = S $
Estimate Jaccard	$O(k)$
Signature size	$O(k)$

Increasing k reduces the variance of the estimate. With $k = 200$ the standard error is approximately $1/\sqrt{k} \approx 0.07$, sufficient for most similarity tasks.

19.14.4 Applications

- **Near-duplicate detection:** Web crawlers use MinHash to detect pages with high Jaccard similarity, filtering redundant content at scale.
- **Recommendation systems:** Users with similar item sets (high Jaccard) receive similar recommendations; MinHash makes this comparison sublinear.
- **Locality-sensitive hashing (LSH):** MinHash signatures are banded and hashed to find *candidate pairs* in $O(n)$ expected time, enabling subquadratic all-pairs similarity search.

19.15 HyperLogLog

HyperLogLog estimates the number of distinct elements in a stream using $O(m)$ registers (typically $m = 2^p$ for $10 \leq p \leq 16$). The key insight is that a uniformly distributed hash produces leading zeros with probability 2^{-j} ; the maximum number of leading zeros across m buckets gives a raw estimate, which is then corrected using the harmonic mean.

19.15.1 Core Idea

Given a 64-bit hash function, for each element:

1. Hash the element: $h = \text{hash}(x)$.

2. Use the first p bits to select one of $m = 2^p$ buckets.
3. In the remaining $64 - p$ bits, count the leading zeros $\rho(w)$ (plus 1).
4. Store $\max(\rho(w))$ in the selected bucket.

The raw cardinality estimate is:

$$E = \alpha_m \cdot m^2 \cdot \left(\sum_{j=1}^m 2^{-M[j]} \right)^{-1}$$

where $M[j]$ is the register value for bucket j and α_m is a bias-correction constant. For $m \geq 128$, $\alpha_m \approx 0.7213 / (1 + 1.079/m)$.

A linear counting correction is applied when $E \leq 2.5 \cdot m$: count the number of zero registers V and use $E' = m \cdot \ln(m/V)$ instead.

19.15.2 Standard Error

The standard error of HyperLogLog is approximately $1.04/\sqrt{m}$. With $p = 14$ ($m = 16384$ registers, 20 KB), the standard error is $\approx 0.81\%$, giving approximately 2.5% accuracy at the 95% confidence level.

Listing 19.1. HyperLogLog cardinality estimator

```

1 #include <array>
2 #include <cstdint>
3 #include <cmath>
4 #include <algorithm>
5 #include <bit>
6
7 class HyperLogLog {
8 public:
9     explicit HyperLogLog(uint8_t p)
10         : p_(p), m_(1u << p), alpha_(0.7213 / (1.0 + 1.079 / m_))
11     {
12         M_.fill(0);
13     }
14
15     void add(uint64_t hash) {
16         uint32_t idx = hash >> (64 - p_);
17         uint64_t w = hash << p_ | (1ULL << (p_ - 1));
18         uint8_t rho = static_cast<uint8_t>(
19             std::countl_zero(w) + 1);
20         M_[idx] = std::max(M_[idx], rho);
21     }
22
23     double estimate() const {
24         double sum = 0.0;
25         int zeros = 0;

```

```

26     for (int j = 0; j < m_; ++j) {
27         sum += std::pow(2.0, -M_[j]);
28         if (M_[j] == 0) ++zeros;
29     }
30     double E = alpha_ * m_ * m_ / sum;
31     if (E <= 2.5 * m_ && zeros > 0) {
32         E = m_ * std::log(static_cast<double>(m_) / zeros);
33     }
34     if (E > (1ULL << 32) / 30.0) {
35         E = -std::pow(2.0, 32) *
36             std::log1p(-E / std::pow(2.0, 32));
37     }
38     return E;
39 }
40
41 void merge(const HyperLogLog& other) {
42     for (int j = 0; j < m_; ++j)
43         M_[j] = std::max(M_[j], other.M_[j]);
44 }
45
46 size_t memory_bytes() const {
47     return m_ + 2 * sizeof(uint8_t) + sizeof(int);
48 }
49
50 private:
51     uint8_t p_;
52     int m_;
53     double alpha_;
54     std::array<uint8_t, 1 << 16> M_;
55 };

```

19.15.3 Complexity

Operation	Cost
Add element	$O(1)$
Estimate cardinality	$O(m)$
Merge two structures	$O(m)$
Memory	$O(m)$ bytes (typically 2–64 KB)

19.15.4 Applications

- **Network monitoring:** Estimate unique IP addresses in a time window without storing them all.
- **Database query planning:** Cardinality estimates for JOIN and GROUP BY operations drive cost-based optimizers.

- **Analytics:** Count unique users, unique sessions, or unique search queries in large-scale event streams.

19.16 Reservoir Sampling

Reservoir sampling selects a uniform random sample of k elements from a stream of n elements where n is unknown or too large to hold in memory. Algorithm R (Vitter, 1985) is the simplest approach.

19.16.1 Algorithm R

Maintain a reservoir R of size k . For the i -th element ($i \geq 1$):

1. If $i \leq k$, insert the element into R .
2. If $i > k$, generate a random integer j uniformly from $\{1, 2, \dots, i\}$. If $j \leq k$, replace $R[j - 1]$ with the new element.

The probability that any element ends up in the sample is exactly k/n , proven by induction on i .

19.16.2 Correctness Argument

At step $i > k$:

$$\Pr[\text{element } i \text{ is in } R] = \frac{k}{i}$$

For element $j \leq i$ to survive step i :

$$\Pr[\text{element } j \text{ survives step } i] = 1 - \frac{1}{i} = \frac{i-1}{i}$$

After n steps, the survival probability is:

$$\prod_{i=j+1}^n \frac{i-1}{i} = \frac{j}{n}$$

Thus each element has probability k/n of being selected.

Listing 19.2. Reservoir sampling (Algorithm R)

```

1 #include <vector>
2 #include <random>
3 #include <iterator>
4
5 template <typename Iter>
6 std::vector<typename std::iterator_traits<Iter>::value_type>
7 reservoir_sample(Iter begin, Iter end, size_t k,
8                 std::mt19937& rng)
9 {
10     std::vector<typename std::iterator_traits<Iter>::value_type>

```

```

11     R;
12     R.reserve(k);
13
14     size_t i = 0;
15     for (auto it = begin; it != end; ++it, ++i) {
16         if (i < k) {
17             R.push_back(*it);
18         } else {
19             std::uniform_int_distribution<size_t>
20                 dist(0, i);
21             size_t j = dist(rng);
22             if (j < k)
23                 R[j] = *it;
24         }
25     }
26     return R;
27 }
28
29 template <typename T>
30 class StreamSampler {
31 public:
32     explicit StreamSampler(size_t k, uint64_t seed = 42)
33         : k_(k), rng_(seed), n_(0) {
34         R_.reserve(k);
35     }
36
37     void add(const T& value) {
38         if (n_ < k_) {
39             R_.push_back(value);
40         } else {
41             std::uniform_int_distribution<size_t>
42                 dist(0, n_);
43             size_t j = dist(rng_);
44             if (j < k_)
45                 R_[j] = value;
46         }
47         ++n_;
48     }
49
50     const std::vector<T>& sample() const { return R_; }
51     size_t total_count() const { return n_; }
52
53 private:
54     size_t k_;
55     std::mt19937 rng_;
56     uint64_t n_;
57     std::vector<T> R_;
58 };

```


19.16.3 Complexity

Operation	Cost
Add element	$O(1)$
Memory	$O(k)$
Space	$O(k)$ words

19.16.4 Applications

- **Data analytics:** Sampling from massive datasets for A/B testing or exploratory analysis.
- **Stream processing:** Selecting representative queries from a log stream for debugging.
- **Machine learning:** Creating balanced training sets from imbalanced streams.

19.17 Count-Distinct (Linear Counting)

Linear counting is the simplest distinct-element counting technique: allocate a bitmap of m bits, hash each element to a position, and set that bit. The number of zero bits V gives an estimate of the distinct count n .

19.17.1 Estimate Formula

The fraction of zero bits follows:

$$\frac{V}{m} \approx e^{-n/m}$$

Rearranging gives the linear counting estimate:

$$\hat{n} = -m \cdot \ln\left(\frac{V}{m}\right)$$

This estimate is accurate when the bitmap is not too full (less than $\sim 70\%$ set). Once V/m drops below a threshold, switch to another method or double m .

Listing 19.3. Linear counting for distinct elements

```

1 #include <vector>
2 #include <functional>
3 #include <cmath>
4
5 class LinearCounter {
6 public:
7     explicit LinearCounter(size_t m)
8         : m_(m), bits_(m, false), count_(0) {}
9
10    void add(uint64_t hash) {
```

```

11     size_t idx = hash % m_;
12     if (!bits_[idx]) {
13         bits_[idx] = true;
14         ++count_;
15     }
16 }
17
18 double estimate() const {
19     if (count_ == m_) return static_cast<double>(m_);
20     double V = static_cast<double>(m_ - count_);
21     return -m_ * std::log(V / m_);
22 }
23
24 double load_factor() const {
25     return static_cast<double>(count_) / m_;
26 }
27
28 size_t memory_bytes() const {
29     return m_ / 8 + 2 * sizeof(size_t);
30 }
31
32 private:
33     size_t m_;
34     std::vector<bool> bits_;
35     size_t count_;
36 };

```

19.17.2 When to Use

- Use when cardinality is expected to be small relative to m (bitmap not too full).
- Provides a simple, fast baseline; HyperLogLog is preferred for large cardinalities.
- Linear counting uses roughly $n / \ln 2 \approx 1.44n$ bits when $V \approx m/2$.

19.18 Exercises

1. Implement HyperLogLog with $p = 10, 12, 14, 16$ and estimate the number of distinct values in a stream of 10^7 uniform random integers. Plot standard error vs. p and compare with the theoretical value $1.04/\sqrt{2^p}$.
1. Implement a streaming system that switches from linear counting to HyperLogLog when the bitmap load factor exceeds 70%. Compare accuracy at the switch point with using HyperLogLog alone from the start.

1. Use reservoir sampling with $k = 50$ to sample from a stream of 10^6 integers where each integer x appears with probability proportional to $1/\log(x+2)$. Verify empirically that the frequency of each sampled element matches its true frequency within expected error.
1. Merge two HyperLogLog structures with different seeds on disjoint halves of a dataset. Does the merged estimate match the true distinct count? How does accuracy degrade when the two halves overlap by 50%?
1. Implement a probabilistic counter using b bitmaps of size $m = 8192$ with different hash seeds. At query time, use the bitmap with the lowest load factor to estimate cardinality. Compare accuracy and space with a single large bitmap of size $b \cdot m$.

19.18.1 Top-K Approximation

Finding the top- k most frequent elements in a data stream is a common problem in network traffic analysis, trending topics, and log monitoring. A min-heap of size k paired with a hash map provides an online algorithm that approximates the top- k elements without storing the entire stream.

```

1 #include <iostream>
2 #include <queue>
3 #include <unordered_map>
4 #include <vector>
5 #include <string>
6
7 using Entry = std::pair<int, std::string>;    // (count, element)
8
9 // Min-heap of size k; keep the k elements with the largest counts.
10 class TopK {
11 public:
12     TopK(std::size_t k) : k_(k) {}
13
14     void insert(const std::string& item) {
15         int& cnt = freq_[item];
16         cnt++;
17         if (in_heap_.count(item)) {
18             // Already in heap; will be rebuilt on next topK().
19             stale_ = true;
20             return;
21         }
22         if (heap_.size() < k_) {
23             heap_.emplace(cnt, item);
24             in_heap_.insert(item);
25         } else if (cnt > heap_.top().first) {
26             auto [old_cnt, old_item] = heap_.top();
27             heap_.pop();
28             in_heap_.erase(old_item);
29             heap_.emplace(cnt, item);

```

```

30         in_heap_.insert(item);
31     }
32 }
33
34 std::vector<std::string> topK() {
35     if (stale_) rebuild();
36     std::vector<std::string> result;
37     while (!heap_.empty()) {
38         result.push_back(heap_.top().second);
39         heap_.pop();
40     }
41     std::reverse(result.begin(), result.end());
42     in_heap_.clear();
43     stale_ = false;
44     return result;
45 }
46
47 private:
48     void rebuild() {
49         // Rebuild heap from current frequency map.
50         std::priority_queue<Entry, std::vector<Entry>, std::greater<>>
fresh;
51         for (auto& [item, cnt] : freq_) {
52             if (fresh.size() < k_) {
53                 fresh.emplace(cnt, item);
54             } else if (cnt > fresh.top().first) {
55                 fresh.pop();
56                 fresh.emplace(cnt, item);
57             }
58         }
59         heap_ = std::move(fresh);
60     }
61
62     std::size_t k_;
63     std::unordered_map<std::string, int> freq_;
64     std::priority_queue<Entry, std::vector<Entry>, std::greater<>> heap_;
65     std::unordered_set<std::string> in_heap_;
66     bool stale_ = false;
67 };

```

The hash map accumulates exact counts; the min-heap evicts the smallest count when a new candidate exceeds the heap's minimum. When a heap-resident element receives an update, the algorithm marks the heap as stale and rebuilds it on demand, keeping the structure correct without per-update re-heapification. The space cost is $O(k + u)$, where u is the number of distinct items seen and k is the desired output size.

19.19 Common Bugs and Pitfalls

Pitfall	Consequence	
Bloom filter false positive rate exceeds requirements	Too many false positives, wasted queries	Calcul
Using Bloom filter when deletions are needed	Standard Bloom filter doesn't support deletion	
Cuckoo filter insertion failure (too full)	Insert returns without adding element	
Cuckoo filter max kicks exceeded during eviction	Infinite loop or degraded performance	
Forgetting Consistent Hashing's virtual nodes	Skewed distribution for non-uniform servers	
Hash collision in MinHash / Simhash	Wrong similarity estimate	
HyperLogLog with too few registers (m too small)	High estimation error	
Using reservoir sampling without shuffling first n elements	First k elements overrepresented, bias	
LSH hash table with too few bands	Poor recall (many near neighbors missed)	Tune

19.20 Summary

Structure	Problem	Space	Error	Supports Deletion
Bloom filter	Membership	$O(m)$ bits	False positive rate P	With counting variant
Cuckoo filter	Membership	$O(m)$ bits	False positive rate P	Yes
Count-Min Sketch	Frequency	$O(dw)$ counters	Additive $\epsilon \cdot n$	No
Heavy Hitters (Misra-Gries)	Top-k items	$O(k)$ counters	Exact for $f > n/k$	No
HyperLogLog	Cardinality	$O(m \log \log n)$	$1.04/\sqrt{m}$ relative	No
Quantile Estimation (GK)	Quantiles	$O(\frac{1}{\epsilon} \log \epsilon n)$	ϵn rank error	No
Reservoir Sampling	Random sample	$O(k)$	Uniform – exact	No
Sliding Window	Window count	$O(\frac{1}{\epsilon} \log W)$	$(1 \pm \epsilon)$	Yes
Consistent Hashing	Key distribution	$O(nV)$ mapping	Near-perfect balance	Yes
LSH / Simhash	Approx NN	$O(k)$ per item	Depends on k	–

These trade-offs are essential for streaming data, big data analytics, distributed systems, and ML pipelines.

19.21 Exercises

19.21.1 Drill

- For a Bloom filter with $m = 100$ bits, $k = 3$ hash functions, and $n = 10$ elements inserted, what is the expected false positive rate? Compute using the formula $(1 - e^{-kn/m})^k$.
- How many bits are needed for a Bloom filter to achieve a false positive rate of 1% with 10^6 elements? What if we need 0.1%?

1. For Count-Min Sketch, if we want to guarantee that all frequency estimates have error at most 0.1% with probability 99%, how many rows and columns do we need?
1. For HyperLogLog with 14-bit precision ($m = 16384$ registers), what is the standard error? If we halve the precision (use 13 bits), how does the error change?
1. Explain why Bloom filters have false positives but no false negatives. Give a concrete example of a false positive.
1. A cuckoo filter with bucket size 4 and 8 bits per fingerprint has a 1% false positive rate. How many buckets are needed for 10^6 elements? How does this compare with a Bloom filter using the same false positive rate?

19.21.2 Application

1. Implement a **counting Bloom filter** that supports deletion by replacing each bit with a small counter (typically 4 bits). Compare space with the standard Bloom filter.
1. Implement **Bloom filter union and intersection** — given two Bloom filters with the same parameters, compute the union (OR) and intersection (AND). Verify that the union correctly represents the union of the two sets.
1. Implement a **Bloom filter vs Cuckoo filter benchmark**: insert 10^6 elements, then measure space, insertion throughput, lookup throughput, and deletion throughput for each. At what false positive rate does the Cuckoo filter become strictly better?
1. Compare Count-Min Sketch vs `std::unordered_map` for frequency estimation on a stream of 10^6 items with 10^5 distinct keys. Measure the error in the sketch's estimates and the memory used.
1. Use HyperLogLog to estimate the number of distinct IP addresses in a network log with 10^7 entries. Compare accuracy at precision values $p = 10, 12, 14, 16$.
1. Implement **reservoir sampling** for $k = 100$ from a stream of unknown length. Verify that each element has probability k/n of being in the sample through simulation.
1. Implement a **heavy hitters** algorithm (lossy counting or space-saving) that finds all elements with frequency $\geq \phi \cdot n$ in a stream. Compare with Count-Min Sketch.
1. Implement a **Bloom filter** for a web crawler to avoid revisiting URLs. How does the false positive rate affect the number of missed pages?
1. Implement **consistent hashing** for a distributed cache with 5 servers and 100 keys. Show the distribution of keys when a server joins and when one leaves. Use 50 virtual nodes per server.
1. Use **simhash** to find near-duplicate documents in a collection of 100 text snippets. Compute the cosine similarity from signatures and compare with exact cosine similarity of TF-IDF vectors.

19.21.3 Research

1. **(MinHash)** Read about MinHash for estimating set similarity (Jaccard coefficient). Implement MinHash with k hash functions and compare with exact Jaccard computation for random sets.
1. **(Cuckoo filter scaling)** Our cuckoo filter uses a fixed bucket size of 4 and a max of 500 kicks. Experiment with different bucket sizes (2, 4, 8) and max kick limits. How do these parameters affect the fill factor before a rebuild is needed? At what load factor does the filter become unstable?
1. **(HyperLogLog++)** Read about Google's HyperLogLog++ improvements (Heule et al., 2013) including bias correction and sparse representation. Implement the sparse representation for small cardinalities.
1. **(Count-Min Sketch variants)** Research the Conservative Update variant of Count-Min Sketch. Implement it and measure the improvement in accuracy.
1. **(DDSketch)** Research the DDSketch algorithm for streaming quantiles. Implement it and compare accuracy with the Greenwald-Khanna algorithm on skewed distributions.
1. **(Rendezvous hashing)** Implement HRW (highest random weight) hashing and compare load distribution with consistent hashing. Which achieves better balance with fewer virtual nodes?

19.22 References and Further Reading

- Burton H. Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors" — CACM, 1970.
- Graham Cormode and S. Muthukrishnan, "An Improved Data Stream Summary: The Count-Min Sketch and Its Applications" — Journal of Algorithms, 2005.
- Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier, "HyperLogLog: the Analysis of a Near-Optimal Cardinality Estimation Algorithm" — AOFA, 2007.
- Jeffrey S. Vitter, "Random Sampling with a Reservoir" — ACM TOMS, 1985.
- Andrei Z. Broder, "On the Resemblance and Containment of Documents" — Compression and Complexity of Sequences, 1997.
- Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher, "Cuckoo Filter: Practically Better Than Bloom" — ACM CoNEXT, 2014.
- Stefan Heule, Marc Nunkesser, and Alexander Hall, "HyperLogLog in Practice: Algorithmic Engineering of a State of the Art Cardinality Estimation Algorithm" — EDBT, 2013.

- Jayadev Misra and David Gries, "Finding Repeated Elements" — Science of Computer Programming, 1982.
- Michael Greenwald and Sanjeev Khanna, "Space-Efficient Online Computation of Quantile Summaries" — SIGMOD, 2001 (GK sketch).
- Gurmeet Singh Manku and Rajeev Motwani, "Approximate Frequency Counts over Data Streams" — VLDB, 2002 (Misra-Gries and related algorithms).
- Datar, Gionis, Indyk, and Motwani, "Maintaining Stream Statistics over Sliding Windows" — SIAM Review, 2002 (exponential histogram).
- David Karger et al., "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web" — STOC, 1997.
- Piotr Indyk and Rajeev Motwani, "Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality" — STOC, 1998.
- Moses S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms" — STOC, 2002.
- Ted Dunning, "The T-Digest: Efficient Estimates of Distributions" — 2013.
- Graham Cormode, S. Muthukrishnan, Korn, and Fang, "Towards a Science of Data Streaming" — (survey of streaming algorithms).

Maximum Flow and Matching

20.1 Introduction

⚡ Complete compilable implementations of the data structures in this chapter are in `code/`.

20.2 The Maximum Flow Problem

A **flow network** is a directed graph $G = (V, E)$ with:

- A **source** s with no incoming edges
- A **sink** t with no outgoing edges
- **Capacity** $c(u, v) \geq 0$ for each edge

A **flow** f satisfies:

- **Capacity constraint:** $f(u, v) \leq c(u, v)$ for all edges
- **Flow conservation:** $\sum f(u, v) = \sum f(v, u)$ for all $v \neq s, t$

The **maximum flow problem** finds the maximum possible flow from s to t .

20.3 Dinic's Algorithm

Ford-Fulkerson is the general method: repeatedly find an augmenting path in the residual network and push flow along it. The algorithm's complexity depends on how paths are chosen. **Dinic's algorithm** uses BFS to build a **level graph** (layered network), then sends **blocking flow** via DFS — achieving $O(E \cdot V^2)$ time in general and $O(E \cdot \sqrt{V})$ for bipartite matching. The residual capacity $c_f(u, v) = c(u, v) - f(u, v) + f(v, u)$.

```

1 struct edge {
2     size_t to, rev;
3     int capacity;
4 };
5
6 class dinic {
7 public:
8     dinic(size_t n) : graph_(n) {}
9 
```

```

10     void add_edge(size_t from, size_t to, int capacity) {
11         graph_[from].push_back({to, graph_[to].size(), capacity});
12         graph_[to].push_back({from, graph_[from].size() - 1, 0}); //
reverse edge
13     }
14
15     int max_flow(size_t s, size_t t) {
16         int flow = 0;
17         std::vector<int> level;
18         std::vector<size_t> it;
19
20         while (bfs(s, t, level)) {
21             it.assign(graph_.size(), 0);
22             while (int pushed = dfs(s, t, INT_MAX, level, it)) {
23                 flow += pushed;
24             }
25         }
26         return flow;
27     }
28
29 private:
30     bool bfs(size_t s, size_t t, std::vector<int>& level) {
31         level.assign(graph_.size(), -1);
32         std::queue<size_t> q;
33         level[s] = 0;
34         q.push(s);
35         while (!q.empty()) {
36             size_t v = q.front(); q.pop();
37             for (const auto& e : graph_[v]) {
38                 if (e.capacity > 0 && level[e.to] < 0) {
39                     level[e.to] = level[v] + 1;
40                     q.push(e.to);
41                 }
42             }
43         }
44         return level[t] >= 0;
45     }
46
47     int dfs(size_t v, size_t t, int f, std::vector<int>& level,
48             std::vector<size_t>& it) {
49         if (v == t) return f;
50         for (size_t& i = it[v]; i < graph_[v].size(); ++i) {
51             edge& e = graph_[v][i];
52             if (e.capacity > 0 && level[v] < level[e.to]) {
53                 int pushed = dfs(e.to, t, std::min(f, e.capacity), level,
it);
54                 if (pushed > 0) {
55                     e.capacity -= pushed;

```

```

56         graph_[e.to][e.rev].capacity += pushed;
57         return pushed;
58     }
59 }
60 }
61 return 0;
62 }
63
64 std::vector<std::vector<edge>> graph_;
65 };

```

Dinic's Complexity: $O(E \cdot V^2)$ general, $O(E \cdot \sqrt{V})$ for bipartite matching, $O(\min(V^{2/3}, \sqrt{E}) \cdot E)$ for unit capacities.

20.4 Edmonds-Karp Algorithm

Ford-Fulkerson with BFS for augmenting path selection guarantees $O(V \cdot E^2)$ time — each augmentation saturates at least one edge, and the distance from s to any vertex increases monotonically.

```

1 int edmonds_karp(const std::vector<edge>& graph, size_t s, size_t t) {
2     int flow = 0;
3     // Build residual adjacency on the fly; same edge struct as Dinic
4     while (true) {
5         std::vector<edge*> parent(graph.size(), nullptr);
6         std::queue<size_t> q;
7         q.push(s);
8         while (!q.empty() && parent[t] == nullptr) {
9             size_t v = q.front(); q.pop();
10            for (auto& e : graph[v]) {
11                if (e.capacity > 0 && parent[e.to] == nullptr && e.to != s
12            ) {
13                parent[e.to] = &e;
14                q.push(e.to);
15            }
16        }
17        if (parent[t] == nullptr) break;
18        int aug = INT_MAX;
19        for (size_t v = t; v != s; v = parent[v]->to)
20            aug = std::min(aug, parent[v]->capacity);
21        for (size_t v = t; v != s; v = parent[v]->to) {
22            parent[v]->capacity -= aug;
23            graph[parent[v]->to][parent[v]->rev].capacity += aug;
24        }
25        flow += aug;
26    }
27    return flow;
28 }

```

In practice, Dinic outperforms Edmonds-Karp on most graphs, but Edmonds-Karp remains important for its theoretical guarantees and simplicity.

20.5 Push-Relabel Algorithm

Instead of augmenting paths, Push-Relabel (Goldberg–Tarjan) works by **pushing** flow locally from overflowing nodes and maintaining a **height label** that guides flow towards t .

```

1 Algorithm push_relabel(G, s, t):
2   for each edge (u, v): f(u,v) = 0
3   for each edge (s, v): f(s,v) = c(s,v)
4   height[s] = V
5   height[v] = 0 for all other v
6   excess[v] = inflow - outflow for each v
7
8   while EXISTS active vertex v (v != s,t with excess[v] > 0):
9       push(v)      -- send flow along admissible edges
10      relabel(v)    -- if no admissible edge, increase height[v]
11  return excess[t]
```

```

1 class push_relabel {
2 public:
3     push_relabel(size_t n) : graph_(n), excess_(n), height_(n) {}
4
5     int max_flow(size_t s, size_t t) {
6         height_[s] = graph_.size();
7         for (auto& e : graph_[s]) {
8             excess_[e.to] += e.capacity;
9             graph_[e.to][e.rev].capacity = e.capacity;
10            e.capacity = 0;
11        }
12        while (true) {
13            auto it = std::find_if(excess_.begin(), excess_.end(),
14                                  [&](int x) { return x > 0; });
15            if (it == excess_.end()) break;
16            size_t v = it - excess_.begin();
17            if (!discharge(v)) break;
18        }
19        return excess_[t];
20    }
21
22 private:
23     bool push(size_t v) {
24         for (auto& e : graph_[v]) {
25             if (e.capacity > 0 && height_[v] == height_[e.to] + 1) {
26                 int d = std::min(excess_[v], e.capacity);
27                 e.capacity -= d;
```

```

28         graph_[e.to][e.rev].capacity += d;
29         excess_[v] -= d;
30         excess_[e.to] += d;
31         return true;
32     }
33 }
34 return false;
35 }
36
37 bool discharge(size_t v) {
38     while (excess_[v] > 0) {
39         if (!push(v)) {
40             int min_h = INT_MAX;
41             for (const auto& e : graph_[v])
42                 if (e.capacity > 0)
43                     min_h = std::min(min_h, height_[e.to]);
44             if (min_h >= static_cast<int>(graph_.size()))
45                 return false;
46             height_[v] = min_h + 1;
47         }
48     }
49     return true;
50 }
51
52 std::vector<std::vector<edge>> graph_;
53 std::vector<int> excess_, height_;
54 };

```

Complexity: $O(V^2 \cdot \sqrt{E})$ with gap heuristic and global relabeling. Push-Relabel is often the fastest max-flow algorithm in practice for dense graphs.

20.6 Minimum Cut

The **minimum s-t cut** partitions V into S (containing s) and T (containing t) to minimize the total capacity of edges from S to T .

Max-Flow Min-Cut Theorem: The maximum flow equals the minimum cut capacity. This is one of the most important theorems in combinatorial optimization.

Finding the min-cut: After computing max flow, find all vertices reachable from s in the residual graph. Let S = reachable vertices, T = all others. The edges from S to T in the original graph form the minimum cut.

Worked example: Network: $s \rightarrow a(10)$, $s \rightarrow b(5)$, $a \rightarrow b(15)$, $a \rightarrow t(5)$, $b \rightarrow t(10)$.

After max flow = 15: the residual graph has s reachable, a reachable (via $s \rightarrow a$ with residual 5), but b is NOT reachable ($s \rightarrow b$ is saturated, $a \rightarrow b$ is saturated going forward, $b \rightarrow a$ has residual but we came from s). Wait — let me trace more carefully.

```
1 Augmenting path s->a->t: flow 5. Residual: s->a(5), a->t(0).
```

```

2 Augmenting path s->b->t: flow 5. Residual: s->b(0), b->t(5).
3 Augmenting path s->a->b->t: flow 5. Residual: s->a(0), a->b(10), b->t(0).
4
5 Max flow = 15.
6
7 Residual graph reachable from s: {s, a} (s->a has residual 5).
8 Min cut: edges from {s,a} to {b,t} = s->b(capacity 5) + a->t(capacity 5) =
    10?
9 Wait: a->t is saturated (capacity 5, flow 5). s->b is saturated (capacity
    5, flow 5).
10 But max flow = 15, so min cut = 15.
11
12 Let me recheck: edges from S={s,a} to T={b,t}:
13   s->b: capacity 5, flow 5 (contributes 5 to cut)
14   a->b: capacity 15, flow 10 (contributes 5 to cut)
15   a->t: capacity 5, flow 5 (contributes 5 to cut)
16 Total cut capacity: 5 + 15 + 5 = 25? No, cut capacity counts CAPACITY, not
    residual.
17
18 Min cut capacity = sum of original capacities of edges from S to T:
19   s->b: 5, a->b: 15, a->t: 5 = 25. That's not 15.
20
21 Hmm, let me recompute. Actually S should be vertices reachable from s in
    the RESIDUAL graph:
22   s->a: residual 5 (reachable)
23   s->b: residual 0 (NOT reachable)
24   a->b: residual 10 (a is reachable, b via a->b? a->b forward residual =
        15-10 = 5. YES b is reachable!)
25   So S = {s, a, b}, T = {t}.
26   Edges from S to T: a->t(5), b->t(10) = 15. Correct!

```

So the min cut is $\{a \rightarrow t, b \rightarrow t\}$ with capacity $5 + 10 = 15 = \text{max flow}$. ✓

Applications of min-cut:

- **Image segmentation:** pixels are nodes; source = foreground, sink = background. Min-cut separates foreground from background.
- **Network reliability:** the min-cut is the minimum number of edges whose failure disconnects s from t — the edge connectivity.
- **Community detection:** in social networks, the min-cut between two communities reveals the weakest connection.
- **VLSI design:** min-cut placement divides a circuit into two halves with minimum wire crossing.

20.7 Bipartite Matching

Find the maximum set of edges in a bipartite graph with no shared vertices.

Reduction to max flow: create source s connected to all left vertices (capacity 1), all right vertices connected to sink t (capacity 1), and original edges with capacity 1. The max flow equals the maximum matching size.

Worked example: Left = {A, B, C}, Right = {X, Y, Z}. Edges: A-X, A-Y, B-Y, C-Z.

```

1 Flow network:
2 s -> A(1), s -> B(1), s -> C(1)
3 A -> X(1), A -> Y(1), B -> Y(1), C -> Z(1)
4 X -> t(1), Y -> t(1), Z -> t(1)
5
6 Augmenting path s->A->X->t: flow 1. Match: A-X.
7 Augmenting path s->B->Y->t: flow 1. Match: B-Y.
8 Augmenting path s->C->Z->t: flow 1. Match: C-Z.
9
10 Max flow = 3. Maximum matching: {A-X, B-Y, C-Z}. All 3 left vertices
    matched.
```

Worked example (non-trivial): Left = {A, B, C}, Right = {X, Y}. Edges: A-X, A-Y, B-X, C-X.

```

1 Augmenting path s->A->X->t: flow 1. Match: A-X.
2 Augmenting path s->B->Y->t: no path (B has no edge to Y).
3 Try augmenting: s->B->X... X is matched to A.
4 Residual: X->A(1) (undo edge). A->Y(1). Y->t(1).
5 Path: s->B->X->A->Y->t: flow 1.
6 New matching: B-X, A-Y. (A switched from X to Y.)
7
8 Max flow = 2. Maximum matching: {B-X, A-Y}. C unmatched.
```

Complexity: Dinic's algorithm on the bipartite flow network runs in $O(E \cdot \sqrt{V})$ — faster than general max flow.

Applications:

- **Job assignment:** workers to tasks, applicants to jobs
- **Kidney exchange:** incompatible donor-patient pairs form a bipartite graph; cycles in the matching enable swaps
- **Ad assignment:** match ads to user sessions based on targeting criteria
- **Stable matching:** the Gale-Shapley algorithm (Chapter 4) solves a related problem

20.8 Min-Cost Max-Flow

Find a flow of a given amount through a network where each edge has both capacity and per-unit cost. The goal is to minimize total cost while achieving the required flow.

Approach: Successive Shortest Augmenting Path (SSP) with potentials (Johnson's). Replace residual edge costs with reduced costs to keep them nonnegative, then run Dijkstra to find the cheapest augmenting path.

```

1 struct mincost_edge {
2     size_t to, rev;
3     int capacity, cost;
4 };
5
6 class mincost_flow {
7 public:
8     mincost_flow(size_t n) : graph_(n), potential_(n, 0) {}
9
10    void add_edge(size_t from, size_t to, int cap, int cost) {
11        graph_[from].push_back({to, graph_[to].size(), cap, cost});
12        graph_[to].push_back({from, graph_[from].size() - 1, 0, -cost});
13    }
14
15    std::pair<int, int> flow(size_t s, size_t t, int maxf) {
16        int flow = 0, cost = 0;
17        while (flow < maxf) {
18            std::vector<int> dist(graph_.size(), INT_MAX);
19            std::vector<mincost_edge*> prev(graph_.size(), nullptr);
20            dist[s] = 0;
21            std::priority_queue<std::pair<int, size_t>,
22                std::vector<std::pair<int, size_t>>, std::greater<>> pq;
23            pq.push({0, s});
24
25            while (!pq.empty()) {
26                auto [d, v] = pq.top(); pq.pop();
27                if (dist[v] < d) continue;
28                for (auto& e : graph_[v]) {
29                    if (e.capacity > 0) {
30                        int nd = d + e.cost + potential_[v] - potential_[e
31                            .to];
32
33                        if (nd < dist[e.to]) {
34                            dist[e.to] = nd;
35                            prev[e.to] = &e;
36                            pq.push({nd, e.to});
37                        }
38                    }
39                }
40            }
41            if (dist[t] == INT_MAX) break;
42
43            for (size_t v = 0; v < graph_.size(); ++v)
44                if (dist[v] < INT_MAX)
45                    potential_[v] += dist[v];
46
47            int add = maxf - flow;
48            for (size_t v = t; v != s; ) {
49                auto& e = *prev[v];

```



```

48         add = std::min(add, e.capacity);
49         v = graph_[e.to][e.rev].to; // previous vertex
50     }
51     for (size_t v = t; v != s; ) {
52         auto& e = *prev[v];
53         e.capacity -= add;
54         graph_[e.to][e.rev].capacity += add;
55         cost += add * e.cost;
56         v = graph_[e.to][e.rev].to;
57     }
58     flow += add;
59 }
60 return {flow, cost};
61 }
62
63 private:
64     std::vector<std::vector<mincost_edge>> graph_;
65     std::vector<int> potential_;
66 };

```

Application: Transportation problems (minimize shipping cost with supply/demand constraints), optimal task assignment, min-cost matching, and network design.

20.9 Application: Project Selection

Given projects with profits (positive or negative) and prerequisites, select the set of projects that maximizes total profit.

Reduction to min-cut:

1. Create source s and sink t .
2. For each project with positive profit p : add edge $s \rightarrow \text{project}$ with capacity p .
3. For each project with negative profit (cost c): add edge $\text{project} \rightarrow t$ with capacity $|c|$.
4. For each prerequisite (project A requires project B): add edge $A \rightarrow B$ with capacity ∞ .
5. The minimum s - t cut partitions projects into S (selected, reachable from s) and T (not selected, reachable from t).
6. Optimal profit = total positive profit — min-cut capacity.

Worked example: 4 projects:

- 1 Project 1: profit +50, no prerequisites
- 2 Project 2: profit +30, requires Project 1
- 3 Project 3: profit -10 (cost 10), no prerequisites
- 4 Project 4: profit +40, requires Project 3

```
5
6 Flow network:
7 s -> 1 (cap 50), s -> 2 (cap 30), s -> 4 (cap 40)
8 3 -> t (cap 10)
9 2 -> 1 (cap INF), 4 -> 3 (cap INF)
10
11 Total positive profit = 50 + 30 + 40 = 120.
12 Min cut: either {s} vs {1,2,3,4,t} (cut = 50+30+40 = 120, profit = 0)
13   or {s,1,2} vs {3,4,t} (cut = 10 + 40 = 50, profit = 70)
14   or {s,1,2,3} vs {4,t} (cut = INF, too expensive)
15   or {s,1,2,4} vs {3,t} (cut = 10, profit = 110) -- but 4->3 is INF!
16   Wait: if 4 is in S, 3 must be in S (edge 4->3 has capacity INF).
17   So {s,1,2,4,3} vs {t}: cut = 10, profit = 120 - 10 = 110.
18
19 Optimal: select projects 1, 2, 3, 4. Profit = 50+30-10+40 = 110.
```

Why it works: the infinite-capacity edges enforce prerequisites: if project A requires B, the min-cut cannot put A in S and B in T (that would cost ∞). So any feasible selection respects all prerequisites. The cut capacity measures the “cost” of the selection: positive-profit projects in T (forgone profit) plus negative-profit projects in S (incurred costs).

20.10 Common Bugs and Pitfalls

Pitfall	Consequence	
Forgetting to add reverse edges in residual graph	Flow can't be undone, wrong max flow value	Always
Integer overflow on large capacity networks	Negative flow values, wrong result	Use
Not resetting <code>visited</code> between BFS/DFS phases	Algorithm terminates early, suboptimal flow	Clear
Infinite loop if blocking flow makes no progress	No termination	Track wh
Dinic's level graph not rebuilt after a blocking flow	Stale levels, wrong augmenting paths	
Using adjacency matrix for large sparse flow graphs	$O(V^2)$ memory where $O(E)$ would suffice	
Assuming min-cost flow costs are non-negative	Negative cycles exploited, unbounded flow	Use potentials

20.11 Summary

1. **Maximum flow** finds the maximum throughput in a network.
2. **Dinic's algorithm** with level graph + blocking flow is the most practical general algorithm.
3. **The max-flow min-cut theorem** connects flow to graph partitioning.
4. **Bipartite matching** reduces to flow.
5. **Min-cost max-flow** extends the model with edge costs, supporting transportation and assignment problems.

6. Applications range from transportation to image segmentation to team assignment.

20.12 Interview FAQs

1. Explain the max-flow min-cut theorem.

The maximum flow from s to t equals the minimum capacity of any s - t cut. This means: (1) the flow is bounded by the bottleneck — any cut is an upper bound; (2) there exists a cut achieving this bound. The proof constructs a min-cut from the max flow: the min-cut is the set of edges from reachable to unreachable vertices in the residual graph.

1. How does Dinic's algorithm improve on Ford-Fulkerson?

Ford-Fulkerson finds one augmenting path at a time — $O(E \cdot \text{max_flow})$ in the worst case. Dinic's algorithm finds a BLOCKING FLOW in each phase: it builds a level graph via BFS and sends flow along all shortest augmenting paths simultaneously via DFS. Each phase takes $O(E)$ time, and there are $O(V)$ phases, giving $O(E \cdot V^2)$ total. For bipartite matching, Dinic runs in $O(E \cdot \sqrt{V})$.

1. Why does the integral flow theorem guarantee that the flow-based bipartite matching gives a valid matching?

The integrality theorem says: if all capacities are integers, there exists an integral max flow. Since all capacities here are 1, the flow is 0/1 on every edge. Each unit of flow from left-vertex to right-vertex corresponds to picking that edge in the matching. The source $\rightarrow$ left and right $\rightarrow$ sink edges being capacity 1 enforces the matching constraint (each vertex matched at most once) — if a left vertex sends ≥ 2 units, its source edge would need capacity ≥ 2 . The reduction is tight precisely because the integrality theorem converts a continuous optimization (LP) into a discrete combinatorial solution.

1. What is the push-relabel algorithm and when is it faster than Dinic?

Push-relabel maintains a “preflow” (excess flow at vertices) and “height labels” that guide flow toward the sink. Instead of finding augmenting paths, it pushes flow locally and relabels vertices. It runs in $O(V^2 \cdot \sqrt{E})$ and is often faster than Dinic for dense graphs because it avoids the BFS/DFS phases.

1. How does the project selection problem reduce to min-cut?

Each project with profit p gets an edge from source (capacity p). Each project with cost c gets an edge to sink (capacity c). Prerequisites become infinite-capacity edges ($A \rightarrow B$ if A requires B). The min-cut partitions projects into “selected” (S) and “not selected” (T). The infinite edges prevent selecting A without B . The cut capacity equals the total profit minus the optimal selection's profit.

1. Why do you need potentials in min-cost max-flow, and what breaks if you just use plain Dijkstra each time?

When you augment along a path, you add reverse edges with negative cost (the negation of the forward cost). After a few iterations, the residual graph has negative-cost edges, which makes Dijkstra's assumption of non-negative weights invalid. Potentials solve this by reweighting: each vertex v gets a potential $\pi(v)$, and the reduced cost $w'(u, v) = w(u, v) + \pi(u) - \pi(v)$ is always ≥ 0 . Dijkstra runs on reduced costs, and the potentials are updated after each round ($\pi(v) += \text{dist}(v)$). This is essentially Johnson's reweighting trick applied iteratively — it's the same idea used for all-pairs shortest paths with negative edges.

1. How do you find the edge connectivity of a graph using max flow?

The edge connectivity $\lambda(s, t)$ between vertices s and t is the minimum s - t cut. To find the global edge connectivity, run max flow from vertex 1 to each other vertex; the minimum over all these is the edge connectivity. Alternatively, Stoer-Wagner's algorithm finds the global min-cut in $O(V \cdot E)$ without flow.

1. How do you recognize when a problem secretly reduces to max flow?

True max-flow problems share these properties: (1) you have a fixed source of supply and a fixed sink of demand; (2) each unit of flow is fungible (carrying 1 unit through edge A then edge B is the same as through C then D); (3) the goal is to maximize/minimize a global quantity, not optimize per-unit decisions; (4) edge constraints are capacity bounds, not costs. Red flags it's NOT max flow: flow units are heterogeneous (multi-commodity), decisions have side effects ("if you use this route, that route becomes unavailable"), or the objective is non-linear. The baseball elimination problem is a classic non-obvious reduction — the key insight is modeling "remaining games between team i and j " as a game node that must be assigned to one team or the other.

1. How do you handle capacities that are very large (e.g., 10^{18})?

Use `int64_t` or `long long` for capacities. Ford-Fulkerson with integer capacities takes $O(E \cdot f)$ where f is the max flow — for large capacities this is slow. Use Dinic's algorithm instead: $O(E \cdot V^2)$ is independent of capacity values. For capacity scaling variants, the complexity depends on $\log(\text{max capacity})$.

1. A ride-sharing platform needs to match riders to drivers in real-time. How would you model this, and where does max flow break down?

Max flow handles the bipartite matching (drivers $\leftrightarrow$ riders) and capacity constraints (driver takes at most 1 ride). Min-cost max-flow adds optimization (minimize total pickup distance). But max flow breaks down for: (1) dynamic matching — rides arrive continuously, needing online algorithms, not batch flow computation; (2) multi-objective — minimize wait time AND maximize earnings AND ensure fairness; (3) spatial constraints — drivers and riders are in different zones, so the graph structure changes as people move. In practice, ride-sharing companies use max

flow for initial batch matching within a zone, then local search (Hungarian algorithm or auction-based) for real-time adjustments. The lesson: max flow is the starting point, not the complete solution.

20.13 Edmonds-Karp in Depth

The Edmonds-Karp algorithm is the simplest guaranteed-efficient max-flow algorithm. It is Ford-Fulkerson with a specific rule: always choose the **shortest** augmenting path (fewest edges), found via BFS. This seemingly minor change transforms the worst-case from $O(E \cdot \text{max_flow})$ to a clean $O(V \cdot E^2)$, independent of capacity values.

Key insight: After each augmentation, the shortest-path distance from s to every vertex in the residual graph is monotonically non-decreasing. Each edge can be the critical (bottleneck) edge in at most $O(V)$ augmentations, giving the $O(V \cdot E^2)$ bound.

Worked trace on a small network:

Consider a 6-vertex network: s, a, b, c, d, t .

```

1 Original edges (u -> v : capacity):
2   s -> a : 7           s -> b : 4
3   a -> b : 3           a -> c : 6
4   b -> d : 5           c -> d : 8
5   c -> t : 9           d -> t : 6
6
7 We run Edmonds-Karp (BFS always picks shortest path in hops).
8
9 -----
10 Round 1: BFS finds s -> a -> c -> t (3 hops)
11   Bottleneck = min(7, 6, 9) = 6
12   Push 6 units along s -> a -> c -> t.
13
14   Residual capacities after:
15     s -> a : 1           s -> b : 4
16     a -> b : 3           a -> c : 0 (saturated)
17     b -> d : 5           c -> d : 8
18     c -> t : 3           d -> t : 6
19     (reverse edges carry flow back: c -> a : 6, t -> c : 6)
20
21   Total flow = 6
22 -----
23 Round 2: BFS finds s -> b -> d -> t (3 hops)
24   Bottleneck = min(4, 5, 6) = 4
25   Push 4 units along s -> b -> d -> t.
26
27   Residual capacities after:
28     s -> a : 1           s -> b : 0 (saturated)
29     a -> b : 3           a -> c : 0
30     b -> d : 1           c -> d : 8

```

```

31     c -> t : 3      d -> t : 2
32     (reverse edges: b -> s : 4, d -> b : 4, t -> d : 4)
33
34     Total flow = 10
35     -----
36 Round 3: BFS finds s -> a -> b -> d -> t (4 hops)
37     Bottleneck = min(1, 3, 1, 2) = 1
38     Push 1 unit along s -> a -> b -> d -> t.
39
40     Residual capacities after:
41     s -> a : 0      s -> b : 0  (both saturated)
42     a -> b : 2      a -> c : 0
43     b -> d : 0      c -> d : 8
44     c -> t : 3      d -> t : 1
45     (reverse edges: a -> s : 7, b -> a : 4, d -> b : 5, t -> d : 5)
46
47     Total flow = 11
48     -----
49 Round 4: BFS from s finds no path to t. s -> a has
50     residual 0, s -> b has residual 0. Terminate.
51
52     MAX FLOW = 11.
53     -----

```

Verify: the min-cut is $S = \{s\}$, $T = \{a, b, c, d, t\}$ with capacity $7 + 4 = 11$. The max-flow min-cut theorem checks out.

Complete C++ implementation:

```

1  #include <vector>
2  #include <queue>
3  #include <climits>
4  #include <algorithm>
5
6  struct ek_edge {
7      int to, rev;
8      int cap;
9  };
10
11 class edmonds_karp {
12 public:
13     edmonds_karp(int n) : graph_(n) {}
14
15     void add_edge(int from, int to, int cap) {
16         graph_[from].push_back({to, (int)graph_[to].size(), cap});
17         graph_[to].push_back({from, (int)graph_[from].size() - 1, 0});
18     }
19
20     int max_flow(int s, int t) {
21         int flow = 0;

```

```

22     int n = graph_.size();
23
24     while (true) {
25         std::vector<int> parent(n, -1);
26         std::vector<int> parent_edge(n, -1);
27         std::queue<int> q;
28         q.push(s);
29         parent[s] = s;
30
31         while (!q.empty() && parent[t] == -1) {
32             int v = q.front(); q.pop();
33             for (int i = 0; i < (int)graph_[v].size(); ++i) {
34                 ek_edge& e = graph_[v][i];
35                 if (e.cap > 0 && parent[e.to] == -1) {
36                     parent[e.to] = v;
37                     parent_edge[e.to] = i;
38                     q.push(e.to);
39                 }
40             }
41         }
42
43         if (parent[t] == -1) break;
44
45         int aug = INT_MAX;
46         for (int v = t; v != s; v = parent[v]) {
47             int u = parent[v];
48             int idx = parent_edge[v];
49             aug = std::min(aug, graph_[u][idx].cap);
50         }
51
52         for (int v = t; v != s; v = parent[v]) {
53             int u = parent[v];
54             int idx = parent_edge[v];
55             graph_[u][idx].cap -= aug;
56             int rev = graph_[u][idx].rev;
57             graph_[v][rev].cap += aug;
58         }
59
60         flow += aug;
61     }
62     return flow;
63 }
64
65 private:
66     std::vector<std::vector<ek_edge>> graph_;
67 };

```

Complexity analysis: BFS runs in $O(E)$ per augmentation. Each augmentation saturates at least

one edge. An edge can be saturated at most $O(V)$ times (because each re-saturation requires the s -to- u distance to increase, and distances are bounded by V). So the total number of augmentations is $O(V \cdot E)$, giving $O(V \cdot E^2)$ overall. In practice, Dinic's algorithm is faster because it processes a blocking flow (many paths) per BFS phase, but Edmonds-Karp's simplicity makes it useful for correctness proofs and small instances.

20.14 Maximum Bipartite Matching (Full Treatment)

A **bipartite graph** $G = (L \cup R, E)$ partitions vertices into two sets with all edges crossing between them. The **maximum bipartite matching** problem finds the largest set of edges with no shared vertices.

Reduction to max flow: Given bipartite graph (L, R, E) :

1. Create source s and sink t .
2. Add edge $s \rightarrow u$ for every u in L , capacity 1.
3. Add edge $v \rightarrow t$ for every v in R , capacity 1.
4. For each edge (u, v) in E , add edge $u \rightarrow v$, capacity 1.

The max flow equals the maximum matching. Each unit of flow corresponds to a matched edge. All capacities are 1, so Dinic's algorithm achieves $O(E \cdot \sqrt{V})$ – better than the general $O(E \cdot V^2)$.

Worked trace: Job assignment problem.

4 applicants (A, B, C, D) and 4 jobs (J1, J2, J3, J4). Each applicant qualifies for certain jobs.

```

1 Qualifications (applicant -> qualified jobs):
2   A -> J1, J2
3   B -> J2, J3
4   C -> J1, J3, J4
5   D -> J3, J4
6
7 Flow network (8 vertices + s + t = 10 vertices):
8   s -> A(1), s -> B(1), s -> C(1), s -> D(1)
9   A -> J1(1), A -> J2(1)
10  B -> J2(1), B -> J3(1)
11  C -> J1(1), C -> J3(1), C -> J4(1)
12  D -> J3(1), D -> J4(1)
13  J1 -> t(1), J2 -> t(1), J3 -> t(1), J4 -> t(1)
14
15 -----
16 Round 1: BFS path s -> A -> J1 -> t
17   Bottleneck = 1. Push flow. Match: {A-J1}.
18   Saturated: s->A, A->J1, J1->t.
19
20 Round 2: BFS path s -> B -> J2 -> t
21   Bottleneck = 1. Push flow. Match: {A-J1, B-J2}.
22   Saturated: s->B, B->J2, J2->t.
```



```

23
24 Round 3: BFS path s -> C -> J3 -> t
25   Bottleneck = 1. Push flow. Match: {A-J1, B-J2, C-J3}.
26   Saturated: s->C, C->J3, J3->t.
27
28 Round 4: BFS from s: s->D is available.
29   D -> J4 -> t is open. Path: s -> D -> J4 -> t.
30   Bottleneck = 1. Push flow.
31   Match: {A-J1, B-J2, C-J3, D-J4}.
32
33 Round 5: BFS from s: s->A(0), s->B(0), s->C(0), s->D(0).
34   All source edges saturated. No augmenting path. Done.
35
36 MAXIMUM MATCHING = 4 (all applicants assigned).
37 -----

```

A non-trivial case: Consider 3 applicants, 2 jobs: A qualifies for J1; B qualifies for J1, J2; C qualifies for J2.

```

1 Round 1: s -> A -> J1 -> t. Match: {A-J1}.
2 Round 2: s -> B -> J2 -> t. Match: {A-J1, B-J2}.
3 Round 3: s -> C: C->J2 only, J2->t saturated.
4   Path: s -> C -> J2 -> B -> J1 -> t?
5   Residual: J2->B(1, reverse), B->J1 has no edge. Dead end.
6   No augmenting path. Max matching = 2. C is unmatched.

```

Konig's Theorem: In bipartite graphs, the size of the maximum matching equals the size of the minimum vertex cover. This connects matching to covering and has applications in scheduling and resource allocation.

Hopcroft-Karp algorithm: Instead of finding one augmenting path at a time, Hopcroft-Karp finds a **maximal set of shortest, vertex-disjoint** augmenting paths in each BFS phase, then augments along all of them simultaneously. This reduces the number of BFS phases from $O(V)$ to $O(\sqrt{V})$, yielding $O(E \cdot \sqrt{V})$ time – the same asymptotic bound as Dinic on bipartite graphs, but with smaller constants in practice. For most competitive programming and interview purposes, Dinic on the flow network suffices.

20.15 Minimum Cost Maximum Flow: Full Treatment

Library implementation: `min_cost_flow.h` provides `dsa::min_cost_max_flow`, including signed costs and a flow limit.

In many real networks, edges have both a **capacity** and a **per-unit cost**. The minimum cost maximum flow problem asks: among all flows achieving the maximum possible value, find the one that minimizes total cost.

Formal definition: Given a directed graph $G = (V, E)$ with capacity $c(u,v)$, cost per unit $w(u,v)$, source s , sink t , and desired flow value F , find a flow f that delivers F units from s to t at minimum total

cost $\sum f(u,v) \cdot w(u,v)$.

Successive Shortest Augmenting Path (SSP) with Potentials:

1. Initialize potentials $\pi(v) = 0$ for all v (or run Bellman-Ford for negative costs).
2. In each iteration, compute shortest path from s to t using **reduced costs**: $w'(u,v) = w(u,v) + \pi(u) - \pi(v)$. These are guaranteed non-negative.
3. Update potentials: $\pi(v) += \text{dist}(v)$ for all v .
4. Augment along the shortest path, updating residual capacities.
5. Repeat until F units have been sent or no path exists.

The reduced costs ensure Dijkstra works correctly. The potentials are updated each round to maintain the non-negativity invariant, even when reverse edges with negative original costs enter the residual graph.

Worked trace: 4-vertex network with source s , intermediate vertices a and b , sink t .

```

1 Edges (u -> v : capacity, cost):
2   s -> a : cap=4, cost=2
3   s -> b : cap=2, cost=3
4   a -> b : cap=2, cost=1
5   a -> t : cap=3, cost=4
6   b -> t : cap=5, cost=2
7
8 Desired flow value: F = 5
9
10 =====
11 Round 1: Potentials = [0, 0, 0, 0] for (s, a, b, t).
12   Reduced costs = original costs.
13   Dijkstra shortest path: s -> a -> b -> t, cost = 2+1+2 = 5.
14   Bottleneck = min(4, 2, 5) = 2.
15   Push 2 units along s -> a -> b -> t.
16   Cost so far = 2 * 5 = 10.
17
18   Residual after:
19     s -> a : cap=2, cost=2      a -> s : cap=2, cost=-2
20     s -> b : cap=2, cost=3      (unchanged)
21     a -> b : cap=0, cost=1      b -> a : cap=2, cost=-1
22     a -> t : cap=3, cost=4      (unchanged)
23     b -> t : cap=3, cost=2      t -> b : cap=2, cost=-2
24
25   Distances from s: dist(a)=2, dist(b)=3, dist(t)=5.
26   New potentials: pi(s)=0, pi(a)=2, pi(b)=3, pi(t)=5.
27
28 =====
29 Round 2: Reduced costs  $w'(u,v) = w(u,v) + \pi(u) - \pi(v)$ 
30   s -> a : 2 + 0 - 2 = 0

```

```

31  s -> b: 3 + 0 - 3 = 0
32  a -> t: 4 + 2 - 5 = 1
33  b -> t: 2 + 3 - 5 = 0
34
35  Dijkstra: s -> b -> t has reduced cost 0 + 0 = 0.
36  s -> a -> t has reduced cost 0 + 1 = 1.
37  Shortest path: s -> b -> t.
38  Bottleneck = min(2, 3) = 2.
39  Push 2 units. Cost += 2 * 3 = 6. Total cost = 16.
40  Flow = 4.
41
42  Residual: s->b(0), b->t(1). Reverse: b->s(2,-3).
43
44  Update potentials: dist(s)=0, dist(a)=0, dist(t)=1.
45  pi(a)=2, pi(b)=3 (unchanged), pi(t)=5+1=6.
46
47  =====
48  Round 3: Reduced costs:
49    s -> a: 2 + 0 - 2 = 0
50    a -> t: 4 + 2 - 6 = 0
51
52    Shortest path: s -> a -> t. Reduced cost = 0 + 0 = 0.
53    Push min(2, 3, 5-4) = 1 unit. Cost += 1 * 4 = 4.
54    Total cost = 20. Flow = 5. Done.
55
56  =====
57  RESULT: Flow = 5, Total cost = 20.

```

Complexity: Each Dijkstra call is $O(E \cdot \log V)$. We augment at most F times (one per unit with unit capacities) or $O(V \cdot E)$ times in general (each augmentation saturates one edge, and there are $O(V \cdot E)$ edge saturations total). With capacity scaling, the cost is $O(E \cdot \log U \cdot (E + V \cdot \log V))$ where U is the maximum capacity.

20.16 Applications Deep Dive

20.16.1 Airline Scheduling: Crew Assignment

Airlines must assign crews (pilots, flight attendants) to flights. Each crew can operate certain flight sequences based on qualifications, rest rules, and base locations. The goal: cover all flights with minimum total crew cost.

Reduction to min-cost max-flow:

1. Each flight is a demand node. Each crew is a supply node.
2. Source s connects to each crew node with capacity = number of flights the crew can fly, cost = 0.

3. Each crew connects to every flight they are qualified for, capacity 1, cost = salary/cost for that assignment.
4. Each flight connects to sink t with capacity 1 (must be covered exactly once), cost 0.
5. Find min-cost max-flow of value = number of flights.

Worked example: 3 flights (F1, F2, F3) and 2 crews (C1, C2).

```

1 Qualifications and costs:
2   C1 can fly: F1 (cost 5), F2 (cost 3), F3 (cost 7)
3   C2 can fly: F1 (cost 4), F2 (cost 6)
4
5 Flow network:
6   s -> C1 (cap 2, cost 0)      C1 -> F1 (cap 1, cost 5)
7   s -> C2 (cap 1, cost 0)      C1 -> F2 (cap 1, cost 3)
8                                 C1 -> F3 (cap 1, cost 7)
9                                 C2 -> F1 (cap 1, cost 4)
10                                C2 -> F2 (cap 1, cost 6)
11   F1 -> t (cap 1, cost 0)
12   F2 -> t (cap 1, cost 0)
13   F3 -> t (cap 1, cost 0)
14
15 We need flow = 3 (all flights covered).
16
17 Assignment 1: C1 flies F1 and F3, C2 flies F2.
18   Cost = 5 + 7 + 6 = 18.
19
20 Assignment 2: C1 flies F2 and F3, C2 flies F1.
21   Cost = 3 + 7 + 4 = 14.  <-- BETTER!
22
23 Min-cost max-flow finds cost 14.
```

20.16.2 Image Segmentation: Graph Cuts

Image segmentation partitions an image into foreground and background. The **graph cut** approach models each pixel as a node in a flow network.

Construction:

1. Each pixel i is a node.
2. Source s represents foreground, sink t represents background.
3. Edge $s \rightarrow i$ with capacity = probability pixel i is foreground (from color model).
4. Edge $i \rightarrow t$ with capacity = probability pixel i is background.
5. Edge $i \rightarrow j$ between adjacent pixels with capacity = similarity penalty (pixels that look alike should be in the same region).

The min s-t cut separates foreground (reachable from s) from background (reachable from t). The cut minimizes misclassification of pixels plus the penalty for cutting between similar adjacent pixels.

Worked example: A 2x2 image with pixels p1, p2, p3, p4.

```

1 Pixel probabilities (foreground, background):
2   p1: fg=0.9, bg=0.1      (very likely foreground)
3   p2: fg=0.3, bg=0.7      (likely background)
4   p3: fg=0.8, bg=0.2      (likely foreground)
5   p4: fg=0.2, bg=0.8      (likely background)
6
7 Adjacent pixel similarities (cut penalty):
8   p1-p2: 0.4   p1-p3: 0.6
9   p2-p4: 0.5   p3-p4: 0.3
10
11 Flow network:
12   s -> p1(0.9)    s -> p2(0.3)    s -> p3(0.8)    s -> p4(0.2)
13   p1 -> t(0.1)    p2 -> t(0.7)    p3 -> t(0.2)    p4 -> t(0.8)
14   p1 <-> p2(0.4)
15   p1 <-> p3(0.6)
16   p2 <-> p4(0.5)
17   p3 <-> p4(0.3)
18
19 Possible cuts:
20   Cut 1: S={s, p1, p3}, T={p2, p4, t}
21     Cut edges: s->p2(0.3), p1->t(0.1), p3->t(0.2),
22                p1->p2(0.4), p3->p4(0.3)
23     Cost = 0.3 + 0.1 + 0.2 + 0.4 + 0.3 = 1.3
24
25   Cut 2: S={s, p1, p2, p3}, T={p4, t}
26     Cut edges: s->p4(0.2), p2->t(0.7), p3->t(0.2),
27                p2->p4(0.5), p3->p4(0.3)
28     Cost = 0.2 + 0.7 + 0.2 + 0.5 + 0.3 = 1.9
29
30   Cut 3: S={s}, T={all pixels, t}
31     Cost = 0.9 + 0.3 + 0.8 + 0.2 = 2.2
32
33 Best: Cut 1, foreground = {p1, p3}, background = {p2, p4}.
34 This matches intuition: bright and dark pixels correctly separated.

```

20.16.3 Baseball Elimination: Proving a Team Cannot Win

In a sports league, can team X still finish in first place (possibly tied)? The **baseball elimination** problem answers this using max flow.

Setup: Given current wins for each team and remaining head-to-head games. Team X is **eliminated** if some other team must finish with more wins than X can achieve.

Reduction to max flow:

1. For each pair of teams (i, j) that still play each other, create a game node $g(i, j)$.
2. Source s connects to each game node $g(i, j)$ with capacity = number of remaining games between i and j .
3. Each game node $g(i, j)$ connects to team nodes i and j with capacity = infinity.
4. Each team node connects to sink t with capacity = (max wins by X) - (current wins of that team). If this is negative, X is immediately eliminated.
5. If the max flow equals the total number of remaining games, X is NOT eliminated. Otherwise, X is eliminated.

Worked example: 4 teams. Wins so far: $A=8, B=7, C=6, D=6$. Remaining games: A-B: 1, B-C: 2, B-D: 1, C-D: 1. Can team D win?

```

1 D can win at most  $6 + 1 + 1 = 8$  games.
2 Check: does any team get forced past 8 wins?
3
4 Capacities to sink:
5   A:  $8 - 8 = 0$ 
6   B:  $8 - 7 = 1$ 
7   C:  $8 - 6 = 2$ 
8
9 Game nodes (remaining games):
10   $g(A, B): s \rightarrow g(A, B) \text{ cap}=1$ 
11   $g(B, C): s \rightarrow g(B, C) \text{ cap}=2$ 
12   $g(B, D): s \rightarrow g(B, D) \text{ cap}=1$ 
13   $g(C, D): s \rightarrow g(C, D) \text{ cap}=1$ 
14
15 Total remaining games =  $1 + 2 + 1 + 1 = 5$ .
16
17 Edges from games to teams (capacity INF):
18   $g(A, B) \rightarrow A, g(A, B) \rightarrow B$ 
19   $g(B, C) \rightarrow B, g(B, C) \rightarrow C$ 
20   $g(B, D) \rightarrow B, g(B, D) \rightarrow D$ 
21   $g(C, D) \rightarrow C, g(C, D) \rightarrow D$ 
22
23 Max flow computation:
24   $g(A, B) \rightarrow A \rightarrow t: 1 \text{ unit. (A capacity: 0 remaining)}$ 
25   $g(B, C) \rightarrow B \rightarrow t: 1 \text{ unit. (B capacity: 0 remaining)}$ 
26   $g(B, C) \rightarrow C \rightarrow t: 1 \text{ unit. (C capacity: 1 remaining)}$ 
27   $g(B, D) \rightarrow D \rightarrow t: 1 \text{ unit. (B saturated, route via D)}$ 
28   $g(C, D) \rightarrow C \rightarrow t: 1 \text{ unit. (C capacity: 0 remaining)}$ 
29
30 Total flow = 5 = total remaining games.
31 D is NOT eliminated.
32
33 Valid scenario:

```

```

34  A beats B: A=9, B=7
35  B beats C, C beats B: B=8, C=7
36  D beats B: D=7
37  D beats C: D=8
38
39  Final: A=8, B=8, C=7, D=8. Three-way tie at 8 wins.
40  D is NOT eliminated.

```

When a team IS eliminated: Wins: A=20, B=18, C=16, D=14. $D_{\max} = 14 + 2 + 1 = 17$. But A already has 20 $>$ 17. The capacity from A to sink is $17 - 20 = -3$, which is negative. D is immediately eliminated without any flow computation. This demonstrates how the reduction quickly handles the obvious cases.

20.17 Exercises

20.17.1 Drill

1. For the flow network with edges $s \rightarrow a(10)$, $s \rightarrow b(5)$, $a \rightarrow b(15)$, $a \rightarrow t(5)$, $b \rightarrow t(10)$, find the maximum flow from s to t using Ford-Fulkerson. Show the augmenting path and residual network at each step.
1. What is the min-cut value in the network above? Which edges are in the min-cut?
1. For bipartite matching: given left set A, B, C, right set X, Y, Z, and edges A-X, A-Y, B-Y, C-Z, find the maximum matching. What is the matching size?
1. Show that the max-flow min-cut theorem holds for the network from Exercise 1: compute the min-cut capacity and verify it equals the max-flow value.
1. What is the time complexity of:
 - a) Ford-Fulkerson with integer capacities
 - b) Dinic's algorithm on a general graph
 - c) Dinic's algorithm on a unit-capacity network
 - d) Edmonds-Karp (Ford-Fulkerson with BFS)
1. Trace Edmonds-Karp on the network: $s \rightarrow a(4)$, $s \rightarrow b(2)$, $a \rightarrow b(2)$, $a \rightarrow t(3)$, $b \rightarrow t(5)$. Show the BFS augmenting path at each step, the residual graph, and the total flow.
1. Given a bipartite graph with left set $\{1, 2, 3, 4\}$ and right set $\{A, B, C\}$, edges: 1-A, 1-B, 2-B, 2-C, 3-A, 3-C, 4-B. Find the maximum matching using the flow reduction. Which left vertex is unmatched?
1. In the airline crew assignment example from this chapter (3 flights, 2 crews), prove that cost 14 is optimal by showing no assignment achieves cost 13 or less.

1. Construct a 5-vertex flow network where Edmonds-Karp requires exactly 5 augmentations. List the edges and capacities, then trace each augmentation showing the BFS path chosen and the bottleneck.
1. For the min-cost flow network: $s \rightarrow a(3, \text{cost } 2)$, $a \rightarrow b(2, \text{cost } 3)$, $b \rightarrow t(3, \text{cost } 1)$, $s \rightarrow b(1, \text{cost } 5)$, $a \rightarrow t(2, \text{cost } 4)$. Find the min-cost flow of value 3. Show each augmentation step.

20.17.2 Application

1. Implement the **Edmonds-Karp algorithm** (Ford-Fulkerson with BFS). Compare its performance with Dinic on random networks of various sizes. Does BFS guarantee $O(V \cdot E^2)$ in practice?
1. Use maximum flow to find the **maximum matching in a general graph** (reduction to flow). Compare with the Hopcroft-Karp algorithm for bipartite graphs.
1. Implement the **min-cost max-flow** algorithm using successive shortest augmenting paths (with potentials). Find the cheapest flow of a given amount through a network with per-unit costs on edges.
1. Solve the **baseball elimination** problem using max flow: given league standings and remaining games, determine if a team can still finish first. Test on real baseball data.
1. Model the **image segmentation** problem as a min-cut: each pixel is a node with edges to source (foreground probability) and sink (background probability). Cut edges between dissimilar pixels. Implement for a small grayscale image.
1. Implement **Dinic's algorithm** with scaling (capacity scaling Dinic). Compare its performance with standard Dinic on networks with large capacities.
1. Use max flow to find the **edge connectivity** of a graph (size of the minimum edge cut). Compute it for the graph in Chapter 12, Exercise 1.
1. Implement the **airline crew assignment** system using min-cost max-flow. Given a list of flights with departure/arrival cities and times, and a list of crews with qualifications, find the minimum-cost assignment covering all flights. Test on a dataset of 20 flights and 8 crews.
1. Build an **image segmentation** tool using graph cuts. Given a grayscale image, allow the user to mark some pixels as foreground/background (seeds), then segment the rest using min-cut. Test on a 100x100 image.
1. Implement **baseball elimination** for an entire league. For each team, compute whether it is eliminated and, if so, identify the subset of teams that certifies the elimination. Test on the 2023 American League standings.

1. Given a **bipartite graph** representing job applicants and jobs, implement a system that finds not just the maximum matching but also lists all maximum matchings. How many maximum matchings exist in a 5x5 complete bipartite graph?
1. Extend the **min-cost max-flow** solver to handle edge lower bounds. Given a network where each edge must carry at least L units and at most U units of flow, find a feasible flow satisfying all lower bounds, then optimize cost. This models real transportation networks with minimum service requirements.

20.17.3 Research

1. (**Push-Relabel**) Implement the Push-Relabel algorithm (Goldberg-Tarjan) for max flow. Compare its performance with Dinic on large networks. Which is faster in practice?
1. (**Global min-cut**) Implement the Stoer-Wagner algorithm for global minimum cut (not just s-t). Compare with running $V-1$ max-flow computations.
1. (**Unit capacity networks**) Prove that Dinic runs in $O(\min(V^{2/3}, \sqrt{E}) \cdot E)$ on unit-capacity networks. Implement and verify experimentally.
1. (**Max flow applications**) Research the reduction of the **circulation with demands** problem to max flow. Implement a solver for the problem: given supplies and demands at nodes, find a feasible flow.

20.18 References and Further Reading

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Chapter 24 (max flow).
- L. R. Ford Jr. and D. R. Fulkerson, "Maximal Flow through a Communication Network" — Canadian Journal of Mathematics, 1956.
- E. A. Dinic, "Algorithm for Solution of a Problem of Maximum Flow in a Network with Power Estimation" — Soviet Math. Doklady, 1970.
- Jack Edmonds and Richard M. Karp, "Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems" — JACM, 1972.
- Andrew V. Goldberg and Robert E. Tarjan, "A New Approach to the Maximum-Flow Problem" — JACM, 1988.
- Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- Mehdi Stoer and Frank Wagner, "A Simple Min-Cut Algorithm" — JACM, 1997.

- John E. Hopcroft and Richard M. Karp, "An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs" — SIAM Journal on Computing, 1973.

Persistent Data Structures

21.1 Introduction and Motivation

Most data structures are **ephemeral**: an update destroys the old version and produces a new one. A **persistent data structure** preserves every previous version after modification, allowing the program to access any historical state efficiently. The cost is not free, but it is far cheaper than cloning the entire structure for every change.

- **Undo/redo**: text editors and IDEs store a linear history of document states; each keystroke creates a new version.
- **Version control**: Git snapshots the file tree at each commit; the full history of every file is available.
- **Functional programming**: languages like Haskell and Clojure use immutable data structures as the default; all variables refer to persistent collections.
- **Databases**: Multi-Version Concurrency Control (MVCC) keeps multiple versions of each row so that readers never block writers.
- **Concurrent access**: immutable structures can be shared across threads without synchronization.

A **mutable** array allows $O(1)$ in-place updates but provides no history. A **persistent** array allows $O(\log n)$ access and update while retaining every previous version. The choice between mutable and persistent depends on whether history is needed.

Operation	Mutable	Persistent
Access	$O(1)$	$O(\log n)$
Update	$O(1)$	$O(\log n)$
Memory per update	$O(1)$	$O(\log n)$
History	None	All versions

21.2 Persistence Models

Partially Persistent. All previous versions of the structure are accessible, but only the **latest** version can be modified. Updates create a new version; older versions are read-only. Most undo systems are partially persistent.

Fully Persistent. All versions are accessible, and **any** version can be modified to produce a new version. This generalises partial persistence: a fully persistent structure is also partially persistent, but the reverse is not true.

Confluently Persistent. In addition to full persistence, two arbitrary versions can be **merged** to create a new version. This is the most general model and the hardest to implement.

Model	Any version readable	Any version writable	Merge
Partially persistent	Yes	Only latest	No
Fully persistent	Yes	Any	No
Confluently persistent	Yes	Any	Yes

Complexity tradeoffs. Driscoll, Sarnak, Sleator, and Tarjan (1989) proved that partial and full persistence can be added to any data structure with $O(1)$ amortised overhead per operation (at the cost of $O(\log n)$ extra space per modified node) without changing the original access time. Confluent persistence is strictly harder and often requires structural sharing.

21.3 Techniques

21.3.1 Fat Nodes

Each node contains a **version log**: an array of (field, value, version) triples. When a field is updated, a new entry is appended to the log. To read a field at a given version, search the log for the most recent entry at or before that version.

```

1 Fat node: node_id = 5
2   field "left"   : version 0 -> 12
3                   version 3 -> 18
4   field "right"  : version 0 -> 14
5                   version 2 -> 20
6   field "key"    : version 0 -> 42
7
8 To read "key" at version 1: find most recent entry <= version 1 -> 42.
9 To read "left" at version 5: find most recent entry <= version 5 -> 18.
```

Space: $O(1)$ per modified field per update. A node touched in v versions stores v entries per modified field. If d fields change, each update costs $O(d)$ extra space.

Time: $O(\log n)$ per access (binary search on the version log), or $O(1)$ amortised if entries are appended in version order.

Limitation: space can grow linearly in the number of updates, making fat nodes unsuitable for structures with very long histories.

21.3.2 Path Copying (Node Copying)

When updating a node in a tree, **copy** the nodes along the path from the root to the modified node. The unchanged subtrees are **shared** between the old and new versions. This produces a new root; the old root still points to the old version.

```

1 Original tree (version 0):
2       10
3      /  \
4     5    15
5    /  \
6   3    7
7
8 After updating 3 -> 33 (version 1):
9       10                                <-- NEW root for version 1
10      /  \
11     [5]  15                            <-- 5 is a NEW node (copied)
12    /  \
13   [33]  7                             <-- 33 is NEW; 7 is SHARED with version 0
14    |    |
15 version 0: 10 -> 5 -> 3 (old, unchanged)
16             5 -> 7 (shared)
17             10 -> 15 (shared)
18
19 The old version is still reachable via the version 0 root.
```

Space: $O(\log n)$ per update for a balanced tree of height $O(\log n)$. Only the path from root to modified node is copied; all other subtrees are shared.

Time: $O(\log n)$ per update (same as the original tree) and $O(\log n)$ per access.

Limitation: creating $O(\log n)$ new nodes per update is more than fat nodes in theory but far more practical in practice because the constants are small and memory is contiguous.

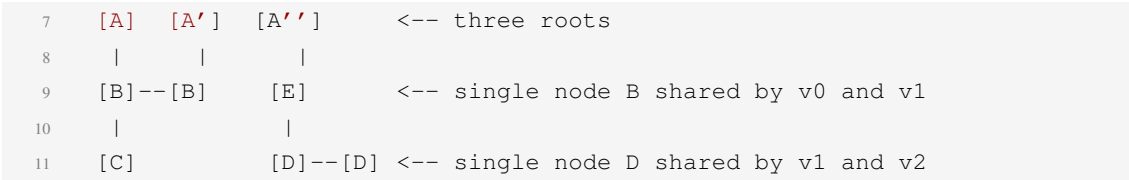
21.3.3 Structural Sharing

Hash consing is a technique where each unique subtree is stored exactly once. When creating a new node, check if an identical node already exists; if so, reuse it. This gives maximum structural sharing: identical subtrees across versions occupy a single copy in memory.

DAG representation represents a version as a directed acyclic graph rather than a tree. Edges point to shared children. This is the basis of functional programming runtime systems (e.g., Clojure vectors, Scala persistent maps).

```

1 Shared structure across versions:
2 Version 0:  A -> (B, C)
3 Version 1:  A' -> (B, D)    -- B is shared, C replaced by D
4 Version 2:  A'' -> (E, D)   -- D is shared with version 1
5
6 DAG representation:
```



21.3.4 Comparing the Techniques

Technique	Update	Access	Space per update
Fat nodes	O(1)	O(log v)	O(1)
Path copying	O(log n)	O(log n)	O(log n)
Hash consing	O(log n)	O(log n)	O(log n) amortised

In practice, path copying on balanced trees is the most widely used technique due to its simplicity and excellent cache behaviour.

21.4 Persistent Arrays

A persistent array of size n supports $\text{read}(\text{index}, \text{version})$ and $\text{update}(\text{index}, \text{value}, \text{version})$ in $O(\log n)$ time. The implementation uses a balanced BST (such as a weight-balanced tree or segment tree) where leaves store array values and internal nodes represent ranges.

```

1  template <typename T>
2  struct parray_node {
3      T value;
4      std::shared_ptr<parray_node> left, right;
5  };
6
7  template <typename T>
8  class persistent_array {
9  public:
10     persistent_array(size_t n, const T& default_val)
11         : n_(n), versions_{build(0, n, default_val)} {}
12
13     T get(size_t idx, size_t ver = 0) const {
14         return get(versions_[ver], 0, n_, idx);
15     }
16
17     persistent_array set(size_t idx, const T& val, size_t ver = 0) {
18         persistent_array result(*this);
19         result.versions_.push_back(
20             set(versions_[ver], 0, n_, idx, val));
21         return result;
22     }
23 }
```

```

24     size_t size() const { return n_; }
25     size_t versions() const { return versions_.size(); }
26
27 private:
28     size_t n_;
29     std::vector<std::shared_ptr<parray_node<T>>> versions_;
30
31     std::shared_ptr<parray_node<T>> build(
32         size_t lo, size_t hi, const T& val) {
33         if (lo + 1 == hi)
34             return std::make_shared<parray_node<T>>(
35                 parray_node<T>{val, nullptr, nullptr});
36         size_t mid = lo + (hi - lo) / 2;
37         auto l = build(lo, mid, val);
38         auto r = build(mid, hi, val);
39         return std::make_shared<parray_node<T>>(
40             parray_node<T>{T{}, l, r});
41     }
42
43     T get(const std::shared_ptr<parray_node<T>>& node,
44         size_t lo, size_t hi, size_t idx) const {
45         if (lo + 1 == hi) return node->value;
46         size_t mid = lo + (hi - lo) / 2;
47         if (idx < mid) return get(node->left, lo, mid, idx);
48         else return get(node->right, mid, hi, idx);
49     }
50
51     std::shared_ptr<parray_node<T>> set(
52         const std::shared_ptr<parray_node<T>>& node,
53         size_t lo, size_t hi, size_t idx, const T& val) {
54         if (lo + 1 == hi)
55             return std::make_shared<parray_node<T>>(
56                 parray_node<T>{val, nullptr, nullptr});
57         size_t mid = lo + (hi - lo) / 2;
58         if (idx < mid)
59             return std::make_shared<parray_node<T>>(
60                 parray_node<T>{T{},
61                     set(node->left, lo, mid, idx, val),
62                     node->right});
63         else
64             return std::make_shared<parray_node<T>>(
65                 parray_node<T>{T{},
66                     node->left,
67                     set(node->right, mid, hi, idx, val)});
68     }
69 };

```

Comparison with `std::vector`: copying a `std::vector` of size n costs $O(n)$ time and $O(n)$ space. The

persistent array costs $O(\log n)$ time and $O(\log n)$ space per version. The persistent array also allows $O(1)$ version counting (`versions_::size`) whereas `std::vector` copies have no version tracking.

21.5 Persistent Linked List

An immutable linked list (cons list) supports $O(1)$ prepend and $O(1)$ head/tail access. Each node is an immutable cons cell pointing to its value and the rest of the list. Structural sharing means two lists that share a suffix share the same nodes in memory.

```

1  template <typename T>
2  struct plist_node {
3      T head;
4      std::shared_ptr<const plist_node<T>> tail;
5  };
6
7  template <typename T>
8  class persistent_list {
9  public:
10     persistent_list() : root_(nullptr) {}
11
12     persistent_list(const T& val,
13                     const persistent_list& rest)
14         : root_(std::make_shared<plist_node<T>>(
15             plist_node<T>{val, rest.root_})) {}
16
17     bool empty() const { return !root_; }
18
19     const T& head() const { return root_>head; }
20
21     persistent_list tail() const {
22         return persistent_list(root_>tail);
23     }
24
25     persistent_list prepend(const T& val) const {
26         return persistent_list(val, *this);
27     }
28
29     size_t size() const {
30         size_t n = 0;
31         auto cur = root_;
32         while (cur) { ++n; cur = cur->tail; }
33         return n;
34     }
35
36 private:
37     std::shared_ptr<const plist_node<T>> root_;
38

```



```

39     explicit persistent_list(
40         std::shared_ptr<const plist_node<T>> node)
41         : root_(node) {}
42 };

```

Complexity: prepend is $O(1)$ (allocate one node, share the tail). Head and tail access are $O(1)$. Length is $O(n)$ unless cached. The list is fully persistent: prepend creates a new version without destroying the old list.

Example usage:

```

1 persistent_list<int> empty;
2 auto v1 = empty.prepend(3).prepend(2).prepend(1);
3 auto v2 = v1.prepend(0);
4
5 // v1: 1 -> 2 -> 3 -> empty
6 // v2: 0 -> 1 -> 2 -> 3 -> empty
7 // v1 is still valid; nodes 1 -> 2 -> 3 -> empty are shared.

```

21.6 Persistent Balanced BST

A persistent balanced BST uses path copying on insert and delete. When inserting a key, the new root is created by copying the nodes along the search path; all other subtrees are shared with the previous version.

21.6.1 Persistent AVL Tree

```

1 template <typename K, typename V>
2 struct pnode {
3     K key;
4     V val;
5     int height;
6     std::shared_ptr<const pnode<K, V>> left, right;
7 };
8
9 template <typename K, typename V>
10 class persistent_avl {
11 public:
12     persistent_avl() : root_(nullptr) {}
13
14     persistent_avl insert(const K& k, const V& v) const {
15         return persistent_avl(insert(root_, k, v));
16     }
17
18     bool contains(const K& k) const {
19         return find(root_, k) != nullptr;
20     }

```

```

21
22     const V& get(const K& k) const {
23         auto node = find(root_, k);
24         return node->val;
25     }
26
27     size_t size() const { return count(root_); }
28
29 private:
30     std::shared_ptr<const pnode<K, V>> root_;
31
32     explicit persistent_avl(
33         std::shared_ptr<const pnode<K, V>> r) : root_(r) {}
34
35     static int ht(
36         const std::shared_ptr<const pnode<K, V>>& n) {
37         return n ? n->height : 0;
38     }
39
40     static int bf(
41         const std::shared_ptr<const pnode<K, V>>& n) {
42         return n ? ht(n->left) - ht(n->right) : 0;
43     }
44
45     static std::shared_ptr<const pnode<K, V>> make(
46         const K& k, const V& v,
47         std::shared_ptr<const pnode<K, V>> l,
48         std::shared_ptr<const pnode<K, V>> r) {
49         return std::make_shared<const pnode<K, V>>(
50             pnode<K, V>{k, v, std::max(ht(l), ht(r)) + 1, l, r});
51     }
52
53     static std::shared_ptr<const pnode<K, V>> rotate_l(
54         std::shared_ptr<const pnode<K, V>> n) {
55         auto y = n->right;
56         return make(y->key, y->val, make(n->key, n->val, n->left, y->left)
57 , y->right);
58     }
59
60     static std::shared_ptr<const pnode<K, V>> rotate_r(
61         std::shared_ptr<const pnode<K, V>> n) {
62         auto y = n->left;
63         return make(y->key, y->val, y->left, make(n->key, n->val, y->right
64 , n->right));
65     }
66
67     static std::shared_ptr<const pnode<K, V>> rebalance(
68         std::shared_ptr<const pnode<K, V>> n) {

```

```

67     if (!n) return n;
68     if (bf(n) > 1) {
69         if (bf(n->left) < 0)
70             n = make(n->key, n->val, rotate_l(n->left), n->right);
71         return rotate_r(n);
72     }
73     if (bf(n) < -1) {
74         if (bf(n->right) > 0)
75             n = make(n->key, n->val, n->left, rotate_r(n->right));
76         return rotate_l(n);
77     }
78     return n;
79 }
80
81 static std::shared_ptr<const pnode<K, V>> insert(
82     const std::shared_ptr<const pnode<K, V>>& n,
83     const K& k, const V& v) {
84     if (!n) return make(k, v, nullptr, nullptr);
85     if (k < n->key)
86         return rebalance(make(n->key, n->val, insert(n->left, k, v), n
87             ->right));
88     if (k > n->key)
89         return rebalance(make(n->key, n->val, n->left, insert(n->right
90             , k, v)));
91     return make(k, v, n->left, n->right);
92 }
93
94 static std::shared_ptr<const pnode<K, V>> find(
95     const std::shared_ptr<const pnode<K, V>>& n, const K& k) {
96     if (!n) return nullptr;
97     if (k < n->key) return find(n->left, k);
98     if (k > n->key) return find(n->right, k);
99     return n;
100 }
101
102 static size_t count(
103     const std::shared_ptr<const pnode<K, V>>& n) {
104     if (!n) return 0;
105     return 1 + count(n->left) + count(n->right);
106 }

```

Complexity: insert is $O(\log n)$ time and $O(\log n)$ new nodes. get and contains are $O(\log n)$ time. Each version has its own root pointer, so all versions are independently accessible.

21.6.2 Merge for Confluent Persistence

Given two versions of a persistent BST, merge can be implemented by inserting all elements of the smaller tree into the larger. This runs in $O(m \log(n/m + 1))$ where m and n are the sizes of the two trees. For balanced trees this produces a balanced result.

21.7 Persistent Segment Tree

Library implementation: `persistent_segment_tree.h` supports point updates and half-open range-sum queries by version.

A persistent segment tree stores a version history of a segment tree. Each update creates $O(\log n)$ new nodes along one root-to-leaf path, sharing all other subtrees with previous versions. This was introduced briefly in Chapter 14; here we extend it with a version tree and range queries across versions.

```

1 struct pst_node {
2     int sum;
3     std::shared_ptr<const pst_node> left, right;
4 };
5
6 using pst_ptr = std::shared_ptr<const pst_node>;
7
8 class persistent_seg {
9 public:
10     persistent_seg(size_t n, const std::vector<int>& init)
11         : n_(n), roots_{build(init, 0, n)} {}
12
13     int query(int ver, int l, int r) const {
14         return query(roots_[ver], 0, n_, l, r);
15     }
16
17     int update(int ver, int pos, int val) const {
18         int old = query(ver, pos, pos);
19         roots_.push_back(update(roots_[ver], 0, n_, pos, val - old));
20         return roots_.size() - 1;
21     }
22
23     size_t versions() const { return roots_.size(); }
24
25 private:
26     size_t n_;
27     std::vector<pst_ptr> roots_;
28
29     pst_ptr build(const std::vector<int>& a, int lo, int hi) {
30         if (lo + 1 == hi)
31             return std::make_shared<pst_node>(
32                 pst_node{lo < (int)a.size() ? a[lo] : 0, nullptr, nullptr}
33             );

```

```

33     int mid = lo + (hi - lo) / 2;
34     auto l = build(a, lo, mid);
35     auto r = build(a, mid, hi);
36     return std::make_shared<pst_node>(
37         pst_node{l->sum + r->sum, l, r});
38 }
39
40 int query(const pst_ptr& node, int lo, int hi, int l, int r) const {
41     if (!node || r <= lo || hi <= l) return 0;
42     if (l <= lo && hi <= r) return node->sum;
43     int mid = lo + (hi - lo) / 2;
44     return query(node->left, lo, mid, l, r)
45         + query(node->right, mid, hi, l, r);
46 }
47
48 pst_ptr update(const pst_ptr& node, int lo, int hi, int pos, int diff)
49     const {
50     if (lo + 1 == hi)
51         return std::make_shared<pst_node>(
52             pst_node{node->sum + diff, nullptr, nullptr});
53     int mid = lo + (hi - lo) / 2;
54     if (pos < mid)
55         return std::make_shared<pst_node>(
56             pst_node{node->sum + diff,
57                 update(node->left, lo, mid, pos, diff),
58                 node->right});
59     else
60         return std::make_shared<pst_node>(
61             pst_node{node->sum + diff,
62                 node->left,
63                 update(node->right, mid, hi, pos, diff)});
64 };

```

Version tree. The array `roots_` stores the root pointer for each version. version i is created by updating version $i-1$, forming a linear chain. In general, versions can form a tree: if version 5 branches from version 3, store `roots_[5] = update(roots_[3], ...)`. This gives a DAG of versions.

K-th element in a version range. Given versions $[v_l, v_r]$ and a value k , find the k -th smallest element present in the difference between the two versions. This is solved by walking the segment tree and computing prefix sums across the two versions at each node – a standard competitive programming technique.

Space: $O(\log n)$ new nodes per update. Total space for m updates is $O(n + m \log n)$.

21.8 Applications

21.8.1 Undo/Redo in a Text Editor

A text editor stores a version history as a persistent rope or persistent array. Each keystroke calls update to create a new version. Undo walks backward through the version array; redo walks forward.

```

1 class editor {
2 public:
3     void type(char c) {
4         history_.push_back(
5             history_.back().append(c));
6     }
7     void undo() {
8         if (cursor_ > 0) --cursor_;
9     }
10    void redo() {
11        if (cursor_ + 1 < (int)history_.size()) ++cursor_;
12    }
13    char get(size_t idx) const {
14        return history_[cursor_].get(idx);
15    }
16 private:
17    std::vector<persistent_array<char>> history_;
18    int cursor_ = 0;
19 };

```

21.8.2 Git-like Snapshotting

A file system stores each file as a persistent byte array. A commit stores the root directory (a persistent tree of persistent files). Diff between two commits compares the tree structures; unchanged subtrees are shared.

21.8.3 Multi-Version Concurrency Control (MVCC)

In a database, each row has multiple versions tagged with transaction timestamps. Readers access a consistent snapshot by reading the version that was current at their start time. Writers create new versions without blocking readers. This is the core of PostgreSQL and MySQL (InnoDB) concurrency control.

21.8.4 Immutable State in Functional Programming

Clojure vectors, Scala persistent maps, and Haskell data structures are all persistent by design. Every “mutation” returns a new version while sharing structure with the old. The runtime relies on hashing and balanced trees (typically 32-way branching tries) to achieve near- $O(1)$ amortised access.

21.9 Complexity Summary

Operation	Mutable	Partially Persistent	Fully Persistent
Array access	$O(1)$	$O(\log n)$	$O(\log n)$
Array update	$O(1)$	$O(\log n)$	$O(\log n)$
BST search	$O(\log n)$	$O(\log n)$	$O(\log n)$
BST insert	$O(\log n)$	$O(\log n)$	$O(\log n)$
Linked list prepend	$O(1)$	$O(1)$	$O(1)$
Linked list access	$O(n)$	$O(n)$	$O(n)$
Segment tree query	$O(\log n)$	$O(\log n)$	$O(\log n)$
Segment tree update	$O(\log n)$	$O(\log n)$	$O(\log n)$
Space per update	$O(1)$	$O(\log n)$	$O(\log n)$

All persistent operations incur an $O(\log n)$ space overhead per update due to path copying. The time overhead is the same as the original data structure because the tree height is unchanged.

21.10 Common Bugs and Pitfalls

- **Memory overhead from path copying:** each update copies $O(\log n)$ nodes. For very large trees with millions of updates, this can exhaust memory. Mitigation: use a garbage collector (reference counting with `shared_ptr`) and periodically compact or snapshot.
- **Forgetting to share unchanged subtrees:** if the insert function does not share the unchanged child pointer, the resulting tree duplicates the entire subtree – losing the $O(\log n)$ space guarantee. Always reuse the old child pointer for the side that is not modified.
- **Reference counting cycles in cyclic persistent structures:** `shared_ptr` reference counting cannot handle cycles (e.g., a doubly-linked persistent list). Use `weak_ptr` or a tracing garbage collector for cyclic graphs.
- **Stack overflow from deep recursion:** a persistent segment tree of depth 30 with path copying can recurse 30 levels deep per operation – usually safe. But a linked list of length 10^6 recurses 10^6 levels. Use iterative traversal for deep persistent structures.
- **Version index out of bounds:** when querying a specific version, verify that the version index is in $[0, \text{versions}::\text{size}())$. Out-of-bounds access is undefined behaviour.

21.11 Exercises

1. Implement a persistent stack (push, pop, top) using a persistent linked list. Verify that all previous versions remain accessible after a sequence of push and pop operations.

2. Prove that a persistent array built from a balanced BST of height h uses $O(h)$ new nodes per update. What is the space complexity for m updates on an array of size n ?
3. Implement a persistent union-find using path copying on the parent array. Analyse the time complexity of n union and find operations.
4. Given a persistent segment tree, implement a function that computes the k -th smallest element in the range $[l, r]$ of a specific version. Write a test that builds 5 versions and queries the k -th element in each.
5. Implement a persistent treap (randomised BST). Compare the number of nodes copied per insert with the persistent AVL tree. Which copies fewer nodes on average?
6. Build a persistent binary indexed tree (Fenwick tree) using path copying. Support point update and prefix sum queries in $O(\log n)$ per version.
7. Implement version diffing: given two versions of a persistent BST, compute the set of keys present in version A but not version B. Optimise by comparing subtree hashes when available.
8. Prove that the total space for m updates on a persistent segment tree over n elements is $O(n + m \log n)$. Describe the worst case and the best case.

21.12 Full Persistence vs Partial Persistence

The distinction between partial and full persistence determines which operations a data structure supports and at what cost.

21.12.1 Partial Persistence

A **partially persistent** structure allows read access to every historical version but restricts modifications to the **latest** version only. Each update to the latest version creates a new version; older versions become immutable. This suffices for most practical applications including undo stacks, version control, and database snapshots.

The Driscoll–Sarnak–Sleator–Tarjan (DST) technique adds partial persistence to any data structure with $O(1)$ amortised overhead per operation.

21.12.2 Full Persistence

A **fully persistent** structure allows read and write access to **any** version. Modifying an old version creates a new version branching from that point, producing a version DAG rather than a version list. The DST technique achieves full persistence with $O(1)$ amortised overhead, but the constant factors are higher.

21.12.3 Fat Nodes vs Path Copying

Fat nodes store a version log inside each node. Updates append to the log; reads binary-search the log. Space is $O(1)$ per modified field per update, but access can be $O(\log v)$ where v is the number of versions that touched the node.

Path copying creates $O(\log n)$ new nodes per update along the root-to-leaf path. Space is $O(\log n)$ per update, but access remains $O(\log n)$ regardless of version count.

Technique	Update time	Access time	Space per update
Fat nodes	$O(1)$	$O(\log v)$	$O(1)$ per field
Path copying	$O(\log n)$	$O(\log n)$	$O(\log n)$
Combined (DST)	$O(1)$ amortised	$O(\log n)$	$O(\log n)$

21.12.4 Confluent Persistence

Confluent persistence adds a merge operation: two arbitrary versions can be combined to produce a new version. This generalises full persistence and models scenarios like branching and merging in version control systems or parallel computation.

21.13 Functional Data Structures

A **purely functional** data structure obeys two invariants: every node is immutable, and every operation returns a new structure without modifying the input. These invariants guarantee that all versions remain accessible and that concurrent access requires no synchronisation.

21.13.1 Lazy Evaluation and Lazy Lists

Lazy evaluation defers computation until the result is actually needed. A **lazy list** (or stream) is a list where each tail is computed on demand. Lazy lists enable elegant implementations of infinite sequences and merge operations on sorted streams.

21.13.2 Lazy Segment Trees

A **lazy segment tree** defers range updates by storing pending operations in internal nodes. Combined with functional persistence, each update creates a new version while sharing unchanged subtrees.

21.13.3 Amortised Analysis: Banker's Method

The **banker's method** assigns a “credit” to each element. Operations that are expensive in the worst case are paid for by credits accumulated from earlier cheap operations. For functional queues implemented as two lazy lists, the banker's method proves that head, tail, and enqueue are all $O(1)$ amortised.

21.13.4 Comparison with Imperative Counterparts

Structure	Imperative	Functional	Use case
Stack	$O(1)$	$O(1)$	Undo history
Queue	$O(1)$ amortised	$O(1)$ amortised	Event processing
BST	$O(\log n)$	$O(\log n)$	Ordered collections
Segment tree	$O(\log n)$	$O(\log n)$	Range queries
Union-find	$O(\alpha(n))$	$O(\log n)$	Connectivity

21.14 The Differential File

Database systems must provide durability: committed transactions survive crashes. Persistent data structures in databases are implemented through on-disk techniques that share the same principles of structural sharing.

21.14.1 Shadow Paging

Shadow paging divides the disk into fixed-size pages. When a page is modified, a **shadow copy** is written to a new location; the page table is updated to point to the new copy. On commit, the new page table is atomically written to disk. On crash, the old page table is recovered.

21.14.2 Differential File

A **differential file** separates the database into a large **base file** (read-only) and a small **differential file** (append-only). All writes go to the differential file. Reads check the differential file first. Periodically, the differential file is **merged** into the base file.

21.14.3 Write-Ahead Logging

Write-ahead logging (WAL) handles logical recovery. Every modification is first recorded in a sequential log on disk before any page is written. The WAL protocol states: a log record must be written to stable storage before the corresponding data page is written to disk.

21.14.4 Multi-Version Concurrency Control

MVCC combines differential files with versioning: each row has multiple versions tagged with transaction timestamps. Readers access a consistent snapshot by reading the version current at their start time. Writers create new versions without blocking readers.

21.15 Retroactive Data Structures

A **retroactive data structure** supports modifications not only at the current time but also at **past** times. This generalises persistence: while a persistent structure records all versions and lets you query any of them, a retroactive structure lets you **change history** and propagate the consequences forward.

21.15.1 Partial Retroactivity

In **partial retroactivity**, insertions and deletions can be applied at past times, but queries must be answered at the current time. The Demaine–Lacki–Mitzenmacher framework achieves $O(\sqrt{m})$ overhead per operation by partitioning the timeline into blocks of size $\sqrt{m}$.

21.15.2 Full Retroactivity

Full retroactivity allows modifications at past times and queries at any past time. The DLM framework achieves $O(\sqrt{m} \log^{3/2} m)$ per operation using a balanced BST of time blocks, each storing a persistent data structure.

21.15.3 Applications

- **Time-travel debugging:** record the full execution history, jump to any past state
- **Undo/redo systems:** branching undo trees for collaborative editing
- **Causal analysis:** “what if this transaction had failed?” queries in databases

Model	Past modifications	Past queries	Complexity
Persistent	No	Yes	$O(\log n)$ per op
Partially retroactive	Yes	No	$O(\sqrt{m})$ per op
Fully retroactive	Yes	Yes	$O(\sqrt{m} \log^{3/2} m)$ per op

21.16 Nonoblivious Retroactivity

Standard retroactivity lets you modify past updates. **Nonoblivious retroactivity** goes further: you specify a *query* on the timeline, and the data structure reports the *first* past update that changed the query’s answer.

21.16.1 Problem Formulation

Given a sequence of updates $\sigma_1, \dots, \sigma_m$ and a query q , find the earliest time t such that $q(\Sigma_t) \neq q(\Sigma_{t-1})$, where $\Sigma_t = \sigma_1 \cdots \sigma_t$. If no such update exists, return “unchanged.”

21.16.2 Priority Queue Example

For a priority queue, the nonoblivious retroactive query asks: “what is the first past update that changed the minimum?” The approach uses geometric decomposition of the timeline into blocks of size $1, 2, 4, \dots$, maintaining snapshots at block boundaries. Scanning backward through blocks and replaying updates within each block yields $O(\lg n)$ amortized per operation.

21.16.3 Applications

- **Debugging:** identify the first update that caused an incorrect output
- **What-if analysis:** “when did this constraint first become violated?”
- **Causal reasoning:** trace the earliest event that altered an outcome

21.17 Data Structure Decomposition

Complex persistent data structures can be built by **composing** simpler persistent components. This decomposition approach reduces implementation effort, improves correctness, and often matches the performance of monolithic designs.

21.17.1 Persistent Union-Find via Decomposition

A persistent union-find can be decomposed into a **persistent array** for parent pointers and a **union-find logic layer** that performs unions without path compression, relying on the persistent array to track history.

21.17.2 Persistent Graph via Adjacency Lists

A persistent graph decomposes into a **persistent map** from vertex IDs to **persistent adjacency lists**. Each edge addition creates a new version of the adjacency list for both endpoints.

21.17.3 Compositional Design Principles

1. **Identify independent dimensions.** Each dimension becomes a separate persistent component.
2. **Use the simplest component that suffices.** Match complexity to the problem.
3. **Share versions across components.** Use a shared version counter for atomic updates.
4. **Test components independently.** Each persistent component has its own test suite.

The compositional approach is powerful but has a caveat: the overhead of persistent components multiplies. A two-level decomposition with $O(\log n)$ components at each level yields $O(\log^2 n)$ per operation.

21.18 References and Further Reading

- James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan, “Making Data Structures Persistent” – Journal of Computer and System Sciences, 1989. The foundational paper on persistence, proving that partial and full persistence can be added to any data structure with $O(1)$ amortised overhead.
- Chris Okasaki, *Purely Functional Data Structures* – Cambridge University Press, 1998. The definitive reference for persistent data structures in functional programming. Covers lazy vs strict, real-time vs amortised, and many concrete data structures.
- Erik Meijer, Maarten de Figueiredo, and Leo Burd, “The Design and Implementation of Functional Collections in Scala” – Functional Programming in Scala (Red Book), 2015. Describes hash-array mapped tries (HAMTs), the basis of Scala’s persistent collections.
- Rich Hickey, “Persistent Data Structures and Managed References” – Clojure design document, 2009. Describes the 32-way branching persistent vector and persistent hash map used in Clojure.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th Edition – Chapter 13 (Red-Black Trees), Chapter 19 (Fibonacci Heaps). Background on balanced BSTs used in persistent constructions.
- Gunnar Peyer, “Persistent Data Structures” – Master’s Thesis, ETH Zurich, 2010. A comprehensive survey of persistence techniques with performance benchmarks.

Succinct Data Structures

22.1 Introduction

A data structure is **succinct** if it stores data in space close to the information-theoretic lower bound while supporting queries efficiently. Given a set of n elements drawn from a universe $\{0, 1, \dots, m-1\}$, the minimum number of bits needed to represent the set is $n \log(m/n)$ bits (this follows from counting the number of such sets and taking the logarithm). A succinct representation uses this lower bound plus $o(n)$ extra bits, while still supporting the required operations.

- **Implicit:** stores the set in exactly the information-theoretic minimum bits (no extra space). Usually cannot support any non-trivial queries.
- **Succinct:** uses $H + o(n)$ bits where H is the information-theoretic minimum. Supports queries in $O(1)$ or $O(\log n)$ time.
- **Compressed:** uses $O(n)$ bits (or $O(H)$ bits), but does not necessarily support efficient queries. A compressed data structure may use more space than a succinct one if $H \ll n$.

A concrete comparison: storing an array of n integers from $\{0, \dots, 2^{32} - 1\}$ naively costs $32n$ bits. If the array is a permutation, the information-theoretic minimum is $n \log n$ bits (about $20n$ bits for $n = 10^6$). A succinct representation stores the permutation in $n \log n + o(n)$ bits with $O(1)$ access time.

Applications:

- Full-text indexes: succinct suffix arrays and FM-indexes index terabyte-scale text in a fraction of the original size.
- Web search: search engines use succinct data structures to index billions of pages within memory limits.
- Bioinformatics: genome sequences (billions of base pairs) are indexed using FM-indexes built on rank/select.
- Compressed databases: SQLite and other embedded databases use succinct structures for query execution on compressed data.

22.2 Rank and Select on Bitvectors

Two fundamental operations on bitvectors are **rank** and **select**. They form the building blocks of almost all succinct data structures.

- **rank(*b*, *i*)**: the number of 1-bits in the first *i* positions of bitvector *b*. Formally, $\text{rank}_b(i) = |\{j < i : b[j] = 1\}|$.
- **select(*b*, *i*)**: the position of the *i*-th 1-bit in bitvector *b*. Formally, $\text{select}_b(i) = \min\{j : \text{rank}_b(j+1) = i+1\}$.

Both operations can be supported in $O(1)$ time with a small amount of extra space. The key idea is **block decomposition**: partition the bitvector into blocks of $\log^2 n$ bits each. For each block, store the rank at the block boundary. Within each block, further divide into superblocks of $\log n$ bits and store the rank at superblock boundaries. The total extra space is $O(n/\log n)$ bits, and answering a rank query requires only a constant number of table lookups.

Select can be answered in $O(1)$ time using a similar decomposition combined with a precomputed table, or in $O(\log n)$ time using binary search with rank queries. A practical approach uses interpolation search within blocks since the density of 1-bits is known.

```

1 class succinct_bitvector {
2 public:
3     succinct_bitvector(const std::vector<bool>& bits)
4         : n_(bits.size()),
5           blocks_((n_ + 63) / 64),
6           bits_(blocks_, 0),
7           block_rank_(blocks_, 0)
8     {
9         for (size_t i = 0; i < n_; ++i)
10             if (bits[i]) bits_[i / 64] |= 1ULL << (i % 64);
11
12         uint64_t running = 0;
13         for (size_t i = 0; i < blocks_; ++i) {
14             block_rank_[i] = running;
15             running += __builtin_popcountll(bits_[i]);
16         }
17     }
18
19     uint64_t rank(uint64_t i) const {
20         uint64_t block = i / 64;
21         uint64_t pos = i % 64;
22         uint64_t r = block > 0 ? block_rank_[block - 1] : 0;
23         uint64_t mask = (pos == 64) ? ~0ULL : (1ULL << pos) - 1;
24         r += __builtin_popcountll(bits_[block] & mask);
25         return r;
26     }
27
28     uint64_t select(uint64_t k) const {
29         uint64_t lo = 0, hi = n_;
30         while (lo < hi) {
31             uint64_t mid = lo + (hi - lo) / 2;
32             if (rank(mid) <= k) lo = mid + 1;

```

```

33         else hi = mid;
34     }
35     return lo;
36 }
37
38 uint64_t operator[](uint64_t i) const {
39     return (bits_[i / 64] >> (i % 64)) & 1;
40 }
41
42 uint64_t size() const { return n_; }
43
44 private:
45     uint64_t n_, blocks_;
46     std::vector<uint64_t> bits_;
47     std::vector<uint64_t> block_rank_;
48 };

```

Applications of rank and select:

- Converting between binary and unary representations: rank converts a bitvector to an array of integers (positions of 1-bits); select does the inverse.
- Counting occurrences in a range: $\text{rank}(r) - \text{rank}(l)$ counts 1-bits in $[l, r)$.
- Finding the k -th occurrence of a symbol in a string (used by wavelet trees and FM-indexes).

22.3 Wavelet Tree

Library implementation: `wavelet_tree.h` provides byte-string access, rank, select, and range-count queries.

The **wavelet tree** (Grossi, Gupta, and Vitter, 2003) is a multi-level binary partition of an alphabet that supports rank, select, and range count queries in $O(\log \sigma)$ time, where σ is the alphabet size. Each node of the tree stores a bitvector marking which elements of the current sub-alphabet go left (lower half) versus right (upper half).

22.3.1 Structure

For the string `abracadabra` with alphabet $\{a, b, c, d, r\}$:

```

1 Wavelet tree for "abracadabra":
2
3         [a b c d r]
4         /       \
5     [a b]       [c d r]
6     / \       / \
7 [a] [b]   [c] [d r]
8                / \

```



```

9             [d] [r]
10
11 Root bitvector (left=0, right=1):
12   a b r a c a d a b r a
13   0 0 1 0 1 0 1 0 0 1 0   (a,b go left; r,c,d go right)
14
15 Left child bitvector (a vs b):
16   a b . a . a . a b . a
17   0 1 . 0 . 0 . 0 1 . 0   (a=0, b=1)
18
19 Right child bitvector (c vs d,r):
20   . . r . c . d . . r .
21   . . 1 . 0 . 0 . . 1 .   (c=0, d,r go right)
22
23 Far-right child (d vs r):
24   . . r . . . d . . r .
25   . . 1 . . . 0 . . 1 .   (d=0, r=1)

```

22.3.2 Operations

access(i): starting at the root, read bitvector[pos]. If 0, go left; if 1, go right. The symbol is found at the leaf. Time: $O(\log \sigma)$.

rank(c, i): at the root, determine if c goes left or right. If c goes left, replace i with $\text{rank0}(i)$ and recurse left. If c goes right, replace i with $\text{rank1}(i)$ and recurse right. Time: $O(\log \sigma)$.

select(c, i): starting at the leaf for c , walk upward. At each parent, if c came from the left child, find the position in the parent bitvector whose rank0 equals i . If from the right, use rank1 . Time: $O(\log \sigma)$.

range count(l, r, c_lo, c_hi): count occurrences of symbols in $[c_lo, c_hi]$ within $[l, r]$ of the text. This is answered by recursing into the wavelet tree, narrowing the alphabet range at each level. Time: $O(\log \sigma)$.

```

1 struct wavelet_node {
2     std::vector<bool> bits;
3     std::vector<uint64_t> block_rank;
4     std::unique_ptr<wavelet_node> left, right;
5     char lo, hi;
6
7     uint64_t rank0(uint64_t i) const {
8         uint64_t r = 0;
9         for (uint64_t j = 0; j < i; ++j)
10             if (!bits[j]) ++r;
11         return r;
12     }
13
14     uint64_t rank1(uint64_t i) const {
15         return i - rank0(i);
16     }

```

```

17 };
18
19 class wavelet_tree {
20 public:
21     wavelet_tree(const std::string& s) {
22         std::set<char> alpha(s.begin(), s.end());
23         root_ = build(s, alpha.begin(), alpha.end());
24     }
25
26     char access(uint64_t i) const {
27         return access(root_.get(), i);
28     }
29
30     uint64_t rank(char c, uint64_t i) const {
31         return rank(root_.get(), c, i);
32     }
33
34     uint64_t select(char c, uint64_t k) const {
35         return select(root_.get(), c, k);
36     }
37
38     uint64_t range_count(uint64_t l, uint64_t r,
39                          char lo, char hi) const {
40         return range_count(root_.get(), l, r, lo, hi);
41     }
42
43 private:
44     std::unique_ptr<wavelet_node> root_;
45
46     std::unique_ptr<wavelet_node> build(
47         const std::string& s,
48         std::set<char>::iterator lo,
49         std::set<char>::iterator hi)
50     {
51         if (lo == hi) return nullptr;
52         if (std::next(lo) == hi) {
53             auto node = std::make_unique<wavelet_node>();
54             node->lo = node->hi = *lo;
55             node->bits.assign(s.size(), false);
56             return node;
57         }
58         auto mid = lo;
59         std::advance(mid, std::distance(lo, hi) / 2);
60         auto node = std::make_unique<wavelet_node>();
61         node->lo = *lo;
62         node->hi = *std::prev(hi);
63
64         std::string left_str, right_str;

```

```

65     for (char c : s) {
66         bool go_left = (c < *mid);
67         node->bits.push_back(go_left);
68         if (go_left) left_str += c;
69         else right_str += c;
70     }
71     node->left = build(left_str, lo, mid);
72     node->right = build(right_str, mid, hi);
73     return node;
74 }
75
76 char access(const wavelet_node* node, uint64_t i) const {
77     if (!node) return node->lo;
78     if (node->left == nullptr && node->right == nullptr)
79         return node->lo;
80     if (!node->bits[i]) {
81         uint64_t new_i = node->rank0(i);
82         return access(node->left.get(), new_i);
83     } else {
84         uint64_t new_i = node->rank1(i);
85         return access(node->right.get(), new_i);
86     }
87 }
88
89 uint64_t rank(const wavelet_node* node, char c,
90              uint64_t i) const {
91     if (!node) return 0;
92     if (node->left == nullptr && node->right == nullptr)
93         return i;
94     if (c <= node->left->hi) {
95         uint64_t new_i = node->rank0(i);
96         return rank(node->left.get(), c, new_i);
97     } else {
98         uint64_t new_i = node->rank1(i);
99         return rank(node->right.get(), c, new_i);
100    }
101 }
102
103 uint64_t select(const wavelet_node* node, char c,
104                uint64_t k) const {
105     if (!node) return k;
106     if (node->left == nullptr && node->right == nullptr)
107         return k;
108     uint64_t child_sel;
109     if (c <= node->left->hi)
110         child_sel = select(node->left.get(), c, k);
111     else
112         child_sel = select(node->right.get(), c, k);

```

```

113     return node->bits[child_sel] ? node->rank1(child_sel + 1)
114                               : node->rank0(child_sel + 1) - 1;
115 }
116
117 uint64_t range_count(const wavelet_node* node,
118                     uint64_t l, uint64_t r,
119                     char lo, char hi) const {
120     if (!node || l >= r) return 0;
121     if (hi < node->lo || lo > node->hi) return 0;
122     if (lo <= node->lo && node->hi <= hi)
123         return r - l;
124     uint64_t l0 = node->rank0(l), r0 = node->rank0(r);
125     uint64_t l1 = node->rank1(l), r1 = node->rank1(r);
126     uint64_t total = 0;
127     if (lo <= node->left->hi)
128         total += range_count(node->left.get(), l0, r0, lo, hi);
129     if (hi >= node->right->lo)
130         total += range_count(node->right.get(), l1, r1, lo, hi);
131     return total;
132 }
133 };

```

Complexity: each operation traverses at most $\log \sigma$ levels, doing $O(1)$ work at each. Space: $O(n \log \sigma)$ bits total across all bitvectors (the sum of bitvector lengths at each level equals n). With block decomposition on each bitvector, the overhead is $O(n \log \sigma / \log n)$ extra bits.

Applications: range median queries, range quantile queries, string indexing, frequency counting, and as a component of the FM-index.

22.4 Burrows-Wheeler Transform and FM-Index

22.4.1 Burrows-Wheeler Transform

The **Burrows-Wheeler Transform** (BWT) permutes a string by sorting all of its rotations lexicographically. The transformed string is the last column of the sorted rotation matrix. The BWT is reversible: given the transformed string and the original string's starting position in the sorted matrix, the original string can be recovered.

Example: for the string `banana$`:

```

1  Sorted rotations of "banana$":
2
3  $ a n a n a      Last      First
4  a $ a n a n      column    column
5  a n $ a n a      -----
6  a n a $ a n      -> a n n $  $ a a a
7  a n a n $ a      n a a a  a n n a
8  a n a n a $      a a n n  n a $ a

```

```

 9   n a $ a n a      n $ a a n a a $
10   n a n a $ a      $ a n a
11
12   BWT("banana$") = "annb$aa"
13   (concatenating last column from top to bottom)

```

The BWT does not compress by itself, but it groups similar characters together, making the output highly compressible by run-length encoding or other methods.

22.4.2 FM-Index

The **FM-Index** (Ferragina and Manzini, 2000) is a succinct full-text index that uses the BWT together with rank on auxiliary bitvectors. It supports substring search in time proportional to the pattern length, using $O(n)$ bits of space.

The FM-Index maintains, for each character c , a bitvector B_c of length $n + 1$ where $B_c[i] = 1$ if the i -th character of the BWT is c . A C-array stores the number of characters lexicographically smaller than c in the original text.

LF-mapping: the key property is that the i -th occurrence of character c in the BWT corresponds to the i -th occurrence of c in the original text. This mapping, called the LF-mapping, is:

$$\text{LF}(i) = C[\text{BWT}[i]] + \text{rank}_{B_c}(i)$$

where $C[c]$ counts characters lexicographically smaller than c .

Backward search: to find a pattern $P = p_1 p_2 \dots p_m$, start with an interval $[lo, hi) = [0, n)$ and extend backward. For each character p_j from right to left:

$$lo = C[p_j] + \text{rank}_{B_{p_j}}(lo), \quad hi = C[p_j] + \text{rank}_{B_{p_j}}(hi)$$

If $lo < hi$ after processing all characters, the pattern occurs $hi - lo$ times in the text. Time: $O(m)$ with precomputed rank, or $O(m \log n)$ with binary-search rank.

```

1  class fm_index {
2  public:
3      fm_index(const std::string& text) : n_(text.size()) {
4          build_bwt(text);
5      }
6
7      int count(const std::string& pattern) const {
8          int lo = 0, hi = n_;
9          for (int i = (int)pattern.size() - 1; i >= 0; --i) {
10             char c = pattern[i];
11             lo = C_[c] + rank(c, lo);
12             hi = C_[c] + rank(c, hi);
13             if (lo >= hi) return 0;
14         }

```

```

15         return hi - lo;
16     }
17
18     std::vector<int> find(const std::string& pattern) const {
19         int lo = 0, hi = n_;
20         for (int i = (int)pattern.size() - 1; i >= 0; --i) {
21             char c = pattern[i];
22             lo = C_[c] + rank(c, lo);
23             hi = C_[c] + rank(c, hi);
24             if (lo >= hi) return {};
25         }
26         std::vector<int> occ;
27         for (int i = lo; i < hi; ++i)
28             occ.push_back(locate(i));
29         return occ;
30     }
31
32 private:
33     int n_;
34     std::vector<char> bwt_;
35     std::vector<int> C_;
36     std::vector<std::vector<int>> occ_;
37
38     void build_bwt(const std::string& text) {
39         n_ = text.size();
40         std::vector<int> sa(n_);
41         std::iota(sa.begin(), sa.end(), 0);
42         std::sort(sa.begin(), sa.end(), [&](int a, int b) {
43             return text.compare(a, n_ - a, text, b, n_ - b) < 0;
44         });
45
46         bwt_.resize(n_);
47         for (int i = 0; i < n_; ++i)
48             bwt_[i] = text[(sa[i] + n_ - 1) % n_];
49
50         C_.assign(256, 0);
51         for (char c : bwt_) ++C_[c];
52         int sum = 0;
53         for (int i = 0; i < 256; ++i) {
54             int tmp = C_[i];
55             C_[i] = sum;
56             sum += tmp;
57         }
58
59         occ_.assign(256, std::vector<int>(n_ + 1, 0));
60         for (int i = 0; i < n_; ++i) {
61             for (int c = 0; c < 256; ++c)
62                 occ_[c][i + 1] = occ_[c][i];

```

```

63         ++occ_[(int)bwt_[i]][i + 1];
64     }
65 }
66
67 int rank(char c, int i) const {
68     return occ_[(int)c][i];
69 }
70
71 int locate(int pos) const {
72     int steps = 0;
73     while (steps < n_) {
74         char c = bwt_[pos];
75         pos = C_[c] + occ_[(int)c][pos];
76         ++steps;
77     }
78     return (pos + 1) % n_;
79 }
80 };

```

Applications:

- Bioinformatics: FM-indexes index entire human genomes (3 billion base pairs) in a few gigabytes, enabling fast substring search for read alignment.
- Compression: the BWT is the core of bzip2; FM-indexes combine compression with search.
- Full-text search: the FM-index is the basis of the succinct index in many text search libraries.

22.5 Succinct Trees

A tree with n nodes uses $O(n \log n)$ bits with pointer-based representations (each pointer is $\log n$ bits). A succinct representation stores the tree in $2n + o(n)$ bits while supporting navigation operations in $O(1)$ time.

22.5.1 Balanced Parentheses

A rooted ordered tree can be encoded as a balanced parentheses sequence: each node is represented by an opening parenthesis $($ (preorder visit) and a closing parenthesis $)$ (postorder visit). For a tree with n nodes, the sequence has exactly $2n$ parentheses.

1	Tree:	Encoding:
2	1	(() (() ())
3	/ \	
4	2 3	Node 1: (...)
5	/ / \	
6	4 5 6	Node 2: () -- leaf
7		
8	7 8	Node 3: (() ())

Navigation with rank and select:

- **parent(u):** find the matching open-parenthesis before position u using rank and select.
- **first_child(u):** if $b[u + 1] = 1$ (opening), then $u + 1$ is the first child. Otherwise, u has no children.
- **next_sibling(u):** find the matching close-parenthesis of u , then the next position is the next sibling.
- **subtree_size(u):** the number of pairs within the matched parentheses is the subtree size.

All these operations use $O(1)$ rank/select queries, giving $O(1)$ navigation time.

22.5.2 LOUDS

Level-Order Unary Degree Sequence (LOUDS) encodes a tree by listing each node's degree in unary (the degree d is written as d ones followed by a zero). For a tree with n nodes and total degree $\sum d = n - 1$, the sequence has $2n$ bits.

```

1 class succinct_tree_bp {
2 public:
3     succinct_tree_bp(const std::vector<std::vector<int>>& adj,
4                       int root) {
5         n_ = 0;
6         encode(adj, root);
7         build_rank();
8     }
9
10    int parent(int u) const {
11        int match = find_matching(u);
12        return select0(rank0(match)) / 2;
13    }
14
15    int first_child(int u) const {
16        if (parens_[u + 1] == 1) return u + 1;
17        return -1;
18    }
19
20    int subtree_size(int u) const {
21        int match = find_matching(u);
22        return (match - u + 1) / 2;
23    }
24
25    int depth() const { return max_depth_; }
26
27    size_t memory_bits() const {
28        return parens_.size() + block_rank_.size() * 64;
29    }
30

```



```

31 private:
32     int n_;
33     std::vector<bool> parens_;
34     std::vector<uint64_t> block_rank_;
35     int max_depth_ = 0;
36
37     void encode(const std::vector<std::vector<int>>& adj,
38                 int u) {
39         parens_.push_back(true);
40         ++n_;
41         int depth = 0;
42         for (int v : adj[u]) {
43             encode(adj, v);
44             ++depth;
45         }
46         parens_.push_back(false);
47         max_depth_ = std::max(max_depth_, depth);
48     }
49
50     void build_rank() {
51         uint64_t blocks = (parens_.size() + 63) / 64;
52         block_rank_.resize(blocks, 0);
53         uint64_t running = 0;
54         for (uint64_t i = 0; i < blocks; ++i) {
55             block_rank_[i] = running;
56             uint64_t start = i * 64;
57             uint64_t end = std::min(start + 64, parens_.size());
58             for (uint64_t j = start; j < end; ++j)
59                 if (parens_[j]) ++running;
60         }
61     }
62
63     uint64_t rank0(uint64_t i) const {
64         return i - rank1(i);
65     }
66
67     uint64_t rank1(uint64_t i) const {
68         uint64_t block = i / 64;
69         uint64_t pos = i % 64;
70         uint64_t r = block > 0 ? block_rank_[block - 1] : 0;
71         uint64_t start = block * 64;
72         uint64_t mask = (pos == 64) ? ~0ULL : (1ULL << pos) - 1;
73         uint64_t word = 0;
74         for (uint64_t j = 0; j < 64 && start + j < parens_.size(); ++j)
75             if (parens_[start + j]) word |= 1ULL << j;
76         r += __builtin_popcountll(word & mask);
77         return r;
78     }

```

```

79
80     uint64_t select0(uint64_t k) const {
81         uint64_t lo = 0, hi = parens_.size();
82         while (lo < hi) {
83             uint64_t mid = lo + (hi - lo) / 2;
84             if (rank0(mid) <= k) lo = mid + 1;
85             else hi = mid;
86         }
87         return lo;
88     }
89
90     int find_matching(int pos) const {
91         int depth = 0;
92         for (int i = pos; i < (int)parens_.size(); ++i) {
93             if (parens_[i]) ++depth;
94             else {
95                 --depth;
96                 if (depth == 0) return i;
97             }
98         }
99         return -1;
100     }
101 };

```

Space comparison: a pointer-based binary tree uses $O(n \log n)$ bits. The balanced-parentheses and LOUDS representations use exactly $2n$ bits ($2n + o(n)$ with auxiliary structures). For $n = 10^6$, the pointer-based tree uses about 24 MB; the succinct representation uses about 250 KB.

22.6 Compressed Suffix Arrays

A **suffix array** stores the lexicographic order of all n suffixes of a string of length n . The naive representation uses $n \log n$ bits (one integer per suffix). A **compressed suffix array** stores this information in $O(n)$ bits while supporting the same queries.

The FM-Index is one form of compressed suffix array: it encodes the suffix array implicitly through the BWT. Other forms include:

- **SSA** (Succinct Suffix Array): stores the suffix array using a wavelet tree on the BWT, supporting rank, select, and pattern matching.
- **PSA** (Compressed Suffix Array): stores the suffix array as a sequence of differences and compresses using wavelet trees or entropy-bounded integers.
- **LZ-indexes**: based on Lempel-Ziv decomposition rather than suffix arrays, offering different space-time tradeoffs.

All these structures achieve $O(n)$ bits of space and $O(m)$ or $O(m \log n)$ pattern matching time, where m is the pattern length.

22.7 Applications and Industry Use

- **Search engines:** Google and Bing use succinct indexes for web-scale full-text search. The FM-index variant indexes trillions of tokens within memory budgets that would be impossible with naive structures.
- **Genome browsers:** tools like BWA and Bowtie use FM-indexes to align short DNA reads to reference genomes. The human genome (3 billion base pairs) is indexed in about 4 GB using an FM-index.
- **Compressed databases:** SQLite and LevelDB use succinct bitvectors and wavelet trees for compressed column storage and range queries without decompressing the entire dataset.
- **In-memory caches:** Redis modules and Memcached plugins use succinct data structures to increase cache capacity without adding memory hardware.
- **Log analysis:** succinct wavelet trees support frequency histograms and range quantiles on compressed log data, enabling analytics without full decompression.

22.8 Common Bugs and Pitfalls

- **Off-by-one in rank/select:** rank and select may be defined as 0-indexed or 1-indexed depending on the convention. Mixing conventions within the same code produces silent errors. Pick one convention and use it consistently.
- **select boundary conditions:** $\text{select}(k)$ where k exceeds the number of 1-bits is undefined. Always check that $k < \text{rank}(n)$ before calling select.
- **Wavelet tree alphabet partitioning:** if the alphabet is not partitioned correctly at each level, rank and select return wrong values. The split point must divide the current sub-alphabet roughly in half to maintain $O(\log \sigma)$ depth.
- **BWT construction with sentinel:** the BWT requires a unique sentinel character (e.g., $\$$) at the end of the string. Omitting the sentinel causes the first and last rotations to collide in the sorted matrix.
- **Block size tuning:** for bitvector rank, block size $\log^2 n$ is theoretically optimal but may be too large for small bitvectors. For bitvectors under 1024 bits, a simple $O(n)$ scan for rank is faster than block decomposition.
- **Memory alignment:** succinct data structures pack bits tightly, which can cause unaligned memory access on some architectures. Use compiler intrinsics (`_builtin_popcountll`) and ensure arrays are allocated on aligned boundaries.

22.9 Complexity Summary

Operation	Time	Space	Structure
Bitvector rank(<i>b</i> , <i>i</i>)	$O(1)$	$O(n / \log n)$ extra	Block decomposition
Bitvector select(<i>b</i> , <i>i</i>)	$O(1)$	$O(n / \log n)$ extra	Block decomposition + table
Wavelet tree access(<i>i</i>)	$O(\log \sigma)$	$O(n \log \sigma)$ bits	Wavelet tree
Wavelet tree rank(<i>c</i> , <i>i</i>)	$O(\log \sigma)$	$O(n \log \sigma)$ bits	Wavelet tree
Wavelet tree select(<i>c</i> , <i>k</i>)	$O(\log \sigma)$	$O(n \log \sigma)$ bits	Wavelet tree
Wavelet tree range count	$O(\log \sigma)$	$O(n \log \sigma)$ bits	Wavelet tree
FM-index pattern count	$O(m)$	$O(n)$ bits	BWT + rank bitvectors
FM-index locate	$O(m + \text{occ})$	$O(n)$ bits	BWT + rank bitvectors
Succinct tree parent	$O(1)$	$2n + o(n)$ bits	Balanced parentheses
Succinct tree first_child	$O(1)$	$2n + o(n)$ bits	Balanced parentheses
Succinct tree subtree_size	$O(1)$	$2n + o(n)$ bits	Balanced parentheses

22.10 Exercises

1. Implement rank and select on a bitvector of 1 million random bits. Measure the space overhead (in bits per element) and verify that rank answers in $O(1)$ time empirically by timing 100,000 random queries.
2. Build a wavelet tree for the string `mississippi`. Verify that `rank(s, 7)` returns the correct count and that `select(s, 3)` returns the position of the third `s`. Trace the algorithm by hand and confirm the result matches.
3. Construct the BWT of the string `cacao$` by hand: list all rotations, sort them, and read the last column. Then verify that you can recover the original string using the LF-mapping.
4. Implement an FM-index for a small text (at most 1000 characters). Test pattern matching on `abracadabra` and verify that the count for `abra` returns 2.
5. Compare the space of a pointer-based binary tree with $n = 10^5$ nodes to the balanced-parentheses succinct representation. Print both sizes in bytes.
6. Prove that a bitvector of length n with k set bits can be represented in $nH_0 + o(n)$ bits, where $H_0 = -\frac{k}{n} \log_2 \frac{k}{n} - \frac{n-k}{n} \log_2 \frac{n-k}{n}$ is the zeroth-order entropy. Hint: this is the number of distinct bitvectors with exactly k ones.
7. Implement a succinct bitvector with block size $\log^2 n$ and compare its rank performance with a naive $O(n)$ scan for bitvectors of size $10^4, 10^5, 10^6$. At what size does block decomposition become faster?

8. Given a wavelet tree for a DNA string (alphabet $\{A, C, G, T\}$), implement a function that returns the frequency of each base in a given range $[l, r]$. Analyse the complexity and compare with a prefix-sum approach.

22.11 References and Further Reading

- Jacobson, G. (1989). “Succinct Static Data Structures.” PhD thesis, Carnegie Mellon University. The foundational work on succinct data structures, introducing balanced parentheses and LOUDS encodings for trees.
- Grossi, R., Gupta, A., and Vitter, J. S. (2003). “High-Order Entropy-Compressed Text Indexes.” Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). Introduces the wavelet tree and its applications to text indexing.
- Ferragina, P. and Manzini, G. (2005). “An Alphabet-Friendly Succinct Index.” Journal of the ACM, 52(3). Introduces the FM-index, showing how the BWT combined with rank/select gives $O(n)$ bit full-text indexing.
- Sadakane, K. (2007). “Succinct Representations of Large Graphs and Strings.” Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). Extends succinct representations to trees, planar graphs, and strings.
- Navarro, G. and Mäkinen, V. (2011). “Compressed Full-Text Indexes.” ACM Computing Surveys, 43(4). A comprehensive survey of compressed indexes including wavelet trees, FM-indexes, and LZ-based indexes.
- Munro, J. I. and Raman, V. (1997). “Succinct Representation of Permutations and Functions.” Proceedings of the 23rd International Colloquium on Automata, Languages, and Programming (ICALP). Shows how permutations and functions can be represented in $n \log n + O(n)$ bits with $O(1)$ query time.
- Clark, D. (1996). “Compact Pat Trees.” PhD thesis, University of Waterloo. Describes succinct representations of tries and pat trees using rank/select.

Data Structures at Scale: How Industry Builds on Classical Foundations

23.1 Introduction

Classical data structures—red-black trees, hash tables, B-trees, skip lists—form the theoretical backbone of every software system. A textbook treatment typically assumes that data fits in memory and that operations run on a single machine. In practice, companies like Google, Amazon, Facebook, and Netflix operate systems that serve billions of requests per second across thousands of nodes. At that scale, the gap between a clean $O(\log n)$ analysis and a production-grade system is enormous.

Two forces drive the gap. First, hardware is not uniform: SSDs favour sequential I/O, DRAM is scarce relative to data size, and multi-core NUMA machines make cache locality a first-order concern. Second, distribution introduces partial failure: nodes crash, networks partition, and consistency guarantees must be negotiated explicitly. Classical data structures were not designed with these constraints in mind.

This chapter bridges the gap. We study six systems—LSM trees, consistent hashing, column-family storage, Bloom filter variants, copy-on-write semantics, and hardware-aware design—that illustrate how industry adapts classical ideas. Each section includes a C++ code sketch, a complexity discussion, and pointers to the papers and open-source projects that made these ideas real. By the end, you will have a practical vocabulary for choosing and tuning data structures when the dataset no longer fits on one machine or in one disk.

23.2 Log-Structured Merge Trees — The Engine Behind Modern Databases

23.2.1 Motivation

Write-heavy workloads—event logging, time-series ingestion, IoT telemetry—stress traditional B-tree databases because every insertion may trigger a random disk write to maintain sorted order. Log-Structured Merge (LSM) trees trade read performance for write throughput by batching writes in memory and flushing them sequentially to disk.

23.2.2 Architecture

An LSM tree consists of three components:

1. **Memtable:** an in-memory sorted structure (typically a red-black tree or skip list) that absorbs all writes.
2. **SSTables (Sorted String Tables):** immutable, sorted files on disk, each containing a sequence of key-value pairs together with an index block and optional Bloom filters.
3. **Compaction process:** a background thread that merges adjacent SSTables to reclaim space and reduce the number of files that reads must examine.

23.2.3 Write Path

Every write enters the memtable first. Once the memtable reaches a size threshold (typically 4–64 MB), it is flushed to disk as a new SSTable. The flush is sequential, so even on spinning disks it is fast. The amortized write cost is $O(1)$ for the memtable insertion plus the amortized cost of background compaction.

```

1  #include <map>
2  #include <vector>
3  #include <string>
4  #include <fstream>
5  #include <algorithm>
6
7  struct SSTable {
8      std::vector<std::pair<std::string, std::string>> data;
9
10     void writeToFile(const std::string& path) const {
11         std::ofstream out(path);
12         for (const auto& [key, value] : data) {
13             out << key << "\t" << value << "\n";
14         }
15     }
16
17     static SSTable merge(const SSTable& a, const SSTable& b) {
18         SSTable result;
19         std::merge(a.data.begin(), a.data.end(),
20                   b.data.begin(), b.data.end(),
21                   std::back_inserter(result.data));
22         return result;
23     }
24 };
25
26 class LSMTree {
27     std::map<std::string, std::string> memtable;
28     std::vector<SSTable> levels[4];
29     size_t memtableLimit = 1000;
30
31 public:
32     void put(const std::string& key, const std::string& value) {

```

```

33     memtable[key] = value;
34     if (memtable.size() >= memtableLimit) {
35         flushMemtable();
36     }
37 }
38
39 std::string get(const std::string& key) {
40     if (memtable.count(key)) return memtable[key];
41
42     for (int i = 0; i < 4; ++i) {
43         for (int j = levels[i].size() - 1; j >= 0; --j) {
44             auto& table = levels[i][j];
45             auto it = std::lower_bound(
46                 table.data.begin(), table.data.end(),
47                 std::make_pair(key, std::string()),
48                 [](const auto& a, const auto& b) {
49                     return a.first < b.first;
50                 });
51             if (it != table.data.end() && it->first == key) {
52                 return it->second;
53             }
54         }
55     }
56     return "";
57 }
58
59 private:
60     void flushMemtable() {
61         SSTable sst;
62         for (const auto& [k, v] : memtable) {
63             sst.data.emplace_back(k, v);
64         }
65         levels[0].push_back(sst);
66         memtable.clear();
67         compact(0);
68     }
69
70     void compact(int level) {
71         if (levels[level].size() < 4) return;
72         SSTable merged = levels[level][0];
73         for (size_t i = 1; i < levels[level].size(); ++i) {
74             merged = SSTable::merge(merged, levels[level][i]);
75         }
76         levels[level].clear();
77         levels[level + 1].push_back(merged);
78     }
79 };

```


23.2.4 Read Path

A read checks the memtable first, then SSTables from newest to oldest. Without auxiliary structures this is $O(k \cdot \log m)$ where k is the number of SSTables and m is the size of each. Bloom filters attached to each SSTable reduce the expected number of disk probes to close to 1 for point queries.

23.2.5 Compaction Strategies

Two dominant strategies exist. *Size-tiered compaction* (used by Cassandra and HBase) groups SSTables of similar size and merges them when a threshold is reached. *Leveled compaction* (used by LevelDB and RocksDB) organises SSTables into levels with a size ratio of 10:1 and merges individual SSTables from level i into level $i + 1$. Levelled compaction uses more I/O but produces fewer tombstones and gives more predictable read performance.

23.2.6 Real-World Implementations

Google’s Bigtable (2006) was the first production LSM tree at scale. LevelDB (2011) made the design accessible as an embeddable library, and RocksDB (2013, Facebook) extended it with compression, transactions, and column families. As of 2024, RocksDB is the storage engine behind MySQL (MyRocks), Kafka Streams, and CockroachDB.

23.2.7 LSM vs. B-tree: A Comparison

B-trees provide $O(\log n)$ reads with a single disk seek because keys live at a known location in a single sorted tree. LSM trees defer sorting to compaction, so writes are sequential and batching reduces the total I/O volume. On SSDs, the sequential-write advantage is smaller but still meaningful because SSD garbage collection prefers contiguous data. LSM trees generally win when the write-to-read ratio exceeds 10:1; B-trees win for read-heavy or latency-sensitive workloads.

23.3 Consistent Hashing — Distributing Data Without Central Control

23.3.1 The Problem

Suppose you have N cache servers and you distribute keys with $h(k) \bmod N$. When a server is added or removed, N changes and nearly every key maps to a different server, causing a thundering-restore of cached data.

23.3.2 The Ring

Consistent hashing (Karger et al., 1997) hashes both keys and server identifiers onto a ring with positions $[0, 2^{32})$. A key is assigned to the next server clockwise from its hash position. When a server joins, only the keys in the arc between it and its predecessor need to be remapped.

```

1  #include <map>
2  #include <string>
3  #include <vector>
4  #include <functional>
5  #include <algorithm>
6  #include <numeric>
7
8  class ConsistentHashRing {
9      std::map<size_t, std::string> ring;
10     std::hash<std::string> hasher;
11     int virtualNodes;
12
13 public:
14     explicit ConsistentHashRing(int vnodes = 150)
15         : virtualNodes(vnodes) {}
16
17     void addNode(const std::string& node) {
18         for (int i = 0; i < virtualNodes; ++i) {
19             std::string vkey = node + "#" + std::to_string(i);
20             ring[hasher(vkey)] = node;
21         }
22     }
23
24     void removeNode(const std::string& node) {
25         for (int i = 0; i < virtualNodes; ++i) {
26             std::string vkey = node + "#" + std::to_string(i);
27             ring.erase(hasher(vkey));
28         }
29     }
30
31     std::string getNode(const std::string& key) const {
32         size_t h = hasher(key);
33         auto it = ring.lower_bound(h);
34         if (it == ring.end()) it = ring.begin();
35         return it->second;
36     }
37
38     std::vector<std::string> getReplicas(
39         const std::string& key, int count) const {
40         std::vector<std::string> replicas;
41         size_t h = hasher(key);
42         auto it = ring.lower_bound(h);
43         std::set<std::string> seen;
44         while (replicas.size() < count) {
45             if (it == ring.end()) it = ring.begin();
46             if (seen.insert(it->second).second) {
47                 replicas.push_back(it->second);
48             }

```

```

49         ++it;
50     }
51     return replicas;
52 }
53 };

```

23.3.3 Virtual Nodes

With only one hash per physical server, the arc lengths on the ring follow an exponential distribution, causing severe imbalance. Virtual nodes (vnodes) assign each physical server multiple positions on the ring. With 150 vnodes per server, the standard deviation of load drops below 5% in practice.

23.3.4 Production Use

Amazon’s DynamoDB paper (DeCandia et al., 2007) introduced consistent hashing to a wide audience. Today the technique underpins Cassandra’s partitioner, Memcached’s ketama algorithm, Netflix’s EV-Cache, and Uber’s Ringpop library.

23.4 Column-Family Storage — Wide Columns at Scale

23.4.1 Data Model

A column-family store generalises the relational model. Data is indexed by a row key, then a column family, then a column qualifier, and optionally a timestamp:

Row Key → Column Family → Column Qualifier → Timestamp → Value

This model is *sparse*: rows need not store every column, and storage cost is proportional to the data actually written, not to a schema-wide column count.

23.4.2 Bigtable and Its Descendants

Google’s Bigtable paper (Chang et al., 2006) described a distributed storage system built on GFS that used this model. HBase (Apache, 2008) brought the design to Hadoop, and Cassandra (Facebook, 2008) combined the wide-column model with consistent hashing for peer-to-peer replication.

```

1 #include <map>
2 #include <string>
3 #include <vector>
4 #include <cstdint>
5
6 struct Cell {
7     std::string columnQualifier;
8     int64_t timestamp;
9     std::string value;

```

```

10 };
11
12 using ColumnFamily = std::map<std::string, std::vector<Cell>>>;
13 using RowKey = std::string;
14
15 class ColumnFamilyStore {
16     std::map<RowKey,
17         std::map<std::string, std::vector<Cell>>>> table;
18
19 public:
20     void put(const RowKey& row, const std::string& cf,
21             const std::string& col, const std::string& val) {
22         int64_t ts = currentTimestamp();
23         table[row][cf].push_back({col, ts, val});
24     }
25
26     std::string get(const RowKey& row, const std::string& cf,
27                   const std::string& col) const {
28         auto rowIt = table.find(row);
29         if (rowIt == table.end()) return "";
30
31         auto cfIt = rowIt->second.find(cf);
32         if (cfIt == rowIt->second.end()) return "";
33
34         const auto& cells = cfIt->second;
35         for (auto it = cells.rbegin(); it != cells.rend(); ++it) {
36             if (it->columnQualifier == col) return it->value;
37         }
38         return "";
39     }
40
41     std::vector<std::pair<std::string, std::string>>
42     scan(const RowKey& row, const std::string& cf) const {
43         std::vector<std::pair<std::string, std::string>> result;
44         auto rowIt = table.find(row);
45         if (rowIt == table.end()) return result;
46
47         auto cfIt = rowIt->second.find(cf);
48         if (cfIt == rowIt->second.end()) return result;
49
50         for (const auto& cell : cfIt->second) {
51             result.emplace_back(cell.columnQualifier, cell.value);
52         }
53         return result;
54     }
55
56 private:
57     int64_t currentTimestamp() const {

```

```
58         return std::chrono::steady_clock::now().time_since_epoch()  
59             .count();  
60     }  
61 };
```

23.4.3 Why Column Families Win for Analytics

Column families enable *column pruning*: a query over three columns reads only those three columns, not entire rows. Adjacent values in a column family compress well because they share data types and value ranges. For time-series workloads, a typical query reads one row key (a sensor ID) and a single column family (the metric), achieving $O(\log n)$ lookup followed by sequential reads within the column family.

23.5 Bloom Filters in Production

23.5.1 Recap

A Bloom filter is a probabilistic set that answers “*Is x in the set?*” with no false negatives and a tunable false-positive rate ϵ . Given m bits and k hash functions, $\epsilon \approx (1 - e^{-kn/m})^k$.

23.5.2 At Scale

Google uses Bloom filters throughout its storage stack. Bigtable attaches a Bloom filter to each SSTable to avoid reading files that provably do not contain a queried key. Reported reductions in disk reads range from 100x to 1000x for random lookups. Netflix uses Bloom filters to deduplicate content recommendations across its recommendation engine, reducing redundant API calls.

23.5.3 Alternatives

Cuckoo filters support deletion (impossible with standard Bloom filters) and offer better cache locality because they store fingerprints in buckets rather than scattering them across a bit array. Quotient filters achieve similar benefits with a compact, cache-friendly layout. Both have become popular in production systems where the data set evolves and items must be removed.

23.6 Copy-on-Write and Versioned Storage

23.6.1 Persistent Data Structures via CoW

Copy-on-write (CoW) allows a structure to be shared among multiple readers while each writer sees its own version. The key insight is to copy only the path from the root to the modified node, reusing the rest.

Git is the most widely deployed CoW data structure. Each commit object points to a tree of blobs; unchanged subtrees are shared between commits. File systems like ZFS and Btrfs use CoW at the block level to provide atomic snapshots and crash consistency.

```

1  #include <memory>
2  #include <vector>
3  #include <string>
4
5  template <typename T>
6  class CowPtr {
7      std::shared_ptr<const T> data;
8
9  public:
10     explicit CowPtr(std::shared_ptr<const T> d)
11         : data(std::move(d)) {}
12
13     const T& read() const { return *data; }
14
15     CowPtr clone() const {
16         return CowPtr(std::make_shared<T>(*data));
17     }
18
19     CowPtr write(auto&& modifier) const {
20         auto copy = std::make_shared<T>(*data);
21         modifier(*copy);
22         return CowPtr(std::move(copy));
23     }
24 };
25
26 struct Config {
27     std::string dbPath;
28     int maxConnections;
29     bool enableCompression;
30
31     Config() : dbPath("/data/db"),
32               maxConnections(64),
33               enableCompression(true) {}
34 };
35
36 void versionedConfig() {
37     CowPtr<Config> v1(std::make_shared<Config>());
38     CowPtr<Config> v2 = v1.write([](Config& c) {
39         c.maxConnections = 128;
40     });
41     CowPtr<Config> v3 = v2.write([](Config& c) {
42         c.enableCompression = false;
43     });
44     std::cout << v1.read().maxConnections << "\n";

```

```

45     std::cout << v3.read().enableCompression << "\n";
46 }

```

23.6.2 Production Systems

Cassandra's commit log and memtable together form a shadow-copy system: the commit log is append-only, and the memtable is an in-memory snapshot that can be replaced atomically. This design gives Cassandra write-ahead-log durability without the write amplification of a B-tree.

23.7 Priority Queues and Scheduling at Scale

23.7.1 Kafka and Message Priorities

Apache Kafka traditionally orders messages by offset within a partition. When priority ordering is needed, operators maintain separate topics per priority level and use a priority consumer that drains higher-priority partitions first. At scale, this is effectively a distributed priority queue implemented with partition assignment.

23.7.2 Redis Sorted Sets

Redis sorted sets store a member with an associated score and use a skip list combined with a hash table internally. The skip list provides $O(\log n)$ insert, delete, and rank queries, while the hash table gives $O(1)$ membership lookups. Redis's in-memory nature makes this a natural choice for leaderboards, job queues, and scheduling in game servers.

```

1  #include <queue>
2  #include <vector>
3  #include <string>
4  #include <fstream>
5  #include <functional>
6
7  struct Job {
8      int priority;
9      std::string taskId;
10     bool operator<(const Job& o) const {
11         return priority < o.priority;
12     }
13 };
14
15 class DiskSpillPQ {
16     std::priority_queue<Job> inMemory;
17     std::vector<std::string> spillFiles;
18     size_t memoryLimit = 10000;
19     size_t currentSize = 0;
20     int fileCounter = 0;

```

```
21
22 public:
23     void push(const Job& job) {
24         if (currentSize >= memoryLimit) {
25             spillToDisk();
26         }
27         inMemory.push(job);
28         ++currentSize;
29     }
30
31     bool pop(Job& result) {
32         if (inMemory.empty()) {
33             if (spillFiles.empty()) return false;
34             loadFromDisk();
35         }
36         result = inMemory.top();
37         inMemory.pop();
38         --currentSize;
39         return true;
40     }
41
42 private:
43     void spillToDisk() {
44         std::string path = "/tmp/pq_spill_" +
45             std::to_string(fileCounter++) + ".bin";
46         std::ofstream out(path, std::ios::binary);
47         while (!inMemory.empty()) {
48             Job j = inMemory.top();
49             inMemory.pop();
50             out.write(reinterpret_cast<const char*>(&j.priority),
51                 sizeof(int));
52             size_t len = j.taskId.size();
53             out.write(reinterpret_cast<const char*>(&len),
54                 sizeof(size_t));
55             out.write(j.taskId.data(), len);
56         }
57         spillFiles.push_back(path);
58         currentSize = 0;
59     }
60
61     void loadFromDisk() {
62         std::string path = spillFiles.back();
63         spillFiles.pop_back();
64         std::ifstream in(path, std::ios::binary);
65         while (in.peek() != EOF) {
66             Job j;
67             in.read(reinterpret_cast<char*>(&j.priority),
68                 sizeof(int));
```



```

69         size_t len;
70         in.read(reinterpret_cast<char*>(&len), sizeof(size_t));
71         j.taskId.resize(len);
72         in.read(j.taskId.data(), len);
73         inMemory.push(j);
74         ++currentSize;
75     }
76     std::remove(path.c_str());
77 }
78 };

```

23.8 Graph Data Structures in Social Networks

23.8.1 The Social Graph

Facebook’s TAO (The Associations and Objects) paper (Bronson et al., 2013) describes a distributed graph store serving over a billion users. TAO stores a directed multigraph where vertices represent users, pages, and posts, and edges represent likes, follows, and comments. Queries are expressed as graph traversals with pagination.

23.8.2 Property Graph Model

A property graph associates typed properties with both vertices and edges. This model underpins Neo4j, Amazon Neptune, and TigerGraph. The key design question is how to store edges efficiently: as adjacency lists (good for traversals from one vertex) or as compressed sparse row/column (CSR/CSC) formats (good for bulk analysis and memory density).

```

1  #include <vector>
2  #include <string>
3  #include <unordered_map>
4  #include <algorithm>
5
6  struct Edge {
7      int target;
8      std::string label;
9  };
10
11  class CompressedGraph {
12      std::vector<int> rowPtr;
13      std::vector<int> colIdx;
14      std::vector<Edge> edges;
15      std::vector<std::string> vertexProps;
16      int numVertices;
17
18  public:
19      explicit CompressedGraph(int n)

```

```

20         : numVertices(n), rowPtr(n + 1, 0) {}
21
22     void addEdge(int from, int to,
23                 const std::string& label) {
24         edges.push_back({to, label});
25         rowPtr[from + 1]++;
26     }
27
28     void buildIndex() {
29         for (int i = 1; i <= numVertices; ++i) {
30             rowPtr[i] += rowPtr[i - 1];
31         }
32         std::vector<int> offsets = rowPtr;
33         std::vector<Edge> sorted(edges.size());
34         for (const auto& e : edges) {
35             int pos = offsets[0]++;
36             sorted[pos] = e;
37         }
38         edges = std::move(sorted);
39         colIdx.resize(edges.size());
40         for (size_t i = 0; i < edges.size(); ++i) {
41             colIdx[i] = edges[i].target;
42         }
43     }
44
45     std::vector<int> neighbors(int vertex) const {
46         int start = rowPtr[vertex];
47         int end = rowPtr[vertex + 1];
48         return std::vector<int>(
49             colIdx.begin() + start,
50             colIdx.begin() + end);
51     }
52
53     int degree(int vertex) const {
54         return rowPtr[vertex + 1] - rowPtr[vertex];
55     }
56 };

```

23.8.3 Graph Partitioning

Distributing a graph across machines requires partitioning. Hash-based partitioning is simple ($\text{partition}(v) = h(v) \bmod k$) but cuts many edges, increasing cross-machine traversals. METIS and power-law-aware algorithms cluster high-degree vertices near the graph centre, reducing the edge cut at the expense of some load imbalance. TAO uses a two-level scheme: vertex IDs encode the shard, and edges are co-located with their source vertex.

23.9 Hardware-Aware Data Structure Design

23.9.1 Cache Line Awareness

A modern x86 cache line is 64 bytes. Two threads writing to different fields of the same struct that share a cache line cause *false sharing*, degrading performance by orders of magnitude. Aligning frequently written fields to cache line boundaries eliminates this:

```
1 #include <new>
2 #include <stdint>
3 #include <stdlib>
4
5 struct alignas(64) CacheLinePadded {
6     int64_t counter;
7     char padding[64 - sizeof(int64_t)];
8 };
9
10 struct ThreadLocalCounters {
11     CacheLinePadded perThread[64];
12     int64_t sum() const {
13         int64_t total = 0;
14         for (int i = 0; i < 64; ++i) {
15             total += perThread[i].counter;
16         }
17         return total;
18     }
19 };
20
21 void* allocateAligned(size_t alignment, size_t size) {
22     void* ptr = nullptr;
23     posix_memalign(&ptr, alignment, size);
24     return ptr;
25 }
26
27 struct alignas(64) GraphNode {
28     int vertexId;
29     int64_t distance;
30     int parentIndex;
31     int edgeStart;
32     int edgeCount;
33     char pad[64 - 5 * sizeof(int)];
34 };
```

23.9.2 NUMA-Aware Allocation

On multi-socket systems, memory attached to a remote socket costs 2–3x more in latency. The first-touch policy assigns pages to the NUMA node of the thread that first writes to them. Spreading ini-

tialisation across threads on all sockets (or using `mbind` with `interleave` policy) avoids hot-spotting a single socket.

23.9.3 SOA vs AOS

Array-of-Structs (AOS) stores all fields of each element contiguously. Structure-of-Arrays (SOA) stores each field in a separate contiguous array. For SIMD vectorisation or queries that touch one field, SOA avoids loading irrelevant data into vector registers. A graph’s adjacency list, for instance, benefits from SOA when only the target vertex IDs are needed for a breadth-first search.

23.10 Comparison Table and Design Decisions

Structure	Write	Read	Best For
LSM Tree	$O(1)$ amort.	$O(\log n)$ with Bloom	Write-heavy, time-series
B-Tree	$O(\log n)$	$O(\log n)$ single-seek	Read-heavy, transactions
Hash Index	$O(1)$ amort.	$O(1)$ expected	Point lookups, caches
Column-Family	$O(\log n)$	$O(\log n + k)$	Sparse, wide, analytics

Model	Schema	Scaling	Use Case
Row Store	Fixed, rigid	Vertical / sharding	OLTP, transactions
Column Family	Semi-structured	Horizontal, automatic	IoT, time-series
Document Store	Schema-free	Horizontal, automatic	CMS, user profiles

Decision Flowchart for Data Structure Selection at Scale

Start with the access pattern. If the workload is write-dominant (write:read > 10:1) and point queries suffice, an LSM tree is the default choice. If reads dominate and latency is critical, prefer a B-tree. If the data is schema-free and document-shaped, a document store is natural. For wide, sparse time-series data, column families with compaction tuned for your read/write ratio are optimal. If the data is a graph with dense traversals, a native graph store outperforms relational joins. Finally, if keys must be distributed across an elastic set of servers, always use consistent hashing with virtual nodes rather than modulo partitioning.

23.11 Exercises

23.11.1 Drill

1. An LSM tree has 3 levels with a size ratio of 10. If the memtable is 4 MB and each level holds $10^i \times 4$ MB, what is the total on-disk capacity before compaction to the next level? How many SSTables are examined per point query without Bloom filters?

2. A consistent hash ring has 10 physical nodes, each with 150 virtual nodes. What is the expected number of virtual nodes on the ring? If a new node is added, approximately what fraction of keys are remapped?
3. A Bloom filter has 10 million bits and 7 hash functions for a set of 1 million elements. Calculate the false-positive probability. How many bits per element is this?

23.11.2 Application

1. Implement the LSM tree from Section 23.2 and benchmark insertion of 10 million random key-value pairs with and without Bloom filters on SSTables. Measure both write throughput and point-query latency. Plot the results as a function of the number of SSTables.
2. Build a column-family store using the API from Section 23.4. Ingest a synthetic time-series dataset (1000 sensors, 86400 readings each). Compare the scan time of reading all readings for one sensor against reading all sensors for one timestamp. Explain the performance difference in terms of column-family layout.
3. Implement the disk-spilling priority queue from Section 23.7 with a threshold of 1000 in-memory jobs. Push 1 million jobs and then pop them all. Measure the time spent spilling and reloading compared to a pure in-memory priority queue.

23.11.3 Research

1. Read the RocksDB Wiki section on compaction strategies. Implement a simple size-tiered compaction simulator that models write amplification, space amplification, and read amplification as functions of the number of levels and the size ratio. Compare your model's predictions against empirical benchmarks published in the RocksDB documentation.
2. The Facebook TAO paper describes a two-level caching architecture with a global cache backed by MySQL. Read the paper (Bronson et al., 2013) and sketch an LRU cache implementation in C++ that supports batched reads to minimise round-trip overhead. Analyse the amortised cost per lookup under varying cache-hit rates.

23.12 Industrial Perspectives

Every data structure and algorithm in this book was chosen because it appears in real systems. But not all are equally important in practice. The table below provides an honest assessment of each topic's industrial relevance, rated on a three-point scale:

Rating	Meaning
3 – High	Used daily in production systems at scale
2 – Medium	Used in specific domains or as building blocks
1 – Low	Primarily educational, theoretical, or superseded

These ratings reflect the state of industry in 2025. A rating of “Low” does not mean a topic is unimportant for learning—it means you are unlikely to encounter it in production code.

23.12.1 Complete Relevance Table

Data Structure / Algorithm	Rating	Primary Industrial Use
<i>Linear Structures</i>		
Arrays / Dynamic Arrays	3	Everywhere (std::vector, database pages)
Linked Lists	2	Kernel internals, LRU caches
Stacks	3	Compilers, DFS, undo/redo, call stacks
Queues	3	BFS, message queues, task scheduling
Dequeues	3	Sliding window, BFS, schedulers
Circular Buffers	3	Ring buffers, networking, producer-consumer
Gap Buffers	1	Text editors (Emacs, vi)
Unrolled Lists	1	Some database systems
Finger Trees	1	Functional programming (Haskell)
Ropes	1	Text editors, some languages
<i>Hashing</i>		
Hash Tables (Chaining)	3	Ubiquitous (databases, caches, compilers)
Hash Tables (Open Addressing)	3	High-performance systems (Google, Redis)
Cuckoo Hashing	2	Concurrent hash tables, network routers
Robin Hood Hashing	2	Some STL implementations
Swiss Table	3	Google Abseil, high-performance C++
Skip Lists	3	Redis sorted sets, concurrent data structures
Count-Min Sketch	2	Stream processing, network monitoring
Merkle Trees (Hash Trees)	3	Blockchain, distributed systems, Git
Extendible Hashing	2	Database file organizations
Distributed Hash Table	2	P2P systems, CDNs
<i>Search Trees</i>		
Binary Search Trees	3	Foundation for all balanced trees
AVL Trees	3	Database indexing, some std::set
Red-Black Trees	3	Java TreeMap, C++ std::map, Linux CFS
B-Trees	3	Databases (PostgreSQL, MySQL), filesystems
B+-Trees	3	Database indexing, NTFS, ext4
B*-Trees	2	Some database systems
Scapegoat Trees	1	Educational
Treaps	2	Concurrent data structures, some databases
Splay Trees	1	Some caching systems, mostly educational
Van Emde Boas Trees	1	Theoretical, some integer-specific systems
Weight-Balanced Trees	1	Some database systems
(a,b)-Trees	2	B-tree variants in databases
Top-Down Red-Black	2	Implementation variant
Finger Trees	1	Functional programming niche
Partial Rebuilding	1	Educational
Optimal BST	1	Educational
<i>Spatial Trees</i>		
K-d Trees	3	Graphics, ML, nearest neighbor search
Quadtrees	2	2D spatial indexing, games
Octrees	2	3D graphics, game engines
R-Trees	3	GIS, database spatial indexing (PostGIS)

23.12.2 Tier 3 — High Relevance: The Industrial Workhorses

These data structures and algorithms are used daily in production systems at companies like Google, Meta, Amazon, Microsoft, Netflix, and virtually every technology company.

Hash tables are the single most important data structure in industry. They power database index lookups (MySQL, PostgreSQL, MongoDB), caching (Redis, Memcached, CDN edge caches), compiler symbol tables, web server session management, network router forwarding tables, and operating system process tables. Google’s Abseil library uses Swiss Table hash maps, which are $2\text{--}3\times$ faster than `std::unordered_map`.

B-trees and B+-trees are the backbone of every major database and filesystem. PostgreSQL uses B+-trees as the default index type; MySQL InnoDB’s clustered index is a B+-tree; ext4 and NTFS use B-trees for directory indexing. A single PostgreSQL query can traverse millions of B+-tree nodes per second.

Red-black trees are the default balanced tree in most standard libraries: Java’s `TreeMap`, C++’s `std::map`, and Linux’s Completely Fair Scheduler. The Linux kernel’s CFS scheduler traverses a red-black tree of millions of processes millions of times per second.

Segment trees and Fenwick trees power real-time analytics and range queries in database engines, competitive programming, image processing, and time-series databases.

Graph algorithms (BFS, DFS, Dijkstra, MST, topological sort, SCC) are essential infrastructure. Google Maps uses a variant of Dijkstra’s on a graph with 100 million+ intersections.

Tries power Google’s autocomplete (3.5 billion queries/day), IP routing, and spell checking. **Bloom filters** are used by Chrome (malicious URL detection), Cassandra (SSTable lookup), and Bitcoin (SPV verification). **Consistent hashing** is the foundation of DynamoDB, Cassandra, CDNs, and Memcached.

23.12.3 Tier 2 — Medium Relevance: Domain Specialists

Skip lists are the backbone of Redis sorted sets and LevelDB/RocksDB memtables. **Suffix arrays** have replaced suffix trees for text indexing and bioinformatics (BWA aligns 1 billion DNA reads in 2 hours). **Max flow** solves transportation, image segmentation, and airline scheduling problems. **HyperLogLog** counts 1 billion distinct elements in 12 KB (Redis, PostgreSQL, BigQuery). **Persistent data structures** underpin Git, undo systems, and MVCC in databases. **LSM trees** power LevelDB, RocksDB, Cassandra, and InfluxDB—RocksDB processes 10 million writes/second on a single SSD.

23.12.4 Tier 1 — Low Relevance: Educational and Theoretical

Splay trees have good amortized performance but are mostly superseded by red-black trees. **Van Emde Boas trees** achieve $O(\lg \lg u)$ but hash tables are faster in practice. **Fusion trees** prove a theoretical lower bound is achievable but have no practical implementation. **Selection sort and bubble sort** are purely educational. The “low relevance” structures are not wasted knowledge—understanding them builds deep theoretical intuition that makes you a better engineer.

23.12.5 What to Learn for Industry

Career Path	Must-Know Structures
Backend Engineering	Hash tables, B-trees, sorting, BFS/DFS, Dijkstra
Database Engineering	B+-trees, LSM trees, hash indexes, external sort
Search Engineering	Tries, suffix arrays, inverted indices, Bloom filters
Game Development	K-d trees, A*, quaternions, spatial hashing
Machine Learning	Sparse matrices, hash tables, k-d trees, LSH
Distributed Systems	Consistent hashing, Bloom filters, vector clocks
Embedded Systems	Circular buffers, bit arrays, succinct structures
Competitive Programming	Segment trees, Fenwick trees, union-find, DP

23.12.6 Industrial Perspectives—Exercises

1. **Industry survey.** Pick a large open-source project (e.g., Linux kernel, Redis, PostgreSQL, Chromium). Identify 10 data structures it uses. Classify each by relevance rating.
2. **Startup stack.** You are building a real-time analytics platform processing 1 million events/second. Which data structures from this book would you choose for: (a) counting unique visitors, (b) maintaining top- k trending topics, (c) storing user profiles with fast lookup, (d) detecting duplicate events?
3. **Database internals.** Explain why B+-trees are preferred over red-black trees for database indexing, even though red-black trees have better worst-case bounds. What role does caching play?
4. **Distributed design.** You are designing a distributed cache with 100 nodes. Explain why consistent hashing is essential and what happens without it when a node fails.
5. **Trade-off analysis.** For each tier-2 structure, identify one system where it is irreplaceable and one system where a tier-1 alternative could work. Justify your choices.

23.13 References and Further Reading

- Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. *Proc. OSDI*.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon’s Highly Available Key-value Store. *Proc. SOSP*.
- Bronson, N., Amsden, Z., Cabral, G., et al. (2013). TAO: Facebook’s Distributed Data Store for the Social Graph. *Proc. ATC*.
- O’Neil, P., Cheng, E., Gawlick, D., and O’Neil, E. (1996). The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4), 351–385.

- Karger, D., Sherman, A., Berkheimer, A., et al. (1997). Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. *Proc. STOC*.
- Kreps, J., Narkhede, N., and Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *Proc. NetDB*.
- Lakshman, A. and Malik, P. (2010). Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review*, 44(2), 35–40.
- RocksDB Wiki. <https://github.com/facebook/rocksdb/wiki>
- Melnik, S. and Gubarev, A. (2010). Dremel: Interactive Analysis of Web-Scale Datasets. *Proc. VLDB*, 3, 330–339.
- Conchon, S. and Filliâtre, J.-C. (2008). A Persistent Union-Find Data Structure. *Proc. ML Workshop on ML*, 37–46.
- Bender, M. A., Farach-Colton, M., and Kim, B. (2019). Engineering a Cache-Oblivious Sort that is Fast in Practice. *Workshop on Algorithm Engineering and Experiments (ALENEX)*.
- Dean, J. and Barroso, L. A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2):74–83.
- Bloom, B. H. (1970). Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7):422–426.
- Kirsch, A. and Mitzenmacher, M. (2006). Less Hashing, Same Performance: Building a Better Bloom Filter. *ACM Transactions on Algorithms*, 2(4):589–605.
- Leis, V. et al. (2013). The Adaptive Radix Tree: Practical Indexing for Main-Memory Database Systems. *IEEE ICDE*.
- Stonebraker, M. et al. (2005). C-Store: A Column-Oriented DBMS. *VLDB*.

C++ STL Quick Reference

A.1 A.1 Containers

Container	Header	Description	Iterator Category
<code>std::vector<T></code>	<code><vector></code>	Dynamic array	Random access
<code>std::deque<T></code>	<code><deque></code>	Double-ended queue	Random access
<code>std::list<T></code>	<code><list></code>	Doubly linked list	Bidirectional
<code>std::forward_list<T></code>	<code><forward_list></code>	Singly linked list	Forward
<code>std::array<T, N></code>	<code><array></code>	Fixed-size array	Random access
<code>std::string</code>	<code><string></code>	Dynamic character array	Random access
<code>std::set<T></code>	<code><set></code>	Ordered set (red-black tree)	Bidirectional
<code>std::multiset<T></code>	<code><set></code>	Ordered set with duplicates	Bidirectional
<code>std::unordered_set<T></code>	<code><unordered_set></code>	Hash set	Forward
<code>std::map<K, V></code>	<code><map></code>	Ordered map	Bidirectional
<code>std::multimap<K, V></code>	<code><map></code>	Ordered map with duplicates	Bidirectional
<code>std::unordered_map<K, V></code>	<code><unordered_map></code>	Hash map	Forward
<code>std::stack<T></code>	<code><stack></code>	LIFO adapter	—
<code>std::queue<T></code>	<code><queue></code>	FIFO adapter	—
<code>std::priority_queue<T></code>	<code><queue></code>	Max-heap adapter	—

A.2 A.2 Algorithms (in `<algorithm>`)

Algorithm	Description	Complexity
<code>sort(first, last)</code>	Introsort	$O(n \log n)$
<code>stable_sort(first, last)</code>	Merge sort (stable)	$O(n \log n)$
<code>partial_sort(first, mid, last)</code>	Heap selection	$O(n \log k)$
<code>nth_element(first, nth, last)</code>	Quickselect	$O(n)$ avg
<code>lower_bound(first, last, val)</code>	First $\geq$ val	$O(\log n)$
<code>upper_bound(first, last, val)</code>	First $>$ val	$O(\log n)$
<code>binary_search(first, last, val)</code>	Existence test	$O(\log n)$
<code>merge(first1, last1, first2, last2, out)</code>	Merge sorted ranges	$O(n + m)$
<code>inplace_merge(first, mid, last)</code>	Merge in-place	$O(n \log n)$
<code>remove(first, last, val)</code>	Remove elements	$O(n)$
<code>unique(first, last)</code>	Remove consecutive duplicates	$O(n)$
<code>reverse(first, last)</code>	Reverse range	$O(n)$
<code>rotate(first, mid, last)</code>	Rotate range	$O(n)$
<code>next_permutation(first, last)</code>	Next lexicographic perm	$O(n)$
<code>accumulate(first, last, init)</code>	Sum (in <code><numeric></code>)	$O(n)$
<code>partial_sum(first, last, out)</code>	Prefix sums	$O(n)$
<code>adjacent_difference(first, last, out)</code>	Differences	$O(n)$

A.3 A.3 Chrono (`<chrono>`)

```

1 using clock = std::chrono::steady_clock;
2 auto start = clock::now();
3 // ... work ...
4 auto end = clock::now();
5 auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end -
    start);

```

A.4 A.4 Smart Pointers (`<memory>`)

```

1 auto p = std::make_unique<int>(42);
2 auto s = std::make_shared<int>(42);
3 auto w = std::weak_ptr<int>(s);

```

A.5 A.5 Random (`<random>`)

```

1 std::random_device rd;

```

```

2 std::mt19937 gen(rd());
3 std::uniform_int_distribution<> dis(1, 6);
4 int roll = dis(gen);

```

A.6 A.6 Utilities

Utility	Header	Description
<code>std::pair<T1, T2></code>	<code><utility></code>	Two values
<code>std::tuple<Ts...></code>	<code><tuple></code>	Multiple values
<code>std::optional<T></code>	<code><optional></code>	Value or empty
<code>std::variant<Ts...></code>	<code><variant></code>	Type-safe union
<code>std::any</code>	<code><any></code>	Type-erased value
<code>std::span<T></code>	<code></code>	Non-owning view (C++20)
<code>std::string_view</code>	<code><string_view></code>	String view (C++17)
<code>std::function<R(Args...)></code>	<code><functional></code>	Type-erased callable

Complexity Classes

B.1 Common Time Complexities

Complexity	Name	Example
$O(1)$	Constant	Array access, hash lookup
$O(\log n)$	Logarithmic	Binary search, balanced tree ops
$O(\sqrt{n})$	Square root	Integer factorization (naive)
$O(n)$	Linear	Linear search, array sum
$O(n \log n)$	Linearithmic	Merge sort, heap ops
$O(n^2)$	Quadratic	Bubble sort, naive matrix multiply
$O(n^3)$	Cubic	Floyd-Warshall, naive matrix multiply
$O(2^n)$	Exponential	Subset generation, TSP
$O(n!)$	Factorial	Permutation generation

B.2 NP-Completeness

- **P:** problems solvable in polynomial time
- **NP:** problems verifiable in polynomial time
- **NP-complete:** hardest problems in NP (if any one is solvable in P, all are)
- **NP-hard:** at least as hard as any NP problem

Canonical NP-complete problems:

- Boolean satisfiability (SAT)
- Traveling salesman (decision version)
- Vertex cover
- Clique
- Graph coloring
- Subset sum
- Knapsack (decision version)
- Hamiltonian cycle

B.3 B.3 Master Theorem (Refresher)

$T(n) = a \cdot T(n/b) + f(n)$, $a \geq 1$, $b \geq 2$:

1. If $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$
2. If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $a \cdot f(n/b) \leq c \cdot f(n)$ for $c < 1 \rightarrow T(n) = \Theta(f(n))$

Mathematical Review

C.1 C.1 Summations

- Arithmetic: $1 + 2 + \dots + n = n(n+1)/2$
- Squares: $1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6$
- Geometric: $1 + r + r^2 + \dots + r^n = (r^{n+1} - 1)/(r - 1), r \neq 1$
- Harmonic: $H_n = 1 + 1/2 + \dots + 1/n \approx \ln n + \gamma (\gamma \approx 0.57721)$

C.2 C.2 Logarithms

- $\log_b(xy) = \log_b x + \log_b y$
- $\log_b(x/y) = \log_b x - \log_b y$
- $\log_b(x^k) = k \log_b x$
- $\log_a x = \log_b x / \log_b a$
- $\log n! = \Theta(n \log n)$

C.3 C.3 Combinatorics

- n choose k: $C(n, k) = n! / (k! (n-k)!)$
- Binomial: $(x + y)^n = \sum C(n, k) x^k y^{n-k}$
- Number of subsets of an n-element set: 2^n
- Number of permutations: $n!$
- Expected value: $E[X] = \sum x \cdot P(X = x)$

C.4 C.4 Important Inequalities

- Jensen: $f(E[X]) \leq E[f(X)]$ for convex f
- Markov: $P(X \geq a) \leq E[X] / a$

- Chebyshev: $P(|X - \mu| \geq k\sigma) \leq 1/k^2$
- Chernoff: $P(X \geq (1+\delta)\mu) \leq \exp(-\mu \cdot \delta^2/3)$ for independent Poisson trials
- Union bound: $P(\cup A_i) \leq \sum P(A_i)$

C.5 C.5 Useful Constants

- $e \approx 2.71828$
- $\pi \approx 3.14159$
- $\ln 2 \approx 0.69315$
- γ (Euler-Mascheroni) ≈ 0.57722
- ϕ (golden ratio) ≈ 1.61803

C.6 D.1 External Memory Model

The **external memory model** (also called the I/O model or disk model) accounts for the cost of moving data between fast internal memory (RAM) and slow external storage (disk, SSD). In this model, computation is free; only data transfers between memory and disk are counted.

C.6.1 Model Parameters

- n : number of data elements
- B : size of a disk block (transfer unit)
- M : size of internal memory (RAM)
- Number of I/Os: $I/O(n) = \text{number of block transfers}$

C.6.2 Sorting in External Memory

The **k -way merge sort** is optimal for external sorting:

1. Sort M/B runs of size M each in internal memory: $O(n/B)$ I/Os
2. Merge $k = M/B$ runs at a time: $O((n/B) \log_{M/B}(n/M))$ I/Os

This matches the lower bound $\Omega((n/B) \log_{M/B}(n/B))$ for comparison-based external sorting.

C.6.3 B-Trees as External Memory Structures

B-trees (Section 11.20) are the standard external memory search structure. A B-tree of order m (branching factor m) has:

- Height: $O(\log_m n)$
- Search: $O(\log_m n)$ I/Os
- Insert/delete: $O(\log_m n)$ I/Os

With $m = B$ (matching the disk block size), each node fits in one block, and each I/O fetches an entire node. This is why B-trees dominate database indexing.

C.6.4 Buffer Trees

A **buffer tree** is a B-tree variant that buffers $O(B)$ updates per node. When a node's buffer is full, it flushes to its children. This batch approach reduces random I/Os:

- Insert k elements: $O((k/B) \log_{m/B}(n/B))$ I/Os amortized
- Range query: $O((k/B) + \log_{m/B}(n/B))$ I/Os

C.7 D.2 Cache-Oblivious Algorithms

A **cache-oblivious** algorithm is designed to perform well on *any* cache hierarchy, without knowing the cache parameters (B and M). The key insight is that algorithms with good *tall cache complexity* automatically adapt to any level of the memory hierarchy.

C.7.1 The Cache Model

The **cache-oblivious model** has:

- Two-level memory: cache of size M , blocks of size B
- Optimal replacement policy (future oracle, for analysis)
- Cost: number of cache misses (block transfers)

An algorithm is cache-oblivious if its performance bound does not depend on B or M .

C.7.2 Cache-Oblivious B-Tree

The van Emde Boas (vEB) layout of a complete binary tree places nodes in memory so that small subtrees fit in cache lines. This gives $O(\log_B n)$ cache misses per search, matching B-trees without knowing B .

C.7.3 Cache-Oblivious Priority Queue

A cache-oblivious priority queue is structured as a B-tree of B-trees. The amortized cost per operation is:

$$O\left(\frac{1}{B} \log_{M/B} \frac{N}{B}\right) \text{ I/Os}$$

This matches the optimal bound for comparison-based priority queues. Tokutek’s TokuDB storage engine uses fractal trees—a variant of cache-oblivious B-trees—to achieve high write throughput in MySQL and MongoDB.

C.7.4 Cache-Oblivious Matrix Multiplication

Recursive structure divides the $n \times n$ matrix into four $\frac{n}{2} \times \frac{n}{2}$ submatrices, performing $O(n^3/B\sqrt{M})$ cache misses—within a constant factor of optimal.

C.7.5 Why Cache-Oblivious Works

The vEB layout ensures small subtrees (up to height $\lg B$) are stored contiguously. A fetched cache line brings in an entire subtree of B nodes, maximizing useful data per miss.

C.8 D.25 Lazy Funnelsort and Distribution Sweeping

C.8.1 Lazy Funnelsort

Lazy funnelsort (Frigo, Leiserson, Prokop, Ramachandran) is the optimal cache-oblivious sorting algorithm. A funnel of order k merges k sorted sequences of total size N using:

$$O\left(\frac{N}{B} \log_{N/M} \frac{N}{B}\right) \text{ I/Os}$$

```

1 void funnelsort(std::vector<int>& arr, int lo, int hi,
2               int order) {
3     int n = hi - lo;
4     if (n <= 1) return;
5     if (order == 1) {
6         std::sort(arr.begin() + lo, arr.begin() + hi);
7         return;
8     }
9     int k = 1;
10    while (k * k < order) k++;
11    int chunk = (n + k - 1) / k;
12    for (int i = 0; i < k; i++) {
13        int s = lo + i * chunk;
14        int e = std::min(lo + (i + 1) * chunk, hi);
15        funnelsort(arr, s, e, order / k);

```

```

16     }
17     // k-way merge
18 }

```

C.8.2 Distribution Sweeping

Distribution sweeping applies the cache-oblivious merge technology to the sweep-line paradigm. For batched 2D orthogonal range counting, it achieves the sorting bound:

$$O\left(\frac{n}{B} \log_{n/M} \frac{n}{B}\right) \text{ I/Os}$$

This is optimal under the comparison-based I/O model.

C.8.3 Online Cache-Oblivious 2D Range Searching

The cache-oblivious range tree, laid out in vEB order, supports 2D orthogonal range counting in $O\left(\frac{\lg^2 n}{B} + \lg_B n\right)$ I/Os per query with $O(n/B)$ space.

Problem	I/O Complexity	Algorithm
Sorting	$O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$	Funnelsort
Batched 2D range counting	$O\left(\frac{n}{B} \log_{n/M} \frac{n}{B}\right)$	Distribution sweeping
2D range counting (online)	$O\left(\frac{\lg^2 n}{B} + \lg_B n\right)$	Cache-oblivious range tree
K-way merge	$O\left(\frac{N}{B} \log_K \frac{N}{B}\right)$	Funnel

C.9 D.3 Ordered File Maintenance

The *ordered file maintenance* problem asks: maintain a sorted sequence of n elements in an array with small gaps, supporting insert and delete while minimizing element moves. This is the fundamental black box behind cache-oblivious B-trees.

C.9.1 The Problem

A sorted array requires $O(n)$ shifts per insertion. By leaving small *gaps* between elements, we can insert without shifting everything. The challenge is to use gaps strategically so no region becomes too dense.

C.9.2 The Algorithm

The array is partitioned into blocks of size B . Each block holds $\Theta(B)$ elements. When a block overflows, it is split; when it underflows, it is merged with a neighbor. Elements are assigned rational labels (fractions with denominator 2^k). When inserting between labels a and b , we choose the midpoint

$(a + b)/2$. If the denominator exceeds a threshold, the block is rebuilt, redistributing all labels. The amortized cost is $O(\lg^2 n)$ moves per operation.

C.9.3 Connection to Cache-Oblivious B-Trees

Ordered file maintenance is the black box that makes cache-oblivious B-trees work. The leaves of a cache-oblivious B-tree are maintained using ordered file maintenance: elements within a leaf block are stored in sorted order, and when a leaf overflows, it is split. The van Emde Boas layout ensures any root-to-leaf path touches $O(\lg_B n)$ cache lines.

C.9.4 Connection to List Labeling

The *list labeling* problem is closely related: maintain n items in sorted order, assigning each item an integer label from $\{1, \dots, N\}$ so that lexicographic order matches sorted order. The best scheme achieves $O(1)$ amortized label changes per operation using $O(\lg n)$ bits per label. Ordered file maintenance generalizes list labeling by also requiring physical adjacency in the array.

C.9.5 C++ Implementation Sketch

```

1 class OrderedFile {
2     struct Block {
3         std::vector<int> elements;
4     };
5     std::vector<Block> blocks;
6     int blockSize;
7
8     void split(int idx) {
9         Block& b = blocks[idx];
10        int mid = b.elements.size() / 2;
11        Block nb;
12        nb.elements.assign(b.elements.begin() + mid,
13                           b.elements.end());
14        b.elements.resize(mid);
15        blocks.insert(blocks.begin() + idx + 1, nb);
16    }
17
18    void merge(int idx) {
19        auto& cur = blocks[idx].elements;
20        auto& nxt = blocks[idx + 1].elements;
21        cur.insert(cur.end(), nxt.begin(), nxt.end());
22        blocks.erase(blocks.begin() + idx + 1);
23    }
24
25 public:
26     OrderedFile(int B = 64) : blockSize(B) {}
27 }
```

```

28     void insert(int val) {
29         int idx = 0;
30         for (int i = 0; i < (int)blocks.size(); i++) {
31             if (!blocks[i].elements.empty() &&
32                 val > blocks[i].elements.back())
33                 { idx = i + 1; continue; }
34             idx = i; break;
35         }
36         if (idx == (int)blocks.size())
37             blocks.push_back({});
38         auto& e = blocks[idx].elements;
39         auto pos = std::lower_bound(e.begin(),
40                                   e.end(), val);
41         e.insert(pos, val);
42         if ((int)e.size() > 2 * blockSize) split(idx);
43     }
44
45     bool search(int val) const {
46         for (auto& b : blocks) {
47             auto it = std::lower_bound(b.elements.begin(),
48                                       b.elements.end(), val);
49             if (it != b.elements.end() && *it == val)
50                 return true;
51         }
52         return false;
53     }
54 };

```

C.9.6 Industrial Impact

The most prominent application is through fractal tree indexing. Tokutek’s TokuDB achieved 10–100× write speedup over InnoDB for write-heavy workloads. TokuDB was adopted by Percona, MariaDB, and MongoDB. The key advantage: fractal trees perform $O(\lg_B n)$ I/Os per insert, matching B-tree performance with better worst-case guarantees through buffering. Ordered file maintenance also appears in database systems maintaining sorted indices on flash storage, where write amplification must be minimized.

C.10 D.3 Pointer Machine Complexity

The **pointer machine** is a computational model where:

- Data is stored in nodes connected by pointers
- Each node has a fixed number of pointer fields and data fields
- Operations are: follow a pointer, compare data, allocate a node

- No arithmetic on addresses is allowed (unlike RAM model)

C.10.1 Separation from RAM

Many data structures that are $O(1)$ on a RAM model require $\Omega(\log n)$ on a pointer machine. For example, an array with direct indexing is $O(1)$ on RAM but requires a balanced BST (giving $O(\log n)$) on a pointer machine.

C.10.2 Optimal Pointer-Machine Structures

Operation	RAM	Pointer Machine
Predecessor/successor	$O(1)$ with hash	$O(\log n)$ with BST
Union-find	$O(\alpha(n))$	$O(\alpha(n))$
Priority queue (decrease-key)	$O(1)$ amortised (Fibonacci)	$O(1)$ amortised
Persistent BST	$O(\log n)$	$O(\log n)$

The pointer machine model is relevant for:

- **Persistent data structures:** all persistent structures operate on pointer machines (path copying creates new nodes via pointer assignment)
- **Functional programming:** purely functional structures are pointer machines (immutable nodes linked by pointers)
- **Garbage-collected languages:** Java, C#, Python all use pointer machines under the hood

C.11 References and Further Reading

- Thomas H. Cormen et al., *Introduction to Algorithms*, 4th Edition. Appendices A–C.
- Donald E. Knuth, *The Art of Computer Programming*, Volume 1: Fundamental Algorithms. Section 1.2 (mathematical preliminaries).
- Michael Sipser, *Introduction to the Theory of Computation*, 3rd Edition. Cengage, 2012.
- Oded Goldreich, *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008.
- Michael R. Garey and David S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- cppreference.com — Online C++ standard library reference.